

# 通关！AI 辅助软件项目开发实战

## ——从立项竞标到交付复盘的 18 天全生命周期训练

六关通关制 · 全生命周期 · 人机协同工作方法

课程教学组 编著

2026 年 9 月

# 前言

## 一、为什么写这本书

AI 编程工具普及之后，软件开发的瓶颈移动了。写代码这件事本身越来越便宜——一个会写提示词的学生，一天产出的代码量可能超过过去的一周。但真实项目的失败很少败在代码写得慢，而是败在另外一些地方：需求没问清楚就开工，承诺写出去兑现不了，质量靠一句“我们觉得没问题”，交付的软件换个环境就跑不起来。写代码不再是瓶颈，工程判断力才是。

学校的课程作业恰恰训练不到这一段。作业没有前置依赖——上一次做得再差，也不影响下一次开始；也没有真实后果——错误在期末才暴露，改不改随你。于是很多毕业生直到入职第一天，才第一次见到需求规格说明书、测试报告和验收单长什么样。

本书要补的就是这一段。它把一个软件项目从招投标到验收移交的全过程，压缩进 18 个工作日，让你完整地走一遍：去争取一个项目，把模糊的意图问成可验证的规格，把规格翻译成设计，把设计变成有测试护栏的代码，把代码连同一整套文档交到别人手上并让对方签字，最后坐下来诚实复盘。AI 全程在场——但它是被管理、被验证、被留痕的助手，不是替你负责的枪手。

## 二、这是一本什么样的书

本书采用通关式设计，全程六关：立项竞标、需求与规格、设计与计划、研发冲刺、质量门禁、交付验收与复盘。每一关设门禁，不过必须返工，返工占用的是本组下一关的时间预算——第 2 关把需求写糊涂了，第 3 关的设计就无从下手，后果立竿见影。

全书围绕一个贯穿始终的样例项目“澜图”展开：一套面向遥感影像智能解译的平台，把 SAM 系列分割模型的能力封装为命令行工具、REST 服务与 QGIS 插件三种形态，让不懂深度学习的遥感作业员也能在几分钟内从一景影像里提取出建筑物或水体的矢量边界。它的规模经过刻意裁剪——24 条功能需求、5 项 MVP、一天能读完的代码量——刚好够用来讲清楚方法。你的小组做的不是澜图，而是自己的题目；澜图是参照系、难度标尺和逆向教材，不是让你照抄的答案。

AI 辅助是本书的主线而非点缀。第 2 章用一整章建立通用方法论：提示词工程、规则文件三件套、Skills 沉淀、长流程上下文管理、AI 初稿的人工评审法、翻车记录制度。第 3~8 章把六关逐一展开，每一关都讲两套做法——人工怎么做（工具、要点、过程），人机协同怎么做（提示词、Skills、检查纪律），并以澜图的真实样例收尾。第 9 章是工具箱：逐日作战手册、答辩问题库、实训题、模板与提示词卡、检查单、工具链清单、FAQ、术语表与拓展阅读。

## 三、这本书适合谁，怎么用

选课学生（《空间信息数字工程综合实习》）：按 18 个工作日的日历执行（见 1.3 节与 9.1 节），每关开工前先读本关导论，交付前对照该关的过关自检清单逐项打钩。

自学者：找一个自己的题目（规模参照澜图），按六关顺序自驱。没有评委就自己扮演甲方，没有助教就用干净环境自测代替批量验证。3~5 人结伴最佳，单人亦可，但评审环节务必找同伴交叉进行——自己写的段落自己不审，是本书反复强调的纪律。

教师与助教：本书与《通关实践大纲》《教师实施与评分手册》配套使用，评分口径、门禁规则与日程组织以后两者为准。

## 四、你需要准备什么

一门编程语言的基本功（能独立写出 500 行可运行的程序）、Git 基础操作、一个 AI 编码工具（Claude Code、Cursor、Kimi CLI 任选其一，全组统一）。开课前完成 Bootcamp：开发环境装好、样例工程在本地跑通。Bootcamp 是入学门槛，不是考核项。

## 五、几点声明

澜图项目及其全部文档是教学再创作，招投标双方、用户与数据均为虚构，如有雷同纯属巧合。

书中出现的需求编号、指标数值，一律以澜图需求基线（spec.md）与正式交付文档为准；若正文与基线冲突，以基线为准。

AI 工具的版本迭代很快，书中的界面截图与具体命令可能过时；但人机分工的原则、质量护栏的纪律、过程留痕的要求不会过时——这些才是本书真正要教的东西。

编著者

2026 年 9 月

## 凡 例

一、本书由来。本书由课程《通关任务 1~6 工作方法指导书》六册整合重编而成，六关框架不变，内容按教材体例重写：每关先讲人工方法，再讲人机协同方法，最后以澜图样例收尾。

二、口径说明。澜图 V3.0 基线为 24 条功能需求、16 条非功能需求、3 条精度需求，其中 ★ 实质性条款 13 条；50 条设计用例对应 122 个自动化测试函数；缺陷看板 18 条。正文引用若与基线冲突，以基线为准。

三、架构版本说明。第 6、7 章部分小节保留了围绕消息队列调度、GPU Worker 池、死信队列展开的教学案例，取自澜图早期架构方案 v1——该方案已在设计评审中被两份架构决策记录（ADR）否决，最终架构为同步执行加模型实例缓存，与样例代码 lantu/ 目录的真实实现一致。保留旧架构案例是有意为之：异步架构在企业真实项目和多数自选题目里仍然常见，案例中的测试设计思路、演练组织方式、度量方法可以直接迁移。阅读约定：凡涉及队列、Worker、死信的段落，请当作另一类架构下的方法示范，不要拿去与 lantu/ 目录的代码逐行对照。

四、配套资源。澜图样例文档集（招标文件、投标文件、需求、设计、计划、测试、交付全套）、样例代码 lantu/ 目录、《通关实践大纲》与《教师实施与评分手册》。

# 目 录

|                                             |     |
|---------------------------------------------|-----|
| 第 1 章 课程导论.....                             | 1   |
| 1.1 课程定位与能力目标.....                          | 1   |
| 1.2 全生命周期与三条线索.....                         | 3   |
| 1.3 通关制：六关、18 个工作日与评分.....                  | 5   |
| 1.4 人机分工总原则.....                            | 7   |
| 1.5 样例项目「澜图」：怎么用，不能怎么用.....                 | 9   |
| 1.6 一条需求的完整旅程：全书导航.....                     | 12  |
| 第 2 章 AI 辅助开发通用方法论.....                     | 14  |
| 2.1 提示词工程：骨架、进阶技巧与使用纪律.....                 | 14  |
| 2.2 规则文件三件套：constitution、AGENTS 与 spec..... | 19  |
| 2.3 Skills：把工作方法沉淀为可复用能力包.....              | 22  |
| 2.4 长流程协作：上下文管理与链式任务.....                   | 26  |
| 2.5 AI 初稿的人工评审：四遍法与五副眼镜.....                | 29  |
| 2.6 翻车模式库与翻车记录制度.....                       | 33  |
| 2.7 红线、伦理与学术诚信.....                         | 39  |
| 2.8 过程留痕：Prompt 日志与模式库建设.....               | 44  |
| 2.9 人机分工矩阵：六关一张表.....                       | 48  |
| 2.10 需求追踪矩阵 RTM：一条链，五个版本.....               | 54  |
| 2.11 文档工程：国标结构、语言规范与程序化构建.....              | 58  |
| 2.12 评审会与团队协作：组织方法、主副修与人人过关.....            | 63  |
| 2.13 过程度量：用数据说话.....                        | 69  |
| 2.14 人机协同对话实录精选.....                        | 76  |
| 第 3 章 第一关：立项竞标.....                         | 79  |
| 3.1 关卡导论：立项竞标要解决什么问题.....                   | 79  |
| 3.2 《项目建议书》的工作方法.....                       | 80  |
| 3.3 《澄清与质询记录》的工作方法.....                     | 83  |
| 3.4 《技术投标书》：人工方法、人机协同与逐章写作指南.....           | 86  |
| 3.5 《项目计划 v1》：WBS 与甘特图.....                 | 96  |
| 3.6 《风险登记册 v1》的工作方法.....                    | 99  |
| 3.7 述标答辩的工作方法.....                          | 102 |
| 3.8 《需求确认书》：锁定 MVP 范围.....                  | 104 |
| 3.9 全程演练：澜图第 1 关完整记录.....                   | 107 |
| 3.10 过关自检与向下一关的交接.....                      | 113 |
| 第 4 章 第二关：需求与规格.....                        | 117 |
| 4.1 关卡导论：从意图到可验证的规格.....                    | 117 |
| 4.2 用户访谈与需求获取：方法与访谈样例精讲.....                | 117 |
| 4.3 用户故事、MoSCoW 优先级与人机协同.....               | 123 |
| 4.4 SRS 编写：国标骨架与章节规划.....                   | 127 |
| 4.5 功能需求六要素：写法与三轮迭代实战.....                  | 129 |
| 4.6 流程图、数据流图与数据字典.....                      | 134 |
| 4.7 SRS 逐节写作示范.....                         | 137 |
| 4.8 spec.md：机器可读规格与结构化内容协同生产.....           | 142 |
| 4.9 双轨同步工程：SRS 与 spec.md 如何永不打架.....        | 146 |
| 4.10 AI 反向澄清：让 AI 当苛刻的评审专家.....             | 147 |
| 4.11 RTM 逐行精讲：追踪链是怎么织出来的.....               | 149 |
| 4.12 质量评审、基线冻结与考核标准.....                    | 151 |
| 4.13 一致性审核实战与常见错误案例集.....                   | 153 |
| 4.14 过关自检与交接.....                           | 156 |
| 第 5 章 第三关：设计与计划.....                        | 161 |

|                                             |     |
|---------------------------------------------|-----|
| 5.1 关卡导论与设计总方法论.....                        | 161 |
| 5.2 概要设计 HLD：人工方法、人机协同与逐章细则.....            | 164 |
| 5.3 详细设计 DLD：人工方法、人机协同与逐模块要点.....           | 169 |
| 5.4 数据库设计：概念、逻辑、物理、运用四段法.....               | 174 |
| 5.5 接口设计与 OpenAPI 契约.....                   | 176 |
| 5.6 ADR：把架构决策写成可审判的文书.....                  | 177 |
| 5.7 非功能设计专题：GB/T 25000.10 逐项落地.....         | 179 |
| 5.8 plan.md、tasks.md 与 Spec-Driven 生成链..... | 181 |
| 5.9 测试计划：测试左移的落地.....                       | 183 |
| 5.10 项目管理计划与质量、配置、风险计划族.....                | 185 |
| 5.11 RTM v3 挂接与从逆向分析到正向设计.....              | 188 |
| 5.12 功能域演练与设计文档体检.....                      | 191 |
| 5.13 样例剖析、基线漂移复盘与设计缺陷案例集.....               | 194 |
| 5.14 过关自检与交接.....                           | 199 |
| 第 6 章 第四关：研发冲刺.....                         | 203 |
| 6.1 关卡导论与冲刺总方法论.....                        | 203 |
| 6.2 冲刺启动：环境、分支与 CI 就绪.....                  | 205 |
| 6.3 PR 门禁五环与 AI 代码评审.....                   | 207 |
| 6.4 测试先行与测试护栏.....                          | 209 |
| 6.5 pytest、契约测试与覆盖率门禁实操.....                | 211 |
| 6.6 安全红线：四条不可协商的底线.....                     | 218 |
| 6.7 性能测试实操：一次完整的优化闭环.....                   | 220 |
| 6.8 精度测试：冻结影像集与哈希登记制度.....                  | 223 |
| 6.9 降级与故障演练：把预案变成肌肉记忆.....                  | 226 |
| 6.10 调试方法论与重构技术债.....                       | 228 |
| 6.11 敏捷仪式：站会、周报与燃尽图.....                    | 231 |
| 6.12 变更管理与 RTM v4 回填.....                   | 234 |
| 6.13 代码文档同步与合格仓库解剖课.....                    | 239 |
| 6.14 PR #47 全流程复盘与编码缺陷案例集.....              | 242 |
| 6.15 过关自检与交接.....                           | 247 |
| 第 7 章 第五关：质量门禁.....                         | 251 |
| 7.1 关卡导论：质量是构建出来的.....                      | 251 |
| 7.2 测试设计总览与用例设计六种技术.....                    | 254 |
| 7.3 用例生产的人机协同与 AI 用法.....                   | 259 |
| 7.4 测试执行管理与 RTM v5 闭环.....                  | 264 |
| 7.5 单元、集成与系统三级测试执行.....                     | 267 |
| 7.6 功能适合性专项：MVP 与★款逐条过堂.....                | 272 |
| 7.7 性能专项：压测场景设计与数据解读.....                   | 273 |
| 7.8 精度专项：算法类需求的验收测试.....                    | 276 |
| 7.9 可靠性与安全性专项.....                          | 279 |
| 7.10 可移植性、兼容性、易用性与维护性专项.....                | 283 |
| 7.11 破坏性测试：攻击者思维与组内红队自测.....                | 290 |
| 7.12 故障演练：15 分钟定位与恢复.....                   | 295 |
| 7.13 缺陷管理与根因分析.....                         | 297 |
| 7.14 质量特性自评表与测试报告.....                      | 302 |
| 7.15 过关自检与交接.....                           | 306 |
| 第 8 章 第六关：交付验收与复盘.....                      | 309 |
| 8.1 关卡导论：验收驱动与三层交付物.....                    | 309 |
| 8.2 文档审计：四维评分、流程与自查清单.....                  | 311 |
| 8.3 一致性核查与可执行性核查.....                       | 315 |
| 8.4 AI 审计：预审三遍法与三单对照.....                   | 319 |
| 8.5 移交态整理与交付包工程.....                        | 323 |
| 8.6 干净环境自测：在助教之前先跑一次.....                   | 326 |

|                            |     |
|----------------------------|-----|
| 8.7 部署运维手册与用户手册.....       | 328 |
| 8.8 缺口收敛与批量验证应对.....       | 333 |
| 8.9 彩排、验收演示与汇报 PPT.....    | 334 |
| 8.10 UAT 组织与验收报告.....      | 341 |
| 8.11 价值论证：效率收益是怎么算出来的..... | 343 |
| 8.12 复盘报告与偏差分析.....        | 344 |
| 8.13 AI 协作量化与净收益估算.....    | 348 |
| 8.14 技术债务台账与经验萃取.....      | 352 |
| 8.15 移交收口与迁移到你自己的题目.....   | 355 |
| 第 9 章 附录与工具集.....          | 367 |
| 9.1 18 个工作日逐日作战手册.....     | 367 |
| 9.2 答辩问题库：六关合集.....        | 371 |
| 9.3 课堂练习与实训题.....          | 383 |
| 9.4 模板与提示词卡集.....          | 391 |
| 9.5 检查单汇总.....             | 400 |
| 9.6 工具链装备清单.....           | 404 |
| 9.7 常见问题解答.....            | 412 |
| 9.8 术语表.....               | 422 |
| 9.9 拓展阅读与资源清单.....         | 430 |

## 第 1 章 课程导论

本章是全书总论，回答三个问题：这门课要把你训练成什么样的人，一个软件项目完整的形状是什么样，有了 AI 之后人还要亲自做哪些事。前三节讲课程本身——能力目标、三条线索、通关规则与日历；后三节讲工作方式——人机分工总原则、样例项目「澜图」的正确用法，以及一条需求从招标条款走到验收证据的完整旅程。1.6 节是全书导航图，建议读完折角。

### 1.1 课程定位与能力目标

#### 1.1.1 这门课要把你变成什么样的人

在此之前，你写过的程序大多有一个共同特征：题目是给定的，标准答案是存在的，验收方式是运行一下看输出对不对。这类训练解决的是「会不会写」的问题。而在真实的工程现场，最常见的失败不是写不出来，是写出了没人要的东西、写完了没人敢用、或者写完了说不清自己写了什么。

这门课要补的正是这一段。它把一个软件项目从招投标到验收移交的全过程压缩进 18 个工作日，让你在毕业前完整地走一遍：去争取一个项目，把模糊的意图问成可验证的规格，把规格翻译成设计，把设计变成有测试护栏的代码，再把代码连同一整套文档交到别人手上并让对方签字确认。

三个转变是这门课的全部意图：从「做题」转向「做事」——目标不再由题面给定，而要自己论证；从「个人技能」转向「工程协作」——你的产出要能被队友接手、被评审质询、被半年后的自己看懂；从「功能实现」转向「价值交付」——衡量标准不是跑通了几个功能，是有没有人愿意接收这个成果。

一句话概括：这门课考的不是你能写多少代码，是你能不能对一个项目负责。

#### 1.1.2 AI 改变了什么，没有改变什么

AI 编码工具把「产出初稿」的成本压到了接近于零。一份两万字的技术方案、一套五十条的测试用例、一个模块的完整实现，都可以在几十分钟内拿到看起来像模像样的第一版。这是真实的效率跃迁，本课程不回避、不限制，反而要求你深度使用并如实记录。

但有三件事 AI 没有改变。第一，判断的责任仍然在人：AI 能给出五个方案，不能替你承担选错的后果。第二，事实的锚点仍然要人来立：AI 会流畅地编造不存在的 API、引用错误的标准条款、给出乐观的性能估算，而这些错误恰恰以「看起来很专业」的形式出现。第三，可追溯性仍然要人来维护：一份没人能说清来源的文档，无论是谁写的，在评审席上都站不住。

所以本书的立场是：AI 是加速器，不是替代者。它让你有可能在 18 个工作日内走完过去需要一学期的流程，代价是你必须建立起一整套约束它、检验它、留痕它的工程纪律。这套纪律就是第 2 章的全部内容。

### 1.1.3 六项能力目标与达成证据

能力目标不是口号，每一条都对应可检查的证据。下表是本课程的能力—证据对照，也是你在第 6 关答辩时举证的骨架。

| 能力目标    | 达成的标志                     | 证据在哪里                         |
|---------|---------------------------|-------------------------------|
| 工程全局观   | 说得清每份交付物的上游来源与下游用途        | 第 6 关答辩、RTM 五个版本              |
| 规格驱动开发  | spec.md 是唯一事实源，代码与文档都由它派生 | spec.md + 双轨一致性核查记录           |
| AI 管控能力 | 规则文件生效、链式任务不漏项、翻车有记录      | constitution.md、tasks.md、翻车记录 |
| 测试护栏意识  | AI 产出一律过测试、评审、CI 三道闸      | PR 门禁记录、覆盖率报告、契约测试            |
| 按国标交付   | 文档结构合规、编号自洽、版本可追溯         | 国标交付文档全集 + 文档审计报告             |
| 团队治理    | 计划、风险、变更、配置四本账都活着         | 项目管理计划、风险登记册、变更单、配置库          |

### 1.1.4 本书怎么用

全书九章，分工明确。1.1 节（课程导论）讲为什么和是什么，开课前读完。1.2 节（通用方法论）讲贯穿六关的横向能力——提示词工程、规则文件、评审方法、留痕纪律、度量方法，每一关都要用，所以抽出来讲一次，建议第一关期间读完，之后随时回查。第 3~8 章（六关工作方法）是主体，按关卡组织，每一份交付物一节，讲清楚人工怎么做、AI 怎么配合、样例长什么样、交之前查什么。2.3 节（附录与工具集）是工具箱：逐日作战手册、答辩题库、模板、检查单、术语表，随用随翻。

三种读法。第一次读，顺序过一遍第 1、2 章，第 3~8 章只读各关导论节，建立地图；进入某一关时，把对应一章全部精读，边读边做；答辩前，读 9.2 节的答辩题库



和 9.5 节的检查单汇总。不建议从头到尾线性通读——第 3~8 章是按交付物组织的操作手册，脱离具体任务读会很枯燥，且记不住。

阅读约定：书中出现的所有需求编号、指标数值、文档编号，一律以 content/spec.md 与 content/ 下的正式文档为准。若书中数字与正式文档冲突，以正式文档为准，并按第 6 关的锚点纪律登记更正。

## 1.2 全生命周期与三条线索

### 1.2.1 一个软件项目的完整形状

如果只用一张图描述软件项目，它不是一条从需求到交付的直线，而是三条并行的线索：一条管住项目本身，一条造出产品，一条把成果变成可以被接收的东西。三条线同时开始、同时结束，任何一条断了，项目都算失败。

很多学生项目只有中间那一条——写代码。结果是功能跑通了，但说不清做了什么、为什么这么做、质量如何、别人怎么用。这门课强制三条线并行，就是要打破「软件项目等于写代码」的错觉。

| 线索  | 关注什么     | 主要产出                      | 主导角色     |
|-----|----------|---------------------------|----------|
| 管理线 | 项目是否可控   | 立项、计划、例会、风险、变更、配置、验收移交    | 项目经理     |
| 研发线 | 产品是否长出来  | 需求、设计、编码、测试、部署            | 架构师 / 开发 |
| 交付线 | 成果是否可被接收 | 投标书、SRS、设计文档、测试文档、手册、验收报告 | 全员，PM 收口 |

### 1.2.2 管理线：让项目可控

管理线回答的问题是：这个项目现在处于什么状态，接下来会发生什么，出了偏差怎么办。它的产出看起来最「不像干活」，却是区分工程与作坊的分水岭。

管理线在六关中的落点：第 1 关立项与初版计划、风险登记册；第 3 关正式的项目管理计划与质量、配置、风险管理计划族；第 4 关站会、周报、燃尽图与变更管理；第 6 关偏差分析、复盘报告与配置库收口。注意管理线的四本账——计划、风险、变更、配置——一旦开了就必须一直活着，只在开头写一次、后面再不更新的账本，在审计时是负分而不是零分。

### 1.2.3 研发线：让产品长出来

研发线是从意图到可运行代码的转化链。本课程采用规格驱动开发（Spec-Driven Development）来组织这条线：先把需求写成机器可读的规格，再由规格派生设计、任务与代码，最后用测试回头验证规格。

与传统流程相比，规格驱动的关键差异在于 spec.md 的地位。它不是 SRS 的简化摘要，而是与 SRS 双轨并存的单一事实源：SRS 面向人（评审、验收、存档），spec.md 面向机器（AI 生成、脚本校验、自动追踪）。两者必须永远一致，维护这份一致性是第 2 关的核心训练之一。

### 1.2.4 交付线：让成果可被接收

交付线最容易被误解为「补文档」。真实的顺序恰恰相反：交付物是在过程中自然长出来的，不是最后补造的。一份在项目结束后三天内赶出来的测试报告，无论写得有多漂亮，都无法解释报告里的数据是哪次执行产生的。

本课程的交付线对标国家软件工程标准的教学对标版：GB/T 8567 的文档结构、GB/T 9385 的需求规格、GB/T 15532 的测试规范、GB/T 25000.10 的质量模型。对标不等于照抄——你需要知道标准的存在、会查会用、让文档达到交付质量，而不是把标准目录复制一遍再填空。

### 1.2.5 AI 引擎：加速器，不是替代者

AI 横穿三条线，但在每条线上的介入深度不同。研发线介入最深——设计草案、代码实现、测试用例都可以由 AI 主笔；交付线次之——文档骨架与初稿可以交给 AI，但结论、承诺、披露必须由人写；管理线介入最浅——风险判断、变更裁决、偏差归因这些事，AI 只能提供素材，不能替你下判断。

四个支柱撑起这台引擎：规格驱动（让 AI 有明确的目标）、规则文件（让 AI 服从项目约定）、链式任务（让 AI 不漏项、不发散）、测试护栏（让 AI 的产出必须通过验证才能进仓）。四者缺一，AI 就会从加速器变成不确定性来源。

### 1.2.6 三条线的自查表

每关结束前，用这张表自查一遍。三列都有内容，这一关才算真正走完；只有中间一列有内容，说明你还在「做题」。

| 关卡         | 管理线             | 研发线                               | 交付线                   |
|------------|-----------------|-----------------------------------|-----------------------|
| 第 1 关 立项竞标 | 立项、WBS、风险登记册 v1 | 需求初探、技术选型论证                       | 项目建议书、技术投标书           |
| 第 2 关 需求规格 | 范围锁定、需求变更机制     | SRS 、 spec.md 、 RTM v1/v2         | 需求确认书、评审记录            |
| 第 3 关 设计计划 | 项目管理计划族、工程基线    | HLD/DLD/DBD/IFD 、 ADR、 plan/tasks | 设计文档集、测试计划            |
| 第 4 关 研发冲刺 | 站会、燃尽图、变更管理     | 代码、单元/契约测试、CI                     | 冲刺报告、RTM v4、Prompt 资产 |
| 第 5 关 质量门禁 | 缺陷看板运营、质量目标核查   | 系统测试、专项测试、缺陷修复                    | 测试报告、质量自评、RTM v5      |
| 第 6 关 交付验收 | 偏差分析、复盘、配置库收口   | 干净环境部署验证                          | 手册、验收报告、审计报告          |

### 1.3 通关制：六关、18 个工作日与评分

#### 1.3.1 为什么用「通关」而不是「交作业」

交作业的隐含规则是：做完了交上去，老师打个分，然后翻篇。这套规则在工程训练里有两个致命问题。一是没有前置依赖——上一份作业做得再差，也不影响下一份开始；二是没有真实后果——错误在期末才被指出，纠正的机会已经没有了。

通关制把这两点改掉。每一关设门禁，门禁不过必须返工，返工占用的是本组下一关的时间预算。这意味着第 2 关把需求写糊涂了，第 3 关的设计就无从下手，你会立刻感受到后果，而不是等到期末。这正是真实项目的运作方式：上游的模糊会在下游放大成灾难。

通关制的公平性来自窗口不重叠：顺延占用的是本组自己的时间，不是全班的。落后的组不会拖慢别人，但会挤压自己后续关卡的余量。

#### 1.3.2 六关全景与课程日历

全课程 18 个工作日，六关窗口互不重叠。日历一律按工作日计算，周末不计入预算、不安排评审、不作为交付日；小组可以自主使用周末，但过程证据的时间戳落在周末不加分也不减分。研发冲刺关独占 7 天、横跨两周，是本日历最重要的取舍：宁可压缩需求、设计与质量环节的窗口，也要保住研发时间。

| 关卡         | 工作日     | 天数 | 本关主线    | 门禁形式         |
|------------|---------|----|---------|--------------|
| 第 1 关 立项竞标 | D1–D4   | 4  | 管理线启动   | 评审           |
| 第 2 关 需求规格 | D5–D6   | 2  | 研发线起点   | 评审           |
| 第 3 关 设计计划 | D7–D8   | 2  | 设计与工程基线 | checklist 快检 |
| 第 4 关 研发冲刺 | D9–D15  | 7  | 研发线主体   | 中期评审         |
| 第 5 关 质量门禁 | D16–D17 | 2  | 交付线攻坚   | 正式评审         |
| 第 6 关 交付验收 | D18     | 1  | 管理线收口   | 验收评审（含企业评委）  |

课前另有 Bootcamp，不设关卡、不计分，采用助教答疑制：开发环境安装、Git 基础、AI 编码工具接入、样例工程本地跑通。它是入学门槛而不是考核项——开课第一天各组须已能在本地运行样例工程，课堂时间不用于环境排障。

### 1.3.3 门禁分级与返工规则

第 1、2、4（中期）、5、6 关为正式评审，由教师与甲方角色共同参与；第 3 关为 checklist 快检，助教执行、教师抽检。分级的理由是设计关的产出以自洽性和完整性为主，适合用清单快速判定；而立项、需求、验收关涉及价值判断与承诺，必须由人当面质询。

第 3 周的中期评审是范围锁定点，规则只有一条：允许砍功能，不允许砍流程。核心功能不超过 5 个，流程完整性永远优先于功能数量。一个做了 3 个功能但六条流程全都走完、证据齐全的项目，得分高于做了 10 个功能却拿不出设计文档与测试证据的项目。

### 1.3.4 评分构成与过程分的采集

过程分不是靠印象打的，它有明确的采集口径。凡是过程分项，都要求「有物、有时、有人」——有可查的产物、有落在过程中的时间戳、有署名负责人。三者缺一，该项证据不予采信。

这条规则直接决定了你的日常习惯：Prompt 日志要当天记，翻车记录要当场写，评审意见要现场留痕，配置库要随时更新。第 6 关的三项审计（文档审计、AI 使用审计、一致性核查）审的就是这些日常动作的沉淀。第 2 章的 2.8 节专门讲留痕方法，那一章要在第 1 关就读完并开始执行。

后补的过程证据在本课程视同缺失；标注了【补录】的后补记录按疏忽处理，扣分但不算失信；伪造过程证据是重罪，直接判该关不过。详见 2.7 节。

### 1.3.5 小组角色与职责

3-5 人一组，模拟企业项目组运作。人少的组可以兼任，鼓励结对，不做强制轮换。项目经理是唯一不轮换的角色——项目的连续性需要一个稳定的责任人。

| 角色          | 职责重心                         | 在哪几关是主力   |
|-------------|------------------------------|-----------|
| 项目经理 PM     | 计划、风险、变更、对外沟通、收口             | 第 1、3、6 关 |
| 架构师         | 技术选型、HLD/DLD、ADR、设计评审        | 第 3、4 关   |
| 开发工程师       | 编码、单元测试、代码评审、重构              | 第 4 关     |
| 测试 / DevOps | 测试设计与执行、CI、部署、故障演练           | 第 3、5、6 关 |
| AI 工程负责人    | 规则文件、Prompt 模式库、AI 使用公约、翻车记录 | 全程        |

AI 工程负责人是这门课新增的角色，也是最容易被轻视的一个。它不是「最会用 AI 的那个人」，而是「负责让全组都用得对」的那个人：维护 constitution.md 与 AGENTS.md、沉淀 Prompt 模式库、组织翻车复盘、在第 6 关准备 AI 使用审计的三单对照材料。

## 1.4 人机分工总原则

### 1.4.1 三个档位：AI 主笔、人机共写、人工独立

人机分工不是一句「AI 辅助」就能带过的。同一份文档里，不同段落的分工档位可以完全不同：结构和素材交给 AI，判断和承诺留给自己。把分工细化到段落级，是这门课与「用 AI 写作业」的根本区别。

| 档位        | 含义               | 人的动作             | 典型场景                |
|-----------|------------------|------------------|---------------------|
| A 档 AI 主笔 | AI 出完整初稿，人审核修订   | 逐句核查事实、逻辑、合规     | 文档骨架、样板代码、测试脚本、格式转换 |
| B 档 人机共写  | 人定框架与判断，AI 填充与打磨 | 先写提纲和结论，再让 AI 展开 | 设计方案、测试用例集、分析报告     |
| C 档 人工独立  | AI 不参与产出，最多做检索   | 全部自己写，可让 AI 事后挑错 | 结论、承诺、披露、评价、归因      |

### 1.4.2 判定三问

拿不准某项工作该用哪个档位时，依次问三个问题。任何一个答案是「否」，档位就要往 C 档移一格。

- 第一问：错了能不能被检出？能用测试、脚本或对照标准立刻验证的，可以放心用 A 档；错误要等到验收才暴露的，往 C 档移。
- 第二问：这句话是不是承诺？凡是对外的承诺、对上的结论、对己的披露，只要写下来就要负责的，一律 C 档。
- 第三问：我有没有能力判断它对不对？如果连 AI 写得对不对都看不懂，说明这块知识你还没掌握，此时用 A 档等于放弃学习，应当先自己做一遍。

第三问是这门课特有的。企业里可以用「反正有人能审」来兜底，课堂上不行——你交上来的东西，最终要在答辩席上由你本人解释。

### 1.4.3 四类绝不外包的工作

无论时间多紧，以下四类工作必须由人独立完成。它们的共同特征是：外包出去，项目就失去了责任主体。

- 结论类：测试是否通过、质量是否达标、项目是否可以验收——判断只能由人做出。
- 承诺类：投标承诺、验收标准、性能指标——写下来就要兑现的话，必须由人斟酌。
- 披露类：翻车记录、偏差分析、技术债台账、AI 使用声明——涉及自我披露的内容，AI 会自动往好听里写，这恰恰破坏了披露的意义。

- 归因类：为什么延期、为什么出这个缺陷、根因是什么——归因需要现场知识，AI 只能给出通用套话。

#### 1.4.4 AI 产出的可信度分级与验证成本

对 AI 产出建立分级预期，能显著降低返工。可信度取决于两个因素：任务是否有客观标准，以及错误是否容易被发现。

| 可信度 | 任务类型                     | 建议验证方式       |
|-----|--------------------------|--------------|
| 高   | 格式转换、样板代码、术语解释、结构生成      | 抽查即可         |
| 中   | 功能实现、测试用例、文档初稿、数据整理      | 逐条过测试或对照基线核查 |
| 低   | 数值估算、标准条款引用、API 存在性、性能预测 | 逐个查证原始出处     |
| 不可用 | 结论、承诺、披露、归因              | 不采用 AI 产出    |

特别提醒「低」这一档。AI 编造 API、张冠李戴需求编号、给出乐观数值估算，是最高频也最难发现的三类错误，因为它们的表述方式与正确内容完全一样。2.5 节的「五副眼镜」就是专门对付这一档的。

#### 1.4.5 从总原则到具体矩阵

本节给的是判定框架。六关各自的分工矩阵——哪份交付物的哪个部分用哪个档位——集中在 2.9 节。建议的用法是：本节读一遍建立直觉，2.9 节的表打印出来贴在工位上，每关开工前对一遍。

### 1.5 样例项目「澜图」：怎么用，不能怎么用

#### 1.5.1 澜图是什么

澜图是本书贯穿六关的样例项目：一套面向遥感影像智能解译的工具集。它把 SAM 系列分割模型的能力封装成三种可用形态——命令行工具、REST 服务、以及可被 QGIS 调用的接口——让不懂深度学习的遥感作业员，也能在几分钟内从一景影像里提取出建筑物或水体的矢量边界。

它的价值命题很朴素：同样一幅影像的地物提取，人工勾绘需要 45 分钟，用澜图



走一遍需要 6 分钟。第 6 关的价值论证章节会讲这个数字是怎么算出来的、为什么不能简单地说「效率提升 7.5 倍」。

| 维度   | 澜图 V3.0 的边界                                |
|------|--------------------------------------------|
| 交付形态 | CLI 命令行 + REST 服务（6 个端点）+ QGIS 插件对接        |
| 核心能力 | 文本提示分割、点/框交互分割、批量处理、结果矢量化导出                |
| 数据存储 | 本地文件：GeoTIFF 影像、GeoPackage 矢量、JSON 工程状态    |
| 规格规模 | 24 条功能需求、16 条非功能需求、3 条精度需求，其中 ★ 实质性条款 13 条 |
| 测试规模 | 50 条设计用例、122 个自动化测试函数、缺陷看板 18 条            |
| 明确不含 | PostGIS、MinIO、Redis、消息队列、Worker 池、独立前端框架   |

### 1.5.2 样例代码 lantu 的边界

lantu/ 目录下是澜图的真实代码。它不是一个玩具工程，也不是一个完整产品，而是一个「刚好够用来讲清楚方法」的规模：代码量能在一天内读完，但涵盖了模型加载、推理调度、格式转换、接口暴露、错误处理这些真实议题。

需要特别说明两处实现细节，因为它们在书中被反复引用。其一，任务执行是同步的，模型在进程内缓存，没有异步队列。这是两份架构决策记录（ADR）的裁决结果，完整论证收录在设计文档集中，这里给出结论与要点。「任务执行模型选型」ADR 的结论是同步阻塞加模型实例缓存，否决了 Celery+Redis 异步队列方案，理由有四：调用方只有 QGIS 插件与冒烟脚本，特征是「发一次请求、拿一次结果」，没有排队需求；真正的瓶颈是模型加载而不是排队，队列治不了这个病，缓存能治；异步会引入任务状态机、死信处理等新失败模式，可靠性账算不过来；30 分钟干净环境部署的硬约束（DT-01★）不允许再增加中间件。「推理服务进程模型」ADR 的结论是单进程、惰性加载、按配置键缓存：常驻 GPU 进程与可用率指标冲突，每次请求重新加载模型则 P95≤3 秒的指标不可能达标，多进程隔离的收益本期用不上；缓存键必须包含模型版本与冻结构造参数以防止静默错配，这一条被固化为工程红线并配了两条回归测试。其二，文本分割阈值在 CLI 与核心模块的默认值是 0.24，而 REST 接口保留了 0.5，这是为向后兼容有意保留的差异，已在接口契约与用户手册中显式声明，不是缺陷。



### 1.5.3 样例的三种正确用法

澜图在这门课里的定位是「方法的载体」，不是「要你复制的答案」。三种用法是被鼓励的：

- 当参照系：写自己的 SRS 时，看澜图的 SRS 某一章是怎么组织的、一条功能需求的六要素是怎么写全的；比较结构，不抄内容。
- 当难度标尺：拿不准自己的项目范围是大了还是小了，用澜图的 24 FR / 5 项 MVP 对一下量级。18 个工作日内能走完全流程的项目，规模大致就在这个区间。
- 当逆向教材：5.11 节专讲「从逆向分析到正向设计」——先读懂 lantu 的实现，反推它的设计决策，再对照真实的 ADR 看自己推得对不对。这是读别人代码最有效的训练方式。

### 1.5.4 三条禁令

以下三种用法会直接损害你的学习效果，也过不了评审：

- 不要把澜图的文档改个名字当作自己的交付物。评审时一句「你们的 FR-07 为什么和澜图的一模一样」就会让整个交付包失去可信度。
- 不要把澜图的技术选型直接搬到自己的题目上。澜图选择同步执行是因为它的负载特征如此；你的题目如果是高并发场景，照搬就是错的。选型必须由你自己的需求推导出来，并写进 ADR。
- 不要用澜图的指标当自己的验收标准。「响应时间 < 3 秒」对澜图是有依据的（基于实测基线），对你的项目可能既不现实也不必要。

### 1.5.5 关于书中的旧架构案例

第 6、7 章的部分小节——尤其是研发冲刺关与质量门禁关——保留了一批围绕消息队列调度、GPU Worker 心跳与接管、死信队列、异步任务状态机展开的教学案例。这些案例取自澜图的早期架构方案 v1，该方案已在设计评审中被上述两份 ADR 否决，与 lantu/ 目录的真实实现不一致。

保留它们是有意为之。异步架构在企业真实项目和多数自选题目里仍然常见，这些案例中的测试设计思路、演练组织方式、度量方法可以直接迁移。阅读约定是：凡涉及队列、Worker、死信的段落，请当作另一类架构下的方法示范，不要拿去与 lantu/

目录的代码逐行对照。

这也是一次真实的工程体验：文档与实现的漂移无处不在。识别漂移、判断哪一方是事实源、并把差异登记下来——这正是第6关一致性核查要训练的能力。

## 1.6 一条需求的完整旅程：全书导航

### 1.6.1 走查目的

本节做一次完整的『针线活』示范：随机选一条功能需求，从招标条款出发，走完需求→验收标准→设计（HLD/DLD/DB/API/ADR）→计划任务→测试用例的全程，验证每一环的衔接质量。走查是评审前最有效的自查手段，也是学生理解『可追踪性』最直观的方式。

### 1.6.2 走查对象：FR-03.1 点提示交互分割（★款、MVP、PE-01）

#### 需求原文与验收标准

SRS V2.0: FR-04.2（★）支持正向点提示交互分割，单轮响应 $\leq 3s$ （P95）；验收标准 AC-FR-04.2-a（功能场景：加载影像→打点→返回掩膜叠加）+AC-FR-04.2-b（精度：冻结影像集  $IoU \geq 0.75$ ）。招标条款侧可溯源至招标文件对应技术条款与评分项。

#### 设计落点

- HLD: 4.1 模块表 M-03 行；4.3.3 交互式分割流程（会话→提示→ $1024^2$  裁剪→常驻 Worker→矢量化回传→确认写入）；9.1 非功能映射表 PE-01 行（常驻推理+交互独立通道+裁剪三设计项）；
- DLD: 第 4 章 M-03 全章——类设计（会话管理、提示编码、推理调用、矢量化）、主流程与异常流（提示无效/Worker 熔断/并发会话）、单元测试要点；
- DB: prompts 表（提示持久化，支撑 FR-03.5 续作）、result\_sets 与 polygons 表；
- API: /segmentation/interactive 端点族（OpenAPI v3 契约，x-roles 标注解译作业员）；
- ADR: ADR-003（常驻 GPU Worker——PE-01 的推理预算 $\leq 2s$  排除了每次加载模型的方案）。

#### 计划与测试落点

- 任务：T-15（推理 Worker 常驻加载，验证=单次推理 $\leq 2s$ ）、T-17（交互分割 API，验证=P95 $\leq 3s$  五十次抽样）、T-20（前端交互界面）；
- 测试：TC-F03-01~0n 功能用例、PT-01 性能抽样、AT-02 精度比对、降级演练（FR-08.4/08.5 联动）。

### 1.6.3 走查结论的写法

走完一圈后写结论：① 链条完整性——本例每一环都有落点，无断链；② 衔接质量——PE-01 的 3s 在 SRS/HLD/DLD/测试计划四处表述一致，且 DLD 把它分解为推理 $\leq 2s$ +渲染 $\leq 1s$ 的预算（比原文更细，但方向一致）；③ 风险点——GPU 显存假设未经实测，列入风险登记册跟踪。结论三段的格式固化下来，评审抽查时照此汇报。

### 1.6.4 学生练习

每组在自己题目上任选 3 条 FR（必须含 1 条★款、1 条 NFR）做同样走查，写一页走查报告。常见暴露问题：NFR 走到 HLD 就断了（第 9 章没做映射）；FR 走到测试断了（用例编号与 FR 域对不上）；ADR 与 DLD 矛盾（ADR 说队列并发度=1，DLD 伪代码里写了自研锁）——矛盾即缺陷，无论哪边对。

## 第 2 章 AI 辅助开发通用方法论

本章收拢不属于任何一关、却在每一关都要用的工作方法：怎么向 AI 提问，怎么让它服从项目约定，怎么审它交回来的东西，怎么把过程留成可查的证据，怎么用数据说明自己干了什么。它不是背景知识，是操作手册——2.8 节的留痕纪律如果不在第一天执行，到第 8 章的 AI 使用审计时就补不回来。建议在第一关期间读完，之后随时回查。

### 2.1 提示词工程：骨架、进阶技巧与使用纪律

#### 2.1.1 提示词的四要素骨架

本课程全部提示词遵循同一骨架——人设、结构、素材、约束，缺一不可：

| 要素 | 作用           | 写法示例                         | 缺失的后果                |
|----|--------------|------------------------------|----------------------|
| 人设 | 锁定角色与专业视角    | “你是招投标合规专家/资深售前/苛刻评委”        | 输出泛泛而谈，无专业纵深         |
| 结构 | 固定输出的章节/表格格式 | “输出六节：1...2...3...；表头固定为...” | 两次生成格式不一致，无法复用       |
| 素材 | 限定事实来源       | “只使用以下事实清单；材料如下：...”         | AI 调用先验记忆“合理想象”，幻觉入场 |
| 约束 | 划清能与不能       | “拿不准的标‘需人工复核’；禁止编造数字”        | 无法溯源的内容混入交付物         |

四要素中“素材”和“约束”是反幻觉的闸门：AI 的事实错误绝大多数不是因为模型不行，而是因为提示词没有关闭它“自由发挥”的通道。教学上要求学生在提示词中显式写出素材范围与禁止事项，并把这一条纳入过程分检查。

#### 2.1.2 提示词使用的五条纪律

- 迭代两轮起步：第一稿永远有毛病，把“批改—重生成”作为默认动作，而不是一次提示定终身；
- 大任务先拆小：先让 AI 出大纲/清单，人确认后再逐节展开——这与 skill 的阶段闸门同理；
- 同类输出喂样例：要求“参照这个格式”时给一个真实样例片段，比十句描述有效；
- 敏感信息不出门：含未公开报价、真实人员信息、涉密数据的材料不送入外部 AI 服务；课程素材一律用化名；

- 留痕：代表性提示词与 AI 输出片段归档，标注人工修改处——这是过程分的直接证据。

### 2.1.3 提示词常见败笔对照

| 败笔写法                          | 问题               | 修正写法                                |
|-------------------------------|------------------|-------------------------------------|
| “帮我写个投标书”                     | 无人设无结构无素材，输出必然泛泛 | 四要素齐备：角色 + 章节结构 + 招标解析报告素材 + 事实溯源约束 |
| “写得越详细越好”                     | 无篇幅与重点约束，低分章节膨胀  | “篇幅与分值匹配，本节目标 N 页”                  |
| “数据你自己看着编”                    | 幻觉闸门大开           | “素材外事实一律标（信息待确认），禁止编造”              |
| 一次喂 10 个任务（“写投标书顺便做个计划再列下风险”） | 注意力分散，各项都浅       | 一次一个交付物，链式推进                        |
| 不检查直接用                        | AI 初稿即终稿         | “AI 出初稿 + 人做核验”写入流程                 |

### 2.1.4 四段式结构详解（P-04 母模式）

- 第一段 角色与目标：不是『你是资深工程师』这种空角色，而是带约束的角色——『你是本项目的后端工程师，遵守 rules/constitution.md，代码评审按 R-01~R-07 清单执行』。角色段的本质是激活约束；
- 第二段 上下文：这是决定成败的一段。必须给三样真实材料——spec 条款原文（不是转述）、相关现有代码的类签名/接口定义（不是名字）、本任务涉及的需求编号。AI 输出质量与上下文真实性正相关，『给名字让它猜』是编造 API 的温床（FR-01 的教训）；
- 第三段 任务：一句话说清做什么，附输入输出契约与边界条件清单。边界条件至少列 3 个（空值、上限、并发）——你不列，AI 默认只处理快乐路径；

- 第四段 约束：禁止事项（禁止新增依赖、禁止修改既有公开接口、禁止生成 spec 之外的参数）+ 质量要求（附测试、docstring 写 Implements: FR-xx）。约束段是从翻车记录里长出来的，每条约束背后都应该有故事。

### 2.1.5 开工仪式：新会话的前五分钟

每次开启新的 AI 会话做编码任务，执行固定初始化流程，避免 AI 在零上下文下裸奔：

- 第 1 步，喂身份与规则：粘贴 constitution 摘要+本次相关的 rules 条目；
- 第 2 步，喂任务包：本任务的 spec 条款、tasks.md 条目、相关代码文件清单；
- 第 3 步，做对齐确认：让 AI 复述任务理解与边界条件，不一致当场纠正——这一步在样例项目拦截了约两成的理解偏差，成本只有两分钟；
- 第 4 步，约定产出格式：先出实现计划（伪代码/步骤清单）让我审，通过后再写代码。计划先行能把大部分方向性错误拦在代码产生之前。

### 2.1.6 反模式剖析

#### AP-01 一句话需求式提问

症状：『帮我写个影像分割接口』。AI 只能按它见过的通用模式自由发挥，产出与项目 spec 无关：参数名不对、错误码不对、连认证都没有。《Prompt 模式库》记录的对照实验：同一任务，AP-01 式提问的产出采纳率 12%，四段式的采纳率 74%。讲给学生的一句话：提示词里省的字，都会在返工里加倍还回来。

#### AP-02 让 AI 自证正确

症状：『检查一下你刚才写的代码对不对』。AI 几乎必然回答『检查过了，没有问题』——它没有真的执行代码，只是在语言层面确认了自己的输出。正确做法是给出具体验证手段：『运行 tests/test\_queue.py 并分析失败原因』『逐行对照 spec AC-03.2 列出符合与不符合项』。让 AI 自证，等于让考生自己批卷。

### 2.1.7 提示词迭代：从翻车到约束

演示一次真实的提示词演进（《Prompt 模式库》P-04 的版本史）：

- v1：三段式（无约束段）→ 产出擅自加了 spec 之外的 retry\_limit 参数；

- v2: 加约束『禁止 spec 之外的参数』→ 产出合规，但编造了不存在的库 API（FR-01 事故）；
- v3: 加约束『只使用上下文给出的 API 签名，其他库调用需标注 [未验证]』→ 编造基本绝迹，偶发时因标注明显被评审秒拦。

这个 v1→v3 的演进讲给学生，能让他们理解：提示词不是一次写对的，是拿翻车记录喂大的。

### 2.1.8 技巧一：少样本示例（Few-shot）

当输出格式要求严格时，给一个完整样例胜过十句描述。用法：在提示词中粘贴一个“输入→输出”的真实例子，再让 AI 处理新输入。任务 1 实例：让 AI 把招标条款改写成响应摘要时，先给一条澜图的真实样例——

【示例】输入：FR-03.1 ★点提示分割：应支持在地图上标注前景点与背景点.....

输出：FR-03.1★ | 点提示分割实时响应 | 特征缓存+轻量解码，单轮 ≤3 秒（4.3、5.2 节）| 无偏离  
请按同样格式处理以下条款：FR-05.1 .....

要点：样例必须本身正确（错的样例会被忠实模仿）；样例与任务同类（别拿响应摘要的样例去要风险登记册）。

### 2.1.9 技巧二：思维链外化（让 AI 先想后写）

复杂判断类任务，要求 AI 先输出分析过程再给结论，可显著减少“拍脑袋”。任务 1 实例：MVP 取舍——

在给出 5 项 MVP 建议之前，请先输出分析：（1）全部候选功能列表；（2）每项按“是否★条款/是否闭环必需/技术风险是否前置”三标准打分；（3）落选项的落选理由。最后再给 5 项结论。

要点：分析过程本身是给人看的审查材料——人不必重做分析，只需检查分析的每一步是否成立。这也呼应第二十一章：让 AI 把推理摆在明处，人类评审才有抓手。

### 2.1.10 技巧三：结构化输出契约

凡是要进入文档或表格的 AI 输出，都在提示词中定义机器可解析的格式（Markdown 表格、JSON、块数组），并加一句“只输出该格式，不要任何解释性文字”。收益：输出可直接进构建管线（第二十章），省掉人工转抄——转抄不仅慢，还会引入转抄错误。实例：编标阶段要求输出合规模矩阵行，格式固定为“条款号|性质|招标要求|响应摘要|章节|偏离”，AI 输出直接粘贴进矩阵表。

### 2.1.11 技巧四：自我批判两轮法

重要输出让 AI 自己先批判一遍再定稿：第一轮生成初稿；第二轮以“挑错者”身份审查自己的初稿并输出修改版。实例（述标讲稿）：

现在重新读你刚才的讲稿，扮演时间官与评委：（1）标出超时风险段落；（2）标出评委会追问的句子；（3）给出修改后的完整第二稿。注意：第二稿必须  $\leq 1900$  字。

要点：自我批判不能替代人工评审（同属一个模型，盲区相同），但能在人工评审前过滤掉约一半浅层问题，让人集中火力在深层问题。

### 2.1.12 技巧五：多角色辩论

对策略性选择（MVP 取舍、记忆点选择、技术路线拍板），让 AI 分饰多角辩论，人做裁判。实例：

就“MVP 是否应包含批量任务功能”进行三方辩论：甲方业务代表（主张包含）、项目经理（主张不包含，保工期）、技术负责人（中立，评估实现成本）。每人两轮发言，最后由各方用一句话总结立场。不要给结论，结论由我来做。

要点：辩论的价值不在 AI 的结论，而在它把各方论据摆全——人做决策时漏掉某个视角的概率大大降低。

### 2.1.13 技巧组合示例：一次完整的编标会话

五个技巧可以串成一条标准动作：四要素骨架立规矩（素材+约束）→ 少样本给格式（响应摘要样例）→ 结构化输出（矩阵行）→ 思维链外化（先分析后响应）→ 自我批判（二稿）→ 人工四遍评审。这条链路在澜图 V2.0 的编标阶段完整跑通过，其产物（偏离表 13 条 ★ 响应）经受住了脚本初检与人工复核的双重检验。

### 2.1.14 提示词的版本管理

- prompts/ 目录入 Git，文件名含用途与版本（extract\_fr\_v3.md）；
- 每次修改写变更注记（为什么改、防什么错）；
- 评审时抽查：随机指一条交付物内容，要求小组展示它是由哪版提示词、什么原始材料产出的——答不上来说明过程不受控。

### 2.1.15 基线投喂的分寸

基线不是越多越好：投喂整份 SRS 会稀释注意力且撑爆上下文。正确做法：按当前任务切取最小充分集——起草 M-05 模块设计时，喂 FR-05.x 原文+ADR-002+tasks 表结构+/tasks 端点片段，别的都不喂。切取责任在人：你必须知道哪几条需求与本任务相关（这正是 RTM 的日常价值——顺着挂接列找基线）。



### 2.1.16 反模式速查

- 裸问型：『帮我写个概要设计』——无基线无约束，产出必然泛泛；
- 倾倒型：把整个仓库塞进上下文——AI 抓不住重点，关键约束被稀释；
- 信任型：AI 输出直接入库——等于把质量交给了运气；
- 纠缠型：在长对话里反复纠偏——及时止损开新对话才是正解。

### 2.1.17 本节清单

- 会写四段式提示词，上下文段给真实材料；
- 执行开工仪式四步，对齐确认不省略；
- 能讲清两个反模式的机理与对照实验数据；
- 理解提示词靠翻车记录迭代的演进逻辑。
- ☐ 格式敏感任务配了少样本样例；☐ 判断类任务要求先分析后结论；
- ☐ 进文档的输出全部结构化；☐ 重要输出经过自我批判二稿；
- ☐ 策略选择用多角色辩论摆论据；☐ 全部技巧产出仍经人工评审。

## 2.2 规则文件三件套：constitution、AGENTS 与 spec

rules/constitution.md 是项目的宪法：人评审按它执行，AI 生成按它约束（开工仪式时注入）。Anthropic 与 OpenAI 的工程指南都强调：机器可读的规则文件是防止 Agent 在长流程中发散的核心手段。本节讲这份文件怎么写、怎么维护、怎么验证生效。

### 2.2.1 宪章的结构（样例实物 2 页）

- 第 1 节 项目身份：一句话项目说明+技术栈+不可协商项（安全红线、spec 源头地位）。AI 用途：锚定生成风格；
- 第 2 节 编码规范：命名、目录约定、错误处理模式、日志规范、docstring 要求（含 Implements: FR-xx 标注）。只写本项目特定约定，通用常识（如『变量名要有意义』）不写——规则文件越长，遵守率越低；
- 第 3 节 评审清单 R-01~R-07：每条一行，可勾选。样例：R-01 需求编号核对、R-07 陌生 API 附文档链接等；

- 第 4 节 AI 使用公约：必须用约束提示词的环节、必须标注 AI-generated 的范围、禁止 AI 主导的活动清单（安全代码、精度结论）；
- 第 5 节 禁止事项：新增依赖需 ADR、禁止 --no-verify、禁止直接推 main、禁止引入训练/样本库相关功能（本项目范围红线）。

### 2.2.2 编写方法：规则从哪来

- 源头一 SRS 与 spec：反常识约定（阈值 0~1、CGCS2000 存储基准）必须进宪章，否则 AI 每次都要重新纠正；
- 源头二 翻车记录：每则翻车的纠正措施如果是规则性的，回流宪章（FR-01→R-07）；
- 源头三 评审高频意见：同一类意见出现 3 次以上，提炼成规则——规则是评审疲劳的解药；
- 编写原则：每条规则可执行、可检查、有出处（注明来自哪次事故或哪条 SRS）。写不出检查方法的规则是口号，删。

### 2.2.3 让规则对 AI 生效

- 注入方式：开工仪式粘贴宪章摘要（全文的 1/4，挑与当前任务相关节）；长会话每 10 轮重注入一次，对抗上下文漂移；
- 生效验证：定期做『规则探测』——故意给 AI 一个会违反某规则的模糊指令，看它是否拒绝或提示。探测不过，说明注入方式或规则表述有问题。样例项目每迭代做 3 次探测，探测记录存档；
- 规则与 CI 的分工：能机器检查的规则（编号、红线、契约）下沉到脚本，AI 约束只管机器查不了的（风格、意图、抽象层级）。别把机器能查的事浪费在提示词里。

### 2.2.4 宪章的维护

- 修改走 ADR：宪章的每次修改附决策记录（为什么加这条），防止规则膨胀与随意性；
- 版本化：宪章随代码仓版本管理，评审时看的是 PR 对应版本的宪章；

- 每迭代瘦身一次：删掉已无触发场景的规则（被 CI 吸收的、问题已根治的），保持 2 页以内。

### 2.2.5 常见错误

| 错误        | 后果              | 纠正                  |
|-----------|-----------------|---------------------|
| 宪章写成百科全书  | AI 抓不住重点，人也记不住  | 2 页上限，只留项目特定+事故教训   |
| 规则无出处     | 评审时『为什么有这条』答不上来 | 每条注明出处（SRS 条目/翻车编号） |
| 只写不验证     | 规则形同虚设而无人察觉     | 每迭代规则探测 3 次         |
| 机器可查的靠提示词 | AI 遵守率不稳，漏检     | 下沉为 CI 脚本           |

### 2.2.6 规则文件：项目的“宪法”

#### 规则文件写什么

为任务 1 建立一个 rules.md（或 AGENTS.md），作为项目宪法，内容五项：

- 事实纪律：所有事实只来自 tender/（招标文件）、facts/（企业资料）、confirmations/（双方确认记录）三个目录；其他一律标注（信息待确认）；
- 术语与编号体系：招标条款编号照抄原文（FR-02.1★）；我方章节编号“第 X 章 / X.Y 节”；禁用自创缩写；
- 文风约定：正式公文语体；数字用法（阿拉伯数字 + 单位空格）；禁用词表（“先进的”“完美的”等宣传词）；
- 输出格式：响应摘要固定句式“结论 + 可验证事实 + 章节指引”；表格优先于大段文字；
- 禁区：不改动已冻结文件（如需求确认书）；不引入未经评审的新承诺；不删除任何 ★ 条款响应。

#### 规则文件怎么用

每次新开会话或开启新子任务时，把 rules.md 全文作为第一条消息发给 AI（或使用支持自动加载规则的客户端），并在提示词末尾加一句：“如本任务要求与 rules.md 冲突，以 rules.md 为准并提示我。”这一句话把宪法变成了最高法——AI 发现冲突时会

主动告警，而不是静默服从最新指令。

2.2.7 本节清单

- 掌握宪章五节结构与 2 页上限原则；
- 规则三源头：SRS、翻车、评审高频意见；
- 会用规则探测验证 AI 侧生效；
- 理解规则与 CI 的分工原则。

2.3 Skills：把工作方法沉淀为可复用能力包

2.3.1 为什么需要 skill：从提示词到规程

提示词解决“这一次怎么说”，skill 解决“每一次都按同一套规程做”。两者的差距在重复性工作中被放大：同一个读标流程，用提示词每次要重述要求、结果格式飘忽；写成 skill 后，AI 加载 SKILL.md 即获得完整的流程、模板、纪律与工具，输出稳定可预期。skill 本质上是“老师傅经验的可执行封装”—— Anthropic 工程指南中“机器可读规则文件”思想的教学化形态。本课程要求学生至少会“用”一个现成 skill（bid-writing），鼓励“建”一个自己项目的 skill（如开发规范、测试检查）。

2.3.2 skill 的解剖结构

一个标准 skill 是一个目录，三层结构各司其职：

| 组成 | 文件                              | 作用                                                         | 写作要点                                                  |
|----|---------------------------------|------------------------------------------------------------|-------------------------------------------------------|
| 入口 | SKILL.md<br>（ frontmatter + 正文） | 触发与总纲：<br>AI 靠 frontmatter 的 description 判断“什么时候用这个 skill” | description 必须写清能力边界 + 触发词；正文只放核心原则与流程骨架              |
| 细节 | references/*.md                 | 按需加载的细则：流程、模板、检查单                                          | 按主题拆分（workflow/compliance/quality/templates），正文引用相对路径 |
| 工  | scripts/*                       | 可执行检查脚                                                     | 规则化、确定性、输出结构化报告，声明                                    |

| 组成 | 文件 | 作用              | 写作要点    |
|----|----|-----------------|---------|
| 具  |    | 本：把“该核对的”变成机器可跑 | “不替代人工” |

设计哲学是“渐进披露”：SKILL.md 正文保持精简（AI 每次必读，占用上下文），细节沉到 references（AI 按需读取），机械核对沉到 scripts（不占用上下文，直接跑）。这套结构直接对应 Agentic 工程的最佳实践：规则文件分层 + 链式任务 + 确定性护栏。

### 2.3.3 样例剖析：bid-writing 的诞生过程

#### 第一步：调研与原则提炼

构建前先回答“这个 skill 要解决什么真实问题”。bid-writing 的输入是对业界 AI 标书工具的调研（tender\_case/AI 标书工具调研.md）：钛投标、文兜智写等工具的共性机制被提炼为四条原则——读标先于写标、合规矩矩阵驱动、按评分标准组织内容、提交前模拟评标，外加反幻觉与人机责任纪律。这些原则写进 SKILL.md 的“核心原则（不可违反）”节，成为整个 skill 的宪法。

#### 第二步：SKILL.md 的写法

SKILL.md 分两段。frontmatter 的 description 是触发关键，写法上有三个要点：（1）用“覆盖什么场景 + 什么时候使用 + 触发词”三段式；（2）触发词穷举用户可能的说法（标书、投标、招标、bid、tender、RFP、偏离表、废标、符合性检查、模拟评标）；（3）写清边界（不做什么），防止误触发。正文则固定为：核心原则 → 四阶段 workflow（每阶段一句话 + 指向 references）→ 必备产物清单 → 质量红线摘要 → 脚本用法 → 长文档指引。全文控制在 60 行以内。

#### 第三步：references 的分层

bid-writing 的四个 references 文件对应编标的四类知识：workflow.md（流程：四阶段详述 + 教学变体）、compliance.md（合规矩矩阵/偏离表/索引表的格式与规则）、quality.md（反幻觉纪律/红黄绿模拟评标/提交前检查单）、templates.md（技术标 14 章结构/评分对照表格式）。拆分标准是“使用时机”——读标阶段只读 workflow，编标阶段读 templates + compliance，查标阶段读 quality。写 references 的要点是“给模板不给范文”：模板是可套用的空架子，范文会被照抄。

#### 第四步：scripts 的开发与实测

check\_bid.py 的开发遵循“规则化、可解释、承认局限”三原则。规则化：只做确定性检查（★条款子串覆盖、占位符扫描、关键数字抽取比对、废标关键词计数），不调用 AI 做语义判断——保证同一输入同一输出。可解释：每条结果给出条款标识、原文截断、命中状态，人一眼能复核。承认局限：报告头部与尾部都声明“初检线索，需人工复核”。开发后必须用真实案例实测：bid-writing 的脚本用澜图招标 + 投标两个 docx 跑了两轮，第一轮误报 8 条“未响应”，分析后发现是条款号与前缀匹配问题，修正匹配逻辑后收敛——测试报告（体检报告-脚本初检.md）本身成为技能包的演示素材。

#### 第五步：打包、安装与验证

打包为 .skill 文件（zip 格式，顶层为技能目录），安装到用户技能目录（如 ~/.config/agents/skills/bid-writing/）。验证三件事：（1）压缩包内目录结构正确（SKILL.md 在技能根目录）；（2）安装后目录完整；（3）新会话中触发词能唤起技能。注意：当前会话的技能索引不会热加载，新 skill 下个会话才生效——教学演示时要提前装好。

### 2.3.4 使用 skill 的正确姿势

- 显式触发：直接说“用 bid-writing 检查这份标书”或使用触发词，比等 AI 自动判断可靠；
- 按阶段推进：skill 规定的阶段闸门（如“目录确认后才编标”）是防呆设计，不要为了快而跳过；
- 产物归档：skill 流程的产物（解析报告、体检报告）是过程证据，按课程要求归档；
- 脚本结果人工复核：脚本报红不恐慌（可能是表述差异），报绿不放心（资格签章它查不了）。

### 2.3.5 学生自建 skill 的练习设计

建议每个小组在任务 2 之后构建一个本组项目的 skill，候选方向：

- （1）dev-rules：本组技术栈的编码规范、目录约定、提交信息格式、CI 门禁要求；
- （2）test-guardian：测试用例规范、覆盖率门禁、缺陷分级与准出标准；

(3) doc-builder: 本组文档模板与三级签署流程。

构建过程本身就是对“把经验显性化”的训练。验收标准: SKILL.md 触发准确、references 可套用、scripts 能跑通且输出可解释、有真实使用记录(附一次使用前后对比)。

### 2.3.6 常见错误与过关检查单

#### 常见错误

- SKILL.md 写成百科全书: 正文几百行, AI 每次加载消耗大量上下文, 细节应沉到 references;
- description 太简略(只有一句话名字), AI 无法判断触发时机; 或太宽泛, 什么任务都触发;
- references 给范文不给模板, 学生照抄内容;
- 脚本做语义判断(调用 AI), 输出不稳定且无法复核;
- 构建后不实测, 直接交付——bid-writing 的第一轮实测暴露了 8 处匹配缺陷, 不测永远不知道。

#### 过关检查单

- □ SKILL.md ≤80 行, description 含场景 + 触发词 + 边界;
- □ references 按使用时机拆分, 内容为模板与规则而非范文;
- □ scripts 为确定性规则检查, 输出结构化报告并声明局限;
- □ 用真实案例实测 ≥2 轮, 缺陷修复记录保留;
- □ 打包 .skill 并安装验证(结构、触发);
- □ 有一次真实使用的过程记录(产物归档)。

### 2.3.7 构建过程: 从实践中长出来

不要先写 skill 再干活——先从本次任务 3 的实操中提取。做法: ① 完成任务 3 过程中, 把每次有效的提示词、每次审查发现的问题类型记入工作日志; ② 收尾时归类: 提示词归提示词、清单归清单、红线归红线; ③ 按五部分结构组装; ④ 用一个新主题(如下一组的题目)做冷启动验证——用 skill 起草一份 HLD 骨架, 看输出是否自然合规, 不合规处就是 skill 的缺陷, 迭代修订。

### 2.3.8 与 constitution/rules 的关系

skill 是『怎么做事』的程序性知识，constitution 是『必须遵守什么』的约束性规则。两者配合：skill 指导流程推进，constitution 在每一步被 AI 读取并约束产出。落地形态：skill 以 SKILL.md 存技能目录，constitution 以 rules 文件存仓库根目录（AI 编码工具自动加载）。任务 3 的 T-01 任务就包含 rules 文件初始化与生效验证。

## 2.4 长流程协作：上下文管理与链式任务

### 2.4.1 问题：为什么长流程会发散

任务 1 是典型的长流程协作：从读标到定稿跨越数十轮对话、数天时间。学生最常遇到的失败不是“AI 答错”，而是“AI 跑偏”——开始还记得“只响应招标原文”，二十轮对话后就开始自由发挥；上午定的章节编号体系，下午又自己发明了一套。发散的三个根源：（1）上下文有限，早期约定被挤出窗口；（2）AI 无持久记忆，每次会话从零开始；（3）多轮追加指令相互覆盖，最终生效的是哪一条谁也说不清。

Anthropic 与 OpenAI 的工程指南给出的答案是同一件事：把约定从“对话流”转移到“文件系统”——规则写进机器可读的文件，任务拆成可勾选的清单，每轮对话开始时让 AI 重新加载它们。对话是易失的，文件是持久的。

### 2.4.2 链式任务：tasks.md 防漏项

把任务 1 的全部工作写成一份 tasks.md 清单，每行一个可勾选任务，标注状态与产物链接：

```
# 任务 1 tasks.md
- [x] T01 读标四张表 → 解析报告.md (D2 已确认)
- [x] T02 澄清函 Q-01~06 提交 (D3)
- [ ] T03 偏离表三表初稿 → 责任人：需求工程师 (D6 前)
- [ ] T04 check_bid 初检 + 红项清零 (D9)
- [ ] T05 三件套生成与正文交叉核对 (D10)
规则：每完成一项勾选并附产物路径；新增任务追加在末尾，不插入中间。
```

每轮与 AI 协作开始时，连同 rules.md 一起提供 tasks.md 的当前状态，并明确“本轮只做 T03”。AI 的工作范围由此被约束在一个勾选框内——这是防止“做着 T03 顺手把 T05 也改了”这类范围蔓延的有效手段。课程要求各组 tasks.md 随归档包提交，它同时是项目管理（进度跟踪）与人机协同（任务分派）的双重证据。

### 2.4.3 分会话策略：一事一议



- 一个会话只干一件事：读标会话、编标会话（按章再拆）、查标会话分开——会话越长，早期约束稀释越严重；
- 会话间靠文件接力：上一会话的产物（解析报告）落盘，下一会话从文件读入，不依赖对话记忆；
- 关键节点重述规则：长会话每 10 轮左右重发一次 rules.md 摘要（或让 AI 先复述当前约束再继续）——复述不出来的约束等于不存在；
- 收敛用“定稿提示词”：收尾时明确“以下内容的最终版，不得再改结构，只允许按我的逐条意见修改”。

#### 2.4.4 与 skills 的关系

rules.md 管“这个项目的 facts 与约定”，skill 管“这类工作的流程与方法”，tasks.md 管“现在做哪一步”——三者是互补的层：方法层（skill，跨项目复用）→ 项目层（rules.md，本项目宪法）→ 任务层（tasks.md，当下状态）。bid-writing 的技能包 + 澜图项目的 rules.md + 每组的 tasks.md，就是任务 1 推荐的完整配置。任务 2 进入开发后，rules.md 扩展为编码规范与目录约定，同一套机制继续生效。

#### 2.4.5 事故复盘：22 轮回退

经过：某成员在一个会话里连续开发交互分割模块，对话到第 22 轮时让 AI『顺便优化一下错误处理』，AI 在优化中把第 3 轮已确认并经评审的响应结构（含 code/data/message 三段）改回了它偏好的另一套结构。该成员连续工作疲劳，没有逐行看 diff 就合并，前端当晚集成时全线报错。损失：约半天返工+一次紧急修复 PR。根因：长会话中 AI 的上下文注意力漂移+人疲劳时放弃逐行审。这条事故催生了本节的全部纪律。

#### 2.4.6 上下文漂移的三个征兆

- AI 重新提出已被否决的方案（它忘了否决记录）；
- AI 对早先确认的事实改口（比如接口结构、参数范围）；
- AI 的输出开始偏离当前任务，主动『顺便』改无关代码。

出现任一征兆，立即停止当前会话，按 23.3 的方法重启。不停下来的代价按经验是每多拖 5 轮约增加 1 小时返工。

### 2.4.7 工程化对策

- 对策一 会话长度预算：编码会话 15 轮为软上限，到 15 轮主动收敛（让 AI 总结当前状态），20 轮硬上限必须重启。复杂任务拆成多个会话串行推进，每个会话一个明确子目标；
- 对策二 状态外化：关键约定不留在对话里，写进文件——spec.md、rules、当前任务的上下文包（context pack：任务说明+相关接口+约定清单，一份 markdown）。新会话开工仪式加载 context pack，而不是指望 AI 记得上一会话；
- 对策三 会话切换清单：重启会话前，让旧会话的 AI 输出一份『交接摘要』（已确认决策、当前代码状态、待办），人工核对后作为新会话的第一条消息。核对这步不能省——AI 的交接摘要同样可能漂移；
- 对策四 重要决策落 ADR：凡在对话中做出的设计决定，当场追加进 ADR 或设计文档。对话是易失的，文档是持久的。

### 2.4.8 context pack 的制作方法

为每个开发任务准备一份 context pack（模板在附录），内容五段：任务卡（tasks.md 条目原文）、spec 条款（相关 AC 原文）、接口签名（涉及的现有类/函数真实签名）、约定清单（本项目特定的反常识约定，如阈值 0~1 浮点）、禁止事项（本任务不许动的范围）。制作耗时约 10 分钟，但样例项目的对比数据显示：带 context pack 的会话首稿采纳率比裸会话高约 25 个百分点。

### 2.4.9 人机协同要点

- 让 AI 自己造 context pack：给 AI 任务卡与仓内文件清单，让它起草 context pack，人工补『约定清单』段——AI 不知道哪些约定是反常识的，这段必须人写；
- 用新会话做交叉验证：重要产出（接口设计、算法方案）另起一个干净会话独立复核，避免复核者被原会话的上下文带偏；
- 长任务用『里程碑存档』：每完成一个子目标，把代码状态与决策存档（commit+摘要），会话崩了或漂移了从存档重启，不从对话历史里捞。

### 2.4.10 本节清单

- □ 项目 rules.md 已建立，含事实纪律、编号体系、禁区；
- □ 每会话开始加载 rules.md + tasks.md，提示词含“冲突以 rules 为准”条款；
- □ tasks.md 全程维护，勾选含产物路径；
- □ 一事一会话，产物全部落盘，不依赖对话记忆；
- □ 定稿节点使用“定稿提示词”锁定结构。
- 能复述 22 轮回退事故与双根因（AI 漂移+人疲劳）；
- 识别三个漂移征兆，执行 15/20 轮预算；
- 会做 context pack，约定清单段人工写；
- 执行会话切换清单与里程碑存档。

## 2.5 AI 初稿的人工评审：四遍法与五副眼镜

### 2.5.1 评审的对象不是“文字”是“风险”

学生常问：“AI 写得又快又顺，人工评审到底评什么？”答案是：评审的对象不是文字通顺度（AI 已经很顺），而是风险——事实风险、逻辑风险、合规风险、立场风险。文字越顺，风险藏得越深。本节给出可操作的四遍评审法，每一遍只查一类风险，避免“通读一遍觉得挺好”的无效评审。

#### 2.5.2 第一遍：事实核查（找“编造的”）

- 核查对象：全部数字、日期、名称、条款号、资质、业绩、版本号；
- 方法：逐条问“出处在哪”。出处只能是招标文件、企业资料、双方确认记录；找不到出处的，删改或标注（信息待确认）；
- 高发区：AI 对“版本号、许可证类型、标准编号、统计数字”的记忆错误率最高——澜图案例第六章开源组件清单要求人逐个核对版本与许可证，原因在此；
- 工具：把 AI 初稿与素材文件并排打开，逐条对照；禁止“凭印象放行”。

#### 2.5.3 第二遍：逻辑核查（找“站不住的”）

- 因果链：每个“因为 A 所以 B”追问一句“真的吗”——AI 擅长生成貌似合理的结果（“采用微服务架构，因此系统可靠性高”——微服务并不必然带来高可靠）；

- 前后一致：同一指标在不同章节是否一致（工期、人月、SLA 时限是重灾区）；
- 论证闭环：承诺有没有配验证方法（“响应 $\leq 3$  秒”后面有没有“如何测”）；
- 方法：画论据链——把结论列在左、论据列在右，连不上的就是缺口。

#### 2.5.4 第三遍：合规核查（找“会废标的”）

- ★条款：逐条对照招标原文，响应是否存在、结论是否无偏离/正偏离；
- 格式：目录、章节编号、签署栏、页码是否符合招标格式章；
- 禁区：占位符、旧项目名残留、AI 自述痕迹（“作为人工智能”）、绝对化宣传词；
- 工具：check\_bid.py 初检 + 附录 A 检查单人工过一遍——机器查得到的靠机器，机器查不到的（语义响应是否成立）靠人。

#### 2.5.5 第四遍：立场核查（找“不像我们说的”）

- 读者视角：这段话是写给评委看的吗？评委关心的（得分点、证据、风险兜底）说了吗？
- 口径一致：与已提交的澄清、建议书、述标讲稿是否同一口径；
- 分寸：承诺是否过度（为显得强而下验收时兑现不了的指标）；建议是否越界（合理化建议变成附加条件）；
- 方法：换位思考朗读——以评委身份读出声，别扭处即问题处。

#### 2.5.6 评审的组织与留痕

- 双人制：AI 操作者不自审，由第二人评审；重大交付物（投标书）全组会审关键章；
- 留痕：评审意见直接写在共享文档批注里，或记入评审记录表（问题/位置/依据/处置/复核人）——评审记录本身是过程分证据；
- 限时：四遍法每遍限时（如 100 页文档每遍 40 分钟），防止评审疲劳导致的“后半本放水”；
- 闭环：每条意见有处置结论（改/不改 + 理由），评审人复核后关闭。

### 2.5.7 让 AI 帮人造评审（但不替人判）

AI 可以做评审的“预筛”：把初稿发给另一个会话的 AI，要求按四遍法输出疑点清单（“列出所有无法从所附素材溯源的事实”“列出与招标文件指标不一致的承诺”）。预筛清单交人工评审组逐条裁定——真的改、误报的解释。

注意两条纪律：

（1）预筛不能替代人工四遍法，它只是提高效率；

（2）用来预筛的 AI 与生成初稿的 AI 最好是不同会话/不同模型，减少“自己审自己”的盲区。

### 2.5.8 模式一：编造不存在的 API

机理：AI 按语言模式生成『应该有』的 API，而非查文档。高发于：小众库的深层接口、版本差异大的库（rasterio、GDAL 这类地理库是重灾区）、它给你『补全』的辅助函数。识别：凡是不眼熟的调用，复制到官方文档搜索；让 AI 在生成时给每个第三方调用标注来源文档链接（P-04 v3 约束）。案例：FR-01 翻车记录（rasterio.windows.Grid）。

### 2.5.9 模式二：需求编号与条款张冠李戴

机理：AI 把编号当普通文本 token 处理，不理解其索引意义。高发于：docstring 里的 Implements 标注、RTM 关联建议、变更影响面分析。识别：所有编号类标注必须人工或脚本核对（check\_refs.py 干这个活）；绝不批量接受 AI 的编号建议。案例：FR-02 翻车记录（FR-03.1 标成 FR-03.3，RTM 错行）。

### 2.5.10 模式三：数值与估算的乐观偏差

机理：AI 生成的数值是『合理 sounding』的，不是实测的。高发于：性能估算、容量规划、超时与重试参数的默认值、测试数据规模。识别：凡数字必问出处——实测的附记录，估算的打折（团队公约打八折），凭空的拒绝。案例：FR-03（缓存命中率估 95% 实测 81%）；另有一例未入库的：AI 给队列超时默认 5s，而 spec 要求按任务规模分级，评审时发现。

### 2.5.11 模式四：过度工程

机理：AI 倾向于展示『完整』的方案——抽象层、设计模式、配置项能加都加。

高发于：新建模块的初稿、让它『完善一下』的指令。识别：对照 spec 问每一层抽象『哪条需求需要你』；用 RTM 反向诊断找孤儿代码。案例：某 PR 初稿为分块逻辑引入了三层策略模式+工厂，实际 spec 只要求两种固定分块方式，评审砍成两个函数，代码量从 280 行降到 90 行。

### 2.5.12 模式五：用常识替代需求

机理：需求与常识冲突时，AI 倾向于顺从常识。这是最隐蔽的一类，因为产出『看起来对』。高发于：领域特定规则（遥感里的坐标参考系细节、本项目阈值按 0~1 浮点而非常见的 0~100 整数）、有反直觉决策的设计（降级时任务重排用退避而非立即重试）。识别：评审时对照 spec 原文逐参数核对，特别是『反常识』的条款——这些条款恰恰是评审价值最高的地方。案例：联调问题①（前端传 0~100、后端按 0~1 解析）的变种，AI 生成的前端校验代码擅自把范围改成它认为合理的 0~100。

### 2.5.13 五类模式的速查卡

| 模式     | 一句话识别法     | 拦截工具              |
|--------|------------|-------------------|
| 编造 API | 不眼熟的调用必查文档 | 评审 R-07 + 生成时标注来源 |
| 编号错配   | 编号一律不当文本信  | check_refs.py     |
| 数值乐观   | 凡数字必问出处    | 估算打折公约            |
| 过度工程   | 每层抽象问需求出处  | RTM 反向诊断          |
| 常识替代需求 | 反常识条款逐字核对  | spec 对照评审         |

### 2.5.14 人机协同要点

- 用 AI 查 AI：把待审代码喂给另一个会话的 AI，按五类模式逐一质询（『这段代码中哪些 API 可能不存在？哪些数值没有出处？』）。交叉质询能抓出约三分之一的问题，但结论仍需人工核实——审 AI 的 AI 也会编；

- 把模式写进 rules：五类模式对应的检查规则沉淀进 rules 文件与 CI 脚本，让防线不依赖个人记忆；
- 度量驱动：每期统计五类模式的拦截分布，哪类高发就在提示词约束里针对性加防。样例项目迭代一的拦截分布是：编造 API 4 次、过度工程 3 次、数值 2 次、编号 2 次、常识替代 1 次。

### 2.5.15 本节清单

- □ 四遍法各留痕（事实/逻辑/合规/立场四类问题分别记录）；
- □ 评审人与 AI 操作者分离；□ 每条意见闭环；
- □ 高风险交付物经全组会审；□ 评审记录归档。
- 背得出五类模式及各自的高发场景；
- 掌握每类的一句话识别法与拦截工具；
- 理解『用 AI 查 AI』有效但结论必须人工核实；
- 会把模式防线沉淀进 rules 与 CI，而非依赖记忆。

## 2.6 翻车模式库与翻车记录制度

任务 4 硬性要求每组至少提交 1 次 AI 翻车的根因分析（《AI 翻车记录》收录了 FR-01/02/03 三则完整记录）。『无记录者扣分』的依据是：深度使用 AI 一个月却零翻车，只有两种可能——没深度使用，或没审视产出。本节讲翻车记录怎么写、根因怎么挖。

### 2.6.1 什么算『翻车』

入库标准：AI 产出导致了实质损失（返工超过 2 小时、引入的缺陷存活超过一个构建周期、或误导了设计决策），且事后能讲清根因。AI 写错一行代码被当场发现不算翻车——那是常态；AI 给的方案错了你没审出来、合进去烧了时间，才算。判定边界上宽一点：宁可多记，不能没有。

### 2.6.2 翻车记录格式（《AI 翻车记录》规范）

| 字段    | FR-01 实际内容（编造 API 事故）   |
|-------|-------------------------|
| 编号与标题 | FR-01 AI 编造不存在的栅格处理 API |

| 字段     | FR-01 实际内容（编造 API 事故）                                               |
|--------|---------------------------------------------------------------------|
| 经过     | AI 生成影像分块代码时使用了 rasterio.windows.Grid 这个不存在的类，审查者未核实即合并，CI 通过后运行期崩溃 |
| 损失     | 返工 6 小时；该 PR 后续被整体重写（PR#41）                                         |
| 直接原因   | AI 编造 API；评审未执行『陌生 API 必须查官方文档』规则                                   |
| 根因（五问） | 见下节分解                                                               |
| 纠正措施   | 评审清单新增 R-07：所有陌生 API 引用必须附官方文档链接；P-04 约束段加入『只使用上下文给出的 API 签名』       |
| 验证     | 此后两个迭代未再发生同类事故；评审拦截记录中 R-07 触发 4 次                                  |

### 2.6.3 五问根因法实操

五问（5-Whys）的关键是每一问都要落到可改变的事实上，不能停在『人的问题』。以 FR-01 为例：

- 为什么运行期崩溃？因为代码调用了不存在的 rasterio.windows.Grid；
- 为什么会调用不存在的类？因为 AI 编造了它；
- 为什么编造的代码进了主干？因为评审者没核实这个陌生 API；
- 为什么没核实？因为当时评审清单没有『陌生 API 必查文档』这条，凭印象觉得像真的；
- 为什么清单没有这条？因为之前没把『AI 会编造 API』列为已知风险模式。→ 根因落在：团队的 AI 风险认知缺口。纠正措施就针对这一层：清单加 R-07，提示词加约束，风险登记册加模式。

教学要点：五问做到第五问时，如果答案还是『某人不小心』，说明没挖到底——不小心的背后是流程没设防。纠正措施必须落在流程/清单/工具上，落在『下次注意』上的措施等于没有措施。

### 2.6.4 三则样例翻车速览

| 编号    | 事故              | 根因落点    | 纠正措施                 |
|-------|-----------------|---------|----------------------|
| FR-01 | 编造 rasterio API | 风险认知缺口  | 评审清单 R-07 + 提示词约束    |
| FR-02 | AI 把需求编号张冠李     | RTM 即时登 | check_refs.py 反向校验脚本 |



| 编号    | 事故                                      | 根因落点       | 纠正措施                    |
|-------|-----------------------------------------|------------|-------------------------|
|       | 戴（FR-03.1 写成 FR-03.3），RTM 登记错误          | 记依赖回忆      |                         |
| FR-03 | AI 性能估算乐观（缓存命中率 95% vs 实测 81%），优化收益承诺落空 | AI 估算无背书机制 | AI 数值估算一律人工复核并打折，写入团队公约 |

### 2.6.5 翻车记录的运营

- 双周复盘会：过一遍本期新翻车记录，提炼共性——如果两则翻车的根因落在同一层（比如都是评审清单缺口），合并纠正措施并升级优先级；
- 向提示词库回流：好的纠正措施往往就是一条提示词约束或一个评审清单项，《AI 翻车记录》与《Prompt 模式库》之间有 6 处这种回流，交叉引用；
- 答辩素材：翻车记录是答辩时证明『我们真的深度用了 AI 且保持了工程判断力』的最硬证据，鼓励学生主动讲。

### 2.6.6 人机协同要点

- AI 辅助五问：让 AI 扮演质询者连续追问，人来回答——AI 追问的锋利度常常超过组员互问，因为它没有人情负担；
- AI 归纳跨记录模式：把多则翻车记录喂给 AI，让它找根因聚类，人工裁决。样例项目 AI 发现『三则翻车有两则根因涉及评审清单缺口』，促成了评审清单的整体升级；
- 底线：翻车事实与损失数据必须人工核实填写，AI 不参与——这类记录的可信度就是它的全部价值。

### 2.6.7 案例一：AI 初稿直接当终稿

情景：小组把 AI 生成的建议书初稿直接排版提交。后果：文中“某权威机构统计，80% 的国土部门已部署 AI 解译”纯属虚构，评审时被要求出示出处，全组哑然，诚信档降档。根因：违反“AI 产出只是初稿”原则，无事实核验环节。正确做法：任何数字必须溯源（招标文件/公开报告/标注教学示例），核验人签字后入稿。

### 2.6.8 案例二：偏离表改写招标原文

情景：为使偏离表“简洁”，把招标要求列改写成摘要。后果：评委核对时需做语义等价判断，两条响应被判“未实质响应”，进入澄清程序险些出局（澜图 V1 的真实问题，V2 改为照抄原文后解决）。根因：把可读性置于可核对性之上。正确做法：招标要求列照抄原文关键句，响应列写可验证事实 + 章节指引。

### 2.6.9 案例三：章节标题缺失导致目录断链

情景：内容写完了，但少了一个一级标题（澜图 V1 第十二章）。后果：目录、索引表、评分对照表全部指向一个“不存在”的章节，评委按评分表找不到“项目团队”，6 分大项起评受损。根因：结构完整性靠通读检查，通读查不出“缺的东西”。正确做法：对照 14 章模板做机械核对（清单比对），脚本或人工均可，但必须“按单查”而不是“看着查”。

### 2.6.10 案例四：占位符带进正式版

情景：业绩章用“××市”占位，计划“定稿前替换”，最后忘了（V1 命中 9 处）。后果：符合性审查直接标记“标书未完成”。根因：没有提交前的占位符扫描。正确做法：check\_bid.py 或全文检索（××、【】、TODO、待补充）作为定稿闸门的必查项；教学场景用化名而不是占位符。

### 2.6.11 案例五：承诺相互打架

情景：第一章声明“质保 12 个月”，第十章 SLA 表写“质保期内 4 小时响应”，而招标文件要求“2 小时响应”；人月计划在第八章写 31、第十二章写 28。后果：评委抽查一致性发现矛盾，响应性降档并被质询。根因：分头写作后无一致性专项核对。正确做法：关键数字（工期/质保/时限/人月）建“单一事实源”表，各章引用同一出处，查标时专项比对。

### 2.6.12 案例六：澄清环节自问自答

情景：自拟题目的小组自己写招标文件、自己提澄清，问题全是“能答的”。后果：评审被认定走过场，过程分扣减。根因：缺少对抗性。正确做法：交换评审——A 组的招标文件由 B 组提澄清问题，提问质量本身纳入评分。

### 2.6.13 案例七：MVP 什么都要

情景：需求确认书列了 9 项“核心功能”，每项都是 Must。后果：后续开发全线吃紧，Sprint 1 就延期；评审指出“不会取舍 = 不懂 MVP”。根因：把 MVP 当功能清单而不是“最小闭环”。正确做法：≤5 项硬约束 + 业务闭环检验（入口—处理—出口缺一不可）+ 范围外清单明确写出。

#### 2.6.14 案例八：风险登记册一写了之

情景：识别了 15 条风险，等级全“中”，应对全“密切关注”，之后再没打开过。后果：双周评审被问及 R03 状态，无人能答；真实风险（GPU 未到位）爆发时无预案。根因：登记册没有进入例会机制。正确做法：双周评审逐项过状态，高风险有触发器与预案，版本随状态升版。

#### 2.6.15 案例九：述标背标书目录

情景：讲稿按“第三章总体方案、3.1 节……”逐章念，8 分钟没讲到功能亮点即超时被打断。后果：述标分接近及格线，评委反馈“没记住任何东西”。根因：按文档结构而非评分表组织叙事。正确做法：每节对应一个评分因素、一个记忆点、一组必说数字；排练按 7 分 40 秒收束。

#### 2.6.16 案例十：过程产物最后补造

情景：过程分检查前夜，小组集中“补写”体检报告与演练记录。后果：文件时间戳与内容版本对不上，被认定为过程造假，过程分清零并通报。根因：把过程产物当“作业”而不是“证据”。正确做法：产物随工作即时归档（解析报告 D2、体检报告 D9、演练录像 D11），版本管理与时间戳天然留痕——过程分考察的本来就是“过程是否真的发生”。

#### 2.6.17 案例库的使用

- 开工前：全组通读一遍，认领“本组最可能犯的两条”，写入组内规则文件；
- 查标时：把十个案例作为检查单的补充——每条案例对应一个可检查项；
- 答辩时：评委抽案例追问，答出“我们的机制是什么”即可得分，答“我们不会犯”不得分。

#### 2.6.18 陷阱一：镜像断言

现象：AI 生成的测试断言不是独立计算的期望值，而是把被测代码的逻辑复述一

遍——被测函数算错了，断言跟着错，测试永远绿。典型样例（真实翻车记录 R-07）：AI 为面积计算函数写测试，断言里直接调用同一个面积计算工具函数算期望值——实现 bug（投影换算漏了一级）在断言里原样复现，测试通过，直到 AT-03 人工抽测才发现面积偏差。

防线：断言独立性检查——期望值必须来自独立于实现的信息源（手工计算的样例、权威工具的输出、SRS 里的数值示例）。评审 AI 测试代码时问一个问题：『如果把被测函数改错，这个断言会变红吗？』答不上来的断言就是镜像断言。本项目入库评审累计打回镜像断言 7 处。

### 2.6.19 陷阱二：不可判定预期

现象：AI 生成的用例预期结果用自然语言糊弄——『返回合理的数据结构』『页面展示正常』『性能可接受』。这些预期无法判定 pass/fail，执行时测试员只能凭感觉打钩，用例形同虚设。根源：AI 面对信息不足（SRS 没写明的细节）时不拒绝输出，用模糊措辞填坑。

防线：可判定性检查进用例入库质检（3.6 节五条铁规第一条）。具体操作：每条预期必须含可观察的断言点（状态码、字段值、数值区间、UI 元素可见性）。实操技巧：让 AI 自查——『把这份用例集中所有无法用程序或规程判定的预期标出来』，AI 自查不可判定预期的准确率意外地高（它知道自己在糊弄），但你不问它就不说。

### 2.6.20 陷阱三：臆造缺陷与臆造步骤

现象两种：其一，AI 生成的缺陷单含从未真实执行的复现步骤（它『推演』系统会出错）；其二，AI 汇总执行结果时把没跑过的用例标成通过（生成日报时自作主张补全缺失数据）。后者更危险——直接污染测试报告的数据根基。

防线：执行记录单一权威源——pass/fail 状态只能由执行系统（或人工签字的执行记录）产生，AI 只读不写；缺陷单提交前人工验证复现步骤真实执行过（在缺陷单加『实际执行人』字段）。本项目执行期抓到过 1 次 AI 日报虚标（2 条阻塞用例被标为通过），此后 AI 日报强制与执行系统 diff。

### 2.6.21 元风险：测试者技能退化

三陷阱之外还有一个慢变量风险：全部用例让 AI 想，队员自己不会想了。课程的教学目标是培养人，AI 是脚手架不是替身。对应机制（与任务 4 一致）：每人必须手

写至少 3 条用例并讲清设计依据（等价类怎么分的、边界值为什么取这个点）；答辩随机抽用例问『这条为什么这么设计』，答『AI 写的』即扣分。

### 2.6.22 防线汇总：测试资产的 AI 门禁

把本节防线收敛成一张门禁表（贴在用例库首页）：

| 风险     | 检查点    | 检查方法                  | 责任人    |
|--------|--------|-----------------------|--------|
| 镜像断言   | 断言独立性  | 改错被测函数，断言应变红          | 用例评审人  |
| 不可判定预期 | 可判定性   | 每条预期有可观察断言点           | 用例入库质检 |
| 臆造缺陷   | 复现真实性  | 步骤实际执行并留执行人           | 缺陷提交人  |
| 数据污染   | 状态单一权威 | pass/fail 只能来自执行系统    | 测试负责人  |
| 技能退化   | 手写配额   | 每人 $\geq 3$ 条手写用例且能答辩 | 组长     |

### 2.6.23 本节清单

- 掌握翻车判定标准（实质损失+可讲清根因）；
- 会用五问法挖到流程层根因，纠正措施落在清单/工具上；
- 熟悉三则样例翻车及其回流机制；
- 理解翻车记录作为答辩证据的价值。
- 能各举一个真实例子讲清三陷阱（R-07 镜像断言等）；
- 门禁表五项检查有执行记录；
- AI 日报与执行系统 diff 机制运行；
- 手写用例配额完成，答辩抽问能扛。

## 2.7 红线、伦理与学术诚信

### 2.7.1 为什么单独讲这一章

AI 辅助把“会不会做”变成了“做得对不对”。任务 1 的全部产物在课程中被视为学

术作业，在真实场景中被视为商业文书——两个身份都有刚性规则。本节划出四条线：数据安全线、知识产权线、学术诚信线、责任归属线。越过任何一条，之前所有方法带来的效率都会归零甚至为负。

### 2.7.2 数据安全线：什么不能送进 AI

- 真实商业投标中的客户未公开信息、成本与报价底数、人员身份证号等，一律不得送入外部 AI 服务；需要 AI 处理的，先做脱敏（名称化名、数字区间化）；
- 课程场景中：自拟题目涉及真实单位时同样化名处理（澜图样例全部使用化名：智图公司、临江市、北江省、清溪县），这既是合规训练也是职业习惯；
- 招标文件本身在真实场景中属于受控资料，使用 AI 处理前须确认采购人的保密要求；教学素材为自编文件，无此限制，但要在报告中说明这一点以示知情。

### 2.7.3 知识产权线：AI 产出与素材的版权

- 不盗图：架构图、界面图、甘特图全部自绘（脚本生成优先），不得从竞品方案、论文、网站截图盗用——澜图案例 18 张插图全部脚本自绘并标注“示意”；
- 引用即标注：引用开源项目的文字或数据（功能点、性能数据、界面截图）一律注明来源；逆向分析形成的招标文件是教学再创作，不与原项目同名同标识（澜图的命名即为规避示例）；
- AI 生成内容的权属：课程约定——AI 生成、经人工实质性修改与核验的内容，视为小组作品，但须在归档材料中附 AI 使用声明（见 2.7）；直接未改的 AI 原文不得声明为原创。

### 2.7.4 学术诚信线：AI 使用声明制度

本课程不要求“不用 AI”，而要求“透明地用”。每份交付物附一段 AI 使用声明，固定三要素：

AI 使用声明：本交付物使用了【工具名称】辅助【环节：如素材结构化/初稿生成/规则检查】。  
全部事实类内容经【姓名】人工核验；AI 原始输出与人工修改记录见归档包【路径】。  
本组对内容的准确性与合规性承担全部责任。日期：.....

- 隐瞒使用 = 学术不端（与抄袭同等处理）；声明使用 + 核验留痕 = 正当方法——两者的区别只在透明度；

- 教师用四遍评审法中的“立场核查”反向验证：答辩追问“这段是你写的还是 AI 写的，改了哪里”，答得出即合规，答不出即存疑。

### 2.7.5 责任归属线：署名即负责

无论 AI 参与多深，签署栏上的名字承担 100% 责任。这条线要让学生形成条件反射：AI 不会产生法律责任，不会出席验收会，不会被评委追问——这些时刻只有人在场。课程中执行“签署前走读”制度：任何成员在自己署名的文件上签字前，必须能回答文件中最关键的三个数字出自哪里。澜图案例的全过程（四遍评审、体检报告、签署栏）都在为这条线提供证据链。

### 2.7.6 四条线的检查单

- □ 送入 AI 的素材已脱敏/为教学素材，无真实敏感信息；
- □ 全部图表自绘或引用标注，无盗图；□ 无与既有项目重名重标识；
- □ 每份交付物附 AI 使用声明三要素；□ 归档包含 AI 原始输出与人工修改记录；
- □ 署名成员能回答本文件关键数字的出处。

### 2.7.7 声明体系：四层声明，层层可追溯

| 层级 | 载体        | 声明什么                | 粒度    |
|----|-----------|---------------------|-------|
| L1 | Prompt 日志 | 每次 AI 会话的输入产出处置     | 单次使用  |
| L2 | PR 描述     | 本 PR 的 AI 参与情况与日志索引 | 每个 PR |
| L3 | 翻车记录      | AI 重大失误的根因与对策       | 每起事件  |
| L4 | 复盘报告      | 全周期 AI 协作量化数据与口径    | 项目级   |

四层声明的内在一致性是审计的总抓手：L4 报告的『起草 58%、保留 44%』必须能从 L1 日志聚合出来；L1 日志必须与 L2 PR 描述互指；L3 翻车记录里的案例必须能在 L1 找到原始会话。四层闭环，一处造假处处对不上——这就是『造一处假等于全程存疑』的数学原理。

### 2.7.8 违规分级：疏忽、失信、伪造

| 级别 | 行为                          | 处理              | 澜图实例            |
|----|-----------------------------|-----------------|-----------------|
| 疏忽 | 漏记日志，发现后打标补录                | 提醒，纳入完整率统计      | 7 次日检缺漏，均合规补齐   |
| 失信 | 补录不打标、时间线美化                 | 扣 0.5~1 分+通报    | PR#36 扣 0.5 分通报 |
| 伪造 | 虚构未发生的会话、篡改时间戳、AI 代写日志充原始记录 | 本关不合格，按课程诚信条款处理 | （教学红线，样例未发生）    |

## 2.7.9 翻车记录为什么强制：深度使用的证明

课程要求每组至少 1 份翻车根因分析，无记录者扣分——理由写得很直白：『没有翻车记录，说明没有深度使用，或没有审视』。澜图组攒了 5 起翻车，最有教学价值的是『AI 生成的坐标转换在跨带投影时偏差 300 米』：AI 自信地给了错误 EPSG 参数，代码 review 没查出来，是红队自测抓到的。翻车记录的结构：会话还原（当时怎么问的）→ 错误点（AI 错在哪）→ 漏检点（人为什么没拦住）→ 对策（此后此类输出加什么校验）。写出来你会发现：翻车的根因几乎从不是『AI 太笨』，而是『人没给上下文』或『人放弃了判断』。这个认知就是课程要的。

### 2.7.10 诚实纪律的经济学：为什么诚实是最优策略

给学生算笔账：漏记补录打标，代价是完整率 97.4% 而不是 100%——审计结论是『完整率优秀』，零扣分；不打标被抓，代价 0.5 分+通报+答辩时被追问；伪造被抓，本关不合格。而伪造的收益是什么？只是把 97.4% 变成纸面 100%。用本关不合格的风险换 2.6 个百分点的完整率——这是最差的交易。审计文化的本质：把『诚实』设计成占优策略的制度，然后要求所有人理解这个设计。

### 2.7.11 人机协同要点

- 诚实纪律没有『人机协同』——纪律本身是人的事，AI 只能用于自查与统计，永远不能用于生成声明内容；
- 可以给 AI 做价值观对齐演练：让它扮演审计员攻击你们的日志体系，找出所有『可能被怀疑伪造』的点，人据此加固记录习惯。

### 2.7.12 红线一：披露类内容的措辞，AI 不得改写



披露类内容：验收报告遗留事项、技术债务台账、AI 使用声明、翻车记录。这类内容的共同特征是『说自己的不足』。AI 的本能是把『目视可辨的边缘锯齿』润色成『边缘平滑度有优化空间』——语义没变，严厉度变了，而严厉度正是披露的诚意。规则：披露内容的每个措辞由责任人亲笔（或口述录入），AI 只允许做拼写检查。验收答辩时评委对遗留事项逐条追问，措辞是你答辩的立场文件——它必须是人想清楚后的话。

### 2.7.13 红线二：结论类内容，AI 不得生成

结论类内容：复盘报告的归因与机制结论、ROI 净收益的最终表述、E/I 系列、验收演示的价值总结。AI 可以给素材、给草稿、给反方观点，但落笔成文的结论必须是人的。理由：结论是你们小组对一个月工作的『判断』，评委验收的正是这个判断力——AI 代下结论等于替考。实操边界：AI 写的归因草稿可以放在桌上，人合上它，用自己的话重写——允许被启发，不允许被代笔。

### 2.7.14 红线三：补写日志是重罪

2.8 节讲过补录打标的规矩，这里升格为红线：任何试图让日志『看起来像当时写的』的行为——改时间戳、模仿笔迹、让 AI 按 commit 倒推生成会话记录——按伪造处理，本关不合格。为什么单独升格：研发关的违规多是疏忽，收口关的『补日志』必然是故意——审计在即，动机明确，没有任何『不小心』的辩解空间。PR#36 的 0.5 分是无意的代价，供参照。

### 2.7.15 红线自查：验收前 24 小时的最后一遍

- 抽查 交付审计类样例文档（文档审计、可移植性验证、AI 使用审计、验收汇报 PPT、复盘报告、验收报告）中所有披露/结论段落：写作过程能否说出（谁写的、什么时候、AI 参与了吗）；
- 日志目录最终扫描：所有补录是否打标，时间戳是否有任何修改痕迹；
- PPT 逐数核对：每个数字能报出溯源路径（文档编号+章节）；
- 全组口头确认三条红线——验收前夜会上每人复述一遍，形成会议记录。

### 2.7.16 本节清单

- 四层声明闭环可互查，任取一条能双向追溯；

- 翻车记录  $\geq 1$  份且含会话还原与对策；
- 补录全部打标，无时间线美化；
- 全组理解违规分级与诚实占优逻辑（答辩可能考）。
- 披露措辞人工定稿，AI 仅拼写检查；
- 结论由人落笔，AI 草稿不入正文；
- 日志无任何时间线修饰，补录全打标；
- 红线自查四项完成并有会议记录。

## 2.8 过程留痕：Prompt 日志与模式库建设

AI 使用审计的通过率，不取决于你们 AI 用得再好，取决于日志攒得多全。澜图组归档完整率 97.4%——丢了的那 2.6% 就是 PR#36 的扣分点。本节讲 Prompt 日志的归档规范：格式、节奏、值班检查，以及为什么这套东西必须 D1 就立规、补录必须打标。

### 2.8.1 归档格式：一条日志的七个字段

| 字段        | 内容                     | 示例                                 |
|-----------|------------------------|------------------------------------|
| 时间戳       | 精确到分钟                  | 2026-08-27 14:32                   |
| 操作者       | 谁发起的会话                 | 丙（后端）                              |
| 任务关联      | 对应 tasks.md 条目 / issue | TASK-118 分割结果矢量化接口                 |
| 工具与模型     | 用的什么                   | Copilot / Claude Code / Kimi       |
| Prompt 摘要 | 要点式摘要（全文存档可附件）         | 要求生成栅格转矢量的异步接口，附 spec FR-06.1 验收标准 |
| 产出处置      | 采纳/修改后采纳/弃用 + 理由       | 修改后采纳：阈值默认值改为 0.5（spec 约定）         |
| 关联 PR     | 合并后的回填字段               | PR#76（合并时由值班员回填）                   |

### 2.8.2 D1 立规：第一周就定下的事

澜图组在 D1 就立了三条规：日志文件按日存放（logs/prompts/2026-08-27.md），格式用统一模板；当日日志当日写，最迟次日站会前补齐；值班员（轮流）每日检查

前一日日志与该日 commit 的对照。为什么必须 D1 立规：AI 使用贯穿全周期，但审计在 D17~D18——如果日志从研发冲刺才开始攒，此前的 AI 使用要么靠回忆补录（不可信），要么干脆缺失（直接扣分）。《AI 使用审计报告》审计能抽 PR 就能对日志，靠的是全程攒出来的 18 天记录。

### 2.8.3 值班员日检：97.4% 完整率怎么来

值班员日检的动作很轻（10 分钟）：打开昨日 commit 列表，逐条问『这条 commit 有没有 AI 参与？有的话日志在哪？』发现缺漏当场 @责任人补齐。这个机制的本质是把『期末大审计』拆成『每日小对账』——期末才发现缺 20 条日志是不可修复的，每天发现缺 1 条是可修复的。澜图组 19 天检出缺漏 7 次，全部 48 小时内补齐。那 2.6% 的缺失发生在 D4 夜班：丙在凌晨改 BUG-136 用了 AI，日志没写，值班员次日检查时该 commit 还没提 PR（无对照物），漏检了。

### 2.8.4 【补录】标注：诚实是归档规范的一部分

日志会缺，这不可避免；缺了怎么办，是纪律问题。澜图组规则：补录的日志必须打【补录】标并注明补录日期。PR#36 栽就栽在没打标——D4 的日志 D7 补写，没标，混在正常日志里，审计靠 commit 时间戳与日志笔迹风格识别出后补痕迹，定性为『日志断档后补未声明』，扣 0.5 分通报。0.5 分不多，但教训值钱：补录不可耻，伪装成原始记录才可耻。打了【补录】标的日志在审计中按『完整率打折扣但诚实无虞』处理。

### 2.8.5 日志的副产品价值：不止为审计

- Prompt 模式库的货源：94 次复用中 Top3（P-04×17、P-06×12、T-P02×9）全部从日志里筛出来；
- 翻车记录的线索：5 起翻车的复盘都从日志还原了『当时怎么问的』；
- 采纳率统计的基础：63% 平均采纳率是逐条日志的『产出处置』字段聚合而来；
- 答辩举证弹药：评委质疑某段代码时，翻到对应日志讲出『为什么这么生成、人改了什么』。

### 2.8.6 人机协同要点

- 日志模板、日检脚本（commit 与日志条目比对）适合 AI 生成与半自动执行；
- 期末统计（采纳率、复用次数、工具分布）让 AI 从日志目录聚合，但聚合口径要人先定义（什么算『修改后采纳』）；
- 红线：日志内容必须人写——AI 生成『看起来像真的』的日志是本课程最重违规之一（见 2.7 节）。

### 2.8.7 从『聊天记录』到『团队资产』

没有模式库时，好提示词的命运是：某人某天调出一个好用的提问方式，用完即弃，队友不知，下周自己也忘。模式库要解决的三个问题：复用（别人能直接用）、演进（踩坑后能改进）、度量（知道哪些模式真的有效）。入库标准三条：用过至少 2 次且有效、有明确的适用场景边界、有失败案例或注意事项。只成功一次的提示词先入『候选区』，复验后转正。

### 2.8.8 模式卡片格式（《Prompt 模式库》规范）

每个模式一张卡片，字段以 P-04（代码生成模式）为例：

| 字段    | P-04 实际内容                                                |
|-------|----------------------------------------------------------|
| 编号与名称 | P-04 四段式代码生成提示词                                          |
| 适用场景  | 依据 spec 实现新功能模块（非探索性代码）                                  |
| 提示词模板 | 角色+上下文（spec 条款原文/相关类签名/编码规范摘要）+任务（输入输出、边界）+约束（禁止事项、测试要求） |
| 使用记录  | 迭代一 12 次，采纳率 74%                                         |
| 失败案例  | 用于探索性脚本时约束段反而限制产出，改用 P-07                                |
| 注意事项  | 上下文段必须给真实类签名，AI 凭名字猜签名是其编造 API 的源头                       |

写模板的关键技巧：把变量部分用尖括号标出（如 <spec 条款原文>），让使用者一眼看到要替换什么；把踩坑总结直接写进约束段（如 P-04 约束里有『禁止生成 spec 之外的参数』一条，来自一次 AI 擅自加参数的事故）。

### 2.8.9 样例模式速览（P-01~P-12）

| 编号   | 名称                      | 场景类别  |
|------|-------------------------|-------|
| P-01 | 需求反询模式（让 AI 对 spec 提问）  | 需求澄清  |
| P-02 | 验收标准生成（Given/When/Then） | 需求→测试 |
| P-03 | 设计草案对比生成                | 设计探索  |
| P-04 | 四段式代码生成                 | 编码主力  |
| P-05 | 代码评审清单生成                | 评审辅助  |
| P-06 | 测试补全（按未覆盖行号）            | 覆盖率提升 |
| P-07 | 探索性脚本生成                 | 一次性工具 |
| P-08 | 重构建议（小步走）               | 重构    |
| P-09 | 假设-验证调试法                | 调试    |
| P-10 | 最小复现构造                  | 调试    |
| P-11 | 红队自查（攻击者视角）             | 安全    |
| P-12 | 文档-代码一致性扫描              | 配置管理  |

另有 2 个反模式（AP-01 一句话需求式提问、AP-02 让 AI 自证正确），2.13 节详讲。

### 2.8.10 模式库的运营机制

- 专人运营：设模式库管理员（可由测试主攻兼任），负责入库审核与季度清理；
- 入库评审：新模式的 PR 要附使用记录（至少 2 次），评审看模板是否真正参数化、注意事项是否来自真实踩坑；
- 使用登记：用哪个模式做了什么 PR，在 PR 描述里记一笔模式编号——这是度量采纳率的数据源；
- 定期淘汰：连续一个迭代无人使用的模式降级到『休眠区』；被新工具链替代的（比如某模式的能力被 rules 文件吸收）归档并注明替代关系。

### 2.8.11 人机协同要点（元层面）

模式库本身是『关于如何用 AI 的文档』，它的建设也能用 AI：让 AI 从使用记录里归纳候选模式（『这三次提问结构相似，可否抽象成模式？』），人工判断是否真有共性；让 AI 给模式起草模板初稿，人工把踩坑经验注入注意事项。但模式的『有效/无效』判定必须来自真实使用记录，禁止 AI 凭理论推断入库。

### 2.8.12 本节清单

- 七字段模板 D1 启用，按日存放；
- 值班员日检机制运转，缺漏 48 小时内补齐；
- 补录一律打【补录】标并注明日期；
- 日志与 PR 双向可查（日志有 PR 字段，PR 描述引日志行号）。
- 理解入库三标准（复验、场景边界、失败案例）；
- 会写模式卡片，模板参数化、注意事项来自真实踩坑；
- 记住 12 个样例模式的分类地图，知道去哪查；
- 掌握运营四机制：管理员、入库评审、使用登记、定期淘汰。

## 2.9 人机分工矩阵：六关一张表

本节把编码期的活动逐项拆成『AI 主导/人机协同/人工主导』三档，给出理由与操作要点。这张矩阵是任务 4 过关条件『AI 策略具体可执行』在编码关的落地答卷。

### 2.9.1 分工矩阵总表（第 4 关）

| 活动     | 分工档   | AI 做什么    | 人做什么              | 依据              |
|--------|-------|-----------|-------------------|-----------------|
| 业务代码初稿 | AI 主导 | 按四段式提示词生成 | 审五类缺陷模式、跑测试       | P-04，采纳率 63~74% |
| 测试用例编写 | 协同    | 按未覆盖行补用例  | 审期望值来源（防镜像）       | 2.3 节           |
| 接口契约维护 | 协同    | 补标注初稿     | 逐条确认 x-fr 与 SRS 对 | 2.2 节           |

| 活动          | 分工档  | AI 做什么            | 人做什么       | 依据       |
|-------------|------|-------------------|------------|----------|
|             |      |                   | 应          |          |
| 性能优化方案      | 协同   | profiling 解读、收益估算 | 假设裁决、估算打折  | FR-03 教训 |
| 精度结论        | 人工主导 | 计算与归因分析辅助         | 达标判定签字     | 验收承诺     |
| 安全敏感代码      | 人工主导 | 红队自查辅助            | 双人评审含安全责任人 | 2.4 节    |
| 变更影响面       | 协同   | 全局搜索清单初稿          | 补漏与批准      | CR-001   |
| 技术决策（ADR 级） | 人工主导 | 方案对比素材生成          | 决策与理由记录    | 决策责任不可外包 |
| 代码合并裁决      | 人工主导 | —                 | 评审意见汇总与拍板  | 门禁责任     |
| 翻车根因分析      | 协同   | 五问质询者             | 事实填写与措施落地  | 2.12 节   |

## 2.9.2 档位判定的三条原则

- 可验证性原则：产出的对错能被机器快速验证的（代码跑测试、标注对脚本），可以让 AI 主导；验证本身需要判断的（精度达标、方案合理），人工主导；
- 责任原则：对外有承诺的（验收指标、安全、决策记录），责任人必须是人，AI 只能辅助；
- 成本原则：AI 做错时人工纠错成本低的（初稿、草稿、清单），放手让 AI 做；纠错成本高的（架构、公共接口、安全逻辑），人工主导从源头控。

## 2.9.3 矩阵的使用方式

- 冲刺计划时按矩阵派活：每个 tasks.md 任务标注分工档，AI 主导任务的关注点是评审人排班，人工主导任务的关注点是 AI 辅助材料准备；

- 新人 onboarding 用矩阵建立边界感：先让学生全人工做一遍小任务（建立手感），再开放 AI 主导档位——直接上 AI 的学生没有『对的手感』，审不出错；
- 答辩时矩阵是证据：能按矩阵讲清每类活动为什么这样分工，比空谈『我们用 AI 提效』有说服力得多。

### 2.9.4 动态调整

矩阵不是定死的。两个迭代中真实的调整：测试数据构造从人工主导降为 AI 主导（因为校验脚本成熟）；变更批准从组内双签升级为教师备案（因为 CR-001 涉及验收指标）。每次调整记录在团队公约的修订历史里，答辩可追。

### 2.9.5 为什么骨架与裁决不外包

骨架（章节结构、模块划分）决定文档的信息组织，外包给 AI 等于把评审的导航权交给不了解评审关注点的一方；裁决（选型、质疑成立与否）是责任行为，AI 不能为决策负责。这两个环节也是答辩时评委必考的『你掌握了什么』——骨架自己搭的人，答辩时对结构的每一个为什么都能答上来。

### 2.9.6 AI 产出的可信度分级

- 高可信（抽查即可）：格式化产出——表格填充、脚本生成、格式转换；
- 中可信（逐项审）：结构化叙述——章节草稿、用例草稿、任务草稿；
- 低可信（必验证）：事实性内容——引用编号、标准条款、数值、外部项目事实。低可信内容一律回到原始出处核验，无一例外。

### 2.9.7 小组协作：角色与接口

设计期的四人分工（样例 plan.md）：架构/后端 A、后端 B、AI/算法、前端/测试。协作的三个接口：

- ① 模块边界接口——谁的模块给谁提供服务，以 OpenAPI/表结构为契约，改动当日通知；
- ② 文档接口——各人写各章，但引用别人的章节用粗粒度引用（见 5.10）；
- ③ 审查接口——互评配对固定（A↔B、丙↔丁），评审发现的缺陷记到作者名下但算全组的账（不追责个人，只改流程）。



## 2.9.8 冲突仲裁与升级

设计分歧三级处理：模块内分歧，模块主责人裁决并记录；跨模块分歧，架构组（A+丙）对撞论证后裁决，写 ADR 备注；涉及范围/基线的分歧，升级组长并走 CCB。所有裁决 24 小时内完成——分歧搁置是设计期最隐蔽的进度杀手，它表现为『文档各自写完了，接口对不上』。

## 2.9.9 分工矩阵总表（第 5 关）

| 活动     | AI 职责   | 人职责            | 交接点/质检            | 实测效果            |
|--------|---------|----------------|-------------------|-----------------|
| 测试计划   | 骨架与模板   | 范围取舍/风险排序/资源分配 | 全组计划会逐段审定         | 初稿 10min, 会议 1h |
| 用例生成   | 三种模式起草  | 入库质检五条铁规       | AI-draft 打标, 评审摘标 | P0 34 条中 28 条起草 |
| 用例评审   | 第一评审质询  | 裁决问题清单         | 质询清单导入评审会         | 平均 6~9 问/集      |
| RTM 挂接 | 文本匹配建初稿 | 审计一对零/一对多      | 异常挂接复核            | 错误率 5~8% 需审     |
| 自动化脚本  | 起草脚本    | 评审三处（断言/清理/耦合） | 先评审再入库            | 25min→8min/条    |
| 测试数据   | 批量+异常数据 | 抽样验证打中目标层      | 每类抽 1 样本验证        | 毒消息 20 变体       |
| 精度测试   | 禁止参与执行  | 全流程人工（冻结纪律）    | 使用登记              | 红线，无例外          |
| 压测     | 脚本与数据整理 | 口径定义/拐点解读/容量建议 | 数字出处逐行核对          | S1~S4 四场景       |
| 失败排查   | 三个候选根因  | 定根因，警惕环境归因     | 业务缺陷人复核           | 首候选命中 60%       |
| 缺陷单    | 从记录起草初稿 | 复现验证+定级        | 实际执行人字段           | 虚标事故 1 次        |
| 缺陷分析   | 图表生成    | 根因分类与解读        | 口径书面判据            | 环节矩阵产           |

| 活动   | AI 职责  | 人职责        | 交接点/质检 | 实测效果      |
|------|--------|------------|--------|-----------|
|      |        |            |        | 出         |
| 报告编写 | 数据组织成表 | 结论/分析/披露口径 | 结论段人写  | 《测试报告》    |
| 自评表  | 一致性核对  | 填报+部分符合措辞  | 表文交叉核对 | 《质量特性自评表》 |

### 2.9.10 矩阵背后的三条原理

读这张表要看出门道，三条原理：

- 产出与判断分离：AI 负责产生候选（草稿、清单、图表、候选根因），人负责判断（取舍、定级、结论、披露）。AI 的产出速度优势用在候选生成上，人的责任落点压在判断上；
- 交接点显性化：每个交接点都有可检查的 artifact（打标、质检单、登记簿、diff 报告）——『人审过了』必须留下证据，否则交接点就是漏洞；
- 责任不可委托：矩阵里『人职责』列的每一项都不许以『AI 已经做过了』为由省略——精度冻结纪律、根因拍板、报告结论、部分符合的披露，这四样是测试关的责任底线。

### 2.9.11 矩阵的裁剪与迁移

迁移到新题目时矩阵结构不变，变三处：精度红线行替换为本领域的核心指标测试（如医疗软件的诊断一致性、财务软件的金额计算——凡是『错得对不起来』的指标都是红线行）；提示词模式按领域改写（『攻击者视角』对任何系统通用，『边界猎手』的字段约束按新 SRS 换）；实测效果列的数据换成自己组的过程数据——这组数据将成为你答辩时人机协同有效性的直接证据。

### 2.9.12 分工矩阵总表（第 6 关）

| 活动        | AI 主导  | 人机共担        | 人工专属 |
|-----------|--------|-------------|------|
| 文档结构完整性初查 | ● 清单扫描 |             |      |
| 文档数据与口径核对 |        | ● AI 报疑点人裁决 |      |

| 活动                | AI 主导     | 人机共担         | 人工专属   |
|-------------------|-----------|--------------|--------|
| 披露/结论措辞定稿         |           |              | ● 亲笔   |
| 哈希清单与 manifest 生成 | ● 脚本化     |              | ● 封存签署 |
| 干净环境自测执行          |           |              | ● 真实执行 |
| 缺口根因分析与修复         |           | ● AI 辅助定位人修复 |        |
| Prompt 日志书写       |           |              | ● 当日亲笔 |
| 日志统计聚合            | ● 脚本      | ● 抽样验证       |        |
| 演示动线设计            |           | ● AI 评审人定稿   |        |
| 演示现场执行            |           |              | ● 手与口  |
| 价值论证基线实测          |           |              | ● 真实测量 |
| 价值论证呈现            |           | ● AI 初稿人注入   |        |
| PPT 制作            | ● pptd 管线 | ● 逐数核对       | ● 结论定稿 |
| 复盘数据聚合            | ● 脚本      | ● 口径人工裁决     |        |
| 复盘归因与 E/I         |           |              | ● 人工落笔 |
| 代码走查答辩            |           | ● AI 模拟追问    | ● 本人应答 |
| 移交签署与冻结           |           |              | ● 双方签字 |

### 2.9.13 矩阵的使用规则

- AI 主导档：人仍需验收产出（脚本要跑通、扫描结果要复核），『主导』不等于『放手』；
- 人机共担档：必须留双人痕迹——AI 的疑点清单+人工的裁决记录，缺一视为未做；
- 人工专属档：AI 连草稿都不出（披露/结论）或只出反方（答辩预演）——这一档的违规在收口关全部按红线处理。

### 2.9.14 模式一：AI 预审、人工终审（文档类）

适用：《文档审计报告》自审、UAT 表、验收报告、复盘报告。流程：AI 按清单扫描出『疑点清单』（不给结论）→ 人逐条裁决（真问题/误报/风格偏好）→ 整改 → AI 复扫确认清零。关键纪律：AI 的输出永远叫『疑点』不叫『结论』——澜图组三遍预审中 AI 报出 47 条疑点，人工裁决后真问题 19 条。误报率六成是常态，所以裁决权必须在人。

### 2.9.15 模式二：AI 聚合、人工归因（数据类）

适用：工时偏差、AI 五指标、ROI 账本、覆盖率汇总。流程：人先定口径文档 → AI 写聚合脚本 → 人抽样手工验证（≥10 条）→ AI 出表出图 → 人写归因与结论。关键纪律：聚合可外包，归因不外包。《项目开发总结报告》每个数字背后的『为什么』都是人写的——AI 写的归因（『效率提升得益于团队协作』）一眼假。

### 2.9.16 模式三：AI 对抗、人工应战（预演类）

适用：答辩准备、价值论证压力测试、走查演练。流程：给 AI 足够材料（报告/PPT/代码），让它扮演特定角色（怀疑的评委/苛刻的审计员/新员工学弟）发起攻击 → 人应战并记录被打穿的点 → 修补 → 换角色再打。澜图组用此模式预演了验收答辩，AI 评委提出的 23 个问题中有 9 个在真实答辩被问到变体。关键纪律：AI 攻击清单要留档——它是你们准备充分度的证据。

### 2.9.17 模式四：AI 起草、人工注入灵魂（文稿类）

适用：PPT 讲稿、E/I 系列初稿、移交说明。AI 出结构化初稿，人做三类注入：真实数字（替换 AI 的占位与『合理推测』）、真实故事（PR#36、坐标偏差 300 米这类只有你们有的细节）、真实判断（保留率 44% 是好是坏，人的解读）。注入三类的文稿与纯 AI 稿的辨识度极高——评委一年看几十份，纯 AI 稿 30 秒识破。

### 2.9.18 本节清单

- 能默画分工矩阵的三档分布与代表活动；
- 掌握可验证性、责任、成本三条判定原则；
- 会把矩阵用于派活、onboarding 与答辩举证。

## 2.10 需求追踪矩阵 RTM：一条链，五个版本

### 2.10.1 本节学习目标

- 理解 RTM 的双重价值：对外（验收证据）与对内（变更影响分析）；
- 掌握 RTM 的列设计与占位符策略；
- 掌握孤儿需求、无源测试等完整性指标的检查方法；
- 学会 AI 辅助生成 RTM 初稿 + 脚本自检的组合流程。

## 2.10.2 RTM 为什么不是形式主义

招标文件 5.10.2(1) 把话说得很白：功能验收“每条需求均有对应测试用例（建立需求追踪关系）”。没有 RTM，验收时就要当场翻箱倒柜证明覆盖——那是灾难现场。对内的价值在变更时体现：甲方提出改一个指标，RTM 让你 30 秒内回答“影响哪些需求、哪些设计、哪些测试用例”；没有 RTM，这个问题靠拍脑袋。澜图案例 V1.0→V2.0 的编号大修之所以敢做，正是因为 RTM 把每条映射摆在明面上，改完一跑自检就知道有没有漏。

## 2.10.3 列设计（澜图样例方案）

功能矩阵七列：招标条款（5.5.x） | 需求条目（FR-xx.y） | 需求摘要 | 验收标准（AC 编号） | 优先级（M/S/C） | 设计元素（占位） | 测试用例（占位）。非功能矩阵六列：招标条款 | 需求条目 | 指标 | 验收方式 | 判定主体 | 测试用例。设计要点：

- “招标条款”列必须原文编号——它是整个矩阵的锚；
- “验收标准”列只写 AC 编号，不写全文（避免与 spec.md 双份漂移）；
- “判定主体”列是非功能表的灵魂：第三方/招标人专家组/监理见证/开发自测，★条款不允许落在“开发自测”；
- 占位列（设计元素、测试用例）在需求关留空但列头保留——它宣告“这两列在第 3、4 关必须回填”，是关卡间交接的契约。

## 2.10.4 完整性指标与自检

- 覆盖率：招标 5.5 节 FR 条目在 RTM 中的出现率 = 100%（脚本核对）；
- ★覆盖：13 条★条款逐条在册且验收主体正确；
- 孤儿需求 = 0：RTM 中每条需求能回溯到招标条款或经裁决的访谈诉求；
- 无源测试 = 0：（第 4 关后检查）每个测试用例能回溯到需求；
- MVP 覆盖：确认书 5 项 MVP 对应的条目全部 M 级且用例号已预留。

自检脚本与第九章的 check\_spec.py 共用框架：读 RTM 表格 → 与招标条目清单做集合运算 → 输出差集报告。教学要求：RTM 每次升版随附当次自检输出（文本存档），作为过关评审的入场券。

### 2.10.5 AI 辅助生成流程

RTM 初稿是 AI 的舒适区：把 spec.md 全文 + 招标★条款清单喂给 AI，要求按固定列格式输出表格初稿。人工审核三处：① 招标条款列是否与原文编号逐字一致（AI 爱“优化”编号格式，如把 5.7.8(1) 写成 5.7.8.1）；② AC 编号是否真实存在于 spec.md（AI 会编造 AC-FR-99 这类幽灵编号——脚本可做存在性校验）；③ 判定主体列是否被 AI 擅自降级（把★条款的“第三方实测”写成“测试验证”）。

### 2.10.6 RTM 的生命周期管理

| 关卡       | RTM 动作                | 验收点             |
|----------|-----------------------|-----------------|
| 任务 2（本关） | 建立功能/非功能两张矩阵；设计与测试列占位 | 覆盖率 100%、孤儿 0   |
| 任务 3（设计） | 回填“设计元素”列（模块/接口/表）    | 每条 [M] 需求有设计落点  |
| 任务 4（测试） | 回填“测试用例”列并双向核对        | 无源测试 0；★条款用例全执行 |
| 任务 5（交付） | 随验收文档归档；作为变更基线        | 验收追踪关系现场演示      |

变更时的操作：变更单 → 定位 RTM 行 → 评估影响列 → 修改 spec.md → 同步 SRS/RTM → 自检报告归档。这套动作在第 5 关的“变更管理”评审中会被要求现场演示一次。

### 2.10.7 从 v2 到 v3：挂接设计元素

任务 2 的 RTM v2 只有两列核心：需求→验收标准。任务 3 的 RTM v3（docx+xlsx 双格式）在此基础上回填五列：设计模块、HLD 章节、DLD 章节、数据表、API 端点、ADR——形成『招标条款→需求→验收标准→设计→测试』的完整链条。回填的规则：逐条 FR 对照设计文档找落点，找不到落点的需求是漏设计，找不到需求的设计是镀金，两者都是缺陷，都必须当日处理。

### 2.10.8 正向核查：需求有没有家

正向核查问『每条需求住在哪』。操作：遍历 24 条 FR+16 条 NFR+3 条精度指标，逐条填写设计列。澜图 v3 的结果：24/24 FR 挂接、16/16 NFR 挂接、13 条★款全覆盖、MVP 5 项全覆盖。核查表的每一行都是评审证据——评委抽查任一行，能顺着编号在 1 分钟内翻到设计文档的具体章节，这就是『可追踪』的实操含义。

### 2.10.9 反向核查：家是不是合法

反向核查问『每个设计元素有没有出生证』。两个反向清单：① 模块清单 M-01~M-10 逐一指认需求来源（一一对应 F-01~F-10，无镀金模块）；② 数据表清单 12 张逐一指认（每张表至少服务一条 FR）。反向核查抓的是 AI 时代的高发病——AI 起草设计时会『顺手』加缓存层、加消息总线、加它认为合理的表，这些无源设计如果不拦截，都会变成工时黑洞和测试负担。

### 2.10.10 断链检查：RTM 的 CI 化

RTM 不是一次性文档，是持续维护的追踪资产。维护规则（写入 RTM v3 末）：任何需求/设计/测试变更须同步更新本矩阵；CI 每周执行断链抽查（抽样 15 行人工核对）；RTM 行更新是任务 DoD 的组成部分。断链率纳入度量（见 2.13 节）：断链率>5% 说明配置管理失控，应触发流程整改而不是闷头补表。

### 2.10.11 工具化：脚本生成与校验

澜图案例的 RTM v3 由脚本生成（t3/build\_rtm3.py 出 xlsx，build\_rtm3\_docx.py 出 docx）：需求行从 v2 提取（JSON 中间格式），设计映射表人工维护在脚本里，双向核查结论自动统计。好处：重生成零成本、无抄写错误、xlsx 与 docx 永远一致。教训同样来自实践：v3 的映射表初版引用了不存在的 HLD 4.3.6~4.3.10 小节，是构建后核对章节清单才发现的——脚本生成保证内部一致，但与设计文档的外部一致性仍需人工核对章节编号。

### 2.10.12 评审视角：RTM 怎么用

评委用 RTM 的三个动作：① 随机抽 3 行正向走查（从 FR 翻到设计章节再翻到测试用例，计时）；② 扫反向清单找镀金；③ 看演进记录（v1→v2→v3 的变更说明）。小组自查时先自己做这三个动作，评审就是开卷考试。

## 2.10.13 本节清单

RTM 是需求工程的“总账”：建账（本关）、记账（逐关回填）、对账（自检脚本）、交账（验收归档）。它的成本集中在建账的一次性投入，回报分布在之后每一关。下一章讲本关的收口：质量评审、基线冻结与考核标准。

## 2.11 文档工程：国标结构、语言规范与程序化构建

### 2.11.1 为什么要对标 GB/T 8567

GB/T 8567《计算机软件文档编制规范》规定了软件开发各阶段文档的章节框架。对标它的意义有三：

- ① 完整性兜底——标准列出的章节是几十年工程经验的检查表，漏章大概率漏事；
- ② 评审共同语言——评委按标准找内容，你的章节名与标准一致，评审成本就低；
- ③ 交付合规——招标文件常直接引用文档标准号。

但注意：对标是『覆盖其要求项』，不是『照抄其章节名』——澜图样例对章节做了面向评审的重排（如把非功能设计单列第 9 章），同时保证标准要求的内容项一项不缺。

### 2.11.2 概要设计说明书的对照

| GB/T 8567 要求项           | 澜图 HLD 落点           | 说明                  |
|-------------------------|---------------------|---------------------|
| 引言（目的/背景/定义/参考）         | 第 1 章               | 1.2 节写明设计输入基线清单     |
| 总体设计（需求规定/运行环境/设计概念/结构） | 第 2 章+第 4 章         | 2.1 设计目标直接复述 SRS 目标 |
| 接口设计                    | 拆出为独立文档（《接口设计说明书》）  | HLD 7.3 节给内部集成契约指针  |
| 运行设计（模块组合/运行控制/运行时间）    | 第 4 章+第 9 章         | 运行时间对应 PE 指标分解      |
| 系统数据结构设计                | 拆出为独立文档（《数据库设计说明书》） | HLD 第 5 章给数据架构总览    |
| 系统出错处理设计                | 第 9 章+各模块异常设计       | 异常分类与错误码规范          |



| GB/T 8567 要求项 | 澜图 HLD 落点 | 说明        |
|---------------|-----------|-----------|
|               |           | 见 DLD 9.3 |

教学要点：做一张『标准要求项→我的章节』对照表放在文档修订记录之后，评委核完整性时一表通过。

### 2.11.3 详细设计说明书的对照

GB/T 8567 要求详细设计按『程序系统结构—模块设计说明』组织，每个模块含：模块描述、功能、性能、输入项、输出项、算法、流程逻辑、接口、存储分配、注释设计、限制条件、测试计划、尚未解决的问题。澜图 DLD 的『五件套』是它的工程化压缩：职责与需求映射（对应模块描述+功能）、类设计（对应算法与接口）、关键流程（对应流程逻辑）、数据与接口（对应存储分配+接口）、单元测试要点（对应测试计划）。『尚未解决的问题』这一项不许省——它就是 6.3 节『待裁决清单』的归宿，评审时这一项的诚实度直接反映团队的过程健康度。

### 2.11.4 数据库设计说明书的对照

标准分外部设计（标识符/使用它的程序/约定）、结构设计（概念/逻辑/物理）、运用设计（数据字典/安全保密/恢复）。澜图《数据库设计说明书》的对应：第 2 章外部设计（实例规划、命名规范、数据三分原则）；第 3~5 章概念/逻辑/物理；第 6 章运用设计（访问层规范、迁移、备份恢复、监控）。学生最容易漏的是外部设计——『这个数据库给谁用、谁来部署、怎么命名』看似琐碎，却是部署验收（MTN-01）的直接依据。

### 2.11.5 裁剪的原则

标准允许裁剪，但裁剪必须满足两条：① 写下载剪声明（哪一节、为什么裁、其内容并入哪章）；② 裁剪不影响验收依据的完整性。反例：把『安全设计』裁掉因为『我们项目不涉及安全』——等保二级条款在 SRS 里白纸黑字（SEC-01），裁剪即漏需求，RTM 反向核查会立刻抓住。

### 2.11.6 工程语言的第一原则：可验证

设计文档的每一句话都应能被证实或证伪。判定方法：把句子读给一个不知情的人听，问『你怎么检查这句话是不是真的？』有明确检查动作的句子合格；只能回答

『我觉得是这样』的句子不合格。对照示例：『系统响应迅速』（不可验证）→『交互点提示单轮响应  $P95 \leq 3s$ ，硬件基线见 5.7.1，抽样 50 次判定』（可验证）。

### 2.11.7 词汇黑名单

- 程度词：高性能、高可用、快速、灵活、强大、先进、友好、优雅——全部改写为数值+机制；
- 推卸词：尽量、尽可能、原则上、一般情况下——设计是承诺，不是愿望；
- 含糊词：等、相关、类似、其他——列举要么闭合（共 5 类：……），要么标注『枚举未完，见附录』；
- 口头词：搞一下、弄出来、跑起来——正式文档用工程动词：实现、部署、验证。

### 2.11.8 引用规范

一切外部引用必须可定位：需求引用到条号（FR-06.6）；标准引用到标准号+年号（GB/T 25000.10-2016）；文档引用到文件名+版本+章节（HLD V3.0 第 9.1 节）；会议结论引用到纪要编号与日期。编号的真实性核验是独立审查步骤——本案例的编号 CL 编号事件就是引用规范失守的直接后果。新规则：任何引用编号在入库前必须在被引文档中实际找到，找不到即删除或走澄清。

### 2.11.9 图表规范

- 每图有编号与图题（图 3-1 系统分层架构图），正文必有引用句（『如图 3-1 所示』）；
- 图中文字 $\geq$ 正文 80% 字号，打印可读；配色考虑黑白打印（灰度可区分）；
- 图与正文同源优先：能脚本生成的图不用手绘，脚本入配置库；
- 表格表头加底纹、跨页重复表头；表格内文字用名词短语不写长句。

### 2.11.10 术语管理

术语表（SRS 1.4）是全项目词汇的唯一权威：设计文档中出现的每个领域名词必须与术语表一致。新建术语的流程：先查术语表→没有再定义→定义进术语表→全文统一。AI 起草是术语混乱的重灾区（同一概念三种叫法），审查时做全文一致性搜索：把关键术语逐个全文搜，检查有无别名出现。澜图案例的统一词表：任务（tasks）、

结果集（result\_sets）、图斑（polygons）、提示（prompts）——任何『作业/批次/斑块/掩膜记录』等别名都是缺陷。

2.11.11 修订记录与版本声明

每份文档第一页后是修订记录表：版本号、日期、修改内容、修改人、审核人。版本号规则：草稿 0.x，评审通过升 1.0，基线后每次变更升小版本并过 CCB。修订记录不是形式——本案例 v3 校订的全部依据就写在修订记录里，评审时它回答了『这版改了什么、为什么改、谁批的』三个必问题。

2.11.12 为什么要“用代码写文档”

任务 1 的投标书超 100 页，用 Word 手工排版会遇到三个顽疾：多人合编格式打架、改一处全文重排、版本无法追溯。澜图案例的解法是程序化构建：内容写成结构化的模块文件（bid\_01~bid\_08），由构建脚本（create\_bid.js）统一渲染为 docx。收益是实打实的：V1→V2 的六处修改从编辑到重新出稿只用了几分钟；18 张插图的修复重绘后一键重建。本节教学生会写、会改这套脚本——它是任务 2 之后所有长文档（设计说明书、测试报告、用户手册）的通用基建。

2.11.13 核心概念：块（block）模型

全部内容抽象为“块”的数组，每块是一个 [类型, 内容, 附加参数] 三元组。澜图案例用到的块类型：

| 块类型      | 写法示例                                | 渲染效果                |
|----------|-------------------------------------|---------------------|
| h1/h2/h3 | ["h1","第三章 总体技术方案"]                 | 三级标题（h1 另起一页）       |
| p        | ["p","正文段落....."]                   | 首行缩进正文段             |
| b        | ["b","（1）条款项....."]                 | 项目符号段               |
| table    | ["table",{header,widths,rows}]      | 带表头底纹的边框表           |
| code     | ["code","python check_bid.py ..."]  | 灰底等宽代码行             |
| img      | ["img","gantt.png",6.3,"图 3-1 ..."] | 居中图片 + 图题（自动按宽高比缩放） |
| pb       | ["pb"]                              | 分页符                 |

理解块模型后，写文档就变成“填数组”：AI 生成内容时直接要求输出块数组格式，

人工审阅后并入模块文件。这就是“AI 生成 + 程序化组装”的接口约定——提示词里写明输出格式为块数组，AI 的产出即可直接构建。

### 2.11.14 动手：最小可运行示例

第一步，准备环境：在工程目录安装 docx 库（`npm install docx`，课程已预置 `node_modules`）。第二步，写内容模块 `my_doc.js`：

```
export default [
  ["h1", "第一章 引言"],
  ["p", "这是第一段正文。"],
  ["table", {"header": ["列一", "列二"], "widths": [4500, 4500],
    "rows": [["A", "B"], ["C", "D"]]}],
];
```

第三步，复制 `create_bid.js` 为 `create_my.js`，把 `import` 换成自己的模块、修改封面与页眉文字。第四步，用课程文档技能构建：`docx build create_my.js` 输出 `.docx`——构建器会自动做语法检查、`lint` 与包校验。第五步，实测页数与字数（Word 或 WPS 打开统计），不达标回到模块补内容。

### 2.11.15 表格的四个坑（澜图实测）

- 行列数必须一致：rows 每行单元格数 = header 数 = widths 数，否则构建报“`undefined slice`”错误且报错信息不指位置——本书稿构建时就踩过一次（13.4 表多写一个空单元格），对策：写一个小扫描脚本预检所有表；
- widths 单位是 DXA（1/20 磅），全表合计约 9000 为宜；列宽决定换行，长文本别给太窄；
- 表头重复：表头行已设置跨页重复（`tableHeader`），长表跨页自动带表头，无需手工处理；
- 单元格内不支持富格式：需要加粗的单元格文字，在内容上用“【】”等符号标识，或改在正文段落表达。

### 2.11.16 图片嵌入与中文乱码防范

- 图片放 `assets/` 目录，构建脚本按文件名读取、按 PNG 头自动算宽高比——插入图只需给出目标宽度（英寸）；
- 中文图片正常显示的关键是绘图时配置 CJK 字体（课程 Python 环境已内置 `NotoSansSC`，调用 `setup_plot` 即可）；

- 文件名只用英文与数字（gantt\_task1.png），中文文件名在某些工具链下会乱码；
- 构建后用 ReadMediaFile 或 Word 打开抽查图片清晰度与图题编号连续性。

### 2.11.17 目录、页眉页脚与封面

构建脚本自动生成三件套：封面（居中大标题块）、目录（TOC 域 + 预估页码缓存，Word 打开后按 F9 刷新为准确页码）、页眉页脚（页眉为文档标题、页脚居中页码）。h1 标题设置 pageBreakBefore，每章自动另起一页——章数多的文档（如本书 19 章）仅此一项就贡献约 19 页，做页数规划时要计入。

### 2.11.18 与 AI 的协作要点

- 让 AI 直接产块数组：提示词写明“输出为 [类型,内容] 数组，可用类型：h1/h2/h3/p/b/table/code，表格含 header/widths/rows”；
- 一章一模块：每个模块独立文件，AI 生成与人审阅的粒度都以模块为单位；
- 构建即校验：构建失败（语法/行列数）立即暴露，比 Word 时代“交稿才发现格式崩了”好得多；
- 版本管理：模块文件与构建脚本入 Git，每次重建产物可复现——过程分所需的“版本历史”天然存在。

### 2.11.19 本节清单

- ☐ 内容全部以块数组组织，无手工排版步骤；
- ☐ 所有表格通过行列数预检；☐ 图片统一 assets/ 目录、英文文件名；
- ☐ 目录域可刷新、页码连续；☐ 构建产物页数实测达标；
- ☐ 模块文件与脚本入版本库，可一键重建。

## 2.12 评审会与团队协作：组织方法、主副修与人人过关

### 2.12.1 评审会的目标与边界

设计评审会的目标只有一个：决定是否冻结设计基线。它不负责替你做设计（那是会前的事），不解决技术争论（争论转线下专题，会上只记录分歧与裁决人）。把评审会开成讨论会是新手组最常见的时间黑洞——90 分钟会议 70 分钟在争论一个字段命名，基线冻结不了了之。

### 2.12.2 会前：准入检查

- 文档齐套：HLD/DLD/DB/API/ADR/plan/tasks/测试计划/管理计划族/RTM v3，缺一项不开会；
- 体检通过：2.11 节清单的机器可检项（禁用词、状态码、章节编号、docx 可打开）全绿，报告归档；
- 预读落实：评委提前 2 天拿到文档，小组提前收齐预读意见并分类（采纳/解释/待议）；
- 走查自证：小组自己先做过 3 条 FR 的端到端走查（见 2.10 节），报告随材料提交。

### 2.12.3 会中：议程与角色

| 环节 | 时长    | 内容                            | 责任人  |
|----|-------|-------------------------------|------|
| 陈述 | 10 分钟 | 只讲三个架构决策与其需求依据，不念目录           | 组长   |
| 走查 | 15 分钟 | 评委用 RTM 抽查 3 行（正向+反向），小组现场翻证据 | 评委主导 |
| 质疑 | 10 分钟 | 围绕预读意见中的『待议』项                 | 全体   |
| 裁决 | 5 分钟  | 通过/有条件通过（整改项编号+责任人+复查时点）/不通过  | 评委会  |

角色分工：主持人控时间（到点就切）、记录员逐条记质疑与结论（评审记录是配置项）、答辩人答『是什么/依据哪条』不辩解过程辛苦。答辩的第一原则：每个问题先给结论再给依据，30 秒答不完的转化为『会后补充材料，明日 12:00 前提交』。

### 2.12.4 会后：闭环管理

有条件通过的整改项进看板（与普通任务同等管理），逐项关闭后由评委复查签字，然后才正式冻结基线。评审记录（出席、意见、裁决、整改项清单）48 小时内归档配置库并通知全员。基线冻结是一个正式宣布的动作：配置管理员发基线公告（基线内容清单+版本号+变更流程入口），从这一刻起任何修改走 CCB。

### 2.12.5 评审会反模式

- 材料会前一小时才发——评委零预读，走查环节变成现场翻书；
- 陈述念 PPT 目录——10 分钟讲不完三个决策说明准备不足；
- 质疑环节辩解『我们时间紧』——工期不是质量的理由，砍范围才是；
- 结论含糊（『基本通过，回头改改』）——整改项不编号等于没有整改。

### 2.12.6 AI 在评审会中的位置

会前可用 AI 做模拟评审（反向质疑法，见 4.5），把 AI 的高质量质疑提前消化；会中不用 AI 实时辅助答辩（评审考的是团队的掌握度，开卷用 AI 等于自证没掌握）；会后可用 AI 整理评审记录草稿，但裁决结论部分必须人工撰写——责任文本不外包。

### 2.12.7 团队协作：角色、主副修与人人过关制

任务 4 的主副修机制规定：每人主攻一个方向（后端/前端/测试/DevOps），同时副修另一个方向，且必须完成至少一次跨方向联调。本节讲这个机制为什么存在、怎么落地、联调单怎么写。

#### 机制解决的问题

3~5 人小组 + AI 辅助，最容易出现的畸形状态是：每个人抱着 AI 各写各的模块，代码能跑但谁也没真正理解别人的部分，集成时连环爆炸，答辩时间到别人的模块就哑火。主副修机制用三条约束打破这个状态：

- 主攻负责制：每个方向有一名主攻，对该方向的代码质量、评审、知识负首责。AI 在该方向的产出由主攻审定，出事故主攻第一责任人；
- 副修参与制：每人必须副修另一方向，副修的最低参与标准是——评审过该方向至少 2 个 PR、独立修过该方向至少 1 个 bug、能在答辩时讲清该方向的架构；
- 联调强制制：跨方向的接口（前后端、后端-队列、服务-存储）必须由两端的人当面联调，产出联调单。禁止『我这边测过了，你那边的问题』式甩锅。

#### 分工矩阵怎么排

以 4 人组为例的推荐排法（5 人组可加一名测试主攻；3 人组则一人兼前端与 DevOps 主攻）：

| 成员 | 主攻     | 副修     | 联调义务                       |
|----|--------|--------|----------------------------|
| A  | 后端     | 测试     | 与 B 联调前后端接口；与 D 联调部署链路     |
| B  | 前端     | 后端     | 与 A 联调前后端接口（OpenAPI 契约核对）  |
| C  | 测试     | DevOps | 与所有人联调：测试环境、CI 反馈闭环        |
| D  | DevOps | 后端     | 与 A 联调部署链路；与 C 联调 CI 与测试环境 |

排矩阵的两个原则：副修方向尽量与主攻有接口关系（联调自然发生）；把最弱的成员放在测试主攻位——测试方向能最快建立全局视野，且测试主攻天然要读所有人的代码。

### 联调单：一次联调的完整证据

联调单是主副修机制的核心交付物，一张表记录一次跨方向联调。字段：日期、参与人（两端）、联调对象（哪个接口/链路）、依据文档（OpenAPI 版本号、SRS 条目）、联调过程（测了哪些用例、发现什么问题）、结论（通过/遗留问题及归属）。样例项目两个迭代共产生 14 张联调单，摘录一张典型内容：

| 字段   | 内容                                                                                                    |
|------|-------------------------------------------------------------------------------------------------------|
| 联调对象 | POST /api/tasks 文本提示分割提交链路（前端→后端→队列；文本提示为异步任务，FR-04.1/FR-05.x）                                        |
| 依据   | openapi.yaml v2.1；SRS FR-04.1；spec.md AC-FR-04.1-a                                                    |
| 过程   | 正常提交 5 例通过；发现 2 个问题：①置信度阈值前端传 0~100 整数、后端按 0~1 浮点解析（CR-001 变更后前端未同步默认值 0.30）；②队满时后端返回 500 而非文档承诺的 429 |
| 结论   | 问题①当天修复（PR#53）；问题②登记缺陷 API-007，次日修复并补契约测试                                                             |

教学要点：联调单的价值在于它把『我们联调过了』变成可检查的证据。评审时抽查联调单，对照问题清单追问修复闭环，能有效防止小组编造联调记录——编的记录往往没有问题发现，而真实联调几乎一定会发现问题。



## 人机协同要点（第4关）

- AI 生成联调检查清单：把接口契约喂给 AI，让它列出联调必测场景（正常、边界、异常、并发），人工裁剪后作为联调单的过程模板。样例项目用这个方法把联调用例从平均 5 个/次提到 9 个/次；
- AI 辅助问题定位：联调发现问题时，把两端的日志一起喂给 AI 做关联分析，它擅长发现『前端传的单位与后端解析的单位不一致』这类跨边界问题；
- 副修学习用 AI 加速：进入陌生代码方向时，先让 AI 生成该模块的导览（调用链、关键类、数据流），人工对照代码核实后再动手——这是副修机制下性价比最高的 AI 用法，但要注意 AI 导览的错误率约 10%，关键结论必须看代码确认。

## 走查的考查点拆解

| 考查点   | 评委在验什么        | 不合格回答的样子        |
|-------|---------------|-----------------|
| 业务上下文 | 懂不懂这段代码服务什么需求 | 『这是队友的需求，我不太清楚』 |
| 技术根因  | 懂不懂问题/方案的机理   | 『反正这样改就不报错了』    |
| 方案权衡  | 懂不懂为什么不那样写    | 『没考虑过别的方案』      |
| AI 角色 | 诚不诚实、有没有把关    | 『都是 AI 写的，我没细看』 |
| 关联影响  | 懂不懂改动的影响面     | 『测试好像有人跑过了吧』    |

## 名下代码索引：走查准备的第一步

每名成员给自己建一份《名下代码索引》：列出自己作为作者的每个 PR，每个 PR 备五句话——需求来源（哪个 FR/哪个 BUG）、问题或目标的机理、方案与备选（为什么选这个）、AI 参与情况（日志行号）、影响面（动了哪些模块、测试怎么验的）。澜图组 4 人 × 平均 9~10 个 PR，每份索引 2~3 页。写索引的过程就是复习：写不出五句话的 PR 就是你的风险点，优先回炉。

## 案例精讲：丁讲 PR#79

还原《验收报告》记录的问答要点。评委：『这个 4096 截断，为什么是截断而不是报错？』丁：『文本提示经 tokenizer 后超长会触发模型内部异常，原实现直接把异

常抛给用户（BUG-136）。方案有两个：前端限制输入长度，或服务端截断并告警。选截断+告警是因为应急场景用户粘贴长地址串是常态，硬性限制会打断作业流；截断在 tokenizer 层做、告警写入响应头，前端弹提示。AI 给了初版方案是前端限长，日志 2026-08-27 下午那条，评审时乙指出限长打断作业流，改为现在的方案。影响面：仅文本提示接口，测试 BUG-136 回归用例 3 条全过。』——这段话业务、根因、权衡、AI 角色、影响面五点全中，全程 90 秒。

### **防 AI 替考：平时就做对的三件事**

- 评审即学习：队友 AI 生成的代码，评审时必须逐行看懂再 Approve——看不懂就问，问就是学习；评审记录里留提问痕迹（这些痕迹答辩时是你的学习证据）；
- 合并即认领：PR 合并后该代码『归你』， regardless of AI 写了多少——认领的标准就是能讲五句话；
- 组内互查演练：每周开发例会上抽一人讲一个 PR（10 分钟），全周期下来每人讲过 3~4 次，验收走查只是第 5 次。

### **人机协同要点（第6关）**

- 名下代码索引可用 AI 从 git log+PR 描述聚合生成初稿，五句话由本人填写；
- 组内互查时让 AI 扮演评委追问（『对这段代码连续追问 5 个为什么』），被问倒的点就是回炉点；
- 红线：走查必须本人讲——用 AI 耳机提示、带小抄照读，一经发现按替考处理。

### **本节清单**

- 理解主副修三条约束的设计意图；
- 会排分工矩阵，遵守副修参与与接口邻接原则；
- 会写联调单，知道真实联调几乎必然发现问题这一评审判据；
- 掌握 AI 生成联调清单与陌生模块导览的用法及核实纪律。
- 每人有名下代码索引，每 PR 五句话；
- 风险 PR（写不出五句话的）已回炉；

- 组内互查每人  $\geq 3$  次，有记录；
- AI 参与情况诚实可引日志，无『我没细看』。

## 2.13 过程度量：用数据说话

### 2.13.1 为什么过程证据与结果文档同等重要

本课程的考核对象不只是『文档写得对不对』，更是『过程做得真不真』。一份完美的 SRS 可能是抄的，也可能是 AI 一键生成无人审定的；评审老师无法仅凭终稿区分这两种情况，必须看过程证据。任务 2 的过程证据由四条线索构成：

- 线索一：版本演进。SRS V0.9（访谈后草稿）→ V1.0（澄清前）→ V2.0（澄清+评审后）→ V3.0（设计评审重锚）→ V3.1（CR-001 阈值变更），每次演进附带《版本修订说明》，列出变更条款、变更原因、依据（CL 编号或评审记录编号）。
- 线索二：澄清记录。 $\geq 10$  条 AI 反向提问（check\_clarify 脚本输出或对话导出）、人工裁决表（每条提问的裁决结论与修改落点）。这是人机协同最直接的证据。
- 线索三：体检报告。check\_spec.py 的历次输出（至少保留 V1.0 的首次报告与 V2.0 的最终报告），用 24 项检查的通过/未通过对比证明质量收敛。
- 线索四：RTM 演进。RTM v1（需求→验收）→ v2（挂接设计编号）→ v3（挂接测试用例），证明需求追踪贯穿全程而非验收前补写。

四条线索全部归档进配置库的『过程证据』目录，命名规则：日期\_文档名\_版本号。归档本身就是软件工程纪律的考核点。

### 2.13.2 四个过程度量的定义与采集方法

度量不是为了造数字给评审看，而是让小组自己对 AI 帮了多少、坑在哪里有量化认知。任务 2 要求记录四个指标：

| 指标   | 定义                     | 采集方法                     | 健康区间    | 异常解读              |
|------|------------------------|--------------------------|---------|-------------------|
| AI 初 | AI 生成的内容块中，人工审定后无修改或仅格 | SRS 修订对照：统计 V0.9 中 AI 生成 | 40%~75% | 过 低<br>（ <30% ）：提 |

| 指标      | 定义                                    | 采集方法                       | 健康区间 | 异常解读                                     |
|---------|---------------------------------------|----------------------------|------|------------------------------------------|
| 稿采纳率    | 式修改的比例                                | 段落数与最终保留段落数                |      | 示词太泛或业务理解不足；过高（>90%）：人工审定流于形式，警惕         |
| 误读驳回率   | AI 澄清提问中，经人工裁决为『AI 误读原文』的比例           | 澄清记录表中裁决类型列统计              | ≤40% | 过高说明 SRS 表述含混（AI 都读不懂，人更读不懂）或提示词未提供足够上下文 |
| 歧义修复闭环率 | 确认的歧义中，在 V2.0 中有明确修改落点的比例             | 澄清记录表逐条核对 SRS 条款编号         | 100% | 未闭环项必须给出书面不修理由，否则是评审硬伤                   |
| 星标覆盖率   | 招标文件 13 条★条款在 SRS/spec.md/RTM 三处的可追踪率 | check_spec.py 星标检查项 + 人工抽查 | 100% | 任何一条遗漏均为重大扣分项，验收阶段会被直击                   |

采集工具：四个指标全部可以用一张 Excel 表维护，无需专门系统。但维护时机是关键——澄清记录当天填，采纳率在每个版本冻结时统计，体检报告在每次构建后立即归档。事后补录的度量数据，评审老师从文件修改时间就能看出来。

### 2.13.3 证据链的组织：配置库目录结构

配置库

```

├─ 01_招标阶段/          # 任务 1 产物（招标文件、投标书、需求确认书）
├─ 02_需求阶段/
│   ├─ 访谈/              # 访谈提纲、记录、确认签字扫描件
│   └─ SRS/
│       └─ SRS_V0.9_访谈后草稿.docx

```



### 2.13.5 数据采集纪律

- 机器采集优先：PR 数、CI 拦截、覆盖率从工具日志取数，禁止手工填写——手工填的度量必然失真；
- 采纳率的定义要统一：AI 生成内容中『少量修改后采纳』与『原样采纳』都算采纳，『推翻重写』不算。定义写进团队公约，全组一致，否则数据不可比；
- 度量不用于个人考核：一旦用于个人排名，数据立刻被注水。度量只用于团队复盘与流程改进——这条纪律要在第一天宣布。

### 2.13.6 迭代复盘会的数据用法

复盘会固定三张图：燃尽图（范围与估算质量）、CI 拦截趋势（防线健康度）、采纳率按模式分布（提示词库该改哪里）。样例项目迭代一复盘的真实决策：契约拦截 3 次都集中在标注缺口→把标注检查从 CI 提前到 pre-commit；P-07 模式采纳率仅 41%→降级到休眠区重写。

### 2.13.7 答辩举证：数据怎么讲

答辩时用『数据+故事』组合：『我们的 PR 一次通过率 68%，听起来不高——但请看这 9 次 CI 拦截里有一次拦住了阈值单位错误，如果没拦住它会在验收演示时炸掉。防线拦截不是我们做得差，是系统在正常排毒。』能把拦截数据讲成正面证据的学生，是真的理解了质量工程。

### 2.13.8 人机协同要点

- AI 做数据采集脚本：从 git log、CI 日志、测试报告抽数据的脚本让 AI 写，口径定义人给；
- AI 做复盘分析初稿：把指标喂给 AI 让它找异常与趋势，人来裁决哪些异常值得行动——AI 倾向把每个波动都解释成问题，人工要过滤噪音；
- 度量报告模板化：周报与复盘的度量段用固定模板，AI 填数、人写解读。

### 2.13.9 度量指标总表

全关度量指标分五族，每族的核心指标与本项目实测值：

| 指标族 | 核心指标                                                                                   | 本项目实测 | 回答的问题           |
|-----|----------------------------------------------------------------------------------------|-------|-----------------|
| 规模族 | 用例总数 50<br>(UT14/IT11/ST14/CT4/PT5/SEC2) ;<br>★条款 13 条; RTM 挂接 24FR+16NFR<br>全闭环       | —     | 测了多少、覆盖全不全      |
| 通过族 | IT 首轮 85/86→回归 86/86; ST 首轮<br>61/64→回归 64/64; UT 312/312                              | —     | 过没过、修复能力如何      |
| 缺陷族 | 24 条(严重 5/一般 10/轻微 9); 密度<br>2.5/KLOC; 环节分布 8/5/2/3/6; 闭环<br>率 100%                    | —     | 发现了多少、什么成色、修完没有 |
| 性能族 | PE-01 P95 2.6s; PE-02 7min12s; PE-<br>03 3min48s ; AT-01 84.2%/88.6% ;<br>CPU 降级 21.4s | —     | 够不够快、够不够准       |
| 过程族 | 冻结集使用登记次数; 演练用时<br>6'12"/8'47"/11'55"; AI 脚本起草率<br>70%; 镜像断言打回 7 处                     | —     | 纪律守没守、AI 用得深不深  |

### 2.13.10 答辩举证的话术结构

数据举证的黄金句式是『问题—数字—出处—含义』四段式。例：评审问『你们的性能到底行不行』——『交互分割在目标并发 2 下 P95 为 2.8 秒（数字），数据在测试报告第 16 章 S1 表（出处），对照 SRS PE-01 条款  $P95 \leq 3$  秒达标，但并发 8 时恶化到 7.9 秒，我们给出的容量建议是单节点交互并发 $\leq 2$ 、每 2 路并发配一个 Worker（含义）』。四段齐全的回答是满分回答；只有数字没有出处会被追问，有出处没有含义显得不懂。

### 2.13.11 数字被追问时的下钻准备

每个上台讲的数字都要准备三层下钻：第一层口径（P95 怎么算的、样本多少）；第二层原始记录（哪次压测、日志在哪）；第三层反例与边界（并发 8 不达标怎么办——容量建议就是答案）。被问倒不可怕，可怕的是没有下钻硬撑——『这个数据我需要核实一下原始记录，会后补充』是专业回答，现场编数字是事故。

### 2.13.12 用数据讲失败

答辩不是只讲成功——18 条缺陷、3 条首轮未过的 ST 用例、被红队攻破的越权，都是必讲素材。讲失败的正确姿势：事实（BUG-133 越权，改 ID 可见他人任务）→ 根因（认证授权分离失误）→ 修复（PR#76 + 6 端点权限矩阵回归）→ 收获（BOLA 进了回归库和设计评审三问议程）。评审眼里，能讲清失败的组比声称没失败的组可信得多。

### 2.13.13 数据背景

本节给出一份虚构但按真实过程还原的十日度量表（覆盖第 3~5 关连续窗口的前 10 个工作日，即 D7~D16：2 天设计 + 7 天研发 + 1 天质量启动；表中 D1~D10 为该窗口的相对日序），并做逐项解读。学生组在自己的项目里按同格式记录。

| 日  | 完成任务    | AI 采纳率 | 评 审<br>缺陷 | RTM 断链 | 待 裁<br>决 开<br>放 | 事件                                    |
|----|---------|--------|-----------|--------|-----------------|---------------------------------------|
| D1 | T-01~04 | —      | 2         | —      | 3               | 环境与 CI 就绪                             |
| D2 | T-05~07 | 55%    | 5         | —      | 5               | HLD 骨 架<br>评审                         |
| D3 | T-08~09 | 62%    | 3         | —      | 4               | MS-1/2 达<br>成                         |
| D4 | T-10~12 | 71%    | 4         | 8%     | 2               | 发现数值漂<br>移 3 处                        |
| D5 | T-13~15 | 58%    | 6         | 5%     | 2               | MS-3 ；<br>GPU 告警 1<br>次               |
| D6 | T-16~18 | 66%    | 3         | 5%     | 1               | PE-01 首次<br>实测 3.4s 超<br>标→裁剪参<br>数调整 |
| D7 | T-19~22 | 70%    | 2         | 3%     | 1               | MS-4 ； 实<br>测 2.8s 达标                 |



| 日   | 完成任务    | AI 采纳率 | 评 审<br>缺陷 | RTM 断链 | 待 裁<br>决 开<br>放 | 事件                  |
|-----|---------|--------|-----------|--------|-----------------|---------------------|
| D8  | T-23~26 | 64%    | 2         | 2%     | 0               | 降级演练一次通过            |
| D9  | T-27~31 | 59%    | 1         | 0%     | 0               | MS-5； 契约测试全绿        |
| D10 | T-32~36 | —      | 1         | 0%     | 0               | MS-6 冻结；评审通过（2 项整改） |

### 2.13.14 逐项解读

- 采纳率在 55%~71% 波动——健康区间；若某日 100%，次日站会必问『昨天审查了吗』；
- 评审缺陷 2→1 单调下降——过程收敛的证据；D5~D6 的小高峰对应新模块集中交付，属正常；
- RTM 断链 8%→0%——D4 第一次抽查发现 8% 断链（三行设计列空缺），当日修复并加 CI 周抽查，此后收敛；
- 待裁决 D8 清零——冻结前 2 天清零，留了缓冲；若 D10 仍有开放项，评审应判有条件通过；
- PE-01 的 3.4s→2.8s 是度量驱动的设计迭代：实测超标→定位到裁剪参数→调整→复测达标，全程 1 天，记录在评审证据里。

### 2.13.15 度量复盘会怎么开

里程碑评审时用 10 分钟过度量表：只讨论三类点——异常尖峰（D5 GPU 告警的根因与复发风险）、趋势拐点（断链率何时开始下降、为什么）、目标偏离（采纳率若持续低于 40% 说明提示词或基线有问题，调整而非硬撑）。复盘结论写入里程碑记录。

### 2.13.16 本节清单

- 掌握四层指标体系与采集纪律；
- 理解『零拦截可疑』『度量不考核个人』两条反直觉原则；
- 会用三张图开复盘会；
- 能把拦截数据讲成答辩正面证据。
- 五族指标总表填满自己组的数据；
- 核心数字各备『问题—数字—出处—含义』话术；
- 每个上台数字有三层下钻准备；
- 至少 2 个失败故事按四段式备好。

## 2.14 人机协同对话实录精选

### 2.14.1 本节说明

本节给出两段完整的多轮对话剧本（教学重演版，基于澜图案例真实过程整理），展示“提问—产出—追问—修订”的实际节奏。剧本中 U=学生（用户），A=AI。注意体会：好的协同不是一次问出完美答案，而是每一轮都让产物离源头更近一步。

### 2.14.2 剧本一：从访谈笔记到用户故事（4 轮）

第 1 轮（U 给出原始素材）：U 粘贴第 1 场访谈笔记（约 800 字，含原话“你把 80% 的明显图斑画出来，剩下的我修”），要求按四栏纪要结构化。A 输出结构化纪要。点评：第一轮只要求结构化，不要求提炼故事——步子小，错误面小。

第 2 轮（U 核对原话）：U 抽查发现 A 把原话润色为“希望自动化完成大部分勾绘”，指出违反“原话逐字”铁律，要求恢复并重新输出“原话引述”栏。A 修正。点评：原话润色是 AI 的顽固倾向，必须在提示词中明令 + 产出后抽查；发现一次就要在小组提示词卡里加一条禁忌。

第 3 轮（U 要求生成故事）：U 要求把痛点清单转成用户故事，每条标来源痛点编号，并执行 INVEST 自检。A 输出 9 条故事，其中 2 条不可测（“系统要快”）。点评：让 AI 自检 INVEST 是有效的，但它的“可测”标准比人宽——“响应迅速”它自认为可测。终审仍是人。

第 4 轮（U 追问价值栏）：U 指着“以便提升效率”问“提升谁的、哪个环节的、怎么度量”，要求重写价值栏。A 改为“以便单期排查周期从 2 个月缩短到 2 周（招标

5.1.2 口径)”。点评：价值栏是故事的优先级证据，逼着 AI 把价值栏写实，等于逼着团队回头查招标背景章节——好问题比好答案重要。

### 2.14.3 剧本二：spec.md 条目抽取（3 轮）

第 1 轮：U 粘贴招标 5.5.2 节原文，用第九章的抽取提示词。A 输出 6 条条目（FR-02.1~02.6）。U 用原文逐条对编号——全部命中。点评：模块级分批抽取（一次一节）比全文一次抽取的错号率低得多。

第 2 轮：U 要求为 FR-02.1 生成 Given/When/Then 验收标准。A 初稿写“Then 分割精度达到招标要求”。U 追问：“招标要求的原文是什么？完整率口径是什么？”A 改写为带  $\geq 80\%$ / $\geq 85\%$  数字的版本，但漏了“面积加权”。U 指出 CL-09 裁决，A 补上。点评：AC 生成必须经过“数字照录检查”——凡是 AC 里的数字，必须能在招标原文或裁决记录中找到出处。

第 3 轮：U 要求把 FR-02.1 的★标记与验收主体写入。A 正确标注 [STAR] 并写“第三方实测”。U 追问依据，A 引用招标 5.10.2(2)“性能验收按 5.7.2 节指标实测”。点评：让 AI 给出处是防幻觉的通用招式——“依据是什么”这五个字应当成为学生的肌肉记忆。

### 2.14.4 剧本的教学用法

- 课堂上演示剧本一的前两轮（15 分钟），学生分组复演后两轮；
- 要求学生小组提交自己项目的“真实对话摘录”2 段（含一次 AI 出错、一次人工纠正），作为过程性考核材料；
- 对话摘录按“轮次/输入摘要/产出摘要/人工处置”四列整理，不允许只贴成功记录——错误与纠正才是学习点。

### 2.14.5 实录一：HLD 数据架构起草（含纠偏）

背景：起草 HLD 第 5 章数据架构。第一轮提示词（节选）：

你是 GIS 软件架构师。基于所附 SRS 1.3 节数据需求与 design-skeleton.md，起草 HLD 1.5 节数据架构，按 5.1~5.4 四节，每节 $\leq 400$  字，叙述标注 FR 编号，存储介质限定 GeoPackage / 项目状态 JSON / 文件系统，禁止训练/样本库内容。

AI 初稿问题：5.2 节画了三条链路，但把『监督链』的日志留存写成 90 天（SRS 是 $\geq 180$  天）。纠偏方式：不重开对话，单点指出『5.2 监督链留存天数与 FR-10.3 不符，应为 $\geq 180$  天，请修正并全文复查其他数值』——数值类错误必须全文复查，因为同类错误往往成簇出现。复查果然又发现一处：瓦片缓存 TTL 写 12h（设计约定 24h）。

教训固化：数值对照是独立审查步骤，不合并到通读。

## 2.14.6 实录二：ADR 反向攻击

背景：ADR-003（常驻 GPU Worker）初稿完成后做反向攻击。提示词：

你是反对者。攻击这份 ADR 的决策（常驻 Worker 方案），找出三个最弱点，要求：每点给出触发场景与失效后果；不考虑『团队不熟』类理由；若某攻击点不成立请说明原因。

AI 给出的三点中两点成立：① 常驻进程内存泄漏（成立→写入后果节：每日低峰重启缓解，关联 REL-01 可用率条款）；② 多版本模型显存线性增长（成立→后果节：限每类模型≤2 在线版本，关联 FR-08.3）；③ 单进程是单点（成立，但本期主动接受——单用户工具无高可用要求，已在 ADR-003 后果节记录并给出容器重启策略）。裁决记录写进 ADR 修订备注。这轮对话的价值：评审会上评委恰好问到了显存问题，小组因后果节已有预案而从容应答。

## 2.14.7 实录三：tasks.md 拆分与校准

背景：从 DLD 模块清单拆 36 个原子任务。AI 草稿的覆盖率良好（模块无遗漏），但出现三类典型问题：① 估算乐观——『数据库迁移脚本 0.25 人日』改为 0.5（含 up/down/up 重放验证）；② 横切依赖遗漏——T-08 审计装饰器几乎被所有后端任务依赖，AI 只标了 2 处，人工补全；③ 验证标准含糊——『训练队列功能正常』这类不可执行表述，改为『队列 FIFO+管理员优先级调整演示』。校准后做最后一道闸：用 RTM 反查每个任务引用的编号真实性。

## 2.14.8 从实录中提炼的协作习惯

- 先骨架后展开，先结构后文字——方向错了立刻重开对话；
- 数值、编号、名称三类内容永远单独复查，不信通读印象；
- 反向攻击成为固定工序，不等到评审会才接受质疑；
- 每轮重要对话的关键结论当日写入文档或工作日志，不留在聊天记录里蒸发。

## 第3章 第一关：立项竞标

从本章起进入六关实战，每关的组织节奏相同：先讲产出是什么、在全局中占什么位置，再讲不用 AI 时人怎么做，然后讲人机怎么配合，接着剖析澜图样例，最后给检查单。本关解决立项：项目建议书、澄清质询、技术投标书、项目计划、风险登记册、述标答辩与需求确认书。「人工方法」部分不是历史陈列——不知道一份投标书在没有 AI 的年代怎么一句一句写出来，就没有能力判断 AI 写的那版哪里不对。

### 3.1 关卡导论：立项竞标要解决什么问题

#### 3.1.1 本关的位置与产出

第 1 关占 D1-D4 四个工作日，是管理线的启动关，也是整个项目唯一一次「还可以换题」的窗口——D3（第一周周三）前必须完成立项二审，那是止损点。过了这个点，题目就锁定了，后面五关都在这个题目上继续。

本关要回答三个问题：这个项目值不值得做（项目建议书），我们凭什么能做成（技术投标书），以及做到什么程度算做完（需求确认书锁定 MVP）。三个问题的答案构成后续所有工作的地基，地基歪了，第 2 关的需求就会跟着歪。

- 《项目建议书》——论证必要性与可行性，回答「为什么要做」
- 《澄清与质询记录》——把甲方模糊的意图问成可以承诺的条款
- 《技术投标书》——十四章的完整技术方案，本关工作量最大的一份
- 《项目计划 v1》——WBS 分解与甘特图，管理线的第一本账
- 《风险登记册 v1》——识别、评估、应对，管理线的第二本账
- 《需求确认书》——锁定 MVP 范围，本关与第 2 关的接力棒
- 述标答辩——8 分钟讲清方案，并扛住评委质询

过关条件：技术投标书通过评审、述标答辩合格、MVP 范围经甲方角色签字确认、项目计划与风险登记册齐备且可执行。四项有一项不合格即为不过，需返工，返工时间从本组第 2 关的预算中扣除。

#### 3.1.2 本关的三个常见陷阱

第一个陷阱是把投标书写成产品说明书。投标书是竞争性文件，它的读者是要在多个方案里选一个的评委，所以每一章都要回答「凭什么选我们」，而不只是「我们打算怎么做」。

第二个陷阱是 MVP 什么都要。18 个工作日的项目，核心功能不超过 5 个。把范围划小不是偷懒，是工程判断——一个小而完整的交付，远胜过一个大而残缺的半成

品。

第三个陷阱是过程产物最后补造。第 1 关的澄清记录、评审意见、Prompt 日志，都是当时不记事后就再也拿不出来的东西。第 6 关的 AI 使用审计从 D1 就已经开始了。

## 3.2 《项目建议书》的工作方法

### 3.2.1 任务定位与交付物

项目建议书是任务 1 链条的起点：在正式投标之前，向招标人（课程中为评审组）证明“我理解你的问题、我有可行的思路、我值得被允许投标”。它不承担逐条响应的义务，但必须把“痛点—思路—能力—效益”四件事讲清楚。配套样例见 tender\_case/01-项目建议书（教学样例）.docx（8 节 3 页）。

### 3.2.2 人工完成方法

#### 所需工具

- 文字处理：Word/WPS（样式、页眉页脚、目录域），建议建立课程统一的样式模板（标题黑体、正文宋体小四、1.5 倍行距），全队共用；
- 资料检索：行业报告（艾瑞、赛迪、自然资源部公开公报）、学术论文（知网/Web of Science）、同类开源项目（GitHub）——用于支撑痛点量化与技术趋势判断；
- 估算工具：Excel 做人月与投资估算表，功能点或类比估算法均可，关键是假设要写出来；
- 协作工具：共享文档（飞书/腾讯文档/语雀）做团队共编，保留修改痕迹作为过程证据。

#### 内容要点

- 痛点必须量化：忌写“效率低、成本高”，要写“单期全域排查约 2 个月、复核工作量占 60%”这类可核验的表述——数字来源要注明（行业报告/招标原文/类比项目）；
- 思路必须有辨识度：一句话说清你的方案与“直接买个软件/直接部署个模型”的区别（样例中的表述是“引擎 + 平台 + 规范”）；
- 能力必须可证：团队、业绩、技术储备都要落到具体事实，教学场景中用化名也要自洽；

- 效益必须对应痛点：每一条预期效益都回答“它解决了第一节哪个痛点”。

### 标准工作过程

(1) 资料研读：通读招标公告与项目背景材料，列出事实清单；(2) 痛点提炼：小组讨论归纳 3-5 个痛点并逐一找量化证据；(3) 方案构思：画一页纸的“思路图”（建设内容三段式）；(4) 估算：按 WBS 粗分解做人月估算，形成投资估算说明；(5) 撰写：按“背景—目标—方案—计划—投资—团队—风险—结论”八节成文；(6) 评审：组内交叉评审后请教师或他组同学扮演招标人提意见；(7) 修订定稿：落实意见，冻结版本。全程约 2-3 天。

## 3.2.3 人机结合完成方法

### 适用 AI 的环节与提示词

AI 在项目建议书环节的最佳定位是“快速成稿的笔杆子 + 不知疲倦的质疑者”。推荐分三轮使用：

第一轮：素材结构化。把招标文件或选题背景材料喂给 AI，要求提炼事实清单与痛点假设：

你是资深售前顾问。请阅读以下项目背景材料，输出两张表：

表 1【事实清单】：材料中出现的所有客观事实（数字、日期、角色、约束），逐条标注出处（章节/段落），不得添加材料中没有的内容；

表 2【痛点假设】：从事实推断的业务痛点，每条注明推断依据，无法找到量化依据的标注“需人工补充数据”。材料如下：.....

第二轮：初稿生成。给人设、结构、素材、约束四要素，要求 AI 逐节生成：

你是遥感应用领域的售前专家，为本科生课程演练撰写《项目建议书》初稿。

结构固定为八节：1 背景与问题理解；2 建设目标与内容；3 初步技术方案概要；

4 初步实施计划与资源估算；5 投资估算与效益分析；6 团队与业绩简介；

7 风险初判与应对；8 结论与建议。要求：

- 第 1 节只使用我提供的事实清单，痛点逐条标注对应事实编号；
- 第 3 节技术路线写“分层架构 + 关键技术名词”，不展开细节（留给投标书）；
- 所有无法溯源的数字一律写成“（教学示例）”或“（待确认）”，禁止编造；
- 全文 1500-2500 字，正式公文语体。事实清单如下：.....

第三轮：对抗审查。让 AI 扮演挑剔的招标人，对初稿提问：

你是该项目招标方的技术负责人，以“是否允许其投标”为目的审阅这份项目建议书。

请输出：（1）最有说服力的 3 点；（2）最薄弱的 3 点（具体到段落）；

（3）你会当面质询的 5 个问题。只批评，不客套。建议书全文如下：.....

### 人工必须完成的部分

- 痛点数字的真实性核验：AI 给的任何数字都要回到原始材料核对，找不到出处的删除或标注；
- “思路辨识度”的拍板：一句话定位是全文的灵魂，AI 可以给候选，但取舍必须由人完成——它决定了后续投标书的叙事主线；
- 估算假设的确认：人月、工期等假设关系承诺，必须小组讨论确认并记录理由；
- 最终签发：盖章签字页由人负责。

#### **人机协同的标准过程（建议1天完成）**

人准备素材与事实清单（1.5 小时）→ AI 结构化与痛点假设（15 分钟）→ 人筛选确认痛点（30 分钟）→ AI 生成初稿（30 分钟）→ 人修订关键段（2 小时，重点改第 1、3、8 节）→ AI 对抗审查 + 人回应（1 小时）→ 人定稿签发（30 分钟）。对比纯人工的 2-3 天，压缩到 1 天的关键是：人把精力集中在“判断与取舍”，把“成稿与挑刺”交给 AI。

#### **3.2.4 “澜图”样例剖析**

样例文件 01-项目建议书（教学样例）.docx 体现了上述方法的三个细节：

- 痛点量化：第 1 节把“周期长、成本高”落到“单期排查约 2 个月、复核工作量下降约 70%”，且效益数字标注了来源性质（同类项目实测·教学示例）；
- 辨识度表述：第 2 节“一引擎、一平台、一套规范”三段式，与投标书第三章架构叙事完全一致——建议书定下的主线贯穿了全部后续文档；
- 风险前置：第 7 节只列三条最致命的风险（精度、进度、数据安全），每条带一句应对，详见《风险登记册》——建议书不做全面风险分析，只展示风险意识。

反向观察：如果删掉第 1 节的量化数字，全文仍然通顺，但说服力塌陷——这正是 AI 初稿最常见的毛病（通顺但空泛），也是人工核验要盯住的第一件事。

#### **3.2.5 常见错误与过关检查单**

##### **常见错误**

- 把建议书写成投标书：逐条响应需求、堆砌技术细节——定位错误，建议书只讲“为什么值得让我投”；



- 痛点空泛：全文没有可核验的数字，或数字无出处；
- 效益与痛点脱节：效益列了一堆，但回答不了“解决哪个痛点”；
- AI 痕迹外露：出现“作为人工智能语言模型”等表述，或引用材料中不存在的事实。

#### **过关检查单**

- □ 每个痛点都有量化表述与出处标注；
- □ 能用一句话说清方案辨识度（电梯演讲测试：30 秒讲给非技术人员听）；
- □ 效益逐条可回溯到痛点；
- □ 全部事实（数字、业绩、人员）经人工核验或有“教学示例/待确认”标注；
- □ 风险节不超过 4 条且每条带应对；
- □ 经过一次“对抗审查”（AI 或他组扮演招标人），问题有书面回应。

### **3.3 《澄清与质询记录》的工作方法**

#### **3.3.1 任务定位与交付物**

澄清是投标截止前投标人向招标人书面提问、消除招标文件歧义的法定环节；质询是评标现场评委对投标人的口头考问。前者决定“标书写得对不对路”，后者决定“评委信不任你”。交付物为《澄清与质询记录》，配套样例见 tender\_case/02-澄清与质询记录（教学样例）.docx（8 条书面澄清 + 8 条质询模拟问答）。这一环节最能体现“人的判断力不可替代”——问什么、怎么答，AI 只能辅助，不能代劳。

#### **3.3.2 人工完成方法**

##### **所需工具**

- 招标文件精读工具：PDF 批注（Adobe/WPS 批注）或打印纸稿 + 荧光笔，歧义处逐条编号贴标签；
- 澄清函模板：固定格式（编号、原文引用、疑问点、建议解释、联系方式），Word 模板化复用；
- 会议记录工具：质询现场分工——一人主答、一人速记（或录音后整理）、一人观察评委反应；

- 问题台账：Excel 记录每个问题的提出时间、答复、对投标方案的影响、是否闭环。

#### **澄清提问的要点**

- 问“影响报价与方案的”，不问“自己能查到的”：好问题改变投标决策（如样例 Q-02“GPU 硬件谁提供”直接影响报价范围）；坏问题暴露没读标（如“工期多久”——原文写得清清楚楚）；
- 引用原文再提问：每条澄清先摘录招标原文（章节号 + 原句），再说清歧义，最后给出“我方的理解建议”——给招标人台阶，也锁定对自己有利的解释；
- 一次问全：澄清窗口通常只有一次，提问前全组做一轮“歧义清扫”，避免漏问；
- 留痕闭环：答复必须书面化（含邮件），口头答复一律请求书面确认，并归档进投标资料库。

#### **质询应对的要点**

- 听全再答：评委的问题经常包含两三个子问题，先复述确认再逐条作答；
- 答证据不答观点：评委问“精度怎么保证”，差回答是“我们很重视”，好回答是“三道防线：原型实测、阈值标定向导、人机复核兜底”（样例质询第 2 条）；
- 不知道就承诺时限：答不上来的问题，记录并承诺“会后 X 小时内书面补充”，比硬编安全得多；
- 主答人一致：技术问题主答人应是架构师或项目经理，避免多人抢答口径不一。

#### **标准工作过程**

（1）全组独立读标并各自标注歧义点；（2）汇总去重，按“对方案影响”排序；（3）逐条起草澄清函（引用原文 + 我方理解建议）；（4）教师/他组模拟招标人审查提问质量；（5）提交并跟踪答复；（6）答复归档，同步修订投标方案；（7）质询准备：预测 20 个问题、撰写应答口径、全员演练两次以上；（8）现场执行分工（主答/速记/观察）；（9）会后 24 小时内整理《澄清与质询记录》并归档。

### **3.3.3 人机结合完成方法**

#### **AI 辅助歧义清扫**

让 AI 通读招标文件，以“投标人视角”找歧义、矛盾与缺失，输出候选澄清问题清单：

你是投标人方的资深售前，目标是找出招标文件中会影响方案设计与报价的歧义点。  
请通读以下招标文件全文，输出候选澄清问题清单，每条包含：  
（1）原文摘录（章节号 + 原句）；（2）歧义或缺失说明；（3）对我方方案/报价的潜在影响；  
（4）建议的提问方式（含我方建议的解释口径）。  
按影响程度排序，最多 12 条。不得发明原文不存在的内容。招标文件如下：.....

人工随后做三件事：删掉“原文已写明”的伪问题（AI 会误报）、合并同类问题、按课程规则选定最终提交清单。

### AI 模拟评委：质询对抗演练

这是 AI 在该环节最有价值的用法——无限次数、不怕难堪的陪练。把投标书全文喂给 AI，让它扮演不同风格的评委：

你是评标委员会中以苛刻著称的技术评委，刚读完这份投标书。  
请提出 10 个质询问题，要求：（1）至少 3 个针对方案中最薄弱的环节；  
（2）至少 2 个质疑关键指标的可信度（如响应时间、精度）；  
（3）至少 1 个针对团队真实性与稳定性；（4）问题要具体，不接受套话能答的。  
投标书全文如下：.....  
（进阶用法）下面进入问答模式：你逐题提问，等我回答后，你先点评我的回答（指出含糊处、失分点），再追问一个更尖锐的问题。开始。

演练纪律：回答必须口头进行（不是打字），全程录音，回放时对照“答证据不答观点”自检。样例文档《澄清与质询记录》第 3 节的 8 条问答，就是这种对抗演练沉淀下来的“口径库”。

### 人工必须完成的部分

- 提问的最终取舍：问什么、不问什么是策略问题（暴露我方理解深度，也暴露我方底牌），必须人来定；
- 现场应答：质询是人与人的信任建立，AI 无法代劳；
- 答复的业务判断：招标人的每个答复都要评估“对方案和报价的影响”，记入台账并触发方案修订。

### 3.3.4 “澜图”样例剖析

样例《澄清与质询记录》的设计有三处值得模仿：

- 澄清表带“影响”列：Q-01~Q-08 每条都标注“对投标方案的影响”（如 Q-02 硬件由甲方提供 → 报价不含硬件），这使澄清记录成为投标方案的边界依据，而非形式文档；

- 质询第 7 条直问 AI 辅助开发可信度：“会不会把 AI 幻觉带进交付物？”——应答没有回避，而是展示流程证据（初稿—核验—体检报告），把质疑转化为加分项；
- 签署栏完整：记录人、投标人代表、评委代表三方确认，体现“留痕闭环”的职业习惯。

### 3.3.5 常见错误与过关检查单

#### 常见错误

- 澄清问题暴露没读标（问原文已明确的内容），反被扣分；
- 只问不追：答复回来后没有同步修订方案，标书与澄清答复自相矛盾；
- 质询应答背稿子：评委追问一层就露馅——口径要内化成“证据链”而不是台词；
- 口头答复未书面化，事后无凭据。

#### 过关检查单

- □ 每条澄清都引用原文、给出我方建议解释；
- □ 澄清答复全部归档，且已逐条评估“对方案/报价的影响”；
- □ 完成至少 2 轮 AI 模拟质询 + 1 轮真人模拟质询，全程录音；
- □ 形成 ≥8 条的应答口径库，主答人能脱稿复述关键两条（精度、性能）；
- □ 记录含三方签署栏，24 小时内归档。

## 3.4 《技术投标书》：人工方法、人机协同与逐章写作指南

### 3.4.1 任务定位与交付物

技术投标书是任务 1 最核心的交付物，也是评标打分的直接对象。它要同时满足两类读者：符合性审查人员（找废标项，翻得快、按表核对）和评标专家（找亮点，看得细、按分打分）。前者要求“零缺陷的形式合规”，后者要求“可验证的内容质量”。配套样例为 tender\_case/ 下的投标技术标 V2（103 页，14 章 + 7 附录）。本节前半讲人工方法，后半讲人机结合与 V1→V2 实战剖析。

### 3.4.2 人工完成方法：读标

#### 所需工具

- 精读批注：PDF 批注或打印稿 + 三色笔（红 = 实质性条款/废标项，黄 = 评分相关信息，蓝 = 格式要求）；
- 条款台账：Excel 建合规矩阵工作表（条款编号/性质/招标要求/响应章节/偏离结论/状态），从读标第一天就建账，而不是写完标书再补；
- 评分表还原：把招标文件第三章的评分标准抄成独立工作表，贴在显眼处——后续每一章的篇幅分配都对照它。

### **读标要点**

- 先找“一票否决”：资格条件、★条款、否决投标条款、投标有效期、格式与签章要求——这五类先单独成表，它们是生存线；
- 再找“打点”：评分因素、分值、得分条件，逐条换算成“我要在标书哪一章写什么”；
- 最后找“暗坑”：前后矛盾的条款、引用不存在的章节、模糊的验收标准——这些是澄清问题的来源（见第三章）；
- 关键数字抄三遍：工期、质保期、投标有效期、限价——抄进台账，编标与查标阶段都要回查。

### **读标产出**

人工读标的产出不是“我读过了”，而是四张表：资格条件响应表、实质性条款清单、评分要点拆解表、澄清问题清单。以“澜图”招标文件为例，读标提取出 13 条实质性 ★ 要求（5.4.2、5.4.4、5.7.6(1)(2)、5.7.8(1) 及 8 条 FR）与 3 条程序性 ★ 规定，这一清单直接决定了偏离表的结构。

## **3.4.3 人工完成方法：编标**

### **章节组织原则**

- 章节顺序对齐评分表：评分表的顺序是“需求理解→总体架构→功能→合规→非功能→实施→测试→业绩→团队→培训→售后”，标书章节就按这个顺序排，让评委按图索骥；
- 篇幅与分值正相关：功能实现 16 分写最厚（样例第四章 + 第五章合计约 30 页），培训 4 分写最简——不要在低分项上浪费笔墨；
- 每章开头“对表”：章首一句话说明本节响应哪个评分因素或哪组条款。

### 响应写作要点

- 响应写“可验证的事实 + 章节指引”，不写空话：差写法“完全满足招标要求”；好写法“特征缓存架构，设计目标 $\leq 2$ 秒，验收承诺 $\leq 3$ 秒，见 7.1 节”；
- ★条款零妥协：逐条响应，结论只允许“无偏离”或“正偏离”；做不到就不投，任何负偏离都不能静默写入；
- 正偏离必须可兑现：写“优于招标要求”的每一条都要自问“验收时怎么证明”；
- 图表自绘并标注“示意”：不盗用第三方版权图，架构图、流程图、甘特图全部自绘（教学场景同样要求）。

### 偏离表编制要点

偏离表是符合性审查的第一站，三条铁律：

（1）“招标要求”列照抄原文关键句，不改写语义——改写就可能被认定为“未实质响应”；

（2）★条款单独成表置于偏离表首节，一条不漏；

（3）偏离表内容与正文严格一致——表里说“见 7.1 节”，7.1 节就必须真有对应内容。这三条在 V1→V2 实战中都出过问题，详见第五章剖析。

## 3.4.4 人工完成方法：查标与定稿

### 查标要点

- 换人查：编标者自查无效（熟视无睹），必须组内交换或请他组互查；
- 接单查：用固定检查单逐项打勾（形式/实质/一致性/文字四类），不依赖临场灵感；
- 双向核对：从招标条款找投标响应（防漏），也从投标承诺找招标依据（防多报）；
- 数字一致性专项：工期、质保期、有效期、人名、金额在投标函、方案、偏离表、简历中逐一比对。

### 定稿要点

- 三件套补齐：实质性条款响应表、条款—章节索引表、评分要点响应对照表——这三张表直接降低评委查找成本，是“印象分”的主要来源；

- 过程产物归档：解析报告、合规矩阵、各轮检查记录随标书一并归档，既是质量证据也是下次投标的知识库；
- 版本冻结：定稿后建立只读基线，任何修改走变更记录。

### 3.4.5 纯人工方法的工作量估计

以 90 页招标文件、100 页投标书计，纯人工的典型投入：读标 2 人日、编标 8–12 人日（14 章）、偏离表与索引表 1.5 人日、查标 2 人日、定稿排版 1 人日，合计约 15–18 人日。其中读标提取、条款核对、一致性比对等“机械劳动”约占 40%——这正是人机结合要压缩的部分。

### 3.4.6 《技术投标书》的工作方法（下）：人机结合与实战剖析

#### 人机结合总流程：四阶段工作流

人机结合编标采用 bid-writing 技能固化的四阶段流程：读标 → 编标 → 查标 → 定稿。每阶段都有明确的 AI 任务、人工闸门和产出物：

| 阶段 | AI 承担                             | 人工闸门（不做完不得进入下阶段）          | 产出物         |
|----|-----------------------------------|---------------------------|-------------|
| 读标 | 解析招标文件，提取资格/评分/废标/★条款，生成解析报告与目录框架 | 人确认资格条件可满足、★条款可响应、目录对齐评分表 | 招标解析报告      |
| 编标 | 按目录逐章起草，同步登记合规矩阵                  | 人核验全部事实类内容（数字、资质、业绩）来源    | 标书初稿 + 合规矩阵 |
| 查标 | 规则脚本初检：★覆盖、占位符、数字一致性、废标关键词        | 人逐条复核报告，红项清零              | 体检报告（红黄绿）   |
| 定稿 | 生成三件套（★响应表/索引表/评分对照表），导出 docx/pdf | 人签发，归档过程产物                | 正式标书 + 归档包  |

#### 读标阶段：提示词与操作

把招标文件全文（或关键章节）喂给 AI，用结构化提示词提取：

你是招投标合规专家。请解析以下招标文件，输出《招标解析报告》，固定六节：

- 1 项目基本信息表（名称/编号/工期/质保/投标有效期/评标办法，逐项标注出处章节）；
- 2 资格条件逐条清单；
- 3 实质性条款（★）逐条清单（原文摘录，不改写）；

4 评分细则表（因素/分值/得分条件）；5 废标（否决投标）条款清单；

6 澄清问题建议（歧义/矛盾/缺失，各给暂定假设）。

纪律：只提取原文存在的内容；拿不准的标“需人工复核”；不得补充你的经验判断。

人工闸门：对照原文抽查 20% 的提取结果（重点查 ★ 条款有无遗漏、数字有无抄错），确认目录框架后再进入编标。澜图案例中，该步骤提取出 13 条实质性 ★ 要求 + 3 条程序性规定，人工复核确认了 2 处脚本难以定位的条款归属（5.7.6(1)(2)、5.7.8(1) 的编号）。产出物见 tender\_case/招标解析报告-v2.md。

## 编标阶段：分章生成与合规矩阵

### 分章生成提示词骨架

你是投标文件撰写专家。请撰写技术标第 X 章《章节名》，要求：

1. 本节对应评分因素【名称，分值】或响应条款【编号清单】，章首一句话点明；
2. 篇幅与分值匹配（本节目标约 N 页）；3. 每条响应写“可验证事实 + 章节指引”；
4. 可用素材：【招标解析报告相关节 + 我方资料库】；素材之外的事实写“（信息待确认）”；
5. 语体正式，图表以文字描述代替（标注“【图：名称】”）。输出后附本节合规矩阵新增行。

### 长文档的质量控制

百页级标书不能一次性生成——长输出必然质量衰减。正确做法是“模块化生成 + 程序化组装”：每章（或每节）单独生成，存为独立模块，最后用脚本合并构建 docx。澜图投标书即采用此方式：bid\_01~bid\_08 九个模块文件 + create\_bid.js 统一构建，任何一章修改后重新构建只需秒级。这一模式同时带来两个教学收益：版本管理清晰（每章可独立评审）、返工成本极低（改一处不必重排全文）。

## 查标阶段：规则脚本 + 人工复核

check\_bid.py 对招标与投标两个 docx 做规则化初检，输出体检报告（用法见第十章）。它检查四件事：★条款覆盖（招标提取的 ★ 条款在投标书中是否出现）、占位符残留（××、【】、TODO）、关键数字一致性（有效期/工期/质保期）、废标关键词响应。必须向学生强调：脚本报“未找到”不等于“没响应”——它只做子串匹配，表述差异会误报；脚本报“一致”也不等于“合规”——资格、签章、报价仍需人工逐项排查。脚本的价值是把 90% 的机械核对自动化，让人聚焦剩下 10% 的判断。

## 实战剖析：V1 → V2 的真实改进

澜图投标书有两个真实版本：V1（未使用技能流程，直接生成）与 V2（按四阶段流程重建）。两版正文技术内容基本相同，差异全部来自流程，对比数据（脚本实测）极具教学价值：



| 指标           | V1          | V2           | 问题性质                         |
|--------------|-------------|--------------|------------------------------|
| ★条款疑似未响应     | 8 条         | 1 条（说明性文字误报） | V1 偏离表只列 12 条且改写原文，脚本与人都不易核对 |
| 占位符 ××       | 9 处         | 0 处          | V1 业绩章用××占位，查标环节暴露           |
| 废标/否决/无效投标声明 | 0 次         | 1/3/3 次      | V1 缺否决条款响应声明                 |
| 第十二章标题       | 缺失（目录断链）    | 补齐           | V1 有团队内容无章节标题——目录/索引/正文不一致   |
| 附录 F 评分对照表   | 无分值、顺序不符    | 按评分表原分值原顺序   | V1 对照表是“自拟维度”，不是评分表本身        |
| 页数/字数        | 100 页/43136 | 103 页/45216  | 增量全部在合规产物，不在正文               |

### 剖析一：最危险的是“目录断链”

V1 的第十二章内容其实写了（岗位表、简历、人月计划），但缺了“第十二章 项目团队”这个一级标题。后果是：目录里没有它、索引表指向它却翻不到、评委按评分表“项目团队 6 分”找章节时找不到——6 分可能因形式缺陷而流失。这类错误人工查标极难发现（内容明明在），而“对照模板结构做机械核对”一查一个准。教训：结构完整性必须用清单核对，不能靠通读。

### 剖析二：“改写招标原文”是好心办坏事

V1 偏离表的“招标要求”列是改写后的摘要（如“容器化一键部署，干净环境 ≤2 小时”），V2 改为照抄原文（“平台应支持以容器化方式（Docker 及 Docker Compose 或同等技术）一键部署……”）。改写版读起来更简洁，但评委核对时要做“语义等价判断”，脚本核对时直接判“未找到”；照抄版让核对变成纯比对。教训：合规文档的可核对性优先于可读性，摘要可以有，但必须与原文并存。

### 剖析三：V2 的字数只多 4.8%

V1→V2 字数从 43136 增至 45216，仅 +4.8%——提升不靠扩写正文，靠补齐合规

产物。这打破了一个常见误解：“标书不好是因为写得不够多”。真实规律是：正文质量决定你能得多少分，合规形式决定你有没有资格得分。V2 的全部改进（原文引用、补标题、补声明、对齐分值）都属于后者，而这部分恰恰是 AI + 脚本最擅长保障的。

## 本节检查单

- □ 有《招标解析报告》，★条款清单经人工复核；
- □ 目录框架对齐评分表顺序，经全组确认后才开始编标；
- □ 偏离表“招标要求”列照抄原文，★条款单独成表且一条不漏；
- □ 完成至少一轮“脚本初检 + 人工复核”，红项清零，体检报告归档；
- □ 三件套（★响应表/索引表/评分对照表）齐备且与正文一致；
- □ 过程产物（解析报告、矩阵、体检报告）全部归档。

### 3.4.7 技术标十四章逐章写作指南

本节对技术标 14 章逐一给出“目的—要点—AI 用法—易错点”四段式指导。章节结构即澜图 V2.0 采用的结构（对应 bid-writing 技能 templates.md 的 14 章模板），每章标注对应评分因素与分值。

## 第一章 投标响应总述

- 目的：符合性审查的第一印象，声明全部实质性承诺；
- 要点：声明逐条编号，覆盖 ★ 条款总响应、工期、质保、知识产权、安全、否决条款知悉（V2 新增第（6）条即属此类）；每一条都能在后文找到对应章节；
- AI 用法：把招标文件的工期/质保/有效期/废标条款原文喂给 AI，要求生成编号化声明初稿，人逐条核对数字；
- 易错点：承诺与后文不一致（声明写 12 个月质保，第十章写 6 个月）——一致性专项必查。

## 第二章 需求理解（评分8分）

- 目的：证明“懂业务”而非“懂技术”；

- 要点：痛点—本质—机遇三段论（澜图的写法：四大痛点 → 手工作坊与规模化需求的矛盾 → 基础模型的机遇）；业务场景走查（谁、在什么情况下、做什么）；需求优先级映射（MoSCoW）；澄清假设显性化；
- AI 用法：AI 起草场景走查与 MoSCoW 初稿，人核对业务常识；
- 易错点：把招标原文复述一遍当作“理解”——评委要的是你自己的洞察。

### **第三章 总体技术方案（评分10分）**

- 目的：架构合理性的主战场；
- 要点：多视图表达（逻辑/数据/接口/部署/状态/安全至少选4个）；每图配文字说明与“设计决策理由”；技术选型对比表（候选/优劣/选择理由）；部署拓扑与环境配置清单；
- AI 用法：AI 生成选型对比表初稿与图的文字描述；架构图用绘图脚本生成（不盗图）；
- 易错点：只有框图没有理由——评委追问“为什么这样分层”时答不出。

### **第四章 功能实现方案（评分16分，权重最高）**

- 目的：逐条响应功能需求；
- 要点：每模块固定四件套——实现思路/界面与交互/异常处理/与条款对应关系；逐条引用 FR 编号；角色权限矩阵；预置内容清单；
- AI 用法：按模块逐个生成（一模块一提示词），生成后立即登记合规矩阵；
- 易错点：只写“做什么”不写“异常怎么办”——异常处理是拉开档次的地方。

### **第五章 关键技术实现方案（支撑功能16分）**

- 目的：论证最难的几件事有可行路线；
- 要点：每项关键技术按“难点—路线—验证安排—风险预案”四段写；澜图的写法值得模仿：分块拼接、特征缓存、文本两级链路、矢量化后处理、GPU 调度，每项都承诺原型验证时点（M3 前）；
- AI 用法：AI 做技术路线的文献/开源实现调研摘要，人判断可行性；
- 易错点：罗列名词不论证（“采用先进的深度学习技术”）；验证安排缺失。

## **第六章 开源组件清单及许可证合规说明（评分6分，★条款）**

- 目的：响应 5.4.4★，证明无知识产权瑕疵；
- 要点：全量清单表（名称/版本/许可证/用途/结论）；强传染性许可证专项分析；准入审查机制；知识产权承诺；
- AI 用法：AI 整理许可证常识表，人核实每个组件的真实版本与许可证（AI 对版本号极易记错——本节是事实核验的重灾区）；
- 易错点：清单“直接依赖”全但“关键间接依赖”漏；版本号与项目实际不符。

## **第七章 非功能设计（评分8分）**

- 目的：性能、安全、可靠性等指标的设计保障；
- 要点：逐指标写“设计措施 + 量化目标 + 验证方法”；等保二级映射表；可靠性计算（可用率、MTTR）；可移植性（第三方部署实测承诺）；技术指标响应总表收口；
- AI 用法：AI 生成等保条款映射初稿，人核对每条的技术落地；
- 易错点：指标响应总表与正文数字不一致。

## **第八章 项目实施方案（评分6分）**

- 目的：证明能按期按质交付；
- 要点：组织结构图、进度（引用计划文档）、里程碑准出、风险摘要（引用登记册）、配置管理、沟通计划、交付物清单、试运行保障；与《项目计划 v1》严格同源；
- AI 用法：从计划文档反向生成方案叙述，保证同源；
- 易错点：方案与计划“两张皮”——人月、里程碑对不上。

## **第九章 质量保障方案（评分6分）**

- 目的：测试与验收的可信度；
- 要点：四级测试策略（单元/集成/系统/验收）+ 准入准出标准；用例样例（从 SRS 派生）；缺陷分级定义；文档质量（三级签署）；验收测试组织；精度评定方法学（本项目特有）；
- AI 用法：AI 按需求条款生成用例框架清单，人补充关键用例细节；

- 易错点：用例与需求脱节（用例不从 SRS 派生）。

## **第十章 培训与售后服务方案（评分4+4 分）**

- 目的：交付后的可持续运行保障；
- 要点：分角色培训表（对象/内容/方式/考核）；SLA 承诺表（故障分级 × 响应/恢复时限）；应急预案；知识转移安排；
- AI 用法：AI 生成培训大纲与 SLA 表初稿，人确认承诺可行性；
- 易错点：SLA 承诺与招标文件时限要求不符（低分且引质询）。

## **第十一章 技术规格偏离表（符合性核心）**

- 目的：逐条响应的正式载体；
- 要点：★条款单独成表置于首节、照抄原文、一条不漏；一般条款逐条响应；非功能与商务条款成表；表内容与正文一致；
- AI 用法：AI 按合规模矩阵生成三表初稿，脚本初检覆盖性；
- 易错点：见第五章 V1/V2 剖析——改写原文、★漏条、与正文不一致。

## **第十二章 项目团队（评分6 分）**

- 目的：证明人有、人够、人稳；
- 要点：岗位配置表、关键人员简历（教学用化名但资历自治）、人月投入计划、稳定性承诺；
- AI 用法：AI 按岗位画像生成简历骨架，人填真实（或化名）事实；
- 易错点：V1 的教训——整章缺一级标题导致目录断链；人月与第八章不一致。

## **第十三章 类似项目业绩（评分6 分）**

- 目的：相关经验的证明；
- 要点：业绩清单表 + 逐案详述（背景/实施要点/可复用经验——落脚在“对本项目有什么用”）；教学场景化名标注；
- AI 用法：AI 把流水账改写成“可复用经验”导向的叙述；
- 易错点：只讲做过什么，不讲与本项目的关联性。

## **第十四章 合理化建议**

- 目的：展示超出要求的业务理解（不构成附加条件）；
- 要点：4-6 条，每条“建议 + 理由 + 对甲方的价值”；澜图的“验收影像集尽早冻结”直指验收争议源头，是范本；
- AI 用法：AI 头脑风暴候选建议，人筛选可落地者；
- 易错点：建议变成附加条件（“建议甲方增加预算采购 XX”），反而扣分。

### 3.5 《项目计划 v1》：WBS 与甘特图

#### 3.5.1 任务定位与交付物

项目计划回答三个问题：做什么（WBS）、什么时候做（甘特图与里程碑）、做到什么程度算完（里程碑准出标准）。它是管理线的核心交付物，也是后续 Sprint 计划、进度管控、验收的基线。配套样例见 tender\_case/03-项目计划 v1（WBS 与甘特图·教学样例）.docx（17 个工作包 + 甘特图 + 5 个里程碑）。

#### 3.5.2 人工完成方法

##### 所需工具

- WBS：先用便签纸/白板做头脑风暴式分解，再落到 Excel 或 WBS Chart 工具定稿；
- 甘特图：Microsoft Project / ProjectLibre（免费）做带依赖关系的专业甘特图；Excel 条件格式做简版；本课程教学样例用 matplotlib 绘制（便于程序化生成与版本管理）；
- 估算：类比估算（参照历史项目）+ 三点估算（最乐观/最可能/最悲观），记录在估算说明中。

##### WBS 分解要点

- 按交付物分解，不按动作分解：写“4.2 全自动分割（可运行增量 1）”，不写“4.2 写代码”——前者可验收，后者没法检查完成度；
- 100% 原则：WBS 必须覆盖项目全部工作（含管理、测试、文档、部署），漏掉的工作一定会在后期冒出来讨债；
- 工作包粒度：工期  $\leq 1$  周或工作量  $\leq 40$  小时，每条带责任岗位与交付物（样例 WBS 共 17 个工作包，全部满足）；

- 编码规则统一：阶段.序号（如 4.2），后续计划、周报、变更都用同一套编码引用。

### 甘特图与里程碑要点

- 先排关键路径：识别最长依赖链（样例中为：原型验证 → 自动分割 → 交互/文本链路 → 系统测试 → 初验），资源向关键路径倾斜；
- 里程碑必须带准出标准：不写“M3 设计完成”，写“设计评审通过 + 原型验证报告确认交互 ≤3 秒”——没有准出标准的里程碑只是日期；
- 留缓冲：每个迭代留 15–20% 缓冲应对缺陷修复，排满的计划必然延期；
- 工期硬约束倒挂：由合同工期（5 个月）倒排各阶段，而不是顺排后求和——顺排必然超期。

### 标准工作过程

- （1）从合同工期与里程碑要求倒排阶段；
- （2）便签头脑风暴做 WBS 初分；
- （3）对照需求清单做 100% 覆盖检查；
- （4）估算各工作包工作量与资源；
- （5）绘制甘特图并识别关键路径；
- （6）设定里程碑准出标准；
- （7）全组评审（重点：有没有漏项、缓冲够不够）；
- （8）冻结 v1 基线。纯人工约 1.5–2 天。

## 3.5.3 人机结合完成方法

### AI 生成 WBS 初稿

你是项目经理。基于以下需求清单与约束（总工期 5 个月、试运行 1 个月、团队 10 人岗位表），请生成项目 WBS 初稿，要求：

1. 按交付物分解，编码规则“阶段.序号”；
2. 工作包粒度 ≤1 周；
3. 每条给出：内容/工期(周)/责任岗位/交付物；
4. 覆盖管理、需求、设计、开发、测试、验收六类工作；
5. 输出为 Markdown 表格，并附一段“你认为最容易漏的三项工作”。需求清单如下：.....

人工复核重点：AI 最常漏的三类工作——项目管理类（它默认只写开发）、文档与培训类（投标承诺了就会忘排）、数据准备类（影像集冻结这类甲方配合事项）。  
样例 WBS 的 1、2.1、6.2 等条目正是为此而设。

## AI 生成甘特图脚本

甘特图不必手画：让 AI 根据 WBS 表直接生成 matplotlib 脚本，人调整排期后运行出图。样例 `gen_gantt_task1.py` 即由此产生，提示词要点是“数据与样式分离”——任务数据写在脚本顶部的列表里，改排期只改数据：

请根据以下 WBS 表生成一个 Python/matplotlib 甘特图脚本，要求：

1. 任务数据集中在脚本顶部的 `tasks` 列表（任务名/开始周/持续周/颜色），便于人工改排期；
2. 里程碑用菱形标注在图底部，标签防重叠（相邻里程碑错层显示）；
3. 中文标签正常显示（使用环境提供的 CJK 字体设置）；
4. 按阶段着色并附图例；
5. 保存 PNG（dpi≥200）供 Word 嵌入。WBS 表如下：.....

教学提示：脚本化出图的价值不在省几分钟画图时间，而在“计划变更时图随数据走”——Sprint 调整后排期一变，重跑脚本即得新图，版本管理里图与计划永远一致。

## 人工必须完成的部分

- 工期倒挂与缓冲分配：涉及对团队真实产能的判断，AI 的估算只能当参照系；
- 准出标准的谈判：M1“验收影像集冻结”这类涉及甲方的准出条件，必须在需求确认阶段谈定；
- 基线冻结与变更控制：v1 冻结、后续升版走 CCB，是管理纪律，AI 无权决定。

## 3.5.4 “澜图”样例剖析

- 倒排结构：5 个月硬约束下，需求 2 周、设计 3 周、三个迭代 11 周、测试初验 3 周、试运行 4 周、验收 1 周——合计 24 周，先满足硬约束再分配余量；
- MVP 优先排期：迭代一、二排核心功能（对应需求确认书的 5 项 MVP），迭代三排增强功能——任何情况下核心范围可先交付，这是进度风险的总闸门；
- 里程碑带准出标准：M3 的准出不是“设计文档写完”，而是“原型验证报告确认关键指标可达”，把技术风险的暴露提前到设计阶段；
- 人月一致性：计划 30.9 人月与投标书第十二章完全一致——计划与标书必须同源，评委交叉核对时任何出入都是减分项。

## 3.5.5 常见错误与过关检查单

### 常见错误

- WBS 写成任务清单（“学习 XX 技术”“开会”），没有交付物；
- 里程碑没有准出标准，到点宣布“完成”；



- 甘特图与 WBS 对不上（图上有 WBS 没有的任务，或反之）；
- 计划与投标书的人月、工期不一致；
- AI 生成的 WBS 漏掉管理/文档/数据准备类工作，人工未补。

#### **过关检查单**

- □ WBS 按交付物分解，每条  $\leq 1$  周，含责任岗位与交付物；
- □ 100% 覆盖检查通过（含管理、文档、数据准备）；
- □ 甘特图与 WBS 条目一一对应，关键路径已标识；
- □ 每个里程碑有可核验的准出标准；
- □ 每迭代留  $\geq 15\%$  缓冲；人月与投标书一致；
- □ 计划注明版本（v1）与变更控制方式（CCB）。

### **3.6 《风险登记册 v1》的工作方法**

#### **3.6.1 任务定位与交付物**

风险登记册是把“我担心会出事”变成“受控的管理对象”的工具：每条风险有编号、有量化等级、有应对、有责任人、有跟踪状态。它是项目计划的姊妹篇——计划假设一切顺利，登记册假设哪里会不顺利。配套样例见 tender\_case/04-风险登记册 v1（教学样例）.docx（12 条风险，2 高 9 中 1 低）。

#### **3.6.2 人工完成方法**

##### **所需工具**

- 识别工具：风险分解结构（RBS，按技术/管理/资源/外部/合规分类枚举）+ 头脑风暴会；
- 评估工具：概率影响矩阵（ $5 \times 5$ ），Excel 画热力图；风险值 = 概率  $\times$  影响；
- 台账工具：Excel/共享表格，字段固定（编号/描述/类别/概率/影响/等级/应对/责任人/状态），随项目滚动更新。

##### **风险识别要点**

- 写“条件—后果”句式：不写“进度风险”，写“开发期仅 11 周，迭代压缩导致缺陷积压”（样例 R04）——可管理的风险必须有明确的触发条件与后果；

- 从承诺反查风险：把投标书的每一条承诺（≤3 秒、≤2 小时、高危清零、精度指标）逐条问“做不到怎么办”——承诺即风险源；
- 从依赖反查风险：凡是不在自己手里的事（甲方供硬件、甲方冻结影像集、第三方实测）都是风险；
- 宁多勿漏：识别阶段不评估、不筛选，先全部登记，评估阶段再分级。

#### **评估与应对要点**

- 概率、影响分开打分：高概率低影响与低概率高影响是不同的管理对象；
- 应对分四类：规避（改变计划绕开）、减轻（降低概率或影响）、转移（合同/保险转嫁）、接受（有预案地硬扛）——每条风险写明选了哪类、具体动作是什么；
- 每条风险有且仅有一个责任人：集体负责 = 无人负责；
- 高等级风险必须有触发器：写明“什么信号出现时启动预案”（如 R03：原型实测单轮 >3 秒即启动解码量化预案）。

#### **标准工作过程**

（1）头脑风暴识别（全员，30–60 分钟，对照 RBS 逐类过）；（2）去重合并，统一“条件—后果”句式；（3）两人独立打分再对齐（概率/影响），减少主观偏差；（4）逐条定应对策略与责任人；（5）评审高等级风险（全组质询责任人）；（6）冻结 v1 并纳入双周例会固定议程。纯人工约 0.5–1 天，但必须双周更新——一次写完永不维护是最大失败模式。

### **3.6.3 人机结合完成方法**

#### **AI 辅助风险识别**

你是资深项目风险管理顾问。基于以下项目材料（招标要求摘要、投标承诺清单、项目计划 WBS），请做风险识别头脑风暴，输出候选风险清单，要求：

1. 每条用“条件—后果”句式描述；
2. 标注类别（技术/管理/资源/外部/合规/质量）；
3. 覆盖以下视角：投标承诺核查、外部依赖核查、关键路径核查、团队能力核查；
4. 至少 12 条，宁多勿漏；
5. 不打分、不提应对——只识别。材料如下：.....

接着让 AI 做第二遍“视角变换”补充：分别扮演测试工程师、甲方代表、运维人员再审一遍，往往能多挖出 3–5 条（样例 R09“验收影像集代表性不足”就是甲方视角的产物）。

#### **AI 辅助评估校准**

打分容易“拍脑袋”，可让 AI 扮演质疑者校准：“我认为 R03 概率 3、影响 4，理由是……请反驳我”。AI 的反方意见能迫使责任人把打分理由显性化——理由写不出来的分数，多半拍错了。

### **人工必须完成的部分**

- 概率与影响的最终评分：依赖对团队真实能力和甲方真实情况的了解，AI 无从知晓；
- 应对策略的选择：规避/转移常涉及商务与合同决策，必须由人权衡；
- 高风险项的处置拍板：是否接受高风险、是否因此调整投标策略，是 go/no-go 级决策。

### **3.6.4 “澜图”样例剖析**

- 承诺反查法的实例：R02（精度）对应 5.7.9 承诺、R03（性能）对应 PE-01 承诺、R08（部署 2 小时）对应 5.7.8★ 承诺——三条高/中风险全部来自投标承诺，验证了“承诺即风险源”；
- 特色风险 R11：“AI 辅助开发引入隐蔽缺陷或杜撰内容”——把课程方法本身纳入风险管理，应对是“AI 仅初稿 + 人工 Review + CI + 三级签署”，这是 AI 时代风险登记册的新类目；
- 状态字段的真实用法：12 条风险中 4 条标“已应对”（澄清已闭环的，如 R05 坐标系、R09 影像集），其余“监控中”——状态不是装饰，是双周例会的议程来源。

### **3.6.5 常见错误与过关检查单**

#### **常见错误**

- 风险写成问题（“人手不足”是当前问题不是风险）或写成空话（“可能存在技术风险”）；
- 只有识别没有应对，或应对写“加强管理”“密切关注”；
- 等级全部评“中”（不敢评高也不敢评低），登记册失去优先级意义；
- 写完归档不再更新，双周评审流于形式。

#### **过关检查单**

- □ 全部风险为“条件—后果”句式，每条有类别、概率、影响、等级；

- □ 每条风险有明确应对（规避/减轻/转移/接受 + 具体动作）与唯一责任人；
- □ 高等级风险（≥15）有触发器与预案；
- □ 至少覆盖：投标承诺、外部依赖、关键路径、团队能力四个视角；
- □ 含 AI 辅助开发相关风险（本课程必查项）；
- □ 写明跟踪机制（双周评审、升版规则），并真的执行。

## 3.7 述标答辩的工作方法

### 3.7.1 任务定位与交付物

述标是把 100 页标书压缩成 8 分钟口头叙事的环节：评委一天听十几家，你的任务不是“讲完方案”，而是“让评委记住三个理由”。招标文件明确“述标 8 分钟 + 质询 ≤10 分钟，答辩情况作为技术方案与团队能力评分的依据之一”。配套样例见 tender\_case/06-述标答辩讲稿与 AI 辅助策略（教学样例）.docx（讲稿约 1900 字 + 答辩要点 + AI 策略专章）。

### 3.7.2 人工完成方法

#### 所需工具

- 计时工具：手机秒表/演讲计时器，每轮演练必计时；
- 录音/录像：演练全程录像，回放检查语速、口头禅、肢体语言；
- 提词卡：A5 卡片，每卡一个小节 + 3 个关键词（不是全文）；
- 演示设备预案：若允许放 PPT，准备翻页笔与离线备份；若为纯口述（本案例默认），练板书式结构词（“第一……第二……第三……”）。

#### 讲稿写作要点

- 字数锁死：中文正式演讲 230–250 字/分钟，8 分钟上限对应 1900–2000 字，排练按 7 分 40 秒收束留余量；
- 按评分表组织，不按标书目录组织：评委手里拿的是评分表，讲稿每一节对应一个评分因素（样例逐节标注“对应评分：需求理解 8 分”），评委边听边打勾；
- 每节只留一个记忆点：需求理解的记忆点是“痛点是工程不是算法”，架构的记忆点是“引擎服务化、应用轻量化”——8 分钟最多让人记住 3 个点；

- 指标复读：关键数字（≤3 秒、≤2 小时、5 个月、99%）必须口头说出，评委记不住条款号但记得住数字。

### 演练过程

（1）写稿：按评分表分值分配时长（16 分的功能讲 90 秒，4 分的培训一句带过）；（2）读稿计时，砍到 1900 字；（3）脱稿演练 ≥3 轮，全程录像；（4）回放复盘：语速、口头禅（“然后”“那个”）、超时点；（5）模拟质询（见第三章 AI 对抗演练）；（6）分工定型：主述 1 人，应答支援 1-2 人，速记 1 人。

## 3.7.3 人机结合完成方法

### AI 生成讲稿初稿

你是述标教练。基于这份投标书（全文附后），为我写一份 8 分钟述标讲稿，要求：

1. 严格按评分表组织段落，每段开头标注【时间码】与【对应评分因素】；
2. 总字数 ≤1900 字；
3. 每段只有一个核心记忆点，用一句话金句概括；
4. 必须口头说出这些数字：交互 ≤3 秒、部署 ≤2 小时、工期 5 个月、高危漏洞清零；
5. 语体为口语化正式演讲，禁用书面长句；
6. 结尾 30 秒收束为“三句话总结”。

### AI 做演讲教练

下面是我的讲稿。请从评委视角批改：（1）哪句话记不住、可删；（2）哪个评分因素没对应上；（3）时间分配与分值是否匹配；（4）给出三句更有力的金句替换。讲稿：.....

进阶：把演练录音转文字稿发给 AI，让它分析口头禅频率、平均语速、超时段落——比人工回放复盘快且客观。

### 人工必须完成的部分

- 现场表达：语气、眼神、停顿是人与人的说服，AI 无法代劳；
- 记忆点的取舍：三个记忆点选哪三个，是策略判断（对应评委构成与竞争格局）；
- 临场应变：被打断、被追问时的应对，只能靠多轮模拟演练内化。

## 3.7.4 “澜图”样例剖析

- 时间码结构：0:00 开场（一句话定调）→ 0:40 需求理解 → 2:00 架构 → 4:00 关键技术 → 5:30 实施质量 → 6:40 AI 策略 → 7:30 收尾，时长与分值严格正相关；
- AI 策略进述标：6:40 一节把“本次投标本身就是 AI 辅助的实演”作为差异化卖点——在 AI 课程语境下，这是最独特的记忆点；

- 收尾三句话：“需求理解到位、方案承诺可验、过程方法可信”——对应评委打分的三条主线，说完即止，不超 8 分钟；
- 答辩红线写进文档：“不承诺标外功能、不报无法兑现的指标、不与评委争辩评分规则”——三条红线是演练时的否决项。

### 3.7.5 常见错误与过关检查单

#### 常见错误

- 讲稿按标书目录念（“第三章是总体方案，第三章第一节是……”）——评委听不出重点；
- 超时：8 分钟讲了 10 分钟，被强制终止在最关键的收尾前；
- 只有观点没有数字，或数字与标书不一致（临场说“2 秒”而标书承诺 3 秒）；
- PPT 依赖：翻页故障即冷场，没有纯口述预案。

#### 过关检查单

- □ 讲稿 ≤1900 字，逐节带时间码与评分因素对应；
- □ 时长分配与分值正相关；三个记忆点明确且全组会背；
- □ 关键数字口头说出且与标书一致；
- □ 完成 ≥3 轮脱稿录像演练 + ≥2 轮模拟质询；
- □ 分工定型（主述/支援/速记），答辩红线全员知悉。

## 3.8 《需求确认书》：锁定 MVP 范围

### 3.8.1 任务定位与交付物

需求确认书是中标后、开发前双方签署的范围基线文件，任务是 1 的收口动作。它把“我们要做什么”从 100 页标书的叙述压缩为一纸可执行的契约：MVP 核心功能 ≤5 项、每项的验收依据、范围边界、变更程序。它的价值在项目后半程才显现——需求蔓延发生时，确认书是双方唯一共同承认的“原点”。配套样例见 tender\_case/05-需求确认书（教学样例）.docx。

### 3.8.2 人工完成方法

#### 所需工具

- 需求追踪矩阵：Excel，列“需求编号—招标条款—投标响应—是否 MVP—验收依据—状态”；
- MoSCoW 排序表：Must/Should/Could/Won't 四分清单，MVP 只能来自 Must；
- 正式文书模板：确认书属准合同文件，用公文模板（标题、依据、正文条款化、签署栏）。

### **MVP 界定要点**

- ≤5 项是硬约束：什么都重要 = 什么都不重要。选择标准三条——招标★条款优先（不满足即废标级）、业务闭环必需（缺了流程跑不通）、技术风险前置（最难的先做先验）；
- 验收依据必须可测：每条 MVP 功能配可量化或可判定的验收标准，引用招标条款编号（样例 MVP-2 的验收依据直接引用 5.7.9 精度指标与 PE-02 性能指标）；
- 范围边界要“双向写”：既写做什么（5 项），也写不做什么（范围外清单）——范围外清单是防止后期扯皮的关键，样例明确列出批量任务高级调度、QGIS 插件、历史影像补配三项边界；
- 变更程序先小人后君子：CCB 组成、申请单格式、评估维度（工期/质量/费用）、双方书面确认——写进确认书本身。

### **标准工作过程**

（1）从投标书与澄清记录导出全部需求条目建追踪矩阵；（2）全组做 MoSCoW 排序，教师/甲方代表参与 Must 项确认；（3）对候选 Must 项做“≤5 取舍”辩论，记录落选理由；（4）为每项 MVP 撰写验收依据（与招标指标对齐）；（5）撰写范围外清单与变更条款；（6）双方（课程中为小组与评审组）逐条走读后签署；（7）基线冻结，后续变更走 CCB。纯人工约 1 天，其中大半时间花在“取舍辩论”——这恰是最不该交给 AI 的部分。

## **3.8.3 人机结合完成方法**

### **AI 辅助需求条目化与验收依据起草**

你是需求工程师。基于以下招标需求条款与投标响应，请输出需求追踪矩阵初稿（Markdown 表）：  
列：需求编号/招标条款原文摘要/投标响应章节/建议 MoSCoW 分级（Must 限 5 项以内）/验收依据草案。  
验收依据要求：可量化指标直接引用招标编号；不可量化的写成“可判定语句”（谁、在什么条件下、验证什么、通过标准）。

分级理由附在最后。材料如下：.....

### AI 做“范围辩论”的反方

我们初步选定 5 项 MVP 如下：.....。请扮演甲方业务负责人，论证为什么你不同意砍掉的功能（从范围外清单里挑 2 项）应该进 MVP；再扮演项目监理，指出我们验收依据中不可测、易扯皮的表述。

反方演练的价值：签署前把甲方的异议预演一遍，确认书的条款就要能顶住这些异议。

### 人工必须完成的部分

- MVP 的最终取舍：范围即承诺，取舍即策略——必须由全组与教师（甲方）面对面谈定；
- 验收依据的可行性确认：承诺的指标必须在原型验证中有支撑，不能为签而签；
- 签署：确认书经签署即产生约束效力（课程中为评审效力），签字前逐条走读。

### 3.8.4 “澜图”样例剖析

- 5 项 MVP 的选择逻辑：MVP-1 影像接入（流程入口）+ MVP-2/3/4 三类分割方式（全部 ★ 条款）+ MVP-5 矢量化导出（成果出口）——构成最小业务闭环，且 ★ 条款全覆盖；
- 验收依据的写法：MVP-1 写“10 GB 影像上传完整性校验通过；1 亿像素首屏 ≤3 秒”——数字全部能在招标文件 PE 指标中找到出处，没有新造承诺；
- 范围外清单留有余地：批量任务写“本期提供基础批量能力，高级调度纳入后续迭代”——不是简单拒绝，而是分期兑现，与投标承诺不冲突；
- 变更条款的颗粒度：含申请单、CCB 组成、评估维度、书面确认，甚至写明“紧急缺陷修复 48 小时内补登”——可执行的程序才是真正的程序。

### 3.8.5 常见错误与过关检查单

#### 常见错误

- MVP 超 5 项（不会取舍），或 5 项不构成业务闭环（只有功能没有出口）；
- 验收依据不可测（“运行流畅”“界面友好”）；
- 只写做什么、不写不做什么，范围边界缺失；
- 变更程序一句话带过，后期无从执行；



- 签署流于形式，双方没有逐条走读。

### 过关检查单

- □ MVP ≤5 项且构成业务闭环，★ 条款全覆盖；
- □ 每项验收依据可测、可判定，指标有招标出处；
- □ 范围外清单明确且与投标承诺不矛盾；
- □ 变更控制条款可执行（CCB/申请单/评估维度/书面确认）；
- □ 双方逐条走读后签署，基线冻结并归档；
- □ 需求追踪矩阵同步建立（确认书 ↔ 招标条款 ↔ 投标响应三向可追）。

## 3.9 全程演练：澜图第1关完整记录

本节以叙事方式复盘澜图案例任务1的完整过程——每天做了什么、AI 产出了什么、人改了什么、踩了什么坑。它是本章前八节所有方法的“带注释的实战录像”，建议读者在动手前通读一遍，建立整体节奏感。注意节奏口径：澜图案例按企业真实编标节奏展开（14天），课程通关则把第1关压缩进D1~D4四天——压缩映射见3.9.13，压缩出来的时间差由第3.4节的人机协同方法承担。文中日期为教学推演。

### 3.9.1 第1天：读标——把90页招标文件变成四张表

上午，全组各自通读招标文件并三色标注。下午做了一件关键的事：没有急着写标书，而是先把招标文件全文交给AI，用结构化提示词提取《招标解析报告》。AI 用约10分钟输出了六节报告：基本信息表、资格条件、★条款清单、评分细则、废标条款、澄清建议。人工复核发现两处问题：一是AI把5.7.6的两条★子款合并成了一条（原文的（1）（2）编号被吃掉）；二是工期条款的出处标错了章节。复核人当场修正，★条款最终确认为13条实质性+3条程序性。当天产出：解析报告确认版+合规矩阵空表+目录框架。当天教训：AI提取快，但编号与出处必须人工逐条回原文核对——这一步花了40分钟，是全天天性价比最高的40分钟。

### 3.9.2 第2天：目录确认会与澄清函

上午开目录确认会：教师扮演招标方顾问，对目录框架提了三个意见——“团队与业绩顺序应按评分表颠倒”“缺评分响应对照表的位置”“第十四章建议控制在2页内”。目录改定后冻结，全组签字——这个签字是闸门，此后任何人改目录都要走变更。下

午写澄清函：AI 歧义清扫给出 11 条候选，人工删掉 3 条“原文已写明”的伪问题、合并 2 条同类，最终提交 6 条。当天教训：澄清问题的质量取决于删，不取决于多。

### 3.9.3 第 3 天：项目建议书——三轮人机接力

按第二章的三轮法执行：AI 结构化素材（15 分钟）→ 人筛选痛点（30 分钟，删掉了 AI 编造的一个行业数据）→ AI 成稿（30 分钟）→ 人重写第 1、3、8 节的关键段（2 小时）→ AI 扮演招标人对抗审查（提出 5 问，其中“你们与直接部署开源模型何异”一问促使全组把“引擎+平台+规范”的表述磨利）→ 定稿。全天 6 小时完成，纯人工估计 2 天。当天教训：AI 对抗审查提出的最尖锐问题，恰恰成为建议书最有力的卖点。

### 3.9.4 第 4-6 天：编标主体——模块化生成流水线

三天完成 14 章初稿。标准动作：每天早会分章到人 → 每人用“分章生成提示词”驱动 AI 出初稿 → 第二人四遍评审（重点事实与合规两遍）→ 合规模矩阵登记 → 晚会对齐口径。第 5 天晚会发现一起典型事故：功能章的“单轮  $\leq 3$  秒”与非功能章的“单轮  $\leq 2$  秒”不一致——架构师在第五章写了设计目标 2 秒，被功能章作者误抄为承诺值。处置：确立“承诺值  $\leq 3$  秒、设计目标  $\leq 2$  秒”的统一口径，两处分别改写。当天教训：分头写作必须有“单一事实源”，关键数字只在一处定义、处处引用。

### 3.9.5 第 7 天：计划与风险——AI 初稿、人工校准

AI 按 WBS 提示词产出 15 个工作包初稿，人工补上漏掉的三项（项目管理、验收影像集冻结、培训），并按交付物导向改写了 4 条动作式表述（“开发后端接口”→“开放接口六类能力联调完成”）。甘特图脚本由 AI 生成、人工调整排期后出图。风险登记册 AI 给出 14 条候选，全组打分辩论 1 小时：R02 精度的影响分从 4 改 5（因为验收影像集冻结后无退路），R10 数据泄露从“中”降“低”（内网部署已有确定性应对）。当天教训：AI 给的分数是参照系，定分必须结合本组实际辩论。

### 3.9.6 第 8 天：查标——脚本初检与红项清零

check\_bid.py 首跑：★疑似未响应 8 条、占位符 9 处、废标关键词 0 命中。全组逐条复核：8 条“未响应”中，3 条是表述差异（如“平台应支持以容器化方式”在标书中写成了“容器化一键部署”），4 条是偏离表改写了原文导致匹配失败，1 条（第十二章标题）是结构缺陷。处置决定：不是改脚本迁就标书，而是改标书迁就原文——偏离表照抄招标原文、补章节标题、业绩章占位符换化名、第一章补否决条款声明。当天

教训：脚本报红先问“标书错还是脚本局限”，多数时候是标书有更合规的写法。

### 3.9.7 第 9 天：定稿——V2 出炉

修复后复检：疑似未响应降到 1 条（招标说明性文字，确认为脚本固有误报），占位符 0，数字一致性全绿。生成三件套，交叉核对索引表指向的章节真实存在。V2 实测 103 页、45216 字，比 V1 多 4.8%——全部增量在合规产物。标书冻结，入版本库打 tag。当天教训：“提升”不等于“写更多”，合规性是独立的质量维度。

### 3.9.8 第 10 天：述标准备——讲稿与对抗演练

AI 按评分表生成讲稿初稿 2400 字，超时——砍到 1900 字的过程中，全组被迫做了“什么最不能删”的取舍（最终保住了三个记忆点）。脱稿演练 3 轮全程录像；AI 模拟评委提出 10 问，其中“AI 辅助开发会不会把幻觉带进交付物”一问最初答得防守，改为展示体检报告与四遍评审记录后，从失分题变成加分题。当天教训：演练录音回放发现的口头禅（平均每分钟 4 个“然后”）比任何老师点评都管用。

### 3.9.9 第 11–12 天：述标答辩与需求确认书

正式述标 7 分 42 秒收束，质询 10 分钟答 6 问，其中 1 问（“若验收影像集与训练分布差异大怎么办”）现场承诺 24 小时内书面补充，并于当晚兑现。需求确认书的 MVP 辩论持续 90 分钟：批量任务功能最终在教师扮演的甲方坚持下以“基础能力进 MVP、高级调度列范围外”的折中落笔——双方签字。当天教训：答不上来的问题给时限承诺比硬编好；MVP 谈判的折中方案要写进边界条款，不能停留在口头。

### 3.9.10 第 13–14 天：归档与复盘

归档包按附录 B 清单整理：解析报告、合规矩阵（含逐日更新记录）、两轮体检报告、V1→V2 对比、代表性提示词片段（标注人工修改处）、演练录像、签署版确认书。复盘会每人讲“一个最有效的提示词 + 一个最坑的幻觉”。全组共识：AI 把 40% 的机械劳动压缩到近零，但最值钱的三件事——目录确认、红项裁定、MVP 取舍——全是人做的。这句话后来被写进了课程海报。

### 3.9.11 演练记录的使用建议

- 各组 D1 开工前通读本节，把 13 个“当天教训”抄入本组 rules.md 的禁区节；
- 每天收工时对照本节相应天数自查：“他们这天踩的坑，我们踩了吗”；

- 复盘会对照 25.10 的全组共识，给出本组版本的回答——这是过程分答辩的最佳素材。

### 3.9.12 全程量化数据总览

把 14 天真实节奏的关键量化数据汇成一张表，学生可直观看到人机协同的“账本”——AI 省下的时间在哪里，人花的时间在哪里：

| 环节         | AI 用时          | 人工用时                | 纯人工估计      | 人工时间花在哪          |
|------------|----------------|---------------------|------------|------------------|
| 读标与解析报告    | 10 分钟          | 2.5 小 时<br>(复核+确认会) | 约 2 人日     | ★条款逐条回原文核对、目录决策  |
| 项目建议书      | 45 分 钟<br>(三轮) | 3.5 小时              | 2-3 人日     | 痛点筛选、关键段重写、对抗回应  |
| 投标书 14 章初稿 | 约 6 小时<br>(分章) | 约 12 小时<br>(评审修订)   | 8-12 人日    | 事实核验、口径统一、关键技术论证 |
| 计划与风险      | 1 小时           | 4 小时                | 1.5-2 人日   | 补漏项、打分辩论、准出标准    |
| 查标（两轮）     | 脚本每次秒级         | 5 小时                | 2 人日       | 红项裁定、原文对齐改写      |
| 述标准备       | 1.5 小时         | 5 小时                | 2 人日       | 脱稿演练、质询应答        |
| 确认书与签署     | 30 分钟          | 3 小时 (含 90 分钟辩论)    | 1 人日       | MVP 取舍谈判         |
| 合计         | 约 10 小时        | 约 35 小时             | 约 17-22 人日 | ——               |

读表要点：总工时压缩约 70%，但人工时间并非“剩下的零头”——它高度集中在判断密集处（核对、决策、谈判、演练）。AI 压缩的是“生产时间”，人保有并强化的是“判断时间”。这张表也是回答评委“AI 到底帮了什么”的最佳证据。

### 3.9.13 总时间线（真实节奏 2 周，课程压缩为 D1~D4）

澜图案例的原始记录按真实编标节奏展开，共 14 天；课程通关把第 1 关压缩为

D1~D4 四天，映射关系为：读标与建议书（第 1~3 天）→D1，编标主体（第 4~7 天）→D2，查标与定稿（第 8~9 天）→D3 上午，述标准备、答辩与需求确认（第 10~12 天）→D3 下午至 D4，归档与复盘（第 13~14 天）→并入 D4 晚的归档节奏。以下为经过“澜图”案例验证的节奏，各组可按题目复杂度微调，但阶段顺序不可颠倒——每个阶段都是下一阶段的输入：

| 日程    | 阶段任务    | 关键动作                         | 当日报出               |
|-------|---------|------------------------------|--------------------|
| D1    | 启动与读标   | 全组通读招标文件；三色笔标注；建合规矩阵台账       | 四张表（资格/★/评分/澄清）初稿  |
| D2    | 读标收口    | AI 生成解析报告；人工复核★清单；目录框架确认会    | 招标解析报告（确认版）        |
| D3    | 澄清      | 歧义清扫；澄清函提交；项目建议书素材准备         | 澄清函 + 事实清单         |
| D4    | 建议书     | AI 初稿 → 人工修订 → 对抗审查          | 项目建议书（定稿）          |
| D5    | 编标启动    | 按分值分工到人；高权重章节先写（功能 16 分）     | 第四、五章初稿            |
| D6-D7 | 编标主体    | 分章生成与人工核验并行；合规矩阵逐日更新         | 全部章节初稿             |
| D8    | 计划与风险   | WBS/甘特图/风险登记册（AI 初稿 + 人工校准）  | 项目计划 v1 + 风险登记册 v1 |
| D9    | 查标      | 脚本初检 + 交叉人工复核；红项清零           | 体检报告（终版）           |
| D10   | 定稿与述标准备 | 三件套补齐；标书冻结；讲稿定稿；分工定型         | 正式标书 + 讲稿          |
| D11   | 演练日     | 脱稿演练 ≥3 轮；AI 模拟质询 ≥2 轮；口径库定稿 | 演练录像 + 口径库         |
| D12   | 述标答辩    | 正式述标 + 质询；24 小时内整理记录         | 澄清与质询记录            |

| 日程  | 阶段任务 | 关键动作                 | 当日出        |
|-----|------|----------------------|------------|
| D13 | 需求确认 | MVP 取舍辩论；逐条走读；<br>签署 | 需求确认书（签署版） |
| D14 | 复盘归档 | 过程产物打包；组内复盘会         | 归档包 + 复盘纪要 |

### 3.9.14 小组分工模板（3-5 人）

| 角色           | 任务 1 中的主责                     | 兼任        | 关键交付         |
|--------------|-------------------------------|-----------|--------------|
| 项目经理（组长）     | 时间线推进、澄清函、计划与风险登记册、述标主述       | 评审组织      | 计划 v1、风险册、讲稿 |
| 技术负责人（架构）    | 投标书第三、四、五章（架构/功能/关键技术）、质询技术应答 | 原型验证方案    | 技术章节         |
| 需求/质量工程师     | 读标解析、偏离表、需求确认书、检查单执行          | 脚本初检执行    | 解析报告、偏离表、确认书 |
| 商务/文档工程师     | 建议书、团队与业绩章、排版与三件套、归档          | 会议记录      | 建议书、索引表、归档包  |
| （第 5 人）测试工程师 | 查标主查、验收依据核对、演练计时与录像           | AI 模拟质询主持 | 体检报告、演练记录    |

分工的三条纪律：（1）每份交付物有唯一主责人，但查标必须换人；（2）AI 操作（提示词撰写与结果初筛）由主责人完成，核验由第二人完成——操作与核验分离；（3）每日 15 分钟站会同步“昨日出/今日计划/卡点”，卡点当日升级不隔夜。

### 3.9.15 三个关键评审节点

- 目录确认会（D2 末）：全组 + 教师确认标书目录对齐评分表——这是编标的闸门，目录错了后面全错；
- 红项清零会（D9）：逐条过体检报告，每条红项指定修复人与时限，当场复核；

- MVP 取舍辩论（D13）：教师扮演甲方，各组陈述 5 项 MVP 的选择逻辑并接受质询——这是任务 1 最有训练价值的 90 分钟。

### 3.9.16 教师观察点（过程分采集）

- D2 看解析报告：★条款是否一条不漏，有无人工复核痕迹；
- D5-D7 看合规矩阵：是否逐日更新，还是最后一天补的（补的没有过程价值）；
- D9 看体检报告迭代：红项从几条降到零，降的过程比结果更说明问题；
- D11 看演练录像：是否真练，还是临场背稿；
- D14 看归档包：七类产物是否齐全（见附录 B）。

## 3.10 过关自检与向下一关的交接

### 3.10.1 任务 1 的产出如何喂给任务 2

| 任务 1 产出        | 任务 2（需求与设计）中的用途                   |
|----------------|-----------------------------------|
| 需求确认书（MVP 5 项） | 直接转化为产品 Backlog 的史诗（Epic）与首批用户故事  |
| 需求追踪矩阵         | 扩展为“需求—设计—代码—测试”四向追踪矩阵            |
| 项目计划 v1        | 迭代一、二细化为 Sprint 1/2 计划，里程碑不变      |
| 风险登记册 v1       | R02/R03 等技术风险转化为 Sprint 1 的技术验证任务 |
| 投标书第三、四章       | 架构六视图与功能方案是概要设计的起点（深化为设计说明书）      |
| 过程产物归档包        | 人机协同过程分延续累计；本组 skill 开始筹建         |

### 3.10.2 衔接期的三个常见断层

- 范围漂移：开发中忘了 MVP 边界，把“范围外清单”的功能做进来了——对策：Sprint 计划会第一项议程是重读需求确认书；
- 文档分叉：标书承诺与设计文档不一致（如标书写三格式导出，设计只做了两种）——对策：需求追踪矩阵每周同步；

- 风险沉睡：登记册不再更新——对策：双周评审雷打不动，状态字段每会必过。

### 3.10.3 术语表

| 术语          | 含义                                             |
|-------------|------------------------------------------------|
| 实质性条款（★）    | 不允许负偏离的招标要求，负偏离即无效投标                           |
| 合规矩阵        | 逐条登记招标要求与投标响应对应关系的工作表                          |
| 偏离表         | 正式投标文件中列明响应与偏离情况的必备章节                          |
| 三件套         | 实质性条款响应表 + 条款—章节索引表 + 评分要点响应对照表                |
| MVP         | 最小可行产品：构成业务闭环的最小功能集（本任务 ≤5 项）                  |
| WBS         | 工作分解结构：按交付物逐层分解的项目工作清单                         |
| 里程碑准出标准     | 进入下一阶段前必须满足的可核验条件                              |
| CCB         | 变更控制组：评估并批准基线变更的正式机制                           |
| 红黄绿体检       | 按严重程度对检查结论分级的方法：红项必须清零                         |
| Spec-Driven | 规范驱动开发：先写机器可执行的规范，再让 AI 依规范生成                  |
| skill（技能包）  | SKILL.md + references + scripts 组成的可复用 AI 工作规程 |
| 四要素提示词      | 人设 + 结构 + 素材 + 约束的提示词骨架                        |
| 护栏          | 防止 AI 产出质量滑坡的自动化检查（测试/静态检查/规则脚本）               |

（指导书全文完）

### 3.10.4 A.1 项目建议书

- □ 痛点量化有出处 □ 一句话辨识度 □ 效益回溯痛点 □ 事实核验/标注 □ 对抗审查有回应

### 3.10.5 A.2 澄清与质询



- □澄清引用原文 + 建议解释 □答复归档并评估影响 □≥2 轮 AI 模拟质询 □口径库 ≥8 条 □三方签署

### 3.10.6 A.3 技术投标书

- □解析报告 + ★清单人工复核 □目录对齐评分表 □偏离表照抄原文、★单独成表 □脚本初检红项清零 □三件套齐备 □过程产物归档

### 3.10.7 A.4 项目计划

- □WBS 按交付物、≤1 周、含责任岗位与交付物 □100% 覆盖 □甘特图与 WBS 一致 □里程碑有准出标准 □人月与标书一致

### 3.10.8 A.5 风险登记册

- □“条件—后果”句式 □应对四类明确 + 唯一责任人 □高风险有触发器 □含 AI 辅助开发风险 □双周跟踪机制

### 3.10.9 A.6 述标答辩

- □讲稿 ≤1900 字带时间码 □时长与分值正相关 □关键数字与标书一致 □≥3 轮录像演练 □分工与红线定型

### 3.10.10 A.7 需求确认书

- □MVP ≤5 项构成闭环 □验收依据可测有出处 □范围外清单 □CCB 变更程序可执行 □双方走读签署 □追踪矩阵建立

| 归档物                      | 来源环节 | 证明什么            |
|--------------------------|------|-----------------|
| 招标解析报告                   | 读标   | AI 提取 + 人工复核的过程 |
| 合规矩阵工作表                  | 编标   | 逐条响应的登记痕迹       |
| 各轮体检报告（含红项清零记录）          | 查标   | 规则初检 + 人工复核的迭代  |
| V1→V2 对比材料               | 定稿   | 流程改进的量化证据       |
| 代表性提示词与 AI 输出片段（标注人工修改处） | 全流程  | 人机分工的真实记录       |
| 澄清/质询记录与演练录音             | 质询准备 | 对抗演练的执行证据       |

| 归档物                | 来源环节  | 证明什么      |
|--------------------|-------|-----------|
| WBS/甘特图/风险登记册的版本历史 | 计划与风险 | 基线管理与滚动更新 |

## 第 4 章 第二关：需求与规格

本关把意图变成可验证的规格：用户访谈与用户故事、按 GB/T 9385 编写 SRS、同步产出机器可读 spec.md、用 AI 反向澄清暴露歧义、建立需求追踪矩阵并完成基线冻结。过关标准只有一条：没有不可测试的需求。

### 4.1 关卡导论：从意图到可验证的规格

#### 4.1.1 本关的位置与产出

第 2 关占 D5–D6 两个工作日，是研发线的起点。它承接第 1 关锁定的 MVP 范围，把甲方的意图变成两份互为镜像的规格：一份给人看的 SRS，一份给机器读的 spec.md。这两份文件此后就是项目的单一事实源——设计要从它派生，代码要向它对齐，测试要拿它当判据。

三天写完一份合规的 SRS 是紧张的，所以本关是全课程中人机协同密度最高的一关。但要清楚哪些能加速、哪些不能：文档骨架、条目展开、格式统一可以交给 AI；而哪条需求该留、优先级怎么排、验收标准定在哪里，这些判断只能由人做。

- 《软件需求规格说明书 SRS》——对标 GB/T 9385，面向人的正式规格
- spec.md——机器可读的结构化规格，Spec-Driven 的核心资产
- 《AI 反向澄清记录》——让 AI 扮演苛刻评审，把模糊之处逼出来
- 《需求追踪矩阵 RTM v1/v2》——追踪链的起点
- 《需求评审记录》与基线冻结说明——V1.0 到 V2.0 的演进证据

过关条件：SRS 通过正式评审、spec.md 通过脚本校验且与 SRS 双轨一致、RTM 覆盖全部 FR、需求基线正式冻结并留下版本记录。

#### 4.1.2 为什么坚持双轨

只写 SRS，AI 读不懂——自然语言文档里的需求条款，AI 抽取时会漏、会错、会自行归并。只写 spec.md，人看不下去，也过不了国标评审——它没有背景、没有论证、没有可读的上下文。

所以两份都要，并且必须永远一致。维护这份一致性听起来是额外负担，实际上是本关最有价值的训练：它逼着你把每一条需求想清楚到「能同时用散文和结构化语句表达」的程度。想不清楚的条款，在转 spec.md 时一定会卡住。

### 4.2 用户访谈与需求获取：方法与访谈样例精讲

### 4.2.1 本节学习目标

- 掌握访谈准备三件套：访谈提纲、对象矩阵、记录模板；
- 学会在教学场景下构造“虚拟画像”替代真实用户，并保证画像的业务真实性；
- 掌握访谈实施的关键技巧：开放提问、追问痛点、捕捉原话；
- 能够产出当日确认的高质量访谈纪要。

### 4.2.2 为什么任务 2 从访谈开始

招标文件写的是“采购方的要求”，用户访谈挖的是“使用者的处境”——两者经常不一致。招标条款 FR-04.1 只写“支持文本提示提取”，但新人作业员“记不住类别代码、不敢调参数”的真实处境，才能解释为什么需要类别词库、参数方案和别名映射。澜图案例中，第 2 场新人访谈直接强化了 FR-02.1 参数业务化说明、FR-02.3 过滤方案、FR-04.3 类别方案三条需求的优先级——这是招标文件里读不出来的。

另一个原因是冲突发现：访谈期望可能与合同条款冲突。澜图案例中“2 小时部署”在用户看来是体验期望（Could），但招标文件 5.7.8(1) 是★实质性条款——最终以合同为准升级为 Must。这类冲突只有同时掌握“用户语言”和“合同语言”的人才能识别。

### 4.2.3 访谈准备三件套

#### 访谈提纲

提纲不是问卷，而是“话题地图”。设计要点：

- 从招标文件倒推：先通读第五章，把每个功能模块转成 1-2 个开放问题（如 FR-03 交互分割 →“您现在修正一个错误图斑的步骤是什么？哪一步最耗时？”）；
- 开放式优先：多问“上一次……是什么情形”，少问“您是否需要……”（诱导性提问只会得到礼貌性肯定）；
- 12 问左右为宜：覆盖背景（20%）、现状流程（30%）、痛点（30%）、期望（20%）；澜图案例即采用 12 问提纲；
- 每场访谈针对角色定制 2-3 个专属问题（对管理员问安全与日志，对复核员问追溯与责任）。

#### 对象矩阵

用一张矩阵保证访谈覆盖所有角色且不漏关键视角：横轴是系统角色（作业员/复

核员/业务管理员/系统管理员），纵轴是经验层次（资深/新人）。澜图案例 4 场访谈覆盖 4 类角色，其中作业员特意覆盖了资深（陈工）与新人两人——新人视角暴露了“上手难”这个资深者意识不到的需求源。

### **记录模板**

固定四栏：①原话引述（逐字记录关键句，带引号）；②事实（流程、数据、周期）；③痛点（说话人的判断）；④线索（指向可能的需求条目）。原话栏最重要——“你把 80% 的明显图斑画出来，剩下的我修，就谢天谢地了”这类原话，是用户故事优先级最硬的证据。

## **4.2.4 教学场景：虚拟画像的构造方法**

课程中多数小组没有真实用户可访谈，采用“虚拟画像 + 角色扮演访谈”：一名组员扮演用户（按画像设定回答），其余组员按真实访谈流程实施。画像要可信，必须满足四条：

- 有岗位事实：年龄、工龄、日常工具、考核方式（如“日均比对影像 6 小时以上”）；
- 有业务痛点：痛点必须能从招标文件背景章节（如澜图招标 5.1.1 四大痛点）找到投影，不能凭空编造；
- 有利益立场：科室负责人关心“按期出数、对上交代”，管理员关心“别给我添运维负担”——立场决定他们访谈中强调什么；
- 有反差点：画像之间要有视角冲突（资深 vs 新人、业务 vs 运维），冲突是需求的矿脉。

虚构不等于随意：画像的全部业务事实必须能溯源到招标文件或公开行业资料，人名地名一律化名并标注“教学示例”。

## **4.2.5 访谈实施要点**

- 开场约定：说明目的（了解现状，不是考核）、时长、记录方式；先取得信任再进正题；
- 跟班观察优先于口头描述：能看真实操作就不只听叙述——用户嘴上说“很简单”，手上却在五个窗口间反复切换；

- 追问三连：用户说“太慢”→追问“慢在哪一步”→“一次要多久”→“多久发生一次”。痛点必须落到频率和时长才可排序；
- 控制倾诉跑偏：礼貌拉回话题地图，但先记下跑偏内容——跑偏处常藏着意料之外的需求；
- 当场复述确认：结束前用自己的话复述一遍理解，请对方纠正。“确认”动作比“记录”动作更重要。

#### 4.2.6 访谈纪要的当日闭环

纪要当日整理、当日发受访人确认，24 小时后记忆衰减会导致细节失真。纪要结构固定为：基本信息（时间/对象/角色）→现状流程→痛点清单（编号）→功能期望→原话摘录→待核实问题。澜图案例的 17 条痛点清单与 21 条用户故事之间可逐条溯源，靠的就是这个编号习惯。AI 可以参与纪要结构化（见第四章），但“发出确认”这个动作必须由访谈人完成并留存对方确认记录。

#### 4.2.7 常见错误与纠偏

| 常见错误            | 后果              | 纠偏                      |
|-----------------|-----------------|-------------------------|
| 问“您要不要 AI 自动分割” | 得到礼貌性肯定，无信息量    | 改问现状：“上次变更调查您怎么做的，花了多久” |
| 只访谈管理者          | 得到的是报表需求，丢掉作业痛点 | 管理者+一线+新人三层覆盖           |
| 记录评价不记录原话       | 优先级判定失据         | 四栏模板，原话带引号              |
| 访谈结束即收尾         | 误解固化进需求         | 纪要当日发回确认，留确认记录          |
| 画像只有人设没有痛点      | 访谈变成闲聊          | 画像四条标准逐项检查              |

#### 4.2.8 本节小结

访谈是需求链的第一公里：它提供招标文件之外的真实处境、冲突线索与优先级证据。人工方法的核心是“结构化准备 + 开放式实施 + 当日闭环确认”。下一章讲从访谈素材到用户故事与 MoSCoW 优先级的转化方法。

#### 4.2.9 访谈样例精讲：三次访谈是怎么做的

本节完整复盘本关样例《用户访谈记录与 MoSCoW 优先级》的三次访谈，讲解每一轮的目标设计、提问技巧、记录方法与产出转化。学生小组应按同样的节奏组织自己的访谈，而不是一次访谈走完所有议程。

## **第一次访谈：业务全景与角色识别（90 分钟）**

对象：业务主管（虚拟画像：省遥感应用中心业务处处长）。目标：不搞清任何细节，只搞清三件事——业务全景、角色边界、痛点排序。

开场用『一天的工作』提问法：请对方描述一个典型工作日从上班到下班做什么。这个问法比『你们需要什么功能』有效十倍——前者引出真实业务流，后者引出对方脑补的功能清单。样例访谈中，正是从『早上先看出图任务队列』这句描述里，挖出了『解译任务排队与优先级调度』这条招标文件没有明写的隐含需求。

角色识别用『谁碰系统』穷举法：沿业务流逐个环节问『这个环节谁操作、谁审核、谁看结果』，最终识别出五类角色——系统管理员、业务管理员、解译作业员、复核员、浏览者。注意复核员是访谈才挖出来的：招标文件只提『精度评估』，没说评估结果要专人复核。这类角色一旦漏掉，权限设计（FR-10.1）将整体返工。

记录要点：访谈记录当场整理，24 小时内发对方确认。样例记录中每条结论都标注了『原话』与『整理』两栏——原话是证据，整理是加工，两者分开是防止『访谈者把自己的理解当成用户的原话』这一最常见失真。

## **第二次访谈：流程深挖与数据摸清（120 分钟）**

对象：解译作业员与数据管理员（两个角色分开谈，各 60 分钟）。目标：把第一次访谈得到的全景，拆成可写进 SRS 的流程步骤与数据五要素。

流程深挖用『逐步回放法』：请作业员以最近一个真实任务为例，从接到任务单开始，一步一步回放操作，访谈者同步画泳道草图，每画一步与对方核对一次。样例中『大幅面影像上传』环节正是在回放中暴露了断点续传的真实痛点——作业员原话『传了 40 分钟断网，从头再来』，这句话后来直接转化为 FR-01.1 的验收场景（Given 10.2GB 影像上传中断、When 网络恢复、Then 从断点续传且总时长不超过……）。

数据摸清用『五要素卡片』：每谈到一类数据，当场填一张卡片——来源、格式、量级、更新频率、质量要求。影像数据卡片样例：来源=卫星下发+历史归档；格式=GeoTIFF 为主、含 JP2；量级=单景 2~15GB、月均 40 景；更新频率=按拍摄计划不定

期；质量要求=几何校正完成、云量<10%。这张卡片直接成为 SRS 数据需求章节的原材料，也是后续存储容量估算的依据。

禁忌：第二次访谈不要谈界面。作业员描述操作时会自然说到『点这个按钮』，访谈者要记下来但立刻拉回流程层——界面是设计阶段的事，需求阶段谈界面会污染需求的抽象层次。

### **第三次访谈：非功能需求与优先级确认（60 分钟）**

对象：业务主管+技术负责人联合访谈。目标两件事：给非功能需求定量，给功能需求定级。

非功能定量用『场景逼问法』：不问『系统要快吗』，问『月底集中出图那几天，同时会有几个人在线、每人提交多大的任务、能接受等多久』。样例中 PE-06『并发 1+30』就是这样逼出来的：主管先说『几十个人用』，追问到『月底最多 30 个作业员同时提交解译任务、1 个管理员批量下发』，才落成可测试的指标。

优先级定级用『砍掉测试法』：把功能清单逐条问『如果上线那天这个功能没有，业务能不能跑』。能跑→不是 Must；业务停摆→Must。样例中『解译结果人工修正』（FR-07.1）原以为必须是 Must，业务主管的回答是『没有修正功能就先导出到桌面 GIS 改，能跑』——于是定为 Should 之上、Must 之下的边界项，最终因与精度闭环强相关被留在 MVP 外但 RTM 中标注二期。这个裁决过程完整记录在访谈纪要中，是答辩时『优先级依据是什么』的标准答案。

### **访谈产物的转化：从记录到需求**

三次访谈的产物分四层转化：① 原话层→存访谈记录附件，永不修改；② 结论层→整理为《访谈结论清单》，每条结论编号（IV-01~IV-23）；③ 需求层→结论映射为 FR/NFR 条目，映射关系写入 RTM 的『来源』列（招标条款号或 IV 编号）；④ 优先级层→MoSCoW 表，每档附定级理由与定级人。

评审老师抽查转化链的标准问法是：『FR-07.1 这条需求哪来的？』合格回答要能在 30 秒内指出：来源=第二次访谈 IV-11，原话在访谈记录附件第 4 页，MoSCoW 定级为 Should 的理由在纪要第 3.2 节。答不上来，说明访谈记录与需求文档是两张皮——这是本关最常见的隐性失分点。



## 4.3 用户故事、MoSCoW 优先级与人机协同

### 4.3.1 本节学习目标

- 掌握用户故事的标准格式与 INVEST 质量原则；
- 能把访谈痛点转化为可验收的用户故事；
- 掌握 MoSCoW 四级判定规则，特别是“用户期望 vs 合同条款”冲突时的裁决顺序；
- 理解 MVP 边界与用户故事优先级之间的关系。

### 4.3.2 用户故事的标准格式与质量原则

标准格式：作为<角色>，我希望<目标>，以便<价值>。三个槽位各有纪律：角色必须是系统用户角色之一（澜图为五类内置角色），不能写“作为用户”；目标是可验证的功能行为，不是情绪（“希望系统好用”不合格）；价值回答“为什么”，它决定优先级判定的依据。澜图样例 US-09“作为解译作业员，我希望交互分割几秒内出结果，以便思路不被打断”——价值栏“思路不被打断”正是 PE-01 指标（≤3 秒）的人性化注脚。

INVEST 六原则逐条落地检查：Independent（故事间尽量独立）、Negotiable（可协商细节）、Valuable（对用户有价值）、Estimable（可估算工作量）、Small（一个迭代内可完成）、Testable（可验收）。教学中最常违反的是 Testable——“系统要快”不可测，“单轮响应 ≤3 秒”才可测。检查方法：对每个故事问“验收时拿什么证据给它打勾？”

### 4.3.3 从访谈到故事的转化规程

- 第一步（素材编号）：把 4 场访谈的痛点清单统一编号（澜图案例 17 条），每条标注来源场次；
- 第二步（一条痛点至少一个故事）：痛点“全自动结果不能直接用”→ 拆出 US-03（点提示补漏）、US-04（背景点剔除误提）、US-10（图斑编辑）三个故事——痛点到故事允许一对多；
- 第三步（招标条款扫描）：把招标 5.5 节每个 FR 条目与已有故事比对——没有故事支撑的条款，说明访谈没覆盖到（如 FR-10.5 数据备份），补写故事并注明“源自招标条款而非访谈”；

- 第四步（查重合并）：措辞不同但验收等价的故事合并，保留来源最多的编号；
- 第五步（评审）：全组逐条朗读，问两个问题——“这条故事的价值栏能说服谁？”“验收证据是什么？”

澜图案例 21 条故事中，18 条源自访谈、3 条源自招标条款补充——这个比例（访谈为主、条款兜底）是健康的；如果 90% 的故事都来自招标文件，说明访谈白做了。

#### 4.3.4 MoSCoW 四级判定规则

| 级别     | 判定规则（必须同时满足）                            | 澜图样例                  |
|--------|-----------------------------------------|-----------------------|
| Must   | ① 缺失则项目验收不通过（★条款、MVP 项）；或 ② 缺失则核心业务流程断裂 | US-02 全自动分割（FR-02.1★） |
| Should | 重要但不致命：缺失可接受短期变通方案；本期完成                 | US-08 批量任务（FR-05.1）   |
| Could  | 增强体验：有时间则做，砍了不影响验收                      | US-21 资源监控看板          |
| Won't  | 本期明确不做，且书面告知用户/甲方（防止默默消失）               | 模型训练平台                |

两条铁律：其一，Must 级必须有合同或确认书依据——“用户很想要”不构成 Must，澜图案例中 US-19（2 小时部署）正是因为命中招标 5.7.8(1)★ 条款才从 Could 升级为 Must；其二，Won't 必须显式列出并与需求确认书的项目边界（招标 5.3.2）一致——“不做清单”是防止范围蔓延的挡箭牌，澜图 spec.md §4 的 6 项范围外条目全部能对应招标 5.3.2 原文。

#### 4.3.5 冲突裁决顺序

当访谈期望、团队判断、合同条款三者冲突时，裁决顺序固定为：合同条款 > 需求确认书 > 用户期望 > 团队偏好。每一次冲突裁决都要写进纪要（谁提出、何时裁决、依据什么），澜图访谈文档第 5 节的“冲突说明”段就是标准写法。教学评审时会故意抽查：“你们这条 Must 的依据是招标哪一条？”答不出条款号的 Must 一律降级。

#### 4.3.6 与 MVP 的关系

MoSCoW 的 Must 集合与《需求确认书》的 MVP 五项是“交叉验证”关系：Must 集合应包含 MVP 全部条目；Must 集合超出 MVP 的部分（如复核标记 FR-06.5、权限审

计 FR-10.1) 属于“本期必须做但不在 MVP 演示范围”，在项目计划中排入迭代二/三。  
若发现 MVP 条目不在 Must 集合中，说明优先级判定出了逻辑错误——这是评审时的快速检查点。

### 4.3.7 本节小结

用户故事把“人的处境”翻译成“可验收的行为”，MoSCoW 把“都想要”翻译成“先后秩序”。人工方法的要点是：故事要过 INVEST，优先级要有合同依据，冲突裁决要留痕。下一章看 AI 如何给这两步提效，以及为什么画像生成这类环节 AI 只能打下手。

### 4.3.8 访谈与用户故事的人机协同

#### 本节学习目标

- 掌握访谈准备阶段 AI 辅助的三个场景：画像草案、提纲草案、纪要结构化；
- 掌握用 AI 批量改写用户故事并做 INVEST 检查的提示词模式；
- 理解本节边界：访谈实施、痛点判断、优先级裁决必须人工完成；
- 会用“样例对照法”检验 AI 产出质量。

#### AI 辅助的适用边界

| 环节     | AI 角色               | 人工角色         | 原因                        |
|--------|---------------------|--------------|---------------------------|
| 虚拟画像草案 | 生成初稿（岗位事实+痛点假设）     | 对照招标文件修订痛点   | AI 会编造不存在但“合理”的细节，需对照源头删减 |
| 访谈提纲草案 | 按模块生成问题清单           | 按对象定制、删诱导性问题 | AI 提纲偏问卷化，需人工改成开放式        |
| 访谈实施   | 不参与                 | 全程人工         | 访谈的价值在信任与现场观察             |
| 纪要结构化  | 录音稿/笔记→四栏纪要初稿       | 核对原话、发回确认    | 原话一个字都不能让 AI 改            |
| 用户故事改写 | 痛点清单→故事初稿、INVEST 检查 | 逐条裁决价值栏与优先级  | 价值判断需要业务立场                |

| 环节        | AI 角色            | 人工角色    | 原因                   |
|-----------|------------------|---------|----------------------|
| MoSCoW 初排 | 按规则给出建议级别<br>+理由 | 对照★条款终审 | AI 不知道哪些是实质性条款，除非喂给它 |

## 画像与提纲生成（提示词示例）

关键技巧：把招标文件相关章节作为上下文喂给 AI，让画像痛点有出处。提示词骨架：

【角色】你是遥感行业业务分析顾问。  
 【上下文】以下是招标文件的项目背景与用户角色定义章节：<粘贴 5.1、5.1.4 原文>  
 【任务】为本项目构造 4 个虚拟用户画像，覆盖：解译作业员（资深/新人各一）、复核员、系统管理员。每个画像包含：岗位事实（年龄/工龄/日常工具）、工作现状、核心痛点（须能对应背景章节的描述）、一句典型口头禅。  
 【约束】不得编造招标文件之外的业务事实；痛点必须标注对应的背景章节编号；人名用化名。

产出后的人工修订重点：检查每条痛点的“章节编号”是否真实存在（AI 可能编造章节号）；删除过于戏剧化的人设细节；澜图案例中“新人不敢调参数”这条痛点就是人工从第 2 场访谈补充、AI 初稿中没有的——AI 擅长写资深用户，容易忽略新手视角。

## 纪要结构化（提示词示例）

【任务】把以下访谈原始笔记整理为四栏纪要（原话引述/事实/痛点/需求线索）。  
 【铁律】①“原话引述”栏逐字摘录，不得改写、不得补全；②无法归入栏目的内容放入“待核实”；③痛点栏每条给出置信度（原话直接支持/推断）。  
 【输出】Markdown 表格 + 末尾列出你发现的原话与需求线索之间的映射建议。

“原话逐字”是纪要场景的第一铁律：AI 有强烈的润色冲动，会把口语化的关键证据改成书面语。对策是在提示词里明令禁止，并在产出后用原文比对抽查 3 处。

## 用户故事批量改写与 INVEST 检查

把痛点清单喂给 AI 批量生成故事初稿，再让 AI 做一轮自检：

【任务】将下列痛点逐条转为用户故事（作为.../我希望.../以便...），每条标注来源痛点编号。  
 【自检】对每条故事执行 INVEST 检查，重点检查 Testable：给出该故事的验收证据建议；不可测的故事请重写。

澜图案例的 21 条故事用此流程，AI 初稿 26 条，人工合并 3 条重复、降级 2 条不可测（“系统要快”“界面要友好”），最终 21 条。人工终审看两点：价值栏是否成立（“以便提升效率”这类空话打回）、角色栏是否在五类内置角色之内（AI 会发明“数据分析师”之类不存在的角色）。

## MoSCoW 的 AI 初排与人工终审

把故事清单 + 招标★条款清单一起喂给 AI，要求“每条给出级别建议 + 判定依据 + 命中条款号”。AI 初排的价值在于迫使团队讨论“依据”而非“感觉”；但终审必须人工对照★条款原件——AI 可能把一般条款错当★条款，或漏掉某个★条款导致 Must 集合不完整。澜图案例中 AI 初排把 US-19（部署时限）排为 Could，人工对照 5.7.8(1) 后升级 Must——这个“AI 初排 + 条款终审”的组合拳就是本关的标准动作。

## 样例对照法

本课程提供的澜图样例（《用户访谈记录与 MoSCoW 优先级》样例）不是用来看的，是用来“对答案”的：小组完成自己的访谈文档后，与样例逐项对比——画像有没有“四条标准”、故事有没有“来源访谈”列、MoSCoW 表有没有“判定依据”列、有没有冲突说明段。结构缺项一目了然。但注意只对结构、不对内容：照抄样例的业务内容等于放弃了自己题目的需求真实性，评审时会被追问击穿。

## 本节小结

访谈环节的人机分工一句话：AI 负责“从文字到文字”的加工（画像草案、纪要结构化、故事改写、优先级初排），人负责“从现实到文字”的获取（访谈、观察）和“从文字到责任”的判定（确认、裁决、签署）。下一章进入 SRS 编写的人工方法。

## 4.4 SRS 编写：国标骨架与章节规划

### 4.4.1 本节学习目标

- 理解 GB/T 9385-2008 对 SRS 的核心要求：正确、无歧义、完整、一致、可验证、可追踪；
- 掌握 SRS 标准骨架各章的写作职责与常见内容；
- 学会从招标文件第五章直接派生 SRS 骨架与功能清单；
- 掌握文档控制页（变更记录、签署）的规范写法。

### 4.4.2 GB/T 9385 的六性要求

9385 的灵魂是六性，每一性都对应可操作的检查动作：

| 特性  | 含义           | 检查动作（本课程做法）                |
|-----|--------------|----------------------------|
| 正确  | 需求真实反映用户需要   | 访谈溯源：每条需求能指出访谈或条款出处        |
| 无歧义 | 一种解释         | AI 反向提问 ≥10 条（第九章）         |
| 完整  | 覆盖全部需求与约束    | 功能清单 100% 覆盖招标条目（脚本核对）     |
| 一致  | 内部不矛盾、与上游一致  | 跨文档指标一致性体检（24 项检查）         |
| 可验证 | 存在有限成本内的判定方法 | 每条 [M] 需求带 Given/When/Then |
| 可追踪 | 双向可溯         | RTM：条款↔需求↔AC↔用例            |

教学评审时，六性各有一个“一票否决”级问题：编号自编造（不可追踪）、指标与招标不符（不正确/不一致）、“系统要快”类表述（不可验证）、两个章节同一指标两个值（不一致）、漏★条款（不完整）、“支持多种格式”不列举（有歧义）。这六问应在小组自审时逐条过。

#### 4.4.3 SRS 标准骨架

澜图样例 SRS 的骨架（可复用，但内容必须重写为你自己的项目）：

- 文档信息：文档控制表（编号/密级/依据/配套文件/三级签署）+变更记录表（版本/日期/变更类型 A·M·D/摘要）；
- 第 1 章 引言：目的、读者对象及阅读建议、文档范围、功能清单及重要程度说明、参考资料、名词解释；
- 第 2 章 综合描述：产品目标与背景、系统上下文（图）、系统用户、运行环境、约束与假设；
- 第 3 章 业务流程：总体流程 + 关键环节流程图；
- 第 4 章 功能需求：按模块分节，条目编号直引招标；
- 第 5 章 数据需求：数据流图、数据字典、数据规模估算；
- 第 6 章 接口需求：外部/内部/开放接口；
- 第 7 章 非功能性需求：性能、精度、安全、可靠、维护、易用；

- 第8章 验收标准与需求追踪：验收分类表 + 指向 RTM；
- 附录：条目对照说明、AI 辅助编制说明。

#### 4.4.4 骨架派生法：从招标文件到 SRS 目录

实操步骤：① 把招标 5.5 节的 F-01~F-10 模块标题原样复制为 SRS 第 4 章的节标题；② 把每个模块下的 FR 条目号原样复制为该节下的子条目标题——此时你就拥有了一个“覆盖率天然 100%”的骨架；③ 对照招标 5.6/5.7/5.8 节标题搭建 SRS 第 5/6/7 章；④ 再补引言与综合描述。澜图 V1.0 的教训正是跳过了第 ①② 步、凭理解自写了模块划分，导致编号体系全面错位——骨架派生法用 10 分钟的机械劳动杜绝这类事故。

#### 4.4.5 功能清单表的编写

“功能清单及重要程度说明”表是评审专家翻阅频率最高的一页，它把 55 个条目的覆盖承诺压缩在一屏之内。列设计：序号、功能模块、需求条目（招标编号）、功能名称、重要程度（AAA/AA/A）、是否 MVP。重要程度的判定规则必须在表前用文字写明（AAA=★条款或 MVP 核心；AA=招标“应”条款；A=“宜/可”条款），否则 AAA 泛滥等于没有分级。澜图样例 55 行中 AAA 13 行——与★条款数加 MVP 补充项吻合，这是评审时的快速勾稽点。

#### 4.4.6 文档控制与变更记录

- 变更类型使用 A（增加）/M（修改）/D（删除）三字母标记，源自参考 SRS 的惯例；
- 每次过关评审冻结一个版本号，评审意见对应的修订在下一版的变更记录中可溯；
- 签署栏三级：编制/校核/审核，教学场景由组长、技术负责人、教师分别签署；
- 密级标注“内部（教学样例）”，地各单位化名——合规意识从教学样例养成。

#### 4.4.7 本节小结

SRS 骨架是“从招标文件机械派生”的，不是“凭理解创作”的——这是本节最重要的一句话。骨架对了，内容差可以迭代；骨架错了（编号错位、章节缺失），内容越好返工越痛。下一章详解功能需求章节的六要素写法。

### 4.5 功能需求六要素：写法与三轮迭代实战

### 4.5.1 本节学习目标

- 掌握“功能描述/功能界面/功能数据/功能操作/业务规则与算法/测试要点”六要素的写法；
- 能判断哪些子功能需要六要素详写、哪些用一览表覆盖；
- 掌握业务规则与算法的分层表述（规则管判定、算法管计算）；
- 会写面向测试人员的“测试要点”。

### 4.5.2 六要素的来源与价值

六要素模式来自行业真实 SRS 实践（本课程参考了某科研管理系统的真实 SRS v5.5）：它迫使编写者回答六个无法回避的问题——做什么（描述）、在哪做（界面）、进出什么数据（数据）、用户能点哪些按钮（操作）、边界与判定逻辑（规则/算法）、怎么算做对（测试要点）。只写“功能描述”的需求文档，在开发阶段会被追问出一百个问题；六要素把这一百个问题前置到需求阶段回答。

### 4.5.3 六要素逐个写法（以澜图 FR-02.1 为例）

#### **功能描述**

一段话说清“谁、对什么、做什么、产出什么”。澜图 FR-02.1：“作业员对选定影像执行全自动分割，输出覆盖全部显著目标的掩膜集合；粒度参数可调（采样点密度、掩膜质量阈值、稳定性阈值、最小图斑面积），每个参数提供业务含义说明与推荐值。”注意参数说明直接照录招标 FR-02.1 的枚举——原文照录是防止“转述失真”的基本功。

#### **功能界面**

列页面清单而非画界面：每个界面一句话说清用途与关键元素。如“新建任务向导页：选择影像/范围 → 选择模型与参数方案 → 参数说明悬浮提示 → 提交”。界面清单是第 3 关原型设计的直接输入。

#### **功能数据**

写输入/输出/查询条件三类，并指向数据字典表号（“写入 D2 任务库，字段见表 5-2”）。需求级数据只到“有什么字段”，物理类型留给设计阶段。

#### **功能操作**

按【操作名】格式逐条列举用户可执行的动作，每个操作一句话。操作清单是测



试用例与权限矩阵的共同来源——FR-10.1 的“权限按功能模块粒度分配”，粒度就落到操作清单上。

**业务规则/算法**

规则与算法分行写：规则以“规则 N：”开头，是可判定的约束（“单任务幅面 >100km² 强制拆分”）；算法以“算法 N：”开头，是计算方法（“重叠区按距离加权融合”）。规则的编号会在澄清记录和测试用例中被引用（如“CL-01 裁决写入规则 2”），不要随意重排。

**测试要点**

每条测试要点对应 spec.md 的一条 AC 或一个业务规则，写成可执行的场景（“构造跨块大型建筑物样本，验证边界连续无错位”）。禁忌写“测试功能是否正常”这类无信息量语句。

**4.5.4 详写与简写的取舍**

55 个条目全部六要素展开会让 SRS 膨胀到无法维护。取舍规则：满足任一条件即详写——① ★实质性条款；② MVP 条目；③ 高风险条目（性能敏感、算法复杂、曾有澄清裁决）；④ 评审组指定。其余条目进“本模块其余子功能要求一览”表（条目/名称/要求要点/测试要点四列）。澜图样例详写 14 条、一览表覆盖 41 条。注意：一览表也必须给出测试要点——覆盖率 100% 的要求（5.7.1(1)）不允许任何条目没有验收路径。

**4.5.5 写作禁忌清单**

| 禁忌              | 反例               | 正例                                                       |
|-----------------|------------------|----------------------------------------------------------|
| 模糊副词            | “系统应快速响应”        | “单轮响应 ≤3 秒（PE-01）”                                       |
| 开放式枚举           | “支持多种格式”         | “ <div>支</div> 持<br>GeoJSON/Shapefile/GeoPackage<br>三格式” |
| 需求里写方案          | “采用 YOLOv8 实现检测” | “文本—检测—分割两级链路<br>（选型留设计）”                                |
| 复制招标原文不加工<br>程化 | 只有“应支持批量处理”      | 补操作、数据、规则、测试要点                                           |

| 禁忌      | 反例              | 正例                        |
|---------|-----------------|---------------------------|
| 功能间隐式依赖 | FR-03 依赖缓存但全文不提 | FR-03.1 规则 1 显式声明缓存机制     |
| 测试要点不可测 | “测试分割效果好不好”     | “冻结集上建筑物完整率 $\geq 80\%$ ” |

## 4.5.6 本节小结

六要素是把“一句需求”展开成“一页契约”的模具。详写抓关键（★/MVP/高风险），简写保覆盖（一览表），每条都有验收路径。下一章讲图与数据——流程图、数据流图、数据字典，这三样是评审专家判断“这份 SRS 是不是练过”的第一眼依据。

## 4.5.7 六要素实战：一个条目的三轮迭代

### 本节学习目标

- 通过 FR-02.1（自动分割）从“差初稿”到“合格条目”的三轮迭代，掌握六要素的打磨过程；
- 学会用评审视角给需求条目找茬；
- 理解每一轮修订的触发源（自查/AI 反向提问/对照招标原文）。

### 第零轮：典型的差初稿（AI 直出，未审定）

以下是 AI 在未给定招标文件上下文时生成的 FR-02.1 初稿（教学中可作为“找茬练习”材料）：

“功能描述：系统应支持对遥感影像进行全自动智能分割，快速准确地提取各类地物目标，输出高质量的分割结果。功能界面：分割功能页面。功能操作：【开始分割】启动分割。测试要点：测试分割功能是否正常。”

用六性检查这份初稿，至少七个问题：①“快速准确”不可测（无指标）；②“各类地物”开放枚举（哪些类？）；③“高质量”无定义；④界面只有一个“页面”（向导步骤缺失）；⑤无功能数据（输入输出不明）；⑥无业务规则（什么影像允许分割？失败怎么办？）；⑦测试要点不可执行。这份初稿如果进入开发，开发会自由发挥，验收时必然扯皮。

### 第一轮：对照招标原文修订（触发源：人工自查）

把招标 FR-02.1 原文摆到旁边逐句对照，补入：选定影像或选定范围两种作用域；

输出“覆盖全部显著目标的掩膜集合”；粒度参数的四个枚举（采样点密度、掩膜质量阈值、稳定性阈值、最小图斑面积）照录；参数要有业务含义说明与推荐值。修订要点：

- 原文的每个“应”字都要落到六要素的某一格里，落不进去的就说明格子缺了（本条“参数说明与推荐值”落进了功能界面的“悬浮提示”和业务规则的“参数说明可见”两处）；
- 招标没写的东西（如具体阈值数值）不编造，写“默认 0.30（原型标定，可调）”并注明出处是标定而非招标；
- 性能口径此时不写死，只在测试要点引用“PE-02”——指标的唯一权威在 spec.md §2 表。

## **第二轮：反向提问触发修订（触发源：AI 澄清 CL-01）**

AI 反向提问：“对多大幅面的影像必须可用？1km<sup>2</sup> 的耗时指标是否意味着整景也承诺？”裁决：单任务幅面上限 100km<sup>2</sup>，超出强制拆分。于是业务规则新增“规则 2：单任务幅面 >100km<sup>2</sup> 强制拆分并提示分批”，测试要点新增“>100km<sup>2</sup> 范围提交验证拆分提示（AC-FR-02.1-c）”。这一轮的价值：歧义在进入开发前被定价——如果没有这条规则，开发会按“任意幅面”实现，验收时按“1km<sup>2</sup>”测量，双方对“合不合格”各执一词。

## **第三轮：一致性审核触发修订（触发源：V1.0→V2.0 审核）**

V2.0 审核发现该条目挂在错误模块下（V1.0 自编“FR-02 全自动分割”在招标里编号碰巧一致，但模块内的条目边界不同——招标把“分块处理”独立为 FR-02.2，V1.0 把它揉在 FR-02.1 的规则里）。修订：分块内容拆出到 FR-02.1 的“算法 1”只做引用，主体移到 FR-02.2 条目；RTM 同步拆行。教训：即使模块号碰巧一致，条目边界也必须与原文逐条对齐。

## **合格条目的验收清单**

- 六要素齐全且每格有实质内容；招标原文的“应”全部落格；
- 指标只引用编号（PE-xx），数值唯一权威在 spec.md；
- 规则有编号且被澄清记录/测试要点引用时可定位；

- 测试要点逐条可执行（有构造数据的方法、有判定标准）；
- 与 RTM 行、spec.md 条目三方一致。

## 4.6 流程图、数据流图与数据字典

### 4.6.1 本节学习目标

- 掌握 SRS 必备四张图：系统上下文图、业务流程图、功能流程图、数据流图；
- 掌握流程图绘制规范（开始/结束、处理、判定、流向）；
- 掌握数据字典七列格式与需求级数据建模的分寸；
- 理解“图与文必须互相对得上”的一致性要求。

### 4.6.2 四张必备图及其职责

| 图                | 回答的问题           | 所在章节    | 澜图样例          |
|------------------|-----------------|---------|---------------|
| 系统上下文图           | 系统边界在哪、与谁交互     | 2.2     | 图 2-1         |
| 总体业务流程图<br>(泳道)  | 业务怎么流转、谁负责哪段    | 3.1     | 图 3-1         |
| 功能流程图（含判定分支）     | 单个功能内部的正常/异常路径  | 3.2-3.4 | 图 3-2/3-3/3-4 |
| 0 层数据流图<br>(DFD) | 数据从哪来、经哪些加工、存到哪 | 5.1     | 图 5-1         |

评审专家的经验是：没有流程图的 SRS，业务流程章节等于没写；没有数据流图的数据需求，数据字典就是无源之水。四张图是底线，不是加分项。

### 4.6.3 流程图绘制规范

- 符号约定：圆角矩形=开始/结束，矩形=处理步骤，菱形（或异色矩形）=判定，平行四边形（或异色矩形）=输入/输出/异常处理；全班统一一套图例；
- 流向规则：主流程自上而下或自左而右；判定分支必须标注“是/否”等分支条件；异常分支必须画归宿（退回重试、记录日志、转人工），不允许画“断头路”；

- 粒度控制：一张图 10–15 个节点为宜；超过则拆分层（总体图 + 子流程图）；澜图样例把全自动分割（3-2）、交互分割（3-3）、复核归档（3-4）拆成三张就是这个道理；
- 图文互查：图中每个处理节点必须在正文中有对应描述；正文提到的异常分支必须在图中有对应路径。互查不一致是评审高频扣分点；
- 工具：Visio/drawio/ProcessOn 均可；本课程样例用 matplotlib 脚本绘制（图随代码版本管理、可重复生成），学生可用任何工具，但要求导出矢量或高分辨率位图、中文字体清晰。

#### 4.6.4 泳道图的画法

涉及多角色协作的流程（如澜图五阶段流程）用泳道图：一列一个角色，流程节点落在操作者泳道内，跨泳道箭头即交接点。泳道图的特殊价值是暴露“责任真空”——画完检查每个阶段是否都有角色认领；澜图样例中“成果矢量化”阶段最初没有复核员的节点，补画时发现“属性检查”无人负责，随即补入。

#### 4.6.5 数据流图（DFD）画法

0 层 DFD 只有三类元素：外部实体（矩形）、加工（编号 1.0/2.0...的圆角框）、数据存储（D1/D2...开口框或异色框）。绘制步骤：① 先摆外部实体（五类角色 + 外部系统）；② 从 SRS 功能模块归并出 4–6 个加工；③ 从数据字典的实体清单摆出数据存储；④ 拉数据流并命名（名词性短语，如“质检合格影像”而非“传输”）。检查规则：每个加工都有进有出；每个存储至少被一个加工读、一个加工写；对外数据流逐条对照安全红线（澜图样例对外仅两路：成果共享、日志调阅——正好与 5.6.3 红线呼应，这就是用图做安全自证的例子）。

#### 4.6.6 数据字典七列格式

沿用参考 SRS 的七列：对象名称、类型、宽度、取值范围、数据来源、空否、备注。各列纪律：

- 对象名称：中文业务名，与正文术语一致（“结果集”不写成“成果集”混用——术语表先行）；
- 类型/宽度：需求级只到“字符/数值/日期时间/几何/文本”粒度，宽度给上限即可；物理设计留给第 3 关；

- 取值范围：枚举必须列全（任务状态 1=排队...5=取消），枚举值与功能章节的规则文字一致；
- 数据来源：输入/自动/选择/计算四选一并写清来源字段（如“自动（SHA-256）”）；
- 备注：放业务解释与跨条目挂接（“支持单结果集  $\geq 50$  万图斑，对应 5.6.2”）。

表的数量：每个核心实体一张表。澜图样例 8 张（影像、任务、提示、结果集、图斑、复核记录、用户、操作日志），覆盖四大数据存储 D1-D4。检查方法：DFD 上的每个数据存储至少对应一张字典表；每张字典表能说出它被哪些功能读写。

### 4.6.7 数据规模估算

数据规模必须照录招标 5.6.2 的口径（试运行  $\geq 50$  景、容量  $\geq 500$  景/3TB、单结果集  $\geq 50$  万图斑、峰值 30 并发），再推导存储规划。禁忌拍脑袋写“预计数据量很大”——规模数字直接决定硬件规划与 PE 指标的合理性，是招标方给定的约束，不是估算的素材。

### 4.6.8 本节小结

图与数据是 SRS 的硬骨架：流程图证伪流程，DFD 证伪边界，数据字典证伪“数据从哪来”。三者共同的验收标准是“图文一致、图表互查、规模有据”。下一章进入本关的方法论核心——spec.md 与 Spec-Driven 开发。

### 4.6.9 数据字典实战：从 0 到 8 张表

#### 推导路径

数据字典不是拍脑袋列的，是推出来的。澜图案例的推导路径：① 从 DFD 的四个数据存储（D1 影像、D2 任务与提示、D3 成果、D4 用户与日志）得到实体大类；② 从功能条目的“功能数据”格收集字段需求（如 FR-02.4“记录所用模型、参数、执行人、执行时间”直接给出结果集表的四个字段）；③ 从非功能条款倒推（FR-10.3 日志“操作人、时间、IP、对象、结果”→ 操作日志表五列）；④ 从验收口径倒推（面积误差  $\leq 3\%$  → 图斑表必须有“面积”字段且标注“编辑保存即重算”）。每张表都能说出“哪几条需求喂出了我”——这是评审时证明数据字典不是摆设的关键。

#### 逐表讲解要点

- 影像表：16 个字段围绕“接入（FR-01.1/01.2）—质检（01.2）—检索（01.3）—坐标（01.4）”四条需求链组织；“质检状态”枚举值 4 个（含“待配准”）是 CL 裁决与坐标处理需求的汇合点；
- 任务表：状态枚举 5 值（排队/运行/成功/失败/取消）与 FR-05.3 文字一一对应；“发起人”字段备注“API 任务记 Token 主体”——接口需求（FR-09）在数据层的投影；
- 结果集表：“时序分组”字段服务 FR-02.6；“状态”枚举（编辑中/待复核/已锁定/归档/退回）与图 3-4 复核流程的状态迁移一一对应；
- 图斑要素表：“复核标记”字段是 FR-06.5 的落点；“编辑历史”是 FR-06.3 留痕的落点；
- 用户表与日志表：字段基本由 FR-10.1/10.2/10.3 原文转写，改造空间最小、一致性要求最高。

## 需求级与物理级的分寸

学生最容易越界写成物理设计（VARCHAR(64)、索引、分区）。分寸线：需求级回答“有什么数据、什么含义、什么约束”；物理级回答“怎么存、怎么索引、怎么优化”。数据字典里出现索引设计、分库分表、缓存策略即越界——那是第 3 关设计文档的内容。反向也越界不得：字段含义不清（如只写“状态”不写枚举值）则欠火候。检查口诀：“类型到类不到型，约束到值不到存”。

## 数据规模与字段设计的联动

招标 5.6.2 给了三个硬数字（500 景/3TB、单结果集 50 万图斑、峰值 30 并发），它们要回写到字典备注里（如结果集表“图斑数量”字段备注“支持  $\geq 50$  万”），并在第 7 章性能条目中被引用。这样“规模—设计—验收”就闭环了：验收时测 50 万图斑加载（FR-06.1 的 PE-08、10 万图斑导出  $\leq 2$  分钟）正是从这条规模要求推导出来的。

## 本节小结

数据字典的价值不在表多，而在“每表有出处、每字段有需求、每规模有回写”。第 3 关数据库设计时，本节的 8 张需求级表将直接映射为物理表设计的输入清单。

## 4.7 SRS 逐节写作示范

第 5 章（功能需求）的写作已在前文六要素与数据需求章节详述，本节示范 SRS 前四章——这些章节常被学生当作『套话区』草草应付，实际上它们是评审老师判断需求理解深度的第一窗口。

#### 4.7.1 第 1 章 引言：四小节各有判分点

- 1.1 编写目的：一句话说清『本文档为谁写、用来干什么』。样例写法：『本文档规定澜图遥感影像智能解译平台的功能与性能需求，作为设计、开发、测试与验收的共同依据，读者为项目开发组、评审专家组与建设单位业务代表。』  
判分点：读者列举完整、用途与验收挂钩。
- 1.2 项目背景：三段式——业务痛点（当前人工解译一景影像平均 3 人日）、建设目标（解译效率提升一个数量级）、项目来源（招标文件编号与需求确认书编号）。禁止写『随着信息技术的发展』式空话开头。
- 1.3 术语定义：只定义本文档用到的、读者可能误解的术语。样例定义了 12 个术语，其中『解译』『瓦片』『断点续传』三个是答辩高频提问点。每个术语附英文与一句话定义，不抄词典。
- 1.4 参考资料：招标文件、需求确认书、GB/T 9385-2008、访谈记录，四项必列，各带版本号与日期。参考资料没有版本号是评审常见扣分点——引用的招标文件若与最终版不一致，需求基线就悬空了。

#### 4.7.2 第 2 章 总体描述：最容易写水的章节，恰恰最见功力

2.1 产品概述：用一张系统上下文图（本关样例 SRS 图 2-1）说清系统边界——系统在内，外部实体（卫星数据下发系统、桌面 GIS、用户）在外，箭头标注数据流向。上下文图是整份 SRS 最重要的一张图：它定义了『什么不归我们管』。样例中明确『卫星数据下发系统只提供下载接口，本平台只下载不上传』（5.6.3(2) 红线的源头），一句话挡住了整个一类需求蔓延。

2.2 用户特征：五类角色各写三行——职责、技术水平、使用频率。技术水平决定界面假设，使用频率决定性能假设。样例中浏览者『偶尔使用、无专业背景』直接导出『浏览界面零培训可用』这条可用性需求。

2.3 约束：照录招标约束并分类——技术约束（国产化部署环境）、进度约束（一个月工期）、合规约束（数据只下载不上传、日志留存 $\geq 180$  天）。每条约束注明出处。



2.4 假设与依赖：写明『GPU 资源由建设单位提供』这类影响方案成立性的前提——假设一旦不成立，对应需求自动触发变更流程，这是给自己留的合规退路。

### 4.7.3 第 3 章 数据需求与第 4 章 接口需求

第 3 章按四类数据（影像、样本、模型、成果）逐类展开五要素，写作模板与推导过程见 4.6 节，此处只强调一个常见错误：学生常把『数据库表设计』写进数据需求章节。数据需求说的是『业务上有什么数据、什么质量要求』，表结构是设计阶段的产物，提前写进 SRS 属于需求越界，评审时会问『你这是需求还是设计』。

第 4 章接口需求逐个写：接口名称、协议、数据格式、调用方向、频率、异常处理。样例的卫星数据下载接口写法：『协议 HTTPS+REST，格式 GeoTIFF+元数据 JSON，方向仅下载，频率按拍摄计划（月均 40 景），异常处理为断点续传+三次重试后告警』。六个字段缺一不可。内部模块间接口不在本节写——那是概要设计的内容。

### 4.7.4 前四章的 AI 协同要点

前四章的叙述性内容适合 AI 起草：把招标文件的对应章节、访谈结论清单、本节写作要点三段材料喂给 AI，提示词模板见附录 B 第 7 卡。但三个地方必须人工写：① 系统上下文图的边界判定（AI 不知道甲方口头交代的红线）；② 假设与依赖（AI 不知道项目真实约束）；③ 每条约束的出处标注（AI 会编造条款号，必须人工核对招标原文）。审定方法：AI 起草后，用『五问法』逐段过一遍，凡是含数字的句子必须能在招标文件或访谈记录中找到出处，找不到就删掉或标注待澄清。

### 4.7.5 SRS 第 5 章逐域写作示范：从招标条款到 FR 组

SRS 第 5 章功能需求是全文的主体（约占 60% 篇幅），也是评审逐条核对的对象。本节以本关样例为蓝本，示范十大功能域（F-01~F-10）各自如何从招标条款展开为一组 FR。每个功能域按『招标原文要点→拆分逻辑→FR 组→易错点』四步讲解。

#### F-01 影像数据管理（FR-01.1~01.6）

招标原文要点（5.4.2★）：支持大幅面遥感影像上传、管理、浏览，单景≥10GB。拆分逻辑：上传（01.1 分片断点续传）、入库（01.2 瓦片合成）、管理（01.3 元数据检索）、浏览（01.4 在线金字塔浏览）、删除（01.5 归档与删除审批）、统计（01.6 存储用量统计）。六条 FR 覆盖『上传—入库—用—管—退』全生命周期——这是数据类功能域的标准拆法，学生组常只写上传和浏览两条，把管理与退出环节漏掉。

易错点：① FR-01.1 的『≥10GB』必须写进描述正文而不是只写『大幅面』；② FR-01.2 备注挂『只下载不上传』红线；③ FR-01.4 的浏览性能与 PE-01 交叉引用，不要在 FR 里重复写性能数字（一处维护原则）。

## **F-02 交互式智能标注 (FR-02.1~02.4)**

招标原文要点（5.4.4★）：点选、框选两种交互方式，实时反馈分割结果。拆分逻辑：02.1 点选标注、02.2 框选标注、02.3 标注结果修正（增删改）、02.4 标注历史与撤销。四条 FR 按『交互方式×操作维度』正交拆分——两种交互方式各一条主 FR，修正与历史是横切能力各一条。

易错点：CL-08 的教训就在这里——AI 曾把『实时反馈』误读为『自动标注无需人工』，人工裁决驳回。本域的核心是『人在环中』（human-in-the-loop），SRS 描述必须写清『用户给出提示点/框→系统返回分割掩膜→用户确认或修正』的交互环，任何让 AI 全自动的描述都违背招标本意。

## **F-03 样本库管理 (FR-03.1~03.4)**

招标原文要点（5.4.5）：样本的创建、分类、版本管理、质量检查。拆分逻辑：03.1 样本集创建与分类、03.2 标注质量检查、03.3 样本版本管理、03.4 样本导入导出。其中 03.3 是 CL-04 澄清的产物——招标只提『版本管理』四个字，澄清后明确『版本号+变更说明+可回滚』三要素写进 FR 描述。

易错点：样本版本与模型版本的关联（训练用哪个版本的样本）属 F-04 域，不要在本域展开——域间引用用『见 FR-04.2』一笔带过，防止两处描述漂移。

## **F-04 模型训练管理 (FR-04.1~04.4)**

招标原文要点（5.4.6★）：训练任务提交、参数配置、过程监控、模型登记。拆分逻辑：04.1 任务提交（MVP）、04.2 训练配置与样本版本绑定、04.3 过程监控与日志、04.4 训练完成模型自动登记入库。GPU 排队策略经 CL-07 澄清写入 04.1：『资源不足时按提交时间排队，管理员可调整优先级』。

易错点：训练框架选型（PyTorch 等）是设计决策不是需求，SRS 只写『支持主流深度学习框架的模型封装格式』，把选型留给任务 3。需求写死技术栈是需求越界，评审必问。

## ***F-05 模型管理与F-06 智能解译 (FR-05.x/FR-06.x)***

F-05 模型管理：05.1 模型登记、05.2 版本对比、05.3 发布与下架。注意 FR-05.1 是 V1.0 遗漏、评审补入的——招标 5.4.6 末尾『训练完成的模型纳入统一管理』一句容易被忽略，漏了它就是覆盖性缺陷。教训：逐句过招标原文，每个句号都是潜在的 FR。

F-06 智能解译（核心功能域，5.4.8★）：06.1 解译任务创建与执行（MVP）、06.2 批量解译、06.3 解译进度监控、06.4 结果预览。06.1 是全书最重要的一条 FR：描述含输入（影像+模型版本）、处理（自动分割+矢量化）、输出（解译成果包）三要素，验收 AC-06.1 绑定 PE-02（1km<sup>2</sup>≤10 分钟）性能场景。

## ***F-07 结果修正与F-08 成果管理 (FR-07.x/FR-08.x)***

F-07：07.1 人工修正（访谈 IV-11 新增）、07.2 修正留痕与对比。本域示范『访谈需求怎么写得与招标需求同等严谨』——07.1 虽无招标条款直接对应，仍按同样格式写描述与 AC，来源列标 IV-11，并在 5.7 节说明其与精度评估闭环的关系。

F-08（5.4.9★）：08.1 成果导出（GeoTIFF/Shapefile/GeoJSON 穷举，CL-02 确认）、08.2 成果元数据与 lineage（哪个模型、哪个版本、哪景影像解译的）、08.3 成果审核发布。lineage（数据血缘）是遥感行业的硬要求，招标没明写但隐含在『可追溯』表述中——这类行业常识需求，访谈确认后写入，来源列标『5.4.9+IV-15』。

## ***F-09 日志审计与F-10 系统管理 (FR-09.x/FR-10.x)***

F-09（5.4.10★）：09.1 操作日志记录（留存≥180 天，正偏离备注）、09.2 日志检索与导出、09.3 审计报表。日志类 FR 的描述必须写清『记录什么』——样例列出登录登出、数据上传下载、任务提交、成果导出、权限变更五类必记事件，这个清单来自合规性访谈，不是拍脑袋。

F-10（5.4.11★）：10.1 权限分级管理（五类角色矩阵）、10.2 用户与组织管理、10.3 系统参数配置。10.1 只做索引指向 5.3 节权限矩阵表——矩阵有 5 角色×12 操作的 60 个格子，单独成节，FR 里写矩阵会变成无法维护的巨长描述。

## ***逐域写作的通用检查***

每写完一个功能域，做四项自查：① 招标原文逐句核对——每句话都有 FR 落点或书面说明（如『此句为描述性语句不构成需求』）；② 生命周期核对——数据类功

能域查『增删改查用退』六环节是否齐全；③ 边界核对——本域 FR 是否越界写了别域的事、是否写成了设计；④ 星标核对——本域★款是否照录、编号是否进 RTM。四项自查记录写入《SRS 编写工作单》，作为过程证据归档。

## 4.8 spec.md：机器可读规格与结构化内容协同生产

### 4.8.1 本节学习目标

- 理解 Spec-Driven Development 的理念及其在需求关的形态；
- 掌握 spec.md 的文件结构：元信息、需求条目、验收标准、范围外、变更记录；
- 掌握 Given/When/Then 验收标准的写法与 [STAR] 标记纪律；
- 理解 spec.md 与 SRS 的双轨制：谁是源头、如何同步。

### 4.8.2 为什么 SRS 之外还要 spec.md

SRS 是写给人读的：叙述充分、格式规范，但对 AI 不友好——段落里的需求无法被程序检索，验收口径埋在长句里无法被执行。而后续关卡中，AI 要依据需求生成设计、代码与测试，它需要的是结构化、无歧义、带验收判据的输入。GitHub Spec Kit 的实践给出的答案是：维护一份机器可读的 spec 作为唯一源头（Single Source of Truth），人类文档由它派生。本课程的双轨制：spec.md 管“准”，SRS 管“读”。

澜图案例的实证：V1.0→V2.0 修订只改了 spec.md 一处源头，SRS 第 4 章的条目表述、RTM 的映射行都随之机械更新——如果没有 spec 源头，同样的修订要在三份 docx 里人肉同步，漏改一处就是新的不一致。

### 4.8.3 spec.md 文件结构

- §0 元信息：版本、编码规则、优先级定义、MVP 范围、招标人——读文件先读元信息；
- §1 功能需求：按招标模块分节（F-01...F-10），每条 FR 带优先级标记 [M]/[S]/[C]/[W] 与一句话定义；
- AC 验收标准：关键条目下挂 AC-FR-xx.y-z 编号条目，Given/When/Then 三段式；
- §2 非功能需求：PE/SEC/REL/MTN/OSS/DOC 分族，指标表照录招标原值；

- §3 验收口径：精度评定方法等跨条目的判定规则；
- §4 范围外（Won't）：显式不做清单，与招标项目边界对应；
- §5 变更记录：每次升版写清依据（哪份裁决、哪次评审）。

#### 4.8.4 Given/When/Then 验收标准写法

GWT 三段式的纪律：Given 写前置条件（环境、数据、状态），When 写单一动作，Then 写可观测、可判定的结果。对比示例：

| 写法 | 示例                                                                             | 判定                 |
|----|--------------------------------------------------------------------------------|--------------------|
| 坏  | When 用户上传影像，Then 系统正常处理                                                        | 不可判定：“正常”无定义       |
| 好  | Given 一幅 10GB CGCS2000 GeoTIFF，When 分片上传（模拟 2 次断点续传），Then 上传完成、完整性校验通过、元数据完整提取 | 可判定：完成/校验/元数据三个检查点 |

三条写作技巧：① Given 里写清数据规格（“1km<sup>2</sup>、0.5m 分辨率”），它是性能指标可复测的前提；② Then 里尽量出现招标原文的数字（≤3 秒、≥80%），数字照录不转述；③ 一个 AC 只测一件事，出现“并且”就考虑拆分。

#### 4.8.5 [STAR] 标记纪律

凡对应招标★实质性条款的 AC，打 [STAR] 标记并注明实测主体（第三方/招标人专家组）。这条纪律在验收时兑现为一句话：带 [STAR] 的条目不接受开发方自测报告。RTM 的非功能表中“判定主体”列即由此派生。教学评审抽查法：随机抽一个 [STAR]，问“验收时谁来测、拿什么数据测、不过怎么办”，三个问题都答得出才算合格。

#### 4.8.6 双轨同步规则

- 改需求先改 spec.md，走变更评审，再同步 SRS 与 RTM——方向不可逆；
- SRS 的叙述可以更丰富（背景、理由、示例），但条目编号、指标、判定口径不得超出 spec.md；
- SRS 附录 A 维护“对照说明”：哪些内容只在 spec.md（AC 全文）、哪些只在 SRS（数据字典、流程图）；
- 每次同步升版后跑一次一致性体检脚本（第十一章），报告归档到配置库。

4.8.7 常见错误

| 错误            | 后果              | 纠偏                   |
|---------------|-----------------|----------------------|
| spec.md 自编号体系 | 与招标文件脱钩，RTM 失效  | 编号直引招标 5.5 节         |
| AC 里写实现方案     | 锁死设计空间，变更困难     | AC 只写可观测结果           |
| 指标四舍五入        | “≥80%”写成“约 80%” | 数字照录原文               |
| 范围外不写         | 验收时被追加需求        | §4 显式列 Won't 并对应招标边界 |
| 改 SRS 不改 spec | 双轨漂移            | 源头纪律 + 体检脚本          |

4.8.8 本节小结

spec.md 是本关对后续所有关卡的最大杠杆：它让 AI 在第 3、4 关的生成有了精确的靶子，让验收在第 5 关有了机械判定的依据。写法上就三件事：编号直引、GWT 可判定、[STAR] 标主体。下一章讲 spec.md 的人机协同生产流程。

4.8.9 spec.md 与结构化内容的人机协同生产

本节学习目标

- 掌握“AI 生成初稿 → 脚本体检 → 人工审定”的三段式生产流程；
- 掌握从招标文件自动生成结构化条目骨架的提示词工程；
- 会用规则脚本做条目覆盖率、★条款、指标一致性体检；
- 理解人工审定环节的“五问法”。

三段式生产流程总览

spec.md 与 SRS 结构化内容（功能清单、数据字典、对照表）的生产分三段：第一段 AI 生成——以招标文件第五章全文为上下文，让 AI 抽取条目、转写验收标准；第二段脚本体检——不依赖 AI 的规则脚本机械核对覆盖率与一致性；第三段人工审定——逐条过“五问法”。澜图样例的 spec.md 即按此流程产出，全程约 4 小时（纯人工编写同等文档约 2-3 天）。

AI 生成：条目抽取提示词

【角色】你是需求工程师，精通 GB/T 9385。

【上下文】以下是招标文件第五章《用户需求书》全文：<粘贴原文>

【任务】抽取全部功能需求条目，按 FR-xx.y 编号逐条转写为结构化条目：编号 / 名称 / 是否★ / 招标原文要求（照录） / 工程化转写（一句话） / 建议验收方式。

【铁律】① 编号与原文完全一致，禁止重新编号；② “招标原文要求”栏逐字照录；③ ★标记只认原文中带★号的；④ 不确定的一律标注“（待人工确认）”，禁止补全。

提示词的四条铁律正是澜图 V1.0 四类缺陷的针对性预防：编号错位（铁律①）、转述失真（铁律②）、★漏标（铁律③）、合理想象（铁律④）。教学要求学生把这段提示词存入小组的 prompts/ 目录并版本管理——提示词是生产资料，与代码同等待遇。

### 脚本体检：check\_spec.py 的设计

体检脚本不判断内容好坏，只做机械核对。澜图样例配套的体检项（24 项）分为四组：

- 覆盖组：招标 55 个 FR 条目、13 条★条款、PE-01~09 在 spec.md 与 SRS 中是否双双出现；
- 一致组：12 项关键指标（≥10GB、≤3 秒、≥80%、≥0.75、≥180 天、≥99% 等）在两文档中是否一致；
- 残留组：旧编号、旧术语（“复核专家”等已废止词）是否残留；
- 结构组：每个 [M] 条目是否带 AC 或实测方式；§4 范围外是否非空。

实现上只是一个 100 行左右的 Python 脚本（python-docx 读 SRS + 正则扫 spec.md），学生应在第 3 关为它补充“设计元素回填率”检查。要点：脚本的检查项清单本身要版本管理，每次评审前冻结当次检查项。

### 人工审定：五问法

脚本绿灯后进入人工终审，逐条问五个问题：

- 一问编号：这条编号在招标文件原文里存在吗？（翻原文，不凭记忆）
- 二问指标：数字与招标原文一致吗？正偏离标注了吗？
- 三问主语：这条需求的行为主体是系统还是人？（“作业员应核对...”是流程要求不是系统需求）
- 四问可测：验收时拿什么证据？谁来测？
- 五问边界：这条与范围外清单冲突吗？与相邻条目重叠吗？

五问法每人每小时约可审 15–20 条，24 条功能需求由 3 人小组分模块并行约 1 小时可完成一轮。审定记录（谁审的、发现什么、改没改）写入配置库——它是“人工核验”这一质量承诺的物证。

## 数据字典的半自动生成

数据字典是 AI 半自动生成收益最高的部分：把 spec.md 的功能条目 + 访谈中的数据线索喂给 AI，要求按七列格式产出实体表初稿。人工审定的重点是三列：取值范围（枚举是否列全、是否与功能章节规则一致）、数据来源（“自动”的必须说清由什么计算）、备注（跨条目挂接是否真实存在）。澜图样例 8 张字典表中，AI 初稿的字段级错误集中在“把设计决策提前”（写出了 VARCHAR(64) 这类物理细节）——需求级字典只到“字符，64”。

## 本节小结

三段式流程把人的时间花在刀刃上：AI 干抽取和转写（快但会编），脚本干核对（严但只认规则），人干判定（慢但负责任）。下一章讲本关最有方法论特色的环节——AI 反向澄清。

## 4.9 双轨同步工程：SRS 与 spec.md 如何永不打架

答辩问题 19 已经给出原则答案，本节讲工程实现。双轨制的最大风险是漂移——SRS 改了一个数字，spec.md 没跟上，三周后测试用例按旧值写，验收时全面返工。防止漂移靠三个机制：单一数据源、生成式同步、一致性体检。

### 4.9.1 单一数据源：结构化内容是唯一真相

把 SRS 中的结构化内容（功能清单表、AC 场景、性能指标表、权限矩阵、术语表）抽取为一个 YAML 主数据文件 req\_data.yaml，作为唯一真相源。SRS 的表格与 spec.md 都由它生成或同步：

```
req_data.yaml (节选) - id: FR-01.1   name: 大幅面影像分片上传   priority: M   star: true
mvp: true   source: [招标 5.4.2]   desc: 支持单景不小于 10GB 的遥感影像分片上传，网络中断后从断点续传
ac: AC-01.1 - id: AC-01.1   for: FR-01.1   given: 一景 10.2GB GeoTIFF 影像上传至 62% 时网络中断
when: 网络恢复后用户重新发起上传   then: 从断点续传，已传分片不重传，全程无需人工重新选择文件
```

这条 YAML 同时喂给两个生成器：gen\_srs\_tables.py 生成 SRS 第 5 章功能清单表与附录的 docx 表格片段；gen\_spec.py 生成 spec.md 对应段落。叙述性章节（背景、用户特征等）仍人工撰写，与结构化内容在文档装配时合流。



## 4.9.2 生成式同步的操作流程

日常维护流程：改需求只改 req\_data.yaml → 跑两个生成器 → 跑 check\_spec.py → 人工核对 diff → 入库。任何直接改 SRS 表格或 spec.md 正文的行为都是违规——下次生成时手工改动会被覆盖，这就是用工程手段强制单一数据源纪律。

生成器本身由 AI 编写（约 150 行 Python），规格就是本节文字加 req\_data.yaml 样例——又是一次 spec 驱动开发的演练。生成器入配置库 tools/，版本与文档同步。

## 4.9.3 一致性体检：check\_spec 的 C 组检查

C1~C6 一致性检查（附录 G）是最后一道闸：条款号一致、数值与招标原文照录比对、角色名统一、版本号一致、红线双侧标注、附录引用存在。其中 C2『数值照录比对』最实用——脚本内置招标关键数值表（10GB、3 秒、10 分钟、180 天、99%、80%/85%/0.75/3%），在 spec.md 与 SRS 文本中全文检索这些数字及其变体（如『90 天』『5 秒』），发现与基线不符的数值立即报警。4.13 节复盘『日志留存 90 天』这类抄写错误，就是被 C2 抓出来的。

## 4.9.4 不建 YAML 的降级方案

对自拟题小组或需求规模小（FR<30 条）的组，YAML 方案可能过重。降级方案：spec.md 为唯一源头，SRS 表格由 gen\_srs\_tables.py 从 spec.md 反向解析生成。纪律不变——仍然只许改一处。无论哪种方案，『单一数据源+生成同步+体检』三件套不能少；纯手工双份维护在本课程一律视为过程缺陷。

## 4.10 AI 反向澄清：让 AI 当苛刻的评审专家

### 4.10.1 本节学习目标

- 掌握反向提问法的提示词设计与操作流程；
- 掌握歧义的分类（口径/边界/异常/冲突）与命中率规律；
- 掌握裁决会议的组织与“修订落点”的闭环要求；
- 能正确区分真实歧义与 AI 误读，避免被 AI 牵着走。

### 4.10.2 什么是反向澄清

常规用法是人问 AI 答；反向澄清是让 AI 就你的 SRS/spec.md 向你提问——扮演最苛刻的评审专家，专找“看起来清楚、细想有两种解释”的地方。GB/T 9385 的“无歧

义”特性靠人工自查很难保证（作者对自己的文字有理解盲区），AI 的“局外人视角”恰好补位。澜图案例首轮 23 问确认 12 条真歧义（命中率 52%），全部裁决后落入 V1.0 基线。

4.10.3 反向提问提示词（教学模板）

【角色】你是验收评审专家，以苛刻著称，曾否决过三份需求文档。  
【上下文】以下是本项目 SRS 与 spec.md 全文：<粘贴>  
【任务】逐条寻找歧义、不可测、不可判定之处并向我提问。每条提问必须：①引用原文句子；②指出至少两种可能的解释；③说明两种解释导致的后果差异；④给出你的判定建议。  
【禁忌】不要问原文可检索到答案的问题；不要问纯表述风格问题；不要问与业务无关的通用软件问题。

提示词设计的三个关键：角色设定给 AI“找茬”的动机；“引用原文 + 两种解释”的格式强迫 AI 把模糊感落成具体分歧；禁忌清单压低误读率——澜图第一轮没加禁忌时 23 问中误读 11 条，加上禁忌清单后第二轮误读率降到约 30%。

4.10.4 歧义四分法与命中率规律

| 歧义类型 | 特征             | 澜图样例                                | 命中率 |
|------|----------------|-------------------------------------|-----|
| 口径类  | 指标有数字但无测量定义    | CL-02（3 秒从何时算起）、CL-09（完整率按个数还是面积）   | 高   |
| 边界类  | 范围、上限、适用集合未界定  | CL-01（幅面上限）、CL-03（词库边界）、CL-08（落盘红线） | 高   |
| 异常类  | 只写正常路径，没写失败怎么办 | CL-06（批量含不合格影像）、CL-10（资源争用）         | 中高  |
| 冲突类  | 两处条款/文档互相矛盾    | CL-07（等保“满足”是设计还是测评）、CL-12（MVP 与刚需） | 中   |

教学提示：让小组按四分法给 AI 的提问归类，命中率最高的口径类和边界类优先裁决；误读多发生在“业务常识类”（AI 用互联网产品经验套政务内网场景，如驳回记录中的“密码找回字段”）——这类直接驳回并记录理由即可，驳回记录本身也是评审加分证据：它证明团队不是照单全收。

4.10.5 裁决会议的组织

- 参会：需求工程师（主持）、架构师（判技术可行性）、甲方代表（拍板业务口径），教学场景由教师客串甲方；
- 议程：逐条宣读 AI 提问与两种解释 → 讨论 ≤5 分钟 → 形成裁决结论（一句话、可执行）→ 指定修订落点（改哪个文件哪一节）→ 记录人当场登记；
- 闭环要求：没有修订落点的裁决等于没裁决。澜图澄清记录每条的“修订落点”段精确到“SRS 4.6 节 FR-06.3 业务规则 1”——评审时会按落点抽查 SRS 是否真的改了；
- 时限纪律：单场裁决会议 ≤90 分钟、≤15 条；超时议题挂起，指定专人线下澄清后再上会。

#### 4.10.6 澄清记录的四要素格式

每条记录固定四段：**【AI 反向提问】** → **【歧义分析】** → **【人工裁决】** → **【修订落点】**。四段缺一不可：缺分析则裁决显得武断；缺落点则无法验证闭环。记录按 CL-xx 编号，被 SRS/spec.md 引用（如“面积加权口径，CL-09 裁决”）——编号的交叉引用让澄清记录从“过程文档”升级为“需求语义的权威注解”。

#### 4.10.7 防滥用：AI 裁决也可能错

澜图案例的 CL-08 是最好的反面教材：第一轮对“瓦片落盘”的裁决（不落盘）本身就不符合招标 FR-01.2，V2.0 审核时才纠正为“只下载不上传”。教训：① 裁决结论若与招标条款冲突，以条款为准并回查裁决过程；② 涉合同红线的裁决（数据、安全、知识产权）必须翻原文，不接受“凭理解”；③ 重大裁决在 RTM 或审核记录中留下二次复核痕迹。

#### 4.10.8 本节小结

反向澄清把 AI 从“枪手”变成“对手”：让它攻击你的文档，再由人来裁决。方法论闭环是：提问（AI）→ 分类（人）→ 裁决（人+甲方）→ 落点（三件套同步）→ 记录（四要素）。下一章讲把所有这些成果登记成账的 RTM。

### 4.11 RTM 逐行精讲：追踪链是怎么织出来的

RTM（需求追踪矩阵）是本关最被低估的交付物。学生组常把它当作『最后补一张表』，结果验收阶段到处断链。本节以样例《需求追踪矩阵 RTM v2》的行结构为例，

讲清每一列怎么填、每一行怎么来。

#### 4.11.1 RTM 的列结构与设计理由

| 列名   | 填写内容                         | 设计理由               | 常见错误                     |
|------|------------------------------|--------------------|--------------------------|
| 需求编号 | FR-01.1 等，与 SRS/spec.md 完全一致 | 追踪锚点，全生命周期不改号      | SRS 改编号后忘同步 RTM，断链       |
| 需求名称 | 动宾短语，≤12 字                   | 快速识别，与 SRS 功能清单表同名 | 同一需求在 RTM 里换了名字          |
| 来源   | 招标条款号 / IV 访谈编号 / CL 澄清编号    | 证明需求不镀金            | 来源列写『讨论决定』这种无证据措辞        |
| 优先级  | M/S/C/W                      | 与 MoSCoW 表一致       | RTM 标 M、MoSCoW 表标 S，两表打架 |
| 验收标准 | AC-01.1 编号                   | 挂接 spec.md 验收场景    | 只写『见 spec』不给编号           |
| 设计元素 | v2 起挂接：模块名/类名                | 任务 3 的挂接点          | 需求阶段就写死类名，限制设计自由         |
| 测试用例 | v3 起挂接：TC-01.1               | 任务 4 的挂接点          | 验收前补填，用例与需求对不上           |
| 状态   | 已确认/已澄清/已冻结                  | 过程管理               | 全部填『已确认』，失去管理意义          |

八列中，需求阶段（v1）必须填满前五列，设计与测试列留空待后续版本挂接。留空不是缺失，是版本纪律——评审老师看到 v1 就把测试列填满的，反而会追问『你们测试用例需求阶段就写完了？怎么写的？』

#### 4.11.2 43 行的构成：24 条 FR + 16 条 NFR + 3 条精度指标

样例 RTM v2 共 43 行：24 条功能需求（FR-01.1~FR-07.3，对应 7 个功能域）+ 16 条非功能需求（PE-01~05 性能、REL 可靠性、MTN 维护性、SEC 安全、DT 可移植、OSS 与 DOC 合规）+ 3 条精度指标（AT-01~03）。这个结构传递一个信号：非功能需求必须进 RTM，与功能需求同等追踪。学生组最常见的 RTM 缺陷就是只追踪功能需求——而非功能需求（性能、可用率、日志留存）恰恰是验收时最容易起争议的地方，

没有追踪行就没有验收依据。

行序规则：先 FR 后 NFR，各自按编号升序；同一 FR 的多条 AC 不另起行，验收标准列内用顿号分隔（如 AC-03.1、AC-03.2）。行数膨胀是 RTM 失控的开始——69 行是精心控制的结果，超过 120 行的 RTM 在一个月工期内必然维护不动。

### 4.11.3 三种典型行的填写示范

- 星标行（FR-02.1 在线交互式分割标注）：来源=招标 5.4.4★；优先级=M；验收=AC-02.1；状态=已冻结；行背景标黄。星标行共 13 条对应 13 条★款，评审时按★款逐条点名，本行必须能 10 秒内翻到。
- 澄清修订行（FR-03.1 样本库管理）：来源=招标 5.4.5+CL-04；状态=已澄清。凡经澄清修改的需求，来源列必须同时挂原条款与澄清编号——这是『澄清闭环』在 RTM 层的体现，缺了 CL 编号，4.10 节讲的歧义修复闭环率就算不出来。
- 访谈新增行（FR-07.1 解译结果人工修正）：来源=IV-11（第二次访谈）；优先级=S；备注二期。这类行是答辩问题 6 的直接证据，来源列的 IV 编号必须与访谈结论清单一一对应。

### 4.11.4 RTM 的维护纪律

三条铁律：① RTM 与 SRS 同版本冻结——SRS 升 V2.0 时 RTM 必须同步升 v2，版本号错位即断链；② 每次需求变更先在 RTM 波及分析（改动行数、波及测试用例数），评估完再改 SRS——顺序反过来就是先斩后奏；③ RTM 用 Excel 维护但纳入配置库，文件名带版本号，禁止多人在线同时编辑（合并冲突会毁掉追踪链）。

自检方法：冻结前做『双向抽查』——正向抽 10 条 FR 看能否走到 AC，反向抽 5 条 AC 看能否走回 FR 与来源条款。双向都通，RTM 才算织好。check\_spec.py 的 R3 项（RTM 行数与 FR+AC 总数一致）是第一道自动闸，但编号一致不等于语义一致，人工抽查不可省。

## 4.12 质量评审、基线冻结与考核标准

### 4.12.1 本节学习目标

- 掌握本关过关评审的组织方式与入场条件；

- 会用一致性审核清单做组间互审；
- 理解基线冻结与变更控制的执行动作；
- 明确本关考核评分细则。

#### 4.12.2 评审组织

评审会 30 分钟/组：10 分钟小组陈述（讲需求链：访谈→故事→spec→SRS→澄清→RTM）+ 15 分钟评委质询 + 5 分钟评议。入场条件（缺一不可）：五份交付物齐套、check\_spec 体检报告绿灯、RTM 自检报告（孤儿=0）、澄清记录 ≥10 条且四要素完整。

#### 4.12.3 质询高频问题（评委视角）

- 随机抽一条需求，现场演示“招标条款 → spec.md → SRS → RTM”链路；
- 你们指标里哪个比招标更严？标注正偏离了吗？为什么这么做？
- AI 澄清的 12 条里，哪条裁决最艰难？甲方代表（教师客串）当时怎么拍的板？
- 挑一条 [STAR]，说清验收主体、数据、不通过的处置；
- 如果甲方明天要求把某性能指标加严一倍，你们的变更流程第一步做什么？（标准答案：定位 RTM 行做影响分析）

#### 4.12.4 基线冻结与变更控制

- 评审通过即冻结基线：spec.md、SRS、RTM 打版本号（V1.0/V2.0...）入配置库；
- 此后任何修改走变更：变更申请 → 影响分析（RTM）→ CCB 评审 → 三件套同步升版 → 体检归档；
- 紧急缺陷修复可先改后补单，48 小时内补登（沿用《需求确认书》第 5 条约定）。

#### 4.12.5 考核评分细则（任务 2，满分 100）

| 评分项     | 分值 | 评分要点                   |
|---------|----|------------------------|
| 访谈与需求获取 | 15 | 画像可信、纪要有原话、故事过 INVEST、 |

| 评分项               | 分值 | 评分要点                         |
|-------------------|----|------------------------------|
|                   |    | MoSCoW 有合同依据                 |
| SRS 质量（GB/T 9385） | 25 | 骨架合规、六要素完整、图文一致、覆盖率 100%     |
| spec.md 质量        | 20 | 编号直引、AC 可判定、[STAR] 标主体、范围外显式 |
| AI 反向澄清           | 15 | ≥10 条、四要素完整、有驳回记录、修订落点可验证    |
| RTM 与追踪           | 15 | 双矩阵完整、孤儿 0、占位策略清晰、自检报告齐      |
| 评审表现              | 10 | 链路演示流畅、质询应答有据、时间纪律           |

否决项（一票不过）：★条款遗漏任一；需求编号自编造；指标与招标原文不符且未标正偏离；澄清记录无修订落点。

## 4.13 一致性审核实战与常见错误案例集

### 4.13.1 案例背景

澜图样例的 V1.0 由 AI 高效生成、表面完整（36 页、结构齐全、图文工整），但在“对照招标文件原文逐条审核”后暴露出 6 类缺陷，被迫发布 V2.0 大修。本节复盘全过程——这是本课程关于“AI 产出只是初稿”最有力的一次实证，各小组在自己的项目上至少应执行一次同规格的审核。

### 4.13.2 六类缺陷与发现手段

| 缺陷                        | 发现手段           | 教训                    |
|---------------------------|----------------|-----------------------|
| 需求编号自编，与招标错位              | RTM 映射时对不上条款号  | 编号直引原文，骨架派生法（第 5.4 节） |
| 角色少两类（漏业务管理员/浏览者）         | FR-10.1★ 原文核对  | ★条款逐字照录，不凭印象          |
| 文件大小限制与招标相反（≤4GB vs ≥4GB） | 指标值体检脚本 + 原文比对 | 指标照录原文，禁止转述           |
| CL-08 裁决与 FR-01.2 冲突      | 二次复核涉红线裁决      | 涉红线裁决必翻原文             |

| 缺陷                  | 发现手段    | 教训        |
|---------------------|---------|-----------|
| 可用率 99.5%、并发 20 等偏差 | 指标一致性体检 | 正偏离必须显式标注 |
| 单位名称、角色称谓不统一        | 术语残留检查  | 图文同名同改    |

#### 4.13.3 审核的标准动作（可复用流程）

- 第一步：从招标文件提取权威清单（★条款、FR 条目、PE 表），存为基准文件；
- 第二步：跑自动体检（覆盖组/一致组/残留组/结构组，24 项）；
- 第三步：人工按“审核清单”（附录 A）过人工项（图文一致、裁决合理、术语统一）；
- 第四步：缺陷分级（致命/高/中/低），致命缺陷不过夜；
- 第五步：修订 → 升版 → 复检 → 写审核记录（发现什么、怎么改的、依据什么）。

澜图 V3.0 终检 24 项全绿：13 条★、24 条 FR、5 条 PE 双文档全覆盖，12 项关键指标一致。完整记录见《任务 2 样例审核记录》。

#### 4.13.4 学生常见错误案例集（10 例）

以下案例按本课程试点班与澜图样例审核中的真实问题整理（去标识化），供自审时对照。

##### 案例1：编号“优化”

某组把招标的“FR-02.1”统一改成“REQ-AUTO-01”，理由是“更语义化”。后果：RTM 与招标文件失去机械对应，验收条款映射全部要人工翻译。规则：编号是锚点，锚点必须原样。

##### 案例2：指标“润色”

某组把“完整率不低于 80%”写成“完整率约 80%”。“约”字让验收无法判定。规则：指标照录，一字不改。

##### 案例3：界面写成效果图

某组在 SRS 里贴了 6 页精美的界面设计图，却没有功能数据与业务规则。后果：



设计与需求错位——图是第 3 关的产出，需求关只需要界面清单与元素描述。规则：需求关的界面只到“页面清单 + 关键元素”。

### **案例4：测试要点写“测试功能是否正常”**

这是无效的测试要点。规则：每条测试要点必须有构造方法 + 判定标准（“构造跨块建筑物样本，验证拼接处无错位”）。

### **案例5：AI 澄清凑数**

某组的 10 条澄清全是“是否需要帮助文档”这类泛泛之问。规则：四要素中“歧义分析”必须给出两种解释及后果差异，给不出的不算歧义。

### **案例6：范围外清单缺失**

验收模拟时教师扮演甲方提出“移动端也要啊”，小组无法招架。规则：spec.md §4 显式列 Won't 并与招标项目边界对应；自拟题则与教师在需求确认时书面冻结。

### **案例7：双轨漂移**

某组评审前夜改了 SRS 的指标却忘改 spec.md，次日体检脚本抓出不一致。规则：先改 spec.md 再同步；体检报告随每次升版归档。

### **案例8：访谈纪要不是访谈的记录**

某组“访谈”实为组内讨论，纪要没有一句原话。规则：原话栏为空即视为未访谈；虚拟画像访谈也要按真实流程走（角色扮演者按画像回答）。

### **案例9：MoSCoW 全部 Must**

某组 21 条故事 19 条 Must。规则：Must 必须有合同/确认书依据；答不出条款号的 Must 当场降级。

### **案例10：图表互不负责**

某组流程图画了“失败重试”分支，正文却只字未提；DFD 上有“D5 缓存库”而数据字典无对应表。规则：图文互查、图表互查列入附录 A 清单，逐条打勾。

## **本节小结**

十个案例的共同根源是同一个：把“看起来完成了”当成“完成了”。需求工程的质量不体现在文档厚度，而体现在每一条都可溯源、可判定、可验收。

## 4.14 过关自检与交接

任务 2 的终点是任务 3 的起点。需求阶段与开发阶段之间的交接质量，直接决定 spec.md 能不能成为 AI 编码的有效输入。本节给出交接的内容清单、格式约定与启航检查——完成本节全部勾选，任务 2 才算真正过关。

### 4.14.1 交接物清单（六件套）

| 交接物           | 版本       | 任务 3 中的用途           | 交接验收标准            |
|---------------|----------|---------------------|-------------------|
| SRS           | V2.0 冻结版 | 设计的业务依据；验收的对照基准     | 评审检查单全过；签字齐全      |
| spec.md       | v2       | AI 编码的直接输入；测试用例生成源头 | check_spec 24 项全过 |
| req_data.yaml | v2（若采用）  | 结构化单一数据源，后续变更入口     | 与 spec.md 生成一致    |
| RTM           | v1       | 设计元素挂接的基表；波及分析工具    | 55+14 行双向抽查通过     |
| 澄清裁决表         | CL-01~12 | 设计争议的裁决依据；验收口径档案    | 每条有落点，闭环率 100%    |
| 提示词库+rules    | 当前版      | 任务 3 编码提示词的基础       | 附录 K 卡片本地化完成      |

六件套以配置库整体移交的方式交接：任务 3 开工会上，需求负责人逐项展示、全组确认、指导教师见证。移交不是形式——任务 3 的架构师（可能是组内另一名同学）将以此为唯一依据做设计，交接会上发现的理解偏差，比开发两周后发现的便宜

十倍。

#### 4.14.2 spec.md 作为编码输入的格式约定

任务 3 的 AI 编码将以 spec.md 为上下文核心。为保证编码 Agent 正确消费，冻结前最后检查三项格式约定：① 每条 FR 的描述自包含——不依赖『见上文』『同前』这类需要全文上下文的表述，因为编码时往往只截取单条 FR 喂给 AI；② AC 的 Given 中数据规模、硬件前提写全——AI 生成测试代码时将直接照用这些数值；③ 约束与红线在第 5 节集中且每条有 RED 编号——编码规则文件（rules）将逐条引用这些编号。

一个实操建议：冻结前做一次『编码预演』——随机抽一条 MVP 功能（如 FR-02.1），把它的 FR+AC+相关 RED 单独喂给 AI，要求生成该功能的接口骨架代码。生成的代码若出现需求中没有的假设（如自创字段、自造默认值），说明 spec 的自包含性不足，立即修订。这次预演同时是任务 3 工作方式的彩排。

#### 4.14.3 需求基线的变更纪律（任务 3 期间的衔接规则）

冻结不是不变——开发阶段必然发现需求问题。纪律是：① 开发中发现的需求缺陷，不得在代码里绕过去，必须回填变更申请；② 变更走 RTM 波及分析→甲方（教师）签认→SRS/spec 升版→体检复跑的完整流程；③ 每次变更在 RTM 对应行的状态列记『已变更-V2.x』，保持追踪链连续；④ 课程结束时，SRS 终版与验收时的实际系统必须一致——验收老师抽查的就是这个一致性。

这条纪律的现实意义：AI 编码时代，『需求缺陷在代码里悄悄绕过去』的成本变得极低——AI 顺手就把逻辑改了，文档却没动。一个月后文档与系统是两张皮，验收时对不上。变更纪律就是给这个最低成本的违规路径装上闸门。

#### 4.14.4 任务 2 过关自查总清单

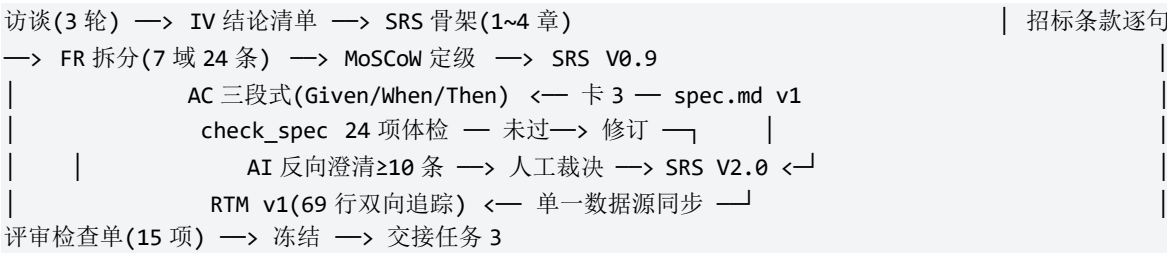
提交评审前，组长按以下清单逐项勾选（全部通过方可提交）：

- 文档齐：SRS V2.0、spec.md v2、RTM v1、澄清裁决表、访谈记录（含确认）、MoSCoW 表、版本修订说明、四指标度量表——八项交付物全部入库且命名规范。
- 数字对：24 条 FR、16 条 NFR、3 条精度指标、13 条★全覆盖、≥10 条澄清、24 项体检全过、MVP≤5 项——六个关键数字逐项核实，与正文一致。

- 链不断：随机抽 10 条需求，从 RTM 走到 AC、走到来源条款、走到访谈原话或招标原文，全程无断点。
- 人能答：全组对 9.2 节的 20 个问题的模拟答辩达标（≥16 题合格），星标条款相关问题人人能答。
- 底稿清：V0.9/V1.0 历史版本、AI 对话记录、体检报告按目录归档，文件修改时间与过程记录吻合（防事后补做）。

五项全勾，任务 2 正式过关。带着冻结的基线、织好的追踪链和一支演练过的队伍，进任务 3——设计与开发关。

#### 4.14.5 全卷回顾：任务 2 方法论一图流



这张图是全书的浓缩。学生完成本关后回看，应能指着图中每一个节点说出：这个环节我们组做了什么、产出了什么、踩过什么坑。能讲清楚，就是真掌握；讲不清楚的节点，就是下一个项目要重点补课的地方。

| # | 检查项                           | 判定 |
|---|-------------------------------|----|
| 1 | 需求编号全部直引招标文件，无自编号             | □  |
| 2 | 13 条★条款逐条在册，验收主体为第三方/招标人      | □  |
| 3 | 指标值与招标原文一致；正偏离显式标注            | □  |
| 4 | 五类角色与招标 FR-10.1 一致，全文（含图）术语统一 | □  |
| 5 | 每张流程图与正文互查一致；异常分支有归宿          | □  |
| 6 | 数据字典每个存储有表、每张表有读写方            | □  |
| 7 | 澄清裁决与招标条款无冲突；涉红线裁决有原文复核记录     | □  |
| 8 | 范围外清单与招标项目边界一致                | □  |
| 9 | RTM 覆盖率 100%、孤儿 0、自检报告归档      | □  |

| #  | 检查项               | 判定 |
|----|-------------------|----|
| 10 | 五份交付物版本号一致，变更记录可溯 | □  |

D5 下午的组内正式评审使用本检查单，质量员逐项宣读、全组表决、记录结论。  
全部通过方可冻结 V2.0；任一『否决项』不通过，当日修订后复审。

| 类别  | 检查项                               | 否决项 |
|-----|-----------------------------------|-----|
| 完整性 | 招标文件功能域（F-01~F-10）逐域有 FR 覆盖，无整域遗漏 | 是   |
| 完整性 | 13 条★款全部有对应 FR/PE 且 RTM 可追踪       | 是   |
| 完整性 | 五类角色在权限矩阵中均有定义                    | 否   |
| 可测性 | 每条 FR 配 AC 且为 Given/When/Then 三段式 | 是   |
| 可测性 | 全部量化指标含数值+单位+前提条件                 | 是   |
| 可测性 | 全文无『等/若干/快速/友好』等模糊词（除引用原文外）       | 否   |
| 一致性 | SRS 与 spec.md 条款号、数值、角色名一致        | 是   |
| 一致性 | SRS 与需求确认书的 MVP 范围一致              | 是   |
| 追踪性 | 每条 FR 来源列非空（招标条款/IV/CL 编号）        | 是   |
| 追踪性 | RTM v1 行数与 FR+NFR 总数一致，双向抽查通过     | 否   |
| 过程  | 澄清记录≥10 条且每条有裁决与修改落点              | 否   |
| 过程  | check_spec 报告 24 项全部通过并归档         | 是   |
| 过程  | 版本修订说明（V1.0→V2.0）完整               | 否   |
| 合规  | 红线条款（只下载不上传等）双侧标注                 | 是   |
| 合规  | 参考资料全部带版本号与日期                     | 否   |

评审记录格式：会议时间、出席人、逐项结论、未通过项的整改责任人与时限、冻结决议签字。评审记录本身也是交付证据，入配置库『02\_需求阶段/评审/』目录。

- 《用户访谈记录与 MoSCoW 优先级》：对象矩阵 → 画像四条标准 → 四场纪要 → 故事清单（带来源列）→ MoSCoW 表（带依据列）→ 冲突说明；

- 《SRS》：文档控制 → 引言（含功能清单表）→ 综合描述 → 业务流程（四图）→ 功能需求（六要素+一览表）→ 数据需求（DFD+字典）→ 接口 → 非功能 → 验收追踪 → 附录；
- spec.md：§0 元信息 → §1 功能（FR+AC）→ §2 非功能（PE 表照录）→ §3 验收口径 → §4 范围外 → §5 变更记录；
- 《AI 反向澄清记录》：方法说明 → CL-xx 四要素条目 → 误读驳回记录 → 统计结论；
- 《RTM》：追踪规则 → 功能矩阵 → 非功能矩阵 → 完整性自检结论。

（本指导书与澜图案例样例文档配套使用：结构可对样例，内容必须原创。）

## 第 5 章 第三关：设计与计划

本关把规格变成设计与计划：概要、详细、数据库、接口四份设计文档，ADR 决策文书，非功能设计逐项对标 GB/T 25000.10，以及 plan/tasks 生成链与测试左移的测试计划。注意防止过度设计与零设计两个极端。

### 5.1 关卡导论与设计总方法论

#### 5.1.1 任务 3 在全生命周期中的位置

通关实践的前两关解决了『做什么』：任务 1 完成招投标与立项，锁定了 MVP 范围与验收依据；任务 2 完成需求与规格，冻结了 SRS、spec.md 与 RTM v2。任务 3 解决『怎么做』——把需求翻译成可实施的技术方案与可执行的工作计划。它是研发线的真正起点：从本关开始，每一句话都会变成代码、表结构、接口和测试用例，设计阶段的含混会在编码阶段以十倍的代价爆发。

任务 3 的时间窗口是 D7~D8（8/18~8/19，2 个工作日），紧随需求基线之后：设计冻结之日，测试计划同步发布（测试左移），开发环境在课前 Bootcamp 已就绪，研发冲刺 D9 直接开工。

#### 5.1.2 任务 3 的交付物清单

| 类别    | 交付物                    | 对标标准                    | 样例文件编号                |
|-------|------------------------|-------------------------|-----------------------|
| 设计文档集 | 概要设计说明书（HLD）           | GB/T 8567 章节结构          | 《概要设计说明书》             |
| 设计文档集 | 详细设计说明书（DLD）           | GB/T 8567；模块级伪代码与单元测试要点 | 《详细设计说明书》             |
| 设计文档集 | 数据库设计说明书               | 概念/逻辑/物理/运用四段           | 《数据库设计说明书》            |
| 设计文档集 | 接口设计说明书 + OpenAPI 契约   | OpenAPI 3.0；HTTP 状态码合法  | 《接口设计说明书》与 OpenAPI 契约 |
| 架构决策  | ADR $\geq 2$ 份（含被否决方案） | MADR/Nygard 格式          | 三份 ADR（存储选型、任务执行模     |

| 类别        | 交付物                | 对标标准               | 样例文件编号                       |
|-----------|--------------------|--------------------|------------------------------|
|           |                    |                    | 型、推理进程模型)                    |
| 研 发<br>计划 | plan.md + tasks.md | 原子任务≤半天、带依赖、可独立验证  | plan.md 与 tasks.md           |
| 测试        | 测试计划               | GB/T 15532；设计冻结即发布 | 《测试计划》                       |
| 管理        | 项目/质量/配置/风险管理计划    | 与任务 1 计划族衔接升级      | 管理计划族（项目管理/质量管理/配置管理/风险管理四册） |
| 追踪        | RTM v3（设计挂接版）      | 需求→设计→测试双向无断链      | RTM v3                       |

### 5.1.3 过关条件

- 设计文档集通过正式评审：章节结构完整、图实一致、每个设计决策可追溯到需求条款；
- ADR 论证质量：候选方案≥2、否决理由具体（禁止『因为我会这个』）、后果诚实（含负面）；
- plan/tasks 可执行：抽查任意 3 个原子任务，能回答『依赖谁、谁来做、怎么算做完』；
- RTM v3 双向核查通过：正向 24 条 FR + 16 条 NFR 全挂接，反向无镀金模块；
- 测试计划与设计同步冻结，性能/精度/安全等专项测试有明确测法与判定阈值。

### 5.1.4 本指导书怎么用

5.1 节讲总方法论；5.2~5.9 节按交付物逐一讲『人工怎么做』与『人机协同怎么做』；5.10 节讲计划族；5.11 节讲 RTM 双向追踪；2.3 节讲 Skills 沉淀与提示词；第 17~21 章是质量保证、样例剖析、反面案例与端到端走查；第 22~24 章面向教师与附录。每一章都紧扣『澜图』教学案例的真实文件展开——请把澜图设计文档集与计划族样例放在手边对照阅读。

体例说明：『人工方法』部分假设没有任何 AI 工具，讲清工具、要点、过程——这是基本功，



也是评审 AI 产出的能力前提：『人机协同』部分讲如何用 AI 把 3 天的活压缩到半天，同时用人工审查守住质量红线。

5.1.5 设计工作总方法论

设计的本质：需求的可实施翻译

设计不是创作，是翻译。每一条设计决策都应当能回答三个问题：

- ① 它满足哪条需求（FR/NFR 编号）？
- ② 它排除了哪些替代方案、为什么（ADR 编号）？
- ③ 它如何被验证（测试用例编号）？

三个问题都有答案的设计才叫『可追踪设计』；答不上来的部分，要么是镀金（无需求来源），要么是漏设计（有需求无落点）——RTM 双向核查就是专门抓这两种病的。

设计分层：概要与详细的分工

| 维度          | 概要设计（HLD）                  | 详细设计（DLD）                     |
|-------------|----------------------------|-------------------------------|
| 回 答 的<br>问题 | 系统由什么组成、怎么部署、为什么这么选型       | 每个模块内部怎么实现、类与接口怎么定义           |
| 读者          | 评审专家、甲方、全组                 | 写代码的人（含 AI）                   |
| 核 心 内<br>容  | 架构视图（技术/数据/部署/集成/安全）、非功能设计 | 模块设计（类、接口、流程、伪代码、异常处理）        |
| 变 更 成<br>本  | 高——动了架构全员返工                | 低——模块内部可迭代                    |
| AI 角色       | 起草候选架构+绘图，人定夺              | 按 spec+constitution 逐模块起草，人审查 |

设计三问工作法

每写一节设计，先在心里过三问：第一问『需求原文怎么说的』——把 FR 原文复制到草稿旁，写完对照删除没有需求支撑的句子；第二问『评委/甲方会怎么挑战我』——把最容易被质疑的决策写成 ADR；第三问『测试怎么测我』——写出该设计对应的验证方式。三问都过不了的设计节，宁可不写。

## AI 时代的设计纪律 (constitution)

Spec-Driven 开发要求仓库里有一份机器可读的『宪章』 (rules/constitution 文件), 把团队共识写成 AI 必须遵守的硬规则。澜图案例的 constitution 包含: ① 技术栈白名单 (HLD 3.2 选型表, 表外组件禁止引入); ② 编码规范 (异常必须含 trace\_id; SQL 禁止字符串拼接); ③ 设计约束 (GPU 独占队列、CPU 降级路径、接口错误码规范); ④ 文档纪律 (任何变更同步 RTM)。AI 生成设计或代码前, 必须先读这份文件——这是防发散的第一道闸。

### 设计评审的四种形态

- 桌面检查: 作者自查, 用 2.12 节的评审清单逐项打勾;
- 同行评审: 组内互评, 重点查『需求覆盖』与『可实现性』;
- AI 反向评审: 让 AI 扮演质疑者, 对设计提  $\geq 10$  条尖锐问题 (做法同任务 2 的 AI 反向澄清);
- 正式评审会: 评委按过关条件裁决, 结论记录在案 (通过/有条件通过/不通过+整改项)。

## 5.2 概要设计 HLD: 人工方法、人机协同与逐章细则

### 5.2.1 产出定位与章节结构

HLD 的读者是评审专家与全组成员, 目标是让评审在 30 分钟内相信『这个方案能实现 SRS 的全部要求, 且团队驾驭得了』。澜图样例 (《概要设计说明书》V3.0) 的章节结构: 1 引言、2 总体设计、3 技术架构、4 功能架构、5 数据架构、6 部署架构、7 集成架构、8 安全架构、9 非功能设计 (GB/T 25000.10 质量特性映射)、10 架构演进与设计追踪。该结构覆盖 GB/T 8567 对概要设计的要求, 并针对评审关注点做了重排。

### 5.2.2 人工完成: 工具与准备

- 输入基线: SRS V2.0、spec.md、RTM v2、需求确认书——四份文件打印或双屏摆放, 设计时逐条对照;
- 绘图工具: draw.io / Visio / PPT 均可, 统一配色与字体 (样例用 matplotlib 程序化绘图, 保证图可由脚本重生成);

- 参考标准：GB/T 8567（文档结构）、GB/T 25000.10（质量特性模型）、等保二级基本要求（安全架构）。

### 5.2.3 人工完成：六大架构视图的写法要点

#### 技术架构（第3章）

先画分层图（接入/应用/任务/数据/基础设施五层），再出选型总表。选型表的每一行必须有三要素：选型+版本、理由、许可证。理由要指向需求或约束（如『OpenLayers：瓦片+矢量叠加+卷帘对比，对应 FR-07.2』），禁止写『流行』『好用』。许可证列对应 OSS-01 合规要求——GPL 系组件出现在商业交付里是硬伤。

#### 功能架构（第4章）

模块划分的第一原则是『与需求域一一对应』：SRS 有 F-01~F-10 十个功能域，HLD 就有 M-01~M-10 十个模块，映射表直接放进 4.1 节。这样做的好处是 RTM 挂接零歧义，反向核查『无镀金模块』一目了然。横切组件（认证、审计、调度）单独列出，因为它们服务所有模块。

#### 数据架构（第5章）

用『数据链路』组织叙述：入库链（影像怎么进来）、解译链（数据怎么被加工）、监督链（行为怎么被记录）。每条链画清存储介质（成果矢量库 GeoPackage / 项目状态 JSON / 文件系统影像目录）与流转方向。数据架构图与数据库设计的 ER 图必须实体一致——图实不一致是评审最常抓的低级错误。

#### 部署架构（第6章）

画节点图：客户端区、应用服务器、GPU 计算节点、数据库、存储。标注节点间协议与最小部署形态（1 台应用服务器+1 台 GPU 节点）。部署图直接支撑 MTN-01『干净环境部署≤2h』的可信度——节点越少、依赖越清楚，部署指标越可信。

#### 集成与安全架构（第7、8章）

集成架构写清边界：哪些外部系统（在线数据源、QGIS 客户端、SMTP 服务）、走什么协议、谁是主谁是从。安全架构对标等保二级做分层控制（网络/主机/应用/数据/管理五层），每层列出具体措施而非口号。

#### 非功能设计（第9章）

这是 v3 新增的硬要求：每条 NFR 都要落到具体设计项，并标注对应 GB/T 25000.10 的哪个质量特性（性能效率/可靠性/安全性/可维护性...）。例如 PE-01（交互

≤3s) → 常驻推理+交互独立通道+1024<sup>2</sup> 裁剪三个设计项。做成映射总表(9.1 节), 评委按表抽查即可。

## 5.2.4 人工完成：过程与工时

人工节奏(4 人组, 约 3 人日): D1 上午架构组闭门出分层图与模块表初稿; D1 下午~D2 分头写各视图; D2 晚交叉互查(重点: 图的实体名与文字一致、每个 NFR 有落点); D3 上午评审演练, 下午定稿。常见卡点: 模块边界争议——裁决规则是『按需求域切, 不按技术层切』。

## 5.2.5 概要设计 HLD: 人机协同方法

### 协同总流程

HLD 的协同流程是『人定骨架 → AI 填肉 → 人动手术 → AI 反向质疑 → 人定稿』五步。骨架(章节结构、模块划分、关键选型)必须人来定, 因为它们是架构决策, 责任在人; 填肉(各视图展开叙述、表格初稿、配图脚本)交给 AI; 动手术(删掉镀金内容、修正与需求不符处)是人工审查的核心动作。

### 第一步：人工搭建设计骨架

在开始任何 AI 对话前, 人工完成三件事: ① 定章节结构(直接采用第 3 章的十章结构); ② 定模块划分表(M-01~M-10 与 F-01~F-10 的映射, 照抄 SRS 功能域, 不发挥); ③ 定关键选型(从任务 1 投标书的技术承诺与 SRS 约束中继承)。把这三件事写成一个 300 字以内的 design-skeleton.md, 作为后续所有 AI 对话的锚点。

### 第二步：AI 起草的提示词模式

设计类提示词用『角色+基线+约束+输出契约』四段式:

你是资深 GIS 软件架构师。请基于以下基线起草《概要设计说明书》第 5 章『数据架构』:

【基线】附件: SRS V2.0 第 3 章数据需求、design-skeleton.md、RTM v2

【约束】① 每条叙述可追溯到 FR 编号, 写法『.....(FR-06.6)』; ② 存储介质限定 GeoPackage 成果库 / 项目状态 JSON / 文件系统; ③ 禁止出现训练、样本库相关内容——本平台为三分割模式体系; ④ 中文字符, 术语表见 SRS 1.4

【输出契约】按 5.1 数据分类与存储策略 / 5.2 数据流架构 / 5.3 数据治理 / 5.4 容量规划 四节输出; 每节≤400 字; 配图表给出 matplotlib 绘图脚本而非贴图

四段式的关键在『约束』段: 把 constitution 中与本节相关的规则全部显式列出。

AI 不知道你的红线, 除非你每次都说。

### 第三步：人工审查清单 (HLD 专用)

| # | 审查项                           | 常见 AI 毛病                    |
|---|-------------------------------|-----------------------------|
| 1 | 每条设计是否标注了 FR/NFR 来源？          | 编造看似合理的无源设计（镀金）             |
| 2 | 模块划分与 SRS 功能域是否一一对应？          | 按技术偏好重新分组，破坏映射              |
| 3 | 图中实体名与正文、DB 设计是否一致？           | 图里用 annotations，正文用 prompts |
| 4 | 选型理由是否指向需求/约束？                | 理由写『生态成熟』『业界主流』             |
| 5 | 非功能设计是否逐项对应 GB/T 25000.10 特性？ | 只写性能，漏安全与可维护性               |
| 6 | 引用澄清记录时编号是否真实存在？              | 编造 CL-xx 编号（本案例血泪教训）        |
| 7 | 性能指标的前提条件是否写明（硬件基线）？          | 只写 $\leq 3s$ ，不写环境前提        |

## 第四步：AI 反向质疑

初稿完成后，换一个对话让 AI 扮演评委：『你是评审专家，以挑刺为目的审读这份 HLD，提出至少 10 条质疑，按严重度排序，每条指出具体章节』。AI 质疑的价值在于不知疲倦且无面子负担；但裁决权在人——质疑分三类处理：成立的立即改、误解的写进评审答疑记录、超出范围的记入二期演进（HLD 10.1）。

## 配图的人机协同

架构图推荐『AI 写脚本、程序生成图』而非『AI 直接出图』：让 AI 输出 matplotlib 绘图脚本（见样例 t3/make\_diagrams3.py），人工检查文字标注后运行。好处：图可重生成、可版本管理、改一个标签不用重画；坏处：美观度不如专业工具——教学场景够用，投标场景可后期美化。图的文字内容必须人工逐字校对，v1/v2 的图实不一致（图里画训练 Worker，需求里无训练）就是这么被发现的。

## 效率对比

| 活动           | 纯人工     | 人机协同        | 质量差异       |
|--------------|---------|-------------|------------|
| HLD 初稿（24 页） | 约 3 人日  | 约 0.5 人日    | 无显著差异（审查后） |
| 架构配图 4 张     | 约 1 人日  | 约 1 小时      | 脚本版更易维护    |
| 反向质疑与整改      | 依赖评审会周期 | 随时可做        | AI 质疑覆盖率更高 |
| RTM 挂接回填     | 约 1 人日  | 约 1 小时（脚本化） | 脚本版零抄写错误   |

红线：骨架与选型决策不外包给 AI；任何 AI 产出未经清单审查不得入基线；审查记录（改了什么、为什么）写入评审记录。

## 5.2.6 HLD 逐章写作细则

### 第1章 引言

四小节：编写目的（给谁看、解决什么决策）、项目背景与设计输入（列出全部基线文件及版本——SRS V2.0/spec.md v2/RTM v2/确认书/投标文件技术标）、参考标准、术语。最容易写虚的是编写目的：不要写『为指导开发』这种废话，写『为设计评审提供裁决依据，为任务 4 编码提供实现约束』。设计输入清单是本节的实质内容——后续一切追踪都从这里出发，清单漏一份文件，追踪链就先天断一环。

### 第2章 总体设计

2.1 设计目标：复述 SRS 目标并翻译为设计语言（『支持全自动/交互/文本三种分割模式』→『三分割模式共享任务与结果模型，交互走独立低延迟通道』）。2.2 设计原则：只写有约束力、可检查的原则（澜图六条：朴素优先、契约先行、测试左移、可追溯、降级可用、合规红线），每条给检查方式。2.3 关键架构决策索引：三份 ADR 的一句话摘要+指针。2.4 架构视图总览：一页图导览后续各章。

### 第3章 技术架构

3.1 分层图+各层职责段；3.2 选型总表（选型+版本+理由+许可证四列齐全）；3.3 关键技术设计（本项目的三个硬核：GPU 独占队列与交互通道并存、两类模型组合推理流水线、大影像分块拼接）；3.4 技术风险与缓解（与风险登记册联动，只写架构级风险）。写作禁忌：分层图与模块表不一致（图里画的服务在 M-01~M-10 里找不到归属）——图实一致检查的头号对象。

## 第4章 功能架构

4.1 模块映射表（与需求域一一对应，见 3.3.2）；4.2 横切组件；4.3 核心业务流程（每流程一段叙事：触发→处理链→产出→异常出口，标注 FR 编号）；4.4 模块间依赖规则（只允许经 API/队列交互，禁止跨模块直读数据库——这条纪律防止模块边界腐化）。流程写作要点：每条流程自问『谁触发、谁等待、失败了谁知道』。

## 第5章 数据架构

三条链路（入库/解译/监督）+存储分层+治理+容量。治理节写数据责任：谁入库、谁清理、留存多久（源影像只读留存、中间掩膜可定期清理、矢量成果需归档）。容量节给估算表：输入参数（单景 1 km<sup>2</sup>、图斑 3000~8000 个、成果库 5~20 MB）→数据集容量结论。容量估算的意义不在准，在于让每个数字都有出处；未实测的估算一律标注【估算，待实测】，实测后回填。

## 第6~8章 部署/集成/安全

部署章：节点图+配置表+环境规划（开发/测试/演示三环境差异）。集成章：外部系统清单（在线数据源、QGIS、SMTP）逐个写协议、方向、失败处理。安全章：等保二级五层控制（见 6.6）。这三章的共同写法：先列清单再给表，每个集成点/控制项都能回答『故障时系统行为是什么』。

## 第9~10章 非功能与演进

第9章见 5.7 节专题。第10章演进：二期候选能力清单（从 SRS『二期』标注项继承）+设计追踪规则（设计变更怎么改文档、怎么过 CCB）。演进节的作用是保护一期：把评审中『为什么不支持 X』的回答标准化为『X 在二期演进清单第 n 项，一期不做是范围决策而非遗漏』。

## 章节间的交叉引用纪律

HLD 内部引用（如 9.1 引用 ADR-003）必须与 ADR 实际编号一致；HLD 引用 DLD/DB 章节时用『详见 DLD 第 n 章』的粗粒度引用，避免下游文档章节调整时连锁失效——粗引用是文档集群的解耦手段。全文构建后跑一次引用核对脚本（提取所有『详见/见第 n 章』模式，对照目标文档章节清单）。

## 5.3 详细设计 DLD：人工方法、人机协同与逐模块要点

### 5.3.1 产出定位

DLD 的读者是写代码的人——包括 AI。判断 DLD 质量的唯一标准：一个不了解前期讨论的人（或 AI），只凭 DLD+spec.md 能否写出符合要求的代码？凡需要『再问问设计的人』的地方，就是 DLD 没写透的地方。澜图样例（《详细设计说明书》V3.0）按模块组织 9 章：M-01~M-06 每模块一章，M-07/08/09 合并为设计要点一章，横切设计一章。

### 5.3.2 模块设计标准五件套

每个模块章必须包含五件，缺一不可：

- 职责与需求映射：本节覆盖哪些 FR，逐条列出（复制 SRS 原文编号）；
- 类/组件设计：类名、职责、关键方法与签名——写到方法级，不写到行级；
- 关键流程：用时序图或编号步骤描述主流程与异常流（异常流至少与主流程等长——线上事故 80% 出在异常路径）；
- 数据与接口：操作哪些表（引用 DB 设计表号）、暴露哪些端点（引用接口设计章节）；
- 单元测试要点：本模块必须覆盖的测试场景清单（测试左移，设计时就想好怎么测）。

### 5.3.3 伪代码的写法

关键算法写伪代码，粒度到『可直接翻译为代码』。以样例 M-05 的 GPU 独占调度为例，伪代码要素：输入、输出、状态转移、并发约束、异常处理。反例：写『然后调用模型进行推理』——这句对 AI 编码毫无信息量；正例：『worker 从 gpu\_queue 阻塞领取任务（并发度=1，实现 FR-05.2 独占语义）→ 校验模型版本匹配，不匹配则 reject 并告警 → 执行推理，心跳每 3s 上报 → 异常按 FR-05.4 重试策略入退避队列』。

### 5.3.4 边界与数值的明确化

DLD 最常见的含糊是数值缺失。规则：凡是 SRS/合同里有数值的，DLD 必须复述并落实到机制；凡是 DLD 自己定的数值，必须给理由。样例中的示范：撤销栈深度上限（FR-01.3 原文）→ 实现为内存撤销栈 + 容量硬限；简化容差过大时降级为不简化（DLD 自决）→ 理由：自相交几何无法被第三方 GIS 打开，见 ADR-001 后果节。



### 5.3.5 人工完成：过程与工时

人工节奏（约 5 人日）：按模块分工（谁开发谁设计），每人每天 1~2 个模块章；每日 15 分钟对齐会只问三个问题——昨天设计的模块接口变了吗？变了通知谁？今天的设计依赖谁的什么产出？模块间接口变化必须当日同步到接口设计与 OpenAPI 契约，这是配置管理的硬要求。

### 5.3.6 常见缺陷（评审高频扣分项）

- 类设计只有名字没有职责——『XxxManager』遍地，职责为空；
- 流程只有阳光路径——没有写取消、超时、重试、幂等；
- 数据操作与 DB 设计脱节——DLD 用了某字段，DB 设计里没有；
- 测试要点泛泛——『测试正常流程』等于没写，要写到场景级（如『乱序分片到达时幂等接收』）。

### 5.3.7 详细设计 DLD：人机协同方法

#### 协同策略：逐模块小步快跑

DLD 与 HLD 的协同策略不同：HLD 一次起草一章效果好，DLD 必须逐模块甚至逐节推进。原因：DLD 细节密度高，AI 上下文里同时装下全部模块的 spec+约束会顾此失彼；且模块间有依赖（M-05 依赖 M-02 的任务模型），顺序推进可以滚动校准。推荐节奏：每个模块一轮『AI 起草→人工审查→定稿入库』，约 1~1.5 小时一个模块。

#### 模块起草提示词模板

你是后端高级工程师。请起草《详细设计说明书》第 6 章 M-05 批量任务调度模块。

【需求基线】SRS FR-04.1~04.4 原文（附）；PE-01/PE-02；ADR-002 决策（同步执行 + 模型缓存，不引入任务队列）

【上游约束】HLD 4.1 模块职责表；DB 设计 tasks 表结构（附）；接口设计 /tasks 端点（附 OpenAPI 片段）

【结构要求】按五件套：① 职责与需求映射 ② 类设计（方法签名级）③ 关键流程（主流程+异常流，含调度伪代码）④ 数据与接口引用 ⑤ 单元测试要点（场景级）

【硬约束】GPU 独占队列并发度=1 实现，禁止自研锁；所有数值复述需求原文；不确定处列『待裁决』清单，禁止自行假设

#### 『待裁决』机制：防AI擅自假设

AI 遇到需求空白时默认行为是『编一个合理假设』——这是设计阶段最危险的行为。对策：提示词中强制『不确定处列待裁决清单』，人工对清单逐条裁决：能定的写回 DLD，需要甲方确认的走澄清流程（任务 1 的澄清机制在研发期继续有效），二

期的记演进清单。澜图案例中『文本提示阈值默认 0.35』就是这样被标记、裁决并回写 SRS 的（标注为原型值，可在验收时校准）。

与spec.md 的联动

DLD 的每个模块起草前，先把 spec.md 中该模块的 Given/When/Then 场景喂给 AI，要求其设计必须能支撑每个场景的实现。审查时反向抽查：随机选 3 个场景，问『这个场景在 DLD 里走哪条流程？哪个类处理？』——答不上来即设计有洞。这个抽查方法叫『场景走查』，是 DLD 评审的核心手段。

人工审查清单（DLD 专用）

| # | 审查项             | 判定方法                     |
|---|-----------------|--------------------------|
| 1 | 五件套齐全？          | 对照 5.2 节逐项打勾             |
| 2 | FR 映射逐条可追溯？     | 每条 FR 在章节首列出且正文有对应设计     |
| 3 | 异常流充分？          | 每个主流程至少有取消/超时/重试/幂等中的适用项 |
| 4 | 数值与 SRS 一致？     | 全文搜数值，与 SRS 逐一对照         |
| 5 | 表/字段与 DB 设计一致？  | 抽查 5 个字段名，在 DB 设计中找到     |
| 6 | 端点与 OpenAPI 一致？ | 抽查 3 个端点路径与参数            |
| 7 | 无擅自假设？          | 待裁决清单全部闭环                |
| 8 | 测试要点场景级？        | 每条要点能直接转成一个测试用例          |

防发散：长流程中的AI 上下文管理

多模块连续起草时，AI 会逐渐『忘记』前面的约束（上下文漂移）。三条对策：  
① 每个模块开新对话，重新喂基线与约束，不在长对话尾部做新模块；② 用 tasks.md 做链式任务管理（Anthropic 工程指南的模式）：AI 每完成一节即在任务清单上勾选并复述下一节约束；③ 关键约束写成 rules 文件持久化，每次对话自动加载，而不是依赖

对话记忆。

### 5.3.8 DLD 逐模块写作要点

#### M-01 影像接入

关键设计点：分片会话状态机（初始化→接收中→合并→验校→入库，失败可恢复）、位图幂等接收（乱序/重复分片）、10GB 稀疏文件处理、CGCS2000 转换的参数留痕。异常流重点：断点续传恢复、MD5 验校失败、磁盘满。测试要点：乱序分片幂等、10.2GB 边界、并发上传互不影响。

#### M-02 全自动分割

关键设计点：分块滑窗参数（块尺寸、重叠率）、跨块图斑合并算法（重叠区去重的几何规则）、图斑过滤方案（面积/置信度阈值的方案化管理）、进度百分比的可信计算（按块计数而非时间估计）。异常流：单块推理失败的重试与跳过策略、取消时的中间结果清理。精度设计：输出格式为精度评估（面积加权）预留接口。

#### M-03 交互式分割

关键设计点：会话模型（绑定影像范围+模型版本）、提示编码与 1024<sup>2</sup> 裁剪、多轮回退的提示栈管理、外部提示导入的格式校验。性能设计：全链路预算表（网络/编码/推理/矢量化/渲染各环节毫秒级分解）。异常流：Worker 熔断时的用户提示、会话超时清理、并发会话资源隔离。

#### M-04 文本提示分割

关键设计点：GroundingDINO 检测+SAM 分割的两级流水线、阈值参数的传递与默认值管理（0.30 为原型值，SRS 已标注可校准）、批量文本的顺序执行与模型复用、未检出兜底（返回空结果+规范提示而非报错）。设计警示：两级模型的版本组合必须绑定记录（哪版检测配哪版分割），否则结果无法溯源。

#### M-05 批量任务调度

关键设计点：GPU 独占语义用队列并发度=1 实现（不自研锁）、创建前校验清单（影像就绪/模型在线/参数合法）、进度回报通道（SSE 主+轮询备）、重试退避、归档转移、SMTP 通知模板。调度伪代码四要素：领取→校验→执行→回收，每步标注异常出口。这是 ADR-002 的直接落地章，写前重读 ADR。

## M-06 结果后处理

关键设计点：矢量化三格式输出的坐标一致性、规整算法参数化、撤销栈 $\geq 20$  步的内存模型、属性方案 JSONB 结构、复核标记的状态流转、面积统计的等积投影选择（误差 $\leq 3\%$  的实现根基）。顶点数熔断（ $>50$  万转离线）在此章写明阈值与理由（ADR-001 后果节）。

## M-07/08/09 要点

M-07 前端：图层管理器状态设计、卷帘对比的实现机制（双图层+裁剪渲染）、性能（瓦片缓存命中）。M-08 模型管理：注册元数据（含许可证字段）、版本状态机（注册 $\rightarrow$ 上线 $\rightarrow$ 下线 $\rightarrow$ 归档）、每类 $\leq 2$  在线版本的硬约束、熔断与降级触发逻辑。M-09 开放接口：Token 生命周期（签发/吊销/过期）、限流算法（令牌桶参数）、审计埋点位置、QGIS 联调要点。

## M-10 与横切设计

五类内置角色的权限矩阵实现（角色-操作二维表 $\rightarrow$ 代码常量，禁散落 if）、会话管理、日志审计装饰器（@audit 的切入点是服务层而非控制器——控制器层会漏掉内部调用）、系统参数的热更新与审计、备份恢复的演练脚本化。横切章还含统一异常（错误码表+trace\_id 贯穿）与部署构建（MTN-01/02 的机制保障）。

## 5.4 数据库设计：概念、逻辑、物理、运用四段法

### 5.4.1 为什么必须四段齐全

学生项目最常见的数据库设计是『直接建表』——只有逻辑设计，没有概念与物理设计。后果：实体关系说不清（概念缺失）、性能无保障（物理缺失）、运维无依据（运用缺失）。GB/T 8567 的数据库设计说明书要求四段齐全，澜图样例（《数据库设计说明书》V3.0）的章节映射：2 外部设计、3 概念设计、4 逻辑设计、5 物理设计、6 运用设计。

### 5.4.2 概念设计（人工主导）

方法：从 SRS 的名词中抽取实体候选，用『是否需独立管理、是否有独立生命周期』两问筛选。澜图案例筛出 12 个实体：users、roles、images、tiles、prompts、tasks、result\_sets、polygons、models、api\_tokens、op\_logs、system\_params。然后画 E-R 图并写评审取舍记录——样例 3.5 节记录了三个关键取舍（任务统一建模、提示与会话分

离、图斑属性 JSONB)，取舍记录是概念设计的灵魂，它比结论更有教学价值。

人工要点：实体命名用业务语言而非技术语言（tasks 不叫 job\_records）；每个实体写一句话业务定义；联系标注基数（1:N/N:M）。AI 辅助点：把 SRS 功能章节喂给 AI 做实体候选抽取（『列出本节所有需要持久化的名词及其生命周期事件』），人工筛选裁决。

### 5.4.3 逻辑设计（人机协同）

逐表展开：表名、业务定义、字段（名/类型/空否/默认值/含义）、主外键、约束。人工守住五条：① 范式到 3NF，反范式必须写理由（样例：polygons.attrs 用 JSONB 存可配置属性方案，理由 FR-06.4 属性方案可配置）；② 空间字段统一坐标基准（CGCS2000/EPSG:4490，对应 SRS 坐标条款）；③ 审计字段全覆盖（created\_by/at、updated\_by/at）；④ 枚举值与 SRS 术语一致；⑤ 外键策略明确（级联规则写清）。

AI 协同点：字段级草稿由 AI 生成（提示词附概念设计实体清单+相关 FR 原文），人工做范式检查与命名裁决。AI 常见毛病要盯：字段类型随意（该用 NUMERIC 用 FLOAT 存面积——面积误差 $\leq 3\%$ 的条款会被浮点误差破坏）、漏索引外键、枚举值自造词。

### 5.4.4 物理设计（人工定策略，AI 算参数）

四件事：存储规划（只读区与可写区分离——源影像只读、掩膜属可重算的中间成果、矢量成果需归档）、索引设计（每个索引注明服务的查询场景，如 GeoPackage 的 geom 字段 R-Tree 空间索引服务按范围检索）、一致性策略（无 DBMS 兜底时，原子写入、引用完整性校验、并发边界都必须在应用层显式设计）、关键操作的性能设计。容量估算让 AI 做算术：给输入参数（影像数量级、单景大小、日增任务数），AI 输出容量表，人工复核假设。

### 5.4.5 运用设计

数据访问层规范（禁拼接 SQL、事务边界规则）、迁移与版本管理（Alembic 迁移可重放：up/down/up 一致）、备份与恢复（对应 FR-10.5：每日备份+恢复演练）、监控告警。运用设计最容易被学生略过，但它直接支撑 MTN/REL 条款验收——部署 $\leq 2h$ （MTN-01）靠的是迁移脚本化，MTTR $\leq 2h$ （REL-02）靠的是备份恢复演练记录。

## 5.4.6 审查清单

- 概念：12 实体是否都能回答『谁创建、谁修改、何时归档/删除』？
- 逻辑：抽查 3 张表与 DLD 模块的数据操作声明是否一致；
- 物理：每个索引能说出服务的查询；op\_logs 分区策略与留存条款一致；
- 运用：迁移可重放有验证方法；备份恢复有演练计划；
- 全局：ER 图、数据架构图、表结构三者实体名完全一致（图实一致）。

## 5.5 接口设计与 OpenAPI 契约

### 5.5.1 定位：契约是研发与测试的共同语言

接口设计说明书面向人，OpenAPI 契约（openapi\_lantu\_v2.yaml）面向机器。两者必须同源：契约是唯一的结构化真相，文档是契约的叙述性解读。契约先行的价值在任务 4 体现得淋漓尽致——契约测试在端点实现前编写，前后端按契约并行开发，联调不再是『猜对方怎么实现的』。

### 5.5.2 人工方法：REST 接口设计要点

- 资源命名：名词复数、层级清晰（/images/{id}/tiles/{z}/{x}/{y}），动词交给 HTTP 方法；
- 异步任务：长耗时操作一律 202 Accepted + 任务 ID + 进度查询端点（全自动分割、批量、导出均如此）；
- 错误模型：HTTP 状态码表达传输层语义（400/401/403/404/409/429/500），业务错误码放响应体（code+message+trace\_id）；
- 鉴权与限流：开放接口用 Token（FR-09.1），限流触发 429 并带 Retry-After；
- 版本策略：URL 主版本（/v1/），契约文件名含主版本号，破坏性变更升主版本。

### 5.5.3 血泪教训：HTTP 状态码合法性

v1 契约曾把业务错误码（如 4001、5003）直接写进 responses 的 HTTP 状态位——这不是合法 HTTP 语义，契约校验器和代码生成器会直接报错。v2 校订：HTTP 层只用标准状态码，业务码放 description 与响应体。教训：契约写完后必须用工具做合法性校验（本课程用 ruamel.yaml 解析+状态码正则扫描，一行脚本的事，别靠肉眼）。

## 5.5.4 人机协同：契约生成与审查

AI 起草契约的提示词要点：给出六类接口清单（影像/分割/任务/结果/模型/系统）、鉴权模型、错误模型约定，要求输出 OpenAPI 3.0 YAML 且『每个端点 summary 标注 FR 编号』。人工审查四查：① 查端点覆盖——六类接口与 FR-09.1 清单一一对应；② 查状态码合法性与错误模型一致性；③ 查角色标注（x-roles 扩展字段）与权限矩阵一致；④ 查 schema 字段与 DB 设计表结构一致。四查都能脚本化，脚本入 CI。

## 5.5.5 契约测试：让契约活起来

契约不是文档，是可执行的测试资产。做法：用契约生成校验代码（T-04 任务），对假实现跑契约测试必须失败（验证测试有效性），对真实现跑必须转绿。CI 中契约测试是合并门禁之一。验收时，六类接口的契约测试全绿记录就是 FR-09.1 的直接证据。

## 5.5.6 QGIS 集成示例的价值

SRS 要求开放接口支持 QGIS 调用（FR-09.1）。设计中专门安排 QGIS 插件联调示例（T-30）：它的意义不在 QGIS 本身，而在『第三方视角验证接口可用性』——自己写的客户端调用自己的接口容易隐性耦合，用一个真实第三方工具的调用流程走一遍，接口设计的坑（认证繁琐、分页别扭、错误信息不可读）全部暴露。

## 5.6 ADR：把架构决策写成可审判的文书

### 5.6.1 为什么 ADR 是 AI 时代的刚需

AI 生成方案的速度极快，但『为什么选 A 不选 B』的论证不会自动产生。没有 ADR 的项目，三个月后没人知道某个选型的原因，AI 重构时会随意推翻隐性决策。ADR（Architecture Decision Record）用一页纸固化一个决策：背景、候选方案、决策、后果。Thoughtworks 技术雷达持续推荐 ADR 的理由同样适用于 AI 协作：决策可追溯是团队（含 AI 成员）治理的基础。

### 5.6.2 什么时候必须写 ADR

- 技术选型：存储、框架、执行模型（如 ADR-001 GeoPackage、ADR-002 同步执行）；

- 架构模式：进程模型、部署形态、一致性策略（如 ADR-003 常驻 GPU Worker）；
- 重大取舍：性能与成本、灵活与简单之间的权衡；
- 推翻既有决策时：写新 ADR 并标记旧 ADR 为『已取代』，保留历史。

### 5.6.3 ADR 的四段结构与写作要点

以样例 ADR-002（任务编排）为例拆解：

- 背景：写清需求的硬约束（FR-05.2 GPU 独占、PE-01 交互 $\leq 3s$ ），让候选方案的评估有标尺；
- 候选方案：至少 2 个，每个写清机制；被否决方案的否决理由必须具体（『自研调度器三类故障各需专门处理』是具体的；『不成熟』是不具体的）；
- 决策：一句话说清选什么、关键机制怎么映射到需求（队列并发度=1 天然实现 GPU 独占语义）；
- 后果：正面负面都要写。负面后果必须带缓解措施（GeoPackage 无并发写保护 → 单会话独占，并发保护登记二期）。只写正面的 ADR 是广告，不是决策记录。

### 5.6.4 反模式

| 反模式      | 症状           | 纠正                                 |
|----------|--------------|------------------------------------|
| 『因为我会这个』 | 决策依据是团队熟悉度   | 决策依据必须是需求条款与可验证指标（样例 ADR-001 未行备注） |
| 唯一候选     | 只写一个方案       | 强制 $\geq 2$ 候选；实在没有就写『维持现状』作为候选    |
| 否决理由空洞   | 『性能不好』 『太重了』 | 量化或机制化：性能差多少、重在哪几个组件               |
| 无负面后果    | 全是优点         | 每个决策必写 $\geq 1$ 条负面+缓解             |
| 决策后不回看   | ADR 写完即归档    | 评审会与变更时复查 ADR 假                    |



| 反模式 | 症状 | 纠正     |
|-----|----|--------|
|     |    | 设是否仍成立 |

### 5.6.5 人机协同

ADR 是 AI 辅助价值最高的设计文体，因为论证结构规整。流程：人工给定决策点与倾向 → AI 起草候选方案与对比维度 → 人工补充真实约束与裁决 → AI 反向攻击（『请扮演反对者，攻击本决策的三个最弱点』）→ 人工把攻击成立的部分写进后果与缓解。注意：AI 极易把否决理由写得礼貌而空洞（『方案 B 虽好，但综合考虑选择 A』），审查时凡是不能落到条款或数字的否决理由一律打回。

### 5.6.6 样例三份 ADR 的覆盖关系

ADR-001（存储）覆盖数据层决策，ADR-002（编排）覆盖任务层决策，ADR-003（进程模型）覆盖 AI 推理决策——三份合起来覆盖了本平台风险最高的三个技术决策点。教学要求 ≥2 份，评分时看的是论证质量而非数量。三份 ADR 在 v3 校订中还承担了一个功能：决策依据全部改为真实 FR/PE 编号后，它们成了 RTM 反向核查的锚点（设计元素→ADR→需求条款）。

## 5.7 非功能设计专题：GB/T 25000.10 逐项落地

### 5.7.1 非功能需求为什么总做不好

功能需求有天然的可观察性（点按钮看结果），非功能需求没有——『系统要快』没法演示。学生项目的 NFR 章节通常三行字：高性能、高可用、易用。评审时这三行等于零。GB/T 25000.10 把软件质量分解为八大特性（性能效率、兼容性、易用性、可靠性、信息安全性、可维护性、可移植性、功能适合性），非功能设计的任务就是：把 SRS 里每条 NFR 翻译成具体设计项，并标注它服务哪个质量特性。

### 5.7.2 映射表的编制方法

澜图 HLD 9.1 节的映射总表是非功能设计的核心载体。编制步骤：① 从 SRS 抽出全部 NFR（14 条：PE-01~09、SEC-01/02、REL-01/02/03、MTN-01~03、OSS-01、DOC-01 等）；② 逐条问『什么设计让它成立』——写不出设计项的 NFR 是假承诺；③ 标注 GB/T 25000.10 特性归类；④ 每条挂测试编号（PT/AT/ST/DT/CT）。四步做完，映射表自然成形。

### 5.7.3 性能效率：从指标到设计项

以 PE-01（交互点提示 $\leq 3s$ ，P95）为例示范分解：端到端 3s 预算 = 网络与渲染 1s + 推理 2s。推理 2s 预算的设计落点：常驻 Worker（ADR-003，消除模型加载冷启动约 20~60s）+ 1024<sup>2</sup> 裁剪（控制单轮计算量）+ 交互独立通道（不被批量任务排队阻塞，FR-05.2 的独占队列是批量侧的纪律，交互侧必须绕行）。三个设计项缺一不可——缺常驻则冷启动爆预算，缺裁剪则大图超时，缺独立通道则排队不可控。写性能设计时永远先列预算分解表，再逐项给机制。

### 5.7.4 可靠性：故障枚举法

可靠性设计的方法是对话式枚举：『它会怎么坏？』列故障清单——GPU 不可用（自动回退 CPU，REL-01）、模型可选依赖缺失（可用性前置检查 + extras 安装提示，REL-01）、磁盘满（临时目录监控 + 中间成果清理策略，REL-03）、误操作（FR-01.3 撤销重做、项目状态引用校验）。每个故障一行：故障→检测手段→影响→恢复手段→指标（REL-01 可用率 $\geq 99\%$ 、REL-02 MTTR $\leq 2h$ ）。枚举清单还可以直接喂给测试计划当故障注入用例的来源——设计与测试共用一张故障表，是测试左移的具体形态。

### 5.7.5 信息安全性：对标等保二级

安全设计按等保二级的控制域分层写（HLD 第 8 章）：网络层（HTTPS、内网隔离）、主机层（容器最小权限、镜像扫描）、应用层（五类角色 RBAC、Token 限流、输入校验对应 REL-03 异常拦截）、数据层（备份加密、日志留存 $\geq 180$  天）、管理层（审计报表、权限审批流程）。每层给措施+验证方式（ST-01 自查表/ST-02 渗透测试）。学生常犯错误是把安全写成单列一节的一段话——等保是按控制项核查的，不分层就没法逐项自查。

### 5.7.6 可维护性与可移植性

MTN-01（干净环境部署 $\leq 2h$ ）→ docker-compose 一键部署+迁移脚本化+部署手册；MTN-02（源码独立构建）→ 依赖锁定（requirements.lock）+构建文档；MTN-03（覆盖率 $\geq 70\%$ ）→ CI 覆盖率门禁。可移植性在国产化约束下具体化为：OS/数据库/中间件的国产替代清单与适配验证记录。这些条目写清楚后，交付验收的部署演示就是照本宣科——验收时的从容都是设计期写出来的。

### 5.7.7 人机协同要点

AI 起草 NFR 映射表效率很高（给它 SRS 非功能章节+模块清单即可），但有两个高频毛病：① 设计项写得太泛（『采用缓存技术提升性能』——必须追问缓存什么、在哪层、TTL 多少、命中率目标）；② 特性归类乱标（把安全性标到易用性）。审查时逐行问『这个设计项具体是什么机制』，答不出具体机制的行一律打回。

## 5.8 plan.md、tasks.md 与 Spec-Driven 生成链

### 5.8.1 两份文件的分工

plan.md 是组长的计划：里程碑、迭代、分工、风险联动、度量——回答『这七天我们怎么赢』。tasks.md 是全员的任务清单：36 个原子任务，每个带依赖、估算、角色、独立验证标准——回答『今天下午我具体做什么、怎么算做完』。前者管方向，后者管执行。

### 5.8.2 里程碑设计的要点

里程碑必须满足三可：可演示（达成标准是看得见的演示，不是『完成 80%』）、可验证（验证方式具体到测试编号）、可砍（每个里程碑知道失败后砍什么）。样例里程碑『分割三模式』：达成标准=FR-02.1/03.1/03.2/04.1 可演示，验证=AT-01 精度初测+PE-01 抽样实测——全是可观察事件。里程碑间距 1~2 天，7 天冲刺排 3~4 个：太密的里程碑失去缓冲意义，太疏则风险暴露太晚。

### 5.8.3 原子任务拆分规则

- 粒度≤0.5 人日：拆不动说明设计没做透，回炉 DLD 而不是硬拆；
- 依赖显式：每个任务列出前置任务编号，依赖图无环（AI 可辅助检查）；
- 独立验证标准：格式=『做什么动作→看到什么结果』，引用 TC/PE/AT 编号优先；
- 角色唯一：一个任务一个主责人，多人协作的任务拆成多个。

反例警示：v1 版 tasks.md 曾出现『训练任务提交+样本版本绑定』这类任务——它依赖一个不存在的需求域。任务拆分完毕后，必须用 RTM 做一次反向检查：每个任务的验证标准引用的编号在 SRS 里真实存在。

### 5.8.4 看板纪律

四列看板（To Do/Doing/Review/Done），Doing 在制品 $\leq 3$ （4 人组上限 4）。两个门禁：DoR（就绪定义）=依赖完成+验收标准明确，不达标不许进 Doing；DoD（完成定义）=验证标准通过+CI 绿+RTM 行更新，三者缺一不可不许进 Done。RTM 行更新写进 DoD 是关键——追踪矩阵的腐化都是从『回头再补』开始的。

### 5.8.5 人机协同：AI 拆任务，人做校准

把 DLD 模块清单+里程碑框架喂给 AI，要求按四规则输出任务表草稿。AI 拆任务的水平通常在 80 分：结构与覆盖不错，但估算普遍乐观、依赖关系常漏（尤其横切依赖：审计、异常处理这类人人都依赖的）。人工校准三件事：① 估算乘以团队系数（新手组建议 $\times 1.5$ ）；② 补横切依赖；③ 抽查验证标准的可执行性（『能演示』不算标准，『TC-F01-01 场景实测通过』才算）。

### 5.8.6 站会与燃尽

每日 15 分钟站会，只围绕看板：昨天移动了哪几张卡、今天动哪张、哪张卡住了。燃尽图由看板数据自动生成（T 任务完成数/日），连续 2 天无移动即触发进度风险（联动风险管理计划 R-05：砍非 MVP 项+启用缓冲日）。MVP 5 项是最后防线——砍范围时，非 MVP 的 44 条 FR 让位，MVP 寸步不让。

### 5.8.7 Spec-Driven 在设计关的落地：spec、plan、tasks 的生成链

#### *GitHub Spec Kit 思想的本土化*

Spec-Driven 开发的核心主张：规格（spec）是唯一真相源，计划（plan）与任务（tasks）由规格派生，代码由任务驱动。GitHub Spec Kit 给出了 specify $\rightarrow$ plan $\rightarrow$ tasks $\rightarrow$ implement 的命令链。本课程把这条链映射到中文工程文档体系：specify=任务 2 的 SRS+spec.md；plan=任务 3 的设计文档集+plan.md；tasks=tasks.md；implement=任务 4。理解这条链，就看懂了任务 3 为什么是这个形状——设计文档集就是 spec 与 tasks 之间的『人类可读+机器可读』双格式计划层。

#### *spec.md 在设计期的新角色*

任务 2 里 spec.md 是验收标准的载体；任务 3 里它升级为设计校验的输入：每个模块设计完成后，用 spec 场景做场景走查（6.4 节）；tasks.md 的验证标准列直接引用 spec 场景编号。spec 的任何变更（澄清裁决导致）必须当日联动 RTM 与设计——spec 变了设计没动，就是 5.13 节漂移事件的微型重演。

## **constitution：机器可读的团队共识**

Anthropic 工程指南的核心建议是：把规则写成 AI 每次执行前必读的文件，而不是依赖对话里的口头约定。澜图 constitution 的四部分（技术栈白名单/编码规范/设计约束/文档纪律）在任务 3 冻结，任务 4 生效。写 constitution 的原则：只写可机械检查的规则（『SQL 禁拼接』可检查，『代码要优雅』不可检查）；每条规则给反例；规则总数控制在 20 条以内——规则太多等于没有规则。

## **tasks.md 的防发散机制**

长流程开发中 AI 的发散（做着 T-16 顺手改了 T-13 的代码）是 Spec-Driven 的头号敌人。tasks.md 的防发散设计：① 原子任务≤半天——任务越小，AI 跑偏的窗口越小；② 每个任务的验证标准就是 AI 的完成定义，完成后必须自证；③ 看板 Doing≤3——人层面的 WIP 限制传导为 AI 的任务串行纪律；④ 每次 AI 编码会话开始先读 constitution+当前任务卡，结束勾选并更新 RTM 行。这套机制在任务 3 的 T-03（CI 流水线）就位，任务 4 直接运行。

## **验收标准（AC）贯穿链**

画一条链给学生背：招标条款 → FR（SRS）→ AC（spec.md）→ 设计元素（HLD/DLD）→ 任务（tasks.md）→ 测试用例（TC）→ 演示脚本（验收）。链上每一环的编号都是下一环的输入。断任何一环，验收时就有一个『这条怎么证明』答不上来。RTM 就是这条链的表格化；本课程所有文档编号体系（FR/AC/TC/PT/AT/T）都是为这条链服务的。

## **常见误解澄清**

- 误解：Spec-Driven 是『写完 spec 让 AI 全自动开发』。实际：spec 是锚点不是自动驾驶，每个环节都有人工裁决点；
- 误解：文档越全越重，AI 时代不需要。实际：恰恰相反——AI 放大了文档的杠杆（文档一次写对，AI 百次读对）；
- 误解：tasks.md 是项目经理的事。实际：在 AI 协作中，任务卡是每个人都直接操作的执行接口。

## **5.9 测试计划：测试左移的落地**

### 5.9.1 为什么测试计划在编码之前发布

传统项目把测试计划放在编码后期，结果是测试时间被开发挤占、用例与需求两张皮。测试左移要求：设计冻结之日测试计划同步冻结。三大收益：① 用例直接来自 SRS 验收标准，需求歧义在设计期暴露而不是在验收期；② 开发者拿到需求时同时拿到判定标准，『怎么算做完』不再有争议；③ 性能/精度等专项测试需要提前准备环境与数据（冻结影像集 D1 就要启动真值准备）。

### 5.9.2 GB/T 15532 章节结构与样例映射

样例《测试计划》（V3.0）的章节：1 引言（依据、对象、追踪载体）、2 测试级别与范围、3 测试策略（左移+专项）、4 进度、5 环境、6 准入准出、7 风险、8 交付物。编制时逐节对照 GB/T 15532 的要求项，不超纲不缺项。

### 5.9.3 编号体系：让用例自己说话

样例的编号设计：TC-Fxx-nn 按功能域编号（看到 TC-F01-01 就知道测影像接入）、PT-01~09 性能、AT-01~03 精度、ST 安全、DT 部署、CT 合规。好处：RTM 挂接机械可查（FR-06.6 → AT-03）；评审抽查高效（说编号即定位）；缺陷报告引用无歧义。编号规则写进配置管理计划，全员遵守。

### 5.9.4 专项测试的设计要点

| 专项          | 测法要点                            | 判定阈值来源                                |
|-------------|---------------------------------|---------------------------------------|
| 性能 PT-01~09 | 交互类 50 次抽样取 P95；批量类实测计时；并发用回放脚本 | SRS PE 条款原文数值                         |
| 精度 AT-01~03 | 冻结影像集（版本锁定+哈希登记）；面积加权指标与手算比对    | AC-FR-02.1-b / AC-FR-04.2-b / FR-06.6 |
| 安全 ST-01/02 | 等保二级控制项自查表+SCA 依赖扫描+渗透测试        | SEC-01/02，高危清零                        |
| 降级演练        | kill GPU Worker 注入故障，秒表计时切换过程   | FR-08.4/08.5、PE-09                    |

| 专项       | 测法要点             | 判定阈值来源               |
|----------|------------------|----------------------|
| 部署 DT-01 | 干净虚拟机一键部署计时，外人执行 | MTN-01 ( $\leq 2h$ ) |

专项测试的共同原则：阈值来自需求原文，测法写明样本量与工具，判定主体明确（谁测、谁签字）。『冻结影像集』是精度测试的命脉——影像集不冻结，精度指标就无法复测，验收时的争议无解。

### 5.9.5 准入准出与缺陷分级

系统测试准入四条件：单测覆盖率 $\geq 70\%$ （MTN-03）、CI 全绿、契约测试全过、MVP 5 项开发自测通过——达不到就不许开始系统测试，这是防止『带病进入测试期』的闸门。缺陷四级：致命（数据丢失/安全红线）修复前冻结一切新功能开发——这条纪律写在计划里，是为了工期压力下有据可依。

### 5.9.6 人机协同

AI 的价值点在两个环节：① 用例草稿生成——把 SRS 每条的验收标准喂给 AI，要求输出正例+反例用例草稿（编号、前置、步骤、预期），人工评审取舍；② 专项方案细化——给出 PE 条款原文与硬件基线，让 AI 起草测试脚本框架（如 50 次抽样计时的 Python 脚本）。人工守住：阈值不许 AI 改、冻结数据集由人登记哈希、准入准出裁决权在评审会。

## 5.10 项目管理计划与质量、配置、风险计划族

### 5.10.1 与任务 1 计划的衔接

任务 1 的《项目计划 v1》是投标期的承诺性计划；任务 3 的《项目管理计划》（V3.0）是开发期的执行性计划，由 v1 升级而来。升级的三处关键：① 范围基线从『投标承诺』切换到『需求确认书+SRS V2.0』；② 进度从 WBS 甘特图细化为 plan.md 里程碑+tasks.md 原子任务；③ 风险管理从登记册 v1 升级为 v2（触发信号更具体、责任到岗）。

### 5.10.2 范围管理：三道闸

样例《详细设计说明书》给出 AI 时代的范围控制三道闸，值得逐字理解：

- 闸一：新需求一律走变更流程（配置管理计划第 5 章），不得直接开发——AI 让加功能变得太容易，变更闸是唯一防线；
- 闸二：镀金禁令——无 RTM 来源列支撑的功能不开发。AI 编码时代的第一条纪律：AI 会『贴心地』顺手实现你没要求的功能，镀金不再是道德问题而是默认行为，必须用机制拦截；
- 闸三：范围蔓延熔断——任务清单超出 plan.md 基线 20% 时，组长召集范围评审，砍二期项保 MVP。

### 5.10.3 进度与资源管理要点

进度管理看板化（见 5.8 节），计划里要写清的是节奏制度：每日站会、每周风险评估会、里程碑评审会的时间与输入输出。资源管理的特殊项是 GPU：稀缺资源必须写使用纪律（样例：全自动/交互/文本推理共用 GPU，队列独占串行对应 FR-05.2，交互走独立通道保障 PE-01）——资源约束直接引用需求条款，避免组内抢资源的争议无据可依。

### 5.10.4 沟通管理

对内：站会+看板+周报复用模板；对外（甲方/评委）：里程碑演示邀请、澄清走正式记录、变更走 CCB。沟通计划的原则：重要事项必有书面载体（演示记录、澄清记录、变更单），口头共识不算数——这在验收争议时是唯一的保护。

### 5.10.5 验收与交付管理

计划里提前锁定验收方式：按需求确认书 MVP 5 项+13 条★款逐项演示，演示记录与验收标准编号一一对应。交付物清单与格式要求（DOC-01 双格式）在计划期就写入配置库目录规范，验收答辩前不做突击补料——『验收不过夜补材料，一切证据在过程中自然沉淀』。

### 5.10.6 人机协同

管理计划的模板化程度高，AI 起草效率极高：给出任务 1 计划 v1+SRS 基线+本节要点，AI 可一稿成形。人工的价值在三处：范围三道闸的条款必须组内逐条讨论达成共识（不是文本问题，是纪律承诺）；资源数字（人日、GPU 时长、AI 额度预估）必须人工填真实的；风险触发信号必须与当前项目实际挂钩（照抄模板的风险项一眼假）。



## 5.10.7 质量、配置、风险管理计划族

### 三册一体的关系

质量管理计划定『做得对不对』的标准与检查机制；配置管理计划定『东西放哪、怎么改、改了谁知道』；风险管理计划定『哪里会出事、出事了怎么办』。三册共用一套语言：基线、变更、度量。分开成册是因为执行责任人不同（质量官/配置管理员/组长），但评审时一起看。

### 质量管理计划要点

- 质量目标量化：覆盖率 $\geq 70\%$ （MTN-03）、契约测试全绿、PE 实测达标、精度 AT 达标——全部来自需求条款；
- CI 五道闸：lint  $\rightarrow$  单测  $\rightarrow$  spec 校验（check\_spec） $\rightarrow$  契约测试  $\rightarrow$  SCA 安全扫描，任一不过禁止合并；
- 评审制度：SRS 评审/设计评审/代码评审各有清单，评审记录归档；
- AI 产出专条：AI 生成代码与文档未经人工审查不得入库；采纳率纳入度量（不是越高越好，40%~75% 是健康区间，100% 说明没审查）。

### 配置管理计划要点

- 配置项全清单：需求基线（SRS/spec/RTM/确认书）、设计文档集、契约文件、代码、测试资产、计划族——每项有标识规范与基线状态；
- 基线建立时点：任务 2 冻结日建需求基线，设计冻结日建设计基线；基线变更必须过 CCB（变更控制委员会，本项目由组长+需求负责人+评委代表组成）；
- 版本规则：代码语义化版本；文档版本号+修订记录页；契约文件名含主版本（openapi\_lantu\_v2.yaml）；
- 归档结构：配置库目录规范（01\_需求/02\_设计/03\_度量/04\_测试/05\_交付），交付时整库移交。

### 风险管理计划要点

方法：概率 $\times$ 影响五级矩阵定级；每条风险四要素齐全——触发信号（可观察的事件，不是感觉）、责任人（到人）、应对措施（平时做什么）、预案（触发了做什么）。样例的高优风险：GPU 环境不可用（触发信号=T-02 验收失败）、交互时延超

标（触发信号=T-17 实测 P95>3s）。每周风险评估会复核全部风险状态；触发信号出现当日升级。风险登记册是活文档，v1→v2 的演进记录（哪些风险关闭、哪些新增、等级变化）本身就是评审素材。

人机协同与审查重点

三册的框架都可 AI 起草，人工灌注项目特异性。审查重点：① 质量计划的指标是否全部能在需求里找到出处（自创指标=验收争议的种子）；② 配置计划的基线清单是否与 RTM/设计文档的实际编号一致；③ 风险项的触发信号是否可观察——『进度可能延误』不是触发信号，『看板连续 2 天无移动』才是。

计划族的常见失败模式

| 失败模式      | 后果            | 对策                        |
|-----------|---------------|---------------------------|
| 计划写完即归档   | 执行无据，验收时补材料   | 度量数据每日进配置库，计划随里程碑滚动修订     |
| 指标无出处     | 验收时『你说达标就达标？』 | 每个指标标注需求条款编号              |
| 变更无记录     | 基线漂移无人察觉      | CCB 单+配置状态报告；本案例 v3 校订即范例 |
| 风险四要素缺斤短两 | 出事后互相推诿       | 每周复核会逐项检查四要素              |

5.11 RTM v3 挂接与从逆向分析到正向设计

5.11.1 从 v2 到 v3：挂接设计元素

任务 2 的 RTM v2 只有两列核心：需求→验收标准。任务 3 的 RTM v3（docx+xlsx 双格式）在此基础上回填五列：设计模块、HLD 章节、DLD 章节、数据表、API 端点、ADR——形成『招标条款→需求→验收标准→设计→测试』的完整链条。回填的规则：逐条 FR 对照设计文档找落点，找不到落点的需求是漏设计，找不到需求的设计是镀金，两者都是缺陷，都必须当日处理。

5.11.2 正向核查：需求有没有家

正向核查问『每条需求住在哪』。操作：遍历 24 条 FR+16 条 NFR+3 条精度指标，

逐条填写设计列。澜图 v3 的结果：24/24 FR 挂接、16/16 NFR 挂接、13 条★款全覆盖、MVP 5 项全覆盖。核查表的每一行都是评审证据——评委抽查任一行，能顺着编号在 1 分钟内翻到设计文档的具体章节，这就是『可追踪』的实操含义。

### 5.11.3 反向核查：家是不是合法

反向核查问『每个设计元素有没有出生证』。两个反向清单：① 模块清单 M-01~M-10 逐一指认需求来源（一一对应 F-01~F-10，无镀金模块）；② 数据表清单 12 张逐一指认（每张表至少服务一条 FR）。反向核查抓的是 AI 时代的高发病——AI 起草设计时会『顺手』加缓存层、加消息总线、加它认为合理的表，这些无源设计如果不拦截，都会变成工时黑洞和测试负担。

### 5.11.4 断链检查：RTM 的 CI 化

RTM 不是一次性文档，是持续维护的追踪资产。维护规则（写入 RTM v3 末）：任何需求/设计/测试变更须同步更新本矩阵；CI 每周执行断链抽查（抽样 15 行人工核对）；RTM 行更新是任务 DoD 的组成部分。断链率纳入度量（见 2.13 节）：断链率>5% 说明配置管理失控，应触发流程整改而不是闷头补表。

### 5.11.5 工具化：脚本生成与校验

澜图案例的 RTM v3 由脚本生成（t3/build\_rtm3.py 出 xlsx，build\_rtm3\_docx.py 出 docx）：需求行从 v2 提取（JSON 中间格式），设计映射表人工维护在脚本里，双向核查结论自动统计。好处：重生成零成本、无抄写错误、xlsx 与 docx 永远一致。教训同样来自实践：v3 的映射表初版引用了不存在的 HLD 4.3.6~4.3.10 小节，是构建后核对章节清单才发现的——脚本生成保证内部一致，但与设计文档的外部一致性仍需人工核对章节编号。

### 5.11.6 评审视角：RTM 怎么用

评委用 RTM 的三个动作：① 随机抽 3 行正向走查（从 FR 翻到设计章节再翻到测试用例，计时）；② 扫反向清单找镀金；③ 看演进记录（v1→v2→v3 的变更说明）。小组自查时先自己做这三个动作，评审就是开卷考试。

### 5.11.7 从逆向分析到正向设计：开源项目案例的用法

#### 教学案例的来路

澜图案例的需求基线来自对遥感分割类开源工具的逆向分析：阅读其源码与文档，提取能力清单（SAM 系列分割、文本提示分割、交互式标注、批量处理、遥感数据接入），再用正向工程的文档体系（招标书→SRS→设计）重新表达为一个完整平台。理解这个来路有助于把握一个重要分寸：逆向提供的是『能力证据』（这件事技术上做得通、生态选型是什么），不是『需求本身』——需求要按教学场景重新裁剪与定义。

## 逆向分析在设计阶段的三个用途

- 选型佐证：HLD 3.2 选型表中『GDAL/rasterio/GeoPandas——遥感分割开源生态反复验证的技术主线』，逆向分析给出的选型理由是『已被同类项目验证』，比『网上说好用』高一个证据等级；
- 工作量校准：读源码知道交互式分割的真实复杂度（提示编码、掩膜后处理、会话状态），任务估算不再拍脑袋；
- 风险识别：源码里发现的坑（大图内存、分块边界、坐标参考系混乱）直接进入风险登记册与 DLD 异常流设计。

## 逆向的纪律：不抄、不绑、不侵权

- 不抄设计：开源项目的内部结构是其作者在其约束下的选择，你的约束（国产化部署、等保、教学工期）不同，直接搬结构等于放弃设计论证；
- 不绑版本：引用其结论时记录版本与日期（『基于 v0.x 源码分析』），开源项目演进快，结论有时效；
- 不侵权：许可证审查先行（OSS-01）；案例项目改名（澜图）并重新设计文档体系，避免与原项目混淆。

## 学生自拟题时的『迷你逆向』

自拟题目的组也应做半天的迷你逆向：找一个与题目最接近的开源项目，读它的 README、目录结构、核心数据模型（models/schema）、API 路由，输出一页纸：它有哪些能力、技术栈是什么、数据模型几张表、哪些地方做得糙（你的机会点）。这页纸是任务 1 项目建议书与任务 3 选型的共同素材。评审时『你参考了哪个同类项目、借鉴了什么、规避了什么』是高频问题，有这页纸就能从容作答。

## 逆向成果的归档

逆向分析笔记（能力清单、选型观察、风险发现）归档配置库，命名如 reverse-analysis-v0.x.md。它是设计文档的合法引用源：『选型依据：逆向分析笔记第 3 节』比『参考了某项目』可追踪得多。本书配套工作区中的逆向分析笔记即为该文档的原始素材。

## 5.12 功能域演练与设计文档体检

### 5.12.1 演练说明

本节做一次手把手的完整演练：以 F-05 批量任务管理域为例，从 SRS 原文出发，产出该域的 HLD 段落、DLD 章节骨架、数据表、接口端点、ADR 需求判断、任务拆分与测试挂接。学生照此流程对自己的每个功能域各做一遍，任务 3 的 60% 工作量即告完成。

### 5.12.2 第一步：需求原文归集

从 SRS 复制 FR-05.1~05.6 原文与验收标准：创建前校验（05.1）、GPU 独占队列（05.2★）、进度监控（05.3）、失败重试（05.4）、结果归档（05.5）、SMTP 通知（05.6）。同时归集相关 NFR：PE-06（并发 1+30）。归集时连同条款的修订历史一起看——05.2 的 GPU 独占语义是澄清过的，澄清记录里有甲方对排队规则的具体期望，这些细节决定设计走向。

### 5.12.3 第二步：决策点识别与 ADR 判断

问：这个域里有哪些『可以这样做也可以那样做』的点？列出：① 任务编排用现成框架还是自研；② 进度回报用 WebSocket 还是轮询；③ GPU 独占用锁还是队列语义。然后判断哪些需要 ADR：规则是『不同选择会导致不同的需求满足方式或不同的风险画像』。①要（ADR-002）；②不要（决策太小，DLD 里一句话说明即可：SSE 主+轮询备）；③不要（它是 ADR-002 决策的直接推论）。ADR 饥渴症（什么都写 ADR）与 ADR 缺失症同样有害。

### 5.12.4 第三步：HLD 段落产出

按 34.4 的流程写法写 4.3.x 节：『批量任务创建（校验影像就绪/模型在线/参数合法，FR-05.1）→ 入 gpu\_queue（并发度=1，GPU 独占语义，FR-05.2★）→ Worker 执行并回报进度（SSE，FR-05.3）→ 失败按退避策略重试（FR-05.4）→ 完成归档结果

集（FR-05.5）→ SMTP 通知提交人（FR-05.6）』。一段话，六个 FR 全部点到，评委照单验收。这段话的人工写作时间约 15 分钟——前提是前两步做扎实了。

### 5.12.5 第四步：DLD 章节骨架

按五件套展开（要点见 5.3）。骨架先写类清单：TaskService（创建/校验）、QueueScheduler（领取/派发/回收）、ProgressReporter（SSE 推送）、RetryPolicy（退避策略）、ArchiveService（归档）、NotificationService（SMTP）。每类一句话职责。然后关键流程伪代码、异常流、测试要点。类清单是 DLD 的脊椎——脊椎歪了后面全歪，类清单必须组内互审后再展开。

### 5.12.6 第五步：数据与接口落点

数据：tasks 表（状态机字段 status、重试计数字段 retry\_count、归档字段 archived\_at——每个字段指认服务哪条 FR）。接口：/tasks 端点族（创建 POST、查询 GET、取消 DELETE、进度 SSE GET /tasks/{id}/events），OpenAPI 契约先行，x-roles 标注角色。每加一个端点，同步更新 RTM 该行的 API 列——顺手更新是 RTM 不过期的唯一秘诀。

### 5.12.7 第六步：任务与测试挂接

拆任务：T-21（创建校验+队列+进度）、T-22（重试+归档+通知）——每个 ≤0.5 人日、依赖明确、验证标准引用条款。挂测试：TC-F05-01（创建校验正例/反例）、PT-06（并发 1+30 场景）。最后回填 RTM 行：FR-05.x → M-05 → HLD 4.1/4.3.x → DLD 第 6 章 → tasks 表 → /tasks/\* → ADR-002 → TC/PT。一行写满，本域设计闭环。

### 5.12.8 演练验收标准

学生演练的判分点：① 六个 FR 在 HLD 段落里全部点名；② ADR 需求判断有理由（包括『不写 ADR』的理由）；③ 类清单职责无重叠无遗漏；④ RTM 行一次写满无空格。四项全对即掌握本关核心动作。

### 5.12.9 设计文档的体检方法

#### 体检总清单

任务 3 文档集冻结前，执行一次完整体检。清单分四层：一致性、合规性、可验证性、可追溯性。本清单由澜图案例的真实缺陷史提炼——每一类都对应一个真实踩

过的坑。

一致性检查

| 检 查 项      | 方法                             | 真实案例                                                      |
|------------|--------------------------------|-----------------------------------------------------------|
| 图 实 一致     | 图中每个实体名与正文/DB 表名逐一比对           | v2 数据架构图画着 annotations/training_tasks，实际模型是 prompts/tasks |
| 文 档 间 一 致  | 模块名、表名、端点名跨文档抽查                | RTM 映射表引用了不存在的 HLD 4.3.6~4.3.10                           |
| 数 值 一致     | 全文搜数值，与 SRS 原文对照               | 交互≤3s 在 SRS/HLD/DLD/测试计划四处须一致                             |
| 版 本 引 用 一致 | 搜所有文档引用编号（SRS V2.0、OpenAPI v3） | 配置计划残留 openapi_lantu_v1 引用                                |

合规性检查

- 标准合规：章节结构对标 GB/T 8567/15532，逐节核对不缺项；
- 契约合规：OpenAPI 用解析器+状态码正则扫描（v1 的非法四位状态码教训）；
- 许可证合规：选型表逐行核许可证，GPL 系红牌（OSS-01）；
- 红线合规：只下载不上传等合同红线，用静态检查（代码库搜回写类 API）验证。

可验证性检查

抽查任意 10 条设计陈述，问『这条怎么测/怎么演示』。可验证的陈述有数值、有机制、有判定主体；不可验证的陈述是形容词（『高性能』『友好的界面』『灵活的架构』）。形容词密度是文档质量的反向指标——全文搜索『高性能/灵活/先进/友好/强大』，每一处都要求改写为可验证表述或删除。

可追溯性检查

即 RTM 双向核查（见 2.10 节），此处强调执行时点：设计冻结前必须跑一轮完整双向核查，核查记录（抽查了哪些行、发现什么、怎么改的）归档配置库。本案例的 v3 校订就是一次完整的可追溯性修复，其记录可直接作为教学演示素材。

## 体检工具化

能脚本化的全部脚本化：禁用词扫描（grep 训练/样本库/编造编号）、状态码扫描、数值对照表、章节编号核对、docx 可打开性验证（用 Word COM 实测页数与可打开性，而非只看文件生成成功）。脚本入 CI 或一键批处理，体检从『靠人细心』变成『靠机制兜底』。体检报告（发现项、严重度、处置）归档——本案例的历次体检报告都在工作区留存，供学生参考格式。

## 5.13 样例剖析、基线漂移复盘与设计缺陷案例集

### 5.13.1 剖析方法

本节以 V3.0 设计文档集为对象做逐册点评：指出每册的教学亮点（可以直接模仿的写法）与已知不足（学生自己的项目中应当做得更好的地方）。阅读样例的正确姿势不是抄结构，而是问『这节为什么这么写』——理解了意图，才能在自己的题目里做出同等质量的判断。

### 5.13.2 概要设计

- 亮点 1：模块与需求域一一对应（4.1 节映射表）——RTM 挂接零歧义；
- 亮点 2：非功能设计单列第 9 章且逐项标注 GB/T 25000.10 质量特性——NFR 不再是口号；
- 亮点 3：2.3 节关键架构决策索引——把 ADR 的指针放进 HLD，评委一翻即达；
- 不足：业务流程仅覆盖 5 个（4.3.1~4.3.5），M-05/07/08/09/10 的流程欠奉——学生项目应补齐或说明裁剪理由。

### 5.13.3 详细设计

- 亮点：M-05 的调度伪代码把 FR-05.2 的 GPU 独占语义落实到『队列并发度=1』——需求到机制的直接翻译；



- 亮点：每模块末尾的单元测试要点是场景级的（『乱序分片到达时幂等接收』），可直接转测试用例；
- 不足：12 页篇幅偏薄，M-07/08/09 合并为一章要点——真实项目每模块都应独立成章；教学工期约束下的取舍，评委理解但会追问。

#### 5.13.4 数据库设计

- 亮点 1：概念设计的评审取舍记录（3.5 节）——把『任务统一建模』等三个关键取舍写成可讨论的决策；
- 亮点 2：6.5 节『与需求的最终对照』——DB 设计自带追踪小节；
- 不足：容量规划基于假设参数，教学中可接受，真实项目需实测数据支撑。

#### 5.13.5 接口设计（《接口设计说明书》与 OpenAPI 契约）

- 亮点：每个端点 summary 标注 FR 编号；v2 修复了 v1 的非法状态码——契约合法性有工具校验记录；
- 不足：示例报文偏少，学生项目应为核心端点配请求/响应示例。

#### 5.13.6 计划族（计划族样例文档（plan.md、tasks.md、《测试计划》与管理计划四册））

- 亮点：范围三道闸与镀金禁令——AI 时代的原创性纪律，值得全班推广；
- 亮点：tasks.md 的验证标准列全部可执行，且与 TC/PE/AT 编号挂钩；
- 不足：度量数据为预估值（AI 采纳率 40%~75% 目标），真实执行后应用实测值回填——这正是任务 4 的素材。

#### 5.13.7 总体评价

V3.0 文档集的最大价值不在任何单册，而在『链』：招标条款→FR→验收标准→设计元素→ADR→测试用例，每一环都能双向走通。学生对标时先查自己的链通不通，再抠单册的写法细节——链不通的文档集，单册写得再漂亮也是散装文档。

#### 5.13.8 反面案例：需求基线漂移事件复盘

##### 事件经过

本节复盘一个真实发生在本教学案例开发过程中的事件——它是本指导书最有价值的章节，因为犯错的正是案例的制作者（AI+教师组合），而发现错误的契机是用

户的一句追问：『这个设计和需求对得上吗？有没有需求追踪矩阵？』

时间线：① 设计样例 v1 基于早期口头约定的需求基线（含训练任务、样本库管理等）构建；② 任务 2 的正式 SRS 冻结为三分割模式体系（全自动/交互式/文本提示，24 条 FR），但设计样例未同步更新；③ v2 深化设计时继续在错误基线上叠加细节（甚至引用了编造的澄清编号 CL-04/CL-07/CL-09）；④ 追问触发核查，发现系统性偏差：设计的功能域与 SRS 的功能域根本性错位；⑤ 全部设计产物返工为 V3.0，RTM v3 从零建立双向追踪。

## 根因分析（五问法）

- 表象：设计文档与 SRS 功能域不一致。为什么？——设计基于旧基线；
- 为什么基于旧基线？——SRS V2.0 冻结时没有触发设计文档的同步流程；
- 为什么没有触发？——配置管理计划里基线变更的联动清单缺失（需求基线变更应强制列出受影响的设计项）；
- 为什么叠加到 v2 才发现？——中间没有任何双向核查动作，RTM v2 只有需求列没有设计列；
- 为什么会出现编造编号？——AI 起草时引用了『看起来合理』的澄清裁决，人工审查未核对编号真实性。这才是最深的一层：引用类内容必须核验存在性。

## 影响面评估

偏差不是局部的：四册设计文档、三份 ADR、plan/tasks、《测试计划》与《项目管理计划》、四张架构配图、OpenAPI 契约——几乎全部任务 3 产物都建立在错误基线上。返工代价约等于重做。这印证了一个工程常识：基线错误发现得越晚，修复成本越高；本事件在文档阶段被发现，若拖到编码阶段，代价将再放大一个量级。

## 修复过程（值得模仿的部分）

- 第一步不是改文档，而是重建 RTM：先把 24 FR+16 NFR 的真实基线提取为机器可读 JSON（rtm\_v2\_rows.json），再逐条挂接设计元素——先立追踪，再修内容；
- 修复按依赖序推进：数据模型（实体）→ 模块结构 → HLD → DLD → DB → API 契约 → ADR → 计划 → 测试计划；下游文档永远跟随上游修正；

- 配图全部程序化重绘（脚本生成），避免『图里还是旧名词』的残留；
- 修复后有终检：禁用词扫描、章节编号核对、Word 实测可打开性与页数——修复质量本身也要验证；
- 校订记录写入 RTM v3 引言——不遮掩偏差，把它变成可追溯的变更历史。

### 预防机制（进入全组规范）

| 机制          | 内容                               | 落点       |
|-------------|----------------------------------|----------|
| 基线变更<br>联动单 | 任何基线变更必须列出受影响配置项清单               | 配置管理计划   |
| 引用核验        | 文档中一切编号引用（CL/FR/TC/HLD 章节）须核验存在性 | 体检清单     |
| 双向核查<br>常态化 | 设计冻结前必跑 RTM 双向核查                 | RTM 维护规则 |
| 禁用词扫描       | 已淘汰的需求域词汇进 CI 扫描清单               | 体检脚本     |
| 诚实文化        | 发现偏差立即上报有奖，遮掩偏差追责                | 团队公约     |

### 给学生的启示

你们在通关实践中几乎必然遇到同类问题：需求澄清后忘了改设计、设计改了没动计划、AI 引用了不存在的条款。这个案例的价值在于示范了正确的处置姿态——不遮掩、先立追踪、按序修复、修复后验证、记录留痕。评审时，一组能说清『我们犯过什么错、怎么发现、怎么修、怎么防』的小组，永远比声称『我们没犯过错』的小组可信。

### 5.13.9 设计缺陷案例集：十个真实的设计缺陷故事

#### 案例集的使用

以下十个缺陷全部来自本教学案例的真实开发过程或历届学生项目的评审记录。每则按『症状→根因→修复→预防』四段展开。审查自己的文档时，把十则当检查表逐条自查——踩过这些坑的人才真正『会做设计』。

#### 案例一：基线漂移（全书第一大坑）

症状：设计的功能域与 SRS 不一致（训练/样本库 vs 三分割模式）。根因：SRS 冻结未触发设计同步。修复：V3.0 全面返工。预防：基线变更联动单+RTM 双向核查。详见 5.13 节。

## 案例二：编造的澄清编号

症状：ADR 引用 CL-04/CL-07/CL-09，澄清记录里实际不存在。根因：AI 起草时生成了『看起来合理』的引用，人工审查只读了语义没核验编号。修复：全部改为真实 FR/PE 条款引用。预防：引用核验入体检清单；AI 提示词加『引用编号必须来自附件基线，不确定列待裁决』。

## 案例三：非法 HTTP 状态码

症状：OpenAPI 契约把业务码 4001/5003 写进 HTTP 状态位。根因：错误模型两层（传输层/业务层）概念混淆。修复：HTTP 用标准码，业务码入响应体+description。预防：契约构建后跑状态码正则扫描（一行脚本）。

## 案例四：图实不一致

症状：数据架构图画着 annotations/training\_tasks，实际数据模型是 prompts/tasks。根因：图在旧基线时期绘制，模型改了图没改。修复：四张配图脚本化重绘。预防：图由脚本生成+图内文字进禁用词扫描范围。

## 案例五：RTM 引用了不存在的章节

症状：RTM 映射表写『HLD 4.3.6~4.3.10』，HLD 实际只有 4.3.1~4.3.5。根因：映射表由脚本生成，脚本里的章节号是人工猜的。修复：核对章节清单后修正脚本重建。预防：构建后跑章节交叉核对脚本；脚本生成只保证内部一致，外部一致必须机器或人工核对。

## 案例六：数值表述漂移

症状：同一指标在不同文档里写法不一（ $\leq 3s$  /  $< 3$  秒 / 3 秒内）。根因：多文档多作者无对照习惯。修复：数值对照表+全文统一。预防：数值以 SRS 为唯一源，其他文档引用条款号而非复述数值（复述必漂移）。

## 案例七：撤销栈深度无出处

症状：某届学生设计写『支持撤销』，无深度无数值。根因：需求里有数值（ $\geq 20$

步）没复述进设计。修复：DLD 写明内存撤销栈+容量 20 硬限+关联 FR-06.3。预防：数值复述检查（搜 SRS 全部数值，逐个确认设计中有落点）。

### 案例八：异常流缺失

症状：分片上传只有阳光路径，断点恢复只字未提。根因：作者按演示路径设计，没按故障枚举设计。修复：按 26.4 故障枚举法补全。预防：五件套审查时专门数异常流条目数，异常流少于主流程一半的模块打回。

### 案例九：任务估算系统性乐观

症状：AI 拆的 36 个任务全部 0.5 人日以内，实际执行一半超时。根因：AI 对『新手组+新领域』的摩擦成本无感知。修复：引入团队系数×1.5 与横切依赖补全。预防：首个迭代实测校准系数，后续迭代用实测系数重估。

### 案例十：体检只看生成成功

症状：构建脚本输出 OK 即认为文档完成，评审前夜发现 Word 打不开（表格闭合错误）。根因：把『文件生成』当『文档有效』。修复：Word COM 实测可打开性与页数。预防：构建流水线最后一步永远是真实打开验证，自动化之。

### 案例集的元教训

十则案例的共同模式：缺陷都不是『不会』，而是『没查』。设计质量的差距不在写作能力，在检查机制的密度——每一类缺陷都对应一条可自动化的检查规则。把教训转化为规则、把规则转化为脚本，是工程能力复利的方式。

## 5.14 过关自检与交接

### 5.14.1 任务 3 交付包结构

任务 3 评审提交的不是散装文件，是一个结构化交付包：

```
t3_交付包/
├─ 00_封面与清单.docx          （交付物清单+版本+页数实测记录）
├─ 01_设计文档/ 设计文档集（概要设计/详细设计/数据库/接口四册） + OpenAPI 契约 yaml
├─ 02_决策记录/ 三份 ADR（存储选型、任务执行模型、推理进程模型） ADR
├─ 03_计划/      plan.md 与 tasks.md + 《测试计划》与管理计划族
├─ 04_追踪/ RTM v3 docx+xlsx
├─ 05_证据/      体检报告、双向核查记录、评审预读意见处理表
└─ 06_工具/      构建脚本与配图脚本（t3/ 目录快照）
```

清单页是关键创新：每份交付物一行，标注版本号与 Word 实测页数——实测页

数证明文档真实可打开，这是把『体检通过』显性化的方式。

5.14.2 评审陈述 PPT 的结构

10 分钟陈述建议 8 页：① 需求基线一句话（24 FR + 16 NFR + 3 AT、MVP 5 项、★13 条）；② 总体架构一页图；③④⑤ 三个 ADR 各一页（只讲取舍与依据）；⑥ 非功能映射表节选；⑦ RTM 双向核查结论；⑧ 计划与风险（里程碑图+TOP3 风险）。禁忌：念目录、贴大段文字、展示 AI 对话截图充数。

5.14.3 通往任务 4：设计基线冻结清单

- 设计评审裁决书归档（通过/有条件通过+整改关闭证据）；
- 基线公告发布（配置管理员，含全部配置项版本号）；
- tasks.md 首批任务 DoR 达标（依赖完成+验收标准明确）；
- CI 五道闸就绪，体检脚本入库；
- rules/constitution 文件生效验证（AI 编码工具实际加载）；
- 待裁决清单零开放项。

5.14.4 任务 4 预告：设计在编码期的活法

设计文档不是交付即死：编码期每个 PR 引用 DLD 章节号；发现设计错误走变更不改基线之外的自由发挥；测试用例从 spec+DLD 双源生成；RTM 挂接测试用例列（T-35）。任务 4 结束时，RTM v4 将完成『需求→设计→代码→测试』的全链——任务 3 打下的追踪骨架在那一刻兑现全部价值。

5.14.5 任务 3 常见时间陷阱

| 陷阱     | 症状            | 对策                        |
|--------|---------------|---------------------------|
| 追求完美   | HLD 改到第七版还没评审 | 冻结点是评审会，不是自我感觉；先评审再迭代     |
| 轻视计划族  | 管理计划最后一晚突击    | 计划族框架 D1 就立，内容随设计滚动填充     |
| RTM 后补 | 设计冻结后补挂接      | 回填与设计同步，T 任务 DoD 含 RTM 更新 |

| 陷阱   | 症状         | 对策               |
|------|------------|------------------|
| 体检太晚 | 冻结前夜发现打开失败 | 体检脚本从第一份文档起就进 CI |

### 5.14.6 结语前的一页自查

提交前最后一晚，全组一起回答六个问题：① 随便抽一条 FR，我们能在 1 分钟内翻到它的设计、表、接口、测试吗？② 我们的三个 ADR，被否决方案的理由能背出来吗？③ SRS 里每个数值，设计里都有落点吗？④ 图里的每个名词，正文里都有吗？⑤ 待裁决清单清零了吗？⑥ 如果评委问『这段是 AI 写的吗』，我们能拿出审查记录吗？六个『能』，安心睡觉；有一个『不能』，明早改完再交。

### 5.14.7 HLD 自查卡

- 设计输入基线清单完整且版本正确？ □ 模块与需求域一一对应？ □ 选型表每行有理由+许可证？ □ 每张图与正文实体名一致？ □ 非功能映射表覆盖全部 NFR 且标注 25000.10 特性？ □ ADR 索引与实际 ADR 一致？ □ 演进清单与 SRS 二期标注一致？

### 5.14.8 DLD 自查卡

- 每模块五件套齐全？ □ 异常流条目≥主流程一半？ □ 全部数值与 SRS 一致？ □ 字段名在 DB 设计中存在？ □ 端点在 OpenAPI 中存在？ □ 待裁决清零？ □ 测试要点可直达用例？

### 5.14.9 DB 自查卡

- 12 实体两问（谁创建/何时归档）均可答？ □ 反范式处有理由？ □ 坐标基准统一？ □ 索引有查询场景？ □ 分区与留存条款一致？ □ 迁移可重放已验证？ □ 备份恢复有演练计划？

### 5.14.10 接口自查卡

- 契约解析通过？ □ 状态码全部合法？ □ 六类接口与 FR-09.1 对应？ □ x-roles 与权限矩阵一致？ □ 异步端点有进度查询？ □ 限流有 429+Retry-After？ □ 核心端点有示例报文？

### 5.14.11 计划族自查卡

- ☐ 里程碑三可？ ☐ 任务四要素齐全且依赖无环？ ☐ 测试专项有测法有阈值出处？  
☐ 质量指标全部有需求出处？ ☐ 配置基线清单与文档实际一致？ ☐ 风险四要素齐全且触发信号可观察？

#### 5.14.12 RTM 自查卡

- ☐ 55+14 全挂接？ ☐ 反向无镀金？ ☐ 引用章节号真实存在？ ☐ ★款与 MVP 全覆盖？ ☐ xlsx 与 docx 同源生成？ ☐ 维护规则与 CI 抽查已声明？



## 第 6 章 第四关：研发冲刺

本关是研发主体：两个迭代冲刺，强制 PR 门禁（AI 生成→提交者自查→队友评审→CI 绿→合并），测试护栏与安全红线，站会、燃尽图与变更管理。AI 写的代码与人工代码同标准评审。

### 6.1 关卡导论与冲刺总方法论

#### 6.1.1 任务 4 在全生命周期中的位置

任务 1~3 解决了『做什么、怎么做』并冻结了全部基线；任务 4 是把设计变成可运行软件的研发冲刺——它是工作量最大、协作密度最高、AI 参与最深的一关，也是管理纪律最容易崩塌的一关。工期压力下，『先跑起来再说』的诱惑无处不在：跳过测试、绕过评审、AI 产出直接合并。本关的训练目标恰恰是在真实工期压力下保持工程纪律——PR 门禁、测试护栏、变更流程，一个都不能少。

#### 6.1.2 两个迭代冲刺的安排

冲刺窗口为 D9~D15（8/20~8/28，7 个工作日，全课程最长的单关窗口），分两个迭代：迭代一（D9~D11）打地基并完成影像接入与首批功能；D11 末中期评审（范围锁定点）；迭代二（D12~D15）完成分割三模式、后处理与收口。任务清单以 tasks.md（36 个原子任务，含 CR-001 引入 1 项）为准，看板驱动，每日站会 15 分钟。

#### 6.1.3 交付物清单

| 类别    | 交付物                | 样例                      |
|-------|--------------------|-------------------------|
| 代码    | 可运行仓库（含测试、CI、脚本）   | sprint_repo/            |
| 过程证据  | PR 记录（自查+评审+CI 证据） | PR#47 评审样例 PR #47       |
| 管理记录  | 迭代计划与燃尽、站会纪要、周报    | 《迭代冲刺计划与燃尽报告》、《站会纪要与周报》 |
| 变更记录  | 变更单（四步流程全链）        | 变更单 CR-001 CR-001       |
| AI 资产 | Prompt 模式库、翻车记录    | 《Prompt 模式库》、《AI 翻车记录》  |
| 追踪    | RTM v4（代码/PR/测试挂接） | RTM v4 xlsx             |

#### 6.1.4 过关条件

- MVP 5 项需求全部可演示，验收标准逐条有测试证据；
- 全部 PR 走五环门禁（AI 生成→自查→Approve→CI 绿→合并），无直推 main 记录；
- 覆盖率门禁达标：总体 $\geq 70\%$ ，核心模块 $\geq 85\%$ （MTN-03），增量代码有覆盖率约束；
- 安全红线零违规（口令不明文、SQL 参数化、输入校验）；SCA 高危清零（SEC-02）；
- 《Prompt 模式库》 $\geq 8$  个模式、《翻车记录》 $\geq 1$  则完整根因分析；
- 变更全部走四步流程：RTM v4 双向闭环；每人至少一张跨方向联调单。

### 6.1.5 本指导书怎么用

1.2 节讲方法论；第 3~13 章讲工程实践的『人工怎么做+AI 怎么帮』（PR 门禁、评审、测试、安全、性能、精度、演练）；第 14~17 章讲管理与追踪机制；第 18~24 章讲 AI 协作的深水区（模式库建设、翻车分析、编码提示词、AI 缺陷模式、上下文管理、重构）；第 25~27 章剖析样例；第 28~41 章是教师指南、度量、迁移、工具链、FAQ、练习、日程与附录。每章紧扣澜图案例的真实产物（sprint\_repo 代码仓、PR #47、CR-001）。

与前三关的关系：任务 3 指导书教你『写出可实施的设计』，本册教你『把设计不打折扣地写出来』。设计期建立的所有纪律（constitution、tasks.md 验证标准、RTM）在本关全部进入执行态。

### 6.1.6 研发冲刺总方法论

#### ***Spec-Driven 的执行段：implement***

GitHub Spec Kit 的命令链 specify→plan→tasks→implement 中，任务 4 是 implement 段。执行段的核心规则：AI 编码的唯一输入是 spec.md（验收标准）+DLD（设计约束）+constitution（团队规则），不允许直接根据口头描述或聊天记录写代码。每个 AI 编码会话的启动仪式：读 constitution → 读当前任务卡（T-xx 含验证标准）→ 读相关契约/设计片段 → 开始工作。

#### ***五环 PR 门禁的逻辑***

门禁五环各有不可替代的功能：AI 生成（效率来源）→提交者自查（第一道质量

闸，清单驱动）→队友 Approve（第二道闸，人审 AI 不可省）→CI 绿（机器闸，五道检查）→合并（配置管理动作，关联任务与 RTM 更新）。五环的哲学：AI 代码与人工代码同标准——不因为『AI 写的』而降低审查，也不因为『AI 写的』而推定有错；审查的唯一对象是代码本身与它的证据。

## **测试护栏：左移的执行形态**

任务 3 的测试计划在任务 4 落地为三条硬规则：① 核心逻辑测试先行或同 PR——先推失败的用例（红）、再推实现（绿），CI 记录里的先红后绿就是证据；② 覆盖率门禁 CI 强制执行，不靠自觉；③ 安全红线静态检查进 CI——红线问题（明文口令、SQL 拼接）在评审人看到之前就被机器拦下。

## **主副修机制**

每人主攻一个方向（后端/前端/测试/DevOps），同时必须完成至少一次跨方向联调。机制的目的：防止『四人四个孤岛、联调期爆炸』。联调以联调单为载体：接口契约版本、双方自测证据、问题清单与关闭记录。联调单是过关材料，也是评审时『协作真实性』的证据。

## **变更管理：四步流程**

研发期任何偏离基线的改动（需求澄清、阈值调整、设计修正）走同一流程：改 spec → 同步 SRS → 改 plan/tasks → 改代码。顺序不可颠倒——先改代码后补文档等于没有流程。变更单 CR-001 CR-001 是完整范例：一个常量（阈值 0.30→0.24）的变更，走完四步耗时 1.5 小时，换来验收零争议。

## **AI 资产管理**

两类资产必须在冲刺中沉淀：《Prompt 模式库》（好提示词复用，收录标准=≥2 次复用+证据）与《翻车记录》（AI 失败的根因分析，≥1 则，无记录扣分——零翻车说明没有深度使用或没有审视）。前者是效率复利，后者是风险疫苗。两者都是评审的必查项。

## **6.2 冲刺启动：环境、分支与 CI 就绪**

### **6.2.1 启动日的就绪清单**

冲刺第一天的唯一目标是『让协作基础设施可信』。就绪清单：① 仓库初始化

（目录规范、rules 文件、pre-commit 钩子）；② docker-compose 最小部署四服务健康全绿；③ CI 五道闸可用（故意提交坏代码验证拦截）；④ 契约测试骨架就位（假实现能被识别失败）；⑤ 看板建立、首批任务 DoR 核对。对应 tasks.md T-01~T-04。这一天不写任何业务代码——基础设施不可信时写的代码，后面都要返工。

### 6.2.2 分支策略

教学规模推荐 trunk-based 简化版：main 为受保护主分支（禁止直推），feature/T-xx-简述 短分支（生命周期≤2 天），squash 合并保持历史线性。长寿命分支（>3 天）是合并地狱的入口；分支命名带任务号，看板与 git 历史天然互查。反模式：每人一条长期分支『最后合并』——那是把集成风险攒到交付前夜。

### 6.2.3 pre-commit 与本地闸

本地闸在 CI 之前拦截，反馈环最短：lint（ruff）、格式化、安全红线扫描（security\_redline.py）、commit message 规范（type(scope): 摘要+任务号）。配置后全组统一拉取。本地闸的原则：能在本地拦的不留给 CI，能在 CI 拦的不留给评审人——闸越靠前，成本越低。

### 6.2.4 CI 五道闸详解

| 闸            | 内容                                       | 拦截案例（真实发生）               |
|--------------|------------------------------------------|--------------------------|
| Gate1 lint   | ruff 静态检查                                | 未使用变量、危险语法               |
| Gate2 单测+覆盖率 | pytest+coverage_gate.py（总体 ≥70%/核心 ≥85%） | T-22 首推覆盖率 68% 被拦        |
| Gate3 引用核验   | check_refs.py: FR/TC/T 编号须存在于基线          | 编造编号（预防基线漂移）             |
| Gate4 契约测试   | contract_check.py: 状态码合法+FR 标注+x-roles   | 本案例修复的 11 处标注缺口          |
| Gate5 安全扫描   | pip-audit+security_redline.py（R1~R4）     | requirements 引入高危 CVE 依赖 |

### 6.2.5 CI 失败的处置纪律

CI 红不许强行合并（管理员权限也不许——配置管理计划写死）；CI 红超过 30

分钟未定位，升级给全组（站会通报）； flaky test（时好时坏的测试）当日处理：要么修要么隔离并登记，不许『重跑碰碰运气』——flaky 测试对 CI 信任的腐蚀比没有测试更快。

## 6.2.6 环境一致性

依赖锁定（requirements.lock）、Python/Node 版本写入 README、docker-compose 固化服务版本。新人入组测试：按 README 半小时跑通全部构建与测试——跑不通就是 README 缺陷，当场修。教学环境的经典灾难『我机器上能跑』，用这一仪式根除。

## 6.3 PR 门禁五环与 AI 代码评审

### 6.3.1 环一：AI 生成的开工仪式

每个编码任务的标准启动：开新对话（不在旧对话尾部续做新任务）→ 喂入四件套（constitution、任务卡 T-xx、相关 spec/契约/设计片段、反镀金条款）→ 先要实现计划不要代码（『先给实现步骤与文件清单，我确认后再写』）→ 确认后生成。开工仪式 3 分钟，省下的返工以小时计。任务卡的验证标准在此刻读给 AI：『完成定义=以下用例通过』。

### 6.3.2 环二：提交者自查

自查清单（样例 PR#47 评审样例 §1 的七项）：需求映射（无镀金）、契约一致、错误模型、测试先行、安全红线、性能埋点、RTM 更新。执行要点：① 清单逐项给证据，不打空勾（『契约一致』要贴 contract\_check 输出）；② AI 自审（P-08 模式）可以先用，但人工逐项过不可省——《迭代冲刺计划与燃尽报告》统计显示自查清单缺项是 PR 打回首因；③ 采纳率随手记录（保留行/总行），进度量表。

### 6.3.3 环三：队友 Approve

评审人指派规则：互评配对固定（甲↔乙、丙↔丁），跨方向抽查每周至少 1 次。评审时限：工作日内 4 小时响应。评审只问四类问题：正确性（逻辑与异常流）、合规性（契约/错误码/红线）、可维护性（命名/结构/注释）、证据充分性（测试与 CI）。Approve 的含义是『我愿为这段代码背书』——放水 Approve 在复盘时追溯。

### 6.3.4 环四：CI 绿

CI 五闸见 3.4。本环的纪律只有一条：绿了才准合并，任何理由的跳闸合并都是配置管理事故。CI 排队拥堵时（冲刺末常见），按里程碑优先级排队，不是按谁催得急。

### 6.3.5 环五：合并与归档

squash 合并，commit 信息含任务号与 FR 号；合并后三分钟动作：任务卡置 ☒、RTM v4 行更新、燃尽数据点入库。这三个动作写在 DoD 里——『合并了但追踪没更新』不算完成。

### 6.3.6 门禁的成本与收益测算

单 PR 门禁成本约 25 分钟（自查 10+评审 15），本冲刺 28 个 PR 合计约 12 人时。收益侧：CI 拦截 9 次缺陷（每次若流入验收期，定位+修复+回归成本 $\geq 2$  人时），镀金拦截 2 次（避免无源功能蔓延），评审知识扩散（全组对彼此模块的熟悉度显著提升）。门禁不是开销，是保险——验收答辩时，PR 记录链就是『过程受控』的直接证据。

### 6.3.7 代码评审：怎么审 AI 写的代码

#### 评审 AI 代码与人工代码的同与不同

标准相同（同一份审查清单），重点不同。人工代码的典型问题是笔误与疲劳错误；AI 代码的典型问题是『自信的系统性偏差』：编造不存在的 API、引用不存在的编号、估算性数值、过度工程化（顺手加抽象层）、常识替代需求（撤销配重做）。评审 AI 代码时，注意力应优先投向这五类，而非逐行读语法——AI 的语法几乎不犯错，错的是事实与意图。

#### 评审五查

| 查   | 问题                  | 工具/方法                   |
|-----|---------------------|-------------------------|
| 查事实 | 引用的 API/编号/参数真实存在？  | 对照契约与设计文档；check_refs.py |
| 查边界 | 空值/越界/并发/超时的处理在吗？   | 异常流清单对照 DLD             |
| 查意图 | 实现的是需求要的，还是 AI 以为的？ | 逐函数指认 FR 编号（反镀金）        |

| 查   | 问题               | 工具/方法                |
|-----|------------------|----------------------|
| 查数值 | 常量有出处吗（需求条款或实测）？ | 数值溯源：无出处即打回          |
| 查测试 | 断言真有效吗（恒真断言识别）？  | 变异思维：把实现改错一处，看用例是否变红 |

## 评审意见的艺术

评审意见分三级标注：blocker（必须改，合并不了）、suggestion（建议改，作者裁决）、question（求澄清）。意见写『为什么』不写『应该怎样』——『这里并发下会丢任务（blocker），因为 dispatch 与 complete 之间无原子性』比『这里改一下』有用十倍。收到意见的作者先回答问题再改代码，24 小时内响应。PR#47 评审样例 §3 的评审对话是标准语气示范。

## 评审节奏与防疲劳

单次评审不超过 400 行 diff、不超过 60 分钟——超了质量断崖。大 PR 拆分原则：按任务卡边界拆，一个 PR 只干一件事（T-36 的教训：顺手改公共签名导致 2 小时返工）。冲刺末的评审拥堵预案：预留『评审对子时间』，评审人把手头任务暂停权交给组长协调。

## AI 辅助评审

评审人可用 AI 做预筛（P-07/P-08 模式）：让 AI 对 diff 提质疑清单，评审人逐条裁决。注意分工——AI 预筛覆盖广度（不漏文件），人审覆盖深度（意图与事实核验）。评审记录里标注『AI 预筛发现 n 条，人工另发现 m 条』，这组对比数据本身就是 AI 能力边界的实证材料。

## 6.4 测试先行与测试护栏

### 6.4.1 先红后绿的标准节奏

核心逻辑的标准开发节奏：① 从 spec 场景+DLD 测试要点生成用例清单（P-04 模式）；② 用例清单人工确认（编号、正反例覆盖）；③ 先提交失败用例（CI 红，截图归档）；④ 提交实现（CI 绿）。先红后绿不是仪式——红的证据说明用例真的会失败，即测试是有效的测试。同 PR 同推（测试与实现一起）允许但须在 PR 注明理由，仅限

小修。

6.4.2 哪些必须先行，哪些可以后补

| 类别                     | 策略              | 理由             |
|------------------------|-----------------|----------------|
| 核心算法（分块拼接/矢量化/面积统计/位图） | 强制先行            | 逻辑密集，回归成本最高    |
| 状态机与调度（gpu_queue）      | 强制先行            | 非法迁移必须有用例看守    |
| API 端点                 | 契约测试先行，业务用例同 PR | 契约是前后端并行开发的前提  |
| 前端交互                   | 关键路径后补，演示前补齐    | 投入产出比低，用演示脚本兜底 |
| 胶水代码（配置/启动）            | 可后补             | CI 冒烟覆盖        |

6.4.3 用例来源链

每条用例必须能回答来源：spec.md 的 AC（主源）、DLD 的单元测试要点（设计源）、故障枚举表（异常源）、翻车记录（教训源——每次翻车至少沉淀一条防复发用例）。四源之外『凭感觉』写的用例鼓励但不计入覆盖率目标的达成论证——验收时只认有来源链的用例。

6.4.4 用例质量标准

- 断言到业务码与数据，不只断言『没报错』——反例断言 `code==1001` 而非 `raises` 即过；
- 边界值齐全：空、最小、最大、越界（撤销 0/1/20/21 步四值法）；
- 独立性：用例间无执行顺序依赖，可任意单跑；
- 可读性：用例名即场景（`test_rollback_rounds`），注释挂 TC 编号；
- 恒真检测：AI 生成用例必查 `assert True` 变体与『catch 后无断言』（翻车台账 L-2）。

6.4.5 测试护栏的 CI 固化

护栏不是口头要求而是 CI 门禁：Gate2 覆盖率（`coverage_gate.py`）、Gate3 引用核



验、Gate4 契约、Gate5 安全。护栏的有效性验证：冲刺初期故意提交一个含明文口令+无测试的分支，观察五闸的拦截位置与报错信息——这个演练（T-03 的一部分）让全组对护栏产生真实信任。

## 6.5 pytest、契约测试与覆盖率门禁实操

### 6.5.1 以 sprint\_repo 为教具

本节以样例仓 tests/ 两个文件为教具逐段精讲。学习目标：会写结构正确的 pytest 用例、会用参数化做反例组、会用 fixture 隔离状态、会读覆盖率报告。先跑通 selfcheck.py（无 pytest 环境的等价断言脚本，11/11 通过记录已归档），再按 README 安装 pytest 运行正式用例。

### 6.5.2 fixture 与依赖注入

test\_interactive.py 的 svc fixture 每次返回全新 Service——状态隔离靠 fixture 默认函数级生命周期。Service 构造函数接收 inference\_gateway 参数（依赖注入），测试注入 FakeGateway——这是『可测试设计』的关键：依赖外部资源（GPU Worker）的组件必须可替换。写实现时就问『测试时这个依赖怎么换』，答不上来的设计要改。

### 6.5.3 参数化反例组

@pytest.mark.parametrize 把四类非法输入压成一条用例族（坐标维度错/类型非法/label 非法）。参数化的价值：新增边界情形只加一行参数，且报告里每种情形独立计数。注意断言到 http\_status==400 且 code==1001——错误模型的双层断言（HTTP 层+业务层），这正是 v1 契约状态码事故教给我们的。

### 6.5.4 状态机测试：穷举非法迁移

test\_gpu\_queue.py 的 TC-F05-09 演示状态机测试法：对每个状态，尝试全部不允许的迁移，逐一断言拒绝。gpu\_queue.py 的 \_ALLOWED 表驱动设计让这种穷举成为可能——状态机测试的完备性取决于迁移表的显式程度（P-02 模式的动机）。

### 6.5.5 时间敏感逻辑的测试

backoff\_seconds 是纯函数，直接断言序列 [30,60,120]。含 time.time() 的逻辑（archived\_at）只断言『已写入』不断言具体值——时间值注入或断言存在性，二选一，不要 sleep 等待真实时间（那是 flaky 测试的温床）。

### 6.5.6 覆盖率报告怎么读

coverage xml/html 三列看：行覆盖率（总体 76%）、分支覆盖率（状态机类模块重点）、未覆盖行清单（逐行指认：是异常分支没测，还是死代码）。未覆盖行分三类处置：补用例（异常分支）、删代码（死代码）、标注豁免（防御性断言，写理由）。核心模块 $\geq 85\%$  门禁的含义：不是『差不多 85』，是 CI 数字说了算。

### 6.5.7 无 pytest 环境的降级方案

教学现场网络受限时装不了 pytest（本案例实测 pip 超时）。降级方案：selfcheck.py 式等价断言脚本——用纯 Python 复刻关键断言，保持『代码逻辑被真实执行验证』的底线。但要在报告中注明：正式 CI 环境必须恢复 pytest 运行，selfcheck 只是应急通道。工具可降级，验证不可缺席。

### 6.5.8 契约测试实操：让接口始终对上 OpenAPI

契约测试回答的问题是：代码实现的接口，与设计文档（OpenAPI Spec）承诺的接口，是不是同一个？本节以样例代码仓 sprint\_repo 中的 contract\_check.py 为教具，讲清契约测试的人工方法与人机协同方法。先回顾一个真实事故：任务 3 交付的 OpenAPI v3 曾缺少 11 处 x-roles 与 x-fr 标注，是 contract\_check.py 在 CI 中抓到后才修复的——这正是契约测试存在的意义。

#### 为什么需要契约测试

前后端并行开发时，接口偏差是最常见的集成事故来源。典型场景：前端按 OpenAPI 里写的字段名 status 取值，后端实际返回的是 state；或者后端把某个字段从字符串改成了数字，前端运行时才崩。人工核对接口文档与代码既慢又不可靠，契约测试把这件事自动化：以 OpenAPI 为唯一事实源，机器比对代码实现。

| 偏差类型  | 典型例子                         | 人工核对的难度 | 契约测试的拦截方式           |
|-------|------------------------------|---------|---------------------|
| 路径不一致 | 代码是 /api/task，文档是 /api/tasks | 低，但量大易漏 | 枚举代码路由与文档 paths 求差集 |
| 方法不一致 | 文档写 POST，代码注册成 PUT           | 中       | 逐路径比对 HTTP 方法集合     |

| 偏差类型   | 典型例子                     | 人工核对的难度 | 契约测试的拦截方式        |
|--------|--------------------------|---------|------------------|
| 必填参数缺失 | 文档要求 prompt 必填，代码没校验     | 高       | 对照 schema 检查校验逻辑 |
| 标注缺失   | 端点缺 x-roles/x-fr 扩展标注    | 高，纯靠自觉  | 脚本强制扫描每个端点       |
| 响应结构漂移 | 错误响应从 code 改成 error_code | 高       | 对关键响应做 schema 断言 |

## 人工方法：契约核对的传统做法

没有自动化工具时，契约核对靠『接口走查会』：前后端各派一人，逐端点念文档、对代码，一份 6 个端点的 API 走查一遍约需 2 小时，且只能抽查式覆盖。要点：

- 建立端点清单：从 OpenAPI 导出端点表（路径、方法、角色、需求编号），逐项打勾；
- 抽样发请求：用 curl 或 Postman 对每个端点至少发一次正常请求和一次异常请求，记录实际响应；
- 双人背靠背：一人看文档写预期，一人跑代码给实际，第三人比对——避免『自己证明自己』。

人工做法的短板很明显：覆盖率低、不可重复、代码一变就要重来。它只适合作为自动化之前的过渡，以及自动化工具的抽样复核。

## 样例教具：contract\_check.py 的工作原理

sprint\_repo 中的 contract\_check.py 是一个约百行的 Python 脚本，做三件事：

- 解析 OpenAPI：加载 openapi.yaml，提取全部 paths、方法、以及每个端点的 x-roles（允许角色）与 x-fr（对应需求编号）扩展标注；
- 扫描代码路由：解析后端路由注册代码，提取实际暴露的路径与方法集合；
- 双向比对：文档有而代码没有的端点报『未实现』；代码有而文档没有的端点报『未登记』；任一端的 x-roles/x-fr 缺失报『标注缺口』。三类问题任一非空，脚本以非零码退出，CI 红灯。

# 课堂演示：本地跑契约检查 `cd sprint_repo python scripts/contract_check.py` # 全绿输出示例：

```
# [contract] 26 endpoints checked: paths OK, methods OK, x-roles OK, x-fr OK # 制造一个缺口试试：把 openapi.yaml 里某个端点的 x-fr 删掉再跑，观察报错
```

## 真实事故复盘：11 处标注缺口是如何被抓到的

任务 3 交付 OpenAPI v3 时，人工评审只核对了路径与字段，没有逐个检查扩展标注。CI 首次挂上 `contract_check.py` 后立即报红：11 个端点缺 `x-roles` 或 `x-fr`。修复过程本身成了最好的教学案例：

- 第一步，按脚本输出清单逐端点补标注，补的时候必须回到 SRS 查该端点对应哪条 FR——有两处一时查不到，暴露的其实是『这个端点为什么存在』的设计疑问，最终对照需求补登或删除；
- 第二步，把『新增端点必须先过 `contract_check` 再提 PR』写进冲刺纪律；
- 第三步，在 PR 模板的自查清单里加一行：新增/修改端点是否同步更新 OpenAPI 并通过契约检查。

教学要点：契约测试的价值不只在拦截错误，更在于它把『文档与代码一致』从一句口号变成了每次提交都被强制执行的纪律。讲这一节时务必现场演示一次故意制造缺口→CI 拦截→修复→转绿的全过程。

## 人机协同：用 AI 编写与维护契约测试

契约检查脚本本身非常适合由 AI 起草，但必须人工审定比对规则。推荐流程：

- 第 1 步（人工）：明确比对规则清单——比什么（路径、方法、参数、标注）、`tolerance` 是什么（文档先行还是代码先行）、报错的退出语义。把规则写成 5~10 条自然语言条目；
- 第 2 步（AI）：用 P-04 类提示词让 AI 按规则清单生成脚本初稿，要求附带自测用例（构造一个故意不一致的样例验证脚本能抓到）；
- 第 3 步（人工）：逐条对照规则清单审脚本，重点看差集方向有没有搞反、空值处理是否正确；
- 第 4 步（AI+人工）：让 AI 批量补标注初稿（它读 SRS 与 OpenAPI 后给出 `x-fr` 建议），人工逐条确认——AI 补标注的采纳率在我们的样例中约七成，剩下三成是它对需求编号的臆断，必须人工否决。

提示词示例：

```
你是后端工程师。请阅读 openapi.yaml 与 SRS 需求清单，为每个端点建议 x-fr 标注（需求编号）。输
```

出格式：端点 | 建议 x-fr | 依据（引用 SRS 原文条款）。规则：找不到明确依据的端点标注 SUSPECT 并说明理由，禁止猜测编号。

## 常见问题

| 问题         | 症状                                  | 处理                             |
|------------|-------------------------------------|--------------------------------|
| 契约先行还是代码先行 | 两边互相等对方先改                           | 本项目规定文档先行：先改 OpenAPI，评审通过后再改代码 |
| 脚本误报       | 路径参数格式 /tasks/{id} 与 /tasks/:id 不一致 | 在脚本内做路径归一化，或统一写法并写进编码规范        |
| 第三方接口怎么管   | 外部服务没有 OpenAPI                      | 为其写一份最小契约文档（Mock 的输入输出），纳入检查   |
| 脚本本身没人维护   | 规则变了脚本没跟上                           | 脚本的修改与业务代码同标准评审，挂进同一 CI        |

## 本节清单

- 能说清契约测试拦截的五类偏差及各自的人工核对难度；
- 能现场跑通 contract\_check.py 并解释每一类报错的含义；
- 能给新端点补齐 x-roles 与 x-fr，且每条 x-fr 都能在 SRS 中指出原文；
- 掌握『AI 建议标注、人工确认』的协同流程，知道 AI 在此处的典型错误是臆造需求编号。

### 6.5.9 覆盖率门禁：70% 不是目标，是地板

样例项目的 CI 覆盖率门禁是：总体行覆盖率不低于 70%，核心模块（queue / worker / geo / security）不低于 85%，并对增量代码单独设门槛。本节讲清这些数字怎么定、怎么测、怎么不被钻空子。

## 覆盖率的正确心智

覆盖率度量的是『测试执行过多少代码』，不是『测试发现了多少问题』。100% 覆盖的代码可能一个断言都没有，20% 覆盖的代码也可能关键路径都被扎实测过。因此覆盖率门禁的定位是地板而不是天花板：它保证『没有任何大片代码完全没人测』，但达到门禁绝不等于测试合格。评审时永远要问第二层问题：这些覆盖里的断言质量

如何？

## 门禁数字的制定方法（人工决策，AI 提供数据）

门禁数字不是拍脑袋来的，推荐按三步定：

- 第 1 步，模块分级：把代码按重要性分级。样例项目的分法是——核心模块（任务队列、GPU 调度、地理计算、安全校验，业务错了就是事故） $\geq 85\%$ ；一般业务模块（路由、序列化） $\geq 70\%$ ；胶水代码（启动脚本、配置加载）不设门禁但要求冒烟。分级依据是任务 3 的 FMEA 风险登记册：严重度高的模块门禁高；
- 第 2 步，基线测量：先跑一次全量覆盖率，看现状是多少。门禁设在『现状上方 5~10 个百分点』，既够得着又不放水。样例项目首测总体 61%，第一轮门禁定 70%，迭代二提到 76%；
- 第 3 步，增量门禁：存量代码的历史债一时还不清，但新写的代码必须干净。增量门禁要求『本 PR 新增/修改的行，覆盖率不低于 80%』，防止一边补旧债一边欠新债。

## 工具链实操

```
# 样例项目的覆盖率工具链（pytest-cov） cd sprint_repo python -m pytest tests/ --cov=src --cov-report=term-missing --cov-report=xml # term-missing 直接在终端标出未覆盖的行号，课堂演示用它 # xml 报告给 CI 与覆盖率可视化工具消费 # 分模块门禁示例（CI 配置片段）： # coverage report --fail-under=70 # 总体 # coverage report --include="src/queue/*,src/worker/*,src/geo/*,src/security/*" --fail-under=85 # 核心
```

读报告的三个要点：一看总体数字是否过线；二看核心模块每个文件的明细，揪出拖后腿的文件；三看未覆盖行号聚在哪里——聚在错误分支（except 块）说明异常路径没测，聚在正常分支说明主流程有缺口，后者更严重。

## 增量覆盖率的落地

增量覆盖率解决的是『老代码覆盖率 60%，新 PR 只改了 20 行且全没测，总体覆盖率几乎不动，门禁形同虚设』的漏洞。落地方式：CI 里用 diff-cover 类工具，比对本 PR 相对主分支的差异行与覆盖率报告的交集。样例项目把增量门槛定为 80%，PR#52（阈值变更）新增 31 行代码、测试覆盖 29 行，增量覆盖率 93.5%，顺利过闸。

## 防钻空子：覆盖率注水识别清单

学生提覆盖率时最常见的四种注水手法，评审时逐条查：

| 注水手法       | 特征                                  | 识别方法                             | 处置                |
|------------|-------------------------------------|----------------------------------|-------------------|
| 空断言测试      | 测试只调用不断言，或 <code>assert True</code> | 抽查覆盖率最高的测试文件，看断言语句密度             | 打回重写，计一次评审拦截      |
| 镜像测试       | 把实现代码抄进断言里当期望值                      | 断言里的字面量与实现里的字面量逐字相同              | 打回，期望值必须来自需求/spec |
| 只测快乐路径     | 全是有效应答，无异常分支                        | 看 <code>except</code> 块的覆盖行号是否全空 | 要求补异常用例           |
| import 式覆盖 | 测试文件 <code>import</code> 了模块就算覆盖    | 覆盖率高但测试用例数极少                     | 看用例数/覆盖率比值，异常即查   |

## 人机协同：用AI补覆盖率

补覆盖率是 AI 辅助的高性价比场景，但有固定的正确姿势：

- 第 1 步（工具）：跑覆盖率报告，导出未覆盖行号清单；
- 第 2 步（AI）：把目标函数源码+未覆盖行号+spec.md 相关验收标准一起喂给 AI，要求『针对未覆盖分支补测试，每个用例注明对应的验收标准或分支意图』。P-06 提示词（测试补全模式）就是这个场景的沉淀；
- 第 3 步（人工）：按 9.5 清单逐用例审查 AI 产物，重点查期望值来源——AI 最喜欢从实现倒推期望值（镜像测试），这是它补覆盖率时最大的系统性偏差；
- 第 4 步（纪律）：AI 补的测试与人工测试同标准评审，提交信息里注明 AI-generated，便于度量采纳率。

样例数据：迭代一中 AI 辅助补测试 41 个用例，评审采纳 29 个（71%），打回的 12 个里 9 个是镜像断言、3 个是无效边界值。这组数字适合在课堂上公布，让学生对『AI 写测试也要审』建立具体预期。

## 本节清单

- 能解释覆盖率是地板而非天花板，并说出断言质量这层问题；



- 掌握门禁三步制定法：模块分级→基线测量→增量门禁；
- 会用 `pytest-cov` 出报告并定位未覆盖聚集区；
- 能识别四种注水手法，评审 AI 补的测试时优先查期望值来源。

## 6.6 安全红线：四条不可协商的底线

样例项目把安全要求浓缩为四条红线（R1~R4），对应等保基线与 SRS 的 SEC-01/SEC-02 非功能需求：口令不明文、SQL 参数化、输入必校验、密钥不入库。本节讲清每条红线的依据、检查方法、以及 AI 编码场景下特有的安全风险。

### 6.6.1 为什么 AI 时代安全红线更重要

AI 生成代码有两个结构性的安全倾向：一是它学到的公开语料里充满了教学式的不安全写法（硬编码口令、字符串拼 SQL），追求『能跑』而非『安全』；二是它对项目级安全上下文无感，比如不知道某个字段来自用户输入而非可信内部。因此 AI 代码的安全评审不是加分项，是必选项。样例项目的纪律是：安全红线由脚本（`security_redline.py`）+ 人工评审双重把关，任一亮红灯 PR 不得合并。

### 6.6.2 四条红线逐条讲

#### R1 口令不明文

要求：用户口令只允许以加盐哈希形式存储与传输（样例用 `bcrypt`，`cost factor` 12）；日志、异常消息、配置文件中不得出现明文口令。对应 SRS 的 SEC-01。检查方法：`security_redline.py` 扫描代码中的 `password` 关键字附近是否存在直接赋值/拼接/日志输出；人工评审抽查认证模块的存储与日志路径。AI 高风险点：AI 写登录示例代码时几乎必然写出明文比较，必须有专人盯着认证相关 PR。

#### R2 SQL 参数化

要求：所有 SQL 语句使用参数绑定，禁止字符串拼接构造查询；ORM 的 `raw query` 接口使用需评审记录理由。检查方法：脚本扫描 `f-string`/百分号/`format` 拼接出现在 `execute/query` 调用中的模式；人工评审所有数据访问层 PR。课堂演示用例：故意让 AI 写一个『按用户名查用户』的函数，约三成概率它首稿会写出拼接式 SQL——这是讲这条红线最好的现场素材。

#### R3 输入必校验

要求：所有外部输入（HTTP 请求体、路径参数、上传文件、环境变量）在进入



业务逻辑前完成校验；校验规则集中在 schema 层（样例用 pydantic 模型），禁止在业务代码里零散写 if。检查方法：contract\_check.py 已把『端点是否声明了请求模型』纳入扫描；人工评审看校验规则是否与 spec.md 的边界一致（比如置信度阈值必须在 0~1 区间）。对应 SRS 的 SEC-02 与 F-04 系列的输入约束。

**R4 密钥不入库**

要求：API Key、数据库连接串、盐值等机密只从环境变量或密钥管理服务读取；.env 文件进 .gitignore；代码仓历史的每次提交都在扫描范围（曾入库再删除也算违规，需轮换密钥）。检查方法：security\_redline.py 扫描密钥模式（正则匹配常见 Key 格式）+ git 历史扫描；CI 的 pip-audit 顺带扫依赖漏洞。

**6.6.3 security\_redline.py 实操**

```
# 课堂演示：红线扫描 cd sprint_repo python scripts/security_redline.py # 全绿输出: [security]
R1 password-hash OK, R2 sql-param OK, R3 input-validation OK, R4 no-secrets OK # 演示拦截: 临时在某文件写一行 password = "admin123" 再跑, 观察 R1 报红与定位行号
```

脚本约 120 行，核心是四组正则+AST 检查的组合。教学上强调：正则扫得出模式，扫不出意图——比如把密钥拆成两个字符串变量再拼接就能绕过正则，所以脚本只是第一道网，人工评审（尤其是安全敏感模块的指定评审人）是第二道。

**6.6.4 等保基线的教学化映射**

本科实践不要求做真正的等保测评，但要求学生知道红线对应的正规出处。映射表写进 SRS 的 SEC 章节，答辩时能说清：

| 红线         | 对应等保 2.0 控制点（教学化） | SRS 条目 | 检查手段           |
|------------|-------------------|--------|----------------|
| R1 口令不明文   | 身份鉴别-口令复杂度与存储保护   | SEC-01 | 脚本+认证模块人工评审    |
| R2 SQL 参数化 | 安全计算环境-数据完整性/防注入  | SEC-02 | 脚本+数据层人工评审     |
| R3 输入必校验   | 安全区域边界-边界防护/入侵防范  | SEC-02 | 契约扫描+schema 评审 |
| R4 密钥不入库   | 安全管理制度-密码与密钥管理    | SEC-01 | 脚本+git 历史扫描    |

### 6.6.5 人机协同要点

- 让 AI 生成『安全自查清单』附在 PR 描述里：改动涉及哪些红线、自查结果如何。AI 列清单的覆盖率比人凭记忆高，但结论必须人工复核——它经常把『用了 ORM』直接等同于『R2 通过』，忽略 raw query 调用；
- 用 AI 做对抗性自查：给 AI 一段刚写好的代码，让它扮演攻击者找出注入/越权点，再把发现转化为测试用例。这是 P-11 提示词（红队模式）的用途，样例项目用它发现了 2 个路径遍历隐患；
- 安全相关的 AI 建议一律升级评审等级：涉及认证、权限、加密的代码，无论谁写，必须双人评审，其中一人为安全责任人。

### 6.6.6 本节清单

- 背得出四条红线及其等保/SRS 出处；
- 会跑 security\_redline.py 并能修复它报的每一类问题；
- 理解脚本与人工双闸的分工：脚本扫模式，人审意图；
- 掌握红队模式提示词，能把 AI 攻击发现转化为防御性测试用例。

## 6.7 性能测试实操：一次完整的优化闭环

性能需求（PE-01~PE-09）是 SRS 里的非功能硬指标：全自动分割单景影像 P95 不超过 3000ms，降级切换不超过 30s。本节以样例项目的真实优化案例（PE-01 实测 3.4s 优化到 2.8s）为主线，讲清性能测试的人工方法与人机协同方法。

### 6.7.1 先建立性能测试的正确姿势

新手做性能测试最常见的三个错误：只测一次就下结论、在自己笔记本上测出结果就当成交付数据、只报平均值不报分位数。正确姿势的四条纪律：

- 环境纪律：性能数据必须注明环境（CPU/GPU 型号、内存、输入影像尺寸），环境不同数据不可比。样例项目规定性能测试统一在测试服务器（RTX 3060 / 32GB）上跑，本地数据只用于调优参考；
- 统计纪律：每项指标至少跑 30 次取 P50/P95，报 P95 为主——因为 SRS 的 PE-01 写的就是 P95；

- 基线纪律：每次性能相关的 PR 合并前，先测基线（改动前）再测改动后，同环境对比，防止『优化』实际是退化；
- 预算纪律：把性能指标写进常量（样例代码里的 `P95_BUDGET_MS = 3000`），测试断言直接引用它，需求改阈值时只改一处。

### 6.7.2 人工方法：手工性能测试怎么做

没有自动化之前的做法，也是理解原理的必经之路：

- 第 1 步，构造标准输入：准备一组冻结影像集（注意：是冻结的测试影像，固定尺寸与内容，登记哈希值，与训练无关），保证每次测试输入一致；
- 第 2 步，打点计时：在代码关键路径插桩（`time.perf_counter`），记录排队时间、推理时间、后处理时间三段；
- 第 3 步，手工记录表格：跑 30 次，逐次记录三段耗时，算 P50/P95，填进性能测试记录表；
- 第 4 步，瓶颈定位：看三段里哪段占大头。3.4s 版本的分段数据显示推理 2.9s 占 85%，优化方向明确——不是改队列，是改推理路径。

### 6.7.3 样例案例精讲：PE-01 从 3.4s 到 2.8s

这是迭代二的真实优化，完整走完『测-析-改-验』闭环，适合作为课堂主案例：

- 测（基线）：迭代一末测得全自动分割 P95 = 3.4s，超预算 13%。性能测试报告里登记为缺陷 PERF-001，级别 Major（不阻塞交付但偏离承诺）；
- 析（定位）：分段数据显示瓶颈在推理段。进一步 profiling 发现两个热点：影像分块的边界重叠区做了两次推理（重复计算约 18%），以及每次推理重新加载了归一化参数（本可缓存）；
- 改（方案）：两个改动——重叠区推理结果复用、归一化参数按模型实例缓存。方案写成一页纸的优化说明，附预期收益估算，走正常 PR 流程（PR#58），不因为是『性能优化』就跳过评审；
- 验（回归）：合并后同环境重测，P95 = 2.8s，且精度回归测试（AT-01）必须同时过——性能优化不得以精度为代价，这是写进测试纪律的。最终 2.8s 达成且 AT-01 精度无变化，PERF-001 关闭。

教学要点：这个案例的完整证据链（基线记录→分段数据→PR#58→回归报告）都在样例仓里，讲课时按证据链走一遍，让学生看到性能工程是『有纪律的实证过程』，不是『拍脑袋调参』。

### 6.7.4 性能测试的自动化

```
# 样例仓中的性能测试（pytest 标记为 perf，CI 夜间跑） # tests/perf/test_pe01.py 的核心结构：
# 1. 加载冻结影像集（校验哈希，防输入漂移） # 2. 预热 3 次（排除冷启动） # 3. 正式跑 30
次，收集耗时 # 4. 断言 p95 < P95_BUDGET_MS # 5. 输出性能记录行，追加进 perf_history.csv
（趋势追踪） cd sprint_repo python -m pytest tests/perf/ -m perf --durations=0
```

性能测试不进每次 PR 的 CI（太慢），进每日夜间构建；但性能敏感的 PR（动推理路径、动队列调度）必须在描述里附手工实测对比数据，这是评审硬要求。

### 6.7.5 人机协同要点

- AI 起草性能测试脚本：给出指标定义与输入要求，让 AI 生成统计代码（分位数计算、结果表格），人工核对统计口径——AI 常犯的错误是用 mean 冒充 P95，或把预热次数计入统计；
- AI 辅助 profiling 解读：把分段耗时数据喂给 AI 让它提出优化假设，但每个假设必须人工评审其代价——样例中 AI 曾建议『缩小分块尺寸减少单次推理时间』，人工评审发现这会增加块数、反而增加总耗时和接缝误差，否决；
- 优化方案的预期收益由 AI 估算、人工背书：AI 估算重复计算占比 18% 基本准确（实测 17%），但它对缓存命中率的乐观假设（估 95% 实测 81%）导致总收益高估，人工评审时把估算打了八折——这个『AI 估算打折』的经验值得写进团队公约。

### 6.7.6 常见问题

| 问题      | 表现                       | 处理                                 |
|---------|--------------------------|------------------------------------|
| 本地快服务器慢 | 本地 2.5s 测试机 3.6s         | 以测试服务器数据为准；排查 GPU 型号与驱动差异          |
| P95 抖动大 | 30 次里 28 次 2.6s、2 次 4.5s | 查抖动根因（GC/磁盘缓存冷/队列竞争），必要时加到 100 次取样 |

| 问题       | 表现                      | 处理                   |
|----------|-------------------------|----------------------|
| 优化引入精度回归 | 速度快了但 AT-01 降了          | 性能与精度同 PR 回归，任一不过即打回 |
| 指标没人看    | perf_history.csv 一个月没更新 | 把性能趋势图挂进周报模板，迭代评审必看  |

### 6.7.7 本节清单

- 背得出性能测试四条纪律（环境/统计/基线/预算）；
- 能复述 3.4s→2.8s 案例的完整闭环与每个环节的证据；
- 会写带分位数统计的性能测试，知道 P95 与 mean 的区别；
- 掌握 AI 估算收益要打折的经验，理解性能优化不得牺牲精度。

## 6.8 精度测试：冻结影像集与哈希登记制度

精度需求（AT-01~AT-03）是本项目区别于普通 CRUD 系统的特殊指标：分割结果要满足量化精度门槛。本节讲精度测试的完整制度：冻结影像集怎么建、指标怎么算、结果怎么登记，以及 CR-001 阈值变更（0.30→0.24）后完整率从 76.8% 升到 84.2% 的来龙去脉。

### 6.8.1 精度测试为什么必须先解决『输入漂移』

普通单元测试的输入是代码里写死的常量，精度测试的输入是一批影像文件。如果这批文件被人换了几张、或被压缩工具重新编码过，前后两次精度数据就不可比——但你根本发现不了。因此精度测试的第一块基石是冻结影像集制度：

- 选定一批影像（样例项目 5 景，覆盖建筑/农田/水体等主要地物类型），版本锁定，一旦选定即冻结，任何人不得增删改；
- 逐文件计算 SHA-256 哈希，登记进《冻结影像集登记表》（文件清单+哈希+尺寸+地物类别）；
- 精度测试脚本运行前第一步是校验全部哈希，任一不符立即中止——宁可不测，不用漂移的输入测；

- 冻结影像集不入代码仓（体积大），放共享存储，仓里只存登记表；新成员的 onboarding 清单里有一项是『拉取冻结影像集并本地校验哈希』。

## 6.8.2 指标定义：完整率/正确率/IoU 怎么算才对

精度三指标的原文口径（SRS V3.1，测试计划 同口径）：AT-01 全自动分割的面积加权完整率 $\geq 80\%$ 、正确率 $\geq 85\%$ （AC-FR-02.1-b）；AT-02 交互分割 IoU $\geq 0.75$ （AC-FR-03.1-b）；AT-03 面积统计相对误差 $\leq 3\%$ （10 组已知几何手算比对）。每一条都有必须抠准的定义，每一个都曾是真实争议点：

- 完整率/正确率的口径：完整率= $|\text{解译} \cap \text{参考真值}| / |\text{参考真值}|$ （漏检惩罚），正确率= $|\text{解译} \cap \text{参考真值}| / |\text{解译}|$ （误检惩罚）；按地物类别分组统计后面积加权汇总（防止大面积类别刷高分）。这个口径在 SRS 里写明，AI 生成计算代码时要把原文喂给它，否则它默认给出全局像素口径；
- 参照真值从哪来：冻结影像集配人工精标注的参照真值矢量（标注本身走两人背靠背+第三人仲裁的流程，标注质量记录在案）。参照真值同样有哈希登记；
- 『不低于』的判定边界：按原始计算值判定，报告里展示保留一位小数；判定标注由评估报告给出『达标/不达标』，不做自动驳回——最终判定权归复核员（五类角色之一）。这类判定规则必须写进 spec.md 的验收标准，

Given/When/Then 的 Then 里写精确。

## 6.8.3 精度测试脚本的结构

```
# tests/accuracy/test_at01.py 的核心结构：# 1. 校验冻结影像集（5 景）与参照真值的哈希（不过即 fail，输出不符文件清单）# 2. 对每景影像跑全自动分割，产出解译矢量结果集# 3. 按地物类别分组计算完整率/正确率，面积加权汇总# 4. 断言 recall_w >= 0.80 and precision_w >= 0.85# 5. 输出逐景、逐类别的明细表，追加进 accuracy_history.csv# 运行：cd sprint_repo python -m pytest tests/accuracy/ -m accuracy
```

精度测试也不进每次 PR 的 CI（跑一次约 20 分钟），进夜间构建与发布前必跑清单；任何动推理路径、阈值参数、后处理逻辑的 PR，必须附精度实测数据。

## 6.8.4 案例：CR-001 阈值变更的精度影响

变更单 CR-001（变更单 CR-001）把文本提示分割（FR-04.1）的置信度阈值默认值从 0.30 降到 0.24，理由是冻结影像集初测：0.30 档 5 景中 2 景召回不足（地物目标漏检）。这个变更完整走了四步流程（改 spec→同步 SRS→改 plan/tasks→改代码），在精度侧的验证过程：

- 变更影响评估阶段：在冻结影像集上对比两档阈值的实测数据——完整率 76.8%→84.2%（升），正确率 91.5%→88.6%（略降但仍在≥85% 判定线之上），两档均满足 AC-FR-02.1-b 口径。数据附在变更单里，是 CCB 有条件批准的关键依据；
- 代码修改：只改阈值常量与相关测试期望值（PR#52），改动 31 行；
- 回归验证：合并后夜间构建重跑精度测试（完整率 84.2%/正确率 88.6% 复现），登记进 accuracy\_history.csv，精度复测报告归档后 CR-001 方可关闭（CCB 附加条件）；
- 文档同步：SRS、spec.md、OpenAPI 中所有出现 0.30 的地方全部改为 0.24——变更单附带的『影响面清单』逐项打勾，漏一处即变更未闭环。

教学要点：CR-001 是全套样例里最有教学价值的变更案例，因为它同时贯穿了配置管理（四步流程）、精度工程（阈值-精度权衡）、测试工程（期望值更新、回归验证）三条线。建议课堂上用 30 分钟完整走一遍这个变更的证据链。

### 6.8.5 人机协同要点

- AI 起草精度统计代码：指标定义原文必须喂给它，输出后人工核对计算口径（面积加权与否、四舍五入规则）——这是 AI 最容易『默认实现』偏航的地方；
- AI 做精度归因分析：精度不达标时，把逐类别明细喂给 AI 让它找模式（哪个类别拖后腿、什么地物特征）。样例中 AI 正确指出『水体类别完整率最低且与影像分辨率强相关』，人工验证属实，指导了后续优化方向；
- 精度结论一律人工背书：AI 可以帮算、帮分析，但『精度是否达标』的结论必须由测试负责人签字确认，因为涉及验收承诺。

### 6.8.6 本节清单

- 能说清冻结影像集制度的三要素：冻结、哈希登记、运行前校验；
- 掌握完整率/正确率的面积加权口径与判定边界规则，知道为什么这些定义要写进 spec.md；
- 能复述 CR-001 的精度影响评估过程，理解阈值变更必须数据驱动；
- 明确精度结论人工背书的底线。

## 6.9 降级与故障演练：把预案变成肌肉记忆

PE-09 要求：GPU 不可用时系统自动降级到 CPU 推理，切换时间不超过 30s，且任务不丢失。这类『故障时才被执行的代码』平时没人碰，最容易藏着 bug。本节讲如何用故障演练（简化版混沌工程）把降级路径真正测出来。

### 6.9.1 为什么降级路径是高危代码

降级路径的三个结构性风险：一是执行频率低，日常测试跑不到；二是它恰好在系统已经出问题时才启动，出错时代价加倍；三是它往往涉及跨组件协调（队列、Worker、状态机），单元测试覆盖不全。业界经验是：没有演练过的降级方案，首次真实触发时约有一半不能按预期工作。教学项目里我们把这句话变成纪律：降级方案必须在交付前演练验证至少三次。

### 6.9.2 人工演练方法：三板斧

- 第一板斧——走查（桌面演练）：不开机，团队围坐，对着架构图走一遍故障剧本：GPU 进程死了→谁发现→怎么发现→任务去哪→用户看到什么。每一步说出依据（哪段代码、哪个状态）。走查能发现逻辑漏洞，比如样例项目走查时发现『GPU Worker 死后，队列里 in-progress 状态的任务没有回收逻辑』——这是任何单元测试都测不出的设计漏洞，后来补了 Task.transit 的回收路径；
- 第二板斧——实测（真杀进程）：在测试环境真的 kill 掉 GPU Worker 进程，掐表测切换时间，验证任务不丢。PE-09 的 30s 指标就是这么实测出来的（三次实测 18s/22s/25s）；
- 第三板斧——复盘：每次演练后填演练记录表：剧本、预期、实际、偏差、改进项。改进项挂进 tasks.md，不许只记录不整改。

### 6.9.3 样例剧本：kill Worker 演练全记录

课堂主案例，对应样例仓 scripts/chaos/ 下的演练脚本：

- 剧本：系统正在处理 3 个分割任务（1 个进行中、2 个排队），此时 GPU Worker 进程崩溃；
- 预期行为：GpuQueue 的心跳机制在 2 个心跳周期（10s）内发现 Worker 失联→进行中的任务按 backoff\_seconds（30/60/120 秒退避）重排→CPU Worker 拉起并接管→全部任务完成，用户端收到降级提示；



- 实测记录：第一次演练发现问题——CPU Worker 接管了任务，但用户端没有收到降级提示（通知逻辑只在启动时判断模式，没有运行中切换的分支）。登记缺陷 CHAOS-001，修复后第二次演练全绿；
- 度量：三次演练的切换时间 18s/22s/25s，满足 PE-09 的 30s 指标；任务零丢失。

```
# 演练脚本用法（必须在测试环境，严禁生产） cd sprint_repo python scripts/chaos/kill_worker.py
--scenario gpu_dies --tasks 3 # 输出：逐事件时间线 + 切换耗时 + 任务终态核对表
```

#### 6.9.4 故障库：让学生有剧本可演

教学上为每组准备故障剧本库，每组至少抽演 3 个。样例工程对应的剧本：GPU 不可用（必演，验证 CPU 降级路径）、模型可选依赖被卸载（验证优雅降级提示）、磁盘写满、进程被强杀后的临时目录残留与项目状态一致性、畸形输入（超长文本提示、伪装扩展名、路径穿越）。\*\*各组的剧本必须按自己的架构重编\*\*——你的系统里不存在的组件，演它没有意义。每个剧本的卡片上只写『故障现象』，预期行为由学生从 spec.md 和设计文档里自己推导——推导不出来，说明设计文档没写清，这本身就是演练要暴露的问题。

#### 6.9.5 人机协同要点

- AI 生成故障剧本：把系统架构描述喂给 AI，让它穷举故障模式（每个组件：死了/慢了/返回错误数据，每种依赖：断连/超时/数据损坏），人工筛选出与本项目相关的 10~15 个。AI 穷举的覆盖面明显优于人脑风暴；
- AI 起草演练脚本：kill 进程、造毒消息这类脚本让 AI 写，但必须在沙箱里先跑、人工审过每一行——演练脚本本身就是危险操作，AI 写的 kill 命令曾把过滤条件写宽，差点杀掉宿主机其他进程（这构成一条翻车记录的好素材）；
- 复盘会议由 AI 做记录员：口述时间线，AI 整理成演练记录表初稿，人工修正后归档。

#### 6.9.6 与测试金字塔的关系

故障演练不替代单元/集成测试，它站在金字塔之上补『系统级负面场景』。教学安排上：单元测试随 PR 跑，集成测试每日跑，故障演练每迭代至少一轮。PE-09 这类指标的验收证据 = 演练记录表 + 实测时间线 + 缺陷修复闭环记录，三样齐全才算过。

### 6.9.7 本节清单

- 理解降级路径的三类结构性风险，认同『未演练的降级方案不可信』；
- 掌握走查-实测-复盘三板斧，能设计并执行一个完整演练；
- 能复述 kill Worker 演练的剧本、发现的问题与度量数据；
- 会用 AI 穷举故障模式，并对 AI 写的危险脚本保持审查纪律。

## 6.10 调试方法论与重构技术债

AI 时代的调试没有消失，而是变了形：bug 更少是语法错，更多是『AI 理解偏了需求』或『组件间假设不一致』。本节给出一套与 AI 协作的调试方法，对应模式库 P-09（假设-验证）与 P-10（最小复现）。

### 6.10.1 调试的第一原则：先复现，再谈修

没有稳定复现的 bug 不许开始修——靠猜的修复十有八九引入新 bug。复现的产出是最小复现用例：能稳定触发问题的最小输入与最少步骤。P-10 提示词让 AI 协助缩小复现：『给定这段失败日志与相关代码，提出 3 个最小化复现方案，每个方案说明排除了哪些变量』。样例项目统计：有最小复现的 bug 平均修复 1.8 小时，没有的平均 5.5 小时。

### 6.10.2 假设-验证循环（P-09）

- 第 1 步 列假设：把可能原因全部列出（AI 很适合做这个——它的假设覆盖面比人广），每条假设写明『如果它成立，应该观察到什么』；
- 第 2 步 排序：按验证成本从低到高排，先验便宜的（查日志、加打印），后验贵的（改代码重跑）；
- 第 3 步 逐条验证：每条假设设计一个判别实验，实验结果必须二值（证实/证伪），不允许『好像是这样』；
- 第 4 步 修复与防复发：定位后修复，并回答『为什么测试没拦住』——补测试或补评审规则，否则同一 bug 还会回来。

教学演示案例（真实）：精度测试偶发失败，AI 列出 5 个假设（输入顺序、浮点累积、并发写、缓存污染、哈希校验边界），按成本排序后第二个实验就定位到『并发写导致部分掩膜文件截断』。若按直觉先怀疑算法，至少浪费半天。

### 6.10.3 与 AI 协作调试的纪律

- 日志喂全不喂截：给 AI 的日志截段会诱导它瞎猜。喂完整上下文，明确标注现象发生的时间点；
- AI 的诊断是假设不是结论：AI 说『问题是 X』时，按 P-09 把它登记为假设之一，设计实验验证后再动手。样例项目纪律：AI 诊断未经实验验证就改代码，评审一票否决；
- 修复必须带回归用例：每个 bug 修复 PR 附一个『没有本修复就会失败』的测试。这个用例是修复正确性的永久证据，也是防复发的哨兵。

### 6.10.4 调试记录

超过 2 小时的调试必须留调试记录（现象、假设清单、验证过程、根因、修复、防复发措施），格式与翻车记录兼容——调试记录是翻车记录的原料库。样例项目《AI 翻车记录》的 FR-02 就是由一份调试记录升级而来。

### 6.10.5 本节清单

- 执行『先复现再修』纪律，会构造最小复现；
- 掌握假设-验证四步，假设按验证成本排序；
- AI 诊断一律登记为假设，验证后再动手；
- 修复必带回归用例，长调试必留记录。

### 6.10.6 重构与技术债：在冲刺中保持代码能改

两个迭代的高速产出必然积累技术债。本节讲：债怎么登记、什么时候还、AI 辅助重构的安全姿势。样例项目的原则：债不可怕，不登记的债才可怕。

#### 技术债登记制度

所有技术债进 tasks.md 的专门区块（TD 编号），登记字段：位置、债务内容、产生原因、不还的代价、建议偿还时机。样例项目两迭代共登记 9 笔，偿还 5 笔，留 4 笔进任务 5（验收关前必须清零）。三笔典型债：

| 编号    | 债务     | 产生原因   | 偿还时机与结果         |
|-------|--------|--------|-----------------|
| TD-02 | 影像分块函数 | 迭代一赶进度 | 迭代二偿还：拆成 4 个函数， |

| 编号    | 债务                         | 产生原因           | 偿还时机与结果                   |
|-------|----------------------------|----------------|---------------------------|
|       | 260 行，圈复杂度超标               | 先求能跑           | PR#61                     |
| TD-05 | 模型加载的测试用的是桩替身，与真实权重加载行为有差异 | 当时测试环境 GPU 未就绪 | 任务 4 未偿还：换 fakeredis+契约测试 |
| TD-08 | 前端阈值组件与后端解析的单位约定靠口头        | CR-001 前的历史遗留  | CR-001 中一并根治（写进 OpenAPI）  |

### 偿还决策：什么时候还

三问决策法：这处债是否在你下一个迭代要动的代码路径上？（在→立即还，带着债改代码等于在流沙上盖房）它的利息是否在增长？（每次相关开发都要绕道→尽快还）它是否影响验收指标？（影响→任务 5 前必还）。三问全否的债可以留着，但留在登记册里见光。

### AI 辅助重构的安全姿势

重构是 AI 辅助的高危区：重构不改变外部行为，判断『改对了没有』全靠测试，测试不全的地方重构就是在雷区跳舞。安全姿势四条：

- 先补网再重构：重构前确认目标代码的测试覆盖率达标（核心模块 85%），不够先补测试。TD-02 的偿还就是先补 11 个用例再动手；
- 小步走，每步可验证：把大重构拆成一串小 PR，每个 PR 只做一种手法（提取函数/改名字/搬文件），CI 绿一个再走下一个。P-08 提示词（重构建议-小步走）的约束段明确写着『一次只做一个手法』；
- AI 出方案，人控节奏：让 AI 给出重构步骤序列，人工审排序（先内层后外层、先无依赖后有关联），执行时每个手法单独会话——重构会话上下文一乱，AI 会把『改结构』和『改行为』混在一起；

- 行为保持的实证：重构 PR 的描述里必须附『行为保持证据』——重构前后同一测试套件全绿、同一性能基线无退化、契约测试无变化。三样缺一，评审不批。

## 重构的反模式

| 反模式     | 表现                    | 后果                                                  |
|---------|-----------------------|-----------------------------------------------------|
| 顺手重构    | 修 bug 的 PR 里夹带重构      | 评审无法聚焦，bug 修复与结构变化纠缠，回退困难                           |
| 大爆炸重构   | 一个 PR 重写整个模块          | 测试网罩不住的边角全部暴露，PR#41 重写 FR-01 事故代码时险些如此，被评审拆成 3 个 PR |
| AI 全权重构 | 让 AI 一次性『优化这个模块』后整体提交 | 行为变化无法归因，样例项目明令禁止                                   |
| 债不登记    | 口头说『这里以后要改』           | 迭代末无人记得，验收前暴雷                                       |

## 本节清单

- 执行技术债登记五字段制度；
- 用三问决策法排偿还时机；
- 掌握 AI 辅助重构四条安全姿势，行为保持证据三件套；
- 识别四种重构反模式。

## 6.11 敏捷仪式：站会、周报与燃尽图

任务 4 要求每日站会 15 分钟、每周提交周报、维护燃尽图。这些仪式在真实企业里经常流于形式，教学上要把它做实——本节以《站会纪要与周报》（站会周报样例）中的两则真实纪要为教材，讲每个仪式怎么做才有用。

### 6.11.1 每日站会：15 分钟的信息对齐

标准三问（昨天做了什么/今天做什么/有什么阻塞）人人会背，但做实的关键在三条规则：

- 对着看板说，不凭记忆说：每人发言时指着看板上自己的任务卡，任务状态当场更新。凭记忆汇报的站会，一周内就会变成形式；

- 阻塞当场指派：说出的每个阻塞必须当场得到处理方案——谁帮、什么时候、或者升级为教师介入。《站会纪要与周报》的纪要显示，迭代一 D11 站会上提出的『GPU 测试环境被另一组占用』阻塞，当场决定改夜间跑精度测试，当天解决；
- 超时转为单聊：任何技术讨论超过 2 分钟，记录为会后单聊议题，站会继续。15 分钟是硬上限。

《站会纪要与周报》摘录（迭代二 D3 站会纪要）：A 完成 PR#52 合并、今天做降级演练脚本；B 的前端联调被阈值单位问题卡住（后来登记为联调问题①）；C 报告 AT 复测达标（完整率 84.2%/正确率 88.6%，归档后 CR-001 可关闭）；D 提醒明天 CI 要升级 pip-audit 规则。全纪要 11 行，这就是合格站会的产出密度。

### 6.11.2 周报：面向读者写作

周报的读者有三个：教师（评进度与风险）、下一周的自己（接续上下文）、答辩评委（追溯过程证据）。因此周报固定四段式：

- 进展：对照迭代目标逐条说完成度，附 PR 号与证据链接，禁止『基本完成』『进展顺利』这类无量词；
- 度量：本周 PR 数/一次通过率/CI 拦截数/覆盖率变化/燃尽状态，一张小表；
- 问题与风险：诚实登记。周报里没问题的组，评审时会被重点抽查——真实开发不可能没问题；
- 下周计划：从 tasks.md 里拉任务，带负责人与预计工时。

### 6.11.3 燃尽图：让它真能预警

燃尽图的纵轴是剩余工时（或任务点数），横轴是迭代天数。做实的三个要点：

- 每日更新，从 tasks.md 的真实状态取数，不许事后补画。样例项目的燃尽图数据由脚本从看板状态自动生成，杜绝手工美化；
- 理想线只是参考，关注偏差趋势：连续两天剩余量不降，说明任务粒度太大或估计失真——这是要拆开任务或重新估算的信号；迭代中期剩余量高于理想线 30% 以上，必须开风险会，砍范围或加人（教学场景是砍范围）；

- 燃尽图的尽头必须落到零：迭代结束时还有剩余任务，要么是范围管理失败，要么是状态更新不实，评审时都要解释。

《迭代冲刺计划与燃尽报告》（迭代冲刺计划与燃尽）中有完整两轮迭代的燃尽数据：迭代一 D4 出现平台期（剩余 62h 连续两天未降），原因是交互分割任务估计 8h 实际花了 22h——这个案例在课堂上讲『估算偏差如何被燃尽图提前暴露』最直观。

### 6.11.4 人机协同要点

- AI 做站会速记与纪要整理：录音转文字后让 AI 按三问结构整理，人工核对阻塞项的处理方案是否记录准确——这是最重要的字段；
- AI 起草周报初稿：把本周 PR 列表、CI 记录、燃尽数据喂给 AI 生成草稿，但『问题与风险』段必须人工写——AI 倾向于把问题写得轻描淡写，而这段的诚实度恰恰是评分点；
- 燃尽图生成自动化：让 AI 写从看板导出数据画燃尽图的脚本（一次投入，天天受益），脚本逻辑人工审。

### 6.11.5 反模式清单

| 仪式  | 反模式   | 症状           | 纠正            |
|-----|-------|--------------|---------------|
| 站会  | 背书式汇报 | 只报做了什么，不报阻塞  | 强制三问，阻塞必答     |
| 站会  | 技术讨论会 | 一个问题聊 20 分钟  | 2 分钟规则，转单聊    |
| 周报  | 流水账   | 罗列动作无结果无度量   | 四段式模板，无量词扣回   |
| 燃尽图 | 事后补画  | 曲线过分平滑       | 脚本自动取数，每日更新   |
| 燃尽图 | 永不归零  | 迭代末挂着一堆『快完成』 | 结束日强制清点，剩余进复盘 |

### 6.11.6 本节清单

- 掌握站会三条做实规则（看板、阻塞指派、超时单聊）；
- 会写四段式周报，理解问题段的诚实度是评分点；

- 会读燃尽图的偏差信号，知道平台期与偏离 30% 各意味着什么动作；
- AI 速记与周报起草可用，问题段必须人写。

### 6.12 变更管理与 RTM v4 回填

任务 4 的纪律是：变更一律走『改 spec → 同步 SRS → 改 plan/tasks → 改代码』四步流程。本节以 变更单 CR-001（变更单 CR-001：文本提示分割置信度阈值默认值 0.30→0.24）为标本，把每一步的操作、证据与常见偷懒做法讲透。

#### 6.12.1 为什么变更必须走流程

口头变更的三个后果：spec 与代码漂移（下次 AI 按旧 spec 生成，冲突）、测试基线失效（期望值还按旧值）、验收争议（评委按 SRS 验收，你说我们口头改了）。四步流程的本质是把『一个想法』变成『一串同步更新的工件』，保证任何时刻 spec、SRS、任务、代码、测试五者对得上。AI 辅助开发的特殊性在于：spec 是 AI 的工作依据，spec 不改而只改代码，等于给 AI 留下错误指令，下次生成必然回退。

#### 6.12.2 变更单的结构（变更单 CR-001 逐字段讲解）

| 字段                     | CR-001 实际内容                                                                                  | 填写要点                   |
|------------------------|----------------------------------------------------------------------------------------------|------------------------|
| 变 更<br>编 号<br>与 标<br>题 | CR-001 文本提示分割置信度阈值默认值调整                                                                      | 编号连续，标题说清改什么           |
| 提 出<br>人 与<br>日期       | 后端主攻，迭代二 D3（源于 T-18 文本分割联调实测）                                                                | 变更可以任何人提，但必须有记录        |
| 变 更<br>动因              | 冻结影像集初测：0.30 档 5 景中 2 景召回不足（漏检），实测数据附后                                                       | 必须数据驱动，附证据，禁止『感觉更好』    |
| 影 响<br>面 清<br>单        | spec.md AC-FR-04.1-a、SRS FR-04.1、DLD 5.2 节、代码常量 DEFAULT_THRESHOLD、2 个测试用例基线（TC-F04-01/04-02） | 逐项列出所有受影响工件，这是四步流程的导航图 |



| 字段   | CR-001 实际内容                                   | 填写要点                    |
|------|-----------------------------------------------|-------------------------|
| 数据论证 | 完整率 76.8%→84.2%、正确率 91.5%→88.6% (≥85% 判定线仍满足) | 对比数据是批准的依据              |
| 回退方案 | 常量改回 0.30 即回退，无数据迁移                           | 每个变更必须想好后路              |
| 批准记录 | CCB（组长+需求负责人+教师代表）有条件通过：四步闭环+精度复测归档后方可关闭      | 教学项目批准权在组内 CCB，重大变更升级教师 |

### 6.12.3 四步流程逐步操作

- 第 1 步 改 spec：先改 spec.md 的验收标准（AC-FR-04.1-a 中的默认阈值与 Given/When/Then），评审通过（commit sp-9d21）。这一步在代码之前，因为 spec 是源头；
- 第 2 步 同步 SRS：SRS 对应条款（FR-04.1）同步修改，版本号升小版本（V3.0→V3.1），修订记录表加一行：谁在什么时间因为 CR-001 改了哪条；
- 第 3 步 改 plan/tasks：tasks.md 新增任务 T-36（阈值默认值变更与复测，0.5 人日，依赖 T-31），排进当前迭代；plan.md 里程碑不变，燃尽图基线 +0.5 人日并注明变更来源；风险登记册新增 R-09（误检率）跟踪；
- 第 4 步 改代码：PR#52 实施（app/segmentation/text\_prompt.py 的 DEFAULT\_THRESHOLD 0.30→0.24，注释引用 CR-001 与 SRS V3.1；TC-F04-01/04-02 断言基线同步更新，先红后绿）。PR 描述引用 CR-001 编号，评审人对照变更单逐项核对影响面清单。

### 6.12.4 变更闭环验证

四步走完不算完，还要验证闭环：全局搜索旧值 0.30 确认清零（文档与代码）、精度复测（完整率 84.2%/正确率 88.6% 达标并归档）、RTM 更新（RTM v4 中 FR-04.1 行的基线版本列同步）。变更单最后有一栏『闭环确认』，三项验证打勾+CCB 关闭签字，CR-001 才归档。

### 6.12.5 反面教材：不走流程的变更

课堂上对照讲一个反例（样例翻车记录 FR-03 的变体场景）：某组口头决定把分页大小从 20 改成 50，只改了后端常量。后果链条：前端按 20 设计的 UI 出现滚动异常→契约测试没覆盖分页参数没拦住→验收演示时评委翻到第二页发现错位→追溯时发现 SRS、spec、代码三处不一致，无法说清哪个是『正确』的。全组在配置管理项上被扣分。这个反例与 CR-001 正例对照，学生对流程必要性的认识会深刻得多。

### 6.12.6 人机协同要点

- AI 生成影响面清单：把变更描述喂给 AI，让它全局搜索所有出现相关参数/条款的工件，输出清单初稿。AI 的全局搜索比人快且不易漏，CR-001 的影响面清单就是 AI 初稿+人工补了前端刻度一项；
- AI 起草变更单：按模板填好各字段，动因与数据论证段人工写（这是思考，不是文书）；
- AI 执行一致性检查：变更后让 AI 扫全部工件查残留旧值，输出残留清单。注意 AI 对『语义上的旧值』识别有限（比如文档里用中文描述的阈值），人工抽查兜底。

### 6.12.7 本节清单

- 背得出四步流程及顺序理由（spec 是源头，代码在最后）；
- 会填变更单全字段，影响面清单与回退方案是重点；
- 理解闭环验证三项（旧值清零、回归、RTM 更新）；
- 能用 CR-001 正例与分页反例解释流程价值。

### 6.12.8 RTM v4 回填：让追踪矩阵活到交付

需求追踪矩阵（RTM）在任务 2 建了 v1（需求→验收标准），任务 3 挂了设计与测试（v2/v3），任务 4 要回填到 v4：每条 FR 挂接代码位置、实现 PR、测试用例与测试状态。RTM v4 是真实回填结果。本节讲回填的操作方法与防失真纪律。

#### **RTM v4 的列结构**

RTM v4（Excel）的列结构，从左到右就是一条需求的生命周期：

| 列         | 内容             | 填 写<br>时机              | 示例（FR-04.1）                                             |
|-----------|----------------|------------------------|---------------------------------------------------------|
| 需 求<br>编号 | FR/NFR 编号      | 任 务<br>2 已<br>填        | FR-04.1                                                 |
| 需 求<br>摘要 | 一句话            | 任 务<br>2 已<br>填        | 文本提示分割与阈值管理                                             |
| 验 收<br>标准 | AC 编号          | 任 务<br>2 已<br>填        | AC-FR-04.1-a/b                                          |
| 设 计<br>章节 | HLD/DD 章节<br>号 | 任 务<br>3 回<br>填        | DLD 5.2 文本分割模块                                          |
| 代 码<br>位置 | 文件:类/函数        | 任 务<br>4 回<br>填        | app/segmentation/text_prompt.py:<br>DEFAULT_THRESHOLD 等 |
| 实 现<br>PR | PR 编号          | 任 务<br>4 回<br>填        | PR#48~52, PR#52(CR-001)                                 |
| 测 试<br>用例 | 用例编号           | 任 务<br>3/4 回<br>填      | TC-F04-01~05, PT-03, AT-01                              |
| 测 试<br>状态 | 通过/阻塞/未测       | 任 务<br>4 持<br>续 更<br>新 | 通过（夜间构建，AT 复测归档）                                        |
| 变 更<br>记录 | CR 编号          | 随时                     | CR-001                                                  |

## 回填的操作流程

- 第 1 步，PR 合并时即时登记：PR 模板里有一栏『影响的需求编号』，合并时把 PR 号填进 RTM 对应行。即时登记是关键——攒到迭代末一起填，必然靠回忆，必然失真；
- 第 2 步，代码位置用工具辅助定位：按需求编号搜代码注释（样例项目纪律：核心函数 docstring 里写 Implements: FR-04.1），搜不到的再人工找。  
check\_refs.py 脚本会反向校验：代码里声明的需求编号必须在 RTM 存在；
- 第 3 步，测试状态从 CI 取数：夜间构建的测试报告映射回用例编号，脚本更新 RTM 的测试状态列。人工只做异常行处理（构建失败、用例缺失）；
- 第 4 步，迭代评审前全量体检：24 条 FR + 16 条 NFR + 3 条精度指标逐行过一遍：有没有需求没挂代码？有没有代码挂了但 RTM 没登记？有没有测试状态是『未测』的 MVP 条目？体检清单输出留档。

## RTM 的三个诊断用法

RTM 不是应付检查的表格，是诊断工具。教会学生三种诊断：

- 正向诊断（需求→实现）：MVP 的 11 条需求每条都必须能一路点下去：需求→设计→代码→PR→测试→通过。任何一环断，就是交付风险。迭代评审用这个方法逐条过 MVP；
- 反向诊断（代码→需求）：抽查代码仓任何文件，问它服务哪条需求。答不上来的代码是『孤儿代码』——要么过度工程（删），要么需求遗漏（补 SRS）。样例项目反向诊断曾发现一个 150 行的格式转换工具类没有对应需求，评审后删除；
- 变更波及诊断：变更时按 RTM 查出受影响需求关联的全部工件，作为影响面清单的骨架。CR-001 的影响面清单一半内容直接来自 RTM 查询。

## 人机协同要点

- AI 维护关联建议：PR 提交时让 AI 读 diff 与 RTM，建议该 PR 关联哪些需求编号，人工确认。样例项目 AI 建议的准确率约 80%，错误主要是把『使用了某模块』误当成『实现了某需求』；
- AI 做 RTM 体检报告：把 RTM 导出喂给 AI，让它列出断链（无代码的需求、无需求的代码、未测的 MVP 条目），人工逐条裁决；

- 纪律底线：RTM 的最终状态由人负责，AI 只做建议与检查。验收时评委按 RTM 抽查，失真由组长承担责任。

## 本节清单

- 掌握 RTM v4 九列结构与各列填写时机；
- 执行四步回填流程，PR 即时登记是纪律核心；
- 会用正向/反向/变更波及三种诊断；
- 明确 AI 建议可采纳但责任在人。

## 6.13 代码文档同步与合格仓库解剖课

任务 4 结束时，spec.md、SRS、OpenAPI、RTM、代码五件工件必须一致。本节讲不一致是怎么发生的、日常怎么防、交付前怎么体检。

### 6.13.1 不一致的三个发生通道

- 通道一 代码先行：改了代码没改文档（最常见的偷懒）。防线：变更四步流程强制 spec 先行；
- 通道二 AI 回写：AI 按旧 spec 生成代码覆盖新逻辑——上下文里 spec 快照过期。防线：context pack 每次任务现做，不用库存；
- 通道三 并行漂移：两人同时改关联工件（一个改 OpenAPI 一个改代码），合并时各对一半。防线：工件的修改窗口在站会上声明，关联改动串行化。

### 6.13.2 日常防线：三个自动化检查

- contract\_check.py: 代码↔OpenAPI 一致（每次 CI）；
- check\_refs.py: 代码↔RTM 的需求编号双向闭合（每次提交）；
- P-12 文档-代码一致性扫描（AI 辅助，每周）：让 AI 对照 spec.md 与代码找语义级漂移（脚本查不了的，比如 spec 说『退避 30/60/120』而代码注释写『指数退避』——字面不矛盾但语义已漂移）。AI 输出漂移疑点清单，人工裁决。样例项目每周扫出 2~4 条疑点，约一半是真漂移。

### 6.13.3 交付前体检（D15 固定动作）

五件对齐体检单，逐项打勾并留证据：

- spec.md 与 SRS：条款编号一一对应，内容无矛盾（AI 交叉比对+人工抽 10 条复核）；
- OpenAPI 与代码：contract\_check 全绿（附当次 CI 记录）；
- RTM 与代码：check\_refs 全绿；24 条 FR 每条有代码位置与测试状态（MVP 5 项必须为『通过』，且代码位置须精确到文件与函数，不接受只写模块名）；
- spec 与测试：每条 AC 有对应用例且最近构建通过；
- 全局残留扫描：变更涉及的历史值（如 0.30）全局清零。

### 6.13.4 不一致的修复优先级

体检发现不一致时按此顺序处理：涉及验收指标的（立即修，阻塞交付）→涉及 MVP 条目的（当日内）→一般条目的（登记技术债，任务 5 前清零）。修复必须改到源头——代码错了改代码，文档错了改文档，禁止『两边都改得含糊一点凑合对上』。

### 6.13.5 本节清单

- 说出三个不一致发生通道及各自防线；
- 执行每周 P-12 语义级扫描；
- D15 按五项体检单完成对齐并留证据；
- 不一致修复改到源头，禁止含糊对齐。

### 6.13.6 sprint\_repo 逐文件剖析：一个合格代码仓的解剖课

本节带学生逐目录走一遍样例代码仓 sprint\_repo，讲清每个部分为什么在那里、评审时看什么。这是任务 4 验收时代码仓检查的对照标准。

#### 仓库顶层结构

```
sprint_repo/      README.md          # 仓的入口：架构一页图 + 十分钟跑通指引
rules/constitution.md# 项目宪章与编码规则（AI 与人共同遵守）  docs/          # spec.md
快照、ADR、联调单归档  src/      api/          # HTTP 层：路由、请求模型、错误码  queue/
# GpuQueue: 任务队列与调度  worker/          # GPU/CPU Worker: 推理执行  geo/
# 地理计算：分块、坐标转换、掩膜后处理  security/          # 认证、输入校验、口令哈希
tests/      unit/          # 单元测试（随 PR 跑）  contract/          # 契约测试数据
perf/          # 性能测试（夜间构建）  accuracy/          # 精度测试（夜间构建）
scripts/      contract_check.py  # 契约检查（2.2 节）  security_redline.py# 安全红线扫描
（2.4 节）  check_refs.py  # 需求编号反向校验（2.11 节）  selfcheck.py  # 交付自检（11 项断言）
chaos/          # 故障演练脚本（2.7 节）  .github/workflows/  # CI 五道闸配置
```

## README：十分钟跑通承诺

README 的验收标准：新成员（或评委）仅凭 README 能在十分钟内跑通系统。结构固定四块：一句话项目说明+架构一页图、环境要求（Python 版本、GPU 可选）、三步启动命令、验证方式（跑 `selfcheck.py` 应输出 11/11）。教学上要求每组在任务 4 验收前做一次『十分钟实测』——找一位别组同学照 README 操作，卡住超过两分钟的地方就是 README 要改的。

## rules/constitution.md：项目的宪法

宪章内容（约 2 页）：编码规范要点、评审清单 R-01~R-07、安全红线 R1~R4 引用、AI 使用公约（哪些环节必须用 AI 约束提示词、哪些产出必须标注 AI-generated）、禁止事项清单。它的双重读者是人评审者与 AI——开工仪式时喂给 AI 的就是这份文件的摘要。4.2 节详解其编写方法。

## src 各模块的评审看点

| 模块       | 核心类/函数                                                                                      | 评审看什么                                                          |
|----------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| api      | 路由注册、请求模型                                                                                   | 与 OpenAPI 一致性（ <code>contract_check</code> 已查，人工抽查语义）；错误码是否按规范 |
| queue    | <code>GpuQueue</code> 、 <code>Task.transit</code> 、 <code>backoff_seconds(30/60/120)</code> | 状态机完整性（Task 各状态转换是否有测试）；退避参数与 spec 一致                          |
| worker   | GPU/CPU Worker、心跳                                                                           | 降级路径代码（kill 演练覆盖）；心跳超时值与 PE-09 指标的关系                           |
| geo      | 分块、坐标转换、掩膜后处理                                                                               | 坐标参考系处理（常识替代需求高发区）；分块重叠逻辑与性能优化的关联                              |
| security | 认证、校验、哈希                                                                                    | 四条红线逐条对照；这部分代码不接受 AI 主导生成                                      |

## tests 的组织纪律

三个分离：随 PR 跑的（unit，秒级）与夜间跑的（perf/accuracy，分钟级）物理分目录，用 pytest marker 区分；测试文件命名与被测模块一一对应（test\_queue.py 测 queue/），评审时按名索骥；每个测试用例的 docstring 写明对应的 AC 或分支意图——这是 RTM 回填『测试用例』列的数据源，也是防镜像测试的评审抓手。

### **scripts 与 CI：质量防线的物理形态**

四个脚本各守一条防线，CI 五道闸（lint→单测+覆盖率→check\_refs→contract\_check→pip-audit+红线）按『快的在前、慢的在后、失败即停』排序。本地 pre-commit 钩子跑 lint 与红线扫描的快速子集，把最低级错误拦在提交前。教学检查点：学生必须能讲出五道闸各自的拦截对象，并举出本组被每道闸真实拦截过的次数（《迭代冲刺计划与燃尽报告》数据：CI 共拦截 9 次）。

### **selfcheck.py：交付自检的 11 项断言**

selfcheck.py 是交付前的最后一道本地闸，11 项断言覆盖：环境（Python 版本、依赖齐全）、数据（冻结影像集哈希）、功能（核心流程冒烟）、质量门（覆盖率达标、红线全绿、契约一致）。验收演示的第一步就是当着评委跑它——11/11 通过是最直观的『交付就绪』信号。

### **本节清单**

- 能画出仓库结构并讲出每部分的评审看点；
- 理解 README 十分钟承诺及其验收方式；
- 掌握 tests 三分离组织与用例 docstring 纪律；
- 讲出 CI 五道闸排序逻辑与 selfcheck 的定位。

## **6.14 PR #47 全流程复盘与编码缺陷案例集**

本节把样例项目最有代表性的 PR#47（交互分割功能，对应 FR-03.1/03.2）从任务卡到合并的全过程逐帧复盘，作为 PR 门禁五环（1.4 节）的完整示范。材料来源：PR#47 评审样例（PR#47 评审样例）。

### **6.14.1 时间线总览**



| 阶段        | 耗时    | 关键事件                                    |
|-----------|-------|-----------------------------------------|
| 任务认领与准备   | 0.5 天 | 从 tasks.md 领 T-19；制作 context pack；开工仪式  |
| AI 初稿生成   | 1 轮会话 | P-04 四段式，先出实现计划，人工审计划后出代码               |
| 自查与本地验证   | 0.5 天 | 跑单测、contract_check、selfcheck；发现 2 处问题自修 |
| 提交 PR 与评审 | 1 天   | 两轮评审意见，7 条修改，补 3 个测试                    |
| CI 与合并    | 0.5 天 | CI 第一次红灯（增量覆盖率 76%<80%），补测试后转绿合并        |

### 6.14.2 准备阶段：context pack 实物

T-19 的 context pack 五段（缩略）：任务卡（交互分割：用户点选/框选提示→常驻 Worker 推理→掩膜矢量化回传）；spec 条款（AC-FR-03.1/03.2 原文，含单轮≤3s 预算与坐标域约定）；接口签名（PromptEncoder.encode、SamInferClient.infer 真实签名）；约定清单（存储基准 CGCS2000/EPSG:4490、对外输出默认 EPSG:4326、掩膜以 GeoJSON 矢量回传、请求裁剪 1024×1024）；禁止事项（不动 GpuQueue 公共接口、不新增依赖）。这份 pack 约 40 行，制作 25 分钟——评审时这份 pack 与 PR 一起提交，是『提示词质量』的检查证据。

### 6.14.3 生成阶段：两轮产出的对比

- 第一轮 AI 输出实现计划：6 步，人工审时发现第 4 步把阈值参数设计成 0~100 整数——违反 context pack 约定清单（AI 的常识替代需求模式发作），纠正后重新确认计划；
- 第二轮产出代码：约 380 行+9 个测试用例。自查发现 2 处问题：一个边界（空提示点集）未处理、一个 docstring 需求编号标错（已按纪律人工修正）。

### 6.14.4 评审阶段：7 条意见全记录

PR#47 评审样例收录了完整评审对话。7 条意见的分类：事实类 1 条（一个 rasterio 调用要求附文档链接，查后确认存在）；边界类 2 条（超大提示框的性能上限、掩膜为空的返回约定）；意图类 1 条（为什么掩膜矢量化放在 API 层而非 geo 层——

答复后评审认可，但要求把理由写进代码注释）；数值类 1 条（裁剪尺寸 1024<sup>2</sup> 要求注明 PE-01 链路预算出处）；测试类 2 条（补并发提交用例、补提示坐标越界用例）。全部修改后 Approve。注意这 7 条的分布——正好覆盖评审五查的每一类，这就是五查清单有效的实证。

### 6.14.5 CI 红灯与增量覆盖率

首次 CI：五道闸中『单测+覆盖率』红灯——新增代码 214 行，测试覆盖 163 行，增量覆盖率 76.2%，差 3.8 个百分点。补了 3 个异常分支用例后 89.7% 转绿。这个红灯的教学价值：增量门禁真实发挥作用，且缺口不大时补测试的成本约 1 小时——门禁设 80% 是『够得着但不放水』的实证。

### 6.14.6 复盘结论

- 时间分布：准备 20%、生成 15%、自查+评审+返修 45%、CI 与合并 20%——AI 把『写代码』压到最短，总工时的重心转移到了验证与评审，这正是 AI 时代的工时结构；
- 四环证据链完整：context pack→会话记录→评审对话→CI 记录，四环全部留档，答辩可逐环追溯；
- 后续关联：迭代二的 CR-001（文本提示阈值默认值 0.30→0.24，PR#52）变更时，RTM 与影响面清单直接定位到 FR-04 行的关联 PR 与用例——良好的登记习惯在变更时兑现了价值。

### 6.14.7 本节清单

- 能按时间线复述 PR#47 全过程与各阶段工时占比；
- 理解 context pack 作为评审证据的意义；
- 掌握 7 条评审意见与五查清单的对应关系；
- 能讲清增量覆盖率红灯的处理过程与门禁定标逻辑。

### 6.14.8 编码期缺陷案例集：10 则真实 bug 的教学解剖

本节收集样例项目两个迭代中 10 则有代表性的缺陷，按『发现环节×根因』组织，每则给出症状、定位过程、修复与预防。用途：课堂讲解、评审培训题库、学生自查对照。

## 案例速查表

| 编号   | 缺陷                            | 发现环节       | 根因类别      | 预防机制                |
|------|-------------------------------|------------|-----------|---------------------|
| D-01 | rasterio 编造 API 运行期崩溃         | 运行期        | 编造 API    | R-07 评审规则           |
| D-02 | 阈值单位前后端不一致<br>( 0~100 vs 0~1) | 联调         | 常识替代需求    | 联调单+契约测试补例          |
| D-03 | 队满返回 500 而非 429               | 联调         | spec 未读细  | 契约测试补响应码断言          |
| D-04 | 分块重叠区重复推理                     | 性能测试       | 实现缺优化意识   | 性能分段打点              |
| D-05 | Task 死信状态无回收路径                | 走查演练       | 设计漏洞      | 故障走查制度              |
| D-06 | 降级时用户无通知                      | 故障演练       | 通知逻辑只判启动态 | 演练剧本覆盖              |
| D-07 | 精度统计误用全局口径                    | 评审         | AI 默认实现   | 指标定义喂原文             |
| D-08 | docstring 需求编号错行              | check_refs | 编号错配      | 脚本反向校验              |
| D-09 | 日志含明文口令字段                     | 红线扫描       | AI 教学式代码  | security_redline.py |
| D-10 | 超时参数与任务规模脱节                   | 评审         | 数值无出处     | 凡数字必问出处             |

## 三则深度解剖

### D-02 阈值单位不一致

症状：联调时前端传 `threshold=35`，后端按 0~1 解析为『35 倍上限』直接报错。定位：两端日志对比一眼定位单位错配。根因：CR-001 前的约定靠口头，AI 写前端校验时按常识选了 0~100（人习惯的百分数）。修复：PR#53 统一 0~1 浮点并写进 OpenAPI。预防：反常识约定必须进 OpenAPI 与 context pack 约定清单——这类缺陷的共性是『正确的代码实现了错误的约定』，测试单端都过，只有联调能抓。

#### D-05 Task 死信无回收

症状：演练中 kill 掉 GPU Worker 后，其手中 in-progress 任务永远卡住。定位：走查时追问『Worker 死了任务去哪』，代码里查无此路径。根因：状态机设计只考虑了正常流转（`queued`→`running`→`done/failed`），没有外部失效驱动的转换。修复：加心跳超时检测+Task.transit 回收转换，补 3 个用例。预防：状态机评审必须画全『异常驱动转换』，不能只画快乐路径——这是设计评审清单的常驻项。

#### D-09 日志明文口令

症状：登录失败日志把请求体整体打印，含 `password` 字段明文。定位：`security_redline.py` 扫描报 R1。根因：AI 生成的错误处理代码用了『调试友好』的整体打印——公开语料里教学代码的常见写法。修复：日志输出走脱敏过滤器，敏感字段白名单制。预防：红线脚本+纪律『日志输出一律过脱敏层』。教学点：AI 的『调试友好』倾向与生产安全要求天然冲突，凡 AI 写的日志/异常输出都要用这副眼镜看一遍。

### 案例库的使用方法

- 评审培训：新评审者上岗前，给 10 则案例的『症状』部分，让其给出定位思路与预防建议，再对照答案；
- 自查对照：提交 PR 前扫一遍速查表，问『我的 PR 有没有犯这十类』；
- 缺陷入库：本组新发现的代表性缺陷按同格式补入案例库，迭代评审时过一遍新增条目——案例库应当随项目生长。

### 本节清单

- 熟悉 10 则案例的根因类别分布（AI 相关 6 则、流程相关 4 则）；
- 能讲透 D-02/D-05/D-09 三则的完整链路；
- 会把案例库用于培训、自查与持续生长。

## 6.15 过关自检与交接

本节给出任务 4 的完整交付清单、封版检查流程，以及任务 4 产物如何成为任务 5（测试与验收关）的输入。

### 6.15.1 交付包清单（对照研发期样例文档：迭代计划、Prompt 模式库、翻车记录、PR 评审样例、变更单、站会周报、RTM v4、代码仓）

| 类别    | 交付物             | 形式   | 质量门                   |
|-------|-----------------|------|-----------------------|
| 管理    | 迭代冲刺计划与燃尽报告     | docx | 两轮迭代数据齐全              |
| AI 资产 | Prompt 模式库      | docx | ≥8 模式含反模式，均有使用记录      |
| AI 资产 | 翻车记录            | docx | ≥1 则完整五问分析            |
| 过程    | 代表性 PR 评审记录     | md   | 含完整评审对话与修改闭环          |
| 过程    | 变更单             | md   | 四步证据+闭环确认             |
| 过程    | 站会纪要与周报         | md   | 覆盖冲刺全程，无缺日            |
| 质量    | RTM v4          | xlsx | 24 FR 全挂接，MVP 全通过     |
| 代码    | sprint_repo 代码仓 | repo | selfcheck 11/11，CI 全绿 |

### 6.15.2 封版检查流程（D15 下午）

- 第 1 步，代码封版：打 tag（sprint2-final），此后只允许文档类提交；
- 第 2 步，五件对齐体检：按 4.3 节体检单执行，证据截图归档；
- 第 3 步，文档齐套检查：对照 40.1 清单逐件核对，缺件当日补齐；
- 第 4 步，演示彩排：完整跑一遍演示流程并计时，录屏兜底文件生成；
- 第 5 步，封版签字：组长+各方向主攻在交付清单上签字，扫描件归档。

### 6.15.3 通往任务 5：移交什么

任务 5（测试与验收关）将从任务 4 继承四样东西，交接质量直接决定下一关的起点：

- 可运行的系统与测试环境：含环境文档与 selfcheck，任务 5 的系统测试在此基础上做；
- 测试基线：现有单测/契约/性能/精度测试套件，任务 5 补充系统级与用户验收测试；
- RTM v4：任务 5 的验收测试用例继续挂接，演进为 v5；
- 遗留问题清单：未清零的技术债、已知缺陷、性能/精度余量——诚实移交，不藏雷。任务 5 评审第一项就是核对移交清单的诚实度。

#### 6.15.4 任务 4 过关条件自评表

- PR 门禁真实运转：每个合并的 PR 有五环证据，抽任一可讲；
- 测试护栏达标：覆盖率过门禁、性能精度实测达标、演练至少一轮；
- 变更全部走四步：无 spec 漂移；
- AI 资产成库：模式库与翻车记录有真实内容且被实际使用；
- 度量可复盘：三层数据（流动/质量/效率）齐全且来自工具。

#### 6.15.5 写给下一关的三句话

- 任务 5 的验收标准在任务 2 就写好了——把 SRS 的验收章节再读一遍；
- 你们在任务 4 攒下的每条防线（CI、红线、契约、演练）在任务 5 都还会再用一次，别拆；
- 遗留问题清单是最容易被扣分的文档——诚实是唯一的策略。

#### 6.15.6 编码期周体检清单（每周五固定执行）

##### A. 流程健康（10 分钟，组长执行）

- 本周所有合并 PR 是否都有评审记录与 AI 使用声明？（git log 抽查）
- 是否有 PR 绕过了 CI 或使用了 --no-verify？（有则本周流程项不合格，需组内说明）
- 变更是否都有变更单且四步齐全？（对照本周 diff 中出现的 spec 关键词变化）
- RTM 是否有未回填行？（导出统计空行数）

##### B. 质量健康（15 分钟，测试主攻执行）

- CI 拦截次数与分布是否异常？（零拦截与激增都需解释）

- 覆盖率趋势：总体与核心是否非降？增量覆盖率本周 PR 是否全过 80%？
- 测试失败史：本周夜间构建失败几次，根因是否都闭环？
- 技术债台账：新增几笔、偿还几笔、有无超期未还的？

#### C. AI 协同健康（10 分钟，模式库管理员执行）

- 本周各模式使用次数与采纳率（异常下跌的模式要查原因）；
- 新增翻车记录是否入库？纠正措施是否有 owner 与期限？
- 规则探测做没做？结果如何？
- 上下文事故（长会话回退类）发生几次？是否都执行了会话切换清单？

### 6.15.7 验收演示脚本模板（15 分钟版）

| 时段          | 环节    | 操作与台词要点                                           | 风险与兜底            |
|-------------|-------|---------------------------------------------------|------------------|
| 0:00-1:30   | 开场    | 一句话项目定位+MVP 范围（11 条 FR）+今日演示议程                    | PPT 双份备份         |
| 1:30-4:00   | 可信度建立 | 现场跑 selfcheck.py（11/11）+展示 CI 最近构建全绿              | 提前 30 分钟跑过一遍预热   |
| 4:00-9:00   | 功能演示  | 按 RTM 的 MVP 行逐条演示：全自动分割→交互分割→任务管理；每条指认对应 FR 编号    | 录屏兜底；每功能控制在 90 秒 |
| 9:00-12:00  | 质量证据  | 展示性能实测（2.6s vs 预算 3.0s）、精度台账（84.2%/88.6%）、故障演练时间线 | 数据页预渲染           |
| 12:00-14:00 | 过程证据  | 讲一个 PR 的五环旅程+一次变更的四步流程（CR-001 式）                  | 选全组最熟的案例讲        |
| 14:00-15:00 | 收尾    | 遗留问题诚实清单+任务 5 计划一句话                               | 不许说『没有问题』        |

### 6.15.8 演示的反模式

- 功能流水账：逐页面点一遍却不提需求编号——评委无法核对验收依据；
- 质量空口：说『性能很好』而不给环境与数据；

- 过程隐身：只展示成品，不讲五环、变更、翻车——任务 4 的一半分数在过程；
- 超时失控：功能演示超时要果断切录屏，保住质量与过程环节的时间。

#### **6.15.9 本节清单**

- 周体检三个区块 35 分钟内完成，结果进周报；
- 演示脚本按模板编制并彩排至少两次；
- 四个演示反模式全组知晓。



## 第 7 章 第五关：质量门禁

本关组织质量攻坚：测试设计挂接 RTM，单元、集成、系统三级执行，八大质量特性逐项专项验证，破坏性自测（组内红队）与故障演练。按 GB/T 15532 组织，按 GB/T 25000.51 验收——只看报告数据，不接受「我觉得没问题」。

### 7.1 关卡导论：质量是构建出来的

任务 5（质量门禁，D16~D17）是整个通关实践的交付线攻坚阶段。前面四关依次回答了『做什么』（招投标与需求）、『怎么做』（设计）、『做出来』（研发冲刺），任务 5 回答最后一个问题：『凭什么说做好了』。它的答案不允许是『我们觉得没问题』——过关条件明确规定：只看报告数据。

#### 7.1.1 任务与交付物全景

| 交付物         | 对应样例        | 核心内容                                          |
|-------------|-------------|-----------------------------------------------|
| 测试报告·测试设计篇  | 《测试报告》      | 用例集（UT 312/IT 86/ST 64）挂接 RTM，六种设计技术，八大特性专项映射 |
| 测试报告        | 《测试报告》      | 三级测试执行结果+八大特性专项结果+缺陷统计+RTM 闭环核查               |
| 测试报告·性能压测专项 | 《测试报告》      | S1~S4 场景、P50/P95/P99 分位数、容量边界与扩容建议            |
| 质量特性自评表     | 随交付包归档      | GB/T 25000.10 八大特性 28 子特性逐项打钩+证据索引            |
| 破坏性自测记录     | 《测试报告》      | 30 条红队用例、有效/无效判定、裁决记录                         |
| 故障演练验收记录    | 《故障演练验收记录》  | 教师随机故障×3 的分钟级时间线与恢复验证                         |
| 缺陷看板与修复记录   | 《缺陷看板与修复记录》 | 18 条缺陷全明细（级别/环节/修复 PR/回归构建）+统计闭环              |

#### 7.1.2 三条过关硬杠（正式评审）

- 高危漏洞清零：SCA 与渗透测试发现的高危项必须修复并复测为零，不接受『已知未修』；

- 集成测试 100% 通过：86/86，一个红点都不行；首轮失败可以，但最终构建必须全绿；
- RTM 闭环：24 条 FR+16 条 NFR 每条至少 1 条状态为『通过』的测试用例，且可点开看执行记录。

三条硬杠的共同特征是机器可核查——这正是本关要建立的工程观：验收不是表态，是证据链。

### 7.1.3 与前四关的衔接

任务 5 不是孤立阶段：测试计划在任务 3 设计冻结时就已发布（测试左移）；单元/契约/性能/精度测试套件在任务 4 已在 CI 里每日运行；RTM v4 已挂到代码与用例。任务 5 要做的是补齐系统级与验收级证据、组织对抗与演练、把全部数据收口成报告。如果前四关偷工减料，任务 5 会全部暴露——这是课程设计的有意安排。

### 7.1.4 总方法论：质量是构建出来的，不是测出来的

#### 一个反直觉的论断

『质量是构建出来的，不是测出来的』——测试只能发现缺陷，不能注入质量。任务 5 的真正目的不是『测一遍』，而是用系统化的验证手段证明前四关构建的质量真实存在，并把发现的少量残留缺陷闭环。样例项目的数据佐证了这个论断：18 条缺陷中系统测试只发现 8 个，其余 16 个来自红队自测、故障演练、压测、走查扫描——如果只靠『测一遍』，三分之二的缺陷会带进交付。

#### 双标准框架：15532 管过程，25000.51 管判定

- GB/T 15532-2008《计算机软件测试规范》：管测试怎么组织——测试级别（单元/集成/系统）、用例结构、记录与报告要求。《测试报告》的章节骨架直接对标它；
- GB/T 25000.51-2016《就绪可用软件产品的质量要求与测试细则》：管验收怎么判定——产品质量要求以 GB/T 25000.10 八大质量特性为框架（功能适合性、性能效率、兼容性、易用性、可靠性、安全性、维护性、可移植性），验收结论逐项给证据。质量特性自评表就是按这八项组织的；
- 二者关系：15532 告诉你测试活动怎么做，25000.51 告诉你验收答卷怎么交。任务 5 的文档体系（设计→执行→报告→自评表）就是这两个标准的落地。

## 八大质量特性的教学化操作

每个特性专项回答三个问题：测什么（子特性分解）、怎么测（手段与用例）、凭什么过（判定标准与证据）。本书第 11~21 章逐一展开八个专项。先建立地图：

| 特性    | 一句话核心问题   | 样例手段           | 样例判定                           |
|-------|-----------|----------------|--------------------------------|
| 功能适合性 | 承诺的功能都对吗  | ST 64 例+RTM 闭环 | MVP 全过、★款全过                    |
| 性能效率  | 够快、扛得住吗   | k6 压测 S1~S4    | $P95 \leq 3s/1km^2 \leq 10min$ |
| 兼容性   | 目标环境都跑吗   | 浏览器×格式矩阵       | 12 组合全过                        |
| 易用性   | 用户会用、好用吗  | 启发式走查 24 点     | 严重可用问题为零                       |
| 可靠性   | 出故障能扛能恢复吗 | 6 故障剧本注入       | 恢复达标、零丢失                       |
| 安全性   | 扛得住攻击吗    | SCA+渗透+等保自查    | 高危清零                           |
| 维护性   | 好改、好查吗    | SonarQube 扫描   | 阻断级为零                          |
| 可移植性  | 换个环境装得上吗  | 干净虚拟机部署        | ≤2h 一键成功                       |

## 本关的AI 协同总原则

测试关的 AI 用法与研发关有质的不同：研发关 AI 是『生产者』（写代码），测试关 AI 更多是『攻击者+分析者』（生成破坏性用例、分析日志、找覆盖盲区）。但判定权全面收归人工——精度结论、安全结论、验收结论三级结论必须人工背书。第 29~32 章详述。

## 本节清单

- 背得出三条过关硬杠及其机器可核查性；
- 讲清 15532 与 25000.51 的分工；
- 画出八大特性的『测什么/怎么测/凭什么过』地图；
- 理解测试关 AI 角色的转变与判定权回收。

## 7.2 测试设计总览与用例设计六种技术

本节以《测试报告》测试设计篇（第 3~10 章）为教材，讲清测试设计的输入、骨架与核心决策。测试设计回答：用什么用例、按什么结构、覆盖到什么粒度，才能把需求基线转化为可执行的验证。

### 7.2.1 输入盘点：设计之前先对齐基线

- 需求基线：SRS V3.1（24 FR + 16 NFR + 3 AT、MVP 5 项、★13 款）——用例的需求出处；
- 接口基线：OpenAPI v3（6 端点）——IT 级用例的契约出处；
- 精度基线：冻结影像集 5 景+参照真值（哈希登记）——AT 用例的输入出处；
- 既有资产：任务 4 已运行的 UT/契约/性能/精度套件——任务 5 复用而非重写，系统级用例是主要新增量；
- 测试计划：级别划分、范围、准入准出——测试设计是它的展开，不允许冲突。

### 7.2.2 用例三级体系：UT/IT/ST 的分工

| 级别    | 验证对象         | 执行时机    | 数量  | 失败语义         |
|-------|--------------|---------|-----|--------------|
| UT 单元 | 单个函数/类的逻辑分支  | 随 PR+夜间 | 312 | 局部逻辑错，修复成本低  |
| IT 集成 | 接口契约与跨组件链路   | 每日构建    | 86  | 组件间假设不一致     |
| ST 系统 | 端到端业务场景与 NFR | 任务 5 集中 | 64  | 验收风险，直接关联 FR |

三级不是随意分的：UT 便宜但离需求远，ST 贴近验收但贵。金字塔原则——大

量 UT 兜底逻辑，适量 IT 锁住契约，精选 ST 验证价值流。样例项目 312:86:64 的比例可供参考（约 5:1.4:1）。

### 7.2.3 用例编号与 RTM 挂接

执行期编号 UT/IT/ST-nnn 与 RTM v4 的设计期编号 TC-Fxx-nn 建立双向映射（一条设计用例可展开为多条执行用例）。映射表随 RTM v5 归档——评委从任一需求编号能一路点到执行记录与构建号，这是过关条件三的操作化。孤儿用例（无需求出处）与孤儿需求（无用例覆盖）都视为断链，check 脚本+人工抽查双重核查。

### 7.2.4 优先级分配

P0=★款或 MVP 必过（34 条 ST 用例覆盖 MVP 5 项）、P1=核心功能与关键 NFR、P2=一般。优先级的用途是执行排序与回归裁剪：时间不够时先保 P0 全过，P1 抽样，P2 记录风险。注意优先级从需求来（★款与 MVP 清单），不是测试人员拍脑袋。

### 7.2.5 人工方法要点

- 先建覆盖矩阵再写用例：24 FR × 三级覆盖矩阵打钩，确认无漏项后才动手写用例正文——防止『写得很爽但漏了一条需求』；
- 用例评审会：用例集与代码评审同标准——双人评审，重点审预期结果的可判定性；
- 试跑校准：正式执行前先试跑 10% 用例，校准环境、数据与耗时估算。

### 7.2.6 人机协同要点

- AI 生成覆盖矩阵初稿：喂 SRS 需求清单，让 AI 输出 需求×级别 的建议覆盖矩阵，人工审定——AI 在穷举覆盖上的完整性优于人；
- AI 起草用例骨架：按模板批量生成，人工填充预期结果的精确数值（这是 AI 最易错处，详见 1.6 节与 3.8 节）；
- AI 做查漏：把 SRS 条款与用例清单喂给 AI，问『哪些需求没有用例』，输出疑点清单人工裁决。

### 7.2.7 本节清单

- 说清三级用例的分工语义与金字塔比例；
- 执行『覆盖矩阵先行』的设计顺序；

- 掌握执行编号与 TC 设计编号的双向映射；
- 优先级从需求来，不从感觉来。

### 7.2.8 用例设计六种技术精讲

《测试报告》第 3 章列出了六种设计技术及其占比。本节逐一讲清每种技术的人工操作方法、在本项目的真实应用、以及 AI 辅助点。掌握标志：给定任一需求条款，能说出该用哪几种技术、为什么。

#### 等价类划分（约30%）

方法：把输入域分成若干等价类，每类取一个代表值——测一个等于测一类。关键在分类依据必须来自需求而非直觉。样例：置信度阈值参数分四类——有效类 [0,1]、无效类 小于 0、无效类 大于 1、类型错误类（字符串/空）。AI 辅助：让 AI 按需求条款穷举等价类划分方案，人工核对分类依据是否每条都能指回 SRS 原文。AI 常按『常见做法』分类（如擅自把 0~1 当成 0~100），这正是核对点。

#### 边界值分析（约25%）

方法：缺陷密集在边界，取 边界-1/边界/边界+1 三点。样例：分片上传的片大小 边界（8MB-1B/8MB/8MB+1B）、分页大小（0/1/上限/上限+1）、提示坐标（-1/0/影像宽/影像宽+1——BUG-134 负坐标缺陷就是边界用例缺失的代价）。AI 辅助：AI 列边界候选很全，但会编造需求里没有的边界（比如臆造『最大文件 2GB』），每个边界必须要求它注明出处条款，注不出的删掉。

#### 场景法（约20%，ST 级主力）

方法：按业务流把多个功能串成端到端场景，分基本流+备选流。样例 ST-009 全自动分割端到端：上传→金字塔→提交分割任务→队列调度→推理→矢量化→结果集→导出，中间每一步都是可观察检查点。设计技巧：先画业务流程图，基本流取最常用路径，备选流从『每个决策点的另一分支』派生。AI 辅助：把 SRS 的功能描述喂给 AI 生成场景图草稿与备选流清单，人工按真实业务裁剪。

#### 状态迁移（约10%）

方法：把对象生命周期画成状态机，用例覆盖每个合法转换+非法转换尝试。样例：任务状态机 `queued`→`running`→`done/failed/cancelled`，外加异常驱动的回收转换

（Worker 失联→重排）。BUG-124（取消后状态残留）就是合法转换覆盖缺口；D-05（死信无回收）是异常转换设计缺口。教学要点：画状态机时强制问『每个状态被异常打断后去哪』——这是新手最容易漏的一整列。AI 辅助：AI 画状态机草稿很快，但异常转换列必须人工逐格追问。

**错误推测（约10%）**

方法：凭经验与故障库直接猜测系统可能出错的地方。它是破坏性自测的主要技术（7.11 节展开）。样例来源：历史缺陷（任务 4 的 D-01~D-10）、AI 缺陷五模式、OWASP Top10、资源耗竭模式（OOM/磁盘满/连接池尽）。AI 辅助：这是 AI 最强的用例设计场景——它的『攻击想象力』覆盖明显优于人脑风暴，样例项目对抗用例 30 条中约一半的攻击角度由 AI 建议。

**判定表（约5%）**

方法：多条件组合规则用判定表穷举，防组合遗漏。样例：权限矩阵（角色 viewer/editor/admin × 操作 读/写/删/管 × 资源归属 自己/他人），3×4×2=24 组合压缩为 12 条用例。AI 辅助：AI 生成判定表骨架可靠，但规则结论列必须人工对照 SRS 权限条款逐格确认——权限错误是安全缺陷，不是功能缺陷，错一格就是事故。

**技术选择速查**

| 需求特征       | 首选技术    | 样例          |
|------------|---------|-------------|
| 参数有明确取值域   | 等价类+边界值 | 阈值 0~1、分页大小 |
| 跨功能业务流     | 场景法     | ST-009 端到端  |
| 对象有生命周期    | 状态迁移    | 任务状态机       |
| 要求高安全性/健壮性 | 错误推测    | 对抗用例        |
| 多条件组合规则    | 判定表     | 权限矩阵        |

**本节清单**

- 六种技术各能讲清方法、样例与 AI 辅助点；
- 知道 AI 在各技术中的典型偏差（编造边界、漏异常转换、权限臆断）；
- 给定需求条款能选对技术组合。

## 7.2.9 用例模板与可判定预期

一条用例的价值 80% 在预期结果字段。本节精讲《测试报告》第 3 章的用例模板，重点训练『可判定预期』的写法——这是用例评审的一票否决项，也是学生作业中最常见的失分点。

### 模板字段逐项讲

| 字段   | 要求                     | 常见错误              |
|------|------------------------|-------------------|
| 用例编号 | 级别-序号-关联需求，全局唯一        | 编号与需求脱钩，事后无法追踪    |
| 关联需求 | FR/NFR 编号，必填           | 写功能名而不写编号         |
| 优先级  | P0/P1/P2，源自 ★款与 MVP 清单 | 凭感觉定级，P0 泛滥       |
| 前置条件 | 环境/数据/账号/状态，可复现        | 『系统正常』这类空话        |
| 测试数据 | 具体数据或冻结集哈希引用           | 『准备一张影像』——多大的？哪张？ |
| 操作步骤 | 编号步骤，第三者可复现            | 跳步、依赖操作者记忆        |
| 预期结果 | 可判定：数值+容差+观察点          | 『结果正常』『响应合理』      |
| 实际结果 | 执行时填，附证据（截图/日志/构建号）    | 事后补填、与预期抄写一致      |
| 缺陷链接 | 失败时关联缺陷编号              | 失败用例无缺陷单          |

### 可判定预期的四个判据

一条预期结果是否合格，用四个问题检验：数值化吗（有具体数值或明确状态）？有容差吗（时间类指标给了上限而非『快』）？可观察吗（说明在哪看——界面元素/响应码/日志/数据库）？第三方可判定吗（换个人执行能得到同样的通过/失败结论）？四问有一问答否，打回重写。

- 反面：『分割结果准确率高』——哪个指标？多高算高？在哪看？



- 正面：『冻结影像集 5 景全部产出结果集，面积加权完整率 $\geq 80\%$ 、正确率 $\geq 85\%$ （AT-01 口径），评估报告标注「达标」』——ST-009 的真实写法。
- 正面：『传 threshold=35 返回 422，响应体 detail 字段含取值域说明「0~1 浮点」』——ST-017，连错误文案的内容都判定了。

### 预期结果的三个信息源

- SRS/spec 原文：验收标准 Given/When/Then 的 Then 直接转写——这是预期结果的第一出处；
- 设计文档：接口约定（状态码、响应结构）、性能预算分配（PE-01 的链路预算 200+300+2000+300ms）；
- 基线实测：精度类指标的参考值来自冻结集实测历史（accuracy\_history.csv），判定线来自 SRS，两者不可混淆——实测值是观测，判定线是承诺。

### 人机协同要点

- AI 批量起草：喂 spec 条款+模板，让 AI 产用例初稿，采纳率约七成；
- 人工审定的焦点就是预期字段：AI 写的预期常犯三错——含糊措辞（『正常』『合理』）、从实现倒推预期（镜像）、编造需求里没有的判定（『响应应包含 17 个字段』——SRS 没说过）；
- 评审技巧：把用例的预期字段单独抽出来评审，遮住操作步骤——如果只看预期就能知道怎么算过，这条用例合格。

### 本节清单

- 九字段模板逐项会填，关联需求与缺陷链接不留空；
- 能用四判据检验并改写一条不合格预期；
- 说清预期的三个信息源与观测/承诺之辨；
- AI 起草用例时审定焦点在预期字段。

## 7.3 用例生产的人机协同与 AI 用法

用例设计是 AI 辅助收益最大的测试活动（批量、结构化、有模板），也是风险最集中的活动（AI 会编造需求）。本节给出经过样例项目验证的协同流水线。

### 7.3.1 协同流水线（五步法）

- 第 1 步 人工划界：选定本批次的需求条款（如 FR-04 文本提示全族），明确技术组合（等价类+边界值+场景法），写明已知边界条件清单——边界清单人工写，因为它来自需求理解，这是整个流水线的质量锚点；
- 第 2 步 AI 起草：按条款+技术组合+模板生成用例草稿。提示词要点：给出需求原文、指定技术、要求每个输入取值注明依据、预期字段引用 SRS 判定线；
- 第 3 步 机器预检：脚本检查用例编号唯一性、需求编号存在性（check\_refs）、必填字段完整性；
- 第 4 步 人工审定：重点三审——预期可判定性（1.5 节四判据）、边界值出处（防编造）、场景真实性（AI 写的业务流是否符合真实操作）；
- 第 5 步 试跑校准：抽 10% 试跑，环境与数据问题在批量执行前暴露。

### 7.3.2 提示词样例（用例生成）

你是测试工程师。基于以下需求条款编写系统测试用例。【需求原文】FR-04.1：文本提示分割.....（粘贴 SRS 原文，含阈值默认值 0.30 与取值域 0~1）【设计技术】等价类+边界值，阈值参数至少覆盖 0/0.30/1.0/越界负值/越界大值/非数值六档【输出格式】用例编号（留空待分配）/标题/前置条件/步骤/预期结果/依据条款【硬性要求】预期结果必须可判定（数值或明确状态码）；每个输入取值后用括号注明依据来源；禁止引入需求条款之外的参数或行为假设。

### 7.3.3 AI 用例的三类系统性偏差（审定清单）

| 偏差     | 表现                            | 审定动作                 |
|--------|-------------------------------|----------------------|
| 编造边界   | 臆造需求里没有的限制（最大文件 2GB、最大并发 100） | 每个边界值要求出处条款，注不出即删    |
| 含糊预期   | 『响应合理』『结果正确』『性能良好』            | 按四判据打回，要求数值化         |
| 快乐路径偏好 | 10 条用例 9 条正常流，异常类不足           | 统计异常用例占比，低于 40% 要求补充 |

### 7.3.4 度量与改进

样例项目数据：AI 起草用例 118 条，审定采纳 83 条（70%）；打回 35 条中，含糊预期 19、编造边界 11、需求误读 5。这组数字值得告诉学生——AI 写用例又快又多，但三成的打回率说明审定环节不可省。同时跟踪趋势：随着提示词中『硬性要求』段按打回原因迭代（把『含糊预期』案例写进提示词），第二批采纳率升到 78%——提

示词是拿打回记录喂大的，与编码期 P-04 的演进逻辑同构。

### 7.3.5 本节清单

- 会执行五步流水线，边界清单人工写；
- 会用审定清单查三类偏差；
- 理解采纳率度量与提示词迭代的关系。

### 7.3.6 AI 在测试设计中的用法：从需求到用例的协同

1.6 节讲了用例生成的人机协同五步法，本节把整个测试设计阶段（不止用例）的 AI 用法系统化：测试计划起草、用例设计、评审质询、RTM 挂接四个环节，AI 各能做什么、做到什么程度、在哪里必须停下来交给给人。所有结论都有本项目实测依据。

#### **测试计划起草：AI 出骨架，人填判断**

测试计划（GB/T 15532 结构）的骨架性内容——章节结构、测试级别划分模板、进度表格式——AI 生成质量很高且快（10 分钟出初稿）。但计划里真正值钱的内容全是判断：范围取舍（哪些不测、为什么）、风险排序（哪个特性风险最高）、资源分配（五天窗口怎么切）。实测做法：AI 出骨架后，全组对着骨架开 1 小时计划会，逐项填入判断并记录理由。AI 在此环节的价值是『让空白页消失』——对着初稿改比从零写快一倍，但别把初稿当终稿。

#### **用例设计：三种提示词模式**

实测有效的三种提示词模式（已沉淀进《Prompt 模式库》）：

- 模式一『需求展开』：喂单条需求 + spec.md 的 Given/When/Then，要求展开为正常流 + 备选流 + 异常流用例——适合功能用例，产出率高（★条款 13 条条款中 28 条由此起草）；
- 模式二『攻击者视角』：喂接口定义，要求扮演攻击者列攻击向量——产出了边界/资源/状态/授权四类向量清单，红队自测的防守准备就靠它（可惜 BUG-131 的体积上限向量当时被人工筛掉了，教训：AI 列的攻击向量宁可多留）；
- 模式三『边界猎手』：喂字段定义与约束，专生成边界值/等价类用例——机械性最强、AI 产出质量最稳的环节，IT-011 这类边界用例如果当时用了模式三，首轮就能拦住。

反模式：整份 SRS 扔给 AI 说『生成全部测试用例』——产出量大但覆盖随机、深度参差、与优先级脱节。用例生成必须以需求条目为单位逐条驱动，RTM 才能挂得住。

## **评审质询：AI 当第一评审**

用例评审会之前，先让 AI 当第一评审过一遍，实测的三段质询提示词：『假设你是甲方验收专家，指出这份测试设计的覆盖盲区』『假设你是开发，指出哪些用例的预期结果不可判定』『对照 SRS 第 X 章，逐条核实用例与需求的一致性』。实测效果：AI 质询平均每条用例集提出 6~9 个问题，其中约一半有效（另一半是它没读懂上下文的误报，人工划掉即可）。人工评审会的议程直接由 AI 质询清单导入——评审会从『大家随便看看』变成『逐条裁决问题清单』，效率翻倍。

## **RTM 挂接：AI 维护，人审计**

RTM v5 的挂接工作（用例编号 ↔ 需求编号的双向登记）量大而机械，AI 从用例文档与 SRS 的文本匹配自动建初版挂接，覆盖率 100% 的初稿 15 分钟出来。但自动挂接的错误率约 5%~8%（把『导出 GeoJSON 格式校验』挂到『导出』需求却漏挂『坐标系输出 4326』需求），必须人工审计——审计方法：只查『一对零』和『一对多』两种异常（没有用例的需求、挂了 5 条以上用例的需求），这两种异常占挂接错误的九成。

## **AI 用例的入库质检标准**

AI 起草的用例进正式用例库前必须过入库质检（这是测试关的铁规，比任务 4 的代码门禁更严）：

- 预期结果可判定（『页面正常』『响应合理』一律打回）；
- 前置条件完整（数据、环境、账号状态）；
- 步骤可独立执行（不依赖上一条用例的副作用）；
- RTM 挂接至少一条需求；
- AI 来源打标（用例库里标 AI-draft 直到人工评审通过摘标）——事故溯源用。

## **本节清单**

- 测试计划 AI 出骨架、人填判断，分工记录在案；
- 三种提示词模式（需求展开/攻击者视角/边界猎手）进模式库；

- AI 第一评审 + 人工裁决的评审流程运转；
- RTM AI 建初稿 + 人审『一对零/一对多』；
- 入库质检五条铁规执行，AI 用例打标溯源。

### 7.3.7 AI 在测试执行中的用法：脚本、数据与排查

测试执行阶段的 AI 用法与测试设计阶段不同：设计阶段 AI 帮你『想』，执行阶段 AI 帮你『做』——写自动化脚本、造测试数据、分析失败日志。本节按执行期的五个场景讲用法与边界。

#### **自动化脚本生成：最高产出的场景**

AI 生成测试自动化脚本是全流程投入产出比最高的用法，本项目的数据：50 条设计用例展开为 122 个自动化测试函数，自动化脚本约七成由 AI 起草。成功的前提是给人给 AI 的输入足够结构化：接口测试直接喂 OpenAPI 规范 + 用例步骤，AI 产出的 `pytest+requests` 脚本基本可跑；UI 测试喂用例步骤 + 页面元素清单，产出 `Playwright` 脚本需人工修一轮选择器。效率对比实测：人工写接口测试脚本平均 25 分钟/条，AI 起草+人工修 8 分钟/条。

纪律：AI 生成的脚本必须先评审再入库，重点审三处——断言是不是真断言（AI 爱写 `assert response` 这种恒真断言）、清理逻辑（测试数据有没有 `teardown`，不然测试库越跑越脏）、环境耦合（有没有把本机路径、本机端口写死）。

#### **测试数据构造：批量与刁钻两相宜**

AI 在测试数据构造上有双重价值。批量面：造正常数据（生成 200 个图斑属性记录、50 个任务历史——压测和分页测试的底料），格式给准了 AI 又快又稳。刁钻面：造异常数据——畸形 GeoJSON（坐标嵌套错、环不闭合）、超大字段、Unicode 边界字符、SQL 注入载荷串。RL-03 毒消息的 20 个变体就是 AI 生成。人工审查要点：AI 造的『畸形』数据有时畸形得不对（它想造坐标越界，结果造了个语法错误——测试的是解析器不是业务校验），每类异常数据抽一个样本人工验证它确实打到目标校验层。

#### **失败排查：AI 初筛，人定根因**

执行期最耗时的环节是用例失败后的排查（是真缺陷还是测试脚本错/环境抖动）。

本项目的分流机制：失败日志 + 用例步骤 + 最近 commit 记录喂 AI，要求给出三个候选根因并按可能性排序。实测：AI 初筛的首候选命中率约 60%（环境类与脚本类问题命中率高，业务逻辑类低），把人从『通读日志』解放为『验证候选』。但定根因的权利永远在人——AI 有个顽固倾向：把缺陷归因于环境（『可能是网络抖动』），因为环境归因在训练语料里最安全。业务缺陷一旦被误判为环境抖动而重跑通过，就漏网了。

## **执行报告与日报：AI 整理，人签发**

每日测试执行的数据汇总（执行数/通过数/新增缺陷/阻塞项）由 AI 从执行系统导出整理成日报，测试负责人核对后签发。省下的时间用在 AI 做不了的事上：阻塞项的升级决策、跨组协调、缺陷定级争议。

## **AI 不能碰的执行红线**

执行阶段四条红线（写进团队规范）：

- 冻结影像集的精度测试全流程人工执行人工判读，AI 不碰——防止无意识的口径漂移（AI 算指标时顺手『优化』公式的风险真实存在）；
- 用例的最终判定（pass/fail）由人确认，AI 可以建议——尤其是『实际结果与预期不符但好像也有道理』的灰色情况；
- 缺陷单提交前人过目——AI 草稿可能含臆造的复现步骤（它没真的跑过）；
- 压测数据的解读口径（P95 怎么算、样本怎么取）人定，AI 执行计算。

## **本节清单**

- 自动化脚本 AI 起草率与返修情况有统计；
- 脚本入库评审三处（真断言/清理/环境耦合）执行；
- 异常数据抽样人工验证打到目标校验层；
- 失败排查 AI 初筛、人定根因，警惕环境归因倾向；
- 四条执行红线成文并执行。

## **7.4 测试执行管理与 RTM v5 闭环**

RTM 闭环是三条过关硬杠中最易被低估的一条：『每条需求至少 1 条通过的测试用例』听起来简单，做到需要机制。本节讲 v4→v5 的回填与核查方法。

### 7.4.1 v5 相对 v4 的新增

任务 4 的 RTM v4 已挂到代码与用例编号；v5 新增两列：执行记录（最近一次通过的构建号+日期）、验收状态（通过/阻塞/未测）。关键区别：v4 记录『有没有测试』，v5 记录『测试过了没有、在哪次构建过的』。

### 7.4.2 闭环核查四步

- 第 1 步 正向全扫：24 FR+16 NFR 逐行检查——有用例吗？用例最近一次执行过了吗？构建号是多少？三问任一断即标红；
- 第 2 步 反向核查：所有执行用例反查需求编号，揪出孤儿用例（无出处的用例往往是过度设计或需求遗漏的信号）；
- 第 3 步 重点深挖：MVP 5 项与★13 款逐条点开看执行记录原文（日志/截图/报告页码），不只信状态字段；
- 第 4 步 签字归档：核查输出（脚本报告+人工抽查记录）归档，测试负责人与组长双签。

### 7.4.3 自动化支持

```
# RTM 闭环核查脚本（在 check_refs 基础上扩展） python scripts/rtm_close.py --rtm RTM-v5.xlsx --junit reports/latest.xml # 输出： # [rtm] FR 24 条：53 条闭环，2 条标红（FR-07.3 用例未执行 / FR-09.4 最近构建失败） # [rtm] NFR 16 条：全部闭环（证据链接齐全） # [rtm] 反向核查：无孤儿用例 # [rtm] MVP 5 项：全部闭环且为用例最新构建
```

### 7.4.4 常见断链模式与处置

| 断链模式     | 成因           | 处置              |
|----------|--------------|-----------------|
| 有用例未执行   | 用例写完没排进执行计划  | 补执行或明确降级（记录理由）  |
| 执行了但状态旧  | 修复后没重跑该用例    | 按 RTM 反查受影响用例回归 |
| 需求变了用例没变 | 变更四步没走全      | 回到变更流程补闭环       |
| 孤儿用例     | AI 生成用例时编造需求 | 删用例或提需求变更，二选一   |

### 7.4.5 人机协同要点

- AI 做断链诊断报告：RTM 导出+测试结果喂给 AI，输出断链清单与修复建议，人工裁决；
- AI 起草验收证据摘要：为每条 MVP 需求生成『需求→用例→构建→结果』的证据链段落，人工核对链接可点、构建号真实；
- 底线：RTM v5 的最终状态由人签字，AI 只做检查与文书。

#### 7.4.6 本节清单

- 执行四步闭环核查，输出归档双签；
- 识别四种断链模式并正确处置；
- 理解 v4『有没有』与 v5『过没过』的区别。

#### 7.4.7 测试执行管理：准入、记录与回归的纪律

测试执行是把设计变成证据的过程。执行阶段的质量取决于三条纪律：准入准出、记录规范、回归策略。本节逐项讲操作。

##### 准入检查（开测前）

- 代码基线：tag sprint2-final，CI 五道闸全绿，selfcheck 11/11——基线不绿不开测，在晃动的基础上测试等于浪费；
- 环境就绪：测试服务器独占（压测期间禁止其他人使用）、冻结影像集哈希校验通过、监控采集开启；
- 用例就绪：用例集评审通过、编号与 RTM 映射完成、执行排期表（哪天执行哪批）已发。

##### 执行记录规范

每条用例执行必须留四样：实际结果（与预期逐字比对，不是『同上』）、构建号、执行人与时间、失败时的证据（截图/日志片段）。纪律依据：《测试报告》的所有结论都要能被评委复核——『构建 CI#899 的 ST-031 执行记录』必须可检索。禁止补录：执行记录的时间戳由系统生成，事后集中补录的记录在时间链上必然露馅（这正是教师鉴别真实性的三板斧之一）。

##### 失败处置流程



- 第 1 步 判定是真缺陷还是用例/环境问题：环境问题（网络抖动、服务未起）修复环境后重跑，不登记缺陷；用例错误（预期写错）改用例并记录变更；产品缺陷才进缺陷看板；
- 第 2 步 缺陷登记五要素：最小复现步骤、环境、期望 vs 实际、级别初判、关联用例；
- 第 3 步 阻塞处理：P0 用例失败阻塞后续关联用例（在执行表标 blocked 而非 skip），修复后连带回归。

## 回归策略

修复后的回归不是『重跑那条失败用例』就完事：按 RTM 反查缺陷位置关联的全部用例组成回归子集（BUG-121 修复动了金字塔构建逻辑，回归子集=ST-001+ST-004+IT 上传族+AT-01），全绿才算关闭。《测试报告》首轮 61/64→回归 64/64 且无新增缺陷，靠的就是这个机制。每轮回归记录新旧构建号对比。

## 执行期人机协同

- AI 做执行日报：当日执行记录喂给 AI 生成进度与风险摘要，人工审；
- AI 辅助失败分类：把失败日志喂 AI 初判『环境/用例/产品』，人来终判——AI 对日志模式识别快，但倾向把产品缺陷轻描淡写为环境问题，终判权必须人在；
- AI 生成回归子集建议：按缺陷 diff 与 RTM 关联推荐回归范围，测试负责人确认。

## 本节清单

- 准入三项检查不绿不开测；
- 执行记录四要素+系统时间戳；
- 失败三支处置（环境/用例/产品）不混淆；
- 回归子集按 RTM 反查，不是单点重跑。

## 7.5 单元、集成与系统三级测试执行

单元测试在任务 4 已随 CI 每日运行，任务 5 要做的是『收官』：确认覆盖率数据真实、修复引入的代码有测试、缺陷修复全部带回归用例。本节讲收官三件事。

### 7.5.1 覆盖率收官核对

《测试报告》数据：总体 78%、核心模块（queue/worker/geo/security）93%、任务 5 修复代码增量 91%。核对三问：

- 数字从哪来：CI 构建 #904 的 coverage.xml，附在报告证据链里——不接受口头报数；
- 涨了为什么：任务 4 末是 76%/92~94%，任务 5 修复 18 条缺陷时按纪律每个修复带回归用例（新增 19 个测试函数），总体涨到 78.4%——覆盖率上升要有出处，无出处的上涨要警惕注水；
- 注水复查：按任务 4 的四种注水手法（空断言/镜像/快乐路径/import 式）抽查新增用例，重点查缺陷修复附带的回归用例——这些用例的天然防注水属性是『修复前应红、修复后转绿』，评审时要求演示这个先红后绿证据。

### 7.5.2 回归用例的质量判据

每条缺陷修复附带的回归用例必须满足：没有该修复它会失败（先红后绿的 git 证据）；断言复现的是缺陷的原始症状而非修复后的实现细节（否则下次重构误报）；登记了关联缺陷编号。BUG-125（崩溃循环）的回归用例就是范例：构造毒消息注入队列，断言 Worker 重试 3 次后进死信且不崩溃——它测的是行为契约，不是实现。

### 7.5.3 覆盖率豁免的正规操作

任务 5 允许豁免但必须显式：防御性分支（理论上不可达的错误处理）可申请豁免，在 CI 配置里用 pragma 注释逐行标注并附理由；禁止为凑数字调低门槛或删除难测代码。豁免清单进测试报告附录，评审可逐项质询。样例项目豁免 3 处（均为第三方库异常转换分支），评审质询时给出了『为什么不可达』的推理，通过。

### 7.5.4 人机协同要点

- AI 生成回归用例草稿：把缺陷描述+修复 diff 喂 AI，产出先红后绿的用例草稿，人工核对断言针对症状；
- AI 做注水扫描：让 AI 按四种注水模式审查测试文件，输出疑点清单人工复核；
- 豁免理由由人写：『为什么不可达』的论证是判断题，AI 只整理格式。

### 7.5.5 本节清单

- 覆盖率收官三问（出处/归因/注水复查）；
- 回归用例三判据（先红后绿/测症状/挂缺陷号）；
- 豁免显式化，禁令两条（调门槛/删代码）。

### 7.5.6 集成测试与契约终验

集成测试 100% 通过是过关条件二。本节讲 IT 用例的构成、首轮 85/86 的失败分析（IT-011），以及契约终验的操作。

#### IT 用例的三层构成

- 端点契约层（6 端点 × 正常/边界/异常）：每个端点至少 3 条——这是 `contract_check` 的自动化补充：脚本查结构一致，用例查行为正确；
- 跨模块链路层：接入层→业务层→能力层→适配层，每条链路测正常流 + 一个断点流（如元数据核验失败时链路应中断而非继续）；
- 变更回归层：每次 CR 后受影响的端点用例进回归集——IT-011 就是这一层的失败案例。

#### 案例精讲：IT-011 为什么红了

时间线还原：CR-001 把文本阈值默认值 0.30→0.24，四步流程走完、PR#52 合并。但 IT-011（阈值边界校验用例）断言里仍引用旧默认值构造的期望值——变更单的影响面清单列了『测试基线 2 个用例（TC-F04-01/04-02）』，而 IT-011 不在其中：它是任务 4 早期按旧值写的集成用例，编号体系不同导致清单漏了它。每日构建红灯，登记 BUG-118。

教学要点有三：一是变更影响面清单要按『全局搜索旧值』兜底，不能只靠 RTM 关联（关联体系有编号断点）；二是这个失败恰恰证明集成测试在工作——它抓到了单元测试抓不到的跨层不一致；三是《测试报告》如实记录了这个首轮失败再回归的过程，诚实记录反而成了评审加分项。

#### 契约终验操作

- `contract_check.py` 最终全绿（26/6 端点路径、方法、x-roles、x-fr 标注）；

- 人工抽查 5 个端点的行为级契约：状态码集合、错误响应结构、分页参数语义——脚本查结构，人查语义；
- OpenAPI 文档与在线文档（/docs 页）一致性截图归档。

## 人机协同要点

- AI 生成端点异常用例矩阵：对每个端点穷举异常类（参数缺失/类型错/越界/越权/并发冲突），人工按风险裁剪；
- AI 辅助失败分析：IT 红灯时把契约 diff 与用例断言喂 AI，初判『代码错了还是用例旧了』——IT-011 这类『用例旧了』的模式 AI 识别很快；
- 纪律：契约修改永远文档先行（先改 OpenAPI 再改代码），这个顺序在任务 4 已立，任务 5 只做核查。

## 本节清单

- 讲清 IT 三层构成与各自拦截的问题类型；
- 复述 IT-011 案例的三层教训；
- 契约终验：脚本全绿+人工语义抽查 5 端点。

### 7.5.7 系统测试执行：用例集的组织艺术

系统测试（ST）是最贴近验收的级别，也是最昂贵的级别——64 条端到端用例，每条都要真实环境、真实数据、真实操作。本节讲执行组织与首轮失败的处置策略。

## 执行编排

- 按风险排序而非按编号：P0（MVP+★款，34 条）第一天全部执行——最早暴露验收级风险；P1 第二天；P2 穿插；
- 环境锁定：ST 期间测试服务器冻结变更（除缺陷修复 PR 外禁止合入），每日开工先跑 selfcheck 11/11 确认基线健康；
- 双人制：执行人+记录人，关键 P0 用例（如 ST-038 降级、ST-009 精度）要求第二人见证签字——验收级证据需要见证；
- 数据纪律：冻结影像集每日执行前哈希校验；测试产生的中间数据每日归档（供缺陷复现与评审复核）。

## 首轮 61/64 的处置复盘

首轮 64 例过 61，失败 3 例（BUG-121/122/123）。处置的教科书动作：

- 不停测：失败用例登记后继续执行其余用例——首轮的目标是摸清全貌，不是遇到红就停；
- 分级响应：BUG-121（大文件断点续传后金字塔未重建，严重）当天修复 PR#73，因为 FR-01.1★ 是 MVP；BUG-122（SHP 字段截断未告警，一般）次日修复；BUG-123（深色模式对比度，轻微）排进质量冲刺日；
- 回归验证：9/1 全量回归 64/64，且 RTM 反查确认修复未波及其他用例；
- 非阻塞发现也登记：首轮另发现 5 项非阻塞缺陷（用例判定通过但观察到异常行为，BUG-124/129/141/142/143）——『用例过了但看着不对』是宝贵的测试直觉，登记不登记是团队质量文化的试金石。

## 系统测试的环境真实性

ST 必须在与交付环境同构的环境执行：Docker Compose 全组件、真实影像数据、真实网络拓扑（哪怕是局域网）。『在我笔记本上过了』不构成 ST 证据。样例项目的一次教训：某用例在开发机过、测试服务器挂，根因是开发机预装了 GDAL 系统依赖而容器内版本不同——自此环境一致性检查（selfcheck 含依赖版本断言）加入每日开工流程。

## 人机协同要点

- AI 生成执行排期表：按优先级、环境互斥（压测时段独占 GPU）、人员分工自动排程，人工调整；
- AI 做失败快报：每个失败用例自动汇总『用例+日志尾+怀疑方向』，加速当日站会决策；
- AI 写不了的部分：端到端操作的真实执行与观察判断——ST 是本关人工含量最高的环节，这本身就是教学点。

## 本节清单

- 执行编排四原则（风险排序/环境锁定/双人制/数据纪律）；
- 首轮失败处置：不停测、分级响应、RTM 回归、非阻塞也登记；

- 环境真实性与开发机陷阱。

## 7.6 功能适合性专项：MVP 与★款逐条过堂

功能适合性是八大特性之首，回答『承诺的功能都对吗』。它的专项证据=ST 用例结果+RTM 闭环+需求逐条核对。本节讲专项的组织方法。

### 7.6.1 三个子特性的核查设计

- 功能完备性：SRS 承诺的 24 条 FR 是否都实现了——用 RTM 正向核查（每条 FR 有代码有用例）回答，防止『漏做』；
- 功能正确性：实现的是否正确——用 ST/IT/UT 执行结果回答，重点是 MVP 5 项与★13 款全部 P0 用例通过；
- 功能恰当性：功能是否契合用户目标——用任务 2 的用户故事回溯核对（每条 MVP 功能对应哪个用户场景），防止『做对了但没用』。

### 7.6.2 MVP 5 项逐条过堂表（模板）

| 需求            | 验收要点                                  | 用例           | 结果 | 证据                  |
|---------------|---------------------------------------|--------------|----|---------------------|
| FR-01.1★      | 10GB 级分片上传+断点续传+金字塔首屏≤3s              | ST-001/004   | 通过 | CI#899 记录           |
| FR-02.1★      | 全自动分割 5 景精度达标+1km <sup>2</sup> ≤10min | ST-009/AT-01 | 通过 | 精度报告 + 《测试报告》第 16 章 |
| FR-03.1/03.2★ | 点/框提示单轮 P95≤3s                        | ST-014/PT-01 | 通过 | 《测试报告》第 16 章 S1     |
| FR-04.1★      | 文本两级链路+阈值默认 0.30                      | ST-017/023   | 通过 | 执行记录                |
| FR-06.1★      | 矢量导出拓扑检查零重叠零缝隙                        | ST-045       | 通过 | 拓扑报告                |

过堂会的形式：评审现场按这张表逐条点——组长现场演示或出示证据，评委随机抽行深挖。《测试报告》的结论可信度建立在每一行都可追溯上。

### 7.6.3 ★款专项：招标文件承诺的兑现

★款（13 条）来自招标文件的关键性条款，违约即废标级别的承诺。专项做法：把 13 条★款单独列表，每条标注兑现证据（哪个用例、哪份报告、哪页）。答辩时这张表是商务可信度的核心——它证明『投标吹的牛全部兑现了』。教学提醒：★款清单在任务 1 需求确认书就已锁定，任务 5 只是逐条兑现核对——承诺管理贯穿全生命周期。

### 7.6.4 人机协同要点

- AI 生成过堂表初稿：从 RTM v5 直接抽取，人工核对证据链接可点；
- AI 做★款兑现差距分析：把招标条款原文与现有证据喂 AI，问『哪条证据链最薄弱』，指导补测优先级；
- 底线：过堂结论由组长逐条确认签字，AI 不参与判定。

### 7.6.5 本节清单

- 三子特性分别用 RTM/执行结果/用户故事回答；
- 会组织 MVP 过堂表并现场举证；
- ★款单独成表，证据链可点。

## 7.7 性能专项：压测场景设计与数据解读

性能专项是任务 5 技术含量最高的部分之一。本节以《测试报告》第 16 章为教材，讲压测设计的核心判断：不同性质的性能指标要用完全不同的压测方法。样例项目审核中发现的最严重错误——对全自动分割做秒级压测——就是没建立这个判断。

### 7.7.1 第一判断：同步指标还是异步指标

拿到性能条款先分类：交互单轮 $\leq 3s$ （PE-01）是同步指标——用户在线等待，用响应时间分位数度量；全自动  $1km^2 \leq 10min$ （PE-02）、文本批量 $\leq 5min$ （PE-03）、10 万图斑导出 $\leq 2min$ （PT-08）是异步指标——任务进队列异步执行，用吞吐与完成时间度量。对异步任务谈 P95 秒级响应是范畴错误：它的『响应』是  $202+任务ID$ （毫秒级），真正要压的是任务完成时间与队列行为。

### 7.7.2 四场景设计逻辑（S1~S4）

| 场景      | 目标指标             | 负载模型               | 要回答的问题            |
|---------|------------------|--------------------|-------------------|
| S1 交互分割 | PE-01 P95≤3s     | 并发阶梯 1/2/4/8       | 承诺口径内达标吗？边界在哪？    |
| S2 在线浏览 | PE-04/06≤2s@30在线 | 并发阶梯 5/10/20/30/50 | 30 在线承诺成立吗？还能再高吗？ |
| S3 异步吞吐 | PE-02/03、PT-08   | 任务流交错提交            | 分钟级指标实测多少？队列稳吗？   |
| S4 降级模式 | PE-09 单轮≤30s     | 并发 1，CPU 推理        | 降级是可用的底线吗？        |

### 7.7.3 并发阶梯怎么定

阶梯不是拍脑袋：下限从设计承诺口径起（PE-06 说 1 推理+30 在线，S1 就从 1 起步），上限推到系统明确失稳为止——压测的目的之一是找到边界，只在舒适区压测等于没压。S1 推到并发 8（明确超预算、限流触发），S2 推到 50（仍达标，说明浏览链路余量充足）。每档持续 10 分钟+预热 3 分钟不计入，保证稳态统计。

### 7.7.4 负载真实性的三个细节

- 数据真实：压测用真实尺寸的影像与真实分布的提示点，不用『1 像素图片』式假数据——假数据测出的性能没有外推价值；
- 思考时间：模拟真实用户的操作间隔（交互分割用户两次提示间隔 2~5s），无思考时间的压测是把系统往死里打，结论不能外推到真实负载；
- 混合与隔离：S3 刻意混合三类异步任务验证队列调度公平性；但 S1/S2 执行期间禁止其他负载——指标测量与行为观察要分开做。

### 7.7.5 人机协同要点

- AI 起草 k6 脚本：给指标定义与负载模型，AI 生成脚本初稿，人工核对统计口径（分位数计算、预热排除）——AI 常把 mean 当 P95，必须查；



- AI 做场景查漏：喂 SRS 性能条款清单，问『还有哪些条款没被压测场景覆盖』——PE-07 首屏 $\leq 3s$ 就是这样补进 S2 的；
- 负载参数的出处人工定：并发阶梯、思考时间、数据尺寸都是策略决策，AI 给建议、人拍板并记录理由。

### 7.7.6 本节清单

- 掌握同步/异步指标的分类压测法；
- 会设计阶梯负载并讲出上下限依据；
- 负载真实性三细节：真实数据、思考时间、混合与隔离。

### 7.7.7 性能专项（下）：数据解读与容量建议

压测跑出数据只是开始，性能工程师的价值在解读：为什么这条曲线长这样、边界意味着什么、容量建议怎么给。本节以《测试报告》第 16 章数据为教材。

#### **读曲线：S1 交互分割的形态分析**

S1 数据形态（并发 1 $\rightarrow$ 2 $\rightarrow$ 4 $\rightarrow$ 8：P95 2.3 $\rightarrow$ 2.8 $\rightarrow$ 4.1 $\rightarrow$ 7.9s）是典型的排队主导曲线：并发 1~2 段平缓（Worker 直接服务，响应=真实推理时间），并发 4 起陡峭上升（gpu\_queue 独占串行，排队时间=前面任务的推理时间累加）。读图要点：拐点位置（并发 2~4 之间）就是系统的有效吞吐边界；拐点前曲线斜率告诉你单请求成本，拐点后斜率告诉你队列行为。排队论直觉：利用率逼近 100% 时响应时间发散——并发 2 档 GPU 利用率已 94%，所以并发 4 的恶化完全可预期。

#### **P99 口径的意义**

任务 4 只用 P95，任务 5 新增 P99 观测口径（ $\leq 4.5s$ ）。为什么加：P95 意味着 5% 的用户体验超线，压测样本 3100 次里约 155 次超标——P99 把观察窗口推得更深，捕捉尾部风险（GC 停顿、缓存失效、磁盘毛刺）。样例数据并发 2 档 P99=3.4s 与 P95=2.8s 差距不大，说明尾部健康；如果 P99 远大于 P95（如 P95=2.8 而 P99=12），说明存在偶发长尾巴，要查根因而不是平均掉。答辩可讲：分位数的选择就是服务质量承诺的严格程度选择。

#### **容量建议的写法**

容量建议必须可执行，三要素：边界值（单 GPU 节点交互并发 $\leq 2$ 、浏览并发

≤50)、扩容单元(每 2 路交互并发加一个 GPU Worker 副本)、约束条件(显存 9.8/12GB, 副本数与单景尺寸不可同时放大)。反模式:『建议增加服务器配置』——加什么、加多少、为什么, 一个都不能少。所有容量数字必须能指回压测数据行。

## 降级数据的呈现

S4 (CPU 降级 P95=14.8s) 的呈现学问: 不要羞于展示降级态比正常态慢 8 倍——这恰是 PE-09 的设计意图(可用性底线, 不是等效体验)。报告里明确写出这个语义, 反而在红队自测的争议裁决中救了场(红队主张 14.8s 是性能缺陷, 我方凭 SRS 条款措辞赢得仲裁)。教学点: 性能条款的措辞精度(『降级态单轮≤30s』而非『降级后性能不降』)在设计时保护了自己。

## 人机协同要点

- AI 做数据初读: 把分档数据喂 AI 让它描述形态与拐点, 人工验证解释合理性——AI 能说出『排队主导』, 但『这与 gpu\_queue 独占串行的设计决策一致』的因果关联要人补;
- AI 起草容量建议初稿, 人工核对每个数字的出处行;
- 异常毛刺分析: P99 离群样本的日志关联分析交给 AI 初筛, 人定根因。

## 本节清单

- 读得出排队主导曲线与拐点含义;
- 讲清 P99 口径的尾部风险价值;
- 容量建议三要素(边界/扩容单元/约束)且数字有出处;
- 降级数据按设计语义呈现, 不藏不饰。

## 7.8 精度专项: 算法类需求的验收测试

遥感解译平台的特殊性在于: 它的核心交付物不是『功能能跑』, 而是『结果够准』。精度专项(AT-01/02/03)是本项目的验收测试主体, 也是全流程纪律要求最高的一环。本节讲清三件事: 冻结影像集为什么必须冻结、三项精度指标怎么测、CR-001 之后怎么复测。

### 7.8.1 冻结影像集: 纪律先于技术

精度测试的第一个原则: 测试集必须在任何模型调优之前冻结。本项目的冻结影

像集是 5 景（覆盖城市建成区、农田、水体、山地林地、城乡交错带五种典型地表），每景带人工精标的真值图斑。为什么只有 5 景：教学周期内 5 景足以覆盖主要地物类型且每景真值判读工作量可控；工程上数量服从统计功效要求，但纪律要求不因数量多少而改变——冻结纪律才是重点。

冻结纪律的三条铁规：

- 冻结之后，任何队员不得用这 5 景做任何形式的调参、选模型、选提示词——连『看一眼结果再改阈值』都算污染；
- 调优只能用独立的开发影像（本项目另有 12 景非冻结开发集），开发集上得出的参数才能进代码库；
- 冻结集的使用权只在测试负责人手里，每次使用必须登记（谁、何时、跑了哪个版本）。

反面教材（其他届真实发生）：某组拿测试集跑了十几次挑了最好的一次报数，答辩时被要求现场用另一组影像复测，精度掉 12 个点，直接按数据造假处理。冻结集就是考试的封卷，启封次数应该一只数得过来。

### 7.8.2 AT-01：全自动分割精度

指标定义（SRS ★款）：面积加权完整率  $\geq 80\%$ ，面积加权正确率  $\geq 85\%$ 。面积加权的意思：大图斑的漏检/错检惩罚大，小图斑惩罚小——符合遥感业务直觉（漏掉一个工业区比漏掉一间平房严重得多）。

测量过程（样例实测）：对 5 景冻结影像逐景跑全自动分割流水线，输出图斑与真值图斑做空间叠置分析，按面积加权公式计算两率。首轮实测：完整率 76%、正确率 89%——完整率未达标（差 4 个百分点），正确率达标。这就是 CR-001 的触发点。

CR-001 复测的完整链条：需求变更（文本提示分割阈值默认 0.30→0.24）→ 走变更流程（改 spec → 同步 SRS → 改 plan/tasks → PR#52 改代码，CI#891）→ 用冻结集复测 → 完整率 84.2%、正确率 88.6%，双达标，任务 T-36 关闭。教学要点：阈值降 0.06 的代价是正确率从 91.5% 掉到 88.6%（误检变多），这是用 2.9 个点正确率换 7.4 个点完整率的权衡——SRS 的门槛设计（正确率 85% 留了余量）预先容纳了这个权衡空间。答辩可讲：好的验收标准不仅要可测，还要为工程权衡留出可操作的弹性。

### 7.8.3 AT-02：交互分割精度

指标定义：点击引导交互分割的 IoU（交并比） $\geq 0.75$ 。IoU 是分割任务的标准指标：预测掩膜与真值掩膜的交集面积除以并集面积，1.0 为完美重合，0.75 是行业可用的底线（再低就要人工大量修正，失去辅助价值）。

测量方法：5 景冻结影像上每景选取若干典型目标（建筑物、地块、水体边界），由测试员按标准点击规程（目标中心单点）发起交互分割，计算每次分割的 IoU 并取平均。实测 0.78，达标。注意测试规程的标准化：点击位置、缩放级别必须统一，否则不同测试员测出的 IoU 没有可比性——这本身就是要写进《测试设计》的规程细节。

### 7.8.4 AT-03：面积量算精度

指标定义：图斑面积量算误差  $\leq 3\%$ （与真值面积相比）。实测 2.1%，达标且余量充足。技术要点：面积量算必须在投影坐标系下做（本项目存储坐标系 CGCS2000/EPSG:4490），在经纬度坐标（EPSG:4326）下直接算面积会引入随纬度变化的系统误差——这是遥感软件的常识陷阱，答辩必问点之一。输出层用 4326 是为了 Web 前端兼容，量算层用 4490 是为了精度正确，两者的转换职责在设计文档里写清了，测试时按设计验证。

### 7.8.5 人机协同要点

- 叠置分析与指标计算的脚本可以由 AI 起草，但公式必须由人对着指标定义逐符号核对——面积加权公式的分母用错一次，全部数据报废；
- 冻结纪律是纯粹的人工管理事项，AI 无从代劳，测试负责人要在看板上维护冻结集使用登记；
- 让 AI 对精度报告做『抬杠审查』：『如果我是甲方专家，我会质疑这份报告的哪三点？』——样例中 AI 提出的『5 景样本量是否支撑 80% 置信结论』正是答辩真问题，预先准备了样本量论证。

### 7.8.6 本节清单

- 冻结影像集 5 景、冻结三铁规、使用登记在案；
- AT-01 两率口径正确（面积加权），CR-001 复测链完整（变更单→PR→CI→复测数据）；
- AT-02 测试规程标准化（点击位置/缩放级别统一）；

- AT-03 在 EPSG:4490 下量算，能解释为什么不用 4326；
- 所有精度数据可溯源到具体影像、具体版本、具体测试运行记录。

## 7.9 可靠性与安全性专项

可靠性回答的问题是『出事了会怎样』：进程死了、网络断了、磁盘满了，系统是崩溃、丢数据、还是按设计降级恢复。本项目的可靠性专项用 6 个故障剧本做故障注入测试，与 3.3 节的故障演练（计时验收）相互支撑：专项验证『行为对不对』，演练验证『人来修快不快』。本节讲剧本设计、注入方法与判定标准。

### 7.9.1 六个故障剧本的设计依据

剧本不是随机整蛊，每个剧本对应一条 SRS 可靠性条款或一个真实架构风险点：

| 编号    | 故障注入                  | 对应条款/风险点    | 预期行为（判定标准）                               |
|-------|-----------------------|-------------|------------------------------------------|
| RL-01 | kill 推理 GPU Worker 进程 | NFR-R1 服务自愈 | Supervisor 自动重启，队列任务不丢，30s 内恢复服务，前端提示可理解 |
| RL-02 | 卸载模型可选依赖              | NFR-R2 依赖降级 | 上传/导出明确报错并提示重试，已传数据不损坏，浏览类功能不受影响         |
| RL-03 | 灌毒消息进任务队列（伪造畸形任务参数）   | 队列消费者健壮性    | 毒消息被校验拦截进死信队列，不拖垮 Worker，后续正常任务照常处理      |
| RL-04 | 磁盘写满 95%              | NFR-R3 资源保护 | 上传被拒绝并明确提示，日志轮转不崩溃，清理后自动恢复               |
| RL-05 | 数据库连接池耗尽（并发 50 长查询）   | NFR-R1 过载保护 | 新请求排队或快速失败，无连接泄漏，压力撤走后 60s 内自愈           |
| RL-06 | 推理中拔掉 GPU（降级切换）       | PE-09 降级条款  | 自动切换 CPU 推理，降级提示可见，单轮≤30s                |

设计要点：每个剧本的『预期行为』必须事先写死（来自 SRS/设计文档），测试只是验证现实与纸面一致——不许边看现象边改标准，那会把测试变成追认。

## 7.9.2 注入方法与数据校验

故障注入的技术手段（教学环境可用）：kill -9 进程、iptables 断端口、tc 注入网络延迟、dd 灌满磁盘、脚本直连消息队列塞畸形消息。每条注入都要设计『恢复步骤』与『数据校验步骤』：

- 恢复步骤：故障撤销后系统能否自愈到正常态（RL-01 实测恢复 6 分 12 秒——含人工定位时间，见 3.3 节演练）；
- 数据校验：故障前后数据一致性比对——任务状态机有没有卡死在中间态（RL-01 实测 1 个进行中的任务卡在 running，需人工重置为 failed 后可重新发起，记为改进项 T-41 方向）；
- 日志校验：故障时刻的日志是否足以定位（演练项实测证明足够，见 24 章）。

## 7.9.3 剧本执行记录

六剧本首轮实测：RL-01/02/04/06 行为符合预期；RL-03 发现毒消息消费后抛异常但未进死信队列、Worker 反复重试 5 次才放弃（BUG-125，严重级——在高峰期会占用推理资源），修复后复测通过；RL-05 发现连接池耗尽后部分请求挂起超过 120s 无超时（BUG-12x，一般级），加超时熔断后通过。两个缺陷的发现直接证明：可靠性行为不注入故障永远测不出来——单元测试和系统测试里 Worker 从来不会死。

## 7.9.4 与故障演练的分工

可靠性专项（本节）与故障演练（3.3 节）常被混淆，区分清楚：专项测系统行为（机器答），演练测团队响应（人答）；专项可以慢慢做、反复做，演练限时 15 分钟、一次定胜负；专项的剧本是演练的题库来源（演练三故障全部出自六剧本），专项把行为修对，演练才有机会在 15 分钟内走通恢复流程。

## 7.9.5 人机协同要点

- 让 AI 对着架构图头脑风暴故障模式（『这个部署架构里，单点故障有哪些』），人按 SRS 条款筛选成剧本；
- 畸形消息、异常输入的构造可以交给 AI 批量生成（RL-03 的毒消息样本由 AI 生成 20 个变体）；
- 恢复后的数据一致性比对脚本由 AI 起草，人核对状态机转移是否符合设计——状态机语义 AI 常想当然。

## 7.9.6 本节清单

- 六剧本逐一对应 SRS 条款或架构风险点；
- 预期行为事先写死，执行只验证不追认；
- 每剧本含恢复步骤与数据校验步骤；
- 发现的缺陷（BUG-125 等）有修复与复测闭环；
- 讲得清专项与演练的分工与承接关系。

## 7.9.7 安全性专项：SCA、渗透与基线核查

安全性专项由三条线组成：依赖成分分析（SCA）——你引用的第三方库有没有已知漏洞；渗透测试——站在攻击者角度打自己的接口；等保基线核查——对照等保 2.0 基本要求逐项签认（本项目 ST-01 共 33 项）。三条线各有工具、各有产出，合起来构成安全证据链。本节逐线讲怎么做。

### SCA：依赖成分分析

现代软件的代码大头是别人的代码：本项目依赖 47 个直接依赖 + 213 个传递依赖。SCA 的任务是把这些依赖逐一比对公开漏洞库（CVE/GHSA），找出带已知漏洞的版本。工具：pip-audit（Python 侧）、npm audit（前端侧）——教学环境够用，工程上还有 Snyk/Dependabot。

实测流程与结果：pip-audit 首轮扫描发现高危 2 项（一个 XML 解析库的实体注入漏洞、一个老版本 Web 框架的请求走私漏洞）——两项均来自传递依赖，直接依赖清单里根本看不见它们，这就是为什么必须扫全依赖树。处置：升级到修复版本（注意传递依赖要靠约束文件钉版本，不能只升直接依赖指望传递依赖跟上），重扫清零。最终口径：高危 2→0，中危 7 项有缓解说明（均为内网部署场景下不可利用的向量，书面说明理由）。

教学要点：『高危清零』是硬门槛，『中危书面说明』是工程现实——不是所有中危都能升（版本冲突），但每一项都要有『为什么可利用性低/已缓解』的书面论证，不能装没看见。

### 渗透测试：站在对面打自己

渗透测试聚焦 OWASP 类风险，本项目实测两条主线：

- SQL 注入 12 个注入点（所有表单字段、查询参数、路径参数）：全部参数化查询拦截，无一穿透——感谢任务 3 设计时定的『ORM 参数化、禁止字符串拼接 SQL』红线；
- XSS 8 个注入点（图斑属性、任务名称、用户输入的一切会回显的地方）：全部转义输出拦截，无一弹窗。

工具与分工：ZAP/Burp 做自动扫描打底（爬虫遍历 6 端点自动加载荷），人工做定向测试补自动扫描的盲区——自动扫描测不出业务逻辑漏洞，本项目最严重的安全问题 BUG-133（BOLA 越权：改 URL 里的任务 ID 就能看到别人的任务结果）就是人工测出来的，详见 3.2 节。

渗透测试的纪律：只在自己的测试环境打，绝不对任何他人系统、公网系统使用这些技术；测试前把账号权限矩阵写在纸上（哪些角色该看到什么），越权测试才有对照基准。

### 等保基线核查（ST-01，33 项）

对照等保 2.0 三级基本要求中适用于本系统的条款，整理成 33 项核查表，逐项签认。核查项分三类：

- 身份鉴别与访问控制（11 项）：口令复杂度与加盐存储、登录失败锁定、会话超时、最小权限——实测口令 bcrypt 加盐、5 次失败锁定 15 分钟、会话 30 分钟超时，全部签认；
- 安全审计（9 项）：操作日志覆盖增删改、日志含时间/主体/事件、日志防篡改（只追加）、日志留存≥6 个月——签认 8 项，1 项部分符合（日志留存策略已配置但未跑满 6 个月，按合同条款说明）；
- 数据完整性与保密性（13 项）：传输 TLS、敏感配置不落库明文、备份策略、剩余信息清除——全部签认。

签认的格式：每项 符合/部分符合/不符合 + 证据（配置截图/测试记录编号/设计文档条目）。33 项全部有证据支撑，这是等保核查与『我觉得挺安全』的区别。

### 安全红线的回归验证

任务 4 定的安全红线（口令不明文、SQL 参数化、输入校验）在任务 5 要做一次全库回归验证：grep 全代码库确认无明文口令模式、无 SQL 字符串拼接、所有端点入



参有校验器。这一步常被认为是重复劳动，但红队自测发现的 BUG-133（越权）恰恰说明：红线回归过的是『写没写』，渗透测的是『漏没漏』，两层都要。

## 人机协同要点

- AI 生成渗透测试用例清单（按 OWASP Top 10 对本系统 6 端点逐一套），人按业务筛掉不适用项（如本系统无文件下载越权场景则划掉对应项并写理由）；
- SCA 报告由 AI 汇总成表格并起草中危缓解说明，人逐项核实『不可利用』的论证是否成立——AI 容易把『内网部署』当成万能挡箭牌，要追问攻击面（本项目 Web 端对园区网开放，内网论证要谨慎）；
- 等保 33 项核查表由 AI 按标准条款生成骨架，人逐项填证据。

## 本节清单

- SCA 全依赖树扫描（47+213），高危清零，中危有书面缓解论证；
- SQL 注入 12 点 + XSS 8 点渗透记录全拦截；
- 业务逻辑漏洞（BOLA）有人工定向测试；
- 等保 33 项逐项签认带证据；
- 安全红线全库回归验证完成；
- 渗透只在自有环境，权限矩阵事先书面化。

## 7.10 可移植性、兼容性、易用性与维护性专项

可移植性回答的问题是『换台机器还活吗』。开发机上能跑不算数——开发机上有三周以来随手装的一堆依赖、改过的环境变量、手动建的目录，这些是『隐形脚手架』。可移植性专项就是拆掉脚手架：在干净环境里仅凭交付物完成部署。本项目用 3 台干净虚拟机实测了 3 次。本节讲部署验证怎么设计、怎么做才算数。

### 7.10.1 干净环境的定义

『干净』必须有定义，否则验收时扯皮。本项目的干净环境标准：全新安装的 Ubuntu 22.04 虚拟机，只装 Docker 与 Docker Compose（这是交付前提，写在部署文档首页），无其他任何项目相关软件、无项目代码缓存、无外网代理特权。部署者只能使用交付物清单里的东西：docker-compose.yml、镜像（或 Dockerfile）、.env.example、部署文档。这模拟的就是甲方机房的真实处境。

为什么强调『仅凭交付物』：专项的目的之一是验证文档的完备性——部署文档漏写一步（比如忘了说某目录要建、某端口要开），干净环境里立刻暴露。3 台虚拟机实测耗时 1h21m / 1h37m / 1h52m，第一次最长，暴露的 4 个文档缺口补齐后逐次缩短——耗时下降曲线本身就是文档质量改善的证据。

### 7.10.2 部署验证的验收点

部署成功不是『容器起来了』，验收点清单：

- 一键起：docker compose up -d 一条命令起全部服务，无人工干预步骤（中途要 hand-edit 配置就算失败项）；
- 健康检：所有健康检查端点转绿，起后 5 分钟内无重启（起得来但反复崩的不算）；
- 冒烟通：起完立刻跑标准冒烟流程（上传→全自动分割→交互→导出）全绿——这一步用真实小影像，证明推理链路在干净环境里真的通，包括 GPU 直通配置；
- 数据持久：重启整套服务后历史任务与数据仍在（挂载卷配置正确的验证）；
- 离线起：断外网状态下能起（镜像已在本地的前提下）——甲方内网场景的关键验证。

### 7.10.3 常见踩坑与对策

教学项目部署验证的高发坑（本项目踩过的）：

- 镜像里藏了宿主路径：某配置文件写了 /home/开发队员/... 的绝对路径，干净环境直接起不来——对策：CI 里加一步『在干净容器内自检配置引用』；
- GPU 直通门槛：nvidia-container-toolkit 在干净虚拟机上的安装步骤漏写进文档——对策：把『交付前提』从一句话扩成一节（含版本号与验证命令 nvidia-smi 容器内可见）；
- 首次启动的初始化顺序：数据库迁移脚本与后端启动的先后依赖没编排，第一次起报连表不存在——对策：compose 里补 healthcheck 依赖或加 entrypoint 等待脚本；

- .env.example 与真实需要的变量不同步（开发中加了新变量忘了更新 example）  
——对策：CI 加『.env.example 与 compose 引用变量集合 diff 为空』检查。

#### 7.10.4 互操作验证：WFS + QGIS

可移植性还包括数据的『可带走』：本项目支持 WFS 标准服务，专项里用 QGIS（第三方 GIS 桌面软件）直连系统的 WFS 端点，验证图斑数据能被标准客户端读取、属性完整、坐标正确。这一步测的是标准符合性——『支持 WFS』不能靠嘴说，拿一个没参与开发的通用客户端连上才算数。实测 QGIS 3.x 直连成功，图斑几何与属性字段完整。

#### 7.10.5 人机协同要点

- 让 AI 对着部署文档做『陌生人模拟』：『假设你只拿到这份文档和 compose 文件，一步步推演部署过程，指出你会卡在哪』——AI 角色扮演能抓到一部分文档缺口，但最终裁决必须靠真实干净环境；
- 部署失败时的日志分析交给 AI 初筛，人定根因；
- .env 变量 diff 检查、配置引用自检这类 CI 检查脚本适合 AI 起草。

#### 7.10.6 本节清单

- 干净环境标准书面化（只装 Docker、仅用交付物）；
- 3 次独立部署实测记录（含耗时与暴露的文档缺口）；
- 验收点五项（一键起/健康检/冒烟通/数据持久/离线起）逐项过；
- 高发坑对策已固化进 CI 或文档；
- WFS 用第三方客户端（QGIS）实测互操作。

#### 7.10.7 兼容性、易用性与维护性专项

兼容性（GB/T 25000.10 质量特性之一）的回答对象是『在哪些环境下能正常工作』。兼容性测试的难点不在技术而在组合爆炸：浏览器 × 数据格式 × 操作系统 × 分辨率，全组合测不起，必须设计矩阵、科学裁剪。本项目实测矩阵：3 浏览器 × 4 影像格式 = 12 个核心组合。本节讲矩阵怎么设计、怎么裁剪、怎么执行。

#### 兼容性维度从哪里来

兼容性维度不许拍脑袋，必须逐项追溯到需求来源。本项目的四个维度与出处：

| 维度   | 取值                                     | 需求出处                  | 业务理由                                              |
|------|----------------------------------------|-----------------------|---------------------------------------------------|
| 浏览器  | Chrome / Edge / Firefox                | SRS NFR-C1            | 甲方机房禁止装 Chrome 之外的国产浏览器白名单里有 Edge；教师开发机用 Firefox  |
| 影像格式 | GeoTIFF / COG / JPEG2000 / 普通 PNG      | SRS FR-01.1 、 FR-01.2 | 甲方存量数据以 GeoTIFF 为主，新采购卫星数据有 JPEG2000；COG 用于云端在线影像 |
| 操作系统 | Ubuntu 22.04（服务端）/ Windows 10+（客户端浏览器） | 部署合同条款                | 服务端合同锁定 Ubuntu 22.04 LTS                          |
| 分辨率  | 1920×1080 及以上                          | SRS NFR-U2            | 大屏会商场景，地图操作区最小可用宽度                                |

每个维度都答得出『谁要求的、写在哪份文档第几条』——这是兼容性矩阵与脑袋清单的区别。

## 组合爆炸与裁剪策略

全组合 = 3 浏览器 × 4 格式 × 2 OS × 2 分辨率 = 48 项，教学周期内不可能全测。裁剪三刀：

- 第一刀（按架构正交性）：服务端兼容性（OS）与数据格式正交于浏览器——格式解析在后端，浏览器只看渲染结果。拆成『浏览器 × 核心交互』和『格式 × 服务端解析』两个子矩阵，48 项降到 12 + 8 项；
- 第二刀（按风险）：PNG 是普通图片格式（教学辅助用途），走标准库解析、风险最低，降为抽查；GeoTIFF/COG/JPEG2000 是业务主力，全组合保留；
- 第三刀（按合同）：服务端 OS 合同锁定单值，不构成矩阵维度，只做单点验证。

裁剪不是省略，每一个被裁掉的组合都要有书面的风险理由（为什么这个组合低风险）。评审时裁剪理由和测试结果一样受检查。

## 执行与记录

12 个核心组合的执行要点：每个组合跑同一条标准冒烟流程（上传 → 全自动分割

→交互修正→导出），判定标准统一（流程走通+关键结果数值一致——同一影像在不同浏览器下分割结果的图斑数、面积应在容差内一致）。记录用矩阵表直接打钩：行是组合、列是冒烟步骤、单元格填通过/失败+缺陷号。本项实测12组合全部通过，其中Firefox+JPEG2000组合发现1个轻微问题（进度条不刷新，BUG-1xx，轻微级，不影响数据正确性）——如实记录，轻微缺陷不阻断验收但要在报告中披露。

浏览器测试的人工技巧：DevTools的设备模拟不能替代真实安装——字体渲染、WebGL后端、文件上传控件的差异只在真实浏览器里暴露。教学环境至少准备三台不同浏览器的真实环境（可以是一台机器装三个浏览器+一台教师机）。

### 人机协同要点（兼容性专项）

- 矩阵维度与组合的穷举可让AI列草稿（『遥感Web应用常见兼容性维度有哪些』），但裁剪决策必须人来拍——AI不知道甲方机房的浏览器白名单；
- 让AI生成兼容性矩阵的记录表骨架，人填实测结果；
- 浏览器自动化（Playwright跨浏览器跑冒烟流程）值得做：一套脚本跑三种浏览器，矩阵执行成本降三分之二——但断言只断言流程走通与关键数值，视觉问题仍要人看。

### 为什么不用『用户问卷』做主证据

教学周期内招募真实遥感用户做大样本问卷不现实，且问卷测的是主观满意度，测不出『用户卡在哪一步』。走查法更适合：由走查员扮演典型用户角色（本项目定义两个画像：遥感解译员小周——天天用，追求效率；应急值班员老钱——三个月用一次，追求不出错），按典型任务走全流程，每一步对照可用性启发式原则检查。问卷可以做（SRS里有满意度NFR），但只作辅助证据。

### 启发式走查的操作规程

采用Nielsen十原则做检查清单，结合遥感业务加两条领域原则（坐标/比例尺永远可见；耗时操作必有进度与取消）。走查流程：

- 定任务：从SRS选6个核心任务（上传影像并发起全自动分割/文本提示分割/交互点击修正/图斑属性编辑与导出/查看任务历史/系统降级提示识别）；
- 独立走查：至少2名走查员各自独立走一遍，各自记录问题（独立是为了避免互相带节奏）；

- 会合定级：走查员会合，合并问题清单，按『阻断任务 / 显著拖慢 / 轻微不便 / 建议优化』四级定级；
- 记录格式：问题描述 + 触发步骤 + 违反的启发式原则编号 + 定级 + 截图。没有截图的问题记录无效。

样例走查结果（《测试报告》）：共记录 14 项，其中显著拖慢 2 项（全自动分割发起后无预计剩余时间——值班员画像下这算显著问题；属性批量编辑入口藏太深），轻微 5 项，建议 7 项。2 项显著问题均已在任务 5 内修复（PR#78 进度估算、PR#79 入口前置），回归走查确认。

### **任务式实测：让外人来做**

走查员是自己人，有知识盲区——真正的易用性证据来自不熟悉系统的人。做法：请邻组一名同学（没接触过本系统的）拿一份一页纸任务单（只写要做什么，不写怎么做），计时完成 3 个任务，观察员在旁记录卡点（沉默超过 30 秒的地方、误操作、求助）。本项目实测：3 个任务平均完成时间 4 分 20 秒，卡点 2 处（文本提示的输入语法示例不够显眼；导出格式选项的缩写 COG 无解释）——这两处都转化为改进项。

答辩要点：『我们请没接触过系统的人 4 分 20 秒完成了核心任务，卡点是这两处、已经这样改了』——这是把主观特性讲成客观证据的标准句式。

### **人机协同要点（易用性专项）**

- 让 AI 按 Nielsen 十原则生成走查清单初稿，人按业务补领域原则；
- AI 不能替你做走查——它看不到真实渲染的界面，更体会不到等待 8 分钟的焦灼。但可以做完走查后把记录喂给 AI 做定级一致性质检（『这条定显著那条定轻微，标准一致吗』）；
- 外人实测的观察记录由 AI 整理成结构化卡点清单，人确认。

### **SonarQube 质量阈配置**

SonarQube 不是跑一遍看报告，而是先配质量阈（Quality Gate）——什么叫过、什么叫不过，事先定死。本项目质量阈：

- 阻断级（Blocker）问题 = 0（硬门槛，一个不放过）；

- 严重级（Critical）问题 = 0（任务 5 内清零，首轮扫描有 2 个：一处空指针隐患、一处资源未关闭）；
- 新增代码覆盖率  $\geq 70\%$ （与 CI 门禁同口径）；
- 代码重复率  $\leq 3\%$ （实测 2.1%）。

首轮扫描结果：阻断 0、严重 2、重复率 2.1%、主要级（Major）31 个。处置策略：阻断和严重清零（必修）；主要级 31 个不全修——逐一 triage，12 个确认为有效问题安排修复，19 个标记为 Won't Fix 并写明理由（如『该复杂度告警来自协议解析的状态机，拆函数反而损害可读性』）。triage 记录本身就是交付物——评审看的不是你修了多少，而是你对每一个问题做了有依据的决策。

## 重复率的治理

重复率 2.1% 的背后故事：首轮扫描是 5.8%，大头在两个地方——三种影像格式的解析代码里各抄了一遍同样的分块读取逻辑，以及前后端各写了一份坐标转换常量表。治理：分块读取抽公共函数（顺带发现三处副本里有一处有个微妙的偏移 bug——重复代码的危害实例化：改 bug 只改了一处副本）；常量表收敛为后端单一权威源、前端从接口获取。降到 2.1% 后剩余重复均为测试夹具里的样板代码，标记 Won't Fix。

## 覆盖率与维护性的联动

覆盖率（整体 78%、核心 93%、增量 91%）不只是测试指标，也是维护性指标：改动一行代码后，有没有测试替你守住行为，决定了『敢不敢改』。联动审查的做法：把 SonarQube 的复杂度热点图和覆盖率热力图叠在一起看——高复杂度 + 低覆盖率的文件是维护性的雷区，本项目用这个方法定位出 2 个雷区文件（推理调度器、格式适配器），专项补了 14 个测试用例后才敢在 CR-001 里动调度器。

## 技术债台账

不是所有问题都能在任务 5 清零——有些债还不起也不该现在还（如『推理引擎未来应迁移到 TensorRT』）。对不还的债，立台账：债务描述、不还不是不能的理由、偿还的触发条件、挂账责任人。本项目台账挂 3 项，全部写进测试报告的『遗留问题』章节并向评审披露。教学点：藏债是事故，亮债是管理。

## 人机协同要点（维护性专项）

- SonarQube 报告由 AI 做首轮 triage 建议（这条建议修、那条建议挂），人拍板——AI 对『这条 Major 该不该修』的判断大约七成像样，剩下三成需要你懂业务语义；
- 重复代码合并的 PR 适合 AI 起草（机械重构是它的强项），但合并后必须跑全量回归——AI 重构偶尔会顺手『优化』掉它看不懂的边界处理；
- 让 AI 对技术债台账做质询（『这项债如果三个月后爆雷，最坏场景是什么』），帮你想全披露口径。

## 本节清单

- 每个兼容性维度有需求出处（文档 + 条目号）；
- 裁剪三刀（正交性/风险/合同）有书面理由；
- 核心 12 组合实测记录完整，失败项关联缺陷号；
- 轻微缺陷如实披露，不藏；
- 能答辩：为什么 Firefox+JPEG2000 要测而 PNG+Edge 只抽查。
- 两个用户画像定义在案；
- 6 任务 × 2 走查员独立走查，记录带截图与原则编号；
- 问题四级定级，显著级以上有修复闭环；
- 外人实测（非本组成员）至少 1 轮，卡点转化为改进项；
- 易用性证据可答辩复述成客观句式。
- 质量阈事先配置（阻断 0/严重 0/重复率≤3%/增量覆盖≥70%）；
- 阻断、严重清零有 PR 闭环；
- Major 级逐条 triage，Won't Fix 有书面理由；
- 重复率治理有实例（5.8%→2.1%），合并后回归通过；
- 复杂度×覆盖率雷区分析并补强；
- 技术债台账 3 项挂账披露，不藏债。

## 7.11 破坏性测试：攻击者思维与组内红队自测

破坏性测试（组内红队自测）是本课程质量门禁最具杀伤力的手段：组内分设红队角色，由未参与对应模块设计的组员担任，按本组 SRS 写破坏性用例攻击己方系统，



红队发现的有效缺陷计入红队成员个人测试分，凑数无效缺陷倒扣。激励机制下，红队有动力往死里打，开发角色有动力在战前把系统做扎实。本节讲规则设计、组织流程与五个真实命中案例的攻击思路。

### 7.11.1 为什么自测测不出这些缺陷

自测有系统性盲区：用例是自己 SRS 里需求的展开，思维路径和设计者同源——设计者没想到的，测试者往往也没想到。红队自测的价值在于换一个『没有参与该模块设计的大脑』来攻击：红队只读 SRS 和界面，不受实现思路污染。本项目实战数据雄辩地证明了这一点：18 条缺陷中 5 条来自红队自测，且含金量极高——5 条里 2 条严重级（BUG-131 上传 OOM、BUG-133 BOLA 越权），全部是自测用例集里没有的攻击向量。

### 7.11.2 规则设计：红队激励与倒扣

规则原文（教学大纲口径）：红队成员发现的有效缺陷计入其个人测试分；凑数无效缺陷倒扣。规则的精妙之处：

- 奖励有效缺陷 → 红队有动力深挖真实问题，而不是交差；
- 倒扣无效缺陷 → 抑制『刷量』冲动，每条用例都要有合理的攻击假设；
- 缺陷定级仲裁权在教师/评委会 → 防止红队与开发角色私下讨价还价。

有效缺陷的判定标准：可复现（附步骤与证据）、违反 SRS 明文条款或公认安全/工程底线、非重复。无效缺陷的典型：臆造型（不看 SRS 就提『我觉得应该有 XX 功能』）、环境型（在不合规环境里跑出问题）、重复型（换措辞报同一问题）。

### 7.11.3 组织流程

破坏性自测的标准流程（以本项目组内红队自测为例）：

- 战前准备：红队只拿 SRS、部署好的系统访问入口、测试账号（含不同权限角色）——注意不给设计文档和已有测试用例（模拟真实攻击的信息不对称）；
- 攻击窗口：48 小时，红队编写并执行破坏性用例，实时提交缺陷单（缺陷单格式与自测一致：步骤/预期/实际/证据/建议定级）；
- 防守角色响应：对每条缺陷单做 确认有效/申辩无效 响应，申辩必须引用 SRS 条款原文；

- 裁决会：对有争议的缺陷单现场裁决，教师主持；
- 计分：有效缺陷数按定级加权，计入红队成员个人测试分，无效缺陷倒扣。

#### 7.11.4 实战数据解读

本项目红队自测共执行 30 条破坏性用例：有效 5 条、无效倒扣 3 条、正常未命中 22 条。数据解读：

- 防守失分集中在两类：资源保护（BUG-131 大文件上传撑爆内存——PE 条款只约了分割性能没约上传体积上限）与访问控制（BUG-133 越权）——暴露了 SRS 条款的覆盖盲区；
- 22 条未命中用例同样值得复盘：每一条都排除了一类攻击向量，印证「功能测试大家都能做，破坏性思维需要练」；
- 无效倒扣 3 条的教训：2 条臆造型 + 1 条环境型——攻击方不看 SRS 就开火的代价。

#### 7.11.5 争议裁决案例精讲：降级态 14.8 秒之争

自测中最精彩的争议：红队在 GPU 降级态测得交互分割单轮 14.8s，主张违反性能条款。开发角色申辩：SRS PE-09 原文为『GPU 不可用时自动降级至 CPU 推理，降级态单轮响应≤30s 并给出降级提示』——14.8s 在降级条款内，且降级提示可见，属设计内行为。教师裁决：我方胜诉。教学要点有三：其一，SRS 条款的措辞精度（降级态单独约定而非笼统一句）在争议时就是防线；其二，申辩必须引用原文，『我们设计就是这样想的』无效，写在纸上的才算数；其三，这也提醒攻击方：攻击前先把 SRS 读三遍，往条款缝隙里打，别往条款明文上撞。

#### 7.11.6 本节清单

- 理解红队激励与倒扣规则与有效/无效缺陷判定标准；
- 战前准备清单正确（红队只拿 SRS+系统+账号，不看设计与用例）；
- 缺陷单格式规范，申辩引用 SRS 原文；
- 有仲裁案例可复盘（降级态 14.8s 之争）；
- 赛后把有效缺陷的攻击向量补进回归用例库——红队自测的遗产是用例库变强。

### 7.11.7 红队案例精讲：五个有效缺陷的攻击思路

本节逐案拆解红队自测命中的五个有效缺陷（BUG-131~135），每个案例讲清四件事：攻击者的思路（他为什么往这打）、缺陷机理（系统为什么中招）、修复方案、以及这件事教给我们什么。这些案例是最好的破坏性思维教材。

#### **BUG-131：大文件上传撑爆内存（严重）**

攻击思路：攻击者注意到 SRS 写『支持单景 $\geq 10\text{GB}$  影像、8MB 分片断点续传』，但没有写上传体积上限与单分片大小校验——条款缝隙。攻击：伪造 Content-Length 声明小、实际持续推送超大分片流。

缺陷机理：上传接收端按声明长度预分配缓冲区并持续读入内存，未做实际字节数与声明的一致性校验，也无单连接内存配额——2GB 并发上传流直接把服务进程 OOM 打挂，服务中断。这是资源保护类缺陷：功能都对，但没有给恶意/异常输入设边界。

修复（PR#77）：分片实际字节数与声明一致性校验、单连接内存配额、超限即 413 拒绝并熔断该连接；补上传体积上限条款进 SRS（走变更流程）。教训：性能条款（快不快）与资源保护条款（撑不撑得死）是两回事，SRS 都要写。

#### **BUG-132：重复提交产生重复任务（幂等）**

攻击思路：攻击者对『发起全自动分割』接口在弱网模拟下快速双击/重试——Web 系统的经典幂等考题。

缺陷机理：创建任务接口无幂等键，同一影像 + 同一参数的两次请求创建了两个任务，两份推理结果并存，前端列表出现双胞胎任务。单个用户双击只是脏数据，并发场景下会造成 GPU 队列被重复任务占满。

修复（PR#75）：客户端生成幂等键（Idempotency-Key 头），服务端对同键请求返回首次任务而不新建；数据库加唯一约束兜底。教训：所有『创建类』接口默认要过幂等之问——同一请求来两次会怎样。

#### **BUG-133：BOLA 越权读取他人任务（严重）**

攻击思路：攻击者拿到两个不同角色的测试账号，用 A 账号发起任务拿到任务 ID，再用 B 账号把 URL 里的任务 ID 换成 A 的——对象级越权（BOLA，API 安全头号风

险)的标准套路,半分钟试出来。

缺陷机理:任务详情接口只校验了『登录了没有』,没校验『这个任务是不是你的/你有没有权看』——认证与授权分离的经典失误。所有带 ID 的 GET 接口全中招。

修复(PR#76):统一资源级鉴权中间件,所有对象访问过『归属或授权』检查;补全 6 端点的权限矩阵回归测试。教训最深:自动扫描工具测不出 BOLA(它不知道数据归属语义),必须人工定向测——这就是为什么渗透测试不能全靠 ZAP。答辩常问:你们怎么保证所有端点都做了对象级鉴权?标准答案:权限矩阵 × 6 端点的逐格回归用例。

### **BUG-134: 负坐标参数返回500 (一般)**

攻击思路:交互分割接口的点坐标参数,攻击者直接喂负数、喂超出影像范围的值——输入边界攻击的基本功。

缺陷机理:坐标未经范围校验直接进几何计算库,库内部断言失败抛未捕获异常,框架兜底返回 500 裸堆栈。两个问题:该 400(客户端错)不该 500(服务器错);堆栈外泄暴露内部路径(信息泄露隐患)。

修复(PR#74):入参 Joi/Pydantic 校验拦截越界坐标返回 400 与可读提示;全局异常处理器脱敏堆栈。教训:500 与 400 的区别是『系统有错』与『你输错了』的区别,混用等于承认系统脆弱。

### **BUG-135: 时区未指定导致任务时间错乱 (一般)**

攻击思路:攻击者创建任务时提交不带时区的 ISO 8601 时间串,再从不同时区的客户端查看任务列表。

缺陷机理:服务端把无时区时间按服务器本地时区解析入库,前端又按浏览器时区渲染,跨时区查看时任务时间偏移 8 小时。时间处理三宗罪之典型:存解析口径不一。

修复(PR#73):全链路统一 ISO 8601 带时区(UTC 存储、前端按本地时区显示),入参缺时区即 400。教训:时间、坐标、编码——凡是『有多个合法解释』的数据类型,接口必须强制单一解释。

## **五个案例的共同模式**

复盘五个案例，攻击思路收敛为四类通用向量：边界攻击（134——喂非法值）、资源攻击（131——喂过量）、状态攻击（132——重复/乱序请求）、授权攻击（133——换身份/换对象）。135 的时区属解释歧义攻击。把这五类向量做成检查单，对着自己系统的每个接口逐格过一遍——这就是从别人的攻击里长自己的本事。本组赛后已将该检查单（6 端点 × 5 向量）补进回归用例库。

## 本节清单

- 五案例能各自讲清攻击思路、机理、修复、教训；
- BOLA 的人工测试必要性讲得清（工具盲区）；
- 五类攻击向量检查单已建立并应用到全部端点；
- 每条修复有 PR 号与回归用例闭环。

## 7.12 故障演练：15 分钟定位与恢复

故障演练是质量门禁的现场考试：教师随机制造故障，团队凭日志与文档在 15 分钟内定位并恢复，达标即过，优秀加分。它考的不是系统（2.12 节已考过系统行为），考的是人——你们的运维知识是不是只在某一个人脑子里、文档是不是真能用、压力下协作是不是乱套。本节讲演练规则、三个实战案例与备战方法。

### 7.12.1 演练规则与评分

规则：教师从题库（即 2.12 节可靠性六剧本的变种）现场随机抽一个故障注入；团队从接到『系统异常』通知起计时；15 分钟内完成 定位（说出故障点与依据）+ 恢复（服务正常且数据校验通过）即达标；10 分钟内完成为优秀加分。允许使用一切交付文档与日志，禁止联系教师求助。

评分维度不只看结果：定位路径是否合理（有没有瞎试）、分工是否有序（有没有三个人同时改一个配置）、恢复后是否做了数据校验（很多人忘了这一步——服务起来了，但任务状态机里一堆僵尸记录）。

### 7.12.2 实战案例一：kill GPU Worker（6 分 12 秒）

故障：推理 Worker 进程被 kill -9。团队动线：值班同学先看健康检查面板（1 分钟内发现 worker-1 红）→ 查 Supervisor 日志确认进程已重启但队列积压（又 1 分钟）→ 发现重启的 Worker 加载模型后未正确声明 ready，任务不派发（2 分钟看日志定位）

→ 按操作卡执行『排空队列 → 重启 Worker → 验证 ready 探针』标准流程（2 分钟）  
→ 数据校验发现 1 个任务卡 running 态，按文档重置为 failed（1 分钟）。合计 6 分 12 秒，优秀档。关键成功因素：健康检查面板一眼定位到组件级、操作卡有现成流程、数据校验步骤写在操作卡最后一行（不会忘）。

### 7.12.3 实战案例二：模型依赖被卸载（4 分 12 秒）

故障：教师卸载了 sam3 的可选依赖。动线：文本提示分割返回 400，错误消息直接给出 pip 安装命令 → 先确认这是设计内的优雅降级而非崩溃（进程存活、其他模型仍可用）→ 用 model check 逐类型验证界定影响范围 → 按提示安装 → 清空模型缓存避免旧状态干扰 → 冒烟验证。合计 4 分 12 秒，达标。**\*\*这次演练实际验证的是 REL-01 的设计价值\*\***：错误消息里带着修复命令，把恢复从『查文档』压缩成『复制粘贴』。对照组同类故障耗时 11 分钟，差别就在于他们的错误消息只写了『模型加载失败』。

### 7.12.4 实战案例三：毒消息卡死队列（11 分 55 秒）

故障：队列被注入畸形任务消息（这是 BUG-125 修复前的历史演练，毒消息不进死信队列，Worker 反复重试）。动线：现象诡异——服务健康检查全绿但任务不推进（『绿而不动』是最难定位的故障形态）→ 查 Worker 日志发现同一消息反复消费报错 → 按文档查到死信队列为空，确认毒消息 → 手动将消息移入死信队列，服务恢复 → 数据校验。合计 11 分 55 秒，压线达标。此案例直接催生了 BUG-125 的修复与剧本库补毒消息变种（T-41）——演练暴露的问题要回到工程改进，不能演练完就翻篇。

### 7.12.5 备战方法：把演练变成可准备的考试

演练看似随机，其实可备：

- 依赖索引表：一页纸列全系统所有依赖（进程/端口/存储/外部服务）与各自的健康检查命令——定位的地图；
- 操作卡：每个已知故障剧本一张卡，写死现象 → 定位步骤 → 恢复步骤 → 数据校验步骤，演练时照着走；
- 慌乱期纪律：前 3 分钟只看不改（看监控、看日志、列假设），禁止重启、禁止改配置——绝大多数演练超时不是找不到故障，是慌着改东西把现场破坏了；

- 轮值演练：每周队内抽签当值班员，故意制造小故障练手——恢复技能长在每个人身上，不是只长在运维担当身上。

### 7.12.6 人机协同要点

- 操作卡由 AI 按故障剧本起草初稿，人实机验证每一步命令——AI 写的命令可能有版本差异，没验证过的操作卡是负资产；
- 演练复盘时把完整时间线喂给 AI，让它找时间浪费点（它的旁观视角常比当事人冷静）；
- 依赖索引表由 AI 从 compose 文件与配置中半自动提取，人核对补全。

### 7.12.7 本节清单

- 三案例（6'12"/8'47"/11'55"）时间线与动线可复述；
- 慌乱期纪律（前 3 分钟只看不改）成文；
- 依赖索引表与操作卡齐备且实机验证过；
- 演练暴露问题有工程改进闭环（T-41/T-42）；
- 轮值演练机制运行，人人练过。

## 7.13 缺陷管理与根因分析

任务 5 期间缺陷会密集涌现（本项目 5 天 24 条），没有管理纪律，缺陷就会变成群里的一堆零散消息——漏修、重复修、修完忘了回归。本节讲缺陷单的标准写法、生命周期管理与看板运营节奏。

### 7.13.1 缺陷单：写好是技术活

一份合格缺陷单的六要素：

- 标题：现象一句话（『上传 2GB 影像服务进程崩溃』，不是『上传有问题』）；
- 环境：版本 tag（sprint2-final）、环境配置、浏览器——缺陷与版本强绑定，脱离版本谈缺陷没有意义；
- 复现步骤：编号步骤，精确到输入值（『上传文件 big.tif（2.1GB），并发 2 个上传』——reviewer 照着做必须 100% 复现，复现不了的缺陷单先打回补步骤）；

- 预期 vs 实际：预期必须引用依据（SRS 条目号/设计文档条目/公认底线），『预期』不是『我觉得应该』；
- 证据：截图、日志片段、压测数据——无证据不立案；
- 定级建议与依据：严重/一般/轻微 + 为什么（影响面 × 严重性）。

定级标准（本项目口径）：严重 = 核心功能不可用/数据损坏/安全越权/服务中断；一般 = 功能受损但有绕行方案/非核心功能错误；轻微 = 界面/提示/易用性问题。定级决定修复优先级：严重不过夜，一般不过迭代，轻微进清单排期。

### 7.13.2 生命周期与状态机

缺陷状态机：新建（Open）→ 确认（Confirmed，测试负责人复核复现与定级）→ 修复中（Assigned，挂责任人）→ 待回归（Fixed，PR 合并 CI 绿）→ 关闭（Closed，测试方回归验证通过——关单权在测试方不在开发方，这是质量权独立的体现）。两个特殊状态：Rejected（非缺陷/无法复现，要写驳回理由）、Deferred（确认但挂账，进技术债台账）。

纪律要点：任何状态流转都要留痕（谁、何时、为什么）；Rejected 也要有闭环——红队自测的无效缺陷倒扣就靠 Rejected 记录支撑裁决；Deferred 必须挂台账，防止『以后再说』变成『永远不说』。

### 7.13.3 看板运营

缺陷看板四列（Open/修复中/待回归/关闭）加一条严重级泳道置顶。运营节奏：

- 每日站会过看板：只过三件事——严重级进展、待回归积压、新增缺陷的定级争议；
- 待回归积压预警：待回归列超过 5 条就要催回归——开发修得快、测试验得慢是常态瓶颈；
- 每日缺陷燃尽：Open+修复中 总数应随验收日逼近而收敛，本项目曲线：8/28 峰值 9 条在制 → 8/31 收敛到 2 条 → 9/1 清零。

18 条缺陷的最终分布：严重 5（BUG-121/125/130/131/133）、一般 10、轻微 9；全部闭环（22 修复 + 2 Deferred 挂台账）。修复 PR 编号 #72~#81 连续可查，CI 运行 #891~#904 全绿——这组编号链就是缺陷管理的审计轨迹。



#### 7.13.4 缺陷数据的总账

缺陷密度 = 缺陷数 / 代码规模：24 条 / 9.6K 行 = 2.5 个/KLOC。这个数没有绝对好坏标准，但有两个用法：与历史项目比（本校往届同类项目 3~6 个/KLOC，2.5 说明质量护栏起了作用）；分模块看（缺陷集中在哪几个模块——本项目上传链路 with 权限模块密度最高，正是架构最复杂处，符合『缺陷聚集在复杂度热点』的经验定律）。

#### 7.13.5 人机协同要点

- 缺陷单初稿可让 AI 从测试执行记录（日志+截图描述）生成，人核对复现步骤可执行性；
- AI 做缺陷去重检查（『这条和 BUG-xxx 是不是同一根因』）效果不错，利用它对全部缺陷单的文本相似性判断；
- 定级争议时让 AI 扮演反方（『这条为什么不该定严重』）帮助把定级理由写严密；
- 但定级的最终拍板、Rejected 的驳回决定必须人来——质量责任不能外包给 AI。

#### 7.13.6 本节清单

- 缺陷单六要素齐全，复现步骤 100% 可复现；
- 定级标准成文，严重不过夜执行有记录；
- 状态机闭环，关单权在测试方；
- 看板每日运营，燃尽曲线收敛可查；
- 缺陷密度 2.5/KLOC 及模块分布有解读；
- PR/CI 编号链构成完整审计轨迹。

#### 7.13.7 缺陷根因分析：发现环节分布的解读

缺陷管理管的是『每条缺陷的闭环』，根因分析管的是『整批缺陷告诉我们什么』。18 条缺陷按发现环节分布：系统测试 8 条、红队自测 5 条、故障演练 2 条、压测 3 条、走查与扫描 6 条。这张分布表是本项目质量工作的最有价值的复盘素材——它回答了『我们的质量护栏哪层强、哪层漏』。本节讲根因分类法、分布解读与改进转化。

## 根因分类法：不止于『谁写错了』

每条缺陷在关闭时要填根因分类（不是追责，是改进输入）。本项目的分类框架：

- 需求缺陷：SRS 本身有漏洞或歧义（BUG-131——上传体积上限根本没写进 SRS）；
- 设计缺陷：设计决策遗漏或错误（BUG-132——幂等设计在详细设计里没考虑）；
- 编码缺陷：设计对、实现错（BUG-134——校验器写了但这个端点漏挂）；
- 测试缺陷：用例没覆盖到（IT-011——CR-001 影响面分析漏了边界用例）；
- 环境/配置缺陷：部署配置问题。

24 条的根因分布：需求 3、设计 5、编码 11、测试 3、配置 2。编码占一半是常态；值得注意的是需求+设计合计 8 条（三分之一）——越上游的缺陷修复成本越高，这 8 条平均每条的修复成本是编码类的 3 倍（要改文档、走变更、改代码、补测试）。

## 发现环节分布的解读

按发现环节读这张表，每个数字都有故事：

- 系统测试 8 条最多——正常，系统测试是覆盖面最大的环节；其中 3 条（BUG-121/122/123）是 ST 首轮 13/14 的未过项，属功能边界问题，说明自测体系对常规路径有效；
- 红队自测 5 条，但严重级占 2 条——单位含金量最高。红队角色没有开发者的思维定势，专打你想不到的地方（幂等、越权、时区）。结论：红队自测是最稀缺的缺陷来源，值得为它留足 48 小时窗口；
- 压测 3 条——全是只有负载下才现形的问题（并发竞态、队列积压下的超时级联）。结论：有些缺陷在功能测试里物理上不可能出现，压测不是性能专项的专利，也是缺陷猎场；
- 故障演练 2 条——可靠性行为缺陷（毒消息、恢复后状态机僵尸）。注入故障才能看见的缺陷；
- 走查与扫描 6 条——工具与走查的产出，单条成本低（改文案、转义输出），量大但修复快。

### 环节有效性矩阵

把两个维度交叉——环节 × 严重度——得到环节有效性矩阵，答辩时的硬货：

| 发现环节  | 缺陷数 | 严重级数 | 单条平均发现成本  | 环节定位                |
|-------|-----|------|-----------|---------------------|
| 系统测试  | 8   | 1    | 中         | 常规功能边界的主防线          |
| 红队自测  | 5   | 2    | 高（48h 集中） | 破坏性思维的稀缺来源，严重缺陷占比最高 |
| 压测    | 3   | 1    | 中         | 负载才现形的并发/资源缺陷       |
| 故障演练  | 2   | 1    | 低（含备战）    | 可靠性行为缺陷的唯一来源        |
| 走查与扫描 | 6   | 0    | 低         | 低成本批量的表面缺陷          |

解读要点：没有任何单一环节能覆盖全部缺陷类型——五个环节各自捕获的缺陷集合交集很小（只有 1 条缺陷被两个环节独立发现过）。这就是『质量是构建出来的』的实证：每一层护栏拦下的是不同种类的漏网之鱼，砍掉任何一层，对应种类的缺陷就直通用户。

### 改进转化

根因分析的终点是过程改进，本项目从分布表转化出四条过程改进（写进测试报告的『经验与改进』章）：

- SRS 评审清单补『资源保护条款』专项（BUG-131 的教训：性能条款与资源保护分开列）；
- 详细设计评审加『幂等/越权/时区三问』固定议程（红队五案例的教训）；
- CR 影响面分析模板加『边界值用例重估』勾选项（IT-011 的教训）；
- 攻击向量检查单（五类）进回归用例库常驻。

### 人机协同要点

- 根因初分类可以让 AI 按缺陷单文本建议，人复核——AI 常把『编码错』和『设计漏』混淆（实现没按设计写，它分不清是设计的锅还是手的锅）；
- 分布表与矩阵的图表由 AI 生成，解读文字人写——数据解读是最不能外包的智力活动；

- 让 AI 对着根因分布抬杠：『如果我是评审老师，我会质疑你的分类口径哪三点』——预先堵住口径漏洞（如 BUG-133 算设计缺陷还是编码缺陷，要有书面判据）。

### 本节清单

- 每条缺陷有根因分类，分类口径有书面判据；
- 环节分布表（8/5/2/3/6）与环节×严重度矩阵产出；
- 能讲清『为什么每层护栏不可缺』（交集小、捕获集互补）；
- ≥3 条过程改进从根因分析转化而来，写进报告。

### 7.14 质量特性自评表与测试报告

《质量特性自评表》是质量门禁的自查总账：对照 GB/T 25000.10 的八大质量特性逐项声明达成情况、挂证据。它既是交付物，也是评审会的议程表——评审老师基本按这张表逐项质询。本节讲自评表的结构、填报纪律与答辩用法。

#### 7.14.1 八大特性与证据映射

GB/T 25000.10 定义的八大质量特性与本项目的证据归属（这张映射表建议直接抄进你的自评表骨架）：

| 质量特性  | 本项目声明                                      | 主证据（交付物）              |
|-------|--------------------------------------------|-----------------------|
| 功能适合性 | 功能覆盖 24 FR + 16 NFR，★款 13 条全过              | ST 报告 64/64、RTM v5 闭环 |
| 性能效率  | 交互 P95 2.3~7.9s（分并发档）、全自动 7min12s、降级 14.8s | 《测试报告》第 16 章含容量建议     |
| 兼容性   | 3 浏览器 × 4 格式 12 组合通过                       | 兼容性矩阵实测记录             |
| 易用性   | 2 画像走查 14 项、外人实测 4'20"                     | 走查记录、卡点改进闭环           |
| 可靠性   | 六故障剧本行为符                                   | 可靠性专项记录、演练报告          |

| 质量特性  | 本项目声明                             | 主证据（交付物）                  |
|-------|-----------------------------------|---------------------------|
|       | 合，三演练达标                           |                           |
| 信息安全性 | SCA 高危 0、渗透全拦截、等保 33 项签认          | SCA 报告、渗透记录、ST-01 核查表     |
| 维护性   | SonarQube 阻断/严重 0、重复率 2.1%、覆盖 78% | 静态分析报告、triage 记录、技术债台账    |
| 可移植性  | 3 台干净环境部署成功、WFS-QGIS 互通           | 部署实测记录（1h21m/1h37m/1h52m） |

### 7.14.2 填报纪律

自评表填报三条铁律：

- 每一项必须挂证据编号（报告章节号/记录编号），『已完成』三个字不构成证据——评审的第一反应永远是『证据呢』；
- 部分达成的项如实标『部分符合』并写差距与补救计划（本项目等保日志留存一项标部分符合，因未满足 6 个月留存期）——自评表的可信度恰恰由你敢于标注的不足撑起来；全绿的自评表反而引发全面复核；
- 填报人与复核人分离：专项负责人填，测试负责人复核证据真实性，组长对全表签字负责。

### 7.14.3 自评表的答辩用法

评审质询按表走，准备答辩也按表走：每一行准备一个『证据一分钟』——老师说『可靠性讲一讲』，你 60 秒内说出：六个剧本、两个缺陷及修复、三场演练用时、一条遗留改进。能在 1 分钟内给出带数字的答案，就是自评表填报质量的直接体现。反过来，如果某一行你讲不满一分钟，说明这项的证据还单薄，评审前补。

### 7.14.4 与测试报告的关系

自评表是索引，测试报告是正本：自评表每项的『主证据』列指向测试报告对应章节。两表一致性必须交叉检查——评审常用手法是随机抽自评表一行，翻到报告对应章节核对数字（抽到过『性能效率』行，核对 P95 数字与《测试报告》第 16 章是否

一致)。数字不一致 = 文档管理失控, 比缺陷本身更扣分。

### 7.14.5 人机协同要点

- 自评表骨架由 AI 按 GB/T 25000.10 八大特性生成, 人填证据编号;
- 让 AI 做表-文一致性核对(把自评表数字与测试报告正文逐一对照)——这类机械核对 AI 又快又细;
- 『部分符合』项的措辞由人写, AI 起草的措辞倾向于淡化差距, 要顶住。

### 7.14.6 本节清单

- 八大特性逐项有声明 + 证据编号;
- 部分符合项如实标注且有补救计划;
- 填报/复核/签字三级分离;
- 每行能讲『证据一分钟』;
- 自评表与测试报告数字交叉核对一致。

### 7.14.7 测试报告编写

《测试报告》是质量门禁的收口文档, 也是评审会的核心质询对象。它的性质决定写法: 这是一份『用数据说话』的证据文档, 不是工作总结——『我们认真测试了』一文不值, 『ST 首轮 13/14、回归 64/64、缺陷 24 条全闭环』才是内容。本节以《测试报告》为样例精讲结构、数据组织与常见败笔。

#### 结构骨架(按GB/T 15532)

《测试报告》的章节结构与各章职责:

- 1 引言: 测试对象(tag sprint2-final)、范围(引用测试计划)、依据标准(GB/T 15532、GB/T 25000.51、SRS v2.1)、环境(RTX 3060 12GB/32GB/Ubuntu 22.04)——版本与环境写死在第一章, 全文数据才有锚点;
- 2 测试执行概况: 三级测试的规模与结果总表(UT 312 / IT 86 / ST 64, 通过率与轮次)——评审第一眼看的表;
- 3 功能适合性: ★款 13 条逐条结果、RTM 闭环声明;

- 4 性能效率：压测数据（引《测试报告》第 16 章，本节给摘要与结论）；
- 5~10 其余六大特性：每特性一节，统一格式『目标 → 方法 → 数据 → 结论 → 遗留』；
- 11 缺陷统计与分析：缺陷看板总账 + 根因分布 + 环节有效性矩阵；
- 12 破坏性自测：攻击用例与裁决记录；
- 13 故障演练：三案例时间线；
- 14 结论与遗留问题：总体结论（建议通过验收）、遗留 2 项（Deferred 缺陷 + 等保日志留存）及补救计划；
- 附录：缺陷清单全表、用例执行明细索引、冻结集使用登记。

### 数据组织的三原则

第一，每个数字可溯源：报告里任何数字都能追到原始记录（压测原始日志、用例执行系统、缺陷看板）。《测试报告》在每个数据表下注明数据源——这不是繁文缛节，答辩时『这个数据哪来的』必须有秒答。第二，首轮与回归分开列：ST 首轮 13/14 → 回归 64/64 要写成两行，抹平成一行『64/64 通过』既造假又丢掉了修复能力的证据——首轮不过、修复、回归过，恰是质量工程的正常叙事。第三，不利数据不藏：BUG-133 越权这种严重缺陷写进报告并在答辩主动讲（我们被攻破了哪里、怎么修的、现在怎么防）——主动暴露的缺陷是测试有效的证据，被评审挖出来的缺陷是测试失职的证据。

### 常见败笔清单

历年评审记录的测试报告败笔（写报告时逐条自查）：

- 只有通过率没有用例数：『通过率 100%』——一共几条用例？3 条也是 100%；
- 覆盖范围与 SRS 对不上：报告测了 50 条需求，SRS 有 55 条，少的 5 条没有交代；
- 缺陷只有列表没有分析：24 条列完没有根因分布与环节解读——数据没变成信息；
- 结论无依据：『系统质量良好』——良好是形容词，报告要名词与数字；

- 遗留问题章节缺失或写『无』：没有任何系统无遗留，写无 = 没有认真收尾。

### 人机协同要点

- 报告骨架由 AI 按 GB/T 15532 模板生成，各章数据从测试管理系统导出后由 AI 整理成表，人写分析与结论——分工铁律：AI 组织数据，人对结论负责；
- 用 AI 做『败笔自查』：把败笔清单喂给 AI 让它对报告初稿逐条检查；
- 首轮/回归数据的整理容易出错（哪条用例第几轮过的），让 AI 从执行记录自动汇总，人抽查 10% 验证；
- 结论段必须人写。AI 写的结论四平八稳没有判断，而结论恰恰是报告里最需要判断的地方。

### 本节清单

- 结构对齐 GB/T 15532，版本环境锚定第一章；
- 每个数字可溯源到原始记录；
- 首轮与回归分行呈现；
- 严重缺陷主动披露并附修复闭环；
- 败笔清单逐条自查通过；
- 遗留问题如实列出带补救计划。

### 7.15 过关自检与交接

#### 7.15.1 附录 A：交付物模板速查

| 交付物          | 骨架来源                | 冻结前体检项                        |
|--------------|---------------------|-------------------------------|
| 测试设计（用例集）    | 六要素用例模板（1.5 节）      | 预期可判定、前置完整、挂接 RTM、AI-draft 摘标 |
| 测试报告         | GB/T 15532 结构（28.1） | 数字可溯源、首轮/回归分行、遗留披露            |
| 《测试报告》第 16 章 | 分场景分档结构（13~14 章）    | 口径三层下钻、容量建议三要素                |
| 缺陷看板         | 四列+严重泳道（25.3）       | 关单权在测试方、流转留                   |



| 交付物     | 骨架来源         | 冻结前体检项                    |
|---------|--------------|---------------------------|
|         |              | 痕、燃尽收敛                    |
| 质量特性自评表 | 八大特性映射（27.1） | 证据编号齐全、部分符合如实标、表文一致       |
| RTM v5  | 双向挂接（2.1 节）  | 55+14 全挂接、无一对零、抽 3 条可顺藤摸瓜 |
| 冻结集登记   | 使用登记表（15.1）  | 启封次数、操作人、版本逐项在案           |
| 对抗战报    | 攻防规则（22 章）   | 有效/无效判定有依据、申辩引原文          |
| 演练记录    | 三案例时间线（24 章） | 用时、定位路径、数据校验步骤齐全          |

### 7.15.2 附录 B：提示词资产清单（建议入库）

- T-P01 需求展开——单条需求+Given/When/Then 展开正常/备选/异常流（见 7.2 模式一）；
- T-P02 攻击者视角——按接口定义列五类攻击向量（见 7.2 模式二、7.11）；
- T-P03 边界猎手——按字段约束生成边界值/等价类用例（见 7.2 模式三）；
- T-P04 第一评审——甲方专家/开发/一致性三视角质询用例集（见 7.3）；
- T-P05 不可判定预期自查——让 AI 标出自己糊弄的预期（见 7.3）；
- T-P06 失败初筛——日志+步骤+commit 给三个候选根因（见 7.13）；
- T-P07 缺陷去重——文本相似性查同根因缺陷（见 7.13）；
- T-P08 陌生人模拟——AI 角色扮演推演部署文档缺口（见 8.6）；
- T-P09 表文一致性核对——自评表数字与报告正文逐一对照（见 8.3）；
- T-AP01 反模式——整份 SRS 一次生成全部用例（见 7.2）。

### 7.15.3 附录 C：关键模板实物

#### C.1 用例模板（六要素）

# 用例编号：XX-NNN（挂接需求 FR-XX.X） 前置条件：环境/数据/账号状态 步骤：编号步骤，输入值精确  
预期结果：可判断断言点（状态码/字段值/数值区间/元素可见性） 优先级：P0/P1/P2（依据：MVP/★款/风

险) 来源标记: 人工 / AI-draft (评审通过摘标)

### C.2 缺陷单模板 (六要素)

# BUG-NNN 标题 (现象一句话) 环境: 版本 tag / 配置 / 浏览器 复现步骤: 编号步骤, 100% 可复现 预期 vs 实际: 预期引用依据 (SRS 条目/公认底线) 证据: 截图/日志/数据 定级建议: 严重/一般/轻微 + 依据 (影响面×严重性) 实际执行人: (防臆造步骤)

### C.3 演练操作卡模板

# 操作卡: 故障名 现象: (用户侧与监控侧表现) 定位步骤: 1. 看依赖索引表 → 2. .... (慌乱期纪律: 前 3 分钟只看不改) 恢复步骤: .... (每步命令实机验证过) 数据校验步骤: .... (任务状态机/数据一致性) 验证: 冒烟流程

### C.4 冻结集使用登记

# 冻结影像集使用登记 日期 / 操作人 / 测试版本(tag) / 用途(AT-0X) / 结果位置 纪律: 冻结集不得用于任何调优; 使用权归测试负责人

## 7.15.4 附录 D: 本指导书引用的样例文件索引

《测试计划》(测试级别/范围/策略/进度, GB/T 15532); 《测试报告》(测试设计 50 条用例、三级测试全记录、缺陷分析、性能压测专项 S1~S4 四场景数据与容量建议、结论与遗留——测试设计与压测内容已并入该报告对应篇章); 《质量特性自评表》(八大特性逐项声明与证据编号, 随交付包归档); RTM v5 (24 FR + 16 NFR + 3 AT 验收挂接闭环, 随交付包归档); 《缺陷看板与修复记录》(24 条全闭环、环节有效性矩阵, 详见《测试报告》缺陷章节)。全部位于教学案例工作区 tender\_case/, 供读者拆解研究。

## 7.15.5 结语

任务 5 教的不是『把测试跑完』——跑测试谁都会。真正要训练的是: 让每个结论都有数据撑腰的纪律(证据链)、让每层护栏拦不同种类的鱼的体系观(环节有效性矩阵)、以及把攻击、故障和翻车都变成养分的学习机制(红队自测、演练、根因分析)。50 条设计用例会过时, 18 条缺陷会被遗忘, 但『只看报告数据, 不接受我觉得没问题』这条规矩, 希望你能带着它走进第一份工作的每一次验收。

## 第 8 章 第六关：交付验收与复盘

本关完成收口：文档审计、可移植性验证、AI 使用审计三线并行，验收现场 20 分钟（演示、价值论证、代码走查），复盘报告把计划与实际的偏差一笔一笔算清，最后移交归档。验收的 20 分钟，靠前 19 天攒下的证据说话。

### 8.1 关卡导论：验收驱动与三层交付物

通关任务 6（交付验收与复盘，D18）是整个课程的管理线收口。前五关你证明了『能把东西做出来』，这一关要证明三件更难的事：你做的文档别人能审计（文档审计）、你交付的包别人能跑起来（可移植性批量验证）、你宣称的 AI 过程别人能查验（AI 使用审计）。然后站在评委面前 20 分钟：10 分钟演示与 UAT、5 分钟价值论证、5 分钟个人代码走查——最后交出一份诚实的复盘报告。

#### 8.1.1 本关与任务 1 投标的呼应

任务 1 你写下了承诺：投标书的方案、需求确认书的 MVP 范围、AI 辅助策略的分工口径。任务 6 就是兑现承诺并接受核对的时刻。教学设计上这是故意的：让学生在低风险环境里完整经历『承诺—构建—验收—审计—复盘』的商业闭环。评审会当场把你的投标书摊开在旁边对照——承诺了什么、交付了什么、差在哪、为什么。

#### 8.1.2 本关四件事的本质

- 文档审计（对照 GB/T 8567 清单逐份四维评分）：考文档的『移交态』——不是写没写，是别人能不能凭它接手；
- 可移植性批量验证（助教干净环境跑部署）：考交付的『独立可复现』——自测三轮不算数，陌生人能跑通才算数；
- AI 使用审计（Prompt 日志与 commit 一一对应）：考过程的『诚实与可追溯』——不是查用没用 AI，是查你宣称的使用是否经得起核验；
- 验收汇报与复盘：考『把 18 个工作日的工作讲清楚并诚实面对偏差』的能力——包括把失分项讲出来。

#### 8.1.3 配套样例索引（交付审计类样例文档（文档审计、可移植性验证、AI 使用审计、验收汇报 PPT、复盘报告、验收报告））

| 样例        | 内容                  | 本指导书对应章节 |
|-----------|---------------------|----------|
| 42 文档审计报告 | GB/T 8567 清单逐份四维评分， | 第 3~8 章  |

| 样例                | 内容                           | 本指导书对应章节  |
|-------------------|------------------------------|-----------|
|                   | 7 项问题整改闭环，93.8 分             |           |
| 43 可移植性批量验证记录     | 助教干净环境 1h12m 部署详录+全班 6 组总表   | 第 9~13 章  |
| 44 AI 使用审计报告      | 4 个 PR 三单对照，1 项不符合扣分通报       | 第 14~16 章 |
| 45 验收汇报 PPT（16 页） | 结论先行的管理汇报式设计                 | 第 17~22 章 |
| 46 项目开发总结报告       | 计划 vs 实际、AI 量化、技术债、经验教训、课程建议 | 第 23~28 章 |
| 47 验收报告（甲方签署）     | UAT 逐条记录、价值论证、代码走查、签署        | 第 17~20 章 |

#### 8.1.4 本指导书的用法

第 3~16 章讲课前三大审计怎么准备（占本关工作量的七成），第 17~22 章讲验收现场 20 分钟怎么打，第 23~29 章讲复盘报告怎么写，第 30~34 章讲收口关的人机协同与评审应对，第 35~41 章是迁移、工具、日程、FAQ 与附录。与前五本指导书一样：先讲人工怎么做（工具/要点/过程），再讲人机协同怎么加速（提示词/审查清单），全部样例数据来自澜图平台真实教学案例。

#### 8.1.5 总方法论：验收驱动与三层交付物

##### **验收现场不该有任何『第一次』**

收口关的总原则一句话：验收现场不该有任何第一次。第一次部署不该在验收机上（课前助教验证过）、第一次演示不该在评委前（彩排至少两轮）、第一次讲自己的代码不该在走查时（组内互讲过）、第一次看审计问题不该在审计报告里（自查清单先跑过）。第 6 关的所有方法论都是这句话的展开：把一切可能出错的环节从『现场』前移到『课前』。

##### **三层交付物盘点法**

复盘从盘点开始。把你的全部产出分三层（验收汇报 PPT 样例第 4 页就是这张图）：

- 产品层：系统本体（tag、镜像、compose、冒烟脚本）——回答『交付了什么』；
- 文档层：国标 14 份 + 机器可读集 + 过程样例——回答『凭什么相信』；
- 机制层：PR 门禁、变更四步、测试护栏、演练资产、Prompt 模式库——回答『下一组人怎么继续』。

多数组只盘点产品层，验收时被问『你们留下了什么可复用的东西』答不上来。三层盘点放进 PPT 和复盘报告，格局立刻不同。

## 证据链收口：从『数据在』到『数据能查到』

前五关积累了大量证据（CI 记录、缺陷单、压测日志、PR 对话、Prompt 日志）。收口关要把它们从『存在』变成『可检索』：每项关键结论标注出处编号（《测试报告》第几章、CI 第几号构建、PR 第几号）。答辩的铁规矩不变：只看报告数据，不接受『我觉得没问题』——而数据的前提是你能在 30 秒内翻出来。

## 收口关的人机分工总览

收口关 AI 的价值集中在三处：一致性核对（跨文档口径比对又快又细）、材料组织（审计清单、移交清单、PPT 骨架、复盘数据汇总）、模拟质询（扮演评委/审计员/陌生人找漏洞）。收口关 AI 的红线也集中在三处：披露与结论的措辞（部分符合、技术债、审计不符合项——AI 倾向淡化）、任何需要签名负责的内容、以及对历史证据的『补写』——补写日志是审计重罪，详见第 14~16 章。

## 本节清单

- 说得清第 6 关四件事各自在考什么；
- 三层交付物盘点完成，每样产出有归属；
- 关键结论全部标注出处编号，30 秒可查；
- 『现场无第一次』原则落实到彩排、自测、互讲计划。

## 8.2 文档审计：四维评分、流程与自查清单

文档审计是课前三大审计的第一项：审计组对照 GB/T 8567 交付清单逐份评分。你的目标不是『应付审计』，而是借审计把文档包推到移交态。本节讲审计怎么打分的（四维口径）、流程怎么走的、你该怎么配合。样例：《文档审计报告》（澜图平

台审计 93.8 分、7 项问题闭环）。

### 8.2.1 四维评分口径

| 维度    | 分值 | 审计要点                            | 典型失分形态                  |
|-------|----|---------------------------------|-------------------------|
| 结构完整性 | 25 | 章节对标 GB/T 8567/9385/15532，无缺项漏项 | 缺章节、缺签署页、缺版本沿革、缺附录      |
| 内容真实性 | 25 | 数据可溯源至原始记录，无量词套话                | 『性能良好』『运行稳定』类表述；数字无出处   |
| 一致性   | 25 | 跨文档口径一致（需求↔设计↔测试↔手册）            | 阈值/编号/术语/日期跨文档漂移        |
| 可执行性  | 25 | 照文档能操作：部署手册实测、用户手册走查            | 步骤缺前置条件、命令不可复制执行、版本号不写死 |

注意评分细则里两条隐含规则：『空洞套话不计分』——真实性维度的杀手锏；『部署手册由助教在干净环境实测』——可执行性维度的判定只认实测，纸面审查看不出来的问题（如《文档审计报告》D-04 版本号未写死）会在部署验证时现形，两个审计环节是联动的。

### 8.2.2 审计流程（以《文档审计报告》为例）

- 第一步 齐套性核查：对照课程 14 份国标清单 + 机器可读文档集逐件点名，缺件即记录（澜图组 14 份+机器可读集齐套）；
- 第二步 逐份四维评分：每份文档给四个维度分，写一句主要依据（不是只给分）；
- 第三步 问题登记：发现问题逐条编号（D-01~D-07），标维度与严重程度；
- 第四步 整改闭环：小组限期整改，审计组复核，闭环证据（commit 链）归档；
- 第五步 结论与放行：总分与『准予进入验收答辩』结论。

### 8.2.3 严重度与处置方式

《文档审计报告》的问题分级实践值得学：一般问题（如 D-03 错误码段位滞后于变更）当日整改+复核；轻微问题（如 D-05 截图未更新）当日整改；约定项（D-06 投标 24 周基线与课程 18 个工作日执行版的双轨口径）不改历史文档，用新文档（复盘报告专章）说明口径——这是配置管理的基本规矩：历史基线文档不许回改，口径差

异用新文档说明。

### 8.2.4 审计高失分区：变更的次生滞后

《文档审计报告》最有教学价值的发现：7 项问题中 3 项（D-03 错误码、D-05 手册截图、D-07 公式符号）全部与同一个变更 CR-001 相关——变更四步流程（spec→SRS→plan/tasks→代码）管住了代码侧，但文档侧的次生滞后（接口文档、用户手册、附录公式）没有进变更影响面清单。教训提炼：变更影响面清单要加『文档同步』勾选项，逐项列出所有引用了被改内容的文档。这也是 8.3 节一致性核查的重点。

### 8.2.5 人机协同要点

- 让 AI 按四维口径扮演审计员预审你的文档包：『对照 GB/T 8567，指出这份文档在结构完整性上的缺项』——AI 对结构缺项的检出率很高；
- 套话检测：让 AI 扫描全部文档找出『无数字、无出处、无量词』的表述（『性能良好』类），这是真实性维度的高效自查；
- 但评分决策与问题定级必须人来——AI 倾向于『看什么都像问题』或『看什么都合格』，取决于你怎么问，它没有立场。

### 8.2.6 本节清单

- 四维口径能背，每条维度的典型失分形态能举两例；
- 理解审计流程五步与问题分级处置（当日整改/约定项）；
- 变更影响面清单已含『文档同步』勾选项；
- AI 预审（结构缺项+套话扫描）已跑过一轮并留存记录。

### 8.2.7 文档自查清单：逐类文档的体检要点

独立审计到来之前，先自己当审计员。本节给出逐类文档的体检清单——每类文档列出审计员必查的点（来自《文档审计报告》审计依据列的逆向工程）。用法：对照清单逐份过，发现问题直接改，改完在清单上打钩留痕。这份清单本身就是你『自查过』的证据。

**投标与计划类（管理线关键文档（《项目建议书》《项目计划》《需求确认书》《项目管理计划》《风险管理计划》《迭代冲刺计划》））**

- 需求确认书：MVP 范围条目数与验收指标是否明确可测；签署页是否齐（甲乙双方+日期）；变更控制条款是否在案；
- 项目计划：WBS 到工作包级；里程碑有准出标准；版本沿革表完整；历史基线  
与执行版的关系是否说明（双轨口径——澜图组在此被记 D-06 约定项）；
- 风险登记册：升版沿革（v1→v2 的衔接说明，澜图组 D-01 失分点）；风险项  
的状态更新是否留痕。

### **需求类（需求类样例文档（《用户访谈记录》《软件需求规格说明书》 《AI 反向澄清记录》，RTM v2））**

- SRS：GB/T 9385 结构完整；每条需求可测试；★款标识清晰；版本沿革  
（V1.0→V3.0→V3.1 的修订记录逐行有依据）；
- spec.md：与 SRS 条目一一对应；公式、阈值、编号与 SRS 无漂移（澜图组 D-  
07 失分点：AT-01 公式脚注  $\Sigma$  下标不一致）；
- AI 反澄清记录：提问数/采纳数/驳回数口径齐全；每条裁决有落点（修订到  
SRS 哪节哪条）；
- RTM：双向挂接无孤儿；版本随关升版（v2→v3→v4→v5 有沿革）。

### **设计类（设计与计划类样例文档（设计文档集、OpenAPI 契约、ADR、 plan/tasks））**

- 概要设计：技术/数据/部署三架构视图齐；非功能设计逐项挂质量特性标准；
- 详细设计：模块到函数级；状态机、错误模型与代码实测行为一致（走查核  
对，不是凭印象）；
- 数据库设计：概念/逻辑/物理三层齐；ER 图与物理表命名一致（澜图组 D-02  
失分点）；
- 接口设计+OpenAPI：端点数与实现一致；错误码段位随变更同步（澜图组 D-  
03 失分点）；ADR 含被否决方案与理由。

### **测试与质量类（测试与质量类样例文档（《测试计划》《测试报告》《质 量特性自评表》《故障演练验收记录》《缺陷看板与修复记录》））**

- 测试计划/设计/报告：GB/T 15532 结构；首轮与回归分行；每个数字可溯源  
（CI 工件号、执行记录）；



- 压测报告：口径三层下钻（怎么算/样本多少/原始日志在哪）；容量建议三要素；
- 自评表：八大特性逐项挂证据编号；部分符合项如实标注（全绿反而引发全面复核）；
- 缺陷看板：状态机闭环；关单权记录；缺陷密度与环节分布有分析。

### **手册类（用户手册/部署运维手册）**

- 用户手册：按用户画像与任务组织（不是按菜单罗列）；截图与当前版本 UI 一致（澜图组 D-05 失分点：变更后截图未更新）；术语表与 SRS 口径一致；
- 部署运维手册：前置条件全部写版本号（澜图组 D-04 失分点）；每步命令可复制执行；含健康检查、数据持久、离线起、常见故障处置（操作卡引用）。

### **机器可读集**

- constitution/AGENTS：规则与实际执行一致（抽查两条规则的遵守证据）；
- spec/plan/tasks：三链一致（spec 改→plan 改→tasks 改的变更链完整）；
- Prompt 模式库/翻车记录：模式有使用记录，翻车有五问根因——零翻车等于没深度使用或没审视。

### **自查的执行组织**

自查不要全组一起过（互相带节奏），建议：每人认领一类文档当『陌生审计员』，交叉审（写文档的人不审自己写的）；每条体检项三种结论：通过/已整改/约定说明；自查记录本身归档（证明你自查过）。澜图组的安排：丁审需求与设计、丙审测试、乙审手册、甲审投标计划与机器可读集，一天完成。

### **本节清单**

- 六类文档体检清单逐项过，结论留痕；
- 交叉审查（不审自己写的）执行；
- 发现问题的整改 commit 链归档；
- 历史基线不回改，口径差异用新文档说明。

## **8.3 一致性核查与可执行性核查**

一致性是四维里最难自查的维度——它的问题不在任何一份文档内部，而在文档之间。《文档审计报告》的三个一致性失分（D-02 表命名、D-03 错误码、D-07 公式符号）揭示了一个共性模式：一致性缺陷绝大多数是变更的次生滞后。本节给出一套可操作的漂移猎杀方法。

### 8.3.1 漂移的三类高发区

- 数值漂移：阈值（0.30/0.24）、指标门槛（80%/85%）、性能预算（3s/10min）、版本号、端点数（6）——同一数值出现在多份文档，改了一处漏了他处；
- 编号漂移：需求编号（V1.0 旧编号 vs 招标编号）、缺陷编号、文档编号（几号文）——尤其在有版本沿革的文档之间；
- 术语与日期漂移：同一事物的叫法（『冻结影像集』vs『验收影像集』）、签署日期、版本日期——日期错一位，证据链的可信度就塌一角。

### 8.3.2 猎杀方法：关键词全库扫描

人工一致性核查的可行方法是『以数值为锚』：列出 20~30 个关键锚点（全部阈值、指标、编号、日期、版本号），对每个锚点做全文档库检索，逐处比对。澜图组的锚点清单（节选）：0.24/0.30（CLI 阈值与变更前值）、0.5（REST 阈值）、84.2%/88.6%（精度实测）、2.6s/3s（PE-01）、7'12"/10min（PE-02）、6（端点数）、24/16/3（需求数）、50/122（设计用例数与自动化测试函数数）、18/47（看板缺陷数与累计缺陷数）、78.4%/70%（覆盖率实测与门禁）、D4（确认书签署日）。每个锚点的全部出现处列成表，口径不一致处即缺陷。

### 8.3.3 变更次生滞后的根治：影响面清单加文档列

D-03/05/07 的共同根源：CR-001 的影响面清单列了 spec/SRS/plan/tasks/代码/测试，没列『还有哪些文档引用了这个阈值』。根治方法：变更单模板加一行『文档影响面』——强制列出接口文档、用户手册、部署手册、指导材料中所有引用点。此后每次变更，文档同步成为流程内动作而不是审计兜底。

### 8.3.4 人机协同要点（AI 的主场之一）

- 锚点扫描是 AI 的强项：把文档库文本喂给 AI，『找出所有提到阈值的地方并列出取值』——AI 做跨文档数值比对又快又细，《文档审计报告》的 D-07 类问题用这个方法 10 分钟能检出；
- 让 AI 生成锚点清单初稿（『从 SRS 提取所有数值型验收标准』），人核对补全；
- 判断『两处表述是否真矛盾』要人来——AI 会把『交互分割 P95 2.6s（并发 2 档）』和『交互分割 14.8s（降级态）』报成矛盾，其实口径不同。一致性核查的难点从来不是找差异，是判定差异是否缺陷。

### 8.3.5 本节清单

- 锚点清单（20~30 个关键数值/编号/日期）建立并全库扫描；
- 每个锚点的出现处成表，口径一致或差异有解释；
- 变更单模板已加『文档影响面』行；
- AI 扫描结果经人工判定（差异≠缺陷的口径判断留痕）。

### 8.3.6 可执行性核查：只认实测，不认纸面

可执行性维度的判定规则最硬：部署手册由助教在干净环境实测，用户手册对照实际系统走查。纸面审查看不出来的问题（缺前置条件、版本号没写死、步骤顺序错）一跑就现形。本节讲如何在审计前自己做足实测，以及两份手册各自的可执行性要点。

#### **部署手册的自测规程**

部署手册的可执行性只有一种验证法：让没写过它的人，在干净环境里，只用它和交付包，从头跑一遍。规程要点：

- 执行人：组内未参与写手册的成员（模拟助教），或邻组互换（更接近真实）；
- 环境：干净虚拟机快照（只装 Docker），不允许执行人问问题——卡在哪就记哪，那就是文档缺口；
- 记录：逐步计时 + 卡点记录 + 成功标准（健康绿/冒烟通/持久化/离线起，见 8.6 节）；

- 闭环：每个卡点转化为手册修订，修订后重跑验证。澜图组三轮自测：  
1h52m→1h37m→1h21m，缺口 4 项逐项修复——耗时下降曲线就是可执行性改善的量化证据。

## 高发缺口清单（历年+样例汇总）

- 前置条件不写版本号（D-04：nvidia-container-toolkit 没写版本——『装最新版』在甲方机房就是事故）；
- .env.example 与 compose 引用漂移（全班 2 组因此跑不通被预扣分——CI 加 diff 检查可根治）；
- 初始化顺序依赖未编排（数据库迁移先于 API 就绪的 healthcheck 依赖）；
- 镜像内残留宿主绝对路径（开发机路径写进配置）；
- 缺运维动作（日志卷清理、备份恢复——《可移植性批量验证记录》G-02）。

## 用户手册的走查规程

用户手册的可执行性验证是『任务式走查』：让不熟悉系统的人（邻组同学最佳）只拿手册完成 3 个核心任务，观察员记录卡点。要点：手册要按任务组织（『如何完成一次受灾农田提取』）而非按菜单组织（『3.1 文件菜单 3.2 编辑菜单』）；截图必须与当前版本 UI 一致——D-05 的教训是变更（CR-001 阈值文案）后截图没更新，审计员拿手册对系统立刻发现货不对板。截图策略：所有截图标注版本号，变更后重截进文档同步清单。

## 人机协同要点

- AI 陌生人模拟：『假设你只拿到这份手册和交付包，逐步推演部署，指出你会卡在哪』——能抓到一部分缺口（《可移植性批量验证记录》G-01 的注释过时类），但抓不住环境真实问题（版本号、路径、时序），实测不可替代；
- 让 AI 对手册做『步骤可执行性』检查：每步是否有明确动作、预期结果、失败处置三要素；
- AI 生成部署手册初稿（从 compose 文件与配置反向生成），人逐步实测修订——这个方向（AI 起草→实测修订）比人写初稿快一倍。

## 本节清单

- 部署手册经非执笔人在干净环境实测  $\geq 1$  轮，卡点全闭环；
- 高发缺口五项逐项排查（版本号/env 漂移/初始化顺序/宿主路径/运维动作）；
- 用户手册经外人任务式走查  $\geq 3$  任务；
- 截图标版本号，变更后重截进文档同步清单。

### 8.4 AI 审计：预审三遍法与三单对照

文档包几十份、几百页，人工审计一遍要一整天——但审计的到来不等你。正确姿势：在独立审计之前，让 AI 按三种角色把文档包先审三遍，把大部分问题消灭在自查阶段。本节给出三套成形的提示词模式与人工复核纪律。

#### 8.4.1 三遍 AI 预审的设计

| 遍次  | AI 角色 | 提示词要点                               | 检出目标                     |
|-----|-------|-------------------------------------|--------------------------|
| 第一遍 | 结构审查员 | 对照 GB/T 8567/9385/15532 章节框架，逐份指出缺项 | 结构完整性问题（缺章节/缺沿革/缺签署页）    |
| 第二遍 | 数据核查员 | 列出文中全部数值主张，逐一追问出处；找出无量词套话           | 内容真实性问题（无出处数字/『性能良好』类表述） |
| 第三遍 | 口径比对员 | 跨文档比对锚点数值/编号/日期（锚点清单见 8.3 节）        | 一致性问题（阈值/编号/术语漂移）        |

三遍分开跑而不是合并成一句『帮我审一下』——单一提示词下 AI 的注意力会稀释，分角色后每遍的检出深度显著提升。这与 8.3 节的锚点法、8.2 节的四维口径是同一套思想：把大问题拆成判定标准明确的小问题，AI 才靠得住。

#### 8.4.2 人工复核纪律：AI 报的每个问题都要裁决

AI 预审的产出是候选问题清单，不是结论。复核三种裁决：

- 确认是真问题：进整改清单（澜图组用这个方法自查掉了 5 个问题，否则《文档审计报告》的问题数会是 12 而不是 7）；
- 误报：AI 没读到上下文（如把投标基线 24 周工期报成『与课程 18 个工作日矛盾』——那是双轨设计，记为约定项即可）；

- 口径分歧：AI 认为是问题、人认为不是（或反之）——写下来由组长裁决，裁决理由留痕。

纪律红线：不许把 AI 预审报告原样当自查报告提交——审计组一眼能看出（问题描述没有人味、没有出处页码、误报未剔除）。AI 预审的价值是降低漏检率，自查报告的价值是证明你思考过。

### 8.4.3 反向用法：让 AI 给自己的审计整改做复核

整改完成后，把『问题—整改—证据』三件套喂给 AI 做复核：『这个问题这样整改是否闭环？还差什么？』它能抓出两类常见半拉子整改：改了正文没改附录（D-07 类）、改了文档没改引用它的另一份文档（次生滞后的滞后）。当然最终复核拍板仍在人。

### 8.4.4 三个实测有效的提示词骨架

- 结构审查：『你是文档审计员。对照【标准章节框架】审查以下文档目录与章节标题，列出：1) 缺失的必备章节；2) 命名与标准不符之处；3) 签署页/版本沿革是否存在。只列问题，不给修改建议。』
- 数据核查：『列出以下文档中全部数值主张（指标、日期、数量、百分比），对每条标注：有出处/无出处/套话（无量词修饰词）。输出表格。』
- 口径比对：『以下是 N 份文档中出现【某锚点】的全部段落，逐段摘录并指出取值是否一致；对不一致处，摘录原文供人工裁决。不要自己判断哪个对。』——最后一句关键：让 AI 举证，人来裁决。

### 8.4.5 本节清单

- 三遍 AI 预审（结构/数据/口径）跑完，候选问题清单在案；
- 每个候选问题有裁决（确认/误报/分歧）与理由；
- 整改三件套（问题—整改—证据）经 AI 复核 + 人工拍板；
- 自查报告为人工撰写，AI 产出仅作输入。

### 8.4.6 AI 使用审计：三单对照怎么过关

《AI 使用审计报告》记录了澜图组的审计实况：全周期 38 个 PR，抽审 4 个（#36/#47/#52/#76），3 个合规、1 个不符合（PR#36 扣 0.5 分通报）。审计不是找茬，

是验证『AI 使用可追溯、可解释』。本节拆解审计方法（三单对照、时序交叉、后补检测），以及澜图组 3 个合规 PR 是怎么轻松过的、PR#36 是怎么栽的、亮点 PR#76 是怎么加分的。

**审计方法论：三单对照**

| 单             | 来源                 | 证明什么                  |
|---------------|--------------------|-----------------------|
| Prompt 日志     | logs/prompts/ 按日归档 | AI 参与了什么、怎么参与的、产出如何处置 |
| commit/ PR 记录 | Git 历史             | 代码实际变更的时间、作者、内容       |
| 代码本体          | PR diff 与仓库现状      | AI 生成痕迹与人工修改痕迹的真实比例   |

三单互相对照：日志说有 AI 参与 → commit 时间窗对得上吗 → diff 里有相应的生成痕迹吗；反过来，diff 有明显的 AI 风格（大段规整代码+完整注释） → 日志里记了吗 → 没记就是隐瞒。三单能对上，合规；对不上，深入问。

**时序交叉与后补检测**

审计的两个技术动作。时序交叉：把日志时间戳、commit 时间戳、PR 评审时间线摆到一条轴上看顺序是否合理——正常顺序是『日志（生成）→ commit（提交）→ 评审记录（打回/通过）→ 日志回填 PR 号』；出现『日志时间晚于 PR 合并时间且非回填字段』即异常。后补检测：补录日志的指纹——时间戳格式与同期日志不一致、笔迹风格突变、对『产出处置』的描述过于圆满（真实日志常有弃用和大改，后补的倾向于写成采纳）。PR#36 正是被时序交叉发现的：D11 夜班 commit 有 AI 痕迹，当日日志无条目，D14 补录未打标。

**案例一：PR#36，0.5 分的全部经过**

PR#36 修复 BUG-136（文本提示长度超 4096 报错）。丙在 D4 凌晨 1 点用 AI 生成修复方案并采纳，当日未写日志，次日值班员检查时该改动尚未提 PR（无 commit 对照物）而漏检，D14 丙凭记忆补写了日志但没打【补录】标。审计后果：扣 0.5 分 + 通报。深层教训三条：凌晨使用 AI 最容易漏记（疲惫+急于收工）；值班员检查的触点不能只挂在 PR 上（要挂 commit）；补录不打标等于把『疏忽』升级为『失信』

——0.5 分扣的是后者不是前者。

## 案例二：PR#76，合规PR的加分姿势

PR#76 是 6 个端点的权限矩阵改造，审计标注为亮点。它做对了什么：日志完整记录了『AI 修点（生成 6 个端点的装饰器代码）、人修面（人工审查权限矩阵表，发现 3 个端点的角色映射与 SRS FR-02.3 矛盾，修正后采纳）』；PR 描述里直接引用日志行号与 spec 条款；评审记录显示队友打回一次（要求补 2 个端点的测试），二次通过。审计员能顺着 PR 看到完整的人机协作闭环——AI 干了体力活（6 个端点样板代码），人干了判断活（权限矩阵的业务正确性），评审干了把关活。这就是『AI 使用可追溯、可解释、有把关』的样板。

## 应对审计的日常姿势（不是迎检姿势）

- PR 描述模板内置『AI 使用』小节：用了吗/日志在哪/采纳还是改采/为什么——写 PR 时顺手填，审计时零准备；
- 答辩预演时练『抽丁讲 PR』：随便一个成员随便抽一个 PR 能讲清 AI 角色（8.9 节详述）；
- 对审计发现的问题态度：认、改、防——PR#36 后澜图组当日就把值班员检查点从 PR 挂到 commit，这是审计后的正确动作；
- 绝不临时『美化』日志：审计员的时序交叉专打美化，一次失信全程存疑。

## 人机协同要点

- 审计前的自查可以让 AI 跑：给它日志目录与 git log，让它做三单对照初查、列异常清单——人再逐条复核（AI 查得到的审计员也查得到，这是好事）；
- 时序交叉图（甘特式时间轴）AI 生成，用于答辩展示你们的可追溯性；
- 红线：任何『让 AI 帮忙把日志补齐得像原始的』的想法，都是本课程最重违规的入口。

## 本节清单

- 三单对照自查通过，异常条目有人工复核结论；
- PR 描述含 AI 使用小节，引用日志行号；
- 至少一个亮点案例（人机分工清晰的 PR）可在答辩展示；



- 审计发现问题后有『认改防』记录。

## 8.5 移交态整理与交付包工程

文档审计的隐性考点是『移交态』——文档包不仅是写完了，还要处于可正式移交的管理状态：版本有沿革、签署有位置、归档有结构。本节讲这三件事怎么做。

### 8.5.1 版本沿革：每份文档的履历表

每份正式文档必须有版本沿革表（版本号/日期/修改人/修改内容/依据）。审计员通过沿革表重建文档的生命周期：SRS 的 V1.0→V2.0（AI 反澄清 12 条修订）→V3.0（设计评审重锚）→V3.1（CR-001 阈值变更），每一跳都有变更单或澄清记录支撑。常见问题：只有最新版没有沿革（说不清怎么来的）；沿革写着『细节优化』（套话，不真实）；变更无依据（哪条 CR？哪次会议？）。沿革表的每一行都应该能回答『这版为什么存在』。

### 8.5.2 签署页：责任的物理形态

需要签署的文档（需求确认书、验收报告、等保核查表、移交清单）必须预留签署页：角色、签署意见栏、签署人、日期。教学环境中签署是仿真，但纪律是真的：先定稿后签署（签署后文档冻结，再改走升版）；签署意见允许写条件（『同意验收，遗留事项按 3.1 约定执行』——《验收报告》甲方签署的原话）；电子签署留痕（在配置库记录签署事件）。验收报告没有签署页等于没验收。

### 8.5.3 配置库归档结构

全部交付物按配置库结构归档（澜图组结构）：

- 01\_立项/：投标书、需求确认书、澄清记录；
- 02\_需求/：SRS 各版本、spec.md、反澄清记录、RTM 各版；
- 03\_设计/：设计文档集、OpenAPI、ADR、plan/tasks；
- 04\_研发/：代码仓 tag、PR 记录、Prompt 日志、模式库、翻车记录；
- 05\_测试/：测试计划/设计/报告、压测、缺陷看板、自评表；
- 06\_验收/：审计报告（42/43/44）、UAT 记录、验收报告（47）、复盘报告（46）、移交清单。

归档的两个纪律：只读化（归档目录设只读，再改走升版）；索引化（根目录一

份 INDEX.md，列出每件交付物的位置、版本、哈希——审计员和移交接收方的入口）。

### 8.5.4 人机协同要点

- INDEX.md 由 AI 从目录结构自动生成（含哈希），人核对版本号——哈希计算交给脚本，不留手工错误；
- 让 AI 检查版本沿革表的『依据列』空项（哪次修订没写依据）；
- 签署页与归档结构有模板（附录 A），AI 按模板批量生成骨架，人填实质内容。

### 8.5.5 本节清单

- 每份正式文档有版本沿革表，每行有依据；
- 需签署文档均有签署页，签署事件留痕；
- 配置库六目录归档 + 只读化 + INDEX.md 索引（含哈希）；
- 抽查：凭 INDEX.md 30 秒内能取出任意一件交付物。

### 8.5.6 交付包工程：包装也是产品

可移植性验证从助教拆开你的交付包那一刻开始打分。《可移植性批量验证记录》中澜图组的交付包有 9 项清单、SHA-256 哈希、preflight 脚本——这些『包装』决定了验收第一印象，也直接决定部署能不能一键跑通。本节讲交付包怎么工程化。

#### 交付包标准内容（澜图组 9 项）

| # | 交付项                         | 作用    | 工程要点                                                 |
|---|-----------------------------|-------|------------------------------------------------------|
| 1 | manifest.txt + SHA-256 哈希清单 | 完整性自证 | 生成脚本化（sha256sum 全目录），验收时逐文件比对                        |
| 2 | docker-compose.yml          | 一键编排  | 依赖编排正确（迁移先于 API）；引用变量与 .env.example 集合一致（CI diff 检查） |
| 3 | 镜像（tar 包或 registry）         | 运行实体  | docker save 打包含版本 tag；镜像内无宿主路径（CI 自检）                |
| 4 | .env.example                | 配置模板  | 全变量注释；与 compose diff 为                               |

| # | 交付项                  | 作用    | 工程要点                       |
|---|----------------------|-------|----------------------------|
|   |                      |       | 空；敏感项留空不留真值                |
| 5 | 部署运维手册               | 操作依据  | 前置条件全写版本号；步骤可复制执行（见 8.7 节） |
| 6 | 冒烟脚本 smoke.sh + 样例数据 | 快速验证  | 部署后 10 分钟内能跑通主流程           |
| 7 | seed 数据与迁移脚本         | 数据初始化 | 随首次启动自动执行，幂等               |
| 8 | README（十分钟指引）        | 第一印象  | 一页纸：是什么/怎么起/怎么验证/出问题看哪     |
| 9 | preflight.sh 前置检查    | 环境自检  | GPU 可见性、磁盘空间、端口占用，失败给可读提示  |

### 哈希清单：完整性的自证

哈希清单的意义在验收场景才会显现：助教第一步就是逐文件 sha256sum 比对——交付包在传输/拷贝中有没有损坏、被篡改，一目了然。生成必须脚本化（find + sha256sum > MANIFEST.sha256），手工列哈希必错。教学点：这是把『信任』变成『验证』的最小例子——你说包是完整的，我不信，我算。

### preflight 脚本：把环境问题挡在部署前

部署失败的头号原因不是包的问题，是环境不满足前置（GPU 驱动没装、磁盘不够、端口被占）。preflight.sh 在 compose up 之前检查这些并给出可读提示（『nvidia-smi 在容器内不可见：请安装 nvidia-container-toolkit≥1.14，见手册 2.1 节』）。澜图组《可移植性批量验证记录》记录：助教执行 preflight 5 分钟通过，无任何歧义——这 5 分钟的顺畅是脚本替手册值班的结果。写 preflight 的原则：每个检查项失败时给出的提示必须包含『装什么版本、看手册哪节』。

### 人机协同要点

- manifest/哈希/preflight 脚本全部适合 AI 起草——机械的 shell 脚本是它的强项，但必须在真实干净环境验证过才算数（AI 写的命令有版本差异）；
- 让 AI 从 compose 与代码反向生成 README 十分钟指引初稿，人按实测修订；

- .env.example 与 compose 的 diff 检查脚本 AI 生成、进 CI——全班 2 组跑不通就是这个检查缺失。

## 本节清单

- 交付包 9 项齐套，manifest+哈希脚本化生成；
- preflight 覆盖 GPU/磁盘/端口，失败提示含版本号与手册章节；
- .env 与 compose diff 检查进 CI；
- README 一页纸经外人实测（十分钟跑通）。

## 8.6 干净环境自测：在助教之前先跑一次

《可移植性批量验证记录》最硬的事实是：澜图组在交给助教之前，自己在干净环境里跑了三轮——第一轮 1h52m、修 4 个缺口，第二轮 1h37m，第三轮 1h21m 零修复。助教那次 1h12m 的顺畅不是运气，是三轮自测把坑全踩完了。本节讲干净环境自测怎么做才有效——不是『在自己机器上再跑一遍』，那没有意义。

### 8.6.1 什么是真正的『干净环境』

干净环境的核心是消除『开发机惯性』：你机器上装了驱动、配过 hosts、放过数据集、改过端口，这些东西让系统在你机器上『恰好能跑』，换台机器全露馅。干净环境三要素：

- 操作系统全新：用虚拟机快照或全新容器主机。澜图组用 Ubuntu 22.04 最小安装镜像做基础快照 clean-u2204-d24-r3，只装 Docker + nvidia-container-toolkit，其余一概不装；
- 网络按约定受限：验收现场通常允许内网，但你的包不能假设能访问外网拉镜像——镜像必须随包交付或预置 registry。第三轮自测专门验证『断外网能否起』；
- 账号非开发者：用新建的普通用户执行部署，不用你自己的账号（你的 .bashrc、ssh key、环境变量都是污染）。

### 8.6.2 快照工作法：可重复的自测

| 步骤 | 操作        | 要点                                                |
|----|-----------|---------------------------------------------------|
| 1  | 做基础快照     | 装完 Docker 与显卡工具链即拍快照，命名含日期与轮次（clean-u2204-d24-r3） |
| 2  | 每轮自测从快照还原 | 还原到基础快照再拷入交付包，保证每轮起点一致                            |
| 3  | 执行部署并全程计时 | 从解压到冒烟通过分段计时，形成收敛曲线                               |
| 4  | 记录每一个卡点   | 卡在哪条命令、报错原文、怎么解决的——缺口清单的来源                        |
| 5  | 修完还原再测    | 修复必须回到快照重测，不允许『热修后继续』（热修掩盖新坑）                     |

### 8.6.3 五步验收点：自测的检查骨架

每轮自测按五个验收点打钩，与助教验收同构：

- 一键起：一条命令（docker compose up -d）拉起全部服务，无人工干预步骤；
- 健康绿：全部容器 healthy，/healthz 返回 200，preflight 全过；
- 冒烟通：smoke.sh 跑通主流程（上传影像→分割→导出），10 分钟内；
- 持久化：重启容器后数据仍在（卷挂载正确），重启后服务自恢复；
- 离线起：断开外网，从快照还原后仍能一键起（镜像不依赖外网拉取）。

### 8.6.4 澜图组三轮收敛：4→2→0 的缺口曲线

| 轮次    | 耗时    | 新发现缺口 | 说明                                                                            |
|-------|-------|-------|-------------------------------------------------------------------------------|
| 第 1 轮 | 1h52m | 4 个   | G-01 显存注释过时（手册写 8GB 实测需 12GB）；G-02 日志卷无清理策略；另 2 项：seed 脚本非幂等、smoke 脚本路径依赖 cwd |
| 第 2 轮 | 1h37m | 2 个   | 修复引入的新问题：seed 幂等改造后首启变慢（迁移与 seed 串行）；smoke 脚本                                 |

| 轮次    | 耗时    | 新发现缺口 | 说明                       |
|-------|-------|-------|--------------------------|
|       |       |       | 在小分辨率样例上超时阈值过紧           |
| 第 3 轮 | 1h21m | 0 个   | 全流程顺畅，耗时收敛到接近助教实测的 1h12m |

教学点：缺口数 4→2→0 与耗时 1h52m→1h21m 双收敛，是可展示给评委的过程证据——证明你们的可移植性不是『碰巧能跑』，是迭代修出来的。第 2 轮冒出 2 个新缺口是常态：修复本身会引入回归，这正是必须整轮重测而不能热修的原因。

### 8.6.5 全班 6 组总表的警示：跑不通的 2 组栽在哪

《可移植性批量验证记录》附的全班汇总：6 组中 2 组在助教机器上跑不通被预扣分。两组的根因——一组 .env 漂移（开发期改了 .env 没同步 .env.example，助教按 example 配置缺变量）；一组缺初始化脚本（数据库迁移靠开发者手工执行过，交付包里没有）。这两个坑的共性：都是『交付包内容与开发实况脱节』，都在自测中必然暴露——前提是自测真的用了干净环境。这两组都跳过了自测环节。

### 8.6.6 人机协同要点

- 让 AI 扮演『苛刻的验收助教』审查部署手册：给它手册全文，要求逐步模拟执行并报告每个歧义点（版本号缺失、路径假设、权限假设）；
- 计时与缺口记录用结构化模板，AI 可帮助整理成收敛曲线图（matplotlib），但缺口判定与修复决策必须人工；
- 红线：AI 不能替你在干净环境执行——自测的全部价值在于真实执行，任何『AI 分析说应该没问题』都不能替代一轮真实还原测试。

### 8.6.7 本节清单

- 有基础快照，命名规范，每轮自测从快照还原；
- 五步验收点逐轮打钩，记录耗时与缺口；
- 至少完成三轮自测或零缺口一轮收口；
- 缺口全部闭环且重测通过，收敛数据可展示。

## 8.7 部署运维手册与用户手册

部署运维手册的读者不是你们组的人，是助教、是甲方运维、是三个月后接手的

新人——他们不知道你的项目结构、没见过你的代码、机器是全新的。手册质量的唯一判据：一个陌生人仅凭手册能否在合理时间内把系统跑起来并守住它。澜图组的做法是『非执笔人实测』：写手册的人不许执行，执行的必须是没有参与写的人。

### 8.7.1 章节结构与写作要点

| 章节         | 内容                | 写作要点                | 高发错误                         |
|------------|-------------------|---------------------|------------------------------|
| 1 系统概述     | 架构图、组件清单、端口表      | 一张部署架构图+组件与端口对照表    | 图画得漂亮但端口表缺失                  |
| 2 前置条件     | 硬件/OS/Docker/驱动版本 | 全部给具体版本号与检查命令       | 『需要 NVIDIA 显卡』这种废话——给型号与显存下限 |
| 3 安装部署     | 分步命令序列            | 每条命令可直接复制执行，注明预期输出  | 路径假设（cd 到哪不说）、权限假设（要不要 sudo） |
| 4 初始化与种子数据 | 迁移、seed、管理员账号     | 说明执行时机与幂等性          | 漏掉初始化（全班 1 组栽在这）             |
| 5 验证与冒烟    | 健康检查、冒烟脚本         | 给出『成功长什么样』的判定标准     | 只说『访问首页看看』                   |
| 6 日常运维     | 启停、备份、日志、清理       | 日志卷清理策略必须有（G-02 教训） | 只有部署没有运维                     |
| 7 故障处置     | 常见故障→症状→定位→恢复     | 对应故障演练场景（杀进程/断网）    | 写成『请联系开发』                    |
| 8 附录       | 配置项全表、环境变量说明      | 与 .env.example 一一对应 | 配置项散落各处无总表                   |

### 8.7.2 命令的写作标准：可复制、可预期、可回退

三条硬标准：可复制——命令块里不许有『根据你的环境修改』这种话，必须给具体值或明确的取值方法；可预期——关键命令后给预期输出（『应看到 4 个容器状态为 healthy』），让执行者知道对错；可回退——破坏性操作（清库、删卷）前给出备份命令。澜图组手册中 47 条命令全部经非执笔人原样复制执行验证，这是《可移植性批量验证记录》1h12m 顺畅通过的直接原因。

### 8.7.3 前置条件：G-01 显存注释的教训

《可移植性批量验证记录》缺口 G-01：手册写『GPU 显存  $\geq 8\text{GB}$ 』，实测大地块分割峰值占用 11.2GB，助教机 12GB 恰好够用——若按手册配 8GB 机器直接 OOM。根因：手册的显存值抄自模型卡说明（理论值），不是实测值。教训上升为规则：所有资源指标（显存/内存/磁盘）必须写实测峰值  $\times 1.2$  余量，并注明测量场景（『1 万平方公里地块、2048 $\times$ 2048 切片』）。让 AI 从压测报告（PE-01/PE-02 场景）提取资源数据填入手册，比拍脑袋抄文档可靠。

### 8.7.4 故障处置章：对着演练场景写

第 7 章故障处置直接对应质量关的三次故障演练（杀 worker 进程 6'12"、断网 8'47"、磁盘写满 11'55"）。每个演练场景沉淀为一条处置卡：症状（什么指标异常/什么报错）→ 定位路径（看哪个日志哪个指标）→ 恢复步骤（具体命令）→ 根因与预防。写这章时回放演练记录，把当时的排查过程文档化——你们在演练里 15 分钟内定位的本事，要变成读者也能 15 分钟定位的文字。

### 8.7.5 非执笔人实测规程

- 执笔人交付手册初稿后即『封手』，测试由组内未参与编写的成员在干净快照上执行；
- 执行者逐条复制命令，任何一处需要『想一下』『问一句』的地方记为缺陷；
- 缺陷分级：阻断（跑不下去）当日必修；歧义（能跑但要猜）次日修；建议类进 backlog；
- 修订后换另一名非执笔人复测——同一人的第二次执行已经『不陌生了』。

### 8.7.6 人机协同要点

- AI 从 compose/脚本/压测报告反推手册骨架效率很高，特别是配置项全表（扫代码里的环境变量读取）；
- 让 AI 做『模拟执行』审查：提示词骨架——『你是从未见过本项目的运维，在全新 Ubuntu 22.04 上仅有 Docker。逐步执行以下手册，报告每一步的歧义、缺失与矛盾，不给通过/不通过，只列问题清单』；
- AI 生成的命令必须在真实环境逐条验证——AI 会编造看似合理的参数（如 docker compose 的版本相关语法）；



- 实测数据的填入手册（显存 11.2GB、部署 1h21m）必须来自真实记录，禁止 AI 估值。

### 8.7.7 本节清单

- 八章结构齐套，前置条件全带版本号与检查命令；
- 全部命令经非执笔人复制执行验证，关键命令有预期输出；
- 资源指标为实测值并注明测量场景；
- 故障处置卡覆盖三次演练场景，含定位路径与恢复命令。

### 8.7.8 用户手册：按任务组织，不按菜单组织

用户手册最常见的失败是『菜单说明书』——逐菜单逐项罗列功能，读者看完知道每个按钮在哪，仍然不会干活。澜图组的用户手册按『用户要完成的任务』组织：应急场景下 30 分钟出受灾图斑、批量地块季度监测、分割结果人工修正与导出。本节讲任务式手册的写法。

### 从用户画像到任务清单

手册写作的起点是回到需求关的用户画像：澜图平台的画像有应急值班员（急、非 GIS 专业、要结果不要参数）、监测工程师（批量、定期、要质量指标）、外业核查员（移动端、要导出交接）。每个画像列出 3~5 个高频任务，手册正文 = 任务数 × 操作卡。画像不明的组，手册注定写成菜单罗列——因为除了菜单你没有别的组织维度。

### 操作卡模板

| 要素   | 内容           | 澜图示例（应急受灾提取）                          |
|------|--------------|---------------------------------------|
| 任务目标 | 一句话说清做完得到什么  | 30 分钟内从 Sentinel-2 影像得到受灾农田矢量图斑与面积统计  |
| 前置条件 | 数据、权限、环境     | 已登录、影像已上传至项目、GPU 服务健康                 |
| 步骤序列 | 编号步骤+截图+预期画面 | 7 步：新建任务→选影像→选全自动分割→调置信度阈值→执行→交互修正→导出 |
| 质量自检 | 怎么确认做对了      | 图斑覆盖率目测合理；面积统计与同期上报量级一致；导出文件能在外业软件打开  |

| 要素   | 内容      | 澜图示例（应急受灾提取）                         |
|------|---------|--------------------------------------|
| 常见卡壳 | 本任务特有的坑 | 影像云量>20% 时结果不可靠（手册明示弃用）；大图建议切片参数保持默认 |

## 截图纪律：真数据、真流程、可复现

截图是用户手册的信誉所在。纪律：截图必须来自真实操作流程（不是设计稿、不是 P 图）；截图所用数据与验收演示数据同源（评审对照手册复现操作时画面能对上）；关键界面截图带步骤编号标注。澜图组手册 41 张截图全部在冻结后的 V3.1 系统上重新截取——UI 在 CR-001 后有过微调，旧截图全部作废重截。这是文档冻结纪律的一部分：系统变，手册图必须跟着变。

## 任务式走查：手册的验收方式

手册写完后做『任务式走查』：找一名未参与开发的人（跨组交换最佳），给他画像和任务目标，只给手册，看能否独立完成。记录：每个任务的完成时间、卡亮点、求助次数。澜图组走查结果：应急任务 34 分钟完成（目标 30 分钟，差 4 分钟原因是阈值调整章节位置太靠后，手册修订把该节前置）。走查发现的问题不是测试问题，是手册问题——人对，手册让人卡住了。

## 人机协同要点

- AI 极擅长把『功能清单』改写为『任务式结构』：给它功能列表与用户画像，让它提出任务清单与操作卡骨架；
- 操作步骤初稿可由 AI 从代码/接口反推，但每一步必须在真实系统上走一遍——AI 不知道你的 UI 实际长什么样；
- 截图无法 AI 生成（UI 必须真实），但截图清单、标注规范、图文交叉检查（文中引用的图号是否存在）适合 AI 批量检查；
- 常见卡壳章的素材来自真实缺陷库（BUG-121~144）与用户反馈，AI 可帮助聚类提炼。

## 本节清单

- 手册按画像×任务组织，无菜单罗列式章节；
- 每个操作卡五要素齐全（目标/前置/步骤/自检/卡壳）；

- 截图全部来自 V3.1 冻结系统实截，与演示数据同源；
- 完成至少一轮任务式走查并闭环修订。

### 8.8 缺口收敛与批量验证应对

8.6 节讲了单组自测方法，本节把视角拉高：在『助教批量验证全班交付包』的制度下，缺口管理是一场倒计时作战。《可移植性批量验证记录》给的两组关键数据——澜图组的 4→2→0 收敛曲线、全班 6 组 2 组跑不通——合起来就是本节的全部论据：缺口收敛有章法，跑不通有共性，都防得住。

#### 8.8.1 缺口的来源学：四类缺口及其占比

| 类型    | 含义       | 澜图三轮实例                     | 预防手段                          |
|-------|----------|----------------------------|-------------------------------|
| 文档性缺口 | 手册与实况不符  | G-01 显存注释过时                | 资源指标一律实测；变更后文档回归扫描            |
| 工程性缺口 | 脚本/配置缺陷  | seed 非幂等、smoke 路径依赖 cwd    | 脚本进 CI；.env 与 compose diff 检查 |
| 环境性缺口 | 对环境的隐性假设 | 日志卷清理缺失（G-02）              | 离线测试、非开发者账号测试                 |
| 回归性缺口 | 修复引入的新坑  | 第 2 轮 seed 变慢、smoke 超时阈值过紧 | 修复后整轮重测，禁止热修继续                |

#### 8.8.2 收敛作战的节奏设计

澜图组把 9/1~9/2 两天划成自测窗口，节奏是：第 1 轮上午跑（预期发现最多）、当天下午修复、第 2 轮次日跑、第 3 轮收口验证。关键纪律：每轮之间必须回到基础快照；修复期间不改任何与缺口无关的东西（防止范围蔓延）；第 3 轮如果再出阻断性缺口，启动应急预案——砍非核心项（如演示数据精简），保一键部署主链路。收敛曲线本身成为验收材料：《可移植性批量验证记录》附了它，评审看到的是工程过程而非一次侥幸。

#### 8.8.3 全班 6 组总表的读法：跑不通的代价结构

全班汇总：6 组中 4 组通过（澜图 1h12m 最快）、2 组跑不通。跑不通两组的代价不只是『预扣分』：助教记录转入验收答辩质询（评委会问为什么跑不通）；修复后

复测占用了该组最后的复盘时间。而两组的根因（.env 漂移、初始化脚本缺失）都是低级错误——不是技术不行，是缺一个 CI 检查、缺一次真实自测。教学点：可移植性验证的淘汰不发生在答辩现场，发生在助教的虚拟机上，发生在你们看不见的 1 小时里。

#### 8.8.4 批量验证日的应对清单

- T-2 日：交付包封存，哈希清单生成，任何改动必须重新出包重新出哈希；
- T-1 日：在自测快照上跑最后一轮，确认零缺口；准备《部署自证表》（五步验收点打钩记录）；
- T 日：保持通讯畅通，助教执行期间能 15 分钟内响应质询（澜图组被问了一次日志卷清理，按手册第 6 章答复即过）；
- T 日：不围观、不干预助教操作——干预即视为手册无效；
- T+1 日：无论通过与否，把助教的执行记录归档进复盘材料（通过的是证据，没通过的是债务）。

#### 8.8.5 人机协同要点

- 缺口登记、收敛曲线绘制、全班总表对比图都是 AI 辅助制图的好场景；
- 让 AI 分析三轮自测记录，识别『修复-回归』模式（哪些修复容易引入新坑），辅助第 3 轮风险评估；
- 应急预案的取舍建议（砍什么保什么）可以让 AI 列选项，但决策在人——砍功能涉及需求确认书约定，必须人工对照确认书裁决。

#### 8.8.6 本节清单

- 缺口按四类登记，每类有预防手段落地；
- 自测节奏含回归轮，修复后必整轮重测；
- 收敛曲线可展示，封存后改动即重出包重出哈希；
- 批量验证日应对五步法已排练，响应人明确。

### 8.9 彩排、验收演示与汇报 PPT

9/2 上午验收，澜图组的 9/1 排得密不透风：上午三审计（文档/部署/AI）并行收尾，下午两轮全流程彩排，晚上设备封存。彩排不是『过一遍 PPT』，是把验收会 20

分钟当作一场演出完整走台——走位、计时、故障预案全部演练。本节讲这两轮彩排怎么做、风险清单怎么管。

### 8.9.1 两轮彩排的分工

| 轮次          | 时间        | 目标             | 规则                                   |
|-------------|-----------|----------------|--------------------------------------|
| 彩排一<br>(走台) | 9/1 14:00 | 流程全走一遍，暴露衔接问题  | 允许中断喊停改流程；计时但不卡时                     |
| 彩排二<br>(实战) | 9/1 16:30 | 全仿真：计时、抽问、故障注入 | 不许中断；教师扮演评委现场抽 PR 走查、提质疑；注入一次故障（拔网线） |

彩排一暴露的问题集中在衔接：演示到价值论证的换场有 40 秒冷场（PPT 翻页人与演示操作人不是同一人，配合没练过）；UAT 表展示顺序与演示动线不一致，评委视角会跳。彩排二注入的拔网线故障：演示系统纯本地部署，无影响——这本身就是设计答案（演示不依赖外网）；但发现备用机热点切换需 50 秒，超 30 秒承诺，改为提前双机同开热备。

### 8.9.2 风险清单与应对卡

| 风险           | 概率 | 影响 | 应对                           |
|--------------|----|----|------------------------------|
| 演示现场崩溃       | 低  | 高  | 热备机 30 秒切换；本地录屏兜底继续讲         |
| 抽中冷门 PR 走查   | 中  | 中  | 名下代码索引全覆盖+互查演练；被抽者 90 秒五句话结构 |
| 评委质疑基线数据     | 中  | 中  | 三重佐证材料备查（访谈记录/文献/实测记录编号随口可引） |
| 计时超支         | 中  | 低  | 内容按 16 分钟排，留 20% 余量；彩排计时取中位数 |
| 设备兼容（投影/翻页笔） | 高  | 低  | 自带转接头与翻页笔，9/1 晚实地试投笔）        |

### 8.9.3 角色分工与口令

20 分钟验收会 4 人全上场：甲主持+价值论证（语速稳），乙演示操作（手稳），

丙答技术质询+代码走查第一候选人，丁记录评委问题+走查第二候选人。关键机制是口令：评委问题超出自己领域时，回『这部分由 X 回答』——不许硬答。彩排练过这个口令，验收时丙被问部署细节、转乙答，评委评价『分工清晰』。丁的记录职责极重要：评委每个问题当场记下，复盘报告与答辩改进全靠这份记录。

#### 8.9.4 验收前夜封存纪律

- 代码冻结：9/1 18:00 后主干只收演示阻断级修复（本次为零）；
- 环境封存：演示机与备份机同版本部署完毕，断网测试通过，不再改动；
- 材料封存：PPT 终版导出 PDF 双备份（U 盘+云），验收报告打印三份预签乙方栏；
- 人的封存：23:00 前收工——凌晨改东西是 PR#36 式事故的高发场景。

#### 8.9.5 人机协同要点

- 风险清单可让 AI 穷举（『列出验收演示可能出问题的 20 个场景』），人按概率×影响筛选建卡；
- 彩排计时数据 AI 汇总成图表，冷场点定位靠录像回看（人工）；
- 扮演评委的质疑预演（AI 追问）在彩排二前做一轮，彩排二由真人教师执行。

#### 8.9.6 本节清单

- 两轮彩排完成，彩排二含抽问与故障注入；
- 风险卡五张以上，每张有具体应对与责任人；
- 角色口令练熟，记录员明确；
- 封存四项 9/1 晚完成，无人熬夜。

#### 8.9.7 验收演示：用 10 分钟讲一个用户故事

《验收报告》记录的验收会 20 分钟里，演示占 10 分钟。澜图组没有放功能串烧，而是演了一个完整场景：『暴雨应急响应——从拿到灾区 Sentinel-2 影像到交出受灾农田图斑与面积统计，30 分钟工作流压缩到 10 分钟演示』。评委记住的不是 6 个端点，是那个动线。本节讲场景化演示怎么设计。

#### **为什么是场景而不是功能清单**

功能串烧的三个死穴：评委看不出功能之间的协作价值；没有业务上下文，技术指标无意义（『分割很快』——多快算快？用来干什么？）；每个功能浅尝辄止，反而暴露边界毛刺。场景化演示天然解决这三点：动线把功能串成价值链，业务压力（应急=限时）给指标赋予了意义，一条走通的主线比十条半拉的支线有说服力。演示设计的第一原则：评委要在 10 分钟里『看到系统被真正使用』，不是『看到系统有很多按钮』。

**澜图动线：暴雨应急 30 分钟压缩到 10 分钟**

| 时间           | 环节    | 演示内容                               | 对应 MVP / 指标                     |
|--------------|-------|------------------------------------|---------------------------------|
| 0'00"-1'00"  | 情境代入  | 暴雨蓝色预警，值班员接到任务：某县受灾农田快速摸底          | 业务价值定调                          |
| 1'00"-2'30"  | 影像接入  | 上传哨兵影像（冻结集外新数据），自动读取元数据            | MVP-1                           |
| 2'30"-5'00"  | 全自动分割 | 一键执行，展示进度与中间结果；小图块 2m41s 出图        | MVP-2（FR-02.1★）；PE-01 指标        |
| 5'00"-7'30"  | 交互修正  | 对误检水域用点提示剔除、框提示补漏掉的田块              | MVP-3（FR-03.1★）、MVP-4（FR-04.1★） |
| 7'30"-9'00"  | 导出交付  | 栅格矢量化导出 Shapefile + 面积统计表          | MVP-5（FR-06.1★）                 |
| 9'00"-10'00" | 收束    | 对照 UAT 表逐项打钩画面，一句话价值总结（45min→6min） | 验收映射                            |

**演示数据纪律：冻结集外的真实影像**

澜图组演示用的是冻结影像集之外的一景新哨兵数据——这是精心选择的信任策略：用冻结集内的数据，评委会怀疑你们过拟合了演示（『这图你们跑过一百遍了吧』）；用全新真实数据，且当着评委的面从上传开始走全流程，等于现场做一次 UAT。风险是真实数据有不确定性（云量、地物复杂度），所以纪律是：提前用 3 景备选影像各跑过完整动线，选表现中上（而非最好）的一景——演示要的是可信，不是完美。选『中上』是反直觉的成熟：表现最完美反而像排练过的样板戏。

**演示的工程保障**

- 双机备份：演示机+热备机同版本同数据，切换 30 秒内；
- 本地录屏兜底：全流程录屏存本地，现场崩溃时切录屏继续讲（《验收报告》附注：澜图组准备了录屏但未用上）；
- 预载状态：影像已上传一半进度这种『作弊』不做——演示诚信与日志诚信同标准；可以预做的是账号登录与项目创建这类与系统能力无关的步骤；
- 时间纪律：彩排三次取中位数，留 20% 余量；10 分钟的演示按 8 分钟内容排。

### 人机协同要点

- 动线设计可以让 AI 做『评委视角』评审：给它动线脚本，问『哪一步你会走神、哪一步你会质疑、哪一步你看不出价值』；
- 演示讲稿初稿 AI 起草，人按口语习惯改写——讲稿要短句、要具体数字（2m41s 而非『不到三分钟』）；
- 红线：演示数据、指标数字必须来自真实运行；让 AI『优化一下演示数据』等同造假。

### 本节清单

- 演示为一个完整业务场景，非功能串烧；
- 动线映射 MVP 五项与关键指标，时长含 20% 余量；
- 演示数据为冻结集外真实数据，备选 ≥3 景实测；
- 双机备份+录屏兜底就位，彩排 ≥3 次。

#### 8.9.8 验收汇报 PPT：16 页讲 20 分钟

样例《澜图遥感影像智能解译平台——验收汇报》共 16 页，深海军蓝 #16283C 主色、铜橙 #C0652B 点缀、米白 #F5F2EA 底。它不是功能罗列，是管理汇报：先给结论，再给证据，最后给承诺。本节以 验收汇报 PPT 样例为样本精讲验收 PPT 的结构逻辑与制作方法。

#### 16 页结构精讲

| 页  | 内容             | 设计意图    |
|----|----------------|---------|
| P1 | 封面：项目名+验收汇报+组名 | 正式感，无特效 |



| 页    | 内容                                                                | 设计意图                   |
|------|-------------------------------------------------------------------|------------------------|
|      | 日期                                                                |                        |
| P2   | 结论页：五项 MVP 全通过+UAT 7 行全钩+部署验证通过                                   | 结论先行——评委 30 秒内知道结果     |
| P3   | 项目全景：一个月历程时间轴（六关）                                                 | 给非技术评委上下文              |
| P4-9 | 演示动线六页（对应 8.9 节动线）                                                | 每页一环节：截图+一句话+对应 MVP/指标 |
| P10  | 质量证据：缺陷 24 清零/覆盖率 78%·93%·91%/IoU 0.78/ 压测 P99                    | 数据墙——全部可溯源到研发与质量类样例文档  |
| P11  | 需求追踪闭环：RTM 摘要（11 条需求全有通过用例）                                       | 管理严谨性的物证               |
| P12  | 价值论证：45min→6min，约 7 倍                                             | 8.11 节方法的一页呈现          |
| P13  | AI 协作量化：起草 58%/保留 44%/复用 94 次/翻车 5 起                              | 诚实披露——含翻车，反而可信         |
| P14  | 技术债务与遗留：DEBT-01~03+处置约定                                           | 不藏缺点（8.10 节原则）         |
| P15  | 团队与分工+致谢                                                          | 人的呈现                   |
| P16  | Q&A 页+备份材料索引（交付审计类样例文档（文档审计、可移植性验证、AI 使用审计、验收汇报 PPT、复盘报告、验收报告）清单） | 告诉评委证据都在哪              |

### 结论式标题：每页标题都是一句话结论

验收汇报 PPT 样例每页标题不写『质量情况』，写『缺陷清零，三级覆盖率全部达标』；不写『价值分析』，写『单地块作业 45 分钟缩至 6 分钟』。评委翻 PPT 只看标题就能 reconstruct 整个论证链。这是管理汇报与课程作业的分水岭：作业式 PPT 的

标题是名词（章节名），汇报式 PPT 的标题是断言（结论）。写法定式：先写页面内容，再把内容压缩成一句不超过 20 字的断言做标题。

## 视觉纪律

- 三色克制：深海军蓝（标题/结构）、铜橙（关键数字/强调，每页 $\leq 2$ 处）、米白（底）——全部页面不超出三色；
- 一页一观点：P10 数据墙看似例外，实际它只有一个观点『质量达标』，数据是证据阵列；
- 数字为大：关键数字（7 倍、0.78、1h12m）字号 $\geq 40\text{pt}$ 铜橙——评委坐三米外也要看得见；
- 截图带边框与来源标注（『系统 V3.1 实测界面，冻结集外影像』）。

## 制作流程：kimi-slides 管线实战

验收汇报 PPT 样例用 kimi-slides 技能管线制作：先写 .pptd 中间稿（每页的结构化描述：版式/标题/要点/图表引用），check 校验无错误后渲染 PPTX，再逐页截图复核。流程要点：pptd 稿即内容稿，内容与版式分离修改；图表（收敛曲线、AI 指标图）用 matplotlib 提前出 PNG 再引用；每轮修改后重新 check+截图复核——验收汇报 PPT 样例的 16 页截图复核抓到过 2 处文字溢出与 1 处图表过期（旧版覆盖率数据），渲染管线让这类错误在交付前暴露。

## 人机协同要点

- pptd 初稿 AI 生成效率高：给它验收报告+结论清单，出 16 页骨架；
- 结论式标题适合 AI 批量改写（『把以下 16 个名词式标题改为断言式，每条 $\leq 20$ 字』），人终审；
- 红线：PPT 中每个数字必须能溯源（交付审计类样例文档（文档审计、可移植性验证、AI 使用审计、验收汇报 PPT、复盘报告、验收报告）编号+章节）——AI 起草时会『合理推测』数字，终稿前逐数核对；图表数据过期是 AI 协作的高发事故，截图复核必须做。

## 本节清单

- 结构含结论页/演示/证据/价值/AI 披露/债务/索引；

- 全部标题为断言式，关键数字大字号克制用色；
- 管线 check 无错，截图复核清零问题；
- 逐数溯源核对通过，无过期图表。

## 8.10 UAT 组织与验收报告

UAT（用户验收测试）的逻辑起点不在第 6 关，在 D4 签《需求确认书》那一刻——验收的全部依据当时就锁死了。《验收报告》的 UAT 表只有 7 行：5 项 MVP 功能 + 11 条需求汇总 + ★款专项，全部通过。表短是因为映射做得早、做得严。本节讲 UAT 怎么组织、验收报告怎么写、遗留事项怎么约定。

### 8.10.1 UAT 用例的来源：确认书逐条映射

| UAT 行        | 来源依据             | 验收方式             | 结果证据               |
|--------------|------------------|------------------|--------------------|
| UAT-1 影像接入   | 确认书 MVP-1        | 上传多源影像（哨兵/高分）    | 元数据自动解析画面          |
| UAT-2 全自动分割  | MVP-2 / FR-02.1★ | 规定地块一键分割         | IoU 0.78 ≥ 阈值 0.75 |
| UAT-3 点框提示交互 | MVP-3 / FR-03.1★ | 正/负点、框提示修正       | 交互响应与结果对比          |
| UAT-4 文本提示   | MVP-4 / FR-04.1★ | 『水体』『农田』文本分割     | 分割结果与响应时间          |
| UAT-5 矢量化导出  | MVP-5 / FR-06.1★ | 导出 Shapefile 与统计 | 外业软件打开验证           |
| UAT-6 需求汇总   | SRS 11 条 MVP 需求  | RTM 全链路抽查        | RTM 闭环表            |
| UAT-7 ★款专项   | 确认书 13 条★款       | 逐条对照演示/报告        | ★款对照表全钩            |

映射的纪律：UAT 每一行都能指出『依据确认书第几条』，不接受没有依据的临时要求——评委现场提新需求（『能不能试试视频数据』），礼貌记录为『遗留事项/二期建议』，不现场承诺。这是确认书作为『范围锚』的实战用法。

### 8.10.2 UAT 执行组织

- 执行人：甲方代表（教师扮演）+ 助教记录，乙方（小组）只操作不解释——系统好不好用，甲方操作时的直觉反应就是评分；
- 环境：助教已验证过的部署实例，不是开发机；
- 顺序：先★款后一般项——★款是合同关键项，先过★款定调；
- 记录：每行当场判定通过/不通过/有条件通过，当场签认，不事后补判。

### 8.10.3 验收报告的结构与签署纪律

验收报告六节：项目概况（一段）、验收依据（确认书+SRS V3.1+GB/T 25000.51）、UAT 结果表（7 行全通过）、质量证据索引（指向《测试报告》（含压测）、《故障演练验收记录》、部署验证记录）、遗留事项与约定、签署页。签署纪律三条：遗留事项必须写明『处理建议+时间建议+双方责任』，不许写『后续完善』这种空话（澜图组 2 项遗留：DEBT-02 边缘锯齿优化列入二期、文本提示长文本性能优化按 DEBT-03 触发条件执行）；有条件通过项必须有整改期限与复验方式；签署日期=验收会当天（9/4），所有签署人当场签。

### 8.10.4 遗留事项：诚实比完美得分高

澜图组带着 DEBT-02/03 两项债务进验收，没藏。验收报告如实列出并给出处置约定——评审反馈是『对系统边界有清醒认知』。对比之下，有组被评委现场发现缺陷后辩称『这不是问题』，信任受损远超缺陷本身。原则：验收不是证明你的系统没缺点，是证明你知道缺点在哪、为什么可接受、怎么处理。

### 8.10.5 人机协同要点

- UAT 表从确认书+SRS 半自动生成（AI 提取条款→生成用例行骨架），人逐条审定验收方式与证据形式；
- 验收报告的『质量证据索引』让 AI 做交叉检查：索引指向的文档编号、章节、指标值是否真实存在且一致；
- 遗留事项措辞 AI 可给草案，但承诺口径（时间/责任）必须人工裁决——这是商务性承诺。

### 8.10.6 本节清单

- UAT 行与确认书条款一一映射，★款优先；
- 执行用部署实例，甲方操作乙方不代答；
- 报告六节齐套，证据索引可跳转验证；
- 遗留事项三要素（建议/期限/责任）齐全，当场签署。

## 8.11 价值论证：效率收益是怎么算出来的

验收会 5 分钟的价值论证环节，澜图组只说了一个数：『人工勾绘该地块受灾农田需 45 分钟，本平台 6 分钟出结果，约 7 倍效率提升』。评委点头。这个数字背后是一整套方法：基线从哪来、口径怎么定、怎么防止技术自嗨。本节讲价值论证的做法。

### 8.11.1 价值论证的反面：技术自嗨清单

先立反面教材。这些说法在验收会上都出现过，全部无效：『我们用了 SAM 大模型，很先进』（评委：先进关我什么事）；『代码覆盖率 78%』（评委：所以能帮我省多少时间？）；『系统有 26 个 API 端点』（评委：.....）。技术自嗨的共性：说的是你们投入了什么，不是用户得到了什么。价值论证只有一个句式：『用户原来\_\_\_\_，现在\_\_\_\_，差值是\_\_\_\_』。

### 8.11.2 基线数据：45 分钟的出处

价值数字的可信度完全取决于基线的出处。澜图组的 45 分钟基线来自三重佐证：需求关用户访谈记录（应急值班员画像：1 万平方公里级地块人工目视勾绘 40~50 分钟）；文献佐证（遥感应急制图作业规范的工时估计）；实测佐证（请一名 GIS 专业人员用传统工具对演示地块实做一次，42 分钟）。三个来源指向同一量级，基线站得住。反面做法：拍脑袋说『人工要一整天』——夸大的基线会被评委一句话戳穿（『我们单位小李半小时就画完了』），整个论证崩盘。

### 8.11.3 口径设计：6 分钟包含什么

6 分钟的口径必须坦白：从影像上传完成到图斑导出，含全自动分割 2m41s + 人工交互修正约 2 分钟 + 导出 40 秒。注意它不含影像下载与预处理（那部分人工流程同样存在）。澜图组在 PPT 脚注里写了这个口径——评委若追问能答得上来，而且坦白口径反而加分：你们没把 6 分钟包装成『端到端全自动 6 分钟』。口径设计三原则：取与基线可比的区间（人工 45 分钟也不含数据获取）；含必要人工（交互修正是流程的

一部分，不藏）；写清边界（什么不算在内）。

#### 8.11.4 价值的第二层：质量与风险的量化

效率之外还有两层可论证。质量：人工勾绘的 IoU 因人而异（访谈记录 0.65~0.75 波动），平台稳定 0.78——『比最好的人强一点，比平均的人强不少，且永不疲惫』。风险：应急场景夜间作业人工错误率上升，平台 7×24 一致。这两层用定性+访谈佐证即可，不必硬造数字。价值论证的克制原则：有一个硬数字（7 倍）做主锚点，其余定性辅助——全是硬数字反而像编的。

#### 8.11.5 人机协同要点

- AI 可协助检索文献基线与整理访谈记录中的工时证据；
- 让 AI 攻击你们的论证：『扮演怀疑的评委，对 45min→6min 的论证提 5 个最尖锐的质疑』——每个质疑都要有答案；
- 红线：基线实测（42 分钟那次）必须真实发生，AI 生成的『模拟基线数据』是造假。

#### 8.11.6 本节清单

- 价值句式为『用户原来/现在/差值』，无技术自嗨表述；
- 主锚点数字有三重基线佐证，口径含边界说明；
- 质量/风险层定性论证有访谈或文献出处；
- 经受过『怀疑评委』预演，每个质疑有答。

### 8.12 复盘报告与偏差分析

样例《项目复盘报告》是收官交付物，章节结构对标 GB/T 8567 的『项目开发总结报告』，但内容不是八股——它回答四个真问题：计划说了什么、实际发生了什么、为什么有差、下次怎么干。评委对复盘报告的期待就一条：敢不敢用数据解剖自己。本节精讲《项目开发总结报告》的写法。

#### 8.12.1 章节结构与 GB/T 8567 映射

| 《项目开发总结报告》章节 | GB/T 8567 对应 | 核心内容                                  |
|--------------|--------------|---------------------------------------|
| 1 引言与项目概述    | 1 引言         | 背景、范围、里程碑回顾（六关时间轴）                    |
| 2 计划与实际偏差分析  | 2 实际开发结果     | 里程碑偏差（M2+1d、M4+0.5d）、工时 480→512h、偏差源表 |
| 3 AI 协作量化分析  | （特色扩展）       | 起草/保留/采纳/打回/复用/翻车/返工/净收益——本课程特色章      |
| 4 质量结果       | 2 实际开发结果（质量） | 缺陷密度 2.5、覆盖率、IoU、压测、对抗净得分 +5          |
| 5 技术债务与移交    | 3 开发工作评价     | DEBT-01~03+观察项，触发条件与偿还计划              |
| 6 经验与课程建议    | 4 经验与教训      | E-01~05 经验、I-01~06 课程改进建议             |

### 8.12.2 复盘的基调：归因于内，归因于机制

复盘写作最容易滑向两种失败文体：表功文（全是亮点，偏差轻描淡写）与检讨文（全是态度，没有数据）。《项目开发总结报告》的基调规则：每条偏差有数据（+18h，不是『略有超支』）、有归因（PE-01 性能攻关，不是『客观原因』）、有机制层结论（性能需求应在设计关做原型验证，不是『下次注意』）。归因于内的意思：不从外部找理由（『老师临时加需求』不算归因，CR-001 的归因有自部分——接口设计时未考虑批量场景）；归因于机制的意思：每条教训指向流程改进，不指向个人反省。

### 8.12.3 数据从哪来：复盘是聚合不是创作

复盘报告的全部数字都应该有上游出处：工时来自周报汇总；里程碑偏差来自 plan/tasks 基线与实际完成日期；AI 数据来自 Prompt 日志聚合；质量数据来自研发与质量类样例文档；债务来自缺陷库与代码评审遗留。写作动作是『聚合+归因』，不是『回忆+创作』。如果某个数字找不到出处，要么补测补算，要么删掉——复盘无出处的数字比没有数字更糟（评委随手一溯源就是信任危机）。这也是为什么全程日志、周报、缺陷库在最后一天集体兑现价值。

### 8.12.4 E 与 I 的写法：经验要可迁移，建议要可操作

《项目开发总结报告》E-01~05（经验）与 I-01~06（课程建议）各有写法。经验的检验标准是『换组换题还能用』：E-02『spec 先行在 CR-001 中省下至少 8h 返工——变更先改 spec 再改代码，文档与代码零漂移』，这条任何组任何题都成立。建议的检验标准是『课程组下学期能执行』：I-03『建议第 2 关增加接口设计预检点，CR-001 类变更 50% 可在设计关拦截』，具体、有数据支撑、指向关卡设计。写不出 E 和 I 的组，往往是过程记录太薄——又回到日志与周报。

### 8.12.5 人机协同要点

- 数据聚合（工时汇总、日志统计、偏差表）AI 半自动化，人审定口径；
- 归因分析用 AI 做『魔鬼代言人』：对每条归因问『这是机制问题还是运气问题？数据支持吗？』；
- 红线：复盘数字 100% 需上游出处；AI 直接生成的『漂亮复盘』（无溯源）是验收后抽查的重灾区。

### 8.12.6 本节清单

- 六节结构对标 GB/T 8567，AI 量化单独成章；
- 每条偏差三要素（数据/归因/机制结论）；
- 全部数字可溯源到上游文档；
- E 可迁移、I 可操作，各  $\geq 3$  条。

### 8.12.7 偏差分析：工时都花到哪去了

《项目开发总结报告》的核心表只有一张：计划工时 480h，实际 512h，净偏差 +32h（+6.7%）。这张表的说服力在于每一行都有名有姓：PE-01 性能攻关 +18h、M3 设计评审延期追赶 +13h、红队自测缺陷修复 +9h、CR-001 变更 +6h、文档 AI 提效 -14h。本节讲偏差分析的方法论——怎么记账、怎么归因、怎么把偏差变成下次的估算能力。

### **里程碑偏差：M2+1d 与 M4+0.5d 的故事**



| 里程碑       | 计划   | 实际     | 偏差    | 归因                                                 |
|-----------|------|--------|-------|----------------------------------------------------|
| M3 设计评审   | 8/19 | 8/20   | +1d   | 非功能设计的原型验证（分割推理压测预演）比预期复杂——事后看是值得的，它把 PE-01 问题提前暴露 |
| M5 质量门禁评审 | 9/1  | 9/1 中午 | +0.5d | 红队自测修复 9h 挤占，半天顺延但守住验收日                            |
| M6 交付验收   | 9/2  | 9/2    | 0     | 9/1 三审计一次通过，无返工                                    |

教学点：M3 的 +1d 是『好偏差』——它买到了风险前置。偏差分析不追求零偏差，追求每个偏差说得清、且同类偏差下次估得准。

### 工时偏差源表：四类来源

| 偏差源        | 工时   | 性质   | 机制层结论                                      |
|------------|------|------|--------------------------------------------|
| PE-01 性能攻关 | +18h | 技术债型 | 推理优化（TensorRT 候选）应在设计关原型验证——已转为 DEBT-01 跟踪 |
| CR-001 变更  | +6h  | 需求型  | 批量导出场景设计关未预研——接口预研入设计关检查单                  |
| 红队自测修复     | +9h  | 质量型  | 对方发现的我方盲区缺陷，修复是计划外但必要——对抗环节价值兑现            |
| 文档 AI 提效   | -14h | 收益型  | SRS/设计/测试文档的结构化章节 AI 半自动，节省如实入账            |

### 记账方法：偏差数据的三条管线

- 周报管线：每周报含『本周计划工时/实际工时/差值及一句话原因』——期末求和即得总表，不靠回忆；
- 看板管线：tasks.md 的原子任务带估时，完成时记实际，偏差在任务粒度就有记录；
- 事件管线：CR-001、PE-01 这类事件型偏差单独建卡（触发/工时/结论），事件卡直接进复盘。

三条管线在期末交叉验证：周报和 + 看板和 + 事件和，三者差值应 <5%——澜图

组差 2.1%（零散沟通时间），在复盘里如实说明。对不上账的复盘报告等于自曝过程管理失效。

## 从偏差到估算能力：毕业带走的真本事

软件工程教育最缺的是估算训练——一个月的项目就是一次完整的『估算-实测-校准』循环。给学生的方法：把偏差源表按类型汇总后，为每类算出『校准系数』（如性能攻关类任务实际/计划=2.3），下次估算同类任务乘上系数。《项目开发总结报告》给了四个系数。评委在这页常问『下个月再做一遍，你们报多少工时？』——正确答案不是 480，是『按校准系数重算的 496，其中性能项已转入设计关预研所以再减 8h』。

## 人机协同要点

- 工时聚合、偏差表生成、三线对账适合 AI 脚本化；
- 校准系数的适用性判断（这个系数外推靠谱吗）是人 + AI 讨论题，AI 给数据人做判断；
- 红线：实际工时不许为了『好看』回改周报——周报是过程证据，回改等于伪造。

## 本节清单

- 里程碑偏差逐条归因，含『好偏差』说明；
- 偏差源表四类齐全，收益型如实入账；
- 周报/看板/事件三线对账差 <5% 或有说明；
- 每类偏差源有校准系数，能回答『再做一遍报多少』。

## 8.13 AI 协作量化与净收益估算

《项目开发总结报告》给出五组 AI 协作核心数：AI 起草代码占比 58%（5590/9643 行）、人工修改后保留 44%（4244 行）、Prompt 平均采纳率 63%、PR 一次通过率 68%（打回率 32%，9 次打回按 4/3/2 归因）、Prompt 复用 94 次。评委必问：『这些数怎么统计的？』答不上口径，数据就是装饰。本节逐个讲口径与统计方法。

### 8.13.1 五个口径的精确定义

| 指标      | 数值  | 口径定义                   | 统计方法                             |
|---------|-----|------------------------|----------------------------------|
| AI 起草占比 | 58% | 初版 diff 中 AI 生成行 / 总行数 | PR 描述 AI 小节逐 PR 填报+期末聚合          |
| 保留率     | 44% | 合并后仍存在的 AI 行 / 总代码行    | git blame 打标聚合（AI 行以日志 +PR 记录认定） |
| 采纳率     | 63% | 采纳+改采会话 / 总会话数         | 日志『产出处置』字段聚合                     |
| 打回率     | 32% | 打回 PR / 总 PR           | 评审记录；9 次归因 4 测试缺失/3 规范/2 设计      |
| 复用次数    | 94  | 模式库 Prompt 被复用次数       | 模式库条目带复用计数（每次引用 +1）              |

### 8.13.2 口径设计的三原则

- 过程留痕可算：每个指标的数据源是过程记录（PR 描述、日志、评审记录），不是期末回忆——期末『补统计』的数一律不可信；
- 口径先于数据：指标定义在第一周立规时就写好（什么算『改采』：AI 代码经人工修改后合入主干），不能期末看着数据定口径——那是挑数；
- 不利数据照报：保留率 44% 意味着 AI 写的代码一半多被人改掉了——这不是丑事，这恰是『人在把关』的量化证明。打回率 32% 同理。

### 8.13.3 打回率 4/3/2 归因：最有信息量的一个数

9 次打回的归因分布：4 次测试缺失（AI 生成代码没同步生成用例，评审拦下——测试护栏在起作用）、3 次规范问题（命名/注释/日志规范）、2 次设计分歧（方案与 ADR 冲突）。这个分布的管理含义：打回不是失败，是门禁工作的证据；4/3/2 的结构说明测试门禁最严、设计一致性有 2 次失守（已回归 ADR 宣贯）。复盘里这类『结构分析』比总数有价值得多。

### 8.13.4 复用 94 次与模式库 Top3

Prompt 复用数 94，Top3：P-04『按 spec 生成接口骨架+Given/When/Then 桩』×17、P-06『生成单测并自评覆盖盲区』×12、T-P02『代码评审清单式自查』×9。复用机制：

模式库每条带复用计数，引用即 +1，期末排序。Top3 的共性——都是『规格驱动型』Prompt（喂 spec 出工件），不是『聊天型』Prompt。这本身就是课程结论的量化支撑：Spec-Driven 的 Prompt 才有复用价值。

8.13.5 人机协同要点（AI 协作量化）

- 聚合脚本（日志→采纳率、PR→打回归因）AI 生成，人抽样验证（抽 10 条手工核对）；
- 口径文档本身人写——这是管理承诺，不外包；
- 答辩被问口径时的标准答法：定义→数据源→抽样验证方式，三段 30 秒。

8.13.6 收益侧 +84h：三笔进项

| 进项     | 工时   | 依据                        | 备注                  |
|--------|------|---------------------------|---------------------|
| 代码生成提效 | +52h | 同类任务的历史参照工时 vs 实际（任务粒度对比） | 接口骨架、单测样板、CRUD 类最受益 |
| 文档半自动化 | +14h | SRS/设计/测试文档结构化章节的 AI 起草   | 即偏差源表的 -14h 项       |
| 测试用例扩编 | +18h | 红队自测准备中 AI 辅助生成用例草稿       | 含人审定时间净额            |

收益估算的方法论要点：必须有『不借助 AI 的参照工时』，参照来自任务估算基线（看板估时即『无 AI 参照』，因为估时是按传统方式训出来的）或同类任务对照。没有参照系的『提效』都是感觉。

8.13.7 成本侧 -23.5h：三笔开销

| 开销      | 工时    | 依据                   | 教训                   |
|---------|-------|----------------------|----------------------|
| 翻车处置与返工 | -11h  | 5 起翻车记录的处置工时求和       | 坐标转换 300m 偏差一起占 4.5h |
| 日志与归档   | -6h   | 19 天日志书写+值班日检        | 制度成本，审计合规的门票         |
| 评审加严    | -6.5h | AI 代码评审比人工代码平均多耗 25% | 买质量的钱，值              |

### 8.13.8 口径坦白：为什么 +61h 比『提效 50%』有说服力

坦白的三个设计：收益成本分列（只报收益不报成本是广告，不是复盘）；每笔有出处（翻车 11h 能拿出 5 份记录，日志 6h 能拿出值班表）；不确定性声明（《项目开发总结报告》原话：『代码生成提效的参照系为任务估时，估时本身有  $\pm 20\%$  误差，故净收益的置信区间约为 +45~75h』）。评委对『+61h』的接受度远高于『效率翻倍』，因为它像工程数据而不是营销话术。这就是『口径坦白方法』：数字越小越具体，越可信。

### 8.13.9 边际结论：AI 在哪赚、在哪赔

账本最有价值的产出是边际结论。赚：规格明确的结构化工作（接口骨架/单测/文档章节/用例扩编）——投入产出比约 4:1；赔：上下文稀缺的开放问题（坐标系细节/性能根因/边界条件）——投入产出比约 0.6:1（翻车的 4.5h 把收益吃成负的）。行动结论：AI 使用预算应向『规格驱动』场景倾斜，向『探索排障』场景限配并强制人工复核——这条结论直接来自账本，不是来自感觉。

### 8.13.10 人机协同要点（AI 净收益估算）

- 账本聚合 AI 脚本化，但参照系的选定必须人工裁决（哪个任务和哪个任务可比）；
- 置信区间的表述方式可参考：让 AI 对每笔账目做敏感性分析（参照系浮动  $\pm 20\%$  时净收益区间）；
- 红线：成本侧漏记是通病（人倾向于忘记录翻车耗时）——翻车记录模板内置工时字段就是堵漏设计。

### 8.13.11 本节清单

- 五指标口径成文，第一周定稿；
- 数据源全部过程留痕，抽样验证过；
- 打回归因到类，有结构分析；
- 模式库带复用计数，Top3 有共性结论。
- 收益成本分列，各  $\geq 3$  笔且每笔有出处；
- 参照系明确，不确定性有声明；

- 边际结论（哪赚哪赔）成文并转化为行动规则；
- 净收益口径能经受『怎么算的』追问。

8.14 技术债务台账与经验萃取

交付不等于无债，等于『债已知情、有主、有还期』。《项目开发总结报告》的技术债务台账：DEBT-01 推理性能优化（TensorRT 候选方案）、DEBT-02 矢量导出边缘锯齿（BUG-142 遗留）、DEBT-03 文本提示长文本性能（BUG-143 遗留），另有观察项一条（等保日志留存）。本节讲债务台账怎么建、怎么写、怎么移交。

8.14.1 债务条目的六要素

| 要素    | DEBT-01 示例                               | DEBT-02 示例                    |
|-------|------------------------------------------|-------------------------------|
| 债务描述  | 分割推理未做 TensorRT 优化，大地块 P99 7min12s 达阈值上限 | 矢量导出图斑边缘锯齿，目视可辨但不影响面积统计精度     |
| 形成原因  | PE-01 攻关止于达标，优化方案未实施（时间约束）               | 抗锯齿方案需改导出管线，评估工作量 12h，验收前不值得动 |
| 影响    | 数据规模超当前 2 倍时将破线                          | 目视质量受损，定量指标无损                 |
| 触发条件  | 接入 >2 万平方公里地块或并发 ≥3                      | 甲方提出出版级制图要求                   |
| 偿还方案  | TensorRT 量化+批处理管线，预估 16h                 | 导出管线叠加抗锯齿后处理，预估 12h           |
| 责任与跟踪 | 移交二期，触发即立项                               | 列入验收报告遗留事项，双方签认               |

8.14.2 债务与缺陷的分野

BUG-142/143 为什么变成 DEBT 而不是继续修？判定逻辑：缺陷修复的收益 < 成本 + 风险，且在约定场景内不影响验收标准，经双方知情同意后降级为债务。关键在『知情同意』——DEBT-02/03 都写进了验收报告的遗留事项并签署。未经甲方知情而自行决定『不修了』，那叫隐瞒缺陷，不叫债务管理。台账里的每条债务必须有这条签认链：发现→评估→披露→签认→入册。

8.14.3 观察项：债务的预备役

等保日志留存观察项：系统日志当前留存 30 天，等保二级要求  $\geq 6$  个月——但本平台当前部署场景（教学/科研内网）未定级等保，故列为观察项而非债务。观察项的语义是『条件性债务』：一旦部署场景变化（对外服务/定级备案），即刻升级为债务并启动整改（日志归档改造，预估 8h）。观察项机制防止两类极端：把远期合规要求全部当期消化（浪费），或完全不记录（遗忘）。

#### 8.14.4 移交与追溯

- 债务台账随交付包移交（配置文件 DEBTS.md），并进入验收报告遗留事项章；
- 每条债务有代码锚点（相关模块/PR 引用），接手方能找到现场；
- 触发条件的监控责任写入部署运维手册（如 DEBT-01：运维章要求监控地块规模与并发，越线即告警）；
- 课程层面：复盘答辩必问『你们的债为什么是这个数』——零债务的组会被追问『是真没债还是没盘债』，后者更糟。

#### 8.14.5 人机协同要点

- 债务盘点可让 AI 扫缺陷库与评审遗留，生成候选清单——但『降级为债务』的决策必须人工+甲方知情；
- 偿还方案的工作量估算 AI 辅助（参照历史同类任务）；
- 红线：台账披露内容不许 AI『润色弱化』——『目视可辨』被改成『几乎不可见』就是披露失真。

#### 8.14.6 本节清单

- 债务条目六要素齐全，签认链完整；
- 债务/缺陷/观察项三级分野清晰；
- 台账随包移交，触发条件进运维手册监控；
- 答辩能解释每条债的『为什么是这个数』。

#### 8.14.7 经验萃取与课程建议

复盘报告的收尾两节——经验（E 系列）与课程建议（I 系列）——是小组留给后者的资产。写得好，下届学生踩着你们的肩膀；写得水，就是『加强沟通、注意质

量』式废话。《项目开发总结报告》E-01~05、I-01~06 全部有数据、有机制、可执行。本节讲萃取方法。

**经验萃取：从事件到机制的三步**

第一步捞事件：翻过程记录（日志/翻车记录/偏差源表/审计结果），列出全周期『如果重来一次我们会不一样』的时刻，澜图组捞出 14 个候选事件。第二步验成色：对每个事件问三个问题——有数据支撑吗？换组换题还成立吗？能转化为一条规则吗？三问全过才入选（14→5）。第三步写机制：把事件改写成『规则+证据+适用条件』句式。

**E 系列样本解剖**

| 编号   | 经验（机制表述）                                                    | 证据                               | 适用条件           |
|------|-------------------------------------------------------------|----------------------------------|----------------|
| E-01 | 需求关的 AI 反向澄清必须人工裁决——12 条反澄清中 11 条被驳回，但命中那 1 条（导出格式歧义）值回全部成本 | 23 提问 11 驳回记录；SRS V2.0→V3.1 修订记录 | 所有用 AI 审文档的场景  |
| E-02 | 变更先改 spec 再改代码——CR-001 零漂移，且省下约 8h 返工                       | CR-001 全流程记录；偏差源表                | 任何有 spec 工件的项目 |
| E-03 | 干净环境自测至少三轮——缺口 4→2→0，助教验证 1h12m 一次通过                        | 《可移植性批量验证记录》三轮记录                 | 有部署交付的项目       |
| E-04 | Prompt 模式库只沉淀『规格驱动型』——Top3 复用 38 次占总量 40%                   | 模式库复用计数                          | AI 深度使用团队      |
| E-05 | 红队自测的计分激励让红队全程不划水——命中 5 条有效缺陷，发现 5 个真实盲区                    | 红队自测记录                           | 有组间机制的班级       |

**I 系列：给课程组的可执行建议**

I 系列的读者是课程组，写法是『建议+依据+预期收益』。样本：I-01『建议第 3 关增加接口预研检查点（依据：CR-001 类变更源于设计关未预研，预期拦截约 50% 类似变更）』；I-02『建议 Prompt 日志模板内置工时字段（依据：ROI 账本成本侧漏记



高发) 』; I-03『建议验收演示统一要求冻结集外数据(依据: 澜图组自证策略效果显著) 』; I-04~06 涉及值班员检查点挂 commit、债务台账模板下发、复盘数据口径前置培训。每条都能在下学期课程设计里落地——不可执行的建议(『希望多讲点大模型原理』) 不进 I 系列, 进附录感想。

### **萃取工作坊: 一小时产出 E 与 I**

- 0~15 分钟: 全员独立翻记录捞事件, 写便利贴(目标每人  $\geq 3$  个);
- 15~40 分钟: 逐张过三问(数据? 可迁移? 规则化?), 合并同类, 14→5;
- 40~55 分钟: 每条由提案人按『规则+证据+适用条件』成稿;
- 55~60 分钟: I 系列头脑风暴, 只收『课程组能执行』的, 当场验证可执行性。

### **人机协同要点**

- 捞事件环节 AI 可预筛(扫日志与翻车记录生成候选事件清单), 人做三问裁决;
- 机制表述的句式打磨 AI 辅助, 但证据引用必须人工核对编号;
- I 系列的可执行性检查让 AI 扮演课程组: 『这条建议你作为课程设计者会怎么落地? 缺什么信息? 』

### **本节清单**

- E  $\geq 3$  条, 每条三件套(规则/证据/适用条件);
- I  $\geq 3$  条, 每条三件套(建议/依据/预期收益) 且课程组可执行;
- 萃取工作坊有过程记录;
- 无『加强沟通』式空话(评审一眼识别)。

## **8.15 移交收口与迁移到你自己的题目**

验收通过不是终点。9/2 下午, 澜图组做最后的收口: 复盘定稿、移交清单签署、配置库冻结归档。一个月后、一年后, 这组资产还要支撑课程展示、学弟参考、就业作品——收口质量决定它的生命周期。本节讲移交清单与配置库的最终规程。

### **8.15.1 移交清单: 双方签字的资产点交**

| 类别    | 移交项                                 | 验收时状态            | 签收确认  |
|-------|-------------------------------------|------------------|-------|
| 代码    | 主干仓库（tag v1.0-acceptance）+完整 Git 历史 | 已冻结              | 甲乙双方签 |
| 文档    | 全套样例文档全集（PDF 归档版+可编辑源文件）            | 交付审计类文档验收后定稿     | 甲乙双方签 |
| 运行资产  | 交付包（镜像+compose+手册+冒烟）               | 《可移植性批量验证记录》验证通过 | 甲乙双方签 |
| 过程资产  | Prompt 日志/周报/评审记录/模式库               | 审计后封存            | 课程组签  |
| 债务与遗留 | DEBTS.md+验收报告遗留事项章                  | 双方知情同意           | 甲乙双方签 |
| 演示资产  | 验收汇报 PPT+演示数据+录屏                    | 验收使用版            | 课程组签  |

### 8.15.2 配置库六目录的最终态

8.5 节建过配置库结构，收口时做最终归位与 INDEX.md 终版：01\_立项与招投标（第 1~2 关样例文档（项目建议书至 SRS 全套））、02\_需求（设计与决策类样例文档（DLD、数据库、接口、OpenAPI、ADR）+确认书）、03\_设计（计划类样例文档（plan/tasks、《测试计划》、管理计划族、RTM v3、迭代冲刺计划）+ADR+plan/tasks）、04\_研发（代码库引用+模式库+翻车记录）、05\_质量（研发与质量类样例文档（Prompt 模式库至缺陷看板））、06\_验收与复盘（交付审计类样例文档（文档审计、可移植性验证、AI 使用审计、验收汇报 PPT、复盘报告、验收报告）+移交清单）。INDEX.md 终版三条增量规则：每份文档标注最终版本号与定稿日期；失效版本移入各目录 \_archive 子目录（不删除——版本沿革是审计材料）；全库再生成一次哈希清单，与交付包哈希分列。

### 8.15.3 冻结与命名：给未来的自己留路标

- 代码：打 tag v1.0-acceptance，主干设保护（此后改动走新分支，不进验收基线）；
- 文档：PDF 归档版命名统一为『编号\_题名\_V 版本\_YYYYMMDD.pdf』（如 47\_验收报告\_V1.0\_20260904.pdf）；

- 演示数据：冻结影像集+演示用景单独归档目录，附 README 说明来源与用途；
- 日志：Prompt 日志目录整体打包，附统计摘要（19 天/完整率 97.4%/Top3 复用）。

#### 8.15.4 收口检查：最后一次 preflight

收口完成前跑一次『元 preflight』：随机抽三份文档能否从 INDEX 30 秒内定位；交接包哈希能否复算通过；tag 与文档版本号是否互相对应；模拟一个未来场景——『学弟想复现验收演示』，仅凭库内材料能否走通（INDEX→06 目录→45 PPT+演示数据+部署手册→交付包）。这个场景测试是收口的终极标准：库的价值在于让不在场的人复现你们的工作。

#### 8.15.5 人机协同要点

- INDEX.md 终版与命名规范化检查 AI 批量执行；
- 『学弟复现』场景测试可以让 AI 先走一遍纸面流程找断点，再人工实测；
- 红线：封存后任何修改必须走『新版本+变更记录』，禁止原位覆盖——原位覆盖是配置管理的重罪。

#### 8.15.6 本节清单

- 移交清单六类双方签收；
- 配置库六目录归位，INDEX 终版含版本与定稿日期；
- tag/PDF/数据/日志四项冻结命名规范；
- 『学弟复现』场景测试通过。

#### 8.15.7 迁移到你自己的题目

##### **复刻练习的设计意图**

教学实践证明，从“看懂样例”到“做出自己的”之间隔着一道鸿沟。复刻练习要求各组以澜图样例为参照物，对自己选定的题目完整走一遍任务 1 流程。复刻不是照抄——样例提供的是结构、粒度和质量标准，内容必须来自本组题目。每完成一份交付物，对照样例做一次“粒度检查”：我的条目数和细致程度与样例相当吗？低于样例粒度的部分，多半是没做透。

## 自拟题目的招标素材从哪来

任务 1 的前提是一份“招标文件”。自拟题目的小组有两条获取路径：

- 路径一（推荐）：逆向生成——参照澜图案例的做法，选定一个真实开源项目或真实业务场景，逆向编写其《招标文件》。要求：含投标人须知、评分表（合计 100 分）、用户需求书（FR 编号条款  $\geq 30$  条，其中 ★ 条款 8–13 条）、格式要求。教师审核后生效；
- 路径二：教师发放——课程组提供 2–3 份备选招标文件（不同行业），小组认领。

自拟招标文件的质量红线：条款必须可测（“应支持…… $\leq X$  秒/ $\geq Y\%$ ”句式）；★条款必须是真正的实质性要求；评分表必须覆盖任务 1 全部交付物。写不出合格招标文件的小组，说明对题目理解不透——这正是该练习的设计目的。

## 逐份交付物的复刻要点

### 项目建议书（对照）

- 结构复刻：八节一节不少；粒度对标：痛点量化条数  $\geq 3$ 、风险条数  $\leq 4$ ；
- 迁移难点：痛点的量化数据——自拟题目没有真实行业数据时，用“假设数据 + 标注（教学示例）”，并在答辩时主动说明假设依据。

### 澄清与质询记录（对照《澄清与质询记录》）

- 澄清  $\geq 6$  条且每条带“对方案的影响”列；质询口径  $\geq 8$  条；
- 迁移难点：自拟招标文件是自己写的，澄清问题容易“自问自答”走过场——对策：交换评审，A 组的标书由 B 组提澄清问题。

### 技术投标书（对照 V2 技能版）

- 硬性对标：14 章结构、实质性条款响应表单独成表且照抄原文、三件套齐备、脚本初检红项清零；
- 迁移难点：篇幅与分值的匹配——自拟题目的评分表是小组自定的，常见毛病是把分值平均分；参照澜图（功能 16 分最高、培训 4 分最低）拉开权重。

### 项目计划（对照）

- 硬性对标：WBS 按交付物分解且  $\leq 1$  周粒度、甘特图与 WBS 一致、里程碑带准出标准；

- 迁移难点：工期——自拟题目若无外部约束，按课程统一要求（18 个工作日总窗口）倒排。

### 风险登记册（对照）

- 硬性对标：≥10 条、“条件—后果”句式、含 AI 辅助开发风险、高风险有触发器；
- 迁移难点：从承诺反查——自拟题目的承诺是自己写的，学生容易把承诺写得很“安全”（无指标），评审时要反向检查“承诺是否低到没有训练价值”。

### 述标讲稿（对照《述标答辩讲稿》）

- 硬性对标：≤1900 字、时间码、评分因素对应、三个记忆点；
- 迁移难点：记忆点的选择——自拟题目的差异化卖点需要真思考，AI 给的候选往往平庸，全组辩论后拍板。

### 需求确认书（对照）

- 硬性对标：MVP ≤5 项构成闭环、验收依据可测有出处、范围外清单、CCB 条款；
- 迁移难点：MVP 取舍——自拟题目没有真实甲方，教师扮演甲方时必须“入戏”提反对意见。

## 复刻质量的三层验收

- 形似（结构齐）：章节、表格、签署栏一样不少——机器可查；
- 神似（粒度当）：条目数、量化程度、可测性与样例相当——对照可查；
- 骨似（逻辑通）：痛点—方案—承诺—计划—风险—MVP 全链路自洽，评委交叉追问不矛盾——只能靠答辩检验。

三层全部通过，任务 1 才算真正“过关”。形似的交给脚本和检查单，神似的交给对照表，骨似的交给答辩——这正是“机器初检 + 人工复核 + 现场评审”三层质量保证在课程中的完整落地。

## 业务系统类（如：社团管理平台、实验室预约系统）

- 需求理解章：重点在角色与流程（谁用、什么流程、现有方式痛点），业务流程图是标配；痛点量化用“原来一次预约要 3 天 5 个环节”这类过程数据；

- 功能章：以业务闭环为主线组织（申请—审批—执行—归档），权限矩阵通常是 ★ 级关注点；
- 关键技术：难点不在算法而在并发与一致性（抢约冲突、状态机），关键技术章写事务设计与冲突消解；
- MVP 界定：选“主流程闭环”五项（如：注册登录、资源发布、在线预约、审批流、记录导出），报表与统计类放范围外；
- 典型风险：需求蔓延（业务方脑洞大）、并发冲突、数据迁移——风险登记册据此侧重。

### **AI 应用类（如：题库智能判卷、论文润色助手——澜图即属此类）**

- 需求理解章：重点在“AI 能力边界”与“人机分工”——哪些环节 AI 自动、哪些必须人工复核，写进需求理解就是加分点；
- 关键技术章：模型选型对比（候选模型/效果/成本/许可证）、效果验证方法（测试集、指标、人工抽检比例）、兜底策略（模型答不出/答错时怎么办）是三大必写项；
- 非功能章：响应时延与并发（模型推理是性能瓶颈）、结果可解释性与留痕（AI 决策要可复核）；
- MVP 界定：把“效果未经验证的功能”排除在 MVP 外——先验证再承诺（澜图把原型验证放在 M3 评审前，就是这个逻辑）；
- 典型风险：效果不达标（必须有量化验收指标与人工兜底）、模型许可证、提示词注入等 AI 特有安全问题。

### **工具/算法类（如：代码查重工具、图像批处理插件）**

- 需求理解章：重点在用户场景与对标分析（与现有工具比的差异点）；
- 功能章：以输入—处理—输出为主线，接口与兼容性（支持什么格式、什么环境）是 ★ 级关注点；
- 关键技术章：算法正确性与性能（复杂度、基准测试数据），附原型 benchmark；
- MVP 界定：核心算法 + 最小可用界面/接口，周边格式支持放范围外；

- 典型风险：算法效果不及预期、运行环境兼容性、开源依赖合规。

差异化套用检查表

| 章节     | 业务系统类侧重      | AI 应用类侧重     | 工具/算法类侧重    |
|--------|--------------|--------------|-------------|
| 需求理解   | 角色与流程、过程数据量化 | AI 能力边界与人机分工 | 用户场景与对标     |
| 功能方案   | 业务闭环、权限矩阵    | 效果验证与兜底策略    | 输入输出与兼容性    |
| 关键技术   | 并发一致性、事务     | 模型选型、效果指标    | 算法正确性与性能    |
| 非功能    | 并发容量、数据安全    | 推理时延、可解释留痕   | 性能基准、环境兼容   |
| MVP 取舍 | 主流程闭环        | 已验证效果的功能     | 核心算法 + 最小接口 |
| 头号风险   | 需求蔓延         | 效果不达标        | 兼容性/依赖合规    |

通用性兜底

三类之外的题目（如硬件结合、游戏、数据可视化），按同一自问清单确定侧重：（1）我的 ★ 条款会是什么？（2）评委最可能质疑什么？（3）最难验证的承诺是什么？——三个问题的答案，就是各章要加厚的地方。拿不准的，回到澜图样例找最近的章节模仿其粒度。

模式库的使用方法

本节给三类常见自拟题目的架构起手式：Web 管理类、AI 应用类、数据工程类。起手式不是模板照抄，而是『这类题目通常的关键决策点+推荐默认选项+必须重新论证的点』。使用时先判断题目类别，再逐项过决策点，每一项问自己：澜图（或本模式）的选择在我的约束下还成立吗？

Web 管理类（如教务/科研/资产管理）

| 决策点  | 默认选项          | 必须重新论证的情形        |
|------|---------------|------------------|
| 架构形态 | 单体+模块化（非微服务）  | 真实并发>500 或团队>8 人 |
| 权限模型 | RBAC+数据权限行列规则 | 存在跨部门复杂审批        |

| 决策点 | 默认选项              | 必须重新论证的情形          |
|-----|-------------------|--------------------|
| 工作流 | 状态机显式建模（状态表+迁移守卫） | 流程>10 个状态时考虑工作流引擎  |
| 报表  | 预聚合+异步导出          | 实时大屏类需求            |
| 审计  | 操作日志同事务落库         | 有等保/财务合规时升级为不可篡改存储 |

该类题目的 NFR 重点是数据一致性（并发编辑冲突）与权限正确性；验收演示的重头戏是权限矩阵全量校验与异常流（重复提交、越权、并发冲突）。

### AI 应用类（如澜图、智能问答、图像识别）

| 决策点     | 默认选项                     | 必须重新论证的情形          |
|---------|--------------------------|--------------------|
| 推理进程模型  | 常驻 Worker+队列（ADR-003 模式） | 多模型高频切换或 GPU 集群    |
| 交互与批量隔离 | 交互独立通道+批量独占队列            | 交互 SLA 宽松（>10s）可合并 |
| 精度验收    | 冻结数据集+版本哈希+加权指标          | 甲方提供权威评测集时直接用      |
| 降级策略    | GPU→CPU 降级+熔断            | 无 CPU 可行路径时改为排队+公告 |
| 模型来源    | 开源模型+许可证审查               | 商用模型须论证成本与合规       |

该类题目的特殊风险：精度指标的举证链（冻结集→测法→报告）必须在设计期锁定，验收期补不出；AI 生成内容是交付物的一部分，其人工审查记录要留档。

### 数据工程类（如采集平台、交换平台、治理工具）

| 决策点  | 默认选项             | 必须重新论证的情形          |
|------|------------------|--------------------|
| 数据架构 | 分层（原始/清洗/服务）+血缘表 | 实时流场景改流批一体         |
| 调度   | 同步执行/模型缓存        | DAG 复杂依赖时用 Airflow |
| 质量规则 | 规则表+执行报告（可配置）    | 规则复杂到需要 DSL 时专项论证  |



| 决策点   | 默认选项       | 必须重新论证的情形       |
|-------|------------|-----------------|
| 交换接口  | 文件+API 双通道 | 涉密或高频场景单通道强化    |
| 回填与重跑 | 幂等设计+按分区重跑 | 有外部副作用的操作须人工确认闸 |

## 模式之外的题目

游戏、嵌入式、工具软件等题目不在三模式内时，方法不变：先列本题目前五个最高风险的技术决策点（写不出五个说明调研不足），每个决策点按 ADR 流程论证。模式库的价值是节省『已知问题』的论证时间，不是替代『未知问题』的论证。

## 直接可搬的（与题目无关）

- PR 门禁五环流程与评审五查清单；
- CI 五道闸的框架（lint/单测+覆盖率/check\_refs/契约/安全），脚本改配置即用；
- 四步变更流程与变更单模板；
- 站会/周报/燃尽三仪式与模板；
- Prompt 模式库的卡片格式与运营机制（P-01~P-12 中约一半模式与题目无关，直接入库）；
- 翻车记录格式与五问法；
- 人机分工矩阵的判定三原则（具体档位按题调整）。

## 需要改造的（按题目性质）

| 要素   | 澜图样例的做法          | 换成管理类系统怎么改                      |
|------|------------------|---------------------------------|
| 性能指标 | 分割 P95≤3s、降级≤30s | 改响应时间类：核心查询 P95≤500ms、并发 50 无错误 |
| 精度测试 | IoU+冻结影像集        | 改为数据正确性测试：造数脚本+ golden 输出比对     |
| 故障演练 | kill GPU Worker  | kill 数据库连接、模拟第三方接口超时            |
| 安全红线 | 四条不变             | 不变，但 R3 输入校验权重更高（表单密集）          |

| 要素                | 澜图样例的做法                   | 换成管理类系统怎么改                  |
|-------------------|---------------------------|-----------------------------|
| 核 心<br>模 块<br>分 级 | queue/worker/geo/security | 按业务定：通常是权限、账务/状态机、数据一致性相关模块 |
| 契 约<br>测 试        | OpenAPI 6 端点              | 同样适用，管理类系统端点更多，收益更大         |

## 必须重建的（与题目强绑定）

- context pack 模板里的『约定清单』：每道题的反常识约定不同（本题的 0~1 浮点阈值、CGCS2000 存储基准，到了你题可能是『金额一律存分』『时间一律 UTC』）——这些约定从 SRS 与评审事故中提炼，无法继承；
- 评审清单的领域条目：R-01~R-07 通用，但要补本题特有条目（管理类系统常补『权限判断必须在服务端』『金额运算禁止浮点』）；
- 故障剧本库：按本题架构重新穷举（方法见 2.7 节，AI 穷举+人工筛选）；
- 度量基线：门禁数字按本题首测重新定，不继承样例的 70/85。

## 迁移检查单

- CI 五道闸在本题仓里全部跑通，且每道闸都被真实触发过至少一次（故意制造违规验证）；
- 第一笔翻车记录已入库（如果冲刺三天还没有，说明审视不够）；
- 本题的约定清单已写出至少 5 条，且每条能说出来历；
- 评审清单含至少 2 条本题特有领域条目；
- RTM 的列结构与即时登记纪律已就位。

## 常见迁移错误

| 错误       | 表现                   | 纠正                   |
|----------|----------------------|----------------------|
| 照抄指标     | 管理系统也写 $P95 \leq 3s$ | 指标从本题 SRS 的非功能需求推导   |
| 跳过重建约定清单 | 直接沿用样例的约定            | 约定清单必须从本题 SRS 与踩坑中生长 |

| 错误        | 表现                    | 纠正               |
|-----------|-----------------------|------------------|
| 照搬分工      | 题目没有 GPU 也设 DevOps 专职 | 按本题技术栈重排主副修矩阵    |
| 仪式先行，内容空心 | 站会开得很勤，看板是空的          | 先建看板与任务卡，仪式对着看板开 |

### 原样迁移层：与题目无关的全部

- 流程骨架：三审计→验收会→复盘→归档的四日结构；
- 工件体系：交付审计类样例文档（文档审计、可移植性验证、AI 使用审计、验收汇报 PPT、复盘报告、验收报告）的文档集结构（审计报告/部署验证/AI 审计/PPT/复盘/验收报告）；
- 纪律体系：四层声明、补录打标、三条红线、封存规程；
- 协同模式：AI 预审人终审/聚合归因分离/对抗预演/起草注魂四模式。

### 按题适配层：替换内容不换结构

| 澜图样例                 | 迁移时替换为                     | 不变的结构               |
|----------------------|----------------------------|---------------------|
| 暴雨应急演示动线             | 你题的核心业务场景（如二手书的『期末收书潮』）    | 场景化动线：情境→操作→交付→价值收束 |
| 45min→6min 价值锚点      | 你题的用户工时基线（实测！）             | 三重基线佐证+口径坦白         |
| 五步验收点（GPU/冒烟/持久化...） | 你题的技术栈检查点（如无 GPU 则改为内存/并发） | 自测三轮+收敛曲线           |
| DEBT-01~03           | 你题的真实债务                    | 六要素+签认链+观察项机制       |
| PR#76 权限矩阵亮点         | 你题的人机分工样板 PR               | 『AI 修点人修面』的叙事结构     |

### 重新推导层：样例没有答案的

- 验收标准：澜图的  $IoU \geq 0.75$  来自遥感领域，你题的核心质量指标必须从你题的用户需求推导——没有『行业默认值』可抄；

- 演示数据：澜图用冻结集外哨兵影像，你题的『信任策略数据』要自己设计（原则同：非排练数据、真实、中上表现）；
- 故障演练场景：杀进程/断网/磁盘满是澜图的运维风险画像，你题（如纯 Web）可能是并发突增/缓存击穿/第三方接口挂掉。

## 迁移自检：十个问题

对照样例做完你题的收口设计后，回答：我们的『1h12m』会是多少（部署耗时可接受上限）？我们的『五步验收点』是什么？我们的演示动线讲什么用户故事？我们的价值锚点基线从哪来？我们的核心 10 个数是哪 10 个？我们的债务会集中在哪？我们的 AI 审计亮点 PR 候选是哪个？我们的故障演练怕什么？我们的 I 系列能给课程组提什么？谁的五句话最弱？——十问答齐，迁移完成。

## 本节清单

- 能区分直接搬/要改造/必重建三类资产；
- 按检查单完成本组迁移验证；
- 避开四个常见迁移错误。
- 三层（迁移/适配/推导）已分类标注到组内收口计划；
- 按题适配层的替换内容全部有出处（实测/访谈/推导记录）；
- 迁移自检十问答齐并归档。

# 第 9 章 附录与工具集

本章是工具箱，不必通读：逐日作战手册在每关开工那天早上看，答辩题库在每次评审前一晚看，模板、检查单、术语表随用随翻。

## 9.1 18 个工作日逐日作战手册

本节把六关的窗口展开为逐日计划，3~5 人小组可直接套用并按本组题目微调。日历按工作日计算，周末不计入预算；日期以示例学期（8/10 周一开学）标注，实际学期按日历平移。计划按 4 人组设计：3 人组把前端与测试职责并入后端，5 人组可增设一名甲方联络员。每日节奏统一为『上午集中生产、下午交叉评审、17:00 前提交当日成果、晚间归档』。

### 9.1.1 全程日历总表

| 关卡         | 工作日     | 日期（示例学期）        | 天数 | 本关主线        | 门禁           |
|------------|---------|-----------------|----|-------------|--------------|
| 第 1 关 立项竞标 | D1–D4   | 8/10 一 – 8/13 四 | 4  | 管理线启动       | 正式评审         |
| 第 2 关 需求规格 | D5–D6   | 8/14 五、8/17 一   | 2  | 研发线起点       | 正式评审         |
| 第 3 关 设计计划 | D7–D8   | 8/18 二 – 8/19 三 | 2  | 设计与工程基线     | checklist 快检 |
| 第 4 关 研发冲刺 | D9–D15  | 8/20 四 – 8/28 五 | 7  | 研发线主体（两个迭代） | 中期评审         |
| 第 5 关 质量门禁 | D16–D17 | 8/31 一 – 9/1 二  | 2  | 交付线攻坚       | 正式评审         |
| 第 6 关 交付验收 | D18     | 9/2 三           | 1  | 管理线收口       | 验收评审         |

研发是全课程的重心：独占 7 天、横跨两周，是全课程最长的单关窗口，这是日历设计里最重要的取舍——宁可压缩需求、设计与质量环节的窗口，也要保住研发时间。研发窗口中间隔着一个周末（D10 与 D11 之间），小组可自主使用周末消化阻塞项与联调欠账，但过程证据的时间戳落在周末不加分也不减分。需求、设计、质量三关被压缩到两天，靠的不是赶工，而是三个提前量：模板与规则文件在上一关就备好、AI 量产章节与用例、评审纪律前置到每一天。

### 9.1.2 第一关（D1~D4，8/10~8/13）：立项竞标

| 日次        | 主线任务      | 关键动作                                           | 当日出口标准             |
|-----------|-----------|------------------------------------------------|--------------------|
| D1 8/10 一 | 读标与立项     | 上午：开标会、精读招标文件、小组组建与角色分工；下午：《项目建议书》开工、评委会质询问题清单 | 建议书 V0.5；质询问题 ≥8 条 |
| D2 8/11 二 | 建议书收口、投标书 | 上午：建议书交叉评                                      | 建议书 V1.0；投标书骨架     |

|           |       |                                             |                                         |
|-----------|-------|---------------------------------------------|-----------------------------------------|
|           | 开工    | 审、定稿提交；下午：技术标 14 章分工，各章四段式起草首轮              | （目录+分工+写作纪律）                            |
| D3 8/12 三 | 投标书量产 | 上午集中写作，下午交叉评审；承诺类段落（指标、工期、验收标准）一律人工定稿       | 投标书 V0.9（全部章节有稿）；项目计划 v1 与风险登记册 v1 成形   |
| D4 8/13 四 | 述标与签约 | 上午：8 分钟述标答辩；下午：逐条核对需求、锁定 MVP ≤5 项，签署《需求确认书》 | 需求确认书签署；关 1 门禁材料齐套（建议书/投标书/澄清记录/计划/风险册） |

本关最容易踩的坑是把四天全花在投标书上，最后一天仓促签确认书。正确顺序相反：D3 晚之前投标书必须到 V0.9，D4 整天留给述标与需求确认——确认书签得细，第 2 关就少返工。等待评审结果的空档不要闲着：访谈提纲、SRS 模板、spec.md 骨架都在这时备好，这是第 2 关两天能打完的前提。

### 9.1.3 第二关（D5~D6）：需求与规格

分工建议（按 4 人组）：需求负责人 1 人——SRS 主笔与冻结签字人，主持 MoSCoW 定级会与澄清裁决会，对需求基线负全责；AI 工程师 1 人——spec.md 生产、check\_spec 脚本维护、提示词库管理，要求全组最强的提示词能力，但无需求裁决权；业务分析师 1~2 人——访谈主谈与记录、RTM 编织、来源条款核对；质量员 1 人（可由需求负责人兼）——评审组织、检查单执行、配置库管理。核心原则：生产与评审不同人，自己写的段落自己不审，AI 产物同样适用。

| 日次        | 主线任务         | 关键动作                                                                                                 | 当日出口标准                   |
|-----------|--------------|------------------------------------------------------------------------------------------------------|--------------------------|
| D5 8/14 五 | 访谈与故事        | 上午：用户访谈（真实用户或虚拟画像，记录含原话证据）；下午：用户故事整理、MoSCoW 定级会、SRS 章节目录与分工钉死                                        | 访谈记录；故事地图；优先级表 V1；SRS 骨架 |
| D6 8/17 一 | SRS 量产、澄清与基线 | 上午：SRS 章节并行写作（六要素纪律）、spec.md 同步生产；下午：AI 反向澄清 ≥10 问、裁决会（限时 90 分钟、每条 8 分钟）；晚：RTM v1 编织、评审、V1.0 冻结与基线公告 | SRS V1.0 基线；澄清记录；RTM v1  |

冻结纪律：D6 晚的 V1.0 冻结是分水岭——冻结后任何人不得直接改 SRS，只能通过澄清-裁决流程改。这条纪律第一天就要宣布，否则澄清记录会沦为『边改边问』的流水账。

进度风险与预案：① 访谈对象约不到——启用虚拟画像访谈（教师扮甲方），但 RTM 来源列必须标注『画像访谈』，不得伪装真实用户；② AI 初稿出不了 20 条以上 FR——不追求条数达标，FR 下限 15 条但必须条条有来源，条数不足说明题目范围小，向指导教师报备后按实际走；③ 裁决会开不完——超时条目降为『待澄清』进风险登记册，不阻塞冻结，但★条款相关的澄清必须当会裁决。

### 9.1.4 第三关（D7~D8）：设计与计划

| 日次 | 主线任务 | 关键动作 | 当日出口标准 |
|----|------|------|--------|
|----|------|------|--------|

|           |           |                                                                                                                           |                     |
|-----------|-----------|---------------------------------------------------------------------------------------------------------------------------|---------------------|
| D7 8/18 二 | 概要设计日     | 上午：HLD 骨架（章节目录+模块职责表+接口清单）、技术/数据/部署三架构图定稿；下午：OpenAPI 契约骨架、ADR 决策点识别与分工                                                    | HLD V0.5；契约骨架可解析    |
| D8 8/19 三 | 详设、计划与评审日 | 上午：DLD 模块五件套并行、数据库概念/逻辑/物理三段推进、ADR 签署（≥2 份含被否决方案）；下午：plan.md+tasks.md 原子任务、测试计划（测试左移）、设计评审（checklist 快检）、RTM v3 挂接、设计基线冻结 | 设计基线公告；计划族齐套；RTM v3 |

设计关只有两天，靠的是三个提前量：规则文件与文档模板在第 2 关就备好；ADR 决策点在需求评审时就开始识别，D8 上午只是落笔；AI 量产章节初稿、人工专攻决策与接口契约。ADR 不要等设计写完再补——决策当场记录，被否决的方案和理由一并写进文档，『因为我会这个』不是理由。

### 9.1.5 第四关（D9~D15）：研发冲刺

两个迭代：迭代一（D9~D11）打地基+首批功能；D11 末中期评审（范围锁定点——允许砍功能，禁止砍流程）；迭代二（D12~D15）功能收尾+质量冲刺。全程 PR 门禁五环：AI 生成→提交者自查→队友 Approve→CI 绿→合并，AI 写的代码与人工代码同标准评审。

| 日次         | 主线任务      | 关键动作                                              | 当日出口标准          |
|------------|-----------|---------------------------------------------------|-----------------|
| D9 8/20 四  | 环境与骨架日    | 仓库、compose、CI、契约骨架四项并行；故意提交一次坏代码，验证 CI 真的会拦人      | CI 绿；骨架端到端可跑    |
| D10 8/21 五 | 数据底座日     | 容器化、GPU/CPU 双模式、契约测试骨架、项目状态结构；AI 起草配置，人工核对门禁是否真拦人 | 底座收口；影像接入链路三通   |
| D11 8/24 一 | 首批功能+中期评审 | 全自动分割链路贯通；下午末：中期评审（增量 Demo+范围锁定，允许砍功能、禁止砍流程）      | 中期评审通过；范围锁定公告   |
| D12 8/25 二 | 分割模式攻坚 I  | 交互分割链路（点提示→掩膜→矢量回传）；PE-01 首次实测，超标当日定位（分段计时埋点）     | 交互链路可演示         |
| D13 8/26 三 | 分割模式攻坚 II | 文本提示分割、批量处理；精度初测（AT-01 冻结影像集）                     | 三模式全部可演示        |
| D14 8/27 四 | 后处理与开放接口  | 矢量化三格式、撤销栈、QGIS 插件联调；降级演练首轮（真杀进程，不是模拟）            | 功能全量可用；降级预案验证过  |
| D15 8/28 五 | 收尾冻结日     | 覆盖率补齐（核心模块 ≥70%）；RTM v4 回填；封版检查；演示彩排两遍            | 迭代验收；代码冻结；交付包雏形 |



弹性规则：D12~D14 为高压日，允许用晚间与中间周末弹性吸收，但 D15 的冻结不动——冻结日滑动是项目失控的第一个信号。任何一天燃尽落后超过 1 天，当晚启动砍范围评审（非 MVP 项让位），不等中期评审。

人机协同要点：每日计划让 AI 按看板状态起草、人工调整——AI 排计划不懂人的疲劳与协作成本；迭代复盘的数据图由脚本自动生成，复盘会只讨论不做表；变更一律走『改 spec→同步 SRS→改 plan/tasks→改代码』流程，先改代码后补文档等于没有流程。

### 9.1.6 第五关（D16~D17）：质量门禁

总原则四条：①测试设计必须在窗口开始前完成大半——D15 之前用例集与冻结影像集已就位，窗口内只执行与补缺；②严重缺陷不过夜——每天 17:00 缺陷清零会，严重级当天必须进修复中；③故障演练预约制——提前与教师约好演练窗口，临时约必撞车；④D17 下午只写报告不执行新测试——报告要有完整数据，数据要有截止时间。

| 日次         | 主线任务          | 关键动作                                                                                              | 当日出口标准                                 |
|------------|---------------|---------------------------------------------------------------------------------------------------|----------------------------------------|
| D16 8/31 一 | 三级测试+专项日      | 启动会分工（测试负责 1、执行 2、专项 1）；冻结集登记启封；上午 UT 全量收官、IT/ST 首轮并行；下午精度专项冻结集实测、k6 压测四场景；红队自测窗口开启（只拿 SRS 与系统入口） | UT/IT/ST 数据在案（样例组：UT 312 条全过）；压测原始日志归档 |
| D17 9/1 二  | 红队收网+演练+回归+报告 | 上午：红队自测收网与争议裁决、故障演练三连；下午：全量回归、报告定稿（数据截止 12:00）；助教并行三审计（文档/可移植性/AI 使用）                             | 《测试报告》V1.0；质量特性自评表；三审计问题单              |

样例组的偏差教训值得借鉴：压测曾因等修复版部署滑期半天——压测依赖系统版本冻结，版本不冻结压测白做；报告编写原计划一天、实际半天完成——前提是用例数据每天都在归档，报告只是汇总不是补数。两天窗口的每一寸都靠 D15 之前的准备换回来。

### 9.1.7 第六关（D18, 9/2 三）：交付验收日

- 09:00–10:00 验收演示：按验收动线演示 5 项 MVP 与 ★ 条款专项，UAT 表逐行过；
- 10:00–11:00 验收汇报与答辩：10 分钟演示与 UAT、5 分钟价值论证、5 分钟个人代码走查；
- 11:00–12:00 验收签署：《验收报告》双方签署，遗留事项与约定落纸面；
- 14:00–15:30 复盘定稿：计划 vs 实际偏差、AI 协作量化数据、技术债台账，复盘报告 V1.0；
- 15:30–16:00 归档移交：配置库六目录归位、全库哈希生成、移交签署、配置库只读化；16:00 后任何人不再碰库。

三审计（文档审计、可移植性批量验证、AI 使用审计）由助教在 D17 并行完成，D18 上午直接带问题单进场。D17 晚务必按 8.6 节做最后一遍干净环境自测——在助教之前先跑一次，跑不通连夜修。

### 9.1.8 每日通用节奏（全六关适用）

- 09:30 站会 15 分钟（昨天/今天/阻塞三件事，每人 1 分钟）；
- 上午集中生产——AI 会话黄金时段，安排最难任务；



- 下午联调、评审、测试——协作类活动集中，人在都在；
- 17:00 前完成当日 PR 与文档提交，给评审和 CI 留时间；17:30 更新看板与燃尽数据；
- 晚间 30 分钟归档：Prompt 日志、翻车记录、配置库归位，当日事当日毕。

### 9.1.9 本节清单

- 全程日历与本组版逐日表在 D1 前排定并张贴；
- 每关门禁材料在窗口最后一天的上午齐套，下午只做评审不赶工；
- 严重缺陷不过夜、报告半天天条执行；
- 里程碑滑移有复盘，教训进手册。

## 9.2 答辩问题库：六关合集

本节汇集历届评审与老师扮甲方时的 20 个高频提问，按『问题—考察点—合格应答要点』三栏组织。答辩前全组应把这份问答集过一遍，并对自己的 SRS 做一次模拟抽查。

### 9.2.1 需求理解类（问题 1~7）

- 问 1：你们系统的用户有哪几类？各自的核心诉求一句话概括。——考察角色识别。合格应答：五类角色脱口而出，每类诉求一句话（如浏览者=零培训看图），能指出复核员是访谈挖出来的隐含角色。
- 问 2：招标文件 5.4.2 条★款你们怎么落实的？——考察星标追踪。合格应答：FR-01.1/01.2 照录，分片断点续传 $\geq 10\text{GB}$ ，AC-01.1 有 Given/When/Then，RTM 第 1~2 行可追踪。
- 问 3：什么是『只下载不上传』？为什么要这条？——考察红线理解。合格应答：数据合规红线，5.6.3(2)；遥感数据涉密管理要求，平台不得反向写入下发系统；已写入 FR-01.2 备注与 2.3 约束。
- 问 4：FR-07.1 人工修正为什么不是 MVP？——考察优先级依据。合格应答：砍掉测试的访谈结论（可导出到桌面 GIS 替代），但因与精度闭环相关列入二期，访谈纪要 3.2 节有记录。
- 问 5：你们的 MVP 是哪五个功能？砍掉的依据是什么？——考察 MVP 管理。合格应答：FR-01.1~01.6、FR-02.1、FR-03.1/03.2、FR-04.1、FR-06.1；依据=需求确认书签署范围+MoSCoW 定级记录。

- 问 6：招标文件没提样本版本管理，你们为什么写进 SRS？——考察需求来源纪律。合格应答：来源=CL-04 澄清裁决+第二次访谈 IV-09；已在 RTM 来源列标注，非镀金——有甲方确认的澄清记录为证。
- 问 7：浏览者能下载成果吗？——考察权限细节。合格应答：不能，仅在线浏览；依据=FR-10.1 权限矩阵第 5 行，该矩阵经 CL-06 澄清确认。

### 9.2.2 非功能需求类（问题 8~13）

- 问 8：PE-01 交互响应≤3 秒，你们怎么验证？——考察可测试性。合格应答：AC-PE-01 已写验收场景：Given 常规作业机（配置见 5.7.1）、When 执行标注操作、Then 95% 操作响应≤3 秒；测试方法=秒表抽样 50 次或自动化埋点。
- 问 9：1km<sup>2</sup>≤10 分钟这个指标，机器配置差怎么办？——考察前提假设。合格应答：指标绑定 5.7.1 规定的基准硬件配置，配置不足属建设单位依赖项（2.4 假设），SRS 已声明。
- 问 10：PE-06『1+30』是什么意思？怎么来的？——考察指标理解。合格应答：1 管理员批量下发+30 作业员并发提交；来源=第三次访谈场景逼问，纪要 3.1 节。
- 问 11：可用率≥99% 你们打算怎么测？一个月工期测得出来吗？——考察指标可操作性。合格应答：验收期按 GB/T 25000 可用性定义做 7×24 小时试运行统计+故障注入演练；长期指标以试运行外推并写进验收方案，这是行业通行做法。
- 问 12：日志留存你们写 180 天，投标方案写的 3 年，到底听哪个？——考察偏离管理。合格应答：SRS 从严基线写招标下限 180 天，投标 3 年为正偏离，实施按 3 年执行；偏离关系在 FR-09.1 备注列声明。
- 问 13：PE-09 降级≤30 秒，降级是什么意思？——考察术语严谨。合格应答：GPU 资源不可用时自动切换到 CPU 推理模式的切换时延；术语表 3.1 节有定义，AC-PE-09 有切换场景。

### 9.2.3 过程与协同类（问题 14~20）

- 问 14: SRS 里哪些内容是 AI 写的? ——考察人机边界。合格应答: 结构化初稿(功能清单表、AC 模板填充) AI 起草+人工审定; 边界判定、约束出处、优先级裁决人工写; 采纳率 58%, 度量表可查。
- 问 15: AI 澄清提了 12 条问题, 你们采纳了几条? 驳回了什么? ——考察澄清质量。合格应答: 采纳 9 条、部分采纳 1 条、驳回 2 条(如 CL-08 属 AI 误读 5.4.4 条款, 驳回理由记录在裁决表)。
- 问 16: check\_spec 体检报告 V1 到 V2 有什么变化? ——考察质量收敛。合格应答: V1 报告 24 项中 6 项未通过(缺 AC 编号 3 处、量化指标无单位 2 处、术语未定义 1 处), V2 全部通过; 两份报告均在配置库。
- 问 17: 如果甲方现在改需求——样本库要支持视频数据, 你们走什么流程? ——考察变更管理意识。合格应答: 不直接改; 登记变更申请→评估影响(RTM 波及分析: SRS/spec/设计/测试四层)→甲方签认→升版本号+修订说明→体检脚本复跑。
- 问 18: 你们组谁对需求最终负责? ——考察责任机制。合格应答: 需求负责人(点名)签字冻结 V2.0; AI 产物由审定人签字, 审定人对内容负责, 不存在『AI 写的所以我没看』。
- 问 19: SRS 和 spec.md 内容不一致时以哪个为准? ——考察双轨制纪律。合格应答: spec.md 为机器可读的源头基准, SRS 为法定交付文档; 两者由同一结构化数据生成/同步, 体检脚本含一致性比对项; 发现不一致视为缺陷, 24 小时内修复。
- 问 20: V1.0 到 V2.0 改了哪六类问题? ——考察复盘深度。合格应答: 覆盖遗漏(FR-05.1 补入)、量化缺失(3 处补单位)、口径歧义(CL-09 面积加权)、来源缺失(2 条补 IV 编号)、术语未定义(降级)、编号断档(FR-04.3 重排)——详见 4.13 节复盘。

## 9.2.4 问题库的使用

本问题库按评审真实高发问题整理, 每问给『考察点+应答要点+自杀答案』。答辩前全组做两轮模拟: 第一轮组长主答, 其他人补充; 第二轮评委(指导教师或邻组扮演)随机抽问, 任何人被点到都要能答。设计答辩的潜规则: 评委不是在考文档,

是在考『这组人是不是真的做了这些设计』。

### 9.2.5 需求追踪类（1~8）

- 1. 抽一条 FR 给我走完它的设计链路。考察点：RTM 真实性。要点：1 分钟内翻到 HLD/DLD/表/端点/测试。自杀答案：现场搜索文件夹。
- 2. 你们的 MVP 是哪几条？为什么锁定这些？要点：背出 11 条编号+需求确认书的锁定逻辑。自杀答案：『大概是最核心的那些』。
- 3. 这个模块（如 M-05）对应哪几条需求？要点：模块-需求域映射脱口而出。自杀答案：翻映射表现场找。
- 4. 有没有无需求来源的设计？（反向核查）要点：坦然回答『反向核查过，十模块均有来源』，如有裁剪项说明理由。
- 5. ★款和普通条款在设计上待遇有何不同？要点：★款有专项设计+专项测试+演示预案（如 FR-05.2 的队列并发度=1）。
- 6. PE-01 的 3 秒怎么保证？要点：预算分解（推理 $\leq 2s$ +渲染 $\leq 1s$ ）+三个机制（常驻/裁剪/独立通道）+测法（50 次抽样 P95）。
- 7. 需求变更了怎么办？要点：CCB 流程+联动清单+RTM 同步，举一个真实变更例子。
- 8. SRS 里哪个需求你们最想推翻？要点：诚实指出一条并给出论证过程（哪怕结论是接受）——展示判断力而非顺从。

### 9.2.6 架构决策类（9~15）

- 9. 为什么选 PostgreSQL 不选 MongoDB？要点：ADR-001 三条否决理由（空间函数弱/关联模型不合/运维栈统一）。
- 10. 被你们否决的方案里，哪个最有可能翻盘？要点：ADR-003 的 Triton（二期 GPU 集群场景记录在案）——展示决策的时效意识。
- 11. GPU 独占队列会不会造成浪费？要点：这是需求条款（FR-05.2）而非设计偏好；交互走独立通道已缓解。
- 12. 单点在哪？怎么办？要点：单进程服务→容器重启策略；GeoPackage 无并发写保护→单会话独占。诚实列单点、并说明为何本期接受，是加分项。

- 13. 为什么不用微服务？要点：4人组18个工作日总工期的运维约束+模块边界纪律可实现同等解耦——朴素优先原则。
- 14. 模型精度达不到80%/85%怎么办？要点：阈值是验收标准不是设计承诺；设计提供的是可测性（冻结集+面积加权算法）；不达标走澄清调整阈值或换模型。
- 15. 设计里最后悔的一个决策？要点：准备一个真实的（如v1的队列三划分过细，v3改为四队列按资源类型分）。

### 9.2.7 过程与 AI 类（16~23）

- 16. 哪些是 AI 写的？你们怎么审的？要点：分工矩阵（37.1）+审查记录+采纳率数据。
- 17. AI 犯过什么错？要点：编造 CL 编号、非法状态码、数值漂移——各配修复与预防（4.13 节案例集就是答案库）。
- 18. 你们的 rules 文件管住了什么？要点：白名单/编码规范/错误码约定，给一条 CI 拦截实例。
- 19. 提示词资产在哪？要点：prompts/ 目录+修订记录，现场翻一个。
- 20. 没有 AI 你们能做出来吗？要点：能，慢3~5倍；AI改变的是效率与覆盖面，不是能力本身——方法都在人工章节里。
- 21. 小组怎么分工？有人划水怎么办？要点：角色互备矩阵+看板数据说话（个人任务移动记录）。
- 22. 评审前你们自己查出了多少问题？要点：体检报告数字（发现n项、严重m项、整改率）——自查密度即过程质量。
- 23. 设计文档以后还会改吗？要点：会，走变更流程；基线是承诺的锚点不是僵化的借口。

### 9.2.8 压力与边界类（24~30）

- 24. 工期砍一半，你先砍什么？要点：砍非 MVP 44 条 FR 的实现（设计保留），保 MVP 5 项+★款+测试闸。

- 25. GPU 坏了两周怎么办？要点：CPU 降级路径+云端备用预案（风险登记册 R-01），首日验证过降级可用。
- 26. 甲方坚持要加训练功能怎么办？要点：变更评估（工作量/对 MVP 的冲击）→二期演进清单→正式变更单；不口头答应。
- 27. 评委：我觉得你们选型不对。要点：先问评委的约束假设（『您担心的场景是……？』），再用条款回应；不顶撞不顺从，回到需求裁决。
- 28. 测试覆盖率 70% 够吗？要点：条款来源（MTN-03）+核心模块 85% 的分层+覆盖率之外的契约/精度/性能测试组合。
- 29. 你们的设计和澜图样例什么关系？要点：方法论继承+领域内容重做（8.15 节迁移表），指出三处刻意不同。
- 30. 如果重来一次，任务 3 你会改变什么？要点：真实的反思（如『RTM 应该从 D2 就建，而不是设计完成后补』）。

### 9.2.9 流程类（8 问）

- 1. 你们的 PR 门禁有哪五环？哪一环拦截过最多问题？（好答案：报得出环数与真实拦截数据）
- 2. 挑一个 PR 讲它的完整生命周期。（好答案：像 3.4 节那样有时间线与证据链）
- 3. CI 五道闸分别拦什么？为什么这个顺序？（好答案：快在前慢在后、失败即停的成本逻辑）
- 4. 你们发生过几次变更？走一遍流程。（好答案：报得出 CR 编号与四步证据）
- 5. 变更如果漏了闭环会怎样？你们怎么防？（好答案：讲分页反例或本组真实经历+闭环验证三项）
- 6. RTM 怎么保证不失真？（好答案：即时登记+check\_refs 脚本+迭代体检）
- 7. 站会开过几次？解决了什么真实阻塞？（好答案：举得出具体阻塞与处理结果）
- 8. 燃尽图出现过平台期吗？怎么处理？（好答案：承认并讲拆任务或砍范围的决策）

### 9.2.10 质量类（8 问）

- 9. 覆盖率是多少？怎么防注水？（好答案：报总体/核心/增量三个数+注水识别清单）
- 10. 性能指标实测多少？在什么环境测的？（好答案：P95+环境+输入哈希）
- 11. 精度怎么测的？参照真值哪来的？（好答案：冻结影像集+标注流程+口径定义）
- 12. 做过故障演练吗？发现了什么？（好答案：讲 kill Worker 与 D-06 式发现）
- 13. 安全红线是哪四条？各被触发过吗？（好答案：背得出+报得出扫描记录）
- 14. 技术债还剩几笔？为什么可以留？（好答案：报得出编号与三问决策）
- 15. 一次通过率 68% 是高是低？（好答案：能辩证讲拦截即排毒）
- 16. 测试先行真的做到了吗？（好答案：承认哪些任务是先行的、哪些是同 PR 的，诚实+数据）

### 9.2.11 AI 协同类（8 问）

- 17. AI 采纳率多少？怎么统计的？（好答案：报数+定义+数据源）
- 18. 最有价值的提示词模式是哪个？（好答案：讲得出版本演进，如 P-04 v1→v3）
- 19. AI 翻过最严重的车是什么？（好答案：按翻车记录讲全字段，含纠正措施的后续验证）
- 20. 怎么防 AI 编造 API？（好答案：R-07+生成时标注来源+CI 拦截数据）
- 21. 哪些环节坚决不用 AI 主导？为什么？（好答案：安全/精度结论/决策，讲责任原则）
- 22. 长会话怎么管？（好答案：15/20 轮预算+context pack+交接摘要）
- 23. 规则文件怎么验证对 AI 生效？（好答案：讲规则探测）
- 24. AI 估算的数据可信吗？（好答案：FR-03 教训+打折公约）

### 9.2.12 协作与迁移类（6 问）

- 25. 主副修怎么排的？副修做了什么实事？（好答案：报得出副修的 PR 与 bug 修复记录）

- 26. 联调发现的最有价值问题是什么？（好答案：讲 D-02 式单位错配+联调单）
- 27. 如果换一道题，哪些资产直接能搬？（好答案：3.7 节三分法）
- 28. 新人加入你们组，第一天做什么？（好答案：onboarding 清单+十分钟跑通实测）
- 29. 组内分歧最大的一次技术决策是什么？（好答案：讲 ADR 与裁决过程）
- 30. 这个冲刺如果重来，最先改什么？（好答案：具体、来自复盘会记录，而非套话）

### 9.2.13 覆盖与闭环

- 问：每条需求都有测试吗？证明给我看。答：RTM v5，24 FR + 16 NFR 全部挂接，现场抽 3 条顺藤摸瓜（需求→用例→执行记录→结果）。
- 问：P0 用例为什么是 34 条？答：MVP 5 项核心需求 × 主流程+异常流，加 ★ 款 13 条全覆盖，去重后 34 条——推导过程在测试设计 1.3 节。
- 问：首轮 ST 有 3 条没过，你们的系统是不是没做完？答：首轮 61/64、修复后回归 64/64——首轮失败+修复+回归恰是质量工程的正常叙事，3 条缺陷的根因与修复见缺陷清单 BUG-121~123。

### 9.2.14 精度与性能

- 问：完整率 84.2% 怎么算的？5 景样本够吗？答：面积加权公式见 SRS 附录，5 景覆盖五类典型地表且已冻结登记；样本量论证在精度报告——同时这是教学周期的约定口径，工程上按统计功效扩展。
- 问：阈值从 0.30 降到 0.24，是不是为了凑指标？答：这是 CR-001 正式变更——首轮完整率 76% 未达标触发的方案调整，走了完整变更流程（spec→SRS→PR#52→复测），代价是正确率 91.5%→88.6%，在 SRS 预留的 85% 门槛内。变更记录可查。
- 问：为什么面积量算在 4490 而不是 4326？答：经纬度坐标直接算面积有随纬度变化的系统误差，量算必须在投影坐标系做——4326 只用于 Web 输出兼容。



- 问：并发 8 的时候 P95 是 7.9 秒，超标了吧？答：PE-01 的并发口径是 1 推理+30 在线，并发 2 档 P95 2.6s 达标；并发 8 已超设计边界，我们实测出边界并给了容量建议（单节点交互并发 $\leq 2$ ，每 2 路并发一个 Worker 副本）——测出边界正是压测的目的。
- 问：P95 怎么统计的？样本量多少？答：每档压测持续 X 分钟、样本约 N 次，k6 内置分位数统计，原始日志在压测报告附录——口径三层下钻随时可查。

### 9.2.15 安全

- 问：高危清零怎么证明？答：SCA 终扫报告（47+213 依赖、高危 0）+ 渗透记录（SQL 12 点、XSS 8 点全拦截）+ 中危 7 项书面缓解论证。
- 问：越权（BOLA）怎么保证所有端点都查了？答：权限矩阵  $\times$  6 端点逐格回归用例，BUG-133 修复（PR#76）时统一加了资源级鉴权中间件并全端点回归——自动工具测不出这类漏洞，必须人工定向测。
- 问：等保 33 项都符合吗？答：32 项符合、1 项部分符合（日志留存策略已配置但未满 6 个月留存期，按合同条款说明补救计划）——部分符合如实标注，自评表 2.2 节。

### 9.2.16 破坏性自测与故障演练

- 问：红队自测你被攻破了几条？答：红队 30 条用例命中 5 条有效，其中 2 条严重（上传 OOM、越权）——我们赛后把五类攻击向量做成了 6 端点  $\times$  5 向量的回归检查单。
- 问：降级态 14.8 秒之争你们怎么赢的？答：PE-09 原文约定降级态单轮 $\leq 30s$  且降级提示可见，14.8s 在设计内——SRS 措辞精度在争议时就是防线。
- 问：故障演练最快 4 分 12 秒，运气还是实力？答：日志索引表 + 操作卡 + 错误消息自带修复命令，三者叠加的结果；最慢那次 10 分 26 秒里，有一半时间花在服务恢复之后继续排查数据一致性上——那不是浪费，是本次演练最有价值的动作。

### 9.2.17 过程与 AI

- 问：你们的缺陷密度 2.5/KLOC 说明什么？答：低于往届同类项目 3~6 的区间，说明任务 4 的测试护栏起了作用；分模块看缺陷聚集在复杂度热点，符合经验定律。
- 问：AI 在测试里干了什么？答：分工矩阵 13 行——用例起草（P0 34 条中 28 条）、脚本生成（效率 25→8 分钟/条）、失败初筛（首候选命中 60%）；红线：冻结集精度全人工、pass/fail 人判、结论人写。
- 问：AI 翻过车吗？答：翻车记录 R-07 镜像断言——面积计算的断言复用被测函数导致恒真，人工抽测才发现；此后断言独立性进入库质检。另外抓到 1 次 AI 日报虚标，此后日报强制与执行系统 diff。
- 问：怎么证明不是 AI 替你们考的试？答：每人手写 ≥3 条用例并能讲设计依据；精度测试全人工有登记；红队攻击用例由人编写（AI 只辅助向量清单）。

### 9.2.18 文档与交付

- 问：测试报告哪个数字你最没底气？答：（诚实回答 + 该数字的口径与原始记录位置）——此问考的是你对数据质量的自知。
- 问：遗留问题是什么？答：Deferred 缺陷 2 条挂技术债台账（含触发条件与责任人）+ 等保日志留存 1 项部分符合——报告 2.8 节全部披露。
- 问：换一批人接手你们的测试体系，跑得起来吗？答：用例库有入库质检标准、执行有规程、看板有节奏、报告有骨架——移交的核心资产是机制不是文档。

### 9.2.19 应答总原则

- 四段式：问题—数字—出处—含义，缺一不可；
- 被问倒：『需要核实原始记录，会后补充』是专业回答，现场编造是事故；
- 失败问题必讲四段：事实→根因→修复→收获；
- 全队都可能被抽，人人过题库。

### 9.2.20 技术型评委

- 问 1：IoU 0.78 在哪个数据集上测的？意图：指标有效性。要点：冻结影像集（D15 冻结），口径含切片方式与类别定义，质量特性自评表 3.2 节。

- 问 2: P99 7min12s 接近阈值, 超大数据怎么办? 意图: 边界认知。要点: DEBT-01 触发条件与偿还方案, 运维手册监控项。
- 问 3: 全自动分割失败时系统行为? 意图: 异常路径。要点: 降级策略(交互模式接管)+可靠性测试记录。
- 问 4: 权限矩阵 6 个端点怎么保证不漏? 意图: 方法可靠性。要点: PR#76 案例——AI 生成+人工对 SRS 逐条核对+评审打回补测试。
- 问 5: 4096 截断为什么不是报错? 意图: 设计权衡。要点: 丁的 PR#79 应答(8.9 节样本)。
- 问 6: 数据库迁移失败怎么回滚? 意图: 运维成熟度。要点: 迁移幂等设计+备份命令+手册第 7 章故障卡。
- 问 7: 坐标系处理有没有坑? 意图: 探翻车。要点: 坦白 300 米偏差翻车+此后加的对照校验(E 系列素材)。
- 问 8: 并发上限多少、依据? 意图: 性能口径。要点: PE-02/PE-03 压测场景与边界条件。
- 问 9: 模型更新(SAM 出新版)怎么办? 意图: 可维护性。要点: 模型抽象层设计+ADR-002 的可替换论证。
- 问 10: 导出的 Shapefile 外业软件兼容性验证过? 意图: 端到端真实。要点: QGIS/ArcGIS 双开验证记录(UAT-5 证据)。

### 9.2.21 管理型评委

- 问 11: 计划 480 实际 512, 教训是什么? 意图: 估算能力。要点: 偏差源表四行+校准系数+『再做一遍报 496』。
- 问 12: CR-001 为什么没在设计关拦住? 意图: 流程有效性。要点: 归因于接口预研缺失+I-01 建议(设计关预研检查点)。
- 问 13: 变更流程会不会太重、拖慢开发? 意图: 流程感。要点: CR-001 全流程 6h 的实测成本 vs 零漂移收益, E-02。
- 问 14: 四人分工工作量均衡吗? 意图: 团队健康。要点: 主副修机制+PR 分布数据+匿名互评摘要。

- 问 15：每日站会 15 分钟真坚持了吗？意图：过程真实性。要点：站会记录索引+三次因攻关顺延的如实说明。
- 问 16：红队自测命中 5 条，是红队强还是系统弱？意图：横向认知。要点：命中率解读+我们从攻击用例学到的两点。
- 问 17：债务为什么不还完再交付？意图：工程判断。要点：收益成本风险权衡+签认链——关键是甲方知情同意。
- 问 18：验收没通过的话你们的预案？意图：风险思维。要点：遗留事项整改路径+复验机制（报告签署页条款）。
- 问 19：文档 47 份是不是过度？意图：比重判断。要点：GB/T 8567 对照表+每份文档在审计/验收/复盘中的实际被用记录。
- 问 20：课程六关哪关设计最该改？意图：批判思维。要点：I 系列精选 1~2 条+依据数据。

### 9.2.22 怀疑型评委

- 问 21：这些文档是不是 AI 批量生成的？意图：辨替考。要点：四层声明+哪些章节 AI 半自动（结构化）哪些人工（叙述/结论）+三遍预审裁决记录。
- 问 22：+61h 的账，参照系太主观了吧？意图：打口径。要点：承认  $\pm 20\%$  误差给区间+任务粒度对照方法+即使下限为正的结论。
- 问 23：保留率 44% 是不是说明 AI 代码不行？意图：诱导否定。要点：不辩护——44% 恰是把关有效的量化证明+打回归因 4/3/2。
- 问 24：日志完整率 97.4%，丢的 2.6% 是什么？意图：探诚信。要点：PR#36 全过程坦白+检查点从 PR 挂 commit 的整改。
- 问 25：演示数据选过吧？意图：演示诚信。要点：坦白 3 景备选上上策略+冻结集外原则+可现场换数据重跑（有底气才敢说）。
- 问 26：45 分钟基线谁测的、怎么测的？意图：基线真实性。要点：GIS 专业人员实测 42 分钟记录+访谈+文献三重佐证。
- 问 27：你们的 PPT 数字和文档对得上吗？意图：抓不一致。要点：逐数溯源核对表的存在+锚点扫描方法+欢迎现场抽查。

- 问 28：翻车 5 起是不是报喜不报忧？意图：探披露。要点：翻车记录全入库可翻+最严重一起（300m）的完整复盘。
- 问 29：复盘 E/I 是你们写的还是 AI 写的？意图：红线探测。要点：萃取工作坊过程记录+AI 角色（预筛/打磨）与人裁决的边界。
- 问 30：给这门课的 AI 使用打个分？意图：元认知。要点：账本数据说话——4:1 与 0.6:1 的边际结论+使用预算倾斜规则。

## 9.3 课堂练习与实训题

本节供学生自测与教师组织讨论使用。自测题侧重“记住并会做”，讨论题侧重“判断与权衡”——后者没有标准答案，给出的只是参考要点。教师可从讨论题中抽取答辩追问。

### 9.3.1 自测题（附答案要点）

#### 第一部分：概念与方法

- 1. 人机分工三原则是什么？——AI 产出只是初稿；事实必须溯源；人是最终责任人。
- 2. 标书偏离表“招标要求”列为什么必须照抄原文？——让核对从语义判断变为文本比对；改写会被判未实质响应（澜图 V1 的 8 条疑似未响应主要由此而来）。
- 3. WBS 分解的两个核心规则？——按交付物分解（不写动作）；工作包  $\leq 1$  周/40 小时且 100% 覆盖。
- 4. 里程碑与准出标准的关系？——没有准出标准的里程碑只是日期；准出标准必须可核验。
- 5. 风险描述的规范句式？——“条件—后果”句式；每条风险有应对策略（规避/减轻/转移/接受）与唯一责任人。
- 6. MVP 选择的三条标准？——★条款优先；业务闭环必需；技术风险前置。
- 7. 提示词四要素？——人设、结构、素材、约束；其中素材与约束是反幻觉闸门。
- 8. skill 的三层结构与各自作用？——SKILL.md（触发与总纲）、references（按需加载的细则）、scripts（确定性检查工具）；设计哲学是渐进披露。

- 9. 四遍评审法各查什么？——事实（编造的）、逻辑（站不住的）、合规（会废标的）、立场（不像我们说的）。
- 10. rules.md、skill、tasks.md 三层各管什么？——项目宪法 / 流程方法 / 当下任务状态。

## 第二部分：实操判断

- 11. check\_bid.py 报告“某 ★ 条款未找到”，下一步怎么做？——不恐慌不改脚本凑数；人工回原文与标书双向核对，判断是真遗漏还是表述差异；多数情况下应修改标书使其包含原文关键句。
- 12. AI 生成的行业数据找不到出处，但“听起来很真”，如何处理？——删除或标注（信息待确认）；宁可少一个论据，不留一个虚构。
- 13. 述标还剩 90 秒但还有两节没讲，怎么办？——跳到收尾三句话；训练时按 7 分 40 秒收束就是为这种情况留的余量。
- 14. 评委问了一个标书之外的功能，怎么答？——不承诺标外功能；答“该功能不在本期范围，可按变更程序评估纳入后续迭代”。
- 15. 甘特图与 WBS 出现不一致，以谁为准？——都不对：先查明原因，修正后两者必须一致；图随数据走（脚本化出图的意义）。

### 9.3.2 课堂讨论题（无标准答案，附参考要点）

- 题 1：AI 写得比大多数学生好，投标书写作训练还有必要吗？——要点：评委视角、核验能力、责任承担都无法外包；“写得好”与“信得过”是两回事；训练目标从“会写”转向“会审、会判、会负责”。
- 题 2：如果招标方也用 AI 评审标书，投标策略应该改变吗？——要点：机器评审重结构与覆盖（三件套、关键词命中更重要），人的评审重信任（述标、质询仍是人的场）；双轨优化。
- 题 3：V2 只比 V1 多 4.8% 的字数，凭什么说“显著提升”？——要点：合规维度的提升不能用字数衡量；废标项是 0/1 问题；工作量省在读标与查标的机械环节。

- 题 4：MVP 辩论中，甲方坚持要第 6 项功能，工期又不许动，你让还是不让？——要点：让不是认输，是交换（范围换资源/换排期/降其他项）；一切让步进变更单，双方签字。
- 题 5：全过程留痕会不会沦为“留痕表演”？——要点：区分“为证据而记录”与“证据自然产生”；版本库时间戳、体检报告迭代是难造假的痕迹；评审看连续性不看单点。
- 题 6：本课程的 AI 使用声明制度能否推广到毕业论文？——要点：透明化是趋势；声明三要素（用了什么/核验了什么/谁负责）的通用性；争议在“实质性修改”的界定。

### 9.3.3 教师使用建议

- 自测题第一部分用于 D9 前的随堂测（10 分钟，闭卷），正确率低于 70% 的组安排补课；
- 第二部分用于查标日后的翻转课堂：每组抽一题现场作答，其他组按“四遍评审法”评判其回答；
- 讨论题用于复盘会与期末答辩追问；鼓励学生给出与参考要点不同但论证充分的答案——方法课没有唯一解。

### 9.3.4 随堂练习（每次课 15~20 分钟）

| #  | 练习                           | 对应章节   | 产出       |
|----|------------------------------|--------|----------|
| E1 | 给 5 条 SRS 原文写设计落点猜测，再对照样例找差异 | 3/4 章  | 一页对照笔记   |
| E2 | 审查一段 AI 起草的 HLD 文本（预埋 5 处缺陷） | 4/17 章 | 缺陷清单+严重度 |
| E3 | 把一段『高性能、灵活扩展』式文字改写为可验证表述     | 27 章   | 改写稿      |
| E4 | 为给定决策点写 ADR                  | 9 章    | ADR 片段   |

| #  | 练习                             | 对应章节    | 产出      |
|----|--------------------------------|---------|---------|
|    | 的被否决方案与理由<br>(20 分钟限时)         |         |         |
| E5 | 拆 5 个原子任务并互评四要素                | 10 章    | 任务卡×5   |
| E6 | 用 RTM 抽查另一组的设计文档 3 行           | 14/20 章 | 走查记录    |
| E7 | 对样例 ADR-002 做反向攻击，找出成立与不成立的攻击点 | 9/21 章  | 攻击清单+裁决 |

### 9.3.5 课后作业

- 作业 1（个人）：读样例《概要设计说明书》第 9 章，为你自己的题目列出 NFR 映射表草稿（至少 8 行）；
- 作业 2（小组）：完成 HLD 骨架（章节结构+模块映射表+选型表草稿），下节课互评；
- 作业 3（小组）：完成任务 3 全部交付物初稿，附体检报告与 RTM 双向核查记录；
- 作业 4（个人）：写一页『本组基线漂移风险自查』——对照 5.13 节预防机制逐条自查。

### 9.3.6 互评的组织

组间互评用交叉评审：A 组评 B 组、B 组评 C 组、C 组评 A 组。互评只看三件事：RTM 走查 3 行、数值一致性抽查 5 处、ADR 论证质量 1 份。互评记录（发现项+证据）交给被评组与教师各一份。互评发现的真缺陷计入发现组的平时分——教会学生『挑剔是能力』。

### 9.3.7 课堂演练（教师带做）

#### 练习 1：契约缺口制造与拦截（30 分钟）

材料：sprint\_repo。步骤：①学生故意删除 openapi.yaml 中 2 个端点的 x-fr 标注；②提交触发 CI；③观察 contract\_check 拦截信息；④修复并转绿。评量：能解释拦截



信息的每一项含义。教学目的：建立『防线真实工作』的体感。

### **练习2：评审五查实战（45 分钟）**

材料：一个预埋了 5 处问题的待审 PR（编造 API 1、边界缺失 1、编号错 1、数值无出处 1、镜像测试 1），学生分组评审，比哪组找得全。讲评时按五查清单逐类对答案。评量：找出 4 处以上合格；能讲出每处属于五查的哪一类。

### **练习3：故障演练观摩（40 分钟）**

教师演示 kill\_worker.py 演练，学生按剧本卡片逐项核对预期与实际。评量：能正确填写演练记录表，并指出时间线上切换耗时的起止点。

### **练习4：AI 对抗评审（40 分钟）**

两组互为攻防：A 组用 AI 生成一段含隐蔽缺陷的代码（允许故意诱导 AI 犯错），B 组评审找问题；然后交换。评量：出题组诱导成功且能讲清缺陷机理得满分；评审组找出全部预埋问题得满分。这是最受欢迎的一个练习，建议在迭代一中段安排。

## **9.3.8 课后作业（个人完成）**

### **练习5：为陌生模块写 context pack**

指定仓内一个学生未接触过的模块，产出 context pack 五段，并用它驱动 AI 完成一个小改动。提交：pack 原文+会话摘要+PR。评量：pack 的约定清单段是否有本题特定内容（抄样例的扣分）。

### **练习6：五问根因分析**

从自己组的真实经历中选一例 AI 翻车（实在没有的选一次评审拦截），写完整翻车记录。评量：第五问是否落到流程层；纠正措施是否可执行、可验证。

### **练习7：覆盖率注水鉴别**

给三份测试代码（分别含空断言、镜像测试、快乐路径三种注水），写鉴别报告。评量：识别准确率+每处的修复建议质量。

### **练习8：变更单起草**

给定一个虚拟变更场景（如『分页大小 20→50』），起草完整变更单含影响面清单。评量：影响面是否覆盖文档与测试；回退方案是否具体。

## **9.3.9 评审考核（计入成绩）**

### **练习9：一次真实 PR 评审**

评审其他组的一个真实 PR，产出评审意见。评量：意见的五查覆盖率与建设性（指出问题+给出依据+建议方向）。

### **练习10：RTM 诊断报告**

对本组 RTM 做正向+反向诊断，输出断链清单与修复计划。评量：诊断的真实发现数（零发现需复核真实性）。

### **练习11：迭代复盘主持**

轮流主持一次复盘会，用三张图（燃尽/CI 拦截/采纳率）驱动讨论，产出改进项。评量：改进项是否具体可执行并跟踪闭环。

### **练习12：演示预案编制**

编制任务 4 验收演示的完整预案：演示脚本、计时、降级录屏、分工。评量：预案的可执行性（教师抽查一个环节让其现场预演）。

## **9.3.10 用例设计练习（对应第 3~5 章）**

练习 1：给定样例需求『FR-03.2 用户可对图斑属性进行编辑并导出』，用等价类与边界值法设计用例集（≥6 条），每条预期必须可判定。验收标准：预期无可判定措辞问题、边界值取点有理由。

练习 2：把练习 1 的用例集喂给 AI 让它以『第一评审』身份质询，记录它提出的问题，人工裁决有效/无效并写理由。验收标准：质询记录 + 逐条裁决意见。

练习 3（镜像断言识别）：给出三份 AI 生成的测试代码（课堂发放），找出其中的镜像断言与恒真断言，改写为独立断言。验收标准：能解释『被测函数改错后断言为什么变红』。

## **9.3.11 压测设计练习（对应第 13~14 章）**

练习 4：为你自己组的项目写一个 k6 压测脚本骨架：stages 分档、thresholds 含 P95 门禁、summary 导出 JSON。验收标准：脚本可运行、thresholds 与 SRS 性能条款一致。

练习 5：给定 S1 形态数据（并发 1/2/4/8 → P95 2.3/2.8/4.1/7.9），写一段 200 字的曲线解读与容量建议。验收标准：指出拐点位置与成因（排队主导）、容量建议三要素齐全。

### 9.3.12 破坏性测试练习（对应第 22~23 章）

练习 6：针对样例系统的『创建任务』接口，按五类攻击向量（边界/资源/状态/授权/歧义）各写 1 条攻击用例。验收标准：每条有攻击假设与判定标准；课后对照 BUG-131~135 自评命中率。

练习 7（仲裁演练）：课堂模拟降级态 14.8s 仲裁——两组分别扮演攻防，裁判组按 SRS PE-09 原文裁决。验收标准：双方陈词必须引用条款原文，不许诉诸『我觉得』。

### 9.3.13 缺陷与报告练习（对应第 24~28 章）

练习 8：给定 5 条缺陷单（其中 2 条故意写坏：复现步骤缺失、预期无依据），按六要素标准质检并打回。验收标准：打回理由逐条对应六要素。

练习 9：给定环节分布数据（8/5/2/3/6），画出环节×严重度矩阵并写 150 字解读。验收标准：能得出『各层护栏捕获集互补、缺一不可』的结论并给数据依据。

练习 10：填一份自评表片段（任选三大特性），每项挂证据编号，其中至少一项标『部分符合』并写补救计划。验收标准：证据编号可追溯、部分符合措辞不淡化。

### 9.3.14 综合练习

练习 11（迁移演练）：按 4.2 节六步迁移法，为你自己组的项目产出迁移决策记录（精度红线识别、专项映射表、裁剪申报）。验收标准：通过 35.6 自检清单。

练习 12（迷你演练）：教师对样例系统注入一个预设故障，小组限时 15 分钟定位恢复，全程按慌乱期纪律执行。验收标准：定位路径记录 + 操作卡使用记录 + 恢复后数据校验记录。

### 9.3.15 审计与文档（对应第 3~9 章）

- 练习 1 锚点扫描（60 分钟）：给定一份含 5 处埋雷漂移的 SRS+设计文档对（教师预制），用锚点扫描法找出全部漂移并给出根因分类。验收：5 处全中且分类正确。
- 练习 2 非执笔人实测（90 分钟）：交换组的部署手册，在干净快照上执行，记录所有『需要想一下』的点。验收：产出缺陷清单  $\geq 3$  条且分级正确。

- 练习 3 交付包体检（45 分钟）：对一份有缺陷的交付包（预制：缺 manifest、.env 漂移、无 preflight）做体检。验收：三项缺陷全检出，另附两条改进建议。
- 练习 4 AI 预审裁决（60 分钟）：用 AI 三遍预审法扫自己组的一份文档，输出疑点清单并逐条裁决。验收：疑点清单+裁决记录（真问题/误报比例）。

### 9.3.16 部署与验证（对应第 10~13 章）

- 练习 5 快照工作法（120 分钟）：从基础镜像拍快照，执行一轮五步验收点自测并计时。验收：五步打钩记录+至少 1 个真实缺口记录。
- 练习 6 preflight 编写（60 分钟）：为本组系统编写 preflight.sh，覆盖 GPU/磁盘/端口三项，失败提示含版本号与手册章节。验收：人为制造 3 种失败场景，提示全部可读可执行。
- 练习 7 收敛曲线绘制（30 分钟）：给定澜图三轮数据（4/2/0 缺口，1h52m/1h37m/1h21m），用 matplotlib 绘制双收敛图并写 100 字解读。验收：图规范+解读有机制层结论。

### 9.3.17 审计与诚实（对应第 14~16 章）

- 练习 8 三单对照（75 分钟）：给定一组日志+git log+diff 材料（预制，含 1 处后补未打标），完成三单对照并输出审计结论。验收：检出后补点，且正确引用时序交叉证据。
- 练习 9 翻车记录撰写（45 分钟）：把本组真实的一次 AI 失误写成翻车记录（会话还原/错误点/漏检点/对策四段）。验收：四段齐，对策可执行。

### 9.3.18 演示与答辩（对应第 17~22、34 章）

- 练习 10 动线设计（90 分钟）：为本组系统设计 10 分钟场景化演示动线（含时间表、MVP 映射、数据策略）。验收：通过 AI 评委质疑预演（≥5 问有答）。
- 练习 11 四段式应答（45 分钟）：全员轮流抽高频 12 问现场作答，录像回放按四段式评分。验收：每人 ≥2 题，四段齐全率 ≥80%。
- 练习 12 模拟打分与补救（60 分钟）：按评分细则对本组当前状态模拟打分，输出补救清单。验收：五维逐项有证据或补救动作，9/1 前闭环。

### 9.3.19 本节清单

- 12 道练习覆盖用例/压测/红队自测/缺陷/报告/迁移/演练全线；
- 每题有验收标准，可直接作为平时成绩依据；
- 练习 6、7、12 建议课堂现场做（对抗性环节现场效果最好）。

## 9.4 模板与提示词卡集

本节汇集任务 1 各交付物的空白模板。模板的价值在于“把方法固化成格式”：学生按格填写即不会漏项，评审按格检查即一目了然。各模板均摘自“澜图”样例并去除具体内容，供各组套用自己的题目。

### 9.4.1 项目建议书结构模板

| 节 | 标题          | 写什么                     | 篇幅指引  |
|---|-------------|-------------------------|-------|
| 1 | 项目背景与问题理解   | 痛点 3-5 个，每个含量化表述与出处     | 1 页   |
| 2 | 建设目标与建设内容   | 目标一句话可测；内容“三段式”概括       | 0.5 页 |
| 3 | 初步技术方案概要    | 总体思路 + 关键技术名词 + 技术栈，不展开 | 1 页   |
| 4 | 初步实施计划与资源估算 | 阶段划分 + 人月粗估（标注估算方法）     | 0.5 页 |
| 5 | 投资估算与效益分析   | 构成说明；效益逐条对应痛点           | 0.5 页 |
| 6 | 团队与业绩简介     | 岗位配置 + 相关业绩（化名须自洽）      | 0.5 页 |
| 7 | 风险初判与应对     | ≤4 条最致命风险，每条一句应对        | 0.5 页 |
| 8 | 结论与建议       | 给招标人的建议 + 投标意向声明 + 签署栏  | 0.5 页 |

### 9.4.2 澄清函模板

| 字段   | 填写说明           |
|------|----------------|
| 澄清编号 | Q-XX，连续编号，台账同步 |
| 招标原文 | 章节号 + 原句照抄     |
| 疑问说明 | 歧义/矛盾/缺失的具体所在  |

| 字段     | 填写说明                 |
|--------|----------------------|
| 我方理解建议 | 建议采用的解释口径（对双方公平且可执行） |
| 影响说明   | 该答复将影响我方方案的哪个部分      |
| 答复与日期  | （招标人填写）              |
| 闭环确认   | （我方）答复已归档并同步修订方案□    |

### 9.4.3 合规矩阵空白表

| 条款编号           | 性质          | 招标要求摘要  | 投标响应摘要    | 响应章节 | 偏离结论                  | 状态                    | 风险标记         |
|----------------|-------------|---------|-----------|------|-----------------------|-----------------------|--------------|
| （如<br>FR-02.1） | ★/ 一般 / 评分项 | 照抄原文关键句 | 方案要点 + 证据 | 章节号  | 无 偏离 / 正<br>偏离 / 负 偏离 | 未开始 / 已起草 / 待核验 / 已确认 | 涉报价/资质/承诺时标★ |
|                |             |         |           |      |                       |                       |              |

### 9.4.4 WBS 空白表

| WBS 编码 | 工作包（交付物导向）       | 主要内容 | 工期(周) | 责任岗位 | 交付物 |
|--------|------------------|------|-------|------|-----|
|        | （写“可验收的东西”，不写动作） |      |       |      |     |
|        |                  |      |       |      |     |

### 9.4.5 风险登记册空白表

| 编号 | 风险描述（条件—后果） | 类别                                    | 概率  | 影响  | 等级   | 应对措施（四类之一+动作） | 责任人 | 状态                    |
|----|-------------|---------------------------------------|-----|-----|------|---------------|-----|-----------------------|
| R  |             | 技术/<br>管理/<br>资源/<br>外部/<br>合规/<br>质量 | 1-5 | 1-5 | =P×I |               |     | 监控中 /<br>已应对 /<br>已关闭 |
|    |             |                                       |     |     |      |               |     |                       |

#### 9.4.6 里程碑准出标准模板

| 里程碑 | 时点 | 准出标准（可核验条件，逐条列）         | 未达标的处置               |
|-----|----|-------------------------|----------------------|
| M1  |    | 1. .... 2. .... 3. .... | （如：顺延并启动赶工预案/缩小迭代范围） |
| M2  |    | 1. .... 2. .... 3. .... |                      |

#### 9.4.7 需求确认书条款模板

| 条款              | 必备内容                          |
|-----------------|-------------------------------|
| 1 目的与依据         | 引用招标文件、澄清记录、投标文件；声明签署后冻结      |
| 2 MVP 核心功能（≤5项） | 每项：功能描述/对应招标条款/验收依据（可测）       |
| 3 范围边界          | 范围外清单（含“分期兑现”表述，避免与投标承诺冲突）    |
| 4 非功能验收基线       | 部署/安全/可靠性/合规各1条，引用招标编号        |
| 5 变更控制          | CCB 组成、申请单、评估维度、书面确认、紧急修复补登时限 |
| 6 签署            | 双方单位、代表签字、日期、份数与效力声明          |

### 9.4.8 述标讲稿骨架模板

| 时间码       | 对应评分因素        | 记忆点（一句话金句） | 必说数字 |
|-----------|---------------|------------|------|
| 0:00–0:40 | 开场定调          |            |      |
| 0:40–2:00 | 需求理解（8 分）     |            |      |
| 2:00–4:00 | 总体架构（10 分）    |            |      |
| 4:00–5:30 | 功能实现（16 分）    |            |      |
| 5:30–6:40 | 实施 + 测试（12 分） |            |      |
| 6:40–7:30 | 差异化卖点（AI 策略）  |            |      |
| 7:30–8:00 | 三句话收尾         |            |      |

### 9.4.9 会议纪要与评审记录模板

任务 1 的每个评审节点（目录确认会、红项清零会、MVP 辩论）都要留书面记录。统一模板如下，评审记录表是过程分的直接证据：

| 字段         | 填写说明                              |      |        |      |      |
|------------|-----------------------------------|------|--------|------|------|
| 会议名称/日期/地点 | 如：标书目录确认会，D2 16:00，线上             |      |        |      |      |
| 参会人/缺席人    | 角色署名（主述/记录/评审），缺席需注明原因            |      |        |      |      |
| 议题清单       | 逐条编号，与议程一致                        |      |        |      |      |
| 决议事项       | 每条决议写“结论 + 责任人 + 完成时限”（不写“讨论了……”） |      |        |      |      |
| 分歧与待办      | 未决事项写明下次处理时间                      |      |        |      |      |
| 签字确认       | 全体参会人；电子档入版本库                     |      |        |      |      |
| 评审记录表头     | 问题描述                              | 位置   | 依据     | 处置   | 复核人  |
| 编号/日期/评    | （四遍法发现的哪                          | 章节/页 | （招标条款/ | 改/不改 | （闭环签 |



| 字段 |      | 填写说明 |               |     |    |
|----|------|------|---------------|-----|----|
| 审人 | 类问题) |      | 检 查 单 条<br>目) | +理由 | 字) |
|    |      |      |               |     |    |

模板使用纪律：模板是起点不是终点——填完后必须自问“这一格的内容经不经得起评委追问”；任何一格填不出实质内容，就是该处工作没做透的信号。

```
# spec.md — <系统名称> 机器可读需求规格 # 版本: v<x.y> | 基准: SRS V<x.y> | 日期: YYYY-MM-DD # 本文件为 AI 编码与自动化体检的输入源头, 修改须同步 SRS 并复跑 check_spec ## 0. 元信息
project: <项目名称> baseline_doc: SRS V<x.y> tender_ref: <招标文件编号> confirmed_scope: <需求确认书编号> ## 1. 角色 - R1 系统管理员: <一句话职责> - R2 ... ## 2. 术语 - 术语 A: <定义> ## 3. 功能需求 ### FR-01.1 <名称> [M] [★] [MVP] source: 招标 5.4.2★ / IV-11 desc: | <含量化指标的需求描述, 禁止『等/若干』> ac: AC-01.1 ### AC-01.1 scenario: | Given <前置条件, 含数据规模与硬件前提> When <触发动作> Then <可观测、可计量的结果> ## 4. 非功能需求 ### PE-01 交互响应 metric: 95% 交互操作响应时间 threshold: <= 3s precondition: 基准硬件配置见 SRS 5.7.1 ac: AC-PE-01 ## 5. 约束与红线 - RED-1: 只下载不上传(招标 5.6.3(2), 双侧标注于 FR-01.2) ## 6. 追踪 rtm: RTM_v<x>.xlsx clarifications: [CL-01 .. CL-12] mvp: [FR-01.1..01.6, FR-02.1, FR-03.1/03.2, FR-04.1, FR-06.1]
```

模板使用三条规定：① 方括号标记（[M]/[★]/[MVP]）是体检脚本的解析锚点，不得改格式；② desc 与 scenario 用 YAML 块标量（|）书写，保持换行；③ 元信息区的版本号、基准文档、确认书编号必须与配置库实际文件一致——体检脚本 C4 项会核对。

## 9.4.10 I.1 第一次访谈提纲（业务全景，90 分钟）

1. 开场（5 分钟）：自我介绍、访谈目的说明、记录与确认方式征得同意 2. 业务全景（25 分钟）：  
- 请描述您部门的一项典型业务从发起到结束的全过程 - 一个典型工作日，您和团队各做什么？ - 目前业务中最耗时的三个环节是什么？ 3. 角色识别（20 分钟）：沿业务流逐环节问 - 这个环节谁操作？谁审核？谁看结果？ - 有没有环节是多人协作或交接的？ 4. 痛点排序（20 分钟）：  
- 如果系统只能解决一个问题，您选哪个？ - 上系统后，您最担心什么？ 5. 边界确认（15 分钟）：  
- 有哪些事是系统绝对不能做的？（红线探测） - 有哪些外部系统要打交道？ 6. 收尾（5 分钟）：总结复述、约定记录确认时限、致谢

## 9.4.11 I.2 第二次访谈提纲（流程与数据，120 分钟）

1. 流程回放（50 分钟）：  
- 以最近一次真实任务为例，从接到任务开始逐步回放 - 每步确认：输入什么？谁做？产出什么？给谁？ - 异常情况：断网/数据错/任务取消时怎么办？ 2. 数据卡片（40 分钟）：每类数据填五要素 - 来源 / 格式 / 量级 / 更新频率 / 质量要求 - 历史数据要不要迁移？保存多久？ 3. 现有工具（20 分钟）：  
- 现在用什么软件做这些事？哪里不好用？ - 有没有必须兼容或必须替代的旧系统？ 4. 收尾（10 分钟）：泳道草图核对、待确认事项清单

## 9.4.12 I.3 第三次访谈提纲（NFR 与优先级，60 分钟）

1. 性能逼问（20 分钟）：  
- 使用高峰是什么时候？多少人同时用？各干什么？ - 哪类操作最不能等？等多久算不能接受？ - 数据量最大的一次任务是什么规模？ 2. 可用性与安全（15 分钟）：  
-

系统宕机多久业务会受实质影响？ - 哪些数据不能泄露？操作要不要留痕？留多久？ 3. 优先级定级（20 分钟）：功能清单逐条 - 上线那天没有这个功能，业务能不能跑？ - 这个功能做一半（只有基本流程）能接受吗？ 4. 收尾（5 分钟）：MoSCoW 初表复述确认

### 9.4.13 卡 1：访谈记录→结论清单

用途：D5 下午，访谈记录整理为 IV 编号结论清单 输入：访谈原始记录全文 提示词：你是需求工程师。阅读以下访谈记录，提取业务结论， 要求：① 每条结论编号 IV-01 起；② 每条附对应原话引用；③ 区分『事实陈述』与『用户诉求』；④ 存疑处标 [待核实]；⑤ 不得添加记录中没有的信息。 验收要点：每条结论能回溯到原话；无 AI 脑补内容

### 9.4.14 卡 2：招标条款→FR 拆分

用途：D6 上午，功能域条款拆分为 FR 组 输入：招标文件某功能域条款原文 + 本组规则文件 提示词：将以下招标条款拆分为功能需求条目。 要求：① 编号 FR-xx.y, xx 为功能域号；② 动宾命名≤12 字；③ 每条含量化指标与来源标注 [招标 x.x.x]；④ ★条款照录原文数值；⑤ 禁止『等/若干/快速』；⑥ 每条同时给出 AC 编号。 验收要点：条款逐句有落点；数值与原文一致

### 9.4.15 卡 3：FR→AC 三段式

用途：D6，为 FR 编写 Given/When/Then 验收场景 输入：FR 条目 + 相关非功能指标 提示词：为以下需求编写验收场景。 要求：① Given 含数据规模与硬件/环境前提；② When 为单一触发动作；③ Then 可观测可计量，含数值与单位；④ 正常场景 1 个+异常场景 1 个。 验收要点：Then 可由第三方测试人员直接执行判定

### 9.4.16 卡 4：AI 反向澄清

用途：D6 下午，对 SRS 提澄清问题 输入：SRS V1.0 全文 提示词：你是甲方评审专家。阅读本 SRS，找出会导致开发方 理解分歧或无法验收的表述，提出问题清单。 要求：① 不少于 10 条；② 每条注明条款位置与歧义点；③ 按『歧义/缺失/矛盾/不可测』分类；④ 只提问不解答。 验收要点：问题具体到条款；非泛泛而问

### 9.4.17 卡 5：MoSCoW 定级辅助

用途：D5 定级会前准备 输入：FR 清单 + 访谈结论清单 提示词：对以下功能需求建议 MoSCoW 优先级。 要求：① 逐条给出 M/S/C/W 建议与理由；② 理由必须引用 访谈结论编号或招标条款；③ 标出定级存疑条目 [需人工裁决]。 验收要点：AI 只建议不定案；定案权在定级会

### 9.4.18 卡 6：SRS 叙述章节起草

用途：D6 第 1~4 章初稿 输入：招标对应章节 + IV 清单 + 本节写作要点 提示词：起草 SRS 第 x 章。 要求：① 遵循 GB/T 9385-2008 结构；② 所有数字与事实必须来自给定材料并标注来源；③ 不确定处标 [待澄清]，不得编造；④ 语言为正式技术文档风格。 验收要点：逐句有出处；无『随着...的发展』式套话

### 9.4.19 卡 7：体检报告解读

用途：check\_spec 未通过项的修复建议 输入：体检报告 + spec.md 提示词：解读以下体检报告的未通过项。 要求：① 逐项说明违反的规则与后果；② 给出最小修复方案；③ 标注哪些必须人工判断（如口径歧义）不得自动改。 验收要点：修复建议不引入新内容

### 9.4.20 卡 8：RTM 骨架生成

用途：D6 RTM v1 编织 输入：spec.md（经 extract\_fr.py 预提取） 提示词：（脚本为主，AI 兜底解析失败项）对解析失败的 FR 条目，按『编号|名称|来源|优先级|AC』五列补全，标 [人工复核]。 验收要点：脚本产物为主，AI 仅处理异常项

9.4.21 卡 9：版本修订说明起草

用途：V1.0→V2.0 修订说明 输入：diff\_srs.py 输出 + 澄清裁决表 提示词：起草版本修订说明。要求：① 每条修订含条款号、旧表述、新表述、修订依据（CL 编号/评审记录）；② 按章节排序；③ 不遗漏 diff 中的任何实质变更。 验收要点：修订条数与 diff 一致；依据列无空

9.4.22 卡 10：评审自查模拟

用途：D6 评审前自查 输入：SRS V2.0 + 附录 J 检查单 提示词：扮演评审专家，按检查单逐项审查本 SRS。要求：① 逐项给出 通过/不通过/存疑 结论与证据位置；② 否决项不通过时说明整改建议；③ 不改变文档内容。 验收要点：结论附条款证据；与人工评审结论比对

9.4.23 卡 11：术语表一致性扫描

用途：C3 检查的人工辅助 输入：SRS 全文 提示词：提取本文全部业务术语，找出：① 同义混用（如复核员/审核员）；② 使用但未定义的术语；③ 定义但 未使用的术语。输出三列清单，不修改原文。 验收要点：与术语表比对后人工裁决统一用词

9.4.24 卡 12：答辩模拟提问

用途：评审前演练 输入：SRS V2.0 + RTM + 澄清记录 提示词：扮演评审老师，参考 9.2 节问答集风格，对这份 SRS 提 10 个答辩问题，其中至少 3 个针对★条款、2 个针对 过程证据。只提问，并给出每个问题的考察点。 验收要点：全组能按问答集标准应答≥8 题

十二张卡片覆盖任务 2 全部 AI 协同点。最后一条规定：每次使用卡片，在度量表中记一行（卡片号、输入规模、输出采纳情况）——这是 2.13 节『AI 初稿采纳率』的数据来源，也是本课程『用证据说话』能力的最后一环。

9.4.25 任务 1 提示词速查库

以下汇总第二至九章的核心提示词（完整版见各章），供课堂速查：

| 场景        | 用途             | 核心指令摘要                              |
|-----------|----------------|-------------------------------------|
| 建议书·素材结构化 | 从背景材料提炼事实与痛点   | 事实清单标注出处；痛点标注依据；无量化依据标“需人工补充”       |
| 建议书·初稿    | 八节结构成稿         | 给定结构与字数；素材外事实标“待确认”；正式公文语体          |
| 建议书·对抗审查  | 招标人视角挑刺        | 最有力 3 点 + 最薄弱 3 点 + 质询 5 问；只批评不客套   |
| 澄清·歧义清扫   | 找招标文件的歧义/矛盾/缺失 | 原文摘录 + 影响分析 + 建议提问口径；按影响排序          |
| 质询·模拟评委   | 对抗演练           | 10 个尖锐问题（薄弱环节/指标可信度/团队真实性）；问答模式逐题追问 |

| 场景        | 用途        | 核心指令摘要                                             |
|-----------|-----------|----------------------------------------------------|
| 读标·解析报告   | 结构化提取招标文件 | 六节固定结构；只提取原文；拿不准标“需人工复核”                           |
| 编标·分章生成   | 按评分因素撰写章节 | 章首点明对应评分因素；响应写“可验证事实 + 章节指引”                       |
| 查标·规则初检   | 机器核对      | check_bid.py --tender 招标文件 --bid 投标书 --out 体检报告.md |
| 计划·WBS 生成 | 工作分解初稿    | 按交付物分解；≤1 周粒度；附“最易漏的三项工作”                          |
| 计划·甘特图脚本  | 程序化出图     | 数据与样式分离；里程碑防重叠；CJK 字体；PNG 供 Word 嵌入                |
| 风险·识别     | 头脑风暴清单    | “条件—后果”句式；四视角覆盖；只识别不打分                             |
| 述标·讲稿     | 8 分钟讲稿    | 按评分表分段 + 时间码；≤1900 字；关键数字必须口头说出                    |
| 述标·教练批改   | 评委视角修改    | 记不住的话可删；时间分配与分值匹配；给三句金句                            |
| 确认书·追踪矩阵  | 需求条目化     | MoSCoW 分级（Must ≤5）；验收依据可量化或可判定                     |
| 确认书·反方演练  | 签署前压力测试   | 甲方代表论证砍错了 + 监理指出不可测表述                              |

以下提示词骨架可直接复制使用（方括号处替换）。全部产出须人工终审，红线场景禁用。

- B.1 文档预审（结构）：『对照以下章节目录与 GB/T 8567 要求，列出本文档的结构缺失与编号断裂，只列疑点不下结论。[文档全文]』
- B.2 文档预审（数据）：『提取本文档全部数字型陈述（指标/日期/版本号），逐项标注疑似不一致或疑似占位符，只列疑点。[文档全文]』
- B.3 锚点扫描：『在以下多份文档中检索值 [0.75 / 78.4% / V3.1] 的全部出现位置，报告上下文差异。[文档集]』

- B.4 模拟执行审查：『你是从未见过本项目的运维，在全新 Ubuntu 22.04 仅有 Docker。逐步执行以下部署手册，报告每步的歧义、缺失与矛盾，不给结论。[手册全文]』
- B.5 三单对照自查：『对照以下日志条目与 git log，列出有 commit 无日志、有日志无 commit 的异常对。[日志+git log]』
- B.6 评委预演：『你是怀疑型评委，针对以下验收材料提 10 个最尖锐的问题，聚焦数据可信与替考嫌疑。[材料]』
- B.7 结论式标题改写：『把以下 PPT 页面标题改写为断言式结论，每条 ≤20 字，不改变事实。[标题列表]』
- B.8 偏差归因魔鬼代言人：『对以下每条偏差归因，指出它是机制结论还是借口，缺什么数据支持。[偏差源表]』

#### 9.4.26 A.1 manifest.txt 骨架

项目：澜图遥感影像智能解译平台 v1.0-acceptance / 打包时间：2026-09-03 10:42 / 打包人：乙 / 内容清单：docker-compose.yml、images/（3 个 tar）、.env.example、docs/（部署运维手册、用户手册）、smoke/（smoke.sh+样例数据）、seed/、README.md、preflight.sh / 校验：见 MANIFEST.sha256，逐文件 sha256sum -c 验证。

#### 9.4.27 A.2 哈希清单生成脚本

```
#!/bin/bash / find . -type f ! -name MANIFEST.sha256 -exec sha256sum {} + | sort > MANIFEST.sha256 / 验证侧：sha256sum -c MANIFEST.sha256 | grep -v OK（应为空）。
```

注意：打包前统一 LF 换行符。

#### 9.4.28 A.3 五步验收点自查清单

☐ 一键起（单命令，无人工干预）/ ☐ 健康绿（全部 healthy+healthz 200+preflight 过）/ ☐ 冒烟通（smoke.sh ≤10 分钟主流程）/ ☐ 持久化（重启后数据与服务恢复）/ ☐ 离线起（断外网一键起）。每轮记录：耗时\_\_缺口数\_\_执行人\_\_。

#### 9.4.29 A.4 验收报告签署页要素

验收结论（通过/有条件通过）/ UAT 结果摘要（7 行全钩）/ 遗留事项表（事项/处理建议/时间建议/双方责任）/ 签署：甲方代表、乙方全体、日期=验收会当天。

### 9.4.30 A.5 移交清单签署页要素

六类移交项逐项打钩（代码/文档/运行资产/过程资产/债务遗留/演示资产）+ 每类双方签收栏 + 配置库只读化时间 + INDEX.md 版本号。

### 9.4.31 A.6 Prompt 日志归档规范（一页版）

七字段：时间戳/操作者/任务关联/工具模型/Prompt 摘要/产出处置/关联 PR。规则：当日日志当日写（最迟次日站会前）；值班员日检 commit 对照；补录打【补录】标+补录日期；归档按日存 logs/prompts/YYYY-MM-DD.md。

## 9.5 检查单汇总

### 9.5.1 附录 A：交付物模板速查（第 3 关）

| 交付物        | 骨架来源              | 冻结前体检项                  |
|------------|-------------------|-------------------------|
| HLD        | 十章结构（见 5.2 节）     | 六视图齐全、NFR 映射表、图实一致      |
| DLD        | 模块五件套（5.2 节）      | 异常流、数值一致、测试要点场景级        |
| DB 设计      | 概念/逻辑/物理/运用四段     | 实体三处一致、索引有场景、备份可恢复      |
| 接口+OpenAPI | 六类接口+契约           | 解析校验通过、状态码合法、四查         |
| ADR        | 四段结构              | 候选 $\geq 2$ 、否决具体、负面+缓解 |
| plan/tasks | 里程碑三可+原子任务四规则     | 依赖无环、验证标准可执行            |
| 测试计划       | GB/T 15532 章节     | 专项测法+阈值出处、准入准出          |
| 管理计划族      | 范围三道闸/CI 五闸/风险四要素 | 指标有出处、基线清单与文档一致         |
| RTM        | 需求行+设计映射+双向核查     | 55+14 全挂接、反向无镀金         |

### 9.5.2 附录 B：提示词资产清单（建议入库）（第 3 关）

- hld\_chapter.md——HLD 章节起草（四段式，见 2.1.4）；
- dld\_module.md——DLD 模块起草（五件套+待裁决机制，见 6.2）；
- adr\_draft.md / adr\_attack.md——ADR 起草与反向攻击（见 9.5/21.2）；
- tasks\_split.md——原子任务拆分（四规则+横切依赖提醒，见 5.8）；
- testcase\_from\_ac.md——验收标准转测试用例草稿（见 7.3）；
- rtm\_fill.md——RTM 设计列回填辅助（见 5.11）；
- doc\_review.md——文档体检（数值/编号/禁用词/形容词密度，见 17 章）。

### 9.5.3 附录 C：推荐阅读

- GitHub Spec Kit 与 GitHub Next 白皮书——Spec-Driven 开发的源头文献；
- Thoughtworks 技术雷达（近期各期）——Spec-Driven 条目、AI 测试护栏、团队治理；
- Anthropic 《Building Effective Agents》与 OpenAI 工程实践指南——rules 文件与链式任务管理；
- GB/T 8567-2006、GB/T 15532-2008、GB/T 25000.10、GB/T 9385——本课程对标标准原文；
- Michael Nygard 《Documenting Architecture Decisions》——ADR 原始文献。

### 9.5.4 附录 D：本指导书引用的样例文件索引（第 3 关）

设计文档集 V3.0（《概要设计说明书》《详细设计说明书》《数据库设计说明书》《接口设计说明书》四册）；OpenAPI v3 契约；ADR 三份（ADR-001 空间数据存储选型、ADR-002 任务执行模型选型、ADR-003 推理服务进程模型）；plan.md 与 tasks.md；《测试计划》；管理计划族（《项目管理计划》《质量管理计划》《配置管理计划》《风险管理计划》四册）；RTM v3（docx+xlsx 双格式）。以上全部位于教学案例工作区 tender\_case/，配图脚本与构建脚本（t3/ 目录）一并开放，供读者研究『文档即代码』的生产方式。

### 9.5.5 结语（第 3 关）

任务 3 教的不是画架构图，而是『让每一句话有出处、让每个决策可审判、让每



条需求有测试等着它』的工程习惯。AI 把写作成本降到接近零之后，设计的竞争点就只剩下两样：判断力的质量，与追踪链的完整。前者靠本章 5.2~5.13 节的方法训练，后者靠 5.11 节的机制保障。祝各位设计评审一次通过。

### 9.5.6 附录 A：交付物模板速查（第 4 关）

| 交付物        | 骨架来源              | 冻结前体检项                       |
|------------|-------------------|------------------------------|
| 迭代计划与燃尽    | 里程碑 + 逐日节奏（4.2 节） | 两轮数据齐全、平台期有解释                |
| PR 评审记录    | 六栏 PR 模板（31.1）    | 五查意见齐、修改闭环、时间链吻合             |
| 变更单        | CR-001 七字段（16.2）  | 影响面穷尽、数据论证、闭环三项打勾            |
| 站会纪要/周报    | 三问+四段式（15 章）      | 阻塞有处理、周报无量词                  |
| RTM v4     | 九列结构（17.1）        | 55+14 全挂接、MVP 全通过、反向无孤儿      |
| Prompt 模式库 | 模式卡片六字段（18.2）     | 入库三标准、使用记录可追                 |
| 翻车记录       | 七字段（19.2）         | 五问到流程层、措施有验证                 |
| 代码仓        | 25 章结构            | selfcheck 11/11、README 十分钟实测 |

### 9.5.7 附录 B：提示词资产清单（建议入库）（第 4 关）

- P-01 需求反询——让 AI 对 spec 提问暴露歧义（见 4.10）；
- P-04 四段式代码生成 v3——含防编造 API 约束（见 2.1.4）；
- P-05 代码评审清单生成——评审辅助（见 6.3）；
- P-06 测试补全——按未覆盖行号补用例，防镜像断言（见 9.6）；
- P-08 重构小步走——一次一个手法（见 6.10）；
- P-09/P-10 调试双模式——假设-验证与最小复现（见 6.10）；
- P-11 红队自查——攻击者视角审安全（见 7.11）；
- P-12 文档-代码一致性扫描——语义级漂移周检（见 8.3）；



- AP-01/AP-02 反模式——一句话提问与让 AI 自证（见 2.1.6）。

9.5.8 附录 C：关键模板实物

C.1 context pack 模板

# Context Pack: T-XX 任务名 ## 1. 任务卡 (tasks.md 原文) ## 2. spec 条款 (相关 AC 原文, 含边界值) ## 3. 接口签名 (涉及类/函数的真实签名, 复制自代码) ## 4. 约定清单 (本项目反常识约定, 逐条注明出处) ## 5. 禁止事项 (本任务不许动的范围/文件)

C.2 变更单模板

# CR-XXX 变更标题 提出人/日期: 变更动因 (附数据): 影响面清单 (spec/SRS/OpenAPI/代码/测试/RTM 逐项): 数据论证 (改动前后对比): 回退方案: 批准记录: 闭环确认 (旧值清零/回归通过/RTM 更新):

C.3 翻车记录模板

# FR-XX 事故标题 经过 (时间线): 损失 (量化): 直接原因: 五问根因 (逐层): 纠正措施 (落在清单/脚本/公约): 验证 (措施生效的证据):

9.5.9 附录 D：本指导书引用的样例文件索引（第 4 关）

《迭代冲刺计划与燃尽报告》；《Prompt 模式库》（P-01~P-12 与反模式）；《AI 翻车记录》（FR-01/02/03 三则完整记录）；PR#47 评审样例；变更单 CR-001；《站会纪要与周报》；RTM v4（代码测试挂接）；sprint\_repo 代码仓（含 scripts/ 下 contract\_check、security\_redline、check\_refs、selfcheck、chaos 五个脚本与 CI 配置）。全部位于教学案例工作区 tender\_case/，供读者拆解研究。

9.5.10 结语（第 4 关）

任务 4 教的不是『让 AI 写代码』——那部分 AI 本来就会。真正要训练的是：给 AI 的产出建立门禁的纪律（第 4~5 章）、用测试把信任变成验证的习惯（第 6~13 章）、以及把每次翻车都兑换成规则的学习循环（第 18~21 章）。两个迭代之后，你的代码仓里最有价值的不是那几千行代码，而是那套五环能空转、防线能自证、经验能沉淀的机制。带着它进任务 5，验收关就是把这套机制再证明一遍。

体检脚本对 spec.md 执行 24 项自动检查，分四组。学生组可自行扩展，但不得删减基线项。

| 组   | 编号    | 检查项                                  | 不通过的典型症状                    |
|-----|-------|--------------------------------------|-----------------------------|
| 结构组 | S1~S6 | 必备章节存在、FR 编号格式、编号连续性、AC 编号配对、术语表存在、版 | FR-4.1 与 FR-04.1 混用；FR-03 后 |

| 组   | 编号    | 检查项                                                                                  | 不通过的典型症状                                  |
|-----|-------|--------------------------------------------------------------------------------------|-------------------------------------------|
|     |       | 本头存在                                                                                 | 直接跳到 FR-05                                |
| 可测组 | T1~T6 | 每条 FR 有 AC、AC 含 Given/When/Then 三段、量化指标含数值与单位、无『等/若干/较快』模糊词、无『用户友好』类不可测形容词、性能指标有前提条件 | 『系统应快速响应』；『支持多种格式』不写清单                    |
| 追踪组 | R1~R6 | 每条 FR 有来源列、★条款全部出现、RTM 行数与 FR+AC 总数一致、澄清编号可解析、MVP 标记与需求确认书一致、术语引用闭环                  | ★条款 5.7.8(1) 在 spec 中找不到；RTM 比 FR 总数少 5 行 |
| 一致组 | C1~C6 | spec 与 SRS 条款号一致、数值与招标原文一致（照录比对）、角色名称全文统一、日期与版本号一致、红线条款双侧标注、附录引用存在                   | 招标写 180 天 spec 写 90 天；同一角色出现『复核员/审核员』两个名字 |

运行纪律：每次 spec.md 修改后立即运行，报告按日期归档；任何一项未通过，该版本不得冻结。体检脚本不是形式——4.13 节复盘的六类缺陷中，有四类是体检脚本先发现的。

## 9.6 工具链装备清单

本关的 AI 协同不是『打开聊天窗口问几句』，而是一套有纪律的工具链。D1 开工前，AI 工程师必须完成本节的全部环境搭建，并向全组演示一遍。工具链四层：对话层、规则层、脚本层、归档层。

### 9.6.1 对话层：选定主工具并固定下来

主工具三选一：Claude Code、Cursor、Kimi CLI。选定标准不是『哪个最强』，而是『全组都会用哪个』——本关大量工作是『AI 生成、人工审定』，审定人必须能复现生成过程，因此全组用同一个工具、同一份配置。选定后写入配置库《工具链说明.md》，课程期间不得更换（换工具=提示词库与对话记录全部失效，度量数据断档）。

账号与成本：一个月实训期内，每组的 AI 调用量约在 300~600 次对话量级。建议由组长统一申请教育额度或团队订阅，费用与账号管理写入项目计划 v1 的资源项——这也是软件项目管理『资源管理』的真实演练。

### 9.6.2 规则层：rules 文件是团队的『AI 立法』

在项目根目录建 .rules 目录（Cursor 用 .cursor/rules，Claude Code 用 CLAUDE.md，Kimi 用 AGENTS.md——按所选工具的实际机制），至少写三条规则文件：

- 需求书写规则：所有需求条目化、动宾短语命名、数字必须带来源标注（[招标 x.x.x] 或 [IV-xx]）；禁止输出『等』『若干』『快速』『友好』；每条 FR 必须同时输出 AC 编号。这条规则让 AI 的默认输出就符合体检脚本的可测组检查。
- 红线规则：只下载不上传；不编造招标条款号；不确定的需求一律标记 [待澄清] 而非自行脑补。红线规则的本质是把甲方的合规要求前置为 AI 的行为约束。
- 格式规则：FR 编号 FR-大类.序号两位数字；AC 编号 AC-对应 FR 号；输出 Markdown 表格时列序固定为『编号|名称|描述|来源|优先级』。格式稳定是后续脚本自动化（提取、比对、生成 RTM 骨架）的前提。

规则文件生效验证：新建对话，只说一句『写三条影像上传的需求』，检查输出是否符合全部规则。不符合就改规则文件，直到一次通过——这个验证过程本身就是一次很好的提示词工程训练。

### 9.6.3 脚本层：check\_spec.py 与辅助脚本

check\_spec.py 的 24 项检查见附录 G，源码约 300 行，可由 AI 工程师在 D2 晚用 AI 辅助生成——生成时把附录 G 的检查项表直接喂给 AI 作为规格，这本身就是一次『用 spec 驱动 AI 写代码』的预演，与任务 3 的开发方式完全同构。

建议再加两个小脚本：① extract\_fr.py——从 spec.md 提取 FR 清单生成 RTM 骨架（自动填编号、名称、来源三列），消灭 RTM 编织中最易错的手工誊抄；② diff\_srs.py——对比两个版本 SRS 的条款差异，辅助编写《版本修订说明》。脚本入配置库 tools/ 目录，与文档同等管理。

### 9.6.4 归档层：对话记录也是项目资产

AI 对话记录必须导出归档：关键对话（SRS 起草、澄清提问、规则验证）导出为 Markdown 存配置库『AI 对话/』目录，命名『日期\_主题\_参与者』。三条理由：① 过程证据——评审时证明人机协同真实发生；② 提示词复用——好的提示词沉淀为附录 B 式的卡片库；③ 责任界定——出现 AI 编造内容时，对话记录能证明审定环节是否尽责。

归档的反面教材：某组用共享账号对话，到期账号回收、记录全失，评审时无法证明澄清过程，需求分被整体降档。教训写在这里，望各组引以为戒。

### 9.6.5 git 与分支纪律

- 分支模型：main（受保护）+ feature 分支（T-编号-简述，如 T-19-interactive-seg），禁止直接在 main 提交；
- 提交信息格式：『[T-19] 交互分割：掩膜编码移至 geo 层 (PR#47)』——任务号在前，RTM 回填时按号检索；
- PR 模板：.github/pull\_request\_template.md 固定六栏——任务卡链接、影响需求编号、自查清单（五查打勾）、AI 使用声明（用了哪些模式、哪些文件 AI 主导）、行为保持证据、评审人指派；
- 教学检查点：抽查 git log，提交信息无任务号的视为纪律违规。

### 9.6.6 pre-commit 本地钩子

```
# .git/hooks/pre-commit (或 pre-commit 框架配置) # 跑三样快速检查, 30 秒内完成: python
scripts/lint_fast.py # ruff 快速规则集 python scripts/security_redline.py # 红线扫描
python scripts/check_refs.py --staged-only # 编号校验 (仅暂存文件) # 任一失败即拒绝提交。
--no-verify 跳过钩子视为严重违规
```

### 9.6.7 CI 配置精讲

CI 五道闸的配置要点（以 GitHub Actions 为例，顺序即配置顺序）：lint（30s）→单测+覆盖率（3min，挂总体 70%/核心 85%/增量 80% 三个门槛）→check\_refs（10s）→contract\_check（20s）→pip-audit+红线（1min）。快的在前：让失败尽快暴露，省 CI 资源也省等待。夜间构建单独一条流水线跑 perf/accuracy/chaos，结果写回度量台账。

### 9.6.8 本地脚本全家福

| 脚本                  | 用途             | 何时跑      | 对应章节   |
|---------------------|----------------|----------|--------|
| selfcheck.py        | 交付自检 11 断言     | 交付前/演示前  | 3.4 节  |
| contract_check.py   | OpenAPI 与代码一致性 | 提交含端点改动时 | 2.2 节  |
| security_redline.py | 四条红线扫描         | 每次提交（钩子） | 2.4 节  |
| check_refs.py       | 需求编号双向校验       | 每次提交（钩子） | 2.11 节 |
| chaos/*.py          | 故障演练           | 每迭代至少一轮  | 2.7 节  |
| burndown.py         | 燃尽图数据生成        | 每日       | 2.9 节  |

### 9.6.9 环境一致性

- Python 版本锁定（.python-version），依赖锁定（requirements.txt 带哈希）；
- 环境问题的处理顺序：先查版本锁定文件是否被遵守，再查系统依赖（GDAL 这类库的系统层依赖是环境不一致的头号来源），最后才改代码；
- 教学检查点：换一台机器从零装环境跑 selfcheck，11/11 通过才算环境文档合格。

### 9.6.10 人机协同要点（第 4 关）

- CI 配置文件让 AI 起草，但门槛数值与闸序人工定（这是策略不是文书）；
- 脚本的首次编写用 P-07（探索性脚本模式），转正后按正式代码标准评审并补测试；
- CI 红了的日志分析可以喂 AI，但修复方案按调试方法论（4.3 节）走，不接受 AI 的『大概是这样』。

### 9.6.11 自动化测试

- pytest + requests（Python 接口测试）：与 OpenAPI 规范配合让 AI 起草脚本；断言必须独立计算；

- Playwright（UI 测试）：跨浏览器（Chromium/Firefox/WebKit）一套脚本，兼容性矩阵的执行利器；选择器脆弱，用 data-testid 属性而非文本定位；
- 坑：测试脚本本身的 bug 会伪装成产品缺陷——脚本先评审再入库，失败先查脚本再查产品。

### 9.6.12 性能测试：k6

本项目压测用 k6（理由：脚本即代码可进仓库、命令行友好易接 CI、分位数内置）。关键用法：stages 定义并发爬坡（1→2→4→8 的分档就是这么写的）；thresholds 把验收线写进脚本（p(95)<3000），压测失败即退出非零——性能门禁自动化的关键；summary export 出 JSON 给报告做数据源。坑：压测机与被测机不要同机（本机压本机，结果偏乐观且互相抢资源）；压测前先暖机（首请求含模型加载，混入统计会污染 P95——本项目常驻 Worker 零冷启动设计部分就是为了这个）。

### 9.6.13 安全工具

- pip-audit / npm audit（SCA）：全依赖树扫描，高危清零；中危要逐项写缓解说明；
- OWASP ZAP（渗透）：爬虫+主动扫描打底，6 端点自动过一遍；主动扫描会真的发攻击载荷，只在测试环境跑，跑完恢复数据；
- 人工定向测试：BOLA、越权、业务逻辑——工具的盲区，本节第 19/23 章已详述。

### 9.6.14 静态分析：SonarQube

质量阈（Quality Gate）先于扫描配置：阻断 0 / 严重 0 / 重复率≤3% / 增量覆盖≥70%。接入 CI 后每次 PR 自动分析。坑：默认规则集对教学项目偏严，triage 时要敢标 Won't Fix 但每条必须写理由；SonarQube 扫描结果与覆盖率数据要对接（jacoco/coverage.py 报告导入），否则复杂度×覆盖率雷区分析做不了。

### 9.6.15 缺陷与看板

GitHub Projects/Issues 足够：缺陷 = Issue（模板化六要素），看板 = Projects 四列 + 严重泳道，PR 关联 Issue 自动联动状态。纪律比工具重要：关单权在测试方、状态流转留痕、严重不过夜。

### 9.6.16 部署验证：Docker + 干净虚拟机

docker compose 一键起是底线；干净虚拟机用 VirtualBox/Hyper-V 快照——每次部署验证前回滚到『只装 Docker』快照，三台并行跑效率最高。验证清单五项（一键起/健康检/冒烟通/数据持久/离线起）写成脚本半自动化。

### 9.6.17 工具链一页图

| 环节    | 工具                     | 产出证据            | 进哪份交付物     |
|-------|------------------------|-----------------|------------|
| 单元/集成 | pytest、Playwright      | 执行记录、覆盖率        | 测试报告 1.2 节 |
| 性能    | k6                     | 分档数据、P95/P99    | 压测报告       |
| 安全    | pip-audit、ZAP、人工       | SCA 报告、渗透记录     | 测试报告安全章    |
| 静态    | SonarQube              | 质量阈报告、triage 记录 | 测试报告维护性章   |
| 缺陷    | GitHub Issues/Projects | 缺陷单、看板、燃尽       | 缺陷清单、根因分析  |
| 部署    | Docker、虚拟机快照           | 部署实测记录          | 测试报告可移植性章  |

### 9.6.18 完整性与打包

| 工具                       | 用途    | 关键用法                                                   | 坑                                 |
|--------------------------|-------|--------------------------------------------------------|-----------------------------------|
| sha256sum / Get-FileHash | 哈希清单  | find . -type f -exec sha256sum {} + > MANIFEST.sha256  | 换行符差异（Win/Linux）导致哈希不一致——打包前统一 LF |
| tar / zip                | 交付包归档 | tar czf 保留权限位                                          | zip 丢 Linux 执行位，脚本到助教机上跑不起来       |
| diff                     | 配置一致性 | diff <(grep -o 'VAR' compose) <(grep VAR .env.example) | 只查过键不查值——值漂移漏检                    |

### 9.6.19 干净环境

| 工具                   | 用途      | 关键用法                   | 坑                      |
|----------------------|---------|------------------------|------------------------|
| VirtualBox/VMware 快照 | 基础环境还原  | 装完 Docker 即拍快照，命名含日期轮次 | 快照链过长崩溃——每轮从基础快照派生而非链式 |
| docker compose       | 一键部署    | up -d --wait 等健康检查     | 健康检查未配导致『起=好』的误判       |
| nvidia-smi           | GPU 可见性 | 容器内执行验证 toolkit        | 宿主可见≠容器可见（G-01 的邻居坑）   |

### 9.6.20 文档与比对

| 工具           | 用途           | 关键用法                      | 坑                              |
|--------------|--------------|---------------------------|--------------------------------|
| Word 比较/修订   | 版本间差异        | 文档冻结前后比对                  | 自动编号域差异产生噪音——先转静态再比            |
| grep/ripgrep | 锚点扫描         | 跨文档搜指标值（0.75/78%）出现次数与一致性 | 全角半角不一致漏匹配（78% vs 78%）         |
| pandoc       | docx 与 md 互转 | 归档 PDF 化前转纯文本审            | 复杂表格转换错位——表格多的文档直接 Word 导出 PDF |

### 9.6.21 PPT 与图表

| 工具                 | 用途        | 关键用法                  | 坑                    |
|--------------------|-----------|-----------------------|----------------------|
| kimi-slides (pptd) | 验收 PPT 管线 | pptd 编写→check→渲染→截图复核 | 渲染后不做截图复核——溢出/过期数据漏网 |
| matplotlib         | 收敛曲线/AI 指 | 统一风格模板，PNG 300dpi     | 图表数据与正文              |



| 工具  | 用途     | 关键用法    | 坑                  |
|-----|--------|---------|--------------------|
|     | 标图     |         | 口径不一致（图 44% 文 43%） |
| OBS | 演示录屏兜底 | 全流程本地录制 | 录屏含麦克风环境音——提前静音检查  |

### 9.6.22 日志与统计

- git log + 自定义脚本：commit 与日志条目对照（值班员日检半自动化）；
- Python/pandas：日志聚合（采纳率/复用计数/ROI 账本），脚本入库存档——统计可复现；
- git blame + 打标：AI 行保留率统计的底层手段。

### 9.6.23 人机协同要点（第 6 关）

- 本节全部脚本适合 AI 生成，但每个脚本首次使用必须在测试数据上验证输出正确性；
- 工具坑清单随用随补，期末归入 I 系列素材（工具链建议）。

| 用途      | 推荐工具                         | 备注          |
|---------|------------------------------|-------------|
| 文档撰写    | Word/WPS + 课程样式模板            | 全队统一模板      |
| 文档程序化构建 | docx 构建脚本（docx-js）           | 长文档模块化生成与组装 |
| 条款核对    | check_bid.py                 | 规则初检，不替代人工  |
| 甘特图     | matplotlib 脚本 / ProjectLibre | 数据与样式分离     |
| 协作共编    | 共享文档（保留修改痕迹）                 | 过程证据        |
| 演练复盘    | 录音录像 + 转文字稿                  | AI 分析语速与口头禅 |
| 版本管理    | Git（文档与脚本同库）                 | 基线与变更留痕     |

（任务 1 工作方法指导书 全文完）

### 9.6.24 本节清单

- 完成分支纪律+PR 模板+pre-commit 三件套配置；

- 能讲出五道闸的顺序逻辑与各闸拦截对象；
- 六类脚本知道用途与运行时机；
- 换机重装能过 selfcheck。
- 每样工具答得出『证据进哪份交付物』；
- k6 thresholds 进 CI 实现性能门禁；
- ZAP 主动扫描只在测试环境；
- 虚拟机快照保证干净环境可重复。
- 五组工具到位，关键命令入库；
- 坑清单已读，高发坑（换行符/执行位/全角）有预防；
- 统计脚本存档，结果可复现。

## 9.7 常见问题解答

本节汇集任务 1 执行中最高频的 20 个问题，按环节分组。答案力求一句话给结论、三句话给理由。收录标准：历届（含澜图案例推演）中至少被问过两次的问题。

### 9.7.1 读标与澄清类

- Q1：招标文件几十页，时间不够逐字读怎么办？——先读四章：投标人须知（废标项）、评标办法（得分点）、用户需求书（响应对象）、投标文件格式（形式要求）；其余章节（合同通用条款等）由 AI 摘要 + 人工扫读。逐字精读只给 ★ 条款与评分表。
- Q2：AI 提取的 ★ 条款条数和人工数对不上，信谁？——信原文。以人工在原文中逐条定位为准；AI 提取是初稿，条数差异通常出在子条款（（1）（2））的合并与拆分上。
- Q3：澄清会不会暴露我们的理解水平，反而不敢问？——问得蠢才会（问原文已写明的），问得准是加分。判断标准：这个问题是否影响方案或报价？影响就问，并附上我方建议口径。
- Q4：招标人没回复澄清，怎么办？——在投标书中把“我方假设”写进需求理解章的“澄清问题与假设”节，明示假设内容——评审时这是标准做法，比静默猜测专业得多。

### 9.7.2 编标类

- Q5: AI 生成的技术方案看着都挺好, 怎么分出高下? ——用三个试金石: 有没有可验证的数字 (没有 = 空话)、数字有没有出处 (没有 = 幻觉)、出处与正文是否一致 (不一致 = 失控)。三关全过的段落才是好段落。
- Q6: 篇幅是不是越长越好? ——不是。篇幅与分值正相关即可; 评委反感注水。澜图 V2.0 的教训恰恰相反: 提升来自合规产物, 不是正文字数。
- Q7: 可以直接把开源项目的文档翻译成方案吗? ——不能。可以逆向理解其能力后用自己的话重新组织 (澜图案例即如此), 且项目名、标识必须更换避免侵权; 直接搬运文字既是抄袭也会在查重中暴露。
- Q8: 组员各写一章, 风格不统一怎么办? ——三件事统一: 样式模板 (字体字号)、术语表 (附录 A 术语)、编号体系 (章节号与条款号规则)。写进 rules.md, AI 生成时也受此约束。
- Q9: 正偏离是不是越多越好? ——不是。正偏离 = 超额承诺, 验收时都要兑现。只写可证明、可兑现的; 为分数写下的每个“优于”, 都是给自己挖的坑。

### 9.7.3 查标与定稿类

- Q10: 脚本说“未找到”, 但我们明明写了相关内容, 算不算响应? ——先按“表述差异”处理: 把招标原文关键句补进响应文字 (这是更合规的写法), 再复检。脚本是倒逼标书向原文对齐的工具, 不是裁判。
- Q11: 查标只有半天时间, 优先查什么? ——按废标风险排序: ★条款响应表 → 关键数字一致性 (工期/质保/有效期) → 占位符扫描 → 章节标题完整性 (对照模板) → 三件套存在性。文字润色类问题最后。
- Q12: 体检报告全绿是不是就稳了? ——不是。脚本只查四类规则项; 资格材料、签章、语义层面的响应质量它一概不知。全绿只是“可以进入人工细评”的入场券。

### 9.7.4 计划与风险类

- Q13: AI 排的工期看起来总是很乐观, 怎么校准? ——用三个锚点: 合同硬约束 (倒排)、团队真实产能 (参考既往作业速度)、缓冲比例 (每迭代  $\geq 15\%$ )。AI 的排期当“最乐观情形”, 人工加上缓冲后才是基线。

- Q14: 风险登记册写多少条合适? ——10-15 条为宜。少于 8 条说明识别不充分（对照“承诺/依赖/关键路径/团队”四视角再挖一轮）；多于 20 条说明粒度太碎，合并同类。
- Q15: 概率和影响打几分有客观标准吗? ——没有绝对标准，但有校准方法：两人独立打分再对齐；分歧超过 1 级的必须各自陈述理由；理由写不出的分数改保守值。

### 9.7.5 述标与确认类

- Q16: 可以照稿念吗? ——可以看提词卡，不能照稿念。训练标志：只看 3 个关键词能讲 60 秒。评委对“念稿”的扣分不在于形式，在于它暴露了对内容的不熟。
- Q17: 质询时被问住，冷场了怎么办? ——标准动作：复述确认问题 → 给出已知的部分 → 对未知部分承诺时限（“这一点我们会后 24 小时内书面补充”）。冷场硬编是最大失分动作。
- Q18: MVP 只有 5 项，评委会不会觉得做得太少? ——恰恰相反。MVP 小而闭环是工程判断力的体现；确认书同时写明范围外清单与后续迭代安排，“少而清楚”永远好于“多而含糊”。

### 9.7.6 协作与规则类

- Q19: 组员直接拿 AI 输出不改就合并进文档，怎么管? ——制度不靠自觉：合并前必须附“核验记录”（核验人、核了什么、改了哪里）；答辩时随机追问“这段改了哪里”，答不出即暴露。翻车案例十就是前车之鉴。
- Q20: rules.md 写了没人看，AI 也不总是遵守，还要坚持吗? ——坚持，但换用法：每会话开始时让 AI 先复述 rules.md 的三条核心再继续（复述不出 = 没加载）；每周组会用 5 分钟过一遍 rules.md 是否需要增补。规则文件是活的，不维护就会死。

### 9.7.7 范围与裁剪

问：设计文档一定要四本（HLD/DLD/DB/API）吗？答：可以合并但内容项不能少。合并时保留四章体系并做裁剪声明。小规模题目（<15 条 FR）可申请 HLD+DLD 合并，评委批准后方可。

问：MVP 只有 9 条，设计要覆盖全部 24 条吗？答：要。设计覆盖全范围（否则二期无从谈起），实现只保 MVP。RTM 状态列标注『二期』即完成区分。

问：ADR 写几份合适？答：硬要求  $\geq 2$ ，建议 3~5。覆盖存储、编排、进程模型三个最高风险点之外，还有真决策点才写——为凑数写的 ADR 评审时反而扣分。

### 9.7.8 深度与粒度

问：DLD 要写到多细？答：判据是『陌生人（或 AI）凭 DLD+spec 能写出代码』。类到方法签名级，流程到分支级，伪代码到可直接翻译级。写到行级代码则过头——那是任务 4 的事。

问：伪代码用什么语言风格？答：结构化的中文步骤或类 Python 伪码均可，关键是输入/输出/状态/异常四要素齐全。禁止直接贴真实代码冒充伪代码（那会锁死实现自由度）。

问：数据库设计要写全部字段吗？答：核心 12 表全字段（样例标准）；边缘表可给结构要点。但 MVP 涉及的表必须全字段+索引+约束。

### 9.7.9 AI 使用边界

问：AI 能画架构图吗？答：让 AI 写绘图脚本，不直接出图。图内文字必须人工逐字校对——图实不一致是本课程最高发的设计缺陷。

问：AI 起草的文档直接交可以吗？答：一票否决项之一（AI 痕迹无人审查）。判定方式很简单：评委问『这一节为什么这么写』，答不上来即视为未审查。

问：AI 额度不够怎么办？答：优先把额度花在审查与反向质疑上，起草可以让 AI 一次多产。额度预估写进项目管理计划（300~600 次对话），超出走备用工具预案。

### 9.7.10 流程与责任

问：设计冻结后发现错误怎么办？答：走变更流程（CCB），不要悄悄改。冻结后的每次变更都是配置管理的演练素材，变更单归档即合规。

问：组内对某选型有分歧怎么裁决？答：写 ADR 让方案对撞，按需求条款裁决；条款不覆盖的，按工期风险裁决；仍有分歧，组长裁决并记录分歧于 ADR 备注。禁止用投票或搁置。

问：测试计划与开发计划冲突（时间不够）听谁的？答：听准入准出标准的——

开发压缩可以砍非 MVP 范围，不可以砍测试闸。这条原则在任务 1 的需求确认书里就预埋了。

### 9.7.11 评审与验收

问：评委会问什么问题？答：三大高频：抽一条 FR 走查（见 2.10 节）、问一个 ADR 的被否决方案、问一个数值的依据。把这三类准备好，评审就是开卷考试。

问：发现样例（澜图）本身有错误怎么办？答：指出属实即加分——教学样例保持真实，包括它的修订历史。发现 v3 仍有残留偏差的组，欢迎按 5.13 节的机制提交校订记录。

问：设计文档有页数要求吗？答：无上限无下限，以内容完备为准（课程统一原则：达到交付质量与内容完备为准，不设页数裁剪）。但请自问：每一页是否都有评委需要的信息？

### 9.7.12 其他

问：可以用别的技术栈吗？答：可以，但选型表给出与需求条款对应的理由，且全组统一。混用栈（一半 FastAPI 一半 Spring）须有 ADR 论证。

问：GPU 没有怎么办？答：CPU 降级路径（FR-08.5）就是为这种情况准备的——首日验证降级可用，性能指标按 CPU 基线重新声明并与评委确认。

问：plan.md 的 7 天冲刺做不完怎么办？答：燃尽连续 2 天落后即触发砍非 MVP 项（R-05 预案）。MVP 5 项是底线，其余皆可让。

问：RTM 用什么工具维护？答：脚本+xlsx（样例 build\_rtm3.py）。允许用任何工具，验收标准是双向核查能跑通，不是工具本身。

问：设计阶段的度量数据要交给谁？答：归档配置库 03\_度量/，验收答辩时作为过程证据附录。度量是证据资产，不是考核大棒。

### 9.7.13 PR 与评审

- Q1：PR 多大算合适？A：改动 200~400 行、一个明确任务为单位。超过 600 行评审质量骤降，拆；少于 50 行的琐碎 PR 合并提交。
- Q2：队友总不评审怎么办？A：评审排班制（每个 PR 指定到人），站会通报超时未审的 PR；评审负载计入分工矩阵。

- Q3: AI 写的代码出 bug 算谁的? A: 算提交者与评审者的——门禁的意义就在于此。AI 没有责任能力, 责任永远在人。
- Q4: 评审意见分歧怎么裁决? A: 先回 spec/设计文档找依据; 文档没有的, 小决策主攻拍板并记录, 大决策升级 ADR。禁止搁置不决。

#### 9.7.14 测试与质量

- Q5: 来不及写测试能先合并吗? A: 不能。测试与代码同 PR 是红线。真来不及的正确做法是把任务拆小, 不是砍测试。
- Q6: 覆盖率差一点达不到怎么办? A: 先看未覆盖行是不是该测的。死代码就删; 防御性分支 (不可达错误处理) 可申请豁免并在 CI 配置里显式标注豁免行——豁免必须显式, 不许偷偷调低门槛。
- Q7: 性能测试在本地跑可以吗? A: 调优参考可以, 交付数据必须在规定测试机。环境差异是性能数据的第一大失真源。
- Q8: 精度差一点 (79.2% vs 80%) 能过吗? A: 不能。要么优化到达标, 要么走变更流程调整指标——变更需要数据论证且教师备案, 不是写个说明就行。

#### 9.7.15 AI 使用 (第 4 关)

- Q9: AI 一写就对, 还要评审吗? A: 要。采纳率高恰是放松警惕的开始——样例项目 FR-01 事故发生在 AI 连续三次产出优质代码之后。
- Q10: 提示词要不要随 PR 提交? A: 关键任务要。context pack 与模式编号写进 PR 描述, 这是 AI 资产度量的数据源。
- Q11: AI 会话记录要存档吗? A: 关键决策会话 (产出被合并的) 导出存档进 docs/ai\_sessions/, 评审追溯与翻车分析用。
- Q12: 能用多个 AI 工具吗? A: 可以, 但同一产出不要多工具反复生成挑选——那是变相的抽卡, 学会把一个工具的产出改对比换工具重抽更有价值。

#### 9.7.16 流程与管理

- Q13: 小改动也要走变更四步吗? A: 看是否动 spec。改 spec 的变更无大小之分, 一律四步; 纯实现细节的修复 (不动 spec/SRS) 走普通 PR。

- Q14: 站会真的每天都有东西说吗? A: 没有进展本身就是信息——说明任务卡住或粒度太大。站会不是报喜会。
- Q15: 燃尽图难看怎么办? A: 难看是信号不是耻辱。中期超 30% 就开风险会砍范围, 砍范围是合格的管理动作; 硬撑到爆才是失败。
- Q16: RTM 回填好烦, 能批量补吗? A: 不能。批量补的就是失真的。PR 模板里填需求编号只要 30 秒, 烦的是事后补, 解决方法是即时登记。

### 9.7.17 交付与答辩

- Q17: selfcheck 有一项不过能演示吗? A: 不能。先修到达 11/11。演示环境的 selfcheck 全绿是入场券。
- Q18: 演示时现场崩了怎么办? A: 按降级预案切到录屏(每组的演示预案必须含录屏兜底), 然后诚实讲崩的原因与后续修复计划——评委对『诚实+有预案』的评价远高于『侥幸没崩』。
- Q19: 答辩被问到不懂的模块怎么办? A: 主副修机制就是防这个的。副修参与达标的人不会完全不懂; 真不懂就承认并请主攻补充, 比硬编好。
- Q20: 时间不够, 先砍什么? A: 砍范围不砍质量。砍功能条目(走变更), 不砍测试、评审、文档。任务 4 的评分逻辑决定了质量项的分比功能项贵。

### 9.7.18 计划与范围

- 问: 五天做这么多专项, 做不完怎么办? 答: 裁剪要申报——写进测试计划的裁剪说明(裁什么、为什么低风险), 不许暗裁。四样底线(RTM/缺陷/红队自测/高危清零)不可裁。
- 问: 测试设计没做完就到执行窗口了? 答: 这是最常见的崩盘开局。预防: 测试左移, 任务 3 完成时测试计划就定稿、P0 用例写完; 任务 4 期间随开发滚动补用例。
- 问: 首轮通过率多少算正常? 答: ST 首轮 90%~95% 是健康水位。100% 先怀疑用例深度, 低于 85% 先怀疑开发关质量。

### 9.7.19 用例与执行



- 问：AI 生成的用例能直接进库吗？答：不能，五条入库质检（可判定/前置完整/独立执行/挂接/打标）一条不能少。
- 问：用例失败先查什么？答：先查测试脚本与环境（三成失败是它们），再查产品。失败排查 AI 初筛但人定根因，警惕 AI 的环境归因倾向。
- 问：50 条设计用例都要自动化吗？答：不。本项目自动化 122 条。易用性走查、精度判读、故障演练天然人工；接口测试天然自动化。
- 问：★条款 13 条怎么定的？答：MVP 5 项核心需求 × 每条至少 1 主流程 + 1 异常流 + ★款全覆盖，逐条从 RTM 筛出来的。

### 9.7.20 精度与性能

- 问：冻结影像集 5 景够吗？答：够教学验收。工程上数量服从统计功效要求，教学上服从工作量约束——但冻结纪律的要求不因数量减少而降低。
- 问：精度差一点达标能通融吗？答：不能。AT-01 首轮完整率 76% 就是没达标——走变更（CR-001）调方案再复测，这才是正确处理；改口径凑达标是造假。
- 问：压测并发上不去（机器不行）怎么办？答：降档声明——如实报告实测档位与容量建议（单节点并发 $\leq 2$ ），把扩容方案写清楚。验收看的是你有没有测出边界，不是边界高不高。
- 问：P95 和平均响应时间用哪个？答：P95（本项目加 P99 观测）。平均值掩盖尾部，5% 用户的糟糕体验会平均消失。

### 9.7.21 破坏性自测与故障演练

- 问：对手组不认真攻击怎么办？答：规则自保——他们的有效缺陷计他们的分，不认真是他们丢分；你的防守质量由你自己的测试深度兜底。
- 问：对手提了我们认为无效的缺陷？答：申辩引 SRS 原文，仲裁会裁决。措辞精度在此时兑现（降级态 14.8s 案例）。
- 问：演练 15 分钟不够怎么办？答：备战在平时——依赖索引表、操作卡、慌乱期纪律、轮值演练。15 分钟考的是准备，不是临场聪明。

### 9.7.22 缺陷与报告

- 问：缺陷定级和开发吵起来怎么办？答：按定级标准（影响面×严重性）逐条对；对不下升级测试负责人裁决。定级争议记录也是评审素材。
- 问：严重缺陷修不完能交付吗？答：不能。严重不过夜、不过关清零；实在修不完的走 Deferred 挂账并向评审披露——藏是事故，亮是管理。
- 问：报告数字和看板对不上怎么办？答：以执行系统/看板为准改报告，并查明哪一步污染了数据（通常是 AI 汇总时补全了缺失数据——所以日报要与执行系统 diff）。
- 问：首轮失败的数据要写进报告吗？答：必须。首轮/回归分行呈现，抹平即造假。

### 9.7.23 AI 使用（第 5 关）

- 问：哪些环节绝对不能用 AI？答：冻结集精度测试的执行与判读、pass/fail 最终判定、根因拍板、报告结论段、部分符合的披露措辞。
- 问：AI 写的断言怎么查镜像？答：问一句『被测函数改错，断言会变红吗』；期望值的来源必须独立于被测实现。
- 问：怎么证明 AI 用得深度而不是走过场？答：《翻车记录》——至少 1 次真实翻车的根因分析。无翻车记录 = 没深度使用或没审视，两者都扣分。

### 9.7.24 文档审计类

问 1：文档审计 93.8 分算什么水平？答：四维均 22+，样例组的水平。90+ 的关键不是『写得多』，是一致性与可执行性——锚点扫描零漂移、命令非执笔人实测过。失分最多的一般是可执行性维（部署手册缺版本号）。

问 2：审计发现的问题改不完怎么办？答：分级处置——阻断类（跑不通/数据矛盾）当日必修；一般类次日修；建议类进 backlog 并在审计报告注明处置方式。《文档审计报告》D-01~07 就是这么分的，『改不完』不等于『没处置』。

问 3：变更后文档滞后总被抓，有根治办法吗？答：变更单加『文档影响面』必填字段——CR 获批时列出所有需同步文档，评审不闭环不合并。D-03/05/07 全是 CR-001 的次生滞后，上这个字段后归零。

问 4：哈希清单必须吗？我们组小。答：必须。它是助教验证的第一道动作，也是你们自证封存后未改动的唯一手段。脚本十行，没有理由不做。

问 5：文档可以验收后再补吗？答：PDF 归档版以验收日为准；验收后只允许新增版本不允许改旧版。补写的文档按新版本走变更记录。

### 9.7.25 部署验证类

问 6：我们系统很简单，也要干净环境三轮自测吗？答：轮次可减，快照不能省。全班跑不通的 2 组里有一组就是『系统简单』的纯 Web 组——.env 漂移跟系统复杂度无关。

问 7：助教机器配置我们不知道，怎么准备？答：preflight 脚本就是为未知环境准备的——GPU/磁盘/端口检查失败给可读提示。再在手册前置条件章写清下限（实测峰值×1.2），剩下的交给脚本。

问 8：部署 1h12m 太久会扣分吗？答：不按绝对时长扣，按『与手册声称的符合度』判。手册写约 1.5h、实测 1h12m，是加分的一致性。手册写 10 分钟实测 1 小时，才是问题。

问 9：离线起这步可以省吗？我们必须要有外网字体/CDN。答：那就不是『可移植』，是设计缺陷。外网依赖全部内化（字体打包、库本地化）——这正是验收要考的。

### 9.7.26 AI 审计类

问 10：日志完整率多少算安全？答：97.4% 是样例水平。90% 以下开始被追问原因；但完整率 100% 且无任何【补录】标反而可疑——审计员会加时序交叉。

问 11：我们 AI 用得少，AI 工程分会低吗？答：不罚用得少，罚说不清。用得少就量化『为什么少』（如核心算法全手写+AI 仅做文档），口径清楚照样拿分。零翻车记录才是明确的扣分信号。

问 12：补录的日志会扣分吗？答：打标的补录只影响完整率数字，不单独扣分；不打标的补录按失信处理（PR#36 的 0.5 分）。

问 13：审计抽哪些 PR？答：随机+风险导向混合：随机抽 2~3 个，另加大 diff/合并快的/夜班提交的必抽。PR#36（凌晨夜班）就是这么被抽中的。

### 9.7.27 演示与答辩类

问 14：演示必须用新数据吗？用跑熟的数据稳一些。答：可以用跑熟的，但评委

会质疑过拟合演示。澜图策略（冻结集外+3 景备选+选中上）是兼顾稳与信的解法。演示诚信与日志诚信同标准。

问 15：走查抽到我但抽到的是队友主导的 PR 怎么办？答：名下代码索引按作者是各建各的，评委抽的是你名下的。评审队友 PR 时问过的问题、留下的评审痕迹，是你懂那段代码的证据——平时评审不划水，此刻不慌。

问 16：20 分钟讲不完怎么办？答：按 16 分钟排内容。彩排计时取中位数留 20% 余量；评委打断提问的时间不可控，内容可压缩的预案（哪页可跳）提前定。

问 17：评委质疑价值基线是编的怎么办？答：三重佐证摆出来（访谈记录编号/文献/实测记录），再坦白口径边界。基线被质疑不是事故，答不出佐证才是。

9.7.28 复盘与归档类

问 18：复盘报告写多少页？答：《项目开发总结报告》7 页（Word 实测）。复盘比密度不比厚度——每个数字有出处、每条偏差有归因，7 页足够。

问 19：净收益是负的怎么办？答：如实报负。负收益+清楚的边际结论（哪赔了、为什么、下规则）比硬凑正数有价值得多——课程考核的是算账能力不是结果正负。

问 20：归档后还能改吗？答：库只读化后不能。发现错误走『新版本+变更记录』补入，原位覆盖是配置管理重罪。这也是 D4 16:00『解散，不许再碰库』的原因。

9.8 术语表

| 术语        | 英文/缩写   | 本课程语境下的定义                                            |
|-----------|---------|------------------------------------------------------|
| 软件需求规格说明书 | SRS     | 按 GB/T 9385 组织的需求基线文档，叙述性内容为主，人工撰写、AI 辅助             |
| 机器可读规格    | spec.md | 用 Markdown+Given/When/Then 编写的结构化规格，AI 编码与自动化体检的直接输入 |
| 双轨制       | —       | SRS 面向人、spec.md 面向机器的并行维护策略，两者内容一致、形态互补              |
| 需求追踪矩阵    | RTM     | 需求编号→验收标准→设计→测试用例→代码的追踪表，本关要求 v1 覆盖需求与验收两级           |

| 术语     | 英文/缩写                  | 本课程语境下的定义                                             |
|--------|------------------------|-------------------------------------------------------|
| 反向澄清   | —                      | 让 AI 对 SRS 提问以暴露歧义的方法，要求 ≥10 条，人工逐条裁决                 |
| MoSCoW | M/S/C/W                | Must/Should/Could/Won't 需求优先级四分法                      |
| 验收标准   | AC                     | Acceptance Criteria，每条 FR 的可测试判据，用 Given/When/Then 表达 |
| 星标条款   | ★                      | 招标文件中不允许负偏离的关键条款，全生命周期 100% 可追踪                       |
| 正偏离    | —                      | 投标承诺优于招标要求的条款（如日志留存 3 年 > 180 天），SRS 从严基线、备注偏离关系      |
| 体检脚本   | check_spec.py          | 对 spec.md 做 24 项自动检查的工具脚本，产物为体检报告                     |
| 五问法    | —                      | 对每条需求连问五个问题（可测吗？有歧义吗？有来源吗？有边界吗？能追踪吗？）的自检法             |
| V 模型左移 | —                      | 在需求阶段就编写验收标准的实践，验收标准与需求同步产生而非测试阶段补写                   |
| 配置库    | —                      | 存放全部过程证据与版本产物的目录体系，命名规则：日期_文档名_版本号                    |
| MVP    | Minimum Viable Product | 最小可行产品，本课程锁定 ≤5 个核心功能并写入需求确认书                         |
| 镀金需求   | Gold Plating           | 无来源、无人要求、自行添加的需求，本课程一律禁止                              |
| 术语     | 英文/缩写                  | 定义                                                    |
| 概要设计   | HLD                    | 定义系统架构与模块划分的设计文档                                      |
| 详细设计   | DLD                    | 定义模块内部实现（类/流程/接口）的设计文档                                |
| 架构决策记录 | ADR                    | 记录一个架构决策的背景/候选/决策/后果的                                 |

| 术语      | 英文/缩写                     | 本课程语境下的定义                    |
|---------|---------------------------|------------------------------|
|         |                           | 一页式文书                        |
| 需求追踪矩阵  | RTM                       | 需求与设计/测试元素的双向映射表             |
| 双向核查    | Bidirectional trace check | 正向查需求有落点+反向查设计有来源            |
| 基线      | Baseline                  | 经正式冻结、变更须过 CCB 的配置项版本        |
| 镀金      | Gold plating              | 无需求来源的功能或设计（AI 时代的高发病）       |
| 待裁决清单   | Open decision list        | AI 起草中标记的不确定项，人工裁决后闭环        |
| 契约测试    | Contract testing          | 以 OpenAPI 契约为依据的接口自动化测试      |
| 冻结数据集   | Frozen dataset            | 版本锁定+哈希登记的精度验收基准数据           |
| 测试左移    | Shift-left testing        | 测试计划与设计同步冻结、用例随编码持续执行        |
| 在制品限制   | WIP limit                 | 看板 Doing 列同时进行的任务数上限         |
| 就绪/完成定义 | DoR/DoD                   | 任务进入开发/进入完成的门禁条件             |
| 变更控制委员会 | CCB                       | 基线变更的裁决组织（本项目：组长+需求负责人+评委代表） |
| 场景走查    | Scenario walkthrough      | 以 spec 场景为线索检验设计完整性的审查方法     |

术语表与 SRS 1.4 的关系：本表只收录设计阶段新增术语；领域术语以 SRS 术语表为准，不重复定义。全文用词一致性审查时两表合并使用。

### 9.8.1 流程与管理

| 术语    | 定义                                    |
|-------|---------------------------------------|
| PR 门禁 | Pull Request 合并前必须通过的检查集合：人工评审+CI 五道闸 |
| 五环    | AI 生成→提交者自查→队友 Approve→CI 绿→合并的完整     |

| 术语     | 定义                                       |
|--------|------------------------------------------|
|        | 链路                                       |
| 燃尽图    | 迭代剩余工作量随时间变化的曲线，用于进度预警                   |
| 主副修    | 每人一个主攻方向+一个副修方向的分工制度                     |
| 联调单    | 跨方向接口联调的过程与结论记录表                         |
| 四步变更流程 | 改 spec→同步 SRS→改 plan/tasks→改代码的变更纪律      |
| CR     | Change Request，变更单                       |
| RTM    | 需求追踪矩阵（Requirements Traceability Matrix） |
| WBS    | 工作分解结构，任务 1 产物，tasks.md 的上游              |

### 9.8.2 质量与测试

| 术语    | 定义                           |
|-------|------------------------------|
| 测试护栏  | 防止缺陷进入主干的自动化测试与检查体系总称        |
| 增量覆盖率 | 本 PR 新增/修改行的测试覆盖率，与存量覆盖率分开考核 |
| 镜像测试  | 期望值从实现代码倒推的无效测试（注水手法之一）      |
| 契约测试  | 验证代码实现与 OpenAPI 文档一致性的自动化检查  |
| 冻结影像集 | 选定后冻结并登记哈希的精度测试输入集合          |
| 故障演练  | 主动注入故障验证系统降级与恢复能力的演练         |
| 毒消息   | 导致消费端反复崩溃的非法消息               |
| P95   | 95 分位数：95% 的请求耗时不超过该值        |
| IoU   | 交并比，分割精度指标；本项目用宏平均口径         |

### 9.8.3 AI 协同

| 术语                      | 定义                       |
|-------------------------|--------------------------|
| Spec-Driven Development | 规格驱动开发：spec 为唯一源头，文档与代码由 |

| 术语           | 定义                                      |
|--------------|-----------------------------------------|
|              | spec 同步派生                               |
| 四段式提示词       | 角色+上下文+任务+约束的提示词结构（P-04）                |
| context pack | 为编码任务准备的上下文包：任务卡+spec 条款+接口签名+约定清单+禁止事项 |
| 开工仪式         | 新 AI 会话的固定初始化流程：喂规则→喂任务包→对齐确认→约定格式      |
| 采纳率          | AI 产出经少量修改或原样被合并的比例                     |
| 翻车记录         | AI 产出造成实质损失的事故根因分析文档                    |
| 五问法（5-Whys）  | 连续追问五层为什么以挖到流程层根因的方法                    |
| 上下文漂移        | 长会话中 AI 遗忘或违背早先约定的现象                    |
| 红队模式         | 让 AI 扮演攻击者审查代码的提示词模式（P-11）              |

#### 9.8.4 标准与规范

| 术语            | 定义                       |
|---------------|--------------------------|
| GB/T 8567     | 计算机软件文档编制规范（设计文档章节结构的依据） |
| GB/T 9385     | 软件需求规格说明规范（SRS 的依据）      |
| GB/T 15532    | 软件测试规范（测试计划的依据）          |
| GB/T 25000.10 | 软件质量模型（非功能需求分类的依据）       |



| 术语   | 定义                                   |
|------|--------------------------------------|
| 等保基线 | 网络安全等级保护的基本要求项，本项目教学化映射为四条安全红线       |
| MVP  | 最小可行产品；本项目锁定 11 条 FR                 |
| ADR  | 架构决策记录（Architecture Decision Record） |

### 9.8.5 测试基础

- RTM（需求追踪矩阵）：需求编号→设计→用例→执行结果的双向追踪表。本项目 RTM v5 覆盖 24 FR + 16 NFR 全闭环。
- P0/P1/P2 用例优先级：P0 = MVP 核心需求与★款的必过用例（34 条），验收不过即否决；P1 = 重要功能；P2 = 一般覆盖。
- 冒烟测试（Smoke）：部署后验证主流程可通的最小用例集。本项目标准冒烟流程：上传→全自动分割→交互修正→导出。
- 回归测试：缺陷修复或变更后重跑相关用例集确认无回退。本项目 IT 回归 86/86、ST 回归 64/64。
- 测试左移：测试计划与用例设计在开发完成前启动。本项目测试计划在任务 3 结束时定稿。

### 9.8.6 精度与性能

- 面积加权完整率/正确率：遥感精度指标，按图斑面积加权的检出两率。本项目 AT-01 门槛 80%/85%，实测 84.2%/88.6%。
- IoU（交并比）：预测掩膜与真值掩膜交集面积/并集面积。AT-02 门槛 0.75，实测 0.78。
- P95/P99：响应时间分位数，95%/99% 的请求快于该值。PE-01 门槛  $P95 \leq 3s$ （并发口径 1 推理+30 在线）。
- 冻结集：精度验收专用的影像集合（5 景），任何调优不得使用，使用须登记。
- 容量建议：由压测数据推导的部署边界与扩容单元。本项目：单 GPU 节点交互并发  $\leq 2$ 。

### 9.8.7 安全与可靠性

- SCA（软件成分分析）：对全部直接与传递依赖扫描已知漏洞。本项目 47+213 依赖，高危 2→0。
- BOLA（对象级越权）：API 未校验资源归属导致的横向越权，OWASP API 安全头号风险。本项目 BUG-133。
- 幂等（Idempotency）：同一请求重复执行结果一致。本项目 BUG-132 后以 Idempotency-Key 头实现。
- 死信队列：无法消费的消息的隔离区。RL-03/BUG-125 后毒消息进死信队列。
- 等保基线：网络安全等级保护 2.0 基本要求的适用条款核查。本项目 ST-01 共 33 项签认。

### 9.8.8 过程与管理

- 缺陷密度：缺陷数/千行代码。本项目 24 条/9.6K 行 = 2.5/KLOC。
- 环节有效性矩阵：发现环节×严重度的交叉分析表，用于评估各测试环节的捕获能力。
- 慌乱期纪律：故障演练前 3 分钟只看不改的纪律（禁重启、禁改配置）。
- 依赖索引表：系统全部依赖及健康检查命令的一页纸清单，演练定位地图。
- 镜像断言：断言逻辑复刻被测实现导致恒真的伪测试。入库质检重点。
- AI-draft 打标：AI 生成的测试资产在人工评审通过前的标记，事故溯源用。

### 9.8.9 标准引用

- GB/T 15532《计算机软件测试规范》：测试文档与过程组织依据。
- GB/T 25000.10：软件质量模型（八大特性），自评表对照框架。
- GB/T 25000.51：就绪可用软件产品的质量要求，验收依据。

按拼音/字母序排列，仅收录本指导书使用且可能有歧义的术语。全课程通用术语见任务 1 指导书附录。

| 术语       | 定义                            |
|----------|-------------------------------|
| UAT（用户验收 | 由甲方按确认书条款执行的验收测试，逐行判定通过/不通过/有 |

| 术语         | 定义                                               |
|------------|--------------------------------------------------|
| 测试)        | 条件通过，当场签认                                        |
| 补录（【补录】）   | 对未及时记录的 Prompt 日志的事后补写，必须打标并注明补录日期；不打标按失信处理      |
| 批量部署验证     | 助教在干净环境执行交付包部署的批量验证，跑不通预扣分                       |
| 干净环境       | 消除开发机惯性的全新操作系统环境（快照/新容器主机/非开发者账号），用于可移植性自测       |
| 故障注入       | 彩排或演练中人为制造故障（拔网线/杀进程）以检验应对能力的测试手段                |
| 口径         | 统计指标的定义与边界（含什么、不含什么、数据源是什么），先于数据确定               |
| 锚点扫描       | 以版本号/指标值/需求编号等关键值为锚，跨文档检索出现位置核查一致性的方法            |
| 名下代码索引     | 个人为代码走查准备的 PR 清单，每 PR 备五句话（需求/机理/权衡/AI 角色/影响面）   |
| 留置事项（遗留事项） | 验收时双方知情同意暂不处理的事项，须写明处理建议、时间与责任                   |
| 偏差源表       | 复盘核心表：计划 vs 实际工时/进度的逐项归因（含收益型负偏差）                |
| 签名链（签认链）   | 债务管理的证据链：发现→评估→披露→签认→入册                          |
| 缺口（Gap）    | 干净环境自测中发现的交付包/手册缺陷，按文档/工程/环境/回归四型分类              |
| 三单对照       | AI 审计方法：Prompt 日志、commit/PR 记录、代码本体三方互查          |
| 收敛曲线       | 多轮自测的缺口数与耗时变化曲线，作为过程质量证据                         |
| 四段式        | 答辩举证话术结构：结论→证据→口径→延伸                             |
| 冻结（Tag 冻结） | 验收基线的代码封存：打 tag（如 v1.0-acceptance）+主干保护，此后改动走新分支 |

| 术语    | 定义                           |
|-------|------------------------------|
| 五步验收点 | 部署自测检查骨架：一键起/健康绿/冒烟通/持久化/离线起 |
| 观察项   | 条件性技术债务：当前场景不构成债务，条件触发即升级    |
| 预扣分   | 批量验证中交付包跑不通即先扣分再进答辩的制度       |

## 9.9 拓展阅读与资源清单

本节汇总任务 1 方法背后的思想来源与延伸资源，按“先读什么、再读什么”排序。每条给出内容梗概与课程内的对应章节，方便按需取用。课程不强制阅读全部，但第一类的三份是方法论的源头，建议至少精读其一。

### 9.9.1 第一类：方法论源头（对应第一、十二、二十四章）

- GitHub Spec Kit 与 GitHub Next 官方白皮书：Spec-Driven Development 的原始阐述，演示如何通过规范驱动 AI 生成代码与 PR。读法：重点看“规范如何写得机器可执行”，对照本书 1.2.1 与读标方法；
- Thoughtworks 技术雷达（近年各期）与其 AI 软件工程白皮书：追踪从 BDD/TDD 到 Spec-Driven 的演进，大量篇幅讨论“AI 时代的测试护栏”与团队治理。读法：重点看护栏（guardrails）相关条目，对照本书 1.2.2、第二十一章；
- Anthropic 工程博客（Building Effective Agents 等）与 OpenAI 的 Agentic 实践指南：Agent 系统的规则文件、工具设计、链式任务与防发散。读法：重点看“何时该用规则、何时该用流程、何时该用 Agent 自主”，对照本书第十二、十章。

### 9.9.2 第二类：招投标与工程标准（对应第四、五、十六章）

- 《中华人民共和国招标投标法》及其实施条例：了解澄清、评标、否决投标的法定框架——课程案例的条款设计以此为底；
- GB/T 8567-2006《计算机软件文档编制规范》：投标与交付文档的章节结构依据（澜图附录 G 的对照表）；
- GB/T 9385-2008《计算机软件需求规格说明规范》：SRS 与需求条款写法的国标依据；
- GB/T 15532-2008《计算机软件测试规范》与 GB/T 25000.51-2016：测试与质量评价的依据；

- GB/T 8566-2022《系统与软件工程 软件生存周期过程》：项目过程框架的依据。读法：国标不必通读，会查会用即可——投标书写“对标某标准”前，至少翻开目录核对一次。

### 9.9.3 第三类：AI 工具与行业实践（对应第十、二十章）

- 本书配套调研报告《AI 标书工具调研.md》（tender\_case/）：钛投标、文兜智写、DeepRFP、Tribble、Arphie 等工具的机制对比，bid-writing 技能的设计输入；
- 各主流 AI 编程助手的规则文件文档（AGENTS.md / .cursor/rules 等）：rules.md 写法的工业参照；
- 遥感分割开源生态（在 GitHub 以「遥感 分割」「remote sensing segmentation」等关键词检索代表性项目）：读其源码与 README，可理解「从零散开源能力到完整平台需求」的逆推过程——这正是澜图案例的来路。

### 9.9.4 第四类：课程内资源（天天要用）

- 澜图全套样例：招标文件、投标技术标 V2、六份任务 1 交付物、对比报告、两轮体检报告（tender\_case/ 目录）；
- bid-writing 技能包：SKILL.md + 4 个 references + check\_bid.py（已安装，触发词“标书/投标/偏离表”）；
- 本指导书的三个附录：检查单汇总（附录 A）、归档清单（附录 B）、工具栈（附录 C）——打印贴墙。

### 9.9.5 如何判断一份资源值不值得读

AI 领域的资料爆炸，筛选比阅读更稀缺。给学生一个三分钟判别法：（1）看来源——官方工程博客、标准组织、持续维护的技术雷达优先，二手解读与营销号降级；（2）看可操作性——读完能否改变你明天的具体动作？只谈趋势不谈做法的，存起来不写进方案；（3）看时效——AI 工具类内容保质期约半年，方法类（如护栏、评审、责任划分）保质期以年计，优先读后者；（4）看是否与课程方法冲突——冲突时以课程方法为基线，把新资源当作“待评估的改进提案”，在复盘会上讨论后再吸收。这个判别法本身，就是本书第二十一章“评审的对象是风险”在阅读习惯上的延伸。

### 9.9.6 阅读路线建议

- 时间紧（只有 2 小时）：本书第一章 + 第五章 + 附录 A，然后直接开工；
- 想深入方法：加读 GitHub Spec Kit 白皮书与 Anthropic 工程博客；
- 想做技能开发：加读第十章 + bid-writing 源码（它是 20 个文件以内的完整可学样本）。
- GB/T 8567《计算机软件文档编制规范》——交付清单与项目开发总结报告结构；
- GB/T 25000.51《系统与软件质量要求和评价（SQuaRE）》——验收质量判据；
- GitHub Spec Kit 官方文档——spec 驱动的变更闭环（CR-001 零漂移的理论源头）；
- Thoughtworks 技术雷达（AI 软件工程专题）——测试护栏与团队治理；
- Anthropic/OpenAI 工程博客（Agentic Systems）——Prompt 日志与人机分工的工程化。

### 9.9.7 F.1 可借鉴的部分

- 流程图绘制规范：参考文档中的业务流程图采用泳道式布局，角色与系统分列——本关样例 SRS 第 5 章的解译作业流程图沿用了同一画法。学生绘制流程图时应对照此规范，而非随手画方框箭头。
- 数据需求的完整性：参考文档对每一类数据给出来源、格式、量级、更新频率、质量要求五要素。本关样例 SRS 的影像数据、样本数据、模型数据、成果数据四类均按此五要素展开（4.6 节有完整推导过程）。
- 接口需求的具体化：参考文档不写『系统应有良好接口』这类空话，而是逐个接口给出协议、数据格式、调用频率、异常处理。这是接口章节的基本功。

### 9.9.8 F.2 不可照搬的部分

- 章节结构：参考文档写于 GB/T 9385-1988 时代，章节按旧版标准组织；本关样例按 GB/T 9385-2008 组织（增加了第 2 章总体描述的用户特征、约束、假设依赖等小节）。照搬旧结构会导致体检脚本第 3 项检查不通过。

- 编号体系：参考文档用流水号（1.1、1.2.....），无大类分层；本关要求 FR-大类.序号 两级编号以对应招标章节。直接用旧编号体系会让 RTM 无法挂接招标条款。
- 验收标准：参考文档无独立验收标准（写作年代尚无此实践）；本关要求每条 FR 配 Given/When/Then 验收标准。这是两代需求工程的代差，不可倒退。

使用参考文档的正确姿势：学其『实』（数据五要素、接口具体化、流程图规范），不学其『形』（旧章节、旧编号）。评审老师提问『参考文档用了没有、怎么用』时，能讲清这个区分的小组即为合格。